BAEUM.AI

Agent Handbook

with LangChain, LangGraph & DeepAgents

AI 에이전트 개발 가이드

Agent Handbook with LangChain, LangGraph & DeepAgents

Copyright © 2026 BAEUM.AI. All rights reserved.

이 책의 코드 예제는 교육 목적으로 자유롭게 사용할 수 있습니다.
본문의 무단 복제 및 배포는 금지됩니다.

초판 발행: 2026

서문

AI 에이전트는 단순한 챗봇을 넘어, 도구를 사용하고, 계획을 세우며, 복잡한 작업을 자율적으로 수행하는 지능형 시스템입니다. 이 책은 세 가지 주요 프레임워크 — **LangChain**, **LangGraph**, **Deep Agents** — 를 활용하여 실전 AI 에이전트를 구축하는 방법을 체계적으로 안내합니다.

> **이 책의 특징**
>
> - **실습 중심** — 59개의 Jupyter 노트북 기반 예제와 실행 결과를 포함합니다
> - **단계적 학습** — 기초부터 프로덕션 배포까지 점진적으로 난이도가 올라갑니다
> - **프레임워크 비교** — 각 프레임워크의 강점과 적합한 사용 사례를 비교합니다
> - **실전 응용** — RAG, SQL, 데이터 분석, ML, 딥 리서치 에이전트를 구현합니다

> **대상 독자**
>
> - Python 기본 문법을 아는 개발자
> - LLM 기반 애플리케이션을 구축하려는 엔지니어
> - AI 에이전트 아키텍처를 체계적으로 배우고 싶은 분

이 책의 구성

파트	주제	챕터
I	에이전트 입문 — 환경 설정부터 세 프레임워크 기초까지	8
II	LangChain — 모델, 도구, 메모리, 미들웨어, 멀티 에이전트	13
III	LangGraph — 상태 그래프, 워크플로, 영속성, 서브그래프	13
IV	Deep Agents — 백엔드, 서브에이전트, 메모리, 하네스	10
V	고급 패턴 — 멀티 에이전트, RAG, SQL, 프로덕션	10
VI	실전 응용 — 5개 실전 에이전트 프로젝트	5

각 챕터는 학습 목표 → 이론 설명 → 코드 실습 → 요약 구조로 구성되어 있습니다.

이 책의 사용법

코드 블록 읽기

이 책의 코드 블록은 두 가지로 구분됩니다:

```python
# Python 코드 블록 — 민트 좌측 보더
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
result = model.invoke("Hello, Agent!")
```

```
실행 결과 블록 — 코랄 좌측 보더
Hello! I'm an AI agent ready to help.
```

박스 유형

> ➡️ TIP
>
> 유용한 팁이나 추가 정보를 제공합니다.

> · NOTE
>
> 중요한 참고 사항이나 배경 지식을 설명합니다.

> ! WARNING
>
> 주의가 필요한 사항이나 일반적인 실수를 경고합니다.

실습 환경

모든 코드는 Python 3.11+ 환경에서 테스트되었습니다. 필요한 패키지 설치와 API 키 설정은 Part I의 첫 번째 챕터에서 다룹니다.

목차

Part I: 에이전트 입문 7
0. 환경 설정 9
1. LLM 기초 12
2. LangChain 입문 15
3. LangChain 대화 18
4. LangGraph 입문 20
5. Deep Agents 입문 23
6. 세 프레임워크 비교 & 다음 단계 26
7. 미니 프로젝트 30
Part II: LangChain 33
1. LangChain 소개 35
2. 첫 번째 에이전트 42
3. 모델과 메시지 시스템 46
4. 도구와 구조화된 출력 51
5. 메모리와 스트리밍 57
6. 미들웨어와 가드레일 62
7. 사람 개입(HITL)과 런타임 71
8. 멀티 에이전트 패턴 76
9. 커스텀 워크플로와 RAG 83
10. 프로덕션 88
11. MCP (Model Context Protocol) 96
12. 프론트엔드 스트리밍 102
13. 가드레일 111
14. Semantic Search 121
Part III: LangGraph 123
1. LangGraph 소개 126
2. Graph API 기초 131
3. Functional API 135
4. 워크플로 패턴 138
5. 에이전트 구축 142
6. 지속성과 메모리 145
7. 스트리밍 152
8. 인터럽트와 타임 트래블 157
9. 서브그래프 162
10. 프로덕션 167
11. 로컬 서버 175
12. 내구성 실행 182
13. API 선택 가이드와 Pregel 191
14. Fault Tolerance 199
15. Backward Compatibility 203
16. Case Studies 205
Part IV: Deep Agents 206
1. Deep Agents 소개 209
2. 첫 번째 에이전트 만들기 215
3. 에이전트 커스터마이징 220
4. 스토리지 백엔드 226
5. 서브에이전트와 태스크 위임 235
6. 장기 메모리 & 스킬 244

7. 고급 기능 252
8. 에이전트 하네스 264
9. 외부 프레임워크 비교 273
10. 샌드박스와 ACP 280
11. 비동기 서브에이전트 289
12. 프로덕션 준비 295
13. 컨텍스트 엔지니어링 302
14. 스트리밍 309
15. 권한 관리 314
16. Models and Tools 320
17. Programmatic Subagents 323
18. Event Streaming 326
19. Permissions 329
20. Quality Profiles and Rubrics 331
Part V: 고급 패턴 333
0. v0 → v1 마이그레이션 가이드 335
1. 미들웨어 시스템 심화 341
2. 멀티에이전트: Subagents 355
3. 멀티에이전트: Handoffs & Router 365
4. 컨텍스트 엔지니어링 & 메모리 심화 377
5. Agentic RAG 390
6. SQL 에이전트 심화 399
7. 데이터 분석 에이전트 407
8. 보이스 에이전트 415
9. 프로덕션 배포 425
10. Deep Agent from Scratch 437
11. Custom Workflow Agent 439
Part VI: LangSmith 440
1. LangSmith Quickstart 442
2. 에이전트 트레이스 구조 451
3. 데이터셋과 평가 루프 459
4. Prompt Hub 버전 관리 467
5. 프로덕션 모니터링 472
6. Agent Evals 479
7. Testing Strategy 483
8. LangGraph Testing 486
9. Runtime Rubric Evaluation 488
Part VII: 실전 응용 489
1. RAG 에이전트 491
2. SQL 에이전트 497
3. 데이터 분석 에이전트 502
4. 머신러닝 에이전트 508
5. 딥 리서치 에이전트 513
6. 멀티모달 PDF RAG 520
7. Content Builder Agent 526
10. Personal Assistant Subagents 530
11. Customer Support Handoffs 532
12. Router Knowledge Base 535
13. Skills SQL Assistant 537
Part VIII: Integrations 539
1. 통합 카테고리 개요 541
2. Anthropic Prompt Caching 546
3. Claude Bash Tool 550
4. Claude Text Editor 554
5. Claude Memory 557
6. Anthropic File Search 561
7. Bedrock Prompt Caching 565
8. OpenAI Moderation 569

부록 A. 용어집 **573**

PART

I

에이전트 입문

Foundations of AI Agents

이 파트에서 다루는 내용

- 0. 환경 설정
- 1. LLM 기초
- 2. LangChain 입문
- 3. LangChain 메모리
- 4. LangGraph 입문
- 5. Deep Agents 입문
- 6. 세 프레임워크 비교
- 7. 미니 프로젝트

00 환경 설정

시작하기 전에

이 시리즈는 LangChain → LangGraph → Deep Agents 순서로 AI 에이전트의 핵심만 빠르게 체험하는 입문 과정입니다.

학습 목표

LEARNING OBJECTIVES

- → `.env` 파일로 API 키를 안전하게 관리하는 방법을 익힌다
- → `ChatOpenAI` 로 LLM 모델을 초기화한다
- → 모델에 간단한 질문을 보내 정상 동작을 확인한다

0.1 API 키 설정

프로젝트 루트의 `.env.example` 을 `.env` 로 복사하고, 아래 키를 입력하세요:

```
OPENAI_API_KEY=sk-...
TAVILY_API_KEY=tvly-...   # 선택
```

키	용도	발급처
`OPENAI_API_KEY`	LLM 호출 (필수)	https://platform.openai.com/api-keys
`TAVILY_API_KEY`	웹 검색 도구 (선택)	https://tavily.com

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("\u2713 API 키 로드 완료")
```

```
✓ API 키 로드 완료
```

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
    os.environ.setdefault("LANGCHAIN_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGCHAIN_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON \u2014 project: {os.environ['LANGCHAIN_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

```
Langfuse tracing ON — https://lf.ddok.ai
```

0.2 모델 초기화

`ChatOpenAI` 는 OpenAI 호환 LLM을 감싸는 LangChain 클래스입니다. 이후 노트북에서 이 `model` 객체를 계속 씁니다.

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 설정 완료:", model.model_name)
```

```
✓ 모델 설정 완료: gpt-4.1
```

0.3 동작 확인

모델에 간단한 메시지를 보내 정상적으로 응답하는지 확인합니다.

```
response = model.invoke("안녕하세요! 한 문장으로 답해주세요.", config=lf_config)
print("\u2713 모델 응답:", response.content)
```

```
✓ 모델 응답: 안녕하세요! 무엇을 도와드릴까요?
```

요약

항목	내용
환경 변수	`load_dotenv()` 로 `.env` 파일 로드
모델	`ChatOpenAI(model="gpt-4.1")`
테스트	`model.invoke("...")` → 응답 확인

01 LLM 기초

메시지, 프롬프트, 스트리밍

에이전트를 만들기 전에, LLM과 대화하는 기본 방법을 익힙니다.

학습 목표

LEARNING OBJECTIVES

- → 메시지의 역할(`system` , `human` , `ai`)을 이해한다
- → 시스템 메시지로 모델의 행동을 제어한다
- → `model.stream()` 으로 실시간 응답을 받는다

1.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 준비 완료")
```

```
✓ 모델 준비 완료
```

1.2 메시지의 세 가지 역할

LLM은 메시지 리스트를 입력으로 받습니다. 각 메시지에는 역할(role), 콘텐츠(content), 메타데이터(metadata)라는 세 가지 핵심 요소가 있습니다.

역할	클래스	설명
`system`	`SystemMessage`	모델의 행동 지침을 설정합니다. 페르소나, 응답 톤, 규칙 등을 정의하여 모델의 초기 동작을 결정합니다.

`human`	`HumanMessage`	사용자의 입력을 나타냅니다. 텍스트뿐만 아니라 이미지, 오디오, 파일 등 멀티모달 콘텐츠를 지원합니다.
`ai`	`AIMessage`	모델의 응답입니다. 텍스트 응답 외에도 `tool_calls` (도구 호출), `usage_metadata` (토큰 사용량) 등의 속성을 포함합니다.

도구 실행 결과를 모델에 전달하는 `ToolMessage` 도 있습니다. `ToolMessage` 는 도구 결과 내용, 도구 호출 ID, 도구 이름을 필수로 포함합니다.

1.3 시스템 메시지로 행동 제어

`SystemMessage` 를 바꾸면 같은 질문에도 전혀 다른 응답을 받습니다. 이것이 프롬프트 엔지니어링의 핵심입니다.

1.4 딕셔너리 형식

메시지 객체 대신 딕셔너리로도 전달할 수 있습니다. LangChain은 메시지 입력을 세 가지 형식으로 지원합니다:

1. 문자열(String): 단순 텍스트 프롬프트에 적합 (예: `model.invoke("Hello")`)
2. 메시지 객체(Message objects): `SystemMessage` , `HumanMessage` 등 타입이 지정된 인스턴스 리스트
3. 딕셔너리(Dictionary): OpenAI Chat Completion API와 동일한 `{"role": ..., "content": ...}` 구조

세 형식 모두 같은 결과를 돌려주니, 상황에 맞는 걸 고르면 됩니다. 딕셔너리 형식은 기존 OpenAI 코드를 LangChain으로 옮길 때 특히 편합니다.

1.5 스트리밍

`model.stream()` 을 사용하면 토큰이 생성되는 대로 바로 출력됩니다. 사용자 체감 속도가 눈에 띄게 빨라집니다.

LangChain 모델은 세 가지 호출 방식을 제공합니다:

- `invoke()` : 동기 호출로 전체 응답을 한 번에 반환
- `stream()` : 토큰 단위로 `AIMessageChunk` 객체를 순차 반환하여 실시간 출력 가능
- `batch()` : 여러 요청을 동시에 처리하여 효율적

스트리밍 중에는 각 `AIMessageChunk` 가 점진적으로 합쳐져 최종 메시지를 구성하며, 토큰 사용량도 함께 추적됩니다.

1.6 배치 호출

`model.batch()` 로 여러 질문을 한 번에 보낼 수 있습니다. `invoke()` 를 반복 호출하는 것보다 여러 요청을 병렬로 처리할 때 효율적입니다.

요약

개념	설명
`SystemMessage`	모델의 페르소나·규칙 설정
`HumanMessage`	사용자 입력
`model.invoke()`	동기 호출 (전체 응답)
`model.stream()`	토큰 단위 실시간 출력
`model.batch()`	여러 요청 동시 처리

02 LangChain 입문

첫 번째 에이전트

LangChain v1의 핵심 API로 도구를 갖춘 에이전트를 만들어 봅니다.

학습 목표

LEARNING OBJECTIVES

- → `@tool` 데코레이터로 커스텀 도구를 정의한다
- → `create_agent()` 로 에이전트를 생성한다
- → `invoke()` 로 에이전트를 실행하고 결과를 확인한다

2.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 준비 완료")
```

```
✓ 모델 준비 완료
```

2.2 도구 만들기

`@tool` 데코레이터를 붙이면 일반 함수가 에이전트 도구가 됩니다.

도구를 정의할 때 알아야 할 핵심 규칙:

- 타입 힌트(Type Hints): 파라미터의 타입 힌트가 도구 입력 스키마를 자동으로 결정합니다. 예를 들어 `a: int` 는 정수형 입력임을 모델에 알려줍니다.
- Docstring: 함수의 docstring이 도구 설명(description)으로 쓰입니다. 모델은 이 설명을 보고 어떤 도구를 쓸지 판단하므로, 간결하고 명확하게 적어 두세요.
- 커스텀 이름/설명: `@tool("custom_name", description="...")` 형태로 이름과 설명을 직접 지정할 수도 있습니다.

- 복잡한 입력: Pydantic `BaseModel` 과 `Field` 로 복잡한 입력 스키마를 정의할 수 있습니다.

```python
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

print("도구 목록:")
for t in [add, multiply]:
    print(f"  - {t.name}: {t.description}")
```

```
도구 목록:
  - add: 두 수를 더합니다.
  - multiply: 두 수를 곱합니다.
```

2.3 에이전트 생성 & 실행

`create_agent()` 는 모델과 도구를 결합해 에이전트를 만듭니다. 에이전트는 내부적으로 ReAct(Reasoning + Acting) 루프를 실행합니다:

```
질문 → 모델이 도구 호출 결정 → 도구 실행 → 결과 관찰 → 반복 또는 최종 응답 생성
```

에이전트의 핵심 구성 요소:

- 모델(Model): LLM이 어떤 도구를 호출할지 판단합니다. 문자열(`"openai:gpt-5"`) 또는 모델 객체를 전달할 수 있습니다.
- 도구(Tools): 에이전트가 수행할 수 있는 액션입니다. 단순 바인딩과 달리 순차 호출, 병렬 실행, 재시도 등을 지원합니다.
- 시스템 프롬프트(System Prompt): 에이전트의 행동을 안내하는 지침입니다.

에이전트는 도구를 여러 번 순차 호출하거나 병렬로 실행할 수 있습니다. 최종 응답을 생성하거나 반복 횟수 제한에 도달하면 루프가 끝납니다.

```python
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="당신은 수학 도우미입니다.",
)
print("\u2713 에이전트 생성 완료")
```

```
✓ 에이전트 생성 완료
```

요약

핵심 API	역할
`\@tool`	함수를 에이전트 도구로 변환
`create_agent()`	모델 + 도구 → 에이전트 생성
`agent.invoke()`	에이전트 실행, 결과 반환

03 LangChain 대화

멀티턴 메모리

에이전트가 이전 대화를 기억하도록 `InMemorySaver`를 연결합니다.

학습 목표

LEARNING OBJECTIVES

- → `InMemorySaver`로 대화 상태를 저장한다
- → `thread_id`로 대화 세션을 구분한다
- → 에이전트가 이전 문맥을 기억하는 멀티턴 대화를 실행한다

3.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 준비 완료")
```

```
✓ 모델 준비 완료
```

3.2 메모리 없는 에이전트의 한계

기본 에이전트는 상태를 저장하지 않습니다. 매번 새로운 대화로 취급합니다.

3.3 InMemorySaver로 메모리 추가

`checkpointer`를 지정하면 에이전트가 대화 히스토리를 저장합니다. `thread_id`로 대화 세션을 구분합니다.

단기 메모리(Short-term Memory)란 단일 대화 스레드 안에서 이전 상호작용의 정보를 유지하는 기능입니다. 여러 번의 사용자 응답을 처리하는 에이전트라면 꼭 필요합니다.

구현 방법:

- `InMemorySaver()` 를 `checkpointer` 파라미터로 전달해 메모리를 켭니다.
- `thread_id` 로 서로 다른 대화 세션을 구분합니다. 같은 `thread_id` 를 쓰면 이전 내용을 이어서 기억합니다.
- 프로덕션에서는 `PostgresSaver` 같은 DB 기반 체크포인터로 영속성을 확보합니다.

대화가 길어지면 토큰 제한을 초과할 수 있으므로, 메시지 트리밍(trimming), 삭제(deletion), 요약(summarization) 등으로 히스토리를 관리하세요.

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model=model,
    tools=[add],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "session-1"}}
print("\u2713 메모리 에이전트 생성 완료")
```

```
✓ 메모리 에이전트 생성 완료
```

3.4 스트리밍으로 실시간 확인

`agent.stream()` 으로 에이전트의 각 단계를 실시간으로 관찰합니다.

에이전트 스트리밍은 오래 실행되는 에이전트의 중간 단계를 실시간으로 들여다볼 때 유용합니다. `stream_mode="updates"` 를 쓰면 각 노드(모델 호출, 도구 실행 등)의 업데이트를 하나씩 받아볼 수 있어서, 에이전트가 어떤 도구를 호출했는지, 어떤 결과를 받았는지 단계별로 확인할 수 있습니다.

요약

개념	설명
`InMemorySaver`	메모리 내 대화 상태 저장 (체크포인터)
`thread_id`	대화 세션 구분 키
`checkpointer=`	`create_agent()` 에 체크포인터 전달
`stream(mode="updates")`	에이전트 실행 단계를 실시간 확인

04 LangGraph 입문

워크플로 만들기

LangGraph의 `StateGraph`로 노드와 엣지를 연결하는 워크플로를 만들어 봅니다.

학습 목표

LEARNING OBJECTIVES

- → `StateGraph`로 상태 기반 그래프를 정의한다
- → 노드(함수)를 등록하고 엣지로 연결한다
- → `compile()` → `invoke()`로 그래프를 실행한다

4.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 준비 완료")
```

```
✓ 모델 준비 완료
```

4.2 첫 번째 그래프

LangGraph의 기본 흐름은 5단계입니다:

```
StateGraph(State) → add_node() → add_edge() → compile() → invoke()
```

StateGraph의 핵심 개념:

LangGraph는 에이전트 워크플로를 그래프로 모델링하며, 세 가지 기본 구성 요소를 씁니다:

1. State(상태): 애플리케이션의 현재 스냅샷을 담는 공유 데이터 구조입니다. 보통 `TypedDict` 나 Pydantic 모델로 정의합니다.
2. Node(노드): 상태를 받아 연산을 수행하고 업데이트된 상태를 돌려주는 함수입니다. 노드가 실제 작업을 담당합니다.
3. Edge(엣지): 현재 상태를 보고 다음에 실행할 노드를 결정합니다. 엣지가 다음 방향을 지시합니다.

`StateGraph` 는 사용자 정의 State 객체를 받는 주요 그래프 클래스입니다. `.compile()` 메서드로 컴파일한 뒤 사용해야 하며, 이 단계에서 구조 검증이 이뤄집니다.

아래 예제는 텍스트의 단어 수를 세는 간단한 1노드 그래프입니다.

4.3 2노드 그래프

두 개의 노드를 순서대로 연결합니다. 첫 번째 노드가 텍스트를 대문자로 변환하고, 두 번째 노드가 단어 수를 셉니다.

```
START → uppercase → counter → END
```

4.4 LLM을 노드로 사용하기

`MessagesState`를 사용하면 LLM 대화를 그래프로 구성할 수 있습니다.

MessagesState란?

`MessagesState`는 LangGraph가 제공하는 사전 정의 상태 클래스로, `messages` 라는 단일 키와 `add_messages` 리듀서를 씁니다. 내부적으로 다음과 같이 정의되어 있습니다:

```
class MessagesState(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` 리듀서는 메시지 ID를 추적해 중복 없이 메시지를 누적하고, JSON을 LangChain Message 객체로 자동 역직렬화합니다. 문서, 메타데이터 같은 추가 필드가 필요하면 `MessagesState` 를 서브클래싱하면 됩니다.

노드의 구조:

노드는 현재 상태(`state`)를 받아 상태 업데이트를 돌려주는 일반 Python 함수(동기/비동기)입니다. LangGraph는 노드를 자동으로 `RunnableLambda` 객체로 변환하여 배치 처리, 비동기 지원, 네이티브 트레이싱 기능을 더합니다.

요약

핵심 API	역할
`StateGraph(State)`	상태 스키마로 그래프 빌더 생성

`add_node()`	노드(함수) 등록
`add_edge()`	노드 간 연결
`compile()`	실행 가능한 그래프 생성
`invoke()`	그래프 실행

05 Deep Agents 입문

올인원 에이전트

Deep Agents SDK의 `create_deep_agent()` 로 도구·메모리·백엔드가 내장된 에이전트를 한 줄로 만들어 봅니다.

학습 목표

> **LEARNING OBJECTIVES**
>
> → `create_deep_agent()` 로 에이전트를 생성한다
>
> → `invoke()` 로 에이전트를 실행한다
>
> → 커스텀 도구를 추가한 에이전트를 만든다

5.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 모델 준비 완료")
```

```
✓ 모델 준비 완료
```

5.2 에이전트 생성

`create_deep_agent()` 는 LangChain 모델을 받아 파일 읽기/쓰기/검색, `execute` , `task` 같은 빌트인 도구가 포함된 에이전트를 반환합니다. 반환 타입이 LangGraph의 `CompiledStateGraph` 이므로 `invoke()` , `stream()` 등을 그대로 쓸 수 있습니다.

Deep Agents란?

Deep Agents는 에이전트 개발을 단순하게 만들기 위해 설계된 프레임워크로, 에이전트 하네스(harness) 역할을 합니다. LangChain의 기본 에이전트 컴포넌트 위에 구축되며, 실행 관리에는 LangGraph를 씁니다.

핵심 기능:

기능	설명
컨텍스트 관리	파일 시스템 도구(`write_file` , `read_file`)로 토큰 한도를 넘는 중간 산출물을 다룹니다.
유연한 저장소	상태 기반, 로컬 디스크, 영구 store, 샌드박스 환경 등 플러거블 백엔드를 지원합니다.
서브에이전트 위임	특정 하위 작업을 전문 서브에이전트에 맡겨 컨텍스트를 격리합니다.
계획 도구	복잡한 작업에서 `TodoListMiddleware` 를 명시적으로 추가하면 `write_todos` 로 단계 관리를 할 수 있습니다.
영구 메모리	LangGraph store/checkpointer와 Deep Agents memory·skills 계층으로 여러 대화에 걸쳐 정보를 유지합니다.

에이전트 생성 방법:

`create_deep_agent()` 에 모델, 도구, 시스템 프롬프트를 전달하면 됩니다. 계획을 실습하려면 `middleware=[TodoListMiddleware()]` 를 함께 전달합니다. 도구 호출을 지원하는 모델이 필요하며, Anthropic, OpenAI 등 다양한 프로바이더를 쓸 수 있습니다.

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
print(f"\u2713 에이전트 생성 완료 (타입: {type(agent).__name__})")
```

```
✓ 에이전트 생성 완료 (타입: CompiledStateGraph)
```

5.3 커스텀 도구 추가

Python 함수에 docstring과 타입 힌트를 작성하면 그대로 도구가 됩니다.

커스텀 도구의 동작 원리:

커스텀 도구는 일반 Python 함수로 작성하며, 다음 두 가지가 자동으로 변환됩니다:

- docstring → 도구 설명 (에이전트가 이 도구를 언제 써야 하는지 판단하는 근거)
- 타입 힌트 → 파라미터 스키마 (에이전트가 올바른 인자를 넘기는 데 사용)

`create_deep_agent()` 의 `tools` 파라미터에 함수 리스트를 전달하면, 빌트인 도구(파일 읽기/쓰기, todo 등)와 함께 에이전트 도구 목록에 추가됩니다. `system_prompt` 파라미터로 에이전트의 행동 방식을 지정할 수도 있습니다.

요약

핵심 API	역할

`create_deep_agent(model)`	빌트인 도구가 포함된 에이전트 생성
`create_deep_agent(model, tools, system_prompt)`	커스텀 도구 + 시스템 프롬프트
`agent.invoke()`	에이전트 실행

06 세 프레임워크 비교 & 다음 단계

LangChain, LangGraph, Deep Agents를 한눈에 비교하고 중급 과정을 안내합니다.

학습 목표

> **LEARNING OBJECTIVES**
>
> → LangChain, LangGraph, Deep Agents 세 프레임워크의 핵심 차이점을 이해한다
>
> → 각 프레임워크의 적합한 사용 사례를 판단할 수 있다
>
> → 중급 과정으로의 학습 경로를 선택할 수 있다

6.1 프레임워크 비교

추상화 수준	높음	중간	매우 높음
핵심 개념	에이전트 + 도구	그래프 + 상태 + 노드	올인원 에이전트
에이전트 생성	`create_agent()`	`StateGraph` → `compile()`	`create_deep_agent()`
실행	`agent.invoke()`	`graph.invoke()`	`agent.invoke()`
커스터마이징	도구/프롬프트/메모리	노드/엣지/상태/리듀서	도구/백엔드/서브에이전트
적합 상황	빠른 프로토타이핑	복잡한 워크플로	파일/태스크 관리가 필요한 에이전트

추가 비교 정보:

특성	LangChain	LangGraph	Deep Agents
모델 지원	모델 무관 (100개 이상 프로바이더)	LangChain 모델 공유	LangChain 모델 공유
라이선스	MIT	MIT	MIT
샌드박스 통합	기본 지원 없음	기본 지원 없음	에이전트가 샌드박스에서 작업 실행 가능

상태 관리	메모리 기반	체크포인터 기반 (타임 트래블 지원)	LangGraph 체크포인터 활용
관측성	LangSmith 연동	LangSmith 네이티브 트레이싱	LangSmith 지원

세 프레임워크는 서로 배타적이지 않습니다. Deep Agents는 내부적으로 LangGraph를 쓰고, LangChain의 모델/도구 인터페이스를 공유합니다. LangChain으로 기본기를 다지고, LangGraph로 복잡한 워크플로를 설계한 뒤, Deep Agents로 프로덕션급 에이전트를 구축하는 순서가 자연스럽습니다.

6.2 어떤 걸 선택해야 할까?

```
"간단한 도구 호출 에이전트가 필요해"     → LangChain
"조건 분기·루프가 있는 워크플로가 필요해" → LangGraph
"파일·서브에이전트·긴 작업 관리가 필요해" → Deep Agents
```

> ➡️ TIP
>
> 세 프레임워크는 서로 배타적이지 않습니다. Deep Agents는 내부적으로 LangGraph를 사용하고, LangChain의 모델·도구 인터페이스를 공유합니다. 계획 도구가 필요하면 `TodoListMiddleware` 를 명시적으로 추가합니다.

6.3 코드 비교 — 같은 질문, 세 가지 방식

아래는 동일한 작업을 세 프레임워크로 처리하는 최소 코드입니다.

LangChain

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

agent = create_agent(model=model, tools=[add])
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

LangGraph

```python
from langgraph.graph import StateGraph, START, END, MessagesState

def chatbot(state):
    return {"messages": [model.invoke(state["messages"])]}
```

```
builder = StateGraph(MessagesState)
builder.add_node("chat", chatbot)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)
graph = builder.compile()
graph.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

Deep Agents

```
from deepagents import create_deep_agent

agent = create_deep_agent(model=model)
agent.invoke({"messages": [{"role": "user", "content": "3+4?"}]})
```

요약

항목	LangChain	LangGraph	Deep Agents
핵심 역할	에이전트 생성 (도구 호출 루프)	워크플로 오케스트레이션 (상태 그래프)	하네스 기반 장기 실행 에이전트
상태 관리	미들웨어·체크포인터 사용	StateGraph 명시적 상태	LangGraph 체크포인터와 백엔드 사용
파일·위임	직접 구성	직접 구성	파일 도구와 `task` 위임 제공
계획 도구	직접 추가	직접 구현	`TodoListMiddleware`를 명시적으로 추가
적합 대상	빠른 프로토타이핑, 도구 호출	복잡한 워크플로, 조건 분기	코딩 에이전트, 데이터 분석, 리서치
학습 곡선	낮음	중간	낮음

미니 프로젝트

→ 07_mini_project.ipynb: 검색 + 요약 에이전트를 만드는 실습

중급 과정

과정	설명	노트북 수
#link("../02_langchain/")[LangChain 중급]	모델/메시지/도구/메모리/미들웨어/멀티에이전트	10
#link("../03_langgraph/")[LangGraph 중급]	Graph API/워크플로/에이전트/퍼시스턴스/서브그래프	10

#link("../04_deepagents/")[Deep Agents 중급]	커스터마이징/백엔드/서브에이전트/메모리/고급 기능	7

권장 학습 순서:

1. LangChain — 모델/도구의 기본기를 다진다
2. LangGraph — 그래프 기반 워크플로를 설계한다
3. Deep Agents — 프로덕션급 에이전트를 구축한다

07 미니 프로젝트

검색 + 요약 에이전트

입문 과정에서 배운 내용을 조합하여, Tavily 웹 검색 도구를 갖춘 리서치 에이전트를 만듭니다.

학습 목표

LEARNING OBJECTIVES

- → Tavily 검색 도구를 직접 정의한다
- → Deep Agents로 리서치 에이전트를 만든다
- → 스트리밍으로 에이전트 실행 과정을 실시간 관찰한다
- → LangChain 에이전트로도 같은 작업을 수행하여 비교한다

7.1 환경 설정

이 노트북에는 `TAVILY_API_KEY` 가 필요합니다. https://tavily.com 에서 무료로 발급받을 수 있습니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY 필요!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY 필요!"

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-4.1")
print("\u2713 환경 준비 완료")
```

```
✓ 환경 준비 완료
```

7.2 검색 도구 정의

Tavily 클라이언트를 감싸는 검색 함수를 만듭니다. docstring과 타입 힌트가 에이전트에 도구 스키마를 알려줍니다.

도구 함수 작성 규칙:

`create_deep_agent()`의 `tools` 파라미터에 전달할 검색 함수를 정의합니다. Deep Agents는 함수의 docstring을 도구 설명으로, 타입 힌트를 파라미터 스키마로 자동 변환합니다. 따라서:

- docstring: 에이전트가 "이 도구를 언제 써야 하는지" 판단하는 근거가 됩니다. 명확하고 구체적으로 적으세요.
- 타입 힌트: 에이전트가 올바른 타입의 인자를 전달하도록 합니다. `Literal` 타입을 쓰면 허용 값을 제한할 수 있습니다.
- Args 섹션: 각 파라미터의 용도를 설명하면 에이전트가 더 정확하게 인자를 고릅니다.

```
from typing import Literal
from tavily import TavilyClient

tavily = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def internet_search(
    query: str,
    max_results: int = 3,
    topic: Literal["general", "news"] = "general",
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색 쿼리
        max_results: 최대 결과 수
        topic: 검색 주제 카테고리
    """
    return tavily.search(query, max_results=max_results, topic=topic)

print("\u2713 검색 도구 준비 완료")
```

```
✓ 검색 도구 준비 완료
```

7.3 Deep Agents 리서치 에이전트

`create_deep_agent()`에 검색 도구와 시스템 프롬프트를 전달합니다.

에이전트의 워크플로:

1. 리서치 수행: 전달된 검색 도구(`internet_search`)로 웹에서 정보를 모읍니다.
2. 컨텍스트 관리: 필요하면 파일 시스템 도구(`write_file`, `read_file`)로 중간 결과를 저장해 토큰 한도를 관리합니다.
3. 결과 종합: 수집한 정보를 분석하고 일관된 보고서로 정리합니다.
4. 계획 관리(선택): 작업 단계 추적이 필요한 수업에서는 `TodoListMiddleware`를 추가해 `write_todos` 도구를 노출합니다.

복잡한 작업에서는 전문 서브에이전트를 만들어 특정 하위 작업의 컨텍스트를 격리할 수도 있습니다.

```
from deepagents import create_deep_agent

research_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 전문 리서처입니다. 웹을 검색한 후 결과를 한국어로 요약하세요.",
)
print("\u2713 리서치 에이전트 생성 완료")
```

```
✓ 리서치 에이전트 생성 완료
```

7.4 스트리밍으로 과정 관찰

`stream(mode="updates")` 로 에이전트가 어떤 단계를 거치는지 실시간으로 확인합니다.

LangGraph 스트리밍 시스템:

LangGraph는 완전한 응답이 준비되기 전에 진행 상황을 점진적으로 보여줘 애플리케이션의 반응성을 높이는 스트리밍 시스템을 제공합니다.

스트림 모드	용도
`values`	각 그래프 단계 후 전체 상태를 스트리밍
`updates`	각 단계 후 상태 변경분만 스트리밍
`messages`	LLM 토큰을 메타데이터와 함께 스트리밍
`custom`	노드에서 사용자 정의 데이터를 스트리밍
`debug`	포괄적인 실행 정보를 스트리밍

`stream()` (동기) 또는 `astream()` (비동기) 메서드로 스트리밍에 접근하며, 여러 모드를 리스트로 넘겨 동시에 쓸 수도 있습니다. 아래 예제에서는 `updates` 모드로 에이전트의 각 단계(도구 호출, 최종 응답)를 실시간으로 출력합니다.

7.5 LangChain 에이전트로 비교

같은 검색 도구를 LangChain `create_agent()` 로도 사용해 봅니다.

LangChain 에이전트와의 차이점:

LangChain의 `create_agent()` 는 모델과 도구를 받아 간단한 도구 호출 에이전트를 만듭니다. Deep Agents와 비교하면:

- LangChain: 도구 호출 에이전트의 기본 형태입니다. 빠른 프로토타이핑에 맞지만, 파일 관리나 서브에이전트 위임은 직접 구성해야 합니다.
- Deep Agents: 파일 관리, 실행 백엔드, 서브에이전트 위임, skills/memory를 하네스 안에서 조립합니다. 계획 도구는 `TodoListMiddleware` 를 추가해 명시적으로 켭니다.

`@tool` 데코레이터를 쓰면 LangChain 도구 인터페이스에 맞게 함수를 변환할 수 있습니다. 시스템 프롬프트와 도구 리스트를 전달하는 패턴은 두 프레임워크가 공유합니다.

요약

이 미니 프로젝트에서 사용한 기술:

기술	출처
`ChatOpenAI` + `load_dotenv`	00_setup
메시지 역할, 스트리밍	01_llm_basics
`\@tool` , `create_agent()`	02_langchain_basics
`InMemorySaver` , `thread_id`	03_langchain_memory
`StateGraph` , `compile()`	04_langgraph_basics
`create_deep_agent()`	05_deep_agents_basics

PART

II

LangChain

Building Blocks for AI Applications

이 파트에서 다루는 내용

- 1. 소개
- 2. 퀵스타트
- 3. 모델과 메시지
- 4. 도구와 구조적 출력
- 5. 메모리와 스트리밍
- 6. 미들웨어
- 7. HITL과 런타임
- 8. 멀티 에이전트
- 9. 커스텀 워크플로와 RAG
- 10. 프로덕션
- 11. MCP
- 12. 프론트엔드 스트리밍
- 13. 가드레일
- 14. Semantic Search

01 LangChain 소개

LangChain v1 프레임워크 개요

학습 목표

LangChain 프레임워크의 구조와 핵심 컴포넌트를 이해합니다.

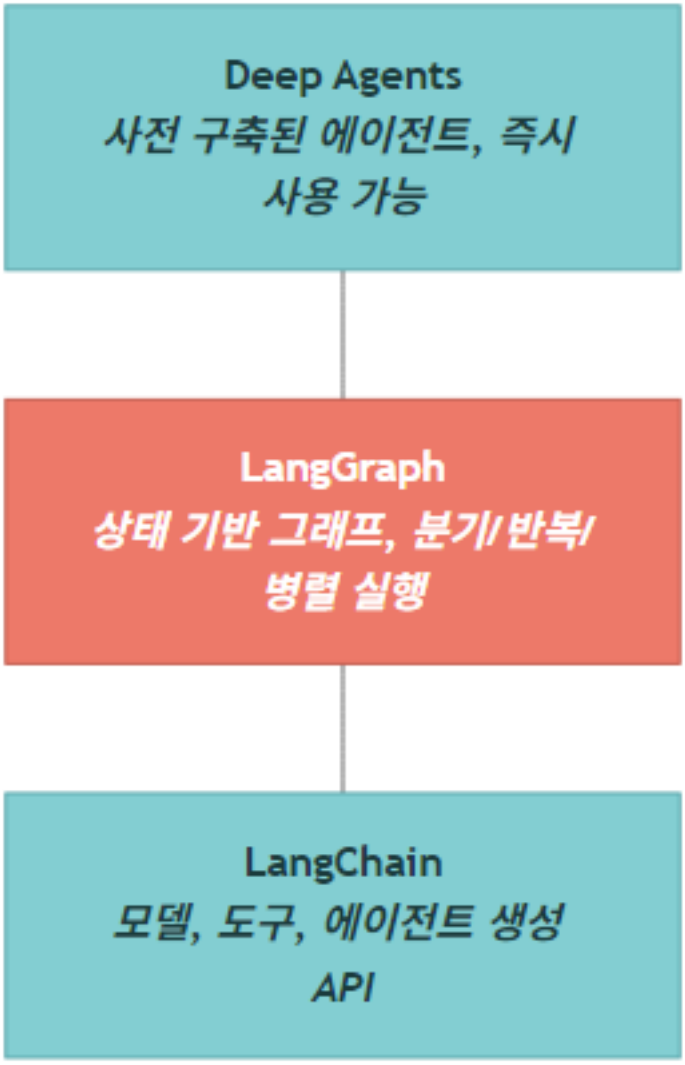
이 노트북을 마치면 다음을 알게 됩니다:

- LangChain v1 프레임워크의 3가지 레이어 구조
- ReAct 에이전트 패턴의 동작 방식
- 핵심 컴포넌트와 주요 API
- 개발 환경 설정 및 검증 방법

1.1 LangChain 프레임워크 개요

LangChain v1은 LLM 기반 에이전트를 구축하는 통합 프레임워크입니다. 3가지 레이어로 구성되며, 각 레이어는 서로 다른 수준의 추상화를 제공합니다.

3가지 레이어 구조



레이어	역할	대상 사용자
LangChain	에이전트 생성의 핵심 API (`create_agent` , `tool` , `ChatOpenAI`)	모든 개발자
LangGraph	복잡한 워크플로 구현 (상태 그래프, 체크포인터, 스트리밍)	중급 이상
Deep Agents	사전 구축된 에이전트 (코딩, 리서치 등)	빠른 프로토타이핑

LangChain v1에서 변경된 주요 사항

항목	이전 (v0.x)	현재 (v1)
에이전트 생성	`create_react_agent()`	`create_agent()`
에이전트 임포트	`from langchain.agents import ...` (다양)	`from langchain.agents import create_agent`

모델 초기화	`ChatOpenAI(...)` 직접 사용	`init_chat_model()` 또는 `ChatOpenAI(...)`
메모리	`ConversationBufferMemory` 등	`InMemorySaver` (LangGraph 체크포인터)
실행 엔진	AgentExecutor	LangGraph 그래프 (내부적으로)

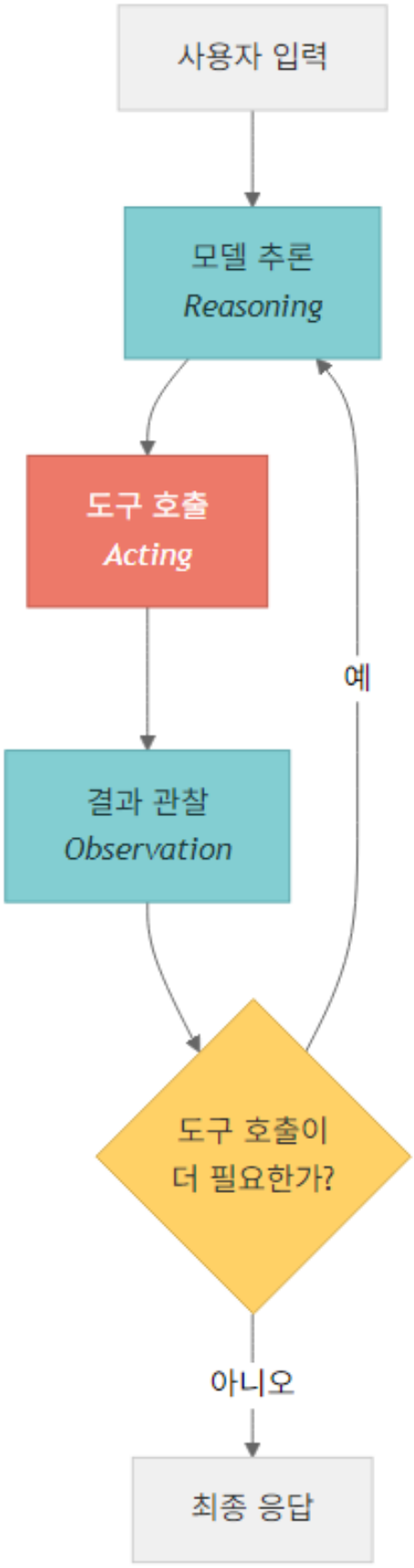
핵심 설계 철학

LangChain v1의 핵심은 모든 에이전트가 LangGraph 그래프로 실행된다는 점입니다. `create_agent()` 로 생성된 에이전트는 내부적으로 LangGraph의 `StateGraph` 로 구현되며, 이로 인해:

- 스트리밍: `stream()` 메서드로 실시간 응답
- 상태 관리: 체크포인터를 통한 대화 히스토리 유지
- 확장성: 커스텀 노드와 엣지 추가 가능

1.2 ReAct 에이전트 패턴

ReAct (Reasoning + Acting) 패턴은 LangChain v1 에이전트의 기본 동작 방식입니다. 에이전트는 다음 루프를 반복합니다:


사용자 입력
모델 추론
Reasoning
도구 호출
Acting
결과 관찰
Observation
도구 호출이
더 필요한가?
예
아니오
최종 응답

핵심 특징

- 자율적 판단: 에이전트가 도구 사용 여부를 스스로 결정합니다
- 다단계 추론: 복잡한 작업을 여러 단계로 분해하여 처리합니다
- 관찰 기반 학습: 도구 결과를 관찰하고 다음 행동을 결정합니다

1.3 주요 컴포넌트 개요

LangChain v1의 핵심 컴포넌트를 표로 정리합니다:

컴포넌트	설명	주요 API
Model	LLM 또는 Chat 모델. 에이전트의 “두뇌” 역할	`ChatOpenAI` , `init_chat_model()`
Tools	에이전트가 사용할 수 있는 함수. 검색, 계산, API 호출 등	`\@tool` 데코레이터, `TavilySearch`
Agent	모델과 도구를 결합한 실행 단위. 내부적으로 LangGraph 그래프	`create_agent()`
Memory	대화 히스토리를 저장하고 관리하는 체크포인터	`InMemorySaver` , `SqliteSaver`
Middleware	요청/응답 처리 파이프라인에 로직 삽입	`prompt` , `before_tool` , `after_model`
State	에이전트 실행 중 관리되는 상태 (메시지, 컨텍스트 등)	`AgentState` , `messages`
Streaming	실시간 응답 스트리밍 지원	`stream()` , `stream_mode="updates"`

1.4 환경 설정 및 설치 확인

LangChain v1 개발에 필요한 패키지와 API 키를 확인합니다.

설치

LangChain v1 기반 실습에 필요한 핵심 패키지를 설치합니다. `uv` 또는 `pip` 중 환경에 맞는 명령을 쓰면 됩니다.

```
# uv 사용자
uv add langchain deepagents langgraph langchain-openai langchain-anthropic

# pip 사용자
pip install -U langchain deepagents langgraph langchain-openai langchain-anthropic
```

> · NOTE
>
> 노트북 안에서 바로 설치하려면 첫 셀에 `%pip install -U langchain deepagents` 를 넣어도 됩니다.

자주 쓰는 기본 모델 ID

프로바이더	권장 모델 ID	메모
OpenAI	`gpt-5.4`	`init_chat_model("openai:gpt-5.4")` 형태로도 호출
Anthropic	`claude-sonnet-4-6`	안정적 추론 + 도구 사용 균형
Google	`gemini-2.5-flash-lite`	경량·저지연용

본문 예시는 별다른 언급이 없으면 `gpt-5.4` 를 기본 모델로 씁니다.

```
# 환경 설정
import subprocess
import importlib

packages = {
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
    "langchain_community": "langchain-community",
    "langgraph": "langgraph",
}

print("=" * 50)
print("LangChain v1 환경 확인")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"\u2713 {package_name}: {version}")
    except ImportError:
        print(f"\u2717 {package_name}: 미설치 \u2192 pip install {package_name}")
```

```
==================================================
LangChain v1 환경 확인
==================================================
✓ langchain: 1.2.10

✓ langchain-openai: installed
✓ langchain-community: 0.4.1
✓ langgraph: installed
```

```
# API 키 확인
from dotenv import load_dotenv
import os

load_dotenv(override=True)

required_keys = ["OPENAI_API_KEY"]
optional_keys = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("필수 API 키:")
for key in required_keys:
    status = "\u2713 설정됨" if os.environ.get(key) else "\u2717 미설정"
    print(f"  {key}: {status}")

print("\n선택 API 키:")
for key in optional_keys:
    status = "\u2713 설정됨" if os.environ.get(key) else "- 미설정 (선택)"
    print(f"  {key}: {status}")
```

```
필수 API 키:
  OPENAI_API_KEY: ✓ 설정됨

선택 API 키:
  TAVILY_API_KEY: ✓ 설정됨
  LANGSMITH_API_KEY: - 미설정 (선택)
```

```python
# LangChain v1 핵심 import 확인
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver

print("\u2713 모든 핵심 모듈 임포트 성공")
print("  - create_agent: LangChain v1 에이전트 생성")
print("  - tool: 도구 데코레이터")
print("  - ChatOpenAI: OpenAI 호환 모델")
print("  - InMemorySaver: 메모리 체크포인터")
```

```
✓ 모든 핵심 모듈 임포트 성공
  - create_agent: LangChain v1 에이전트 생성
  - tool: 도구 데코레이터
  - ChatOpenAI: OpenAI 호환 모델
  - InMemorySaver: 메모리 체크포인터
```

요약

항목	설명
LangChain v1	에이전트 중심 프레임워크, `create_agent()` API 기반
3계층 구조	모델 → 도구 → 미들웨어
ReAct 패턴	추론(Reasoning) → 행동(Action) → 관찰(Observation) 반복
핵심 API	`create_agent()`, `\@tool`, `invoke()`, `stream()`

02 첫 번째 에이전트

`create_agent()` 로 시작하기

학습 목표

LangChain v1의 `create_agent()` 로 에이전트를 생성하고 실행합니다.

이 노트북을 마치면 다음을 할 수 있습니다:

- `@tool` 데코레이터로 커스텀 도구를 정의
- `create_agent()` 로 에이전트를 생성
- `invoke()` 로 에이전트를 실행하고 결과를 확인
- `stream()` 으로 실시간 스트리밍 응답을 받기
- `InMemorySaver` 로 멀티턴 대화를 구현

설치

처음 실행하는 환경이라면 핵심 패키지를 먼저 깔아 둡니다. 이미 깔려 있다면 다음 셀은 건너뛰어도 됩니다.

```
# uv 환경이라면 아래 한 줄로 충분합니다.
# !uv add langchain deepagents langgraph langchain-openai

# pip 환경이라면 다음을 씁니다.
%pip install -q -U langchain deepagents langgraph langchain-openai
```

2.1 환경 설정

OpenAI를 통해 모델을 설정합니다. `ChatOpenAI` 는 OpenAI 호환 API를 지원하므로, `base_url` 을 변경하여 OpenAI를 사용할 수 있습니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)
print("✓ 모델 설정 완료:", model.model_name)
```

2.2 간단한 도구 만들기

`@tool` 데코레이터로 에이전트가 사용할 도구를 정의합니다.

도구를 정의할 때 주의할 점:

- docstring은 필수입니다. 에이전트가 도구의 용도를 이해하는 데 씁니다.
- 타입 힌트를 쓰면 에이전트가 올바른 인자를 전달할 수 있습니다.
- 도구 이름은 함수명에서 자동으로 생성됩니다.

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

print("도구 목록:")
for t in [add, multiply]:
    print(f"  - {t.name}: {t.description}")
```

```
도구 목록:
  - add: 두 수를 더합니다.
  - multiply: 두 수를 곱합니다.
```

2.3 에이전트 생성

`create_agent()` 로 모델과 도구를 결합합니다.

생성된 에이전트는 내부적으로 LangGraph 그래프로 구현되며, `invoke()`, `stream()` 등의 메서드를 제공합니다.

> ➡️ TIP
>
> LangChain v1에서는 `create_react_agent()` 대신 `create_agent()` 를 사용합니다.

```
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[add, multiply],
    system_prompt="당신은 수학 도우미입니다. 제공된 도구를 사용하여 계산하세요.",
)
print("\u2713 에이전트 생성 완료")
print(f"  타입: {type(agent).__name__}")
```

```
✓ 에이전트 생성 완료
  타입: CompiledStateGraph
```

2.4 에이전트 실행

`invoke()` 로 에이전트를 실행합니다.

에이전트에 메시지를 전달하면, 내부적으로 ReAct 루프가 실행됩니다:

1. 모델이 질문을 분석하고 도구 호출을 결정
2. 도구가 실행되고 결과를 반환
3. 모델이 결과를 바탕으로 최종 응답을 생성

```python
# 전체 메시지 흐름 확인
print("전체 메시지 흐름:")
print("=" * 50)
for msg in result["messages"]:
    role = msg.type if hasattr(msg, 'type') else msg.get('role', 'unknown')
    content = msg.content if hasattr(msg, 'content') else msg.get('content', '')
    print(f"[{role}] {content[:200]}")
    print("-" * 50)
```

```
전체 메시지 흐름:
==================================================
[human] 15 + 27은 얼마인가요?
--------------------------------------------------
[ai]
--------------------------------------------------
[tool] 42
--------------------------------------------------
[ai] 15 + 27은 42입니다.
--------------------------------------------------
```

2.5 스트리밍 실행

`stream()` 으로 실시간 응답을 받습니다.

스트리밍을 쓰면 에이전트의 각 단계(모델 추론, 도구 호출, 최종 응답)를 실시간으로 확인할 수 있습니다.
`stream_mode="updates"` 로 각 노드의 업데이트를 순차적으로 받을 수 있습니다.

2.6 멀티턴 대화

`InMemorySaver` 로 대화 상태를 유지합니다.

`InMemorySaver` 는 메모리 내에서 상태를 저장하며, `thread_id` 로 대화 세션을 구분합니다.

> ➡️ TIP
>
> LangChain v1에서는 LangGraph의 체크포인터로 대화 히스토리를 관리합니다.

2.7 Tavily 검색 도구 연동 (선택)

웹 검색 도구를 추가하여 실제 정보를 검색합니다.

Tavily는 AI 에이전트를 위해 설계된 검색 API입니다.

`TAVILY_API_KEY`가 설정된 경우에만 이 셀이 실행됩니다.

2.8 컨텍스트 주입 — `context_schema`와 `context`

호출 시점마다 달라지는 정보(사용자 ID, 권한, 부서 등)는 `context_schema`로 타입을 정의하고 `invoke()` 시 `context=...`로 전달합니다. 시스템 프롬프트나 도구 안에서 `runtime.context`로 꺼내 쓸 수 있어, 상태(state)와 메시지 이력을 어지럽히지 않고 메타데이터를 주입하기 좋습니다.

> ➡️ TIP
>
> state_schema는 TypedDict 서브클래스여야 합니다. dataclass·Pydantic은 `context_schema` 쪽에 쓰고, 상태 그래프 본문은 `TypedDict`로 정의하는 것이 LangChain v1 / LangGraph의 권장 패턴입니다.

> ➡️ TIP
>
> 에이전트 네이밍은 snake_case로. `name="research_assistant"` 처럼 단일 단어 또는 snake_case가 트레이싱과 멀티에이전트 라우팅에서 깔끔하게 보입니다. 공백·하이픈·대문자는 피하세요.

요약

이 노트북에서 다룬 내용:

주제	핵심 API	설명
도구 정의	`\@tool`	함수에 데코레이터를 추가하여 에이전트 도구로 변환
에이전트 생성	`create_agent()`	모델 + 도구 + 시스템 프롬프트를 결합
동기 실행	`agent.invoke()`	완전한 응답을 한 번에 반환
스트리밍 실행	`agent.stream()`	각 단계의 업데이트를 실시간으로 반환
멀티턴 대화	`InMemorySaver` + `thread_id`	체크포인터로 대화 상태를 저장/복원
검색 도구	`TavilySearch`	웹 검색으로 실시간 정보에 접근
컨텍스트 주입	`context_schema` + `context=`	호출별 메타데이터를 상태와 분리해 전달

03 모델과 메시지 시스템

LangChain v1에서 다양한 LLM 모델을 설정하고, 메시지 타입을 활용하여 대화를 구성하는 방법을 학습합니다.

학습 목표

LEARNING OBJECTIVES

- → LangChain v1의 모델 초기화 방법(`init_chat_model` , `ChatOpenAI`)을 이해합니다
- → `invoke()` , `stream()` , `batch()` 세 가지 호출 패턴을 학습합니다
- → `SystemMessage` , `HumanMessage` , `AIMessage` , `ToolMessage` 등 메시지 타입을 이해합니다
- → 멀티모달 메시지(이미지 입력)를 구성하는 방법을 익힙니다

3.1 환경 설정

`.env` 파일에서 API 키를 로드하고, OpenAI를 통해 모델을 초기화합니다.

```python
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# OpenAI를 통한 모델 초기화
model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 초기화 완료:", model.model_name)
```

3.2 모델 프로바이더 비교

LangChain v1은 `init_chat_model()` 로 다양한 프로바이더의 모델을 통합된 방식으로 초기화할 수 있습니다.

프로바이더	모델 문자열 형식	필요 패키지	환경 변수

OpenAI	`"openai:gpt-5.4"`	`langchain-openai`	`OPENAI_API_KEY`
Anthropic	`"anthropic:claude-sonnet-4-6"`	`langchain-anthropic`	`ANTHROPIC_API_KEY`
Google	`"google:gemini-2.5-flash-lite"`	`langchain-google-genai`	`GOOGLE_API_KEY`
AWS Bedrock	`"bedrock:anthropic.claude-v3"`	`langchain-aws`	AWS credentials
Azure	`"azure:gpt-4o"`	`langchain-openai`	`AZURE_OPENAI_API_KEY`
Ollama	`"ollama:llama3"`	`langchain-ollama`	(로컬 실행)

> · NOTE
>
> 참고: OpenAI를 사용하는 경우, `ChatOpenAI` 에 `base_url` 과 `api_key` 를 직접 지정하여 OpenAI API 형식의 서비스(vLLM, LMStudio, Ollama 등)에 접근할 수 있습니다.

3.3 init_chat_model() 사용법

`init_chat_model()` 은 LangChain v1이 제공하는 통합 모델 초기화 함수입니다. 프로바이더별 패키지가 설치되어 있으면, 문자열 하나로 모델을 생성할 수 있습니다.

OpenAI를 쓸 때는 `ChatOpenAI` 를 직접 사용하는 편이 더 간편합니다.

3.4 invoke(), stream(), batch() 패턴

LangChain v1의 모든 모델은 세 가지 호출 패턴을 지원합니다:

메서드	설명	반환 타입
`invoke()`	단일 입력에 대한 단일 응답	`AIMessage`
`stream()`	토큰 단위로 스트리밍 응답	`Iterator[AIMessageChunk]`
`batch()`	여러 입력을 동시에 처리	`List[AIMessage]`

3.5 메시지 타입

LangChain v1의 메시지 시스템은 대화의 각 역할을 명확히 구분합니다:

메시지 타입	역할	설명
`SystemMessage`	시스템	모델의 행동 방식을 지시하는 시스템 프롬프트
`HumanMessage`	사용자	사용자가 입력하는 메시지

`AIMessage`	AI	모델이 생성한 응답
`ToolMessage`	도구	도구 실행 결과를 모델에 전달

메시지 리스트를 구성하여 `model.invoke()`에 전달하면, 대화 맥락을 유지한 응답을 받을 수 있습니다.

3.6 멀티모달 메시지 (이미지 입력)

LangChain v1에서는 `HumanMessage`의 `content`에 텍스트와 이미지를 함께 전달할 수 있습니다. 이미지는 URL 또는 base64 인코딩으로 전달하며, 비전(Vision)을 지원하는 모델에서만 동작합니다.

```
content = [
    {"type": "text", "text": "설명 텍스트"},
    {"type": "image_url", "image_url": {"url": "이미지_URL"}},
]
```

3.7 토큰 사용량 집계 — `UsageMetadataCallbackHandler`

비용·쿼터를 추적하려면 `UsageMetadataCallbackHandler`를 콜백으로 붙입니다. 모델·에이전트 호출이 끝난 뒤 누적 토큰 수치를 dict로 받을 수 있어, 한 세션의 합산이나 모델별 비교에 바로 씁니다.

```
from langchain_core.callbacks import UsageMetadataCallbackHandler

cb = UsageMetadataCallbackHandler()

# 단일 모델 호출에 콜백 부착
_ = model.invoke(
    "LangChain v1을 한 줄로 소개해 주세요.",
    config={"callbacks": [cb]},
)

# 동일 핸들러를 여러 호출에 재사용하면 누적 집계됩니다.
_ = model.invoke(
    "한 줄 더 추가해 주세요.",
    config={"callbacks": [cb]},
)

print("누적 usage_metadata:")
print(cb.usage_metadata)
```

3.8 Runtime Config — `run_name` · `tags` · `metadata` · `max_concurrency`

`config` 파라미터에는 단순 콜백만이 아니라 LangSmith·Langfuse에서 식별에 쓰는 메타데이터를 함께 실어 보낼 수 있습니다. `batch()` 호출에서는 `max_concurrency`로 동시 실행 상한을 정해 토큰 폭주를 막습니다.

```
runtime_cfg = {
    "run_name": "intro-greeting",
    "tags": ["demo", "ch03"],
    "metadata": {"experiment": "v1_runtime_config"},
    "max_concurrency": 5,
```

```
}

# 단일 호출 — run_name/tags/metadata가 트레이스에 노출됩니다.
resp = model.invoke("한국어로 짧게 인사해 주세요.", config=runtime_cfg)
print("응답:", resp.content)

# batch — max_concurrency로 동시 실행 상한을 정합니다.
batched = model.batch(
    [f"숫자 {i}을 한글로 적어 주세요." for i in range(1, 7)],
    config=runtime_cfg,
)
for i, r in enumerate(batched, start=1):
    print(f"{i}: {r.content}")
```

3.9 연결 회복력 — `max_retries` 와 `timeout`

프로덕션에서는 일시적 5xx, 레이트리밋, 네트워크 끊김이 잦습니다. `init_chat_model()` 이나 `ChatOpenAI` 에 `max_retries` 와 `timeout` 을 미리 잡아 두면 지수 백오프 재시도가 자동으로 끼어들고, 한 호출이 무한 대기에 빠지는 일도 막을 수 있습니다.

3.10 표준 출력 — `LC_OUTPUT_VERSION` / `output_version="v1"`

LangChain v1은 모델 응답을 `content_blocks` (텍스트·이미지·툴콜·추론 블록의 통일된 리스트)로 노출하는 표준 직렬화를 도입했습니다. 사용 방법은 두 가지입니다.

- 환경 변수 `LC_OUTPUT_VERSION=v1` 을 띄우면 모든 모델이 자동으로 v1 포맷을 씁니다.
- 또는 모델 생성 시 `output_version="v1"` 을 직접 지정합니다.

content_blocks를 쓰면 멀티모달·툴콜·추론 토큰을 일관된 방식으로 다룰 수 있어, 멀티 프로바이더 코드를 깔끔하게 유지할 수 있습니다.

3.11 프롬프트 캐싱 — Implicit vs Explicit

긴 시스템 프롬프트나 RAG 컨텍스트를 반복 호출할 때, 프롬프트 캐싱은 비용·지연을 크게 줄여 줍니다. 두 가지 흐름이 있습니다.

- Implicit caching — 프로바이더(예: OpenAI gpt-5.4)가 동일 prefix를 자동 감지·재사용합니다. 호출자는 코드 변경이 거의 없어도 됩니다.
- Explicit caching — Anthropic 계열은 `prompt_cache_key` 나 `cache_control` 블록으로 캐싱 단위를 명시합니다. 동일 키 호출이 들어오면 캐시 hit가 보장됩니다.

캐시 히트 여부는 응답의 `usage_metadata.input_token_details.cache_read` 등에 잡힙니다. 위 `UsageMetadataCallbackHandler` 로 함께 추적하면 운영 중 효과를 즉시 측정할 수 있습니다.

요약

이 노트북에서 학습한 핵심 내용:

항목	설명
`init_chat_model()`	프로바이더 문자열로 모델을 통합 초기화
`ChatOpenAI(base_url=...)`	OpenAI 등 커스텀 엔드포인트 사용
`invoke()`	단일 입력 → 단일 응답
`stream()`	토큰 단위 스트리밍 응답
`batch()`	여러 입력을 동시에 처리
`SystemMessage`	시스템 지시사항 설정
`HumanMessage`	사용자 입력 메시지
`AIMessage`	AI 응답 메시지 (대화 이력용)
`ToolMessage`	도구 실행 결과 전달
멀티모달 메시지 + `output_version="v1"`	`content_blocks` 로 텍스트·이미지 통일 직렬화
`UsageMetadataCallbackHandler`	호출 누적 토큰·캐시 통계 수집
Runtime config	`run_name` · `tags` · `metadata` · `max_concurrency` 등 트레이스 친화 설정
`max_retries` · `timeout`	운영용 연결 회복력
`LC_OUTPUT_VERSION=v1`	환경 변수 한 줄로 표준 출력 적용
Prompt caching	implicit(자동)·explicit(`prompt_cache_key`)로 비용·지연 절감

04 도구와 구조화된 출력

LangChain v1에서 `@tool` 데코레이터로 커스텀 도구를 만들고, `with_structured_output()` 으로 구조화된 응답을 받는 방법을 학습합니다.

학습 목표

LEARNING OBJECTIVES

- → `@tool` 데코레이터로 도구를 만들고 스키마를 확인합니다
- → Pydantic 모델로 복잡한 입력 스키마를 정의합니다
- → `create_agent()` 에 도구를 연결하여 에이전트를 구성합니다
- → `ToolRuntime` 으로 도구에서 런타임 컨텍스트에 접근합니다
- → `with_structured_output()` 으로 구조화된 출력을 설정합니다
- → `ToolStrategy` 와 `ProviderStrategy` 의 차이를 이해합니다

4.1 환경 설정

API 키를 로드하고 OpenAI 모델을 초기화합니다.

```python
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv(override=True)

# OpenAI를 통한 모델 초기화
model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 초기화 완료:", model.model_name)
```

4.2 @tool 데코레이터 기본

함수에 `@tool` 을 붙이면 에이전트가 사용할 수 있는 도구가 됩니다. LangChain은 함수의 이름, docstring, 타입 힌트를 자동으로 파싱하여 도구 스키마를 생성합니다.

```
from langchain.tools import tool

@tool
def my_tool(param: str) -> str:
    """Tool description for the LLM."""
    return result
```

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """도시의 현재 날씨를 조회합니다."""
    weather_data = {
        "Seoul": "맑음, 15\u00b0C",
        "Tokyo": "흐림, 12\u00b0C",
        "New York": "비, 8\u00b0C",
    }
    return weather_data.get(city, f"날씨 데이터를 사용할 수 없습니다: {city}")

# 도구의 스키마 확인
print("도구 이름:", get_weather.name)
print("도구 설명:", get_weather.description)
print("입력 스키마:", get_weather.args_schema.model_json_schema())
```

```
도구 이름: get_weather
도구 설명: 도시의 현재 날씨를 조회합니다.
입력 스키마: {'description': '도시의 현재 날씨를 조회합니다.', 'properties': {'city': {'title': 'City', 'type':
'string'}}, 'required': ['city'], 'title': 'get_weather', 'type': 'object'}
```

4.3 Pydantic 복잡한 스키마

더 복잡한 입력 구조가 필요할 때는 Pydantic `BaseModel` 로 스키마를 정의합니다. `@tool(args_schema=MySchema)` 형태로 전달하면, LLM이 파라미터 구조를 정확히 파악할 수 있습니다.

- `Field(description=...)` : 각 필드에 대한 설명을 LLM에 전달
- `Field(default=...)` : 기본값 설정

```
from pydantic import BaseModel, Field

class SearchQuery(BaseModel):
    """데이터베이스 쿼리용 검색 파라미터입니다."""
    query: str = Field(description="검색 쿼리 문자열")
    max_results: int = Field(default=5, description="반환할 최대 결과 수")
    category: str = Field(default="all", description="검색 카테고리: all, tech, science, news")

@tool(args_schema=SearchQuery)
def search_database(query: str, max_results: int = 5, category: str = "all") -> str:
    """고급 필터링 옵션으로 데이터베이스를 검색합니다."""
    return f"'{category}' 카테고리에서 '{query}'에 대한 {max_results}개의 결과를 찾았습니다"

print("복합 스키마:", search_database.args_schema.model_json_schema())
```

```
복합 스키마: {'description': '데이터베이스 쿼리용 검색 파라미터입니다.', 'properties': {'query': {'description': '검색 쿼리 문자열', 'title': 'Query', 'type': 'string'}, 'max_results': {'default': 5, 'description': '반환할 최대 결과 수', 'title': 'Max Results', 'type': 'integer'}, 'category': {'default': 'all', 'description': '검색 카테고리: all, tech, science, news', 'title': 'Category', 'type': 'string'}}, 'required': ['query'], 'title': 'SearchQuery', 'type': 'object'}
```

4.4 도구를 에이전트에 연결

`create_agent()`에 도구 리스트를 전달하면, 에이전트가 상황에 맞는 도구를 자동으로 선택하여 실행합니다.

```python
from langchain.agents import create_agent

agent = create_agent(
    model=model,
    tools=[tool1, tool2],
    system_prompt="...",
)
```

> · NOTE
>
> 참고: LangChain v1에서는 `create_react_agent`가 제거되었습니다. 반드시 `create_agent`를 사용하세요.

4.5 ToolRuntime

`ToolRuntime`을 쓰면 도구 함수 내에서 현재 대화 상태(state)에 접근할 수 있습니다. 메시지 이력, 설정값 등 런타임 컨텍스트를 활용하는 도구를 만들 때 유용합니다.

```python
@tool
def my_tool(runtime: ToolRuntime) -> str:
    messages = runtime.state["messages"]
    # ...
```

4.5.1 `runtime.execution_info`

`ToolRuntime`은 단순 state 접근을 넘어, 현재 실행의 메타 정보를 들고 있습니다. 도구가 어떤 thread/run/node에서 호출됐는지 알 수 있어 로깅·디버깅·재시도 추적에 유용합니다.

> ➡️ TIP
>
> 일부 필드는 `deepagents ≥ 0.5.0` 또는 `langgraph ≥ 1.1.5`에서 노출됩니다.

4.5.2 `runtime.server_info`

LangGraph Platform·LangSmith 같은 호스팅 환경에서는 어시스턴트 ID, 사용자 식별자가 자동으로 채워집니다. 도구 안에서 이 정보를 꺼내 권한 분기·감사 로그에 씁니다.

4.5.3 `\@wrap_tool_call` — 도구 호출 가로채기

`@wrap_tool_call` 데코레이터는 모든 도구 호출 전·후에 끼어들어 재시도·관측·에러 처리를 한 곳에서 처리할 수 있게 해 줍니다. `ToolCallRequest` 로 도구 이름과 인자를 확인하고, 실패 시 fallback `ToolMessage` 를 반환합니다.

```
from langchain.messages import ToolMessage

try:
    from langchain.tools import wrap_tool_call, ToolCallRequest
except ImportError:
    # 일부 빌드에서는 langchain.agents 하위로 노출됩니다.
    from langchain.agents.tool_executor import wrap_tool_call, ToolCallRequest  # type: ignore


@wrap_tool_call
def safe_executor(request: ToolCallRequest, handler):
    """도구 호출 전후로 로그를 남기고, 예외는 ToolMessage로 흡수합니다."""
    print(f"[tool_call] {request.tool_call['name']} args={request.tool_call.get('args')}")
    try:
        result = handler(request)
        print("[tool_call] ok")
        return result
    except Exception as exc:  # noqa: BLE001
        print(f"[tool_call] failed: {exc}")
        return ToolMessage(
            content=f"도구 호출이 실패했습니다: {exc}. 다른 방식으로 답하세요.",
            tool_call_id=request.tool_call["id"],
        )


# 사용 예 — middleware/agent 옵션에 따라 전달 위치가 달라질 수 있습니다.
# wrapped_agent = create_agent(
#     model=model,
#     tools=[get_weather],
#     middleware=[safe_executor],
# )
print("safe_executor 정의 완료 — agent 생성 시 middleware로 등록하세요.")
```

4.5.4 `Command` 업데이트 + `ToolMessage` 동반 반환

도구가 단순 문자열이 아니라 상태 업데이트를 함께 반환해야 할 때는 `Command` 를 씁니다. 핵심은 `Command.update["messages"]` 에 `ToolMessage(tool_call_id=runtime.tool_call_id)` 를 같이 실어야 한다는 점입니다. 이 짝이 빠지면 모델이 도구 응답을 못 받았다고 판단해 같은 도구를 무한 반복할 수 있습니다.

> ➡️ **TIP**
>
> `runtime.tool_call_id` · `Command` API는 `deepagents ≥ 0.5.0` 또는 `langgraph ≥ 1.1.5` 에서 안정화됐습니다.

4.6 구조화된 출력

`with_structured_output()` 을 쓰면 모델의 응답을 Pydantic 모델이나 dataclass 형태로 직접 받을 수 있습니다. 에이전트 없이 모델에서 직접 사용하는 방식입니다.

```
structured_model = model.with_structured_output(MySchema)
result = structured_model.invoke("...")
# result는 MySchema 인스턴스
```

4.7 ToolStrategy vs ProviderStrategy

에이전트에서 구조화된 출력을 사용하는 두 가지 전략이 있습니다:

전략	설명	장점
`ToolStrategy`	도구 호출 메커니즘을 활용하여 구조화된 출력 생성	모든 모델에서 동작, 안정적
`ProviderStrategy`	프로바이더의 네이티브 구조화 출력 기능 사용	더 빠르고 정확 (지원 모델 한정)

`response_format` 파라미터에 전략을 지정하여 에이전트의 최종 응답을 구조화할 수 있습니다.

> · NOTE
>
> 버전 메모. `ProviderStrategy(..., strict=True)` 같은 엄격 검증 옵션은 `langchain ⩾ 1.2` 에서 안정화됐습니다. 그 이전 버전에서는 `ToolStrategy` 로 대체하거나 패키지를 업그레이드하세요.

> · NOTE
>
> Gemini는 비공식 지원. Google Gemini 계열은 구조화 출력 동작이 빠르게 바뀌고 있어, 노트북 예시는 OpenAI 기준으로 제공합니다. Gemini로 옮길 때는 응답 검증 코드를 별도로 두는 편이 안전합니다.

요약

이 노트북에서 학습한 핵심 내용:

항목	설명
`\@tool` 데코레이터	함수를 에이전트용 도구로 변환
`args_schema`	Pydantic 모델로 복잡한 입력 스키마 정의
`create_agent()`	모델과 도구를 연결하여 에이전트 생성
`ToolRuntime`	도구 내에서 런타임 상태(대화 이력 등) 접근
`runtime.execution_info`	`thread_id` · `run_id` · `node_attempt` 등 실행 메타
`runtime.server_info`	호스팅 환경의 `assistant_id` · `user` 정보
`\@wrap_tool_call`	모든 도구 호출에 공통 로깅·에러 핸들링 주입
`Command` + `ToolMessage`	상태 업데이트와 도구 응답을 한 번에 반환
`with_structured_output()`	모델 응답을 Pydantic/dataclass로 구조화

`ToolStrategy`	도구 호출 방식의 구조화된 에이전트 출력
`ProviderStrategy`	프로바이더 네이티브 구조화 출력 (`strict` 는 `langchain\ ⩾ 1.2`)

05 메모리와 스트리밍

LangChain v1 에이전트의 메모리 시스템과 스트리밍 모드를 학습합니다.

학습 목표

LEARNING OBJECTIVES

- → 단기 메모리(Short-term Memory): `InMemorySaver` 로 `thread_id` 기반 대화 상태를 유지하는 방법을 이해합니다.
- → 장기 메모리(Long-term Memory): `InMemoryStore` 로 대화 간에 지속되는 메모리를 구현합니다.
- → 메시지 트리밍: 긴 대화에서 토큰 예산 내로 메시지를 제한하는 방법을 배웁니다.
- → 스트리밍 모드: `values` , `updates` , `messages` , `custom` 등 다양한 스트리밍 모드의 차이를 이해합니다.

5.1 환경 설정

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

5.2 단기 메모리: InMemorySaver

단기 메모리는 하나의 대화 세션 내에서 이전 메시지를 기억하는 메커니즘입니다.

- `InMemorySaver` 는 체크포인터(checkpointer)로서 에이전트의 상태를 메모리에 저장합니다.
- `thread_id` 로 서로 다른 대화 세션을 구분합니다.
- 같은 `thread_id` 를 사용하면 이전 대화 컨텍스트가 유지됩니다.

5.3 다른 thread_id로 독립된 대화

서로 다른 `thread_id`를 사용하면 완전히 독립된 대화 세션이 생성됩니다. 이전 세션의 컨텍스트는 공유되지 않습니다.

5.4 메시지 트리밍

대화가 길어지면 토큰 수가 증가하여 비용과 성능에 영향을 줍니다. 메시지 트리밍으로 토큰 예산 내에서 가장 관련성 높은 메시지만 유지할 수 있습니다.

- `trim_messages` : 최근 N개의 메시지 또는 토큰 예산 내의 메시지만 유지합니다.
- `strategy="last"` : 가장 최근 메시지를 우선 유지합니다.
- `include_system=True` : 시스템 메시지는 항상 포함합니다.

5.5 장기 메모리: InMemoryStore

장기 메모리는 대화 세션 간에 지속되는 정보를 저장합니다.

- `InMemoryStore`는 키-값 저장소로서 사용자 선호도, 설정 등을 저장합니다.
- 도구의 `ToolRuntime` 파라미터로 스토어에 접근할 수 있습니다.
- `thread_id`와 무관하게 모든 세션에서 동일한 데이터에 접근 가능합니다.

단기 메모리와 장기 메모리의 차이:

구분	단기 메모리 (Checkpointer)	장기 메모리 (Store)
범위	하나의 `thread_id` 내	모든 세션에 걸쳐
저장 대상	대화 메시지 히스토리	사용자 선호도, 학습 데이터
수명	세션 종료 시 (또는 영구)	명시적 삭제 전까지 영구
접근 방식	자동 (에이전트 내부)	도구를 통해 명시적

namespace 설계 패턴

`store.put(namespace, key, value)`의 `namespace`는 튜플로 다층 분리가 가능합니다. 사용자별로 격리하려면 `runtime.context.user_id`를 namespace 의 일부로 포함시키는 것이 권장 패턴입니다.

```
namespace = ("preferences", runtime.context.user_id)  # 사용자별 격리
```

이렇게 하지 않으면 모든 사용자가 같은 메모리 영역을 공유하게 됩니다.

Production 환경 — PostgreSQL Store

프로덕션에서는 메모리 휘발성이 없는 `PostgresStore`를 권장합니다.

```
pip install langgraph-checkpoint-postgres
```

```python
from langgraph.store.postgres import PostgresStore
store = PostgresStore.from_conn_string("postgresql://...")
```

> ! WARNING
>
> ⚠️ 주의: `create_agent(..., store=store)` 로 명시적으로 전달하지 않으면 도구 내부의 `runtime.store` 는 `None` 이 됩니다. 도구 호출이 실패하므로 반드시 store 를 전달하세요.

5.6 스트리밍 모드

에이전트의 실행 과정을 실시간으로 관찰할 수 있는 스트리밍 기능을 제공합니다. 용도에 따라 다양한 스트리밍 모드를 선택할 수 있습니다.

스트리밍 모드 비교표

모드	설명	용도
`values`	각 단계의 전체 상태	디버깅, 상태 추적
`updates`	각 노드의 업데이트만	진행 상황 표시
`messages`	메시지 토큰 단위	채팅 UI
`custom`	사용자 정의 이벤트	커스텀 진행률

version="v2" + GraphOutput — 백엔드 스트리밍 표준 패턴

LangChain v1 의 새 스트리밍 API 는 `version="v2"` 플래그를 사용합니다. 이 모드에서는 `invoke()` 가 dict 가 아닌 `GraphOutput` 객체를 반환하며, `.value` (최종 상태)와 `.interrupts` (HITL 인터럽트 리스트)로 접근합니다.

`stream(..., version="v2")` 는 각 청크에 `type` 필드를 포함하므로 `updates` / `messages` / `custom` 을 한 스트림에서 식별 가능합니다.

tool_calls 누적과 chunk_position == "last"

토큰 단위 스트리밍에서 tool call 인자는 청크별로 부분 JSON 으로 도착합니다. 마지막 청크에서만 완전한 인자 형태가 보장되므로, `chunk_position == "last"` 가 표시되는 시점까지 누적해야 안전합니다.

content_blocks 로 reasoning vs text 분리

Claude / OpenAI o-series 등 reasoning 모델은 메시지 내용을 블록 단위로 반환합니다. `msg.content_blocks` 로 `text` / `reasoning` / `tool_use` 를 분리해 UI 에서 다르게 렌더링할 수 있습니다 (예: reasoning 은 접기, text 만 강조).

멀티 모드 스트리밍 · 서브그래프 · 토글

`stream_mode` 는 리스트로 전달하면 여러 모드를 동시에 받을 수 있고, 각 청크가 `(mode, data)` 튜플로 도착합니다. HITL 인터럽트는 `"__interrupt__"` 라는 특수 키로 `updates` 모드에 섞여 옵니다.

- `subgraphs=True` — 멀티 에이전트/서브그래프 사용 시 자식 그래프의 청크까지 받습니다.
- `model = ChatOpenAI(model="...", disable_streaming=True)` — 모델 자체의 토큰 스트리밍을 끄고 싶을 때.

stream_mode="custom" — get_stream_writer 로 도구에서 진행률 발행

`stream_mode="custom"` 은 도구 또는 미들웨어 안에서 임의의 진행률·중간 결과를 스트림으로 흘려보내는 모드입니다. `create_agent` 로 만든 에이전트에서도 동작하며, 도구 내부에서 `get_stream_writer()` 로 writer 를 얻어 호출하면 됩니다.

```
from langgraph.config import get_stream_writer
from langchain.tools import tool

@tool
def long_running_search(query: str) -> str:
    """진행률을 스트림으로 보고하는 검색 도구."""
    writer = get_stream_writer()
    writer({"event": "search_start", "query": query})
    # ... 실제 검색 로직 ...
    writer({"event": "search_done", "hits": 42})
    return f"'{query}' 결과 42건"

# 스트리밍 — stream_mode="custom" 으로 받기
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "LangChain 문서 검색"}]},
    stream_mode="custom",
):
    print(chunk)
```

여러 모드를 동시에 받으려면 `stream_mode=["updates", "custom"]` 처럼 리스트로 지정하면 됩니다.

요약

이 노트북에서 학습한 내용:

개념	구현	설명
단기 메모리	`InMemorySaver` + `thread_id`	하나의 대화 세션 내 컨텍스트 유지
세션 격리	다른 `thread_id` 사용	독립된 대화 세션 관리
메시지 트리밍	`trim_messages` + 미들웨어	토큰 예산 내 메시지 제한
장기 메모리	`InMemoryStore` + `ToolRuntime`	namespace 에 `runtime.context.user_id` 포함으로 사용자별 격리
Production Store	`PostgresStore` (`langgraph-checkpoint-postgres`)	영속화된 장기 메모리
스트리밍 (values/updates/messages)	`stream_mode=...`	단계 전체 / 노드별 / 토큰 단위
스트리밍 (custom)	`get_stream_writer()`	도구 안에서 진행률 직접 발행

버전 v2	`version="v2"` + `GraphOutput`	`result.value` / `result.interrupts` 로 접근
tool_calls 누적	`chunk_position == "last"`	AIMessageChunk + 연산자로 누적
content_blocks	`msg.content_blocks`	reasoning / text / tool_use 분리
멀티 모드	`stream_mode=["updates", "messages"]` + `"__interrupt__"`	HITL 인터럽트 캡처

핵심 포인트:

- 단기 메모리는 `thread_id` 로 격리되며, 같은 세션 내에서만 컨텍스트가 유지됩니다.
- 장기 메모리는 `("preferences", user_id)` 처럼 namespace 의 일부에 `user_id` 를 포함해 사용자별로 격리합니다.
- `store` 를 `create_agent` 에 전달하지 않으면 `runtime.store` 가 `None` 이라 도구가 실패합니다.
- `get_stream_writer()` 는 `create_agent` 안의 도구에서도 사용 가능합니다 — 별도 `StateGraph` 가 필요하지 않습니다.
- `version="v2"` 로 받은 결과는 `GraphOutput` — `result.value["messages"]` / `result.interrupts` 로 접근하세요.
- 토큰 스트리밍에서 tool_calls 의 args 는 `chunk_position == "last"` 시점까지 누적해야 완전합니다.

06 미들웨어와 가드레일

LangChain v1 에이전트의 미들웨어(Middleware) 시스템과 가드레일(Guardrails)을 학습합니다.

학습 목표

LEARNING OBJECTIVES

- → 미들웨어 개념: 에이전트 실행 파이프라인의 각 단계에 훅(hook)을 추가하는 방법을 이해합니다.
- → 빌트인 미들웨어: `SummarizationMiddleware` 등 기본 제공 미들웨어를 사용합니다.
- → 커스텀 미들웨어: `@before_model`, `@after_model`, `@wrap_model_call`, `@dynamic_prompt` 데코레이터로 커스텀 미들웨어를 구현합니다.
- → 가드레일: 안전하지 않은 입력/출력을 차단하는 방법을 배웁니다.

6.1 환경 설정

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

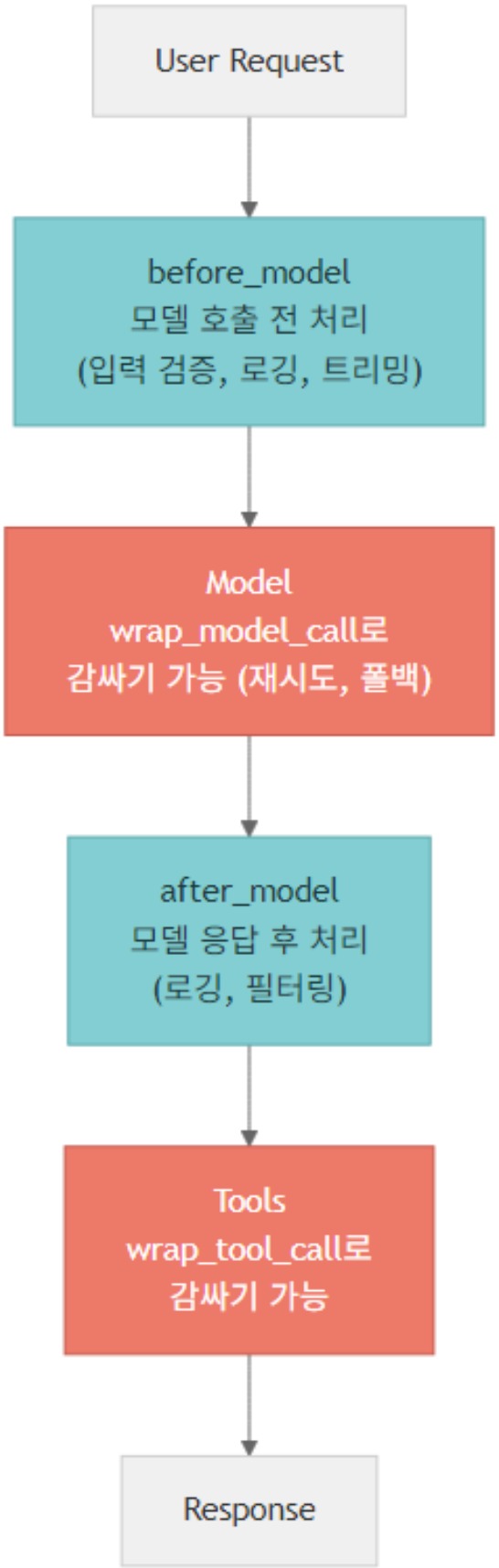
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

6.2 미들웨어 개념

미들웨어는 에이전트 실행 파이프라인의 각 단계에 훅(hook)을 추가하여 동작을 제어하는 메커니즘입니다.


User Request
before_model
모델 호출 전 처리
(입력 검증, 로깅, 트리밍)
Model
wrap_model_call로
감싸기 가능 (재시도, 폴백)
after_model
모델 응답 후 처리
(로깅, 필터링)
Tools
wrap_tool_call로
감싸기 가능
Response

5가지 미들웨어 훅:

훅	실행 시점	주요 용도
`@before_model`	모델 호출 전	입력 검증, 메시지 수정, 가드레일
`@after_model`	모델 응답 후	출력 로깅, 응답 필터링
`@wrap_model_call`	모델 호출 감싸기	재시도, 폴백, 캐싱
`@wrap_tool_call`	도구 호출 감싸기	도구 실행 제어
`@dynamic_prompt`	프롬프트 생성 시	런타임 프롬프트 변경

6.3 빌트인 미들웨어

LangChain v1은 자주 쓰는 패턴을 빌트인 미들웨어로 제공합니다. `SummarizationMiddleware`는 대화가 길어지면 이전 메시지를 자동으로 요약하여 토큰 사용량을 줄입니다.

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """정보를 검색합니다."""
    return f"'{query}'에 대한 검색 결과"

# SummarizationMiddleware — 긴 대화를 자동 요약
from langchain.agents.middleware import SummarizationMiddleware

summarization = SummarizationMiddleware(
    model=model,
    trigger=("messages", 10),
)

agent_with_summary = create_agent(
    model=model,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[summarization],
)
print("SummarizationMiddleware 에이전트 생성 완료")
```

```
SummarizationMiddleware 에이전트 생성 완료
```

빌트인 미들웨어 카탈로그

`SummarizationMiddleware` 외에도 v1 은 자주 쓰는 패턴을 빌트인으로 제공합니다.

미들웨어	용도	비고
`SummarizationMiddleware`	대화 요약	`trigger=("messages", N)` 또는 `("tokens", N)`

`AnthropicPromptCachingMiddleware`	Anthropic 프롬프트 캐싱	시스템 메시지·도구 정의 캐시 → 비용 절감
`BedrockPromptCachingMiddleware`	AWS Bedrock 프롬프트 캐싱	Anthropic 모델을 Bedrock 으로 호출할 때
`ModelFallbackMiddleware`	모델 폴백	첫 모델 실패 시 다음 모델로 자동 전환
`ContextEditingMiddleware`	컨텍스트 동적 편집	오래된 tool_use 메시지 제거 등
`PatchToolCallsMiddleware`	도구 호출 후처리	Deep Agents 의 핵심 컴포넌트 — tool call 결과를 상태에 반영

`ContextSize` 트리거 튜플은 어디서나 동일한 형식입니다:

- `("tokens", 100_000)` — 토큰 수 임계값
- `("messages", 20)` — 메시지 개수 임계값
- `("fraction", 0.8)` — 모델 `.profile["max_input_tokens"]` 대비 비율

```python
# Anthropic / Bedrock 프롬프트 캐싱 — 시스템 프롬프트가 크거나 도구가 많을 때 효과적
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware
from langchain_aws.middleware.prompt_caching import BedrockPromptCachingMiddleware

# Anthropic 직접 호출 (claude-sonnet-4-6 등)
anthropic_cache = AnthropicPromptCachingMiddleware()
# Bedrock 경유 호출
bedrock_cache = BedrockPromptCachingMiddleware()

# 예: Anthropic 모델 + 프롬프트 캐싱
# from langchain_anthropic import ChatAnthropic
# anthropic_model = ChatAnthropic(model="claude-sonnet-4-6")
# agent_cached = create_agent(
#     model=anthropic_model,
#     tools=[search],
#     system_prompt="...",
#     middleware=[anthropic_cache],
# )

print("Anthropic/Bedrock 프롬프트 캐싱 미들웨어 생성 완료")
```

```python
# ModelFallbackMiddleware — 첫 모델 호출이 실패하면 추가 모델로 순차 폴백
from langchain.agents.middleware import ModelFallbackMiddleware

primary = ChatOpenAI(model="gpt-5.4")
backup_1 = ChatOpenAI(model="gpt-5-nano")
# backup_2 = ChatAnthropic(model="claude-sonnet-4-6")  # 다른 프로바이더 폴백도 가능

fallback = ModelFallbackMiddleware(primary, backup_1)

agent_fallback = create_agent(
    model=primary,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[fallback],
)
print("ModelFallbackMiddleware 에이전트 생성 완료 — primary 실패 시 backup_1 로 폴백")
```

```python
# ContextEditingMiddleware — 토큰 임계값 초과 시 오래된 tool_use 메시지를 자동 제거
# trigger=("tokens", 100_000) 처럼 ContextSize 튜플로 시점을 결정합니다.
```

```
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

context_editor = ContextEditingMiddleware(
    edits=[
        ClearToolUsesEdit(
            trigger=("tokens", 100_000),   # 100K 토큰 넘으면 발동
            keep=("messages", 3),           # 가장 최근 3개 메시지의 tool_use 는 유지
        ),
    ],
)

agent_edited = create_agent(
    model=model,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[context_editor],
)
print("ContextEditingMiddleware 에이전트 생성 완료")
print("  → 100K 토큰 초과 시 최근 3개 외 오래된 tool_use 메시지를 자동 제거")
```

6.4 커스텀 미들웨어: @before_model

`@before_model` 데코레이터는 모델이 호출되기 전에 실행됩니다.

주요 용도:

- 입력 메시지 로깅
- 메시지 수정 또는 필터링
- 입력 검증 (가드레일)
- 컨텍스트 추가

6.5 커스텀 미들웨어: @after_model

`@after_model` 데코레이터는 모델 응답이 생성된 후에 실행됩니다.

주요 용도:

- 모델 출력 로깅
- 응답 필터링 또는 수정
- 도구 호출 감시
- 출력 품질 검증

6.6 @wrap_model_call

`@wrap_model_call` 데코레이터는 모델 호출 자체를 감싸서 재시도, 폴백, 캐싱 등의 로직을 구현할 수 있습니다.

`handler` 함수로 원래의 모델 호출을 실행하며, 이 호출 전후에 커스텀 로직을 추가합니다.

```
from langchain.agents.middleware import (
    wrap_model_call,
    ModelRequest,
    ModelResponse,
)
import time
```

```
@wrap_model_call
def retry_on_error(request: ModelRequest, handler) -> ModelResponse:
    """실패 시 지수 백오프로 모델 호출을 재시도합니다.

    - request: ModelRequest — messages / tools / system_message / response_format 보유
    - handler: 다음 단계를 실행하는 콜러블 (request 를 받아 ModelResponse 반환)
    """
    max_retries = 2
    for attempt in range(max_retries + 1):
        try:
            return handler(request)
        except Exception as e:
            if attempt < max_retries:
                wait = 2 ** attempt
                print(f"  재시도 {attempt + 1}/{max_retries} ({wait}초 대기)")
                time.sleep(wait)
            else:
                raise

agent_retry = create_agent(
    model=model,
    tools=[search],
    system_prompt="당신은 유용한 어시스턴트입니다.",
    middleware=[retry_on_error],
)
print("재시도 미들웨어 에이전트 생성 완료")
```

request.override — 요청을 변경해서 핸들러로 전달

`ModelRequest` 는 불변(immutable)에 가까운 객체로 다뤄지므로, 일부 필드만 바꾸려면 `request.override(...)` 로 새 객체를 만들어 핸들러에 넘깁니다. 변경 가능한 필드:

- `messages` — 메시지 리스트
- `tools` — 사용 가능한 도구 리스트
- `system_message` — 시스템 프롬프트
- `response_format` — 구조화 출력 스키마

```
# request.override 로 시스템 메시지·도구 필터를 동적으로 조정
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse


@wrap_model_call
def restrict_tools_for_anonymous(request: ModelRequest, handler) -> ModelResponse:
    """비로그인 사용자에게는 검색 도구만 허용하고 시스템 프롬프트도 교체합니다."""
    user_id = request.runtime.context.get("user_id") if hasattr(request, "runtime") else None

    if not user_id:
        # 비로그인 — 도구 화이트리스트 적용 + 시스템 메시지 교체
        safe_tools = [t for t in request.tools if t.name == "search"]
        new_request = request.override(
            tools=safe_tools,
            system_message="당신은 비로그인 사용자를 돕는 제한된 어시스턴트입니다.",
        )
        return handler(new_request)

    return handler(request)


print("request.override 패턴 미들웨어 등록 준비 완료")
```

6.7 @dynamic_prompt

`@dynamic_prompt` 데코레이터는 런타임에 시스템 프롬프트를 동적으로 변경합니다.

주요 용도:

- 현재 날짜/시간 정보 추가
- 사용자별 맞춤 프롬프트
- 상태에 따른 행동 변경
- A/B 테스트

6.8 @wrap_tool_call

`@wrap_tool_call` 데코레이터는 도구 호출 자체를 감싸서 실행 전후에 커스텀 로직을 추가할 수 있습니다.

`@wrap_model_call`이 모델 호출을 감싸듯, `@wrap_tool_call`은 도구 실행을 감쌉니다. `handler` 함수를 호출하면 원래 도구가 실행되며, 그 전후로 타이밍 측정, 로깅, 에러 핸들링 등을 구현할 수 있습니다.

주요 용도:

- 실행 시간 측정: 도구별 성능 모니터링
- 로깅: 도구 입력/출력 기록
- 에러 핸들링: 도구 실패 시 폴백 처리
- 접근 제어: 특정 도구 호출 차단 또는 제한

도구 호출 상태 업데이트 — `state_schema`

도구 실행 횟수처럼 기본 agent state에 없는 값을 저장하려면 `@wrap_tool_call(state_schema=...)`로 추가 state 필드를 선언합니다. `handler(request)` 결과를 `messages`에 다시 넣고, 같은 `Command(update=...)`에서 카운터를 함께 갱신해야 다음 노드가 누적 값을 볼 수 있습니다.

```python
from typing_extensions import NotRequired, TypedDict
from langgraph.types import Command
ToolAuditState = TypedDict("ToolAuditState", {"messages": list, "tool_call_count": NotRequired[int]})

@wrap_tool_call(state_schema=ToolAuditState)
def count_tool_calls(request, handler):
    result = handler(request)
    count = request.state.get("tool_call_count", 0) + 1
    return Command(update={"messages": [result], "tool_call_count": count})
```

6.9 가드레일

가드레일은 안전하지 않은 입력이나 출력을 차단하는 메커니즘입니다. 미들웨어로 구현하며, 금지된 키워드 감지, 프롬프트 인젝션 방어, 민감 정보 필터링 등에 씁니다.

`@before_model` 훅에서 구현하는 것이 가장 효과적입니다. 모델에 전달되기 전에 위험한 입력을 차단할 수 있기 때문입니다.

@hook_config(can_jump_to=…) — 조건부 그래프 점프

가드레일이 위험 감지 후 단순히 예외를 던지는 대신, 그래프의 특정 노드로 점프하도록 만들 수 있습니다. `@hook_config` 로 점프 가능한 대상을 선언하고, 훅에서 `{"jump_to": "end"}` 같은 dict 를 반환하면 됩니다.

점프 대상:

- `"end"` — 즉시 종료
- `"tools"` — 도구 노드로 점프
- `"model"` — 모델 노드로 재진입

비동기 미들웨어 컨벤션

`ainvoke` / `astream` 환경에서는 비동기 변형을 사용합니다. 데코레이터 또는 클래스 메서드 모두 `a-` 프리픽스로 통일됩니다.

동기	비동기
`\@before_model`	`\@abefore_model`
`\@after_model`	`\@aafter_model`
`\@wrap_model_call`	`\@awrap_model_call`
`\@wrap_tool_call`	`\@awrap_tool_call`
`\@dynamic_prompt`	`\@adynamic_prompt`

비동기 핸들러 안에서 `await handler(request)` 처럼 await 해야 합니다.

요약

이 노트북에서 학습한 미들웨어 타입:

미들웨어 타입	데코레이터 / 클래스	실행 시점	주요 용도
Before Model	`\@before_model`	모델 호출 전	입력 로깅, 검증, 가드레일
After Model	`\@after_model`	모델 응답 후	출력 로깅, 필터링
Wrap Model	`\@wrap_model_call`	모델 호출 감싸기	재시도, 폴백, 캐싱
Wrap Tool	`\@wrap_tool_call`	도구 호출 감싸기	타이밍, 로깅, 에러 핸들링
Dynamic Prompt	`\@dynamic_prompt`	프롬프트 생성 시	런타임 프롬프트 변경
Builtin (요약)	`SummarizationMiddleware`	trigger 충족 시	대화 요약
Builtin (캐싱)	`AnthropicPromptCachingMiddleware`, `BedrockPromptCachingMiddleware`	모델 호출 전	프롬프트 캐싱 비용 절감
Builtin (폴백)	`ModelFallbackMiddleware(primary, *backups)`	모델 호출 실패 시	다른 모델로 자동 폴백
Builtin (편집)	`ContextEditingMiddleware(edits=[…])`	트리거 충족 시	오래된 tool_use 제거

Builtin (Deep Agents)	`PatchToolCallsMiddleware`	도구 호출 후	tool 결과를 상태에 반영
Guardrail	`\@hook_config(can_jump_to=[...])` + `{"jump_to": "end"}`	모델 호출 전	위험 입력 시 그래프 점프

핵심 포인트:

- `ModelRequest` 는 `request.override(messages=..., tools=..., system_message=..., response_format=...)` 로 부분 변경합니다.
- `ContextSize` 트리거 튜플은 `("tokens", N)` / `("messages", N)` / `("fraction", 0~1)` 형식이며 `model.profile["max_input_tokens"]` 와 결합해 동적 임계값을 만들 수 있습니다.
- 가드레일은 예외 대신 `@hook_config(can_jump_to=[...])` + `{"jump_to": "end"}` 패턴이 더 안전합니다 (사용자에게 거절 메시지를 남길 수 있음).
- 비동기 환경에서는 `@abefore_model`, `@aafter_model`, `@awrap_model_call` 처럼 `a-` 프리픽스 변형을 씁니다.
- 커스텀 미들웨어 시그니처: `def hook(request: ModelRequest, handler) → ModelResponse`.

07 사람 개입(HITL)과 런타임

학습 목표

도구 실행 전 사람의 승인을 받고, 런타임 컨텍스트를 주입합니다.

이 노트북에서 다루는 내용:

- Human-in-the-Loop (HITL): 에이전트가 위험한 도구를 실행하기 전에 사람의 승인을 받는 패턴
- ToolRuntime: 도구 실행 시 런타임 컨텍스트(사용자 정보 등)를 주입하는 방법
- 컨텍스트 엔지니어링: 동적으로 프롬프트와 도구를 제어하는 기법
- MCP (Model Context Protocol): 도구 서버를 표준 프로토콜로 연결하는 방식

7.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

# requires deepagents>=0.5.0 or langgraph>=1.1.5
model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

7.2 Human-in-the-Loop 개념

에이전트가 도구를 호출하기 전에 사람의 승인을 요청합니다.

왜 필요한가?

자율적으로 동작하는 에이전트는 강력하지만, 이메일 전송, 파일 삭제, 결제 처리 같은 되돌릴 수 없는 작업에서는 사람의 확인이 필요합니다.

워크플로

```
에이전트 → 도구 호출 제안 → [중단(interrupt)] → 사람 승인/거부 → 도구 실행 → 결과 반환
```

LangChain v1에서는 `HumanInTheLoopMiddleware` 와 `InMemorySaver` (체크포인터)를 결합하여 이 패턴을 구현합니다. 체크포인터는 에이전트의 상태를 저장하여 중단 후 재개할 수 있게 합니다.

7.3 HumanInTheLoopMiddleware

`HumanInTheLoopMiddleware` 는 도구 호출 시 실행을 자동으로 중단하고 사람의 승인을 기다리는 미들웨어입니다. `InMemorySaver` 체크포인터와 함께 사용하여 중단된 상태를 보존합니다.

라이프사이클

내부적으로 `after_model` 훅에서 `HITLRequest` 를 생성합니다.

```
모델이 tool_call 생성 → after_model 훅 발동 → HITLRequest
{
  "action_requests": [...],     # 어떤 도구를 어떤 인자로 호출하려는지
  "review_configs":  [...]      # 도구별 허용 decision 목록
}
→ interrupt 발생 → result.interrupts 에 노출 → 사람이 Command(resume=...) 로 재개
```

transient vs persistent 변경

- transient (요청 단위): `request.override(messages=..., tools=...)` — 한 번의 모델 호출에만 적용. 미들웨어가 끝나면 원복됩니다.
- persistent (상태 저장): `ExtendedModelResponse` + `Command(update={...})` — 그래프 상태를 영속적으로 수정. 다음 호출에도 반영됩니다.

권한 가드 같은 일회성 변경은 transient 로, 사용자 프로필이 바뀐 경우는 persistent 로 처리하는 것이 일반적입니다.

```python
from langchain.agents import create_agent
from langchain.tools import tool
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """지정된 수신자에게 이메일을 보냅니다."""
    return f"{to}에게 이메일 전송 완료: {subject}"

@tool
def delete_file(path: str) -> str:
    """지정된 경로의 파일을 삭제합니다."""
    return f"파일 삭제 완료: {path}"

# 도구별로 허용 가능한 decision 목록을 dict 형식으로 지정합니다.
# - approve : 그대로 실행
# - edit    : 인자를 수정해서 실행
# - reject  : 도구 실행 거부 (사유 전달)
# - respond : 도구를 실행하지 않고 사람이 직접 응답
hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email":  {"allowed_decisions": ["approve", "edit", "reject", "respond"]},
        "delete_file": {"allowed_decisions": ["approve", "reject"]},  # 편집 없이 승인/거부만
    },
)

agent = create_agent(
    model=model,
    tools=[send_email, delete_file],
```

```
    system_prompt="당신은 이메일을 보내고 파일을 관리할 수 있는 어시스턴트입니다.",
    middleware=[hitl],
    checkpointer=InMemorySaver(),
)

print("HITL 에이전트 생성 완료")
print("  -> 도구 호출 시 사람의 승인을 위해 중단됩니다 (approve/edit/reject/respond)")
```

7.4 interrupt와 Command(resume=…) 패턴

HITL 에이전트는 2단계로 동작합니다:

1. 1단계 (invoke): 에이전트가 도구 호출을 제안하면 자동으로 중단(interrupt)됩니다.
2. 2단계 (Command(resume=…)): 사람이 결정을 내려 실행을 재개합니다.

4가지 decision 형식

`Command(resume={"decisions": [...]})` 의 각 decision 은 dict 입니다:

```
# 1) 승인 – 도구를 그대로 실행
Command(resume={"decisions": [{"type": "approve"}]})

# 2) 편집 – 도구 인자를 수정해서 실행
Command(resume={"decisions": [{"type": "edit", "args": {"to": "alice@example.com"}}]})

# 3) 거부 – 도구를 실행하지 않고 사유를 모델에 전달
Command(resume={"decisions": [{"type": "reject", "message": "수신자가 잘못되었습니다."}]})

# 4) 응답 – 도구를 실행하지 않고 사람이 직접 답변을 작성 (모델에는 ToolMessage 로 전달)
Command(resume={"decisions": [{"type": "respond", "message": "이미 어제 보냈으니 건너뛰세요."}]})
```

decision 리스트의 순서는 `result.interrupts[0].value["action_requests"]` 의 순서와 일치해야 합니다.

7.5 ToolRuntime – 도구에서 런타임 정보에 접근합니다

`ToolRuntime` 은 도구가 실행될 때 런타임 컨텍스트(현재 사용자 정보, 세션 데이터 등)에 접근할 수 있게 해주는 메커니즘입니다.

핵심 아이디어

- 도구 함수에 `runtime: ToolRuntime[T]` 파라미터를 추가합니다.
- `T` 는 개발자가 정의하는 컨텍스트 데이터 클래스입니다.
- 에이전트 생성 시 `context_schema=T` 를 지정하고, 호출 시 `context=T(...)` 로 값을 전달합니다.

runtime.execution_info – 실행 메타데이터에 접근

`runtime.execution_info` 는 현재 실행에 대한 메타데이터를 담고 있습니다 (LangGraph 1.1.5+ / Deep Agents 0.5.0+).

필드	설명
`thread_id`	현재 대화 스레드 ID – checkpointer 키와 동일

`run_id`	이번 invoke/stream 한 번의 고유 ID — 로깅/추적용
`attempt`	재시도 횟수 (1부터 시작) — 폴백·재시도 미들웨어에서 유용

도구나 미들웨어 안에서 분기·로깅에 쓰입니다.

runtime.server_info — 호스트 서버에서 주입되는 메타데이터

LangGraph Platform / Server 환경에서 실행될 때 `runtime.server_info` 가 채워집니다.

필드	설명
`assistant_id`	배포된 assistant 의 ID
`graph_id`	그래프 ID
`user`	인증된 사용자 정보(있다면)

로컬에서 실행하면 `None` 또는 빈 객체가 됩니다. 멀티 테넌트 배포에서 권한 분리·로깅에 활용합니다.

7.6 컨텍스트 엔지니어링 – 동적으로 프롬프트와 도구를 제어합니다

컨텍스트 엔지니어링은 에이전트에게 전달되는 프롬프트, 도구, 메시지 히스토리를 동적으로 조작하는 기법입니다.

주요 활용 사례

- 사용자 역할에 따라 다른 시스템 프롬프트 제공
- 상황에 따라 사용 가능한 도구 필터링
- 긴 대화 히스토리 요약 및 정리

`dynamic_prompt` 미들웨어를 쓰면 매 요청마다 프롬프트를 커스터마이즈할 수 있습니다.

7.7 MCP(Model Context Protocol) 연동 개요

MCP는 도구 서버를 표준 프로토콜로 연결하는 방식입니다. LangChain 1.4의 새 표면은 베타 `langchain.mcp.MCPAdapter` 입니다.

LangChain v1에서의 MCP 지원

- `MCPAdapter` 는 URL, 로컬 `Path` , FastMCP `Client` , `MCPConfig` 를 target으로 받습니다.
- `async with MCPAdapter(...)` 블록 안에서 `await adapter.list_tools()` 로 도구를 가져오고 에이전트 호출까지 수행합니다.
- 서버의 elicitation 요청은 콜백이 아니라 LangGraph interrupt로 멈추고 `Command(resume={"responses": {...}})` 로 재개합니다.

요약

이 노트북에서 학습한 핵심 내용:

개념	설명	핵심 API
HITL	도구 실행 전 사람의 결정 요청	`HumanInTheLoopMiddleware(interrupt_on={"tool": {"allowed_decisions": [...]}})`
Decisions	4가지 dict 형식	`approve` / `edit` / `reject` / `respond`
재개	dict 기반 resume	`Command(resume={"decisions": [{"type": ..., ...}]})`
GraphOutput	v2 결과 객체	`result.value` / `result.interrupts`
ToolRuntime	도구에서 런타임 컨텍스트 접근	`ToolRuntime[T]` , `context_schema=T`
execution_info	실행 메타데이터	`runtime.execution_info.thread_id / run_id / attempt`
server_info	서버 메타데이터	`runtime.server_info.assistant_id / graph_id / user`
컨텍스트 엔지니어링	동적 프롬프트/도구 제어	`dynamic_prompt` 미들웨어
MCP	표준화된 도구 프로토콜	`MCPAdapter + list_tools()`

핵심 포인트:

- `version="v2"` 호출은 `GraphOutput` 을 반환 — `result.value["messages"]` / `result.interrupts` 로 접근하세요.
- `interrupt_on` 은 dict 형식 (`{"tool_name": {"allowed_decisions": [...]}}`) — 도구별로 허용 결정을 다르게 설정 가능합니다.
- decision 의 4가지 타입(`approve` / `edit` / `reject` / `respond`)은 모두 `Command(resume={"decisions": [{...}]})` 형식의 dict 입니다.
- `runtime.execution_info` 와 `runtime.server_info` 는 LangGraph 1.1.5+ / Deep Agents 0.5.0+ 에서 사용 가능합니다.
- 미들웨어 변경의 범위: transient(`request.override`) vs persistent(`ExtendedModelResponse` + `Command(update=...)`).

08 멀티 에이전트 패턴

학습 목표

5가지 멀티 에이전트 패턴을 이해하고 구현합니다.

이 노트북에서 다루는 내용:

- Subagents: 메인 에이전트가 전문 서브에이전트를 도구로 호출
- Handoffs: `Command(goto=...)`로 에이전트 간 상태 전환
- Skills: 단일 에이전트가 상황에 따라 전문 프롬프트를 로드
- Router: 분류기가 입력을 적절한 에이전트로 라우팅
- Custom: 개발자가 완전히 제어하는 복잡한 워크플로

8.1 환경 설정

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("모델 준비 완료:", model.model_name)
```

```
모델 준비 완료: gpt-4.1
```

8.2 멀티 에이전트 패턴 비교

아래 표는 5가지 멀티 에이전트 패턴을 비교합니다. 각 패턴은 서로 다른 상황에 적합하므로, 프로젝트 요구사항에 맞게 선택해야 합니다.

패턴	라우팅 주체	상태 공유	적합한 상황
Subagents	메인 에이전트	도구로 격리	병렬 처리, 분산 개발
Handoffs	도구 호출	상태 전환	순차적 멀티홉
Skills	단일 에이전트	프롬프트 교체	도메인 특화

Router	분류기	병렬 실행	멀티 도메인
Custom	개발자 정의	완전 제어	복잡한 워크플로

핵심 차이점

- Subagents는 각 서브에이전트가 독립적으로 실행되어 결과만 반환합니다.
- Handoffs는 대화 상태가 에이전트 간에 전달됩니다.
- Skills는 하나의 에이전트가 여러 역할을 전환합니다.
- Router는 입력을 분류한 뒤 적절한 에이전트에게 위임합니다.

패턴 선택 매트릭스 (요구사항 → 최적 패턴)

#	요구사항	최적 패턴	이유
1	One-shot 단순 작업 — 한 번의 도구 호출로 끝나는 단일 도메인	Single agent (멀티 에이전트 불필요)	오버헤드만 늘어남. `create_agent` 한 개로 충분
2	반복(repeat) 호출 — 같은 도구를 여러 번 호출해 데이터를 축적	Subagents	메인이 결과를 모으며, 각 subagent 호출이 독립 컨텍스트로 격리됨
3	병렬 멀티 도메인 — 분류 후 도메인별 전문가에게 동시 분배	Router (+ `Send`)	분류 노드 → fan-out. 동시 실행으로 지연 최소화
4	대용량 컨텍스트 — 문서 1만 토큰 이상을 부분별로 분석	Subagents (격리된 컨텍스트) 또는 Skills	메인 에이전트 토큰을 보존. Skills는 프롬프트 교체로 추가 도구 없이 처리
5	팀 기반(team-based) — 여러 역할이 서로 결과를 검토·반복	Handoffs	`Command(goto=...)` 로 대화 상태가 다음 역할로 전달됨
6	다이렉트 대화(direct conversation) — 사용자와 끊김 없이 대화하면서 부서 이관	Handoffs (subgraph + conversation history)	메시지 히스토리가 보존되어 사용자가 같은 톤·맥락으로 계속 진행 가능

성능 비교 (3-step task 기준)

같은 3-step 작업을 패턴별로 수행했을 때 도구 호출 횟수와 컨텍스트 비용(approx.):

시나리오	패턴	도구 호출 수	컨텍스트(approx.)	비고
One-Shot (단일 질의)	Single agent	1 call	3K	베이스라인
Repeat (반복 검색·집계)	Subagents	4–5 calls	9K	권장

Multi-Domain (병렬 분배)	Router (+ Send)	3 calls	9K	분류 1 + 도메인 fan-out 병렬

> ➡️ TIP
>
> 수치는 `docs/langchain/22-multi-agent.md` 기준 추정치. 실제는 모델·프롬프트·tool schema에 따라 ±30% 변동.

Router vs Supervisor 용어

> ➡️ TIP
>
> Router — 단순 함수형 라우팅. 입력을 분류한 뒤 결정론적으로 다음 노드를 선택합니다. 대화 상태를 인식하지 않고, 한 번 위임하면 끝납니다. (예: `classify_query()` → `math` / `code` / `general` 분기)

>

> ➡️ TIP
>
> Supervisor — 대화 인식(conversation-aware) 감독자. 메시지 히스토리를 보면서 다음 단계에 어떤 subagent를 호출할지 LLM이 판단합니다. 반복 호출, 결과 통합, 재시도 의사결정이 가능합니다. (LangChain v1에서는 `create_agent` + subagent 도구 묶음 = 사실상 supervisor 패턴)

>

> · NOTE
>
> 본 노트북의 8.3은 Supervisor(LLM이 위임 판단), 8.6은 Router(함수형 분류)입니다.

8.3 서브에이전트 패턴

메인 에이전트(감독자)가 전문 서브에이전트를 도구로 호출하는 패턴입니다.

특징

- 각 서브에이전트는 도구 함수로 캡슐화됩니다.
- 메인 에이전트가 어떤 서브에이전트를 호출할지 판단합니다.
- 서브에이전트의 내부 상태는 메인 에이전트와 격리됩니다.
- 병렬 실행이 가능하여 성능에 유리합니다.

8.3.1 Subagent-local history — `checkpointer=True`

서브에이전트를 `create_agent` 로 만들 때 `checkpointer=True` 를 전달하면, 그 서브에이전트만의 로컬 대화 히스토리가 유지됩니다. 메인 에이전트는 서브에이전트의 내부 multi-turn 추론을 보지 않고 최종 결과만 받습니다.

- 메인 컨텍스트 보존 (서브의 reasoning step 미노출)
- 서브 내부에서는 도구 호출-결과-재추론 multi-turn 유지
- 부모와 자식이 별도 checkpointer를 가질 수 있음 (격리)

8.3.2 단일 dispatch tool — 서브에이전트 노출 방식 3가지

서브에이전트가 많아지면 도구 슬롯 폭증을 막기 위해 단일 `dispatch` 도구 하나로 묶고, 어떤 서브를 호출할지는 인자로 받습니다. 노출 방식은 세 가지가 있습니다.

방식	적합 규모	장점	단점
(A) System prompt 열거	\< 10개	간단·즉시 발견	프롬프트 토큰 증가, 대규모 시 흐려짐
(B) `Literal` 인자 제약	10–30개	tool schema가 enum을 강제 → 모델 hallucination 감소	명단을 코드에서 관리해야 함
(C) Tool-based discovery	30+ 개	`list_subagents()` / `describe(name)` 도구로 동적 탐색. 무한 확장	호출 1-2회 추가

8.3.3 부모 state 주입 — `ToolRuntime[None, CustomState]`

서브에이전트 도구가 부모 그래프의 커스텀 state(예: `user_id` , `tenant` , 누적 결과)에 접근해야 할 때 `ToolRuntime[ContextSchema, StateSchema]` 로 의존성을 주입합니다. 도구 인자 시그니처에 명시하면 LangChain이 자동으로 그래프 state를 전달합니다.

8.3.4 비동기 서브에이전트 — 3-tool 패턴

장시간(분 단위) 실행되는 서브에이전트는 동기 호출로 메인을 블록하면 안 됩니다. `start_job` / `check_status` / `get_result` 세 도구로 분리하면 메인 에이전트가 백그라운드 작업을 폴링하면서 다른 일을 병행할 수 있습니다.

도구	역할	반환
`start_job(task)`	작업 제출 (즉시 반환)	`job_id`
`check_status(job_id)`	진행 상태 폴링	`pending` / `running` / `done` / `failed`
`get_result(job_id)`	완료된 결과 회수	최종 출력 (미완료 시 에러)

8.3.5 `Command` 반환으로 부모 state 업데이트

도구가 단순 문자열 대신 `Command(update={...})` 를 반환하면, 부모 그래프의 state를 직접 갱신할 수 있습니다. 누적 로그·집계 결과·중간 산출물을 메시지 히스토리 밖에 두고 싶을 때 유용합니다.

8.4 핸드오프 패턴

`Command(goto=...)` 로 에이전트 간 상태를 전환하는 패턴입니다.

특징

- 도구가 `Command` 객체를 반환하여 다른 에이전트로 전환합니다.
- 대화 상태(메시지 히스토리)가 다음 에이전트로 전달됩니다.
- `StateGraph` 로 에이전트 간 흐름을 정의합니다.
- 고객 서비스의 부서 이관 같은 순차적 멀티홉 시나리오에 적합합니다.

8.4.1 단일 에이전트 핸드오프 — `\@wrap_model_call` 미들웨어

위의 multi-subgraph 방식은 부서가 많아질수록 노드 수가 폭증합니다. docs 권장 1순위는 단일 에이전트에 `@wrap_model_call` 미들웨어를 붙여, 모드(`role`)에 따라 `system_prompt` 와 `tools` 를 동적으로 교체하는 방식입니다.

- 노드 1개 — `StateGraph` 불필요
- 상태 자연 전이 — `role` 필드만 업데이트하면 다음 호출부터 새 프롬프트·도구 셋 적용
- 핸드오프 = 도구 호출 — `transfer_to_sales` 가 state의 `role` 을 바꾸고 다음 모델 호출 시 sales 페르소나로 작동

8.4.2 Subgraph 핸드오프의 conversation history 규칙

서브그래프 방식(8.4 원본 예제)으로 핸드오프를 만들 때는 부모 그래프의 메시지 히스토리에 정확히 2개만 흘러야 다음 에이전트가 정상 작동합니다.

#	메시지	누가 만드는가	역할
1	`AIMessage(tool_calls=[transfer_to_X])`	라우터 에이전트의 LLM	"이관 결정"의 흔적
2	`ToolMessage(tool_call_id=...)`	`transfer_to_X` 도구 자체	OpenAI tool-call 규칙: 모든 tool_call에는 짝 ToolMessage 필요

부모 history에 흘리지 말아야 할 것:

- 라우터의 중간 reasoning 메시지 (서브그래프 내부에서 처리)
- 도구의 길고 자세한 결과 (요약만 ToolMessage로)

왜 중요한가:

- 다음 에이전트(sales/support)는 이 2개 메시지를 보고 방금 이관됐다를 인지함
- OpenAI/Anthropic은 짝 없는 `tool_calls` 가 있으면 다음 turn에서 에러를 냅니다
- 부모 history가 깨끗할수록 컨텍스트가 가볍고, 같은 thread에서 추가 이관이 가능

→ 위 8.4.1의 `wrap_model_call` 패턴은 이 규칙을 자동으로 따릅니다 (도구가 `ToolMessage` 하나만 흘리고 state.role을 갱신).

8.5 스킬 패턴

단일 에이전트가 상황에 따라 전문 프롬프트를 로드하는 패턴입니다.

특징

- 하나의 에이전트가 여러 “스킬”을 가집니다.
- 각 스킬은 특화된 시스템 프롬프트입니다.
- 에이전트가 필요한 스킬을 동적으로 로드합니다.
- 여러 에이전트를 관리하지 않고도 하나의 에이전트로 다양한 작업을 처리할 수 있습니다.

8.6 라우터 패턴

분류기가 입력을 적절한 에이전트로 라우팅하는 패턴입니다.

특징

- 먼저 쿼리를 분류(classify)합니다.
- 분류 결과에 따라 적절한 전문 에이전트(도구)로 위임합니다.
- 멀티 도메인 시스템에서 유용합니다.
- 분류 로직은 규칙 기반 또는 LLM 기반으로 구현할 수 있습니다.

8.6.1 진짜 fan-out 라우터 — `Send` 로 병렬 분배

위 8.6 예제의 `from langgraph.types import Send` 는 사실 dead import였습니다(분류 후 단일 분기만 실행). 실제 라우터의 강점은 분류 결과를 여러 노드에 동시에 fan-out하는 능력입니다. 한 질문이 여러 도메인에 걸칠 때 `Send(node, payload)` 리스트를 반환하면 LangGraph가 병렬로 실행합니다.

8.7 패턴 선택 가이드

어떤 멀티 에이전트 패턴을 선택해야 할까요? 아래 가이드를 참고하세요.

결정 트리

1. 에이전트가 독립적으로 작업 가능한가?

- YES -> Subagents (병렬 실행, 결과 취합)
- NO -> 다음 질문으로

1. 대화 상태가 에이전트 간에 전달되어야 하는가?

- YES -> Handoffs (상태 전환, 멀티홉)
- NO -> 다음 질문으로

1. 하나의 에이전트가 여러 역할을 전환하면 되는가?

- YES -> Skills (프롬프트 교체)
- NO -> 다음 질문으로

1. 입력을 분류한 뒤 적절한 처리기로 보내면 되는가?

- YES -> Router (분류 후 위임)
- NO -> Custom (완전 커스텀 그래프)

실용적 권장사항

- 처음에는 가장 단순한 패턴(Subagents 또는 Skills)으로 시작하세요.

- 요구사항이 복잡해지면 Handoffs나 Router로 전환하세요.
- Custom 패턴은 다른 패턴으로 해결할 수 없을 때만 사용하세요.
- 패턴을 조합할 수도 있습니다 (예: Router + Handoffs).

요약

이 노트북에서 학습한 핵심 내용:

패턴	핵심 API	사용 시기
Subagents	`create_agent` + 도구 함수	독립적 병렬 작업
Handoffs	`Command(goto=...)`, `StateGraph`	상태 전환이 필요한 멀티홉
Skills	도구로 프롬프트 로드	단일 에이전트 다중 역할
Router	분류 도구 + 전문 도구	멀티 도메인 분류
Custom	`StateGraph` 완전 제어	복잡한 비즈니스 로직

핵심 원칙

- 단순한 것부터 시작하고, 필요에 따라 복잡도를 높이세요.
- 각 에이전트는 하나의 책임만 가지도록 설계하세요.
- 에이전트 간 인터페이스(입력/출력)를 명확히 정의하세요.

09 커스텀 워크플로와 RAG

학습 목표

LangGraph `StateGraph`로 커스텀 워크플로를 만들고, RAG 패턴을 구현합니다.

이 노트북에서 다루는 내용:

- `StateGraph`의 기본 구조 (노드, 엣지, 상태)
- 조건부 엣지를 활용한 분기 처리
- `create_agent`를 워크플로 노드로 통합
- RAG (Retrieval-Augmented Generation) 패턴 구현

9.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

```
환경 준비 완료.
```

9.2 StateGraph 기초

LangGraph의 핵심 빌딩 블록을 살펴봅니다.

개념	설명
노드(Node)	처리 단위. 함수 또는 에이전트가 될 수 있습니다
엣지(Edge)	노드 간 연결. 실행 흐름을 정의합니다
상태(State)	노드 간 공유 데이터. `TypedDict`로 정의합니다

`StateGraph` 는 이 세 가지를 조합하여 복잡한 워크플로를 구성할 수 있게 합니다.

9.3 조건부 엣지

상태에 따라 다른 경로로 분기합니다. `add_conditional_edges` 를 쓰면 런타임에 동적으로 다음 노드를 선택할 수 있습니다.

아래 예제에서는 입력 텍스트를 분류한 후, 카테고리에 따라 서로 다른 핸들러로 라우팅합니다.

9.4 에이전트를 워크플로에 통합

`create_agent` 로 만든 에이전트를 `StateGraph` 의 노드로 사용합니다. 이렇게 하면 여러 에이전트를 파이프라인으로 연결하여 복잡한 작업을 처리할 수 있습니다.

아래 예제에서는 리서치 에이전트와 작성 에이전트를 순차적으로 연결합니다.

> · NOTE
>
> 참고 — 3노드 RAG 파이프라인 패턴 `docs/langchain/23-custom-workflow.md` 에서는 RAG 시나리오에서 자주 쓰이는 Rewrite → Retrieve → Agent 3노드 구성을 소개합니다. 1. Rewrite: 사용자의 모호한 질문을 검색 친화적인 쿼리로 재작성 (LLM 노드) 2. Retrieve: 벡터 스토어에서 관련 문서를 가져오기 (결정론적 함수 노드) 3. Agent: 검색 컨텍스트와 도구로 최종 응답 생성 (`create_agent` 노드)

>

> ➡️ TIP
>
> 본 셀의 `research → writer` 예제는 동일한 패턴(LLM 노드 두 개를 직렬 연결)을 따르며, 단지 검색 노드가 에이전트의 도구로 흡수된 형태입니다.

9.5 RAG (Retrieval-Augmented Generation) 개요

RAG는 외부 지식을 검색하여 LLM의 응답을 보강하는 패턴입니다. 크게 3가지 접근 방식이 있습니다:

패턴	설명	특징
기본 2단계	검색 → 생성	단순하고 빠름
에이전틱 RAG	에이전트가 검색 도구를 호출하여 반복	유연하고 정확
하이브리드	키워드 + 시맨틱 검색 결합	검색 품질 향상

- 기본 2단계: 쿼리로 문서를 검색한 후, 검색된 문서를 컨텍스트로 LLM에 전달하여 답변을 생성합니다.

- 에이전틱 RAG: 에이전트가 검색 도구를 사용하여 필요한 정보를 반복적으로 검색하고, 충분한 정보를 모은 후 답변합니다.

RAG 5가지 빌딩 블록

LangChain은 RAG를 구성하는 다섯 가지 컴포넌트를 표준화된 인터페이스로 제공합니다.

#	빌딩 블록	대표 클래스	역할
1	Document Loaders	`PyPDFLoader`, `WebBaseLoader`, `DirectoryLoader`	파일·웹·DB 등 다양한 소스를 표준 `Document` 객체로 적재
2	Text Splitters	`RecursiveCharacterTextSplitter`, `MarkdownHeaderTextSplitter`	긴 문서를 임베딩 가능한 청크로 분할 (chunk_size, overlap)
3	Embedding Models	`OpenAIEmbeddings`, `HuggingFaceEmbeddings`	텍스트를 의미 기반 벡터로 변환
4	Vector Stores	`FAISS`, `Chroma`, `Pinecone`, `Milvus`	임베딩을 저장하고 유사도 검색 수행
5	Retrievers	`vectorstore.as_retriever()`, `MultiQueryRetriever`, `ContextualCompressionRetriever`	검색 인터페이스 추상화. 재순위·필터·하이브리드 지원

이 노트북에서는 2·3·4번을 직접 사용하고, 에이전트가 도구를 통해 Retriever 역할을 수행합니다. 각 블록의 상세 사용법은 `08_integration/04_document_loaders` `05_retrievers` 디렉터리에서 다룹니다.

9.6 간단한 RAG 구현

텍스트를 청크로 분할하고, 간단한 키워드 기반 검색으로 RAG를 구현합니다. 벡터 스토어 없이도 RAG의 핵심 개념을 이해할 수 있습니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 샘플 문서
documents = [
    "LangChain is a framework for building applications with large language models. It provides tools for prompt engineering, memory management, and agent creation.",
    "LangGraph is a low-level orchestration framework for building stateful agents. It uses a graph-based approach with nodes and edges.",
    "Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add logging, guardrails, and custom logic.",
    "Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom workflows.",
    "RAG (Retrieval-Augmented Generation) combines information retrieval with text generation to provide grounded, factual responses.",
]

# 텍스트 분할
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=50,
)

chunks = []
for doc in documents:
```

```
    chunks.extend(text_splitter.split_text(doc))

print(f"원본 문서: {len(documents)}개")
print(f"분할 청크: {len(chunks)}개")
for i, chunk in enumerate(chunks):
    print(f"  [{i}] {chunk[:80]}...")
```

```
원본 문서: 5개
분할 청크: 5개
  [0] LangChain is a framework for building applications with large language models. I...
  [1] LangGraph is a low-level orchestration framework for building stateful agents. I...
  [2] Middleware in LangChain v1 allows you to intercept and modify agent behavior at ...
  [3] Multi-agent systems in LangChain support five patterns: subagents, handoffs, ski...
  [4] RAG (Retrieval-Augmented Generation) combines information retrieval with text ge...
```

9.7 FAISS 벡터 스토어 (선택)

임베딩 기반 유사도 검색을 쓰면 키워드 매칭보다 훨씬 정확한 검색이 가능합니다. 아래는 FAISS 벡터 스토어를 사용하는 예시입니다.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# 임베딩 모델  생성
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 벡터 스토어 생성
vectorstore = FAISS.from_texts(chunks, embeddings)

# 검색
results = vectorstore.similarity_search("LangChain agent patterns", k=3)
for doc in results:
    print(doc.page_content)
```

```
Multi-agent systems in LangChain support five patterns: subagents, handoffs, skills, router, and custom
workflows.
LangChain is a framework for building applications with large language models. It provides tools for prompt
engineering, memory management, and agent creation.
Middleware in LangChain v1 allows you to intercept and modify agent behavior at every step. You can add
logging, guardrails, and custom logic.
```

요약

이 노트북에서 배운 내용:

주제	핵심 내용
StateGraph 기초	노드, 엣지, 상태로 워크플로를 구성합니다
조건부 엣지	`add_conditional_edges` 로 런타임에 분기 처리합니다
에이전트 통합	`create_agent` 를 StateGraph 노드로 사용하여 파이프라인을 구성합니다

RAG 패턴	검색 + 생성을 결합하여 사실 기반 응답을 제공합니다
벡터 스토어	FAISS 등으로 임베딩 기반 유사도 검색이 가능합니다

다음 노트북에서는 에이전트를 프로덕션 환경으로 배포하는 방법을 알아봅니다.

10 프로덕션

학습 목표

에이전트를 테스트, 배포, 모니터링하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- LangSmith Studio를 사용한 로컬 개발 및 디버깅
- `GenericFakeChatModel`로 결정론적 에이전트 테스트
- 트라젝토리 기반 테스트로 도구 호출 순서 검증
- Agent Chat UI로 웹 기반 대화
- LangGraph Platform 및 자체 서버 배포
- LangSmith를 활용한 관측성(Observability)

10.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

10.2 LangSmith Studio

로컬에서 에이전트를 개발하고 디버깅합니다.

준비물:

- `langgraph.json` 설정 파일
- `langgraph dev` 명령으로 로컬 서버 실행
- Studio UI(hosted)에서 인터랙티브 테스트

Studio 핵심 기능

기능	설명
Hot-reloading	프롬프트·툴 코드를 저장하면 서버 재시작 없이 즉시 반영
Full execution trace	각 노드/툴 호출의 입·출력·메타데이터를 풀 트레이스로 확인
Thread replay	과거 스레드를 불러와 임의 시점부터 재실행(time-travel)
Exception capture	예외 발생 시 주변 상태(state·메시지·툴 인자)까지 함께 캡처

접속 URL

로컬 dev 서버는 `http://127.0.0.1:2024` 에서 떠 있고, Studio UI 는 hosted 엔드포인트로 접속합니다.

```
https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

```python
# langgraph.json 설정 예시
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:agent"
    },
    "env": ".env"
}

print("langgraph.json 설정 예시:")
print(json.dumps(langgraph_config, indent=2))
print("\n실행 방법:")
print("  $ langgraph dev")
print("  → http://localhost:2024 에서 Studio UI 접근")
```

```
langgraph.json 설정 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}

실행 방법:
  $ langgraph dev
  → http://localhost:2024 에서 Studio UI 접근
```

10.3 에이전트 테스트

LangChain 에이전트는 보통 세 갈래로 테스트합니다.

분류	목적	도구

Unit	단일 노드·툴·프롬프트가 결정론적으로 동작하는지 검증	`GenericFakeChatModel` , `pytest`
Integration	실제 모델·외부 시스템과 함께 end-to-end 흐름 검증	LangSmith dataset, 실서비스 모델
Evals	에이전트 트라젝토리를 결정론 매칭 또는 LLM-as-judge로 평가	`agentevals` , LangSmith Evaluators

`GenericFakeChatModel` 을 쓰면 실제 API 호출 없이 결정론적으로 에이전트를 테스트할 수 있습니다.

- API 비용 없이 테스트 가능
- 항상 동일한 결과 → CI/CD 파이프라인에 적합
- 에이전트 로직(도구 호출, 분기 등)을 독립적으로 검증

```python
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_capital(country: str) -> str:
    """국가의 수도를 반환합니다."""
    capitals = {"Korea": "Seoul", "Japan": "Tokyo", "France": "Paris"}
    return capitals.get(country, "알 수 없음")

# 가짜 모델로 결정론적 테스트
fake_model = GenericFakeChatModel(
    messages=iter([
        AIMessage(content="대한민국의 수도는 서울입니다.")
    ])
)

# 테스트 에이전트
test_agent = create_agent(
    model=fake_model,
    tools=[get_capital],
    system_prompt="당신은 지리 전문가입니다.",
)

print("GenericFakeChatModel 테스트:")
print("  → 결정론적 응답으로 에이전트 동작을 테스트합니다")
print("  → CI/CD 파이프라인에서 API 호출 없이 테스트 가능")
```

```
GenericFakeChatModel 테스트:
  → 결정론적 응답으로 에이전트 동작을 테스트합니다
  → CI/CD 파이프라인에서 API 호출 없이 테스트 가능
```

10.4 트라젝토리 평가 — agentevals

`agentevals` 패키지는 에이전트의 메시지 시퀀스(트라젝토리)를 평가합니다. 도구 호출이 의도한 순서로 일어났는지 결정론적으로 매칭하거나, LLM-as-judge 로 자연어 기준에 따라 채점할 수 있습니다.

Trajectory match 모드 4종

모드	의미

`strict`	동일한 툴이 동일한 순서로 호출되어야 통과
`unordered`	동일한 툴 집합이 호출되면 순서 무관
`subset`	기대 트라젝토리가 실제 트라젝토리에 부분집합으로 포함되면 통과
`superset`	실제 트라젝토리가 기대 트라젝토리를 포함하면 통과

```
# 트라젝토리 테스트 예시
def test_agent_trajectory():
    """에이전트가 예상된 순서로 도구를 호출하는지 테스트합니다."""
    result = test_agent.invoke(
        {"messages": [{"role": "user", "content": "대한민국의 수도는 어디인가요?"}]}
    )

    messages = result["messages"]

    # 검증: 메시지가 존재하는지
    assert len(messages) > 0, "에이전트가 응답하지 않았습니다"

    # 검증: 마지막 메시지가 AI 응답인지
    last_msg = messages[-1]
    assert hasattr(last_msg, 'content'), "마지막 메시지에 content가 없습니다"

    print("✓ 트라젝토리 테스트 통과")
    print(f"  메시지 수: {len(messages)}")
    print(f"  최종 응답: {last_msg.content[:100]}")

try:
    test_agent_trajectory()
except Exception as e:
    print(f"테스트 참고: {e}")
```

```
테스트 참고:
```

```
# agentevals — 트라젝토리 결정론 매칭
# 설치: %pip install -U agentevals
from agentevals.trajectory.match import create_trajectory_match_evaluator
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage

# 기대 트라젝토리: get_capital 도구가 한 번 호출되어야 함
expected = [
    HumanMessage(content="대한민국의 수도?"),
    AIMessage(content="", tool_calls=[{"name": "get_capital", "args": {"country": "Korea"}, "id": "1"}]),
    ToolMessage(content="Seoul", tool_call_id="1"),
    AIMessage(content="서울입니다."),
]

# 실제 트라젝토리(에이전트 실행 결과를 그대로 전달)
actual = expected  # 데모 — 실제로는 agent.invoke(...) 결과의 messages

for mode in ("strict", "unordered", "subset", "superset"):
    evaluator = create_trajectory_match_evaluator(trajectory_match_mode=mode)
    result = evaluator(outputs=actual, reference_outputs=expected)
    print(f"  [{mode:9s}] score={result['score']}")
```

10.5 Agent Chat UI

에이전트와 대화할 수 있는 웹 UI입니다. LangGraph 서버와 연결하여 브라우저에서 직접 에이전트를 테스트할 수 있습니다.

주요 기능:
- 실시간 스트리밍 채팅
- 도구 호출 시각화
- 대화 분기(branching)
- Human-in-the-loop 승인

```
print("Agent Chat UI 설정:")
print("=" * 50)
print("""
# 1. Agent Chat UI 설치 — 정식 CLI
$ npx create-agent-chat-app --project-name my-chat-ui

# 또는 GitHub 레포를 직접 클론
$ git clone https://github.com/langchain-ai/agent-chat-ui.git
$ cd agent-chat-ui && npm install && npm run dev

# 2. LangGraph 서버 시작
$ langgraph dev
#   → 로컬 dev 서버: http://127.0.0.1:2024

# 3. Agent Chat UI 환경설정
#   - Deployment URL: http://127.0.0.1:2024 (로컬) 또는 LangGraph Platform URL
#   - Graph ID:       langgraph.json 의 graphs 키 (예: "agent")
#   - LangSmith API Key: LangSmith 트레이싱을 같이 보고 싶을 때만
""")
print("주요 기능:")
print("  - 실시간 스트리밍 채팅")
print("  - 도구 호출 시각화")
print("  - 대화 분기(branching)")
print("  - Human-in-the-loop 승인")
```

10.6 배포

배포는 크게 LangGraph Platform(관리형), 자체 Docker, 그리고 레거시 FastAPI 래핑 세 트랙이 있습니다. 신규 프로젝트는 Platform 트랙을 권장합니다.

Platform 정식 워크플로

1. 에이전트 코드를 GitHub 레포로 push
2. LangSmith 콘솔 → Deployments → + New Deployment
3. 레포·브랜치·`langgraph.json` 경로·env 를 지정하고 Deploy
4. Deployment URL + Graph ID 를 클라이언트(Chat UI / SDK)에 연결

langgraph-sdk 설치

LangGraph 서버(로컬·Platform 모두)와 통신하려면 `langgraph-sdk` 가 필요합니다.

```
%pip install -q langgraph-sdk
```

```
print("배포 옵션:")
print("=" * 50)
print("""
# 옵션 1 (권장) — LangGraph Platform
#   1) 에이전트 코드를 GitHub repo 에 push
#   2) https://smith.langchain.com/deployments → "+ New Deployment"
#   3) repo · branch · langgraph.json 경로 · env 를 지정
```

```
#   4) Deploy 클릭 → Deployment URL + Graph ID 발급
#   (과거 안내되던 `langgraph deploy` CLI 한 줄 패턴은 더 이상 정식 경로가 아닙니다.)

# 옵션 2 — 자체 Docker
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent

# 옵션 3 (deprecated) — FastAPI/Flask 직접 래핑
#   레거시 통합 용도로만 권장. Platform/SDK 가 제공하는 스레드 관리·체크포인트·
#   스트리밍 프로토콜을 직접 구현해야 하므로 신규 코드에는 사용하지 마세요.
from fastapi import FastAPI

app = FastAPI()

@app.post("/chat")
async def chat(message: str):
    result = agent.invoke({"messages": [{"role": "user", "content": message}]})
    return {"response": result["messages"][-1].content}
""")
```

```
# langgraph-sdk 로 배포된 에이전트 스트리밍 호출 (Python)
# 실제 실행하려면 위에서 langgraph dev 가 떠 있거나 Platform deployment URL 이 있어야 합니다.
example = '''
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://127.0.0.1:2024")  # 로컬 dev 서버
# Platform 의 경우: get_sync_client(url=DEPLOYMENT_URL, api_key=LANGSMITH_API_KEY)

for chunk in client.runs.stream(
    None,                       # thread_id — None 이면 새 스레드 생성
    "agent",                    # Graph ID = langgraph.json 의 graphs 키
    input={"messages": [{"role": "user", "content": "안녕하세요"}]},
    stream_mode="updates",
):
    print(chunk)
'''
print(example)
```

```
# REST 로 직접 호출 — assistant_id + input + stream_mode 페이로드 형태
rest_example = '''
curl -X POST http://127.0.0.1:2024/runs/stream \\
  -H "Content-Type: application/json" \\
  --data '{
    "assistant_id": "agent",
    "input": {"messages": [{"role": "user", "content": "안녕하세요"}]},
    "stream_mode": "updates"
  }'

# Platform deployment 에서는 LangSmith API key 를 함께 보냅니다.
#   -H "X-Api-Key: $LANGSMITH_API_KEY"
'''
print(rest_example)
```

10.7 관측성

LangSmith로 에이전트 동작을 추적합니다. 트레이싱을 활성화하면 에이전트의 모든 실행 단계를 기록하고 분석할 수 있습니다.

LangSmith에서 확인할 수 있는 정보:

- 각 에이전트 호출의 전체 실행 흐름
- 모델 입/출력, 도구 호출, 토큰 사용량

- 지연 시간, 에러, 비용 추적

```
# tracing_context — 코드 블록 단위로 트레이싱을 강제 ON/OFF
import langsmith as ls

example = '''
import langsmith as ls

# 환경변수가 꺼져 있어도 이 블록 안에서는 트레이싱이 켜집니다.
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "안녕하세요"}]})

# 반대로 민감 구간만 꺼두기
with ls.tracing_context(enabled=False):
    agent.invoke({"messages": [{"role": "user", "content": "PII 포함"}]})
'''
print(example)
```

```
# 트레이스에 태그·메타데이터 주입 — 대시보드에서 필터·검색 키로 사용
example = '''
agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요"}]},
    config={
        "tags": ["production", "v1.0"],
        "metadata": {"user_id": "123", "session_id": "456"},
    },
)
'''
print(example)
```

10.8 프로덕션 체크리스트

에이전트를 프로덕션에 배포하기 전에 아래 항목을 확인하세요.

항목	도구	상태
단위 테스트	`GenericFakeChatModel` , `pytest`	
트라젝토리 테스트	커스텀 검증 함수	
관측성 설정	LangSmith 트레이싱	
에러 처리	`try/except` , 재시도 로직	
보안	API 키 관리, 입력 검증, 가드레일	
배포 환경	Docker, LangGraph Platform	
모니터링	LangSmith 대시보드, 알림 설정	
문서화	API 문서, 에이전트 동작 설명	

요약

이 노트북에서 배운 내용:

주제	핵심 내용
LangSmith Studio	`langgraph dev` 로 로컬에서 에이전트를 시각적으로 디버깅합니다
에이전트 테스트	`GenericFakeChatModel` 로 API 호출 없이 결정론적 테스트를 수행합니다
트라젝토리 테스트	도구 호출 순서와 최종 응답을 검증합니다
Agent Chat UI	웹 브라우저에서 에이전트와 대화하고 도구 호출을 시각화합니다
배포	LangGraph Platform, Docker, FastAPI 등으로 배포합니다
관측성	LangSmith로 실행 흐름, 토큰 사용량, 비용을 추적합니다

LangChain v1 에이전트 과정을 마칩니다. 기본 개념부터 프로덕션 배포까지, 에이전트 개발의 전체 라이프사이클을 다루었습니다.

11 MCP (Model Context Protocol)

학습 목표

MCP(Model Context Protocol)로 외부 도구와 컨텍스트를 에이전트에 연결하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- MCP의 개념과 아키텍처(서버/클라이언트/호스트)를 이해한다
- `langchain.mcp.MCPAdapter` 베타 API로 MCP 서버에 연결한다
- `MCPConfig` 와 `await adapter.list_tools()` 패턴으로 MCP 도구를 에이전트에 주입한다
- Stdio와 streamable-http 전송 방식의 차이를 안다
- 다중 MCP 서버를 연결하고 인증·Elicitation·미들웨어로 확장한다

11.1 환경 설정

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("환경 준비 완료.")
```

11.2 MCP 개념

MCP(Model Context Protocol)는 외부 도구와 컨텍스트를 표준화된 방식으로 LLM에 제공하기 위한 오픈 프로토콜입니다. LangChain 1.4에서는 `langchain.mcp.MCPAdapter` 가 베타 표면으로 제공되며, FastMCP 클라이언트 수명주기 위에서 도구를 LangChain Tool로 변환합니다.

아키텍처 구성 요소

구성 요소	역할	예시
MCP 서버	도구, 리소스, 프롬프트를 노출	파일 시스템 서버, DB 서버, API 래퍼
MCP 클라이언트	서버에 연결하여 도구를 가져옴	`MCPAdapter` / FastMCP `Client`

호스트	클라이언트를 관리하고 LLM과 연결	LangChain 에이전트, IDE

`MCPAdapter`는 현재 LangChain Tool 변환에 초점을 둡니다. 프롬프트와 리소스 래퍼는 아직 일대일 대응 API가 없으므로 FastMCP 클라이언트로 직접 읽는 영역입니다.

11.3 `langchain[mcp]` 설치와 베타 MCPAdapter

LangChain 1.4 기준 MCP 도구 연동은 `langchain.mcp.MCPAdapter` 베타 표면을 사용합니다. 실행 환경에는 `langchain[mcp]` extra와 MCP 서버 구현에 필요한 `mcp` / `fastmcp` 계열 패키지가 필요합니다.

이전 별도 adapter 패키지는 마이그레이션 비교 대상으로만 다루고, 새 실습 코드는 `MCPAdapter`와 표준 `MCPConfig`를 기준으로 작성합니다.

```
# MCP 베타 어댑터 설치 명령어
print("MCP 베타 어댑터 설치:")
print("  uv add langchain[mcp] mcp")
print()
print("주요 진입점:")
print("  - from langchain.mcp import MCPAdapter")
print("  - async with MCPAdapter(target) as adapter:")
print("        tools = await adapter.list_tools()")
print("  - target: URL, Path, FastMCP Client, ClientGroup, MCPConfig")
print("  - MCPConfig: {'mcpServers': {'name': {'command': ..., 'args': [...]}}}")
print("주의: 도구 호출은 adapter context 안에서 수행해 연결 수명주기를 분명히 합니다.")
```

11.4 Stdio 전송

Stdio(Standard I/O) 전송은 로컬 서브프로세스로 MCP 서버와 통신합니다. 개발 및 테스트 환경에 적합합니다.

```
from pathlib import Path
import json, tempfile, sys
server_path = Path(tempfile.gettempdir()) / "lc_mcp_math_server.py"
server_code = """from fastmcp import FastMCP
mcp = FastMCP("math")
@mcp.tool()
def add(a: int, b: int) -> int:
    return a + b
if __name__ == "__main__":
    mcp.run(transport="stdio")
"""
server_path.write_text(server_code)
stdio_config = {"mcpServers": {"math": {"command": sys.executable, "args": [str(server_path)]}}}
print("Stdio MCPConfig:")
print(json.dumps(stdio_config, indent=2, ensure_ascii=False))
print(f"\n서버 파일: {server_path}")
```

MCP 서버와 어댑터 연결이 되는지 모델 호출 없이 먼저 확인합니다. 이 셀은 도구 discovery와 직접 tool invocation만 수행하므로 오프라인 검증에 적합합니다.

```
from langchain.mcp import MCPAdapter

async with MCPAdapter(stdio_config) as adapter:
    tools = await adapter.list_tools()
```

```
    add_tool = next(t for t in tools if t.name.endswith("add"))
    result = await add_tool.ainvoke({"a": 2, "b": 3})
print([t.name for t in tools], result[0]["text"])
```

11.5 SSE/HTTP 전송

HTTP(streamable-http) 전송은 웹 기반 통신으로, 원격 MCP 서버에 연결할 때 사용합니다. 인증 헤더와 커스텀 설정을 지원합니다.

```
# HTTP/streamable-http 전송 설정 예시
http_config = {
    "mcpServers": {
        "weather": {"url": "https://weather-mcp.example.com/mcp"}
    }
}
import json
print("HTTP MCPConfig:")
print(json.dumps(http_config, indent=2, ensure_ascii=False))
print("\n사용 패턴: async with MCPAdapter(http_config) -> await adapter.list_tools()")
```

인증 — FastMCP Client가 인증을 소유

원격 MCP 서버 인증은 서버별 설정 딕셔너리에 임의 `auth` 객체를 넣는 방식이 아니라 FastMCP `Client`를 먼저 구성하고 그 클라이언트를 `MCPAdapter`에 넘기는 방식으로 설명합니다. 이렇게 하면 토큰·OAuth·전송 수명 주기가 한곳에 묶입니다. 커스텀 HTTP 인증이 필요하면 FastMCP가 쓰는 `httpx2.Auth` 호환 객체를 사용하고, 일반 `httpx.Auth` 객체를 섞지 않습니다.

방식	예시	비고
정적 토큰	`Client(url, auth=token)`	실습에서는 환경변수에서 토큰 문자열을 읽습니다
OAuth	`Client(url, auth="oauth")`	FastMCP가 OAuth 흐름을 시작합니다. 운영 환경은 영구 토큰 저장소를 별도로 구성합니다
여러 서버	`ClientGroup({"name": Client(...)})`	서버별 인증을 유지한 채 하나의 adapter target으로 넘깁니다

실제 연결과 도구 호출은 아래처럼 `async with MCPAdapter(client_or_group)` 블록 안에서 수행합니다.

```
# FastMCP Client / ClientGroup 기반 인증 구성 예시(객체 생성만 수행)
from fastmcp import Client, ClientGroup

weather_token = "${WEATHER_MCP_TOKEN}"
weather_client = Client("https://weather-mcp.example.com/mcp", auth=weather_token)
calendar_client = Client("https://calendar.example.com/mcp", auth="oauth")
auth_group = ClientGroup({"weather": weather_client, "calendar": calendar_client})
print(type(auth_group).__name__, list(auth_group.clients))
```

11.6 MCP 도구 로드 및 에이전트 통합

MCP 서버에서 가져온 도구를 LangChain 에이전트에 바인딩하는 패턴입니다.

11.7 다중 MCP 서버 연결

여러 서버는 표준 `MCPConfig` 의 `mcpServers` 아래에 둡니다. `transport` 키를 직접 넣기보다 `command` / `args` 또는 `url` 로 전송 방식을 추론하게 두는 것이 새 표면의 기본 흐름입니다.

```
# 다중 MCP 서버 설정 예시
import json, sys
multi_server_config = {"mcpServers": {
    "math": {"command": sys.executable, "args": [str(server_path)]},
    "weather": {"url": "https://weather-mcp.example.com/mcp"},
    "database": {"command": "npx", "args": ["-y", "@modelcontextprotocol/server-postgres"], "env":
{"DATABASE_URL": "postgresql://..."}},
}}
print("다중 MCPConfig:")
print(json.dumps(multi_server_config, indent=2, ensure_ascii=False))
print("도구 이름은 다중 서버에서 server_tool 형식으로 prefix 됩니다.")
```

11.7b Elicitation — 서버가 사용자 입력을 요청

MCP 서버는 도구 실행 중 사용자에게 추가 입력을 요청할 수 있습니다. `MCPAdapter` 를 LangChain agent와 함께 쓰면 현대 MCP 서버가 반환하는 `InputRequiredResult` 가 LangGraph interrupt로 노출됩니다. 기존 server-initiated `ctx.elicit(...)` 푸시 방식은 현재 FastMCP/MCP 조합에서 이 interrupt 루프로 답할 수 없으므로, 교육 예제에서는 `InputRequiredResult` 호환 서버를 전제로 둡니다.

`version="v2"` 응답에서는 interrupt payload의 `requests[0].key` 를 읽고, 같은 key 아래에 `accept` / `decline` / `cancel` 응답을 넣어 재개합니다.

action	의미
accept	사용자가 동의하고 `content` 에 값을 제공합니다
decline	사용자가 거부했으므로 서버가 대체 흐름을 선택합니다
cancel	사용자가 취소했으므로 도구 실행을 중단합니다

```
# Elicitation — interrupt payload의 requests[0].key로 재개한다
elicit_example = '''
from langgraph.types import Command
first = await agent.ainvoke(input_state, config=config, version="v2")
payload = first.interrupts[0].value; key = payload.requests[0].key
resume = {"responses": {key: {"action": "accept", "content": {"value": "사용자 입력"}}}}
final = await agent.ainvoke(Command(resume=resume), config=config, version="v2")
'''
print(elicit_example)
```

11.8 도구 호출 미들웨어

MCP 도구도 LangChain Tool로 변환된 뒤에는 일반 도구와 같은 middleware 표면을 사용합니다. 인증은 FastMCP `Client`에 두고, 호출 로깅·정책·응답 필터링은 `@wrap_tool_call` 같은 LangChain middleware에서 처리합니다.

사용 사례	설명
호출 로깅	MCP 메타데이터가 있는 도구 호출을 기록
요청 정책	위험한 도구 이름이나 인자를 검사
응답 필터링	민감한 정보를 ToolMessage에서 제거
재시도 연결	도구 호출 예외를 보고 LangChain 쪽 재시도 정책 적용

```
# 도구 호출 정책은 LangChain middleware에서 처리한다
middleware_note = '''
from langchain.agents.middleware import wrap_tool_call
@wrap_tool_call
def guard_mcp_tool(request, handler):
    if (request.tool.metadata or {}).get("mcp"):
        print(f"MCP policy check: {request.tool_call['name']}")
    return handler(request)
'''
print(middleware_note)
```

11.9b 도구 수명주기 — adapter context 안에서 실행

이전 adapter 패키지의 세션 바인딩 헬퍼는 새 `langchain.mcp`의 직접 대응 API가 아닙니다. 교육 예제에서는 `async with MCPAdapter(...)` 블록 안에서 도구 목록을 만들고 에이전트 호출까지 끝내 연결 수명주기를 명확히 합니다.

```
# Adapter context 안에서 도구 발견과 agent 호출을 함께 수행
stateful_example = '''
from langchain.mcp import MCPAdapter
async def run_with_adapter_lifetime():
    async with MCPAdapter(stdio_config) as adapter:
        tools = await adapter.list_tools()
        agent = create_agent(model=model, tools=tools)
        return await agent.ainvoke({"messages": [{"role": "user", "content": "2와 3을 더하고 결과를 5와 곱해
주세요."}]})
'''
print(stateful_example)
```

11.10 커스텀 MCP 서버 작성

FastMCP 라이브러리를 쓰면 데코레이터로 간편하게 MCP 서버를 구축할 수 있습니다.

요약

이 노트북에서 배운 내용:

주제	핵심 내용
MCP 개념	외부 도구와 컨텍스트를 표준화된 방식으로 LLM에 제공하는 오픈 프로토콜입니다
Stdio 전송	로컬 서브프로세스를 통한 통신으로, 개발/테스트에 적합합니다
Streamable HTTP	웹 기반 통신으로, 원격 서버 연결 및 인증을 지원합니다
에이전트 통합	`async with MCPAdapter(...)` 안에서 `await adapter.list_tools()` 후 `create_agent(tools=...)`로 연결합니다
다중 서버	`MCPConfig.mcpServers` 또는 FastMCP `ClientGroup`으로 여러 서버를 묶습니다
인증	bearer/OAuth 인증은 `MCPAdapter`에 넘기기 전 FastMCP `Client`에 구성합니다
Elicitation	서버의 추가 입력 요청은 `payload.requests[0].key` 기반 interrupt/resume 흐름으로 처리합니다
미들웨어	도구 호출 로깅·정책은 `\@wrap_tool_call` 같은 LangChain middleware로 적용합니다
수명주기	도구 발견과 agent 호출은 adapter context 안에서 함께 실행합니다

> REFERENCES
>
> - MCP (Model Context Protocol)

12 프론트엔드 스트리밍

학습 목표

LLM 응답을 실시간으로 스트리밍하여 사용자에게 전달하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:

- LangChain SDK의 스트리밍 기초(`.stream()` , `.astream_events()`)를 이해한다
- Agent `stream_events(..., version="v3")` 의 projection(`messages` , `tool_calls` , `values` , `output`)을 실제로 소비한다
- `useStream` React 훅의 구조와 사용법을 안다
- Python SDK로 실시간 스트리밍을 소비하는 방법을 익힌다
- Headless Tools와 Custom Stream Channels로 UI 확장 지점을 구분한다

12.1 환경 설정

```python
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

print("환경 준비 완료.")
```

12.2 Python SDK 스트리밍 기초

`.stream()` 메서드는 모델 응답을 토큰 단위로 실시간 전달합니다. 전체 응답이 완성되기 전에 부분 결과를 볼 수 있습니다.

12.3 astream_events()

`.astream_events()` 는 비동기 방식으로 모든 내부 이벤트를 스트리밍합니다. 모델 호출, 도구 실행, 체인 단계 등을 세밀하게 추적할 수 있습니다.

주요 이벤트 타입

이벤트	설명
`on_chat_model_stream`	모델 토큰 스트리밍
`on_chat_model_start`	모델 호출 시작
`on_chat_model_end`	모델 호출 완료
`on_tool_start`	도구 실행 시작
`on_tool_end`	도구 실행 완료

```python
import asyncio

async def stream_events_demo():
    """astream_events()로 이벤트 스트리밍 예시"""
    print("이벤트 스트리밍:")
    print("-" * 40)
    async for event in model.astream_events(
        "파이썬의 장점 2가지",
        version="v2",
    ):
        kind = event["event"]
        if kind == "on_chat_model_stream":
            content = event["data"]["chunk"].content
            if content:
                print(content, end="", flush=True)
        elif kind == "on_chat_model_start":
            print(f"[모델 호출 시작]")
        elif kind == "on_chat_model_end":
            print(f"\n[모델 호출 완료]")

await stream_events_demo()
```

```
이벤트 스트리밍:
----------------------------------------
[모델 호출 시작]

파
이
썬
의
 장
점
 두
 가지
는
 다음
과
 같습니다
.


1
.
 **
코
드
가
 간
결
하고
 읽
기
... (truncated)
```

12.4 Event Streaming v3 — projection 기반 agent stream

LangChain 1.3부터 agent 실행에는 `stream_events(..., version="v3")` 를 우선 검토합니다. 기존 `.stream()` 은 토큰을 직접 받는 저수준 API이고, v3 event streaming은 메시지, 도구 호출, 상태, 서브에이전트, 확장 채널, 최종 출력을 projection 단위로 나눕니다.

Projection	의미	UI 매핑
`stream.messages`	모델 메시지와 텍스트 delta	채팅 버블
`stream.tool_calls`	도구 호출 시작·인자·결과	도구 타임라인
`stream.values`	agent state snapshot	디버그/상태 패널
`stream.output`	최종 state	저장/후처리
`stream.subgraphs` / `stream.subagents`	하위 그래프·서브에이전트 이벤트	멀티에이전트 카드
`stream.extensions`	커스텀 projection 채널	진행률·차트·도메인 UI

이 API는 LangGraph v3 event streaming 위에 올라가므로 LangGraph/Deep Agents와 같은 UI 패턴을 공유합니다.

```python
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """도시 날씨를 조회합니다."""
    return f"{city}: 맑음"

streaming_agent = create_agent(
    model="openai:gpt-5.4", tools=[get_weather],
    system_prompt="날씨 질문은 반드시 get_weather 도구를 사용하세요.",
)
stream = streaming_agent.stream_events(
    {"messages": [{"role": "user", "content": "서울 날씨 알려줘"}]},
    version="v3",
)
for message in stream.messages:
    if str(message.text):
        print(str(message.text), end="", flush=True)
print("\noutput keys:", list(stream.output.keys()))
```

12.5 useStream React 훅

`useStream` 은 LangGraph SDK에서 제공하는 React 훅으로, LangGraph 서버와의 스트리밍 통신을 간편하게 처리합니다.

기본 사용법

```tsx
import { useStream } from "@langchain/langgraph-sdk/react";

function Chat() {
  const stream = useStream({
```

```
    assistantId: "agent",
    apiUrl: "http://localhost:2024",
  });

  const handleSubmit = (message: string) => {
    stream.submit({
      messages: [{ content: message, type: "human" }],
    });
  };

  return (
    <div>
      {stream.messages.map((message, idx) => (
        <div key={message.id ?? idx}>
          {message.type}: {message.content}
        </div>
      ))}
      {stream.isLoading && <div>Loading...</div>}
      {stream.error && <div>Error: {stream.error.message}</div>}
    </div>
  );
}
```

주요 반환값

속성	타입	설명
`messages`	`Message[]`	현재 스레드의 전체 메시지
`isLoading`	`boolean`	스트림 진행 여부
`error`	`Error \\], [null`	에러 객체
`interrupt`	`Interrupt`	중단 요청 (HITL)
`submit()`	`function`	메시지 전송
`stop()`	`function`	스트림 중단

12.6 useStream 설정 옵션

파라미터	필수	기본값	설명
`assistantId`	O	—	에이전트 식별자 (배포 대시보드에서 확인)
`apiUrl`	—	`localhost:2024`	에이전트 서버 URL
`apiKey`	—	—	배포된 에이전트 인증 토큰
`threadId`	—	—	기존 대화 스레드에 연결
`onThreadId`	—	—	스레드 생성 시 콜백
`reconnectOnMount`	—	`false`	컴포넌트 마운트 시 진행 중 스트림 재연결

`onCustomEvent`	—	—	커스텀 이벤트 핸들러
`onUpdateEvent`	—	—	상태 업데이트 핸들러
`onMetadataEvent`	—	—	메타데이터 이벤트 핸들러
`messagesKey`	—	`"messages"`	메시지를 담는 상태 키
`throttle`	—	`true`	상태 업데이트 배치 처리
`initialValues`	—	—	캐시된 초기 상태

12.7 스레드 관리와 재연결

스레드 ID 관리

`threadId`를 관리하면 대화를 지속하거나 이전 대화를 불러올 수 있습니다.

```
const [threadId, setThreadId] = useState<string | null>(null);

const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  threadId,
  onThreadId: setThreadId,
});

// threadId를 URL 파라미터나 localStorage에 저장하여 지속성 확보
```

페이지 새로고침 후 재연결

`reconnectOnMount`를 활성화하면 페이지 새로고침 후에도 진행 중이던 스트림에 자동 재연결됩니다.

```
const stream = useStream({
  apiUrl: "http://localhost:2024",
  assistantId: "agent",
  reconnectOnMount: true, // sessionStorage 사용
});

// 커스텀 스토리지 사용
const stream = useStream({
  reconnectOnMount: () => window.localStorage,
});
```

12.8 브랜칭과 메시지 편집

브랜칭을 쓰면 대화 히스토리의 특정 지점에서 대체 경로를 만들 수 있습니다. 메시지를 편집하거나 AI 응답을 재생성할 때 유용합니다.

```
{stream.messages.map((message) => {
  const meta = stream.getMessagesMetadata(message);
```

```
  const parentCheckpoint = meta?.firstSeenState?.parent_checkpoint;

  return (
    <div key={message.id}>
      {message.content}

      {/* 사용자 메시지 편집 */}
      {message.type === "human" && (
        <button onClick={() => {
          const newContent = prompt("Edit:", message.content);
          if (newContent) {
            stream.submit(
              { messages: [{ type: "human", content: newContent }] },
              { checkpoint: parentCheckpoint }
            );
          }
        }}>
          Edit
        </button>
      )}

      {/* AI 응답 재생성 */}
      {message.type === "ai" && (
        <button onClick={() =>
          stream.submit(undefined, { checkpoint: parentCheckpoint })
        }>
          Regenerate
        </button>
      )}
    </div>
  );
})}
```

핵심: `checkpoint` 파라미터로 특정 시점의 상태로 돌아가 새로운 분기를 생성합니다.

12.9 커스텀 스트리밍 이벤트

에이전트에서 커스텀 데이터를 클라이언트로 스트리밍할 수 있습니다. 진행 상황, 중간 결과 등을 실시간으로 전달할 때 유용합니다.

```
# 커스텀 스트리밍 이벤트 — Python writer 패턴
print("커스텀 스트리밍 이벤트 패턴 (Python 서버 측):")
print("=" * 50)
print("""
from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    \"\"\"데이터를 분석합니다.\"\"\"
    if config.writer:
        # 진행 상황을 클라이언트에 스트리밍
        config.writer({
            "type": "progress",
            "message": "데이터 로딩 중...",
            "progress": 25,
        })
        # ... 처리 ...
        config.writer({
            "type": "progress",
            "message": "분석 완료!",
            "progress": 100,
        })
    return '{"result": "분석 완료"}'
```

```
""")
print("클라이언트(React) 측: onCustomEvent 콜백으로 수신")
print('  onCustomEvent: (data) => setProgress(data.progress)')
```

```
커스텀 스트리밍 이벤트 패턴 (Python 서버 측):
==================================================

from langchain.tools import tool
from langchain.agents.types import ToolRuntime

@tool
async def analyze_data(
    data_source: str, *, config: ToolRuntime
) -> str:
    """데이터를 분석합니다."""
    if config.writer:
        # 진행 상황을 클라이언트에 스트리밍
        config.writer({
            "type": "progress",
            "message": "데이터 로딩 중...",
            "progress": 25,
        })
        # ... 처리 ...
        config.writer({
            "type": "progress",
            "message": "분석 완료!",
            "progress": 100,
        })
    return '{"result": "분석 완료"}'

클라이언트(React) 측: onCustomEvent 콜백으로 수신
  onCustomEvent: (data) => setProgress(data.progress)
```

12.10 멀티 에이전트 스트리밍

여러 에이전트가 협업하는 환경에서는 각 에이전트의 메시지를 구분하여 표시해야 합니다. 메타데이터의 `langgraph_node`로 메시지 출처를 식별합니다.

```tsx
// 노드별 설정
const NODE_CONFIG: Record<string, { label: string; color: string }> = {
  researcher: { label: "Research Agent", color: "blue" },
  writer:     { label: "Writing Agent",  color: "green" },
  reviewer:   { label: "Review Agent",   color: "purple" },
};

// 메시지 렌더링
function AgentMessage({ message, stream }) {
  const metadata = stream.getMessagesMetadata?.(message);
  const nodeName = metadata?.streamMetadata?.langgraph_node;
  const config = NODE_CONFIG[nodeName];

  return (
    <div className={`bg-${config?.color}-950/30 p-4 rounded-lg`}>
      <div className={`text-${config?.color}-400 text-sm font-bold`}>
        {config?.label ?? "Agent"}
      </div>
      <div>{message.content}</div>
    </div>
  );
}
```

이벤트 콜백 정리

콜백	용도	스트림 모드
`onUpdateEvent`	그래프 단계 후 상태 업데이트	`updates`
`onCustomEvent`	에이전트의 커스텀 이벤트	`custom`
`onMetadataEvent`	실행 및 스레드 메타데이터	`metadata`
`onError`	에러 처리	—
`onFinish`	스트림 완료	—

12.11 Headless Tools와 Custom Stream Channels

최신 LangChain/LangGraph UI 문서는 프론트엔드 확장을 두 축으로 나눕니다.

기능	언제 쓰나	프론트엔드 연결
Headless Tools	서버에는 tool schema만 두고, 실제 실행은 브라우저/디바이스 API에서 해야 할 때	`useStream({ tools: [...] })`, `stream.toolCalls`
Custom Stream Channels	모델 토큰이 아닌 별도 진행률·상태·차트 데이터를 흘려보낼 때	`useExtension`, `useChannel`

Headless tool은 "도구 실행을 서버가 아니라 클라이언트에서 하게 할까"의 문제입니다. 서버 tool은 `interrupt()` 로 실행을 프론트엔드에 넘기고, 브라우저 구현은 같은 tool 이름과 schema를 mirror한 뒤 `.implement(...)` 로 붙입니다.

```
const stream = useStream<AgentState>({
  apiUrl: AGENT_URL,
  assistantId: "headless_tools",
  tools: [memoryPut, memoryGet, geolocationGet],
});
```

Custom channel은 "기본 메시지 스트림 밖의 데이터를 어디로 보낼까"의 문제입니다. 최신값만 필요하면 `useExtension("channel-name")`, 로그/히스토리가 필요하면 `useChannel(stream, ["custom:channel-name"])` 로 분리합니다.

요약

이 노트북에서 배운 내용:

주제	핵심 내용
SDK 스트리밍	`.stream()` 으로 토큰 단위 실시간 응답을 받습니다

astream_events	비동기 이벤트 스트리밍으로 모델/도구 호출을 세밀하게 추적합니다
Event Streaming v3	`stream_events(..., version="v3")` 로 메시지·도구·상태·서브에이전트·확장 projection을 분리합니다
useStream	React 훅으로 LangGraph 서버와 스트리밍 통신을 간편하게 처리합니다
스레드 관리	`threadId` 와 `reconnectOnMount` 로 대화 지속성을 확보합니다
브랜칭	`checkpoint` 기반으로 대화의 대체 경로를 생성합니다
커스텀 이벤트	`writer` 패턴으로 진행 상황 등 커스텀 데이터를 스트리밍합니다
멀티에이전트	`langgraph_node` 메타데이터로 에이전트별 메시지를 구분 표시합니다
Headless Tools	`useStream({ tools })` 로 브라우저 구현을 연결하고 `stream.toolCalls` 로 상태를 표시합니다
Custom Channels	`useChannel` 로 진행률·차트 등 별도 스트림을 수신합니다

REFERENCES

- LangChain Event Streaming: https://docs.langchain.com/oss/python/langchain/event-streaming
- LangGraph Event Streaming: https://docs.langchain.com/oss/python/langgraph/event-streaming
- Streaming
- UI (Agent Chat UI & useStream)
- Headless Tools / Custom Stream Channels

13 가드레일

학습 목표

에이전트의 입력과 출력을 검증하고 필터링하는 가드레일을 설정하는 방법을 알아봅니다.

이 노트북에서 다루는 내용:
- 가드레일의 개념과 필요성을 이해한다
- 결정론적 가드레일과 모델 기반 가드레일의 차이를 안다
- PII 감지 미들웨어를 설정한다
- Human-in-the-Loop 가드레일을 구현한다
- 커스텀 before/after 가드레일을 작성한다

13.1 환경 설정

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5.4",
)

from langchain.agents import create_agent
from langchain.tools import tool

print("환경 준비 완료.")
```

```
환경 준비 완료.
```

13.2 가드레일 개념

가드레일(Guardrails)은 에이전트 실행 과정에서 콘텐츠를 검증하고 필터링하는 안전 메커니즘입니다.

왜 가드레일이 필요한가?
- 개인정보(PII) 유출 방지
- 프롬프트 인젝션 공격 차단
- 부적절하거나 유해한 콘텐츠 차단
- 비즈니스 규칙 및 컴플라이언스 준수

- 출력 품질 및 정확성 검증

두 가지 접근법

접근법	방식	장점	단점
결정론적	정규식, 키워드 매칭, 명시적 규칙	빠르고, 예측 가능하며, 비용 효율적	미묘한 위반 사항 놓칠 수 있음
모델 기반	LLM이나 분류기로 의미를 분석	미묘한 문제도 감지	느리고 비용이 높음

가드레일 적용 시점

```
사용자 입력 → [입력 가드레일] → 에이전트 실행 → [출력 가드레일] → 응답
                    ↑                                  ↑
              before_agent                       after_agent
```

13.3 PII 감지 미들웨어

PIIMiddleware는 이메일, 신용카드 번호, IP 주소 등 개인식별정보(PII)를 자동으로 감지하고 처리합니다.

전략	결과
`redact`	`[REDACTED_EMAIL]` 로 대체
`mask`	부분 가리기 (예: 마지막 4자리만 표시)
`hash`	결정론적 해시로 대체
`block`	감지 시 예외 발생

```python
# PII 감지 미들웨어 설정 예시
print("PII 감지 미들웨어 설정:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-5.4",
    tools=[customer_service_tool, email_tool],
    middleware=[
        # 이메일 주소를 [REDACTED_EMAIL]로 대체
        PIIMiddleware("email",
            strategy="redact",
            apply_to_input=True,
            apply_to_output=False,          # default
            apply_to_tool_results=False),  # default

        # 신용카드 번호를 부분 마스킹 (****-****-****-1234)
        PIIMiddleware("credit_card",
            strategy="mask",
            apply_to_input=True,
            apply_to_tool_results=False),  # default
```

```
        # API 키 감지 시 차단 (커스텀 정규식)
        PIIMiddleware("api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True,
            apply_to_tool_results=False),  # default
    ],
)
""")
print("내장 PII 타입: email, credit_card, ip, mac_address, url")
print("커스텀 감지: detector 파라미터에 정규식 또는 함수 전달")
print()
print("apply_to_* 파라미터 (기본값 False):")
print("  - apply_to_input          사용자 메시지 검사")
print("  - apply_to_output         LLM 최종 응답 검사")
print("  - apply_to_tool_results   도구 반환값 검사")
print("→ 최소 하나는 True로 명시해야 미들웨어가 동작합니다.")
```

13.4 Human-in-the-Loop 가드레일

HumanInTheLoopMiddleware는 민감한 작업을 실행하기 전에 사람의 승인을 요구합니다. 금융 거래, 데이터 삭제, 외부 통신 등 고위험 작업에 씁니다.

```
# Human-in-the-Loop 가드레일 예시
print("Human-in-the-Loop 가드레일:")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,       # 승인 필요
                "delete_db": True,         # 승인 필요
                "search": False,           # 자동 실행
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# 1단계: 에이전트 실행 → send_email에서 중단
result = agent.invoke(
    {"messages": [{"role": "user", "content": "팀에 이메일 보내"}]},
    config=config,
)
# → 중단됨: send_email 실행 전 승인 대기

# 2단계: 승인 후 재개
result = agent.invoke(
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config,
)
""")
print("핵심: checkpointer가 있어야 중단/재개가 가능합니다.")
print("거부 시: {\"type\": \"reject\"}로 도구 실행을 막을 수 있습니다.")
```

```
Human-in-the-Loop 가드레일:
==================================================

from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-4.1",
    tools=[search_tool, send_email_tool, delete_db_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,       # 승인 필요
                "delete_db": True,         # 승인 필요
                "search": False,           # 자동 실행
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": "review-123"}}

# 1단계: 에이전트 실행 → send_email에서 중단
result = agent.invoke(
    {"messages": [{"role": "user", "content": "팀에 이메일 보내"}]},
    config=config,
)
... (truncated)
```

13.5 커스텀 입력 가드레일 — before_agent

`before_agent` 훅은 에이전트 실행 시작 전에 요청을 검증합니다. 세션 수준의 인증, 비율 제한, 콘텐츠 필터링 등에 씁니다.

```python
# 커스텀 입력 가드레일 — ContentFilterMiddleware 클래스
print("커스텀 입력 가드레일 (클래스 방식):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    \"\"\"결정론적 가드레일: 금지 키워드가 포함된 요청을 차단합니다.\"\"\"

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None

        content = first_message.content.lower()
        for keyword in self.banned_keywords:
            if keyword in content:
                return {
```

```
                    "messages": [{
                        "role": "assistant",
                        "content": "부적절한 내용이 포함되어 있습니다."
                    }],
                    "jump_to": "end"
                }
        return None
""")
print("핵심: jump_to='end'로 에이전트 실행을 건너뛰고 즉시 응답합니다.")
print("None을 반환하면 다음 단계(에이전트 실행)로 진행합니다.")
```

```
커스텀 입력 가드레일 (클래스 방식):
==================================================

from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

class ContentFilterMiddleware(AgentMiddleware):
    """결정론적 가드레일: 금지 키워드가 포함된 요청을 차단합니다."""

    def __init__(self, banned_keywords: list[str]):
        super().__init__()
        self.banned_keywords = [kw.lower() for kw in banned_keywords]

    @hook_config(can_jump_to=["end"])
    def before_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        first_message = state["messages"][0]
        if first_message.type != "human":
            return None

        content = first_message.content.lower()
        for keyword in self.banned_keywords:
            if keyword in content:
... (truncated)
```

13.6 커스텀 출력 가드레일 — after_agent

`after_agent` 훅은 에이전트 실행 완료 후 최종 출력을 검증합니다. 모델 기반 안전성 검사, 품질 검증 등에 씁니다.

```
# 커스텀 출력 가드레일 — SafetyGuardrailMiddleware 클래스
print("커스텀 출력 가드레일 (클래스 방식):")
print("=" * 50)
print("""
from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    \"\"\"모델 기반 가드레일: LLM으로 응답 안전성을 평가합니다.\"\"\"

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-5.4-mini")
```

```
    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        safety_prompt = f\"\"\"Evaluate if this response is safe.
        Respond with only 'SAFE' or 'UNSAFE'.

        Response: {last_message.content}\"\"\"

        result = self.safety_model.invoke(
            [{"role": "user", "content": safety_prompt}]
        )

        if "UNSAFE" in result.content:
            last_message.content = (
                "안전하지 않은 응답입니다. 다시 질문해주세요."
            )
        return None
""")
print("핵심: 별도의 경량 모델(gpt-5.4-mini)로 안전성을 평가합니다.")
print("UNSAFE 판정 시 응답 내용을 안전한 메시지로 교체합니다.")
```

```
커스텀 출력 가드레일 (클래스 방식):
==================================================

from langchain.agents.middleware import (
    AgentMiddleware, AgentState, hook_config
)
from langgraph.runtime import Runtime
from langchain.messages import AIMessage
from langchain.chat_models import init_chat_model
from typing import Any

class SafetyGuardrailMiddleware(AgentMiddleware):
    """모델 기반 가드레일: LLM으로 응답 안전성을 평가합니다."""

    def __init__(self):
        super().__init__()
        self.safety_model = init_chat_model("gpt-4.1-mini")

    @hook_config(can_jump_to=["end"])
    def after_agent(
        self, state: AgentState, runtime: Runtime
    ) -> dict[str, Any] | None:
        if not state["messages"]:
            return None

        last_message = state["messages"][-1]
        if not isinstance(last_message, AIMessage):
            return None

        safety_prompt = f"""Evaluate if this response is safe.
... (truncated)
```

13.7 데코레이터 방식 가드레일

클래스 대신 데코레이터를 쓰면 간결하게 가드레일을 정의할 수 있습니다.

```
# 데코레이터 방식 가드레일
print("데코레이터 방식 가드레일:")
print("=" * 50)
```

```
print("""
from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# 입력 가드레일 — 데코레이터
@before_agent(can_jump_to=["end"])
def content_filter(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"금지 키워드를 차단합니다.\"\"\"
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{"role": "assistant",
                    "content": "부적절한 요청입니다."}],
                "jump_to": "end"
            }
    return None

# 출력 가드레일 — 데코레이터
@after_agent(can_jump_to=["end"])
def safety_check(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    \"\"\"응답에 민감한 내용이 없는지 확인합니다.\"\"\"
    last = state["messages"][-1]
    if hasattr(last, 'content') and '비밀번호' in last.content:
        last.content = "민감한 정보가 포함된 응답입니다."
    return None

# 에이전트에 적용
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool],
    middleware=[content_filter, safety_check],
)
""")
print("데코레이터 방식은 간단한 가드레일에 적합합니다.")
print("복잡한 로직(상태 관리, 초기화 등)은 클래스 방식을 사용하세요.")
```

```
데코레이터 방식 가드레일:
==================================================

from langchain.agents.middleware import (
    before_agent, after_agent, AgentState, hook_config
)
from langgraph.runtime import Runtime
from typing import Any

banned_keywords = ["hack", "exploit", "malware"]

# 입력 가드레일 — 데코레이터
@before_agent(can_jump_to=["end"])
def content_filter(
    state: AgentState, runtime: Runtime
) -> dict[str, Any] | None:
    """금지 키워드를 차단합니다."""
    if not state["messages"]:
        return None
    content = state["messages"][0].content.lower()
    for kw in banned_keywords:
        if kw in content:
            return {
                "messages": [{"role": "assistant",
                    "content": "부적절한 요청입니다."}],
                "jump_to": "end"
```

```
            }
    return None

# 출력 가드레일 — 데코레이터
... (truncated)
```

13.8 다중 가드레일 조합

여러 가드레일을 `middleware` 리스트에 순서대로 추가하여 다층 방어를 구성합니다.

```python
# 다중 가드레일 조합
print("다중 가드레일 조합 (다층 방어):")
print("=" * 50)
print("""
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, send_email_tool],
    middleware=[
        # Layer 1: 결정론적 입력 필터
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit"]
        ),

        # Layer 2: PII 보호 (입력 + 출력)
        PIIMiddleware("email",
            strategy="redact", apply_to_input=True),
        PIIMiddleware("email",
            strategy="redact", apply_to_output=True),

        # Layer 3: 민감 도구 사람 승인
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email": True}
        ),

        # Layer 4: 모델 기반 안전성 검사
        SafetyGuardrailMiddleware(),
    ],
)
""")
print("실행 순서:")
print("  입력 → [ContentFilter] → [PII 입력] → 에이전트 실행")
print("       → [HITL 승인] → [PII 출력] → [Safety] → 응답")
print()
print("팁: 빠른 결정론적 가드레일을 앞에, 느린 모델 기반을 뒤에 배치")
```

```
다중 가드레일 조합 (다층 방어):
==================================================

from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, HumanInTheLoopMiddleware
)

agent = create_agent(
    model="gpt-4.1",
    tools=[search_tool, send_email_tool],
    middleware=[
        # Layer 1: 결정론적 입력 필터
        ContentFilterMiddleware(
            banned_keywords=["hack", "exploit"]
        ),
```

```
        # Layer 2: PII 보호 (입력 + 출력)
        PIIMiddleware("email",
            strategy="redact", apply_to_input=True),
        PIIMiddleware("email",
            strategy="redact", apply_to_output=True),

        # Layer 3: 민감 도구 사람 승인
        HumanInTheLoopMiddleware(
            interrupt_on={"send_email": True}
        ),

        # Layer 4: 모델 기반 안전성 검사
        SafetyGuardrailMiddleware(),
... (truncated)
```

13.9 프로덕션 가드레일 패턴

모범 사례

패턴	설명	구현 방식
다층 방어	여러 가드레일을 조합하여 단일 실패점 제거	`middleware=[layer1, layer2, ...]`
빠른 실패	결정론적 검사를 먼저 실행하여 비용 절감	결정론적 → 모델 기반 순서
입출력 분리	입력과 출력에 각각 적합한 가드레일 적용	`before_agent` + `after_agent`
그레이스풀 거부	차단 시 사용자에게 친절한 안내 제공	`jump_to="end"` + 안내 메시지
로깅 및 모니터링	가드레일 트리거 이벤트를 기록	LangSmith 트레이싱 연동
폴백 전략	가드레일 자체가 실패할 경우의 대비책	`try/except` + 기본 정책
테스트	가드레일 동작을 단위 테스트로 검증	`GenericFakeChatModel` 활용

도메인별 가드레일 예시

도메인	주요 가드레일
의료	PII(환자정보), 의학적 조언 면책, 응급상황 감지
금융	PII(계좌정보), 투자 면책, HITL(거래 승인)
고객서비스	감정 분석, 에스컬레이션 감지, PII 마스킹
교육	연령 적합성 검사, 학술 정직성, 콘텐츠 필터링

요약

이 노트북에서 배운 내용:

주제	핵심 내용
가드레일 개념	에이전트 실행 과정에서 콘텐츠를 검증하고 필터링하는 안전 메커니즘입니다
PII 감지	`PIIMiddleware`로 이메일, 신용카드 등 개인정보를 자동 감지/처리합니다
HITL	`HumanInTheLoopMiddleware` 로 민감한 도구 실행 전 사람의 승인을 요구합니다
커스텀 입력	`before_agent` 훅으로 에이전트 실행 전 요청을 검증합니다
커스텀 출력	`after_agent` 훅으로 에이전트 실행 후 응답을 검증합니다
데코레이터	`\@before_agent` , `\@after_agent` 데코레이터로 간결하게 정의합니다
다중 조합	여러 가드레일을 `middleware` 리스트에 추가하여 다층 방어를 구성합니다

REFERENCES

- Guardrails

14 Semantic Search

RAG 전에 검색만 먼저 만들기

학습 목표

LEARNING OBJECTIVES

- → Semantic Search와 RAG Agent의 차이를 구분합니다.
- → LangChain 문서 객체, text splitter, embedding, vector store의 최소 흐름을 연결합니다.
- → LLM 호출 없이도 검색 품질을 점검하는 작은 평가 루프를 만듭니다.

14.1 환경 설정

공식 Learn은 Semantic Search를 RAG 앞 단계로 분리합니다. 이 장은 생성 답변 없이 "문서를 잘 찾는가"만 확인합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain_core.documents import Document
from langchain_core.embeddings import DeterministicFakeEmbedding
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

14.2 작은 문서 컬렉션 만들기

실제 PDF 대신 짧은 문서 세 개를 사용합니다. 검색 파이프라인 구조는 PDF, 웹 문서, Markdown에도 동일하게 적용됩니다.

```
raw_docs = [
    Document(page_content="LangChain agents combine models, tools, and middleware.", metadata={"source":
"langchain"}),
    Document(page_content="LangGraph persists state with checkpointers and stores.", metadata={"source":
```

```
"langgraph"}),
    Document(page_content="Deep Agents include planning, files, subagents, and skills.",
metadata={"source": "deepagents"}),
]
len(raw_docs)
```

14.3 청크로 나누기

Semantic Search의 기본 단위는 "문서 전체"가 아니라 검색 가능한 chunk입니다.

```
splitter = RecursiveCharacterTextSplitter(chunk_size=60, chunk_overlap=10)
chunks = splitter.split_documents(raw_docs)

[(doc.metadata["source"], doc.page_content) for doc in chunks]
```

14.4 임베딩과 벡터스토어

실습 안정성을 위해 deterministic fake embedding을 사용합니다. 실제 서비스에서는 `OpenAIEmbeddings`, provider embedding, 또는 로컬 embedding 모델로 교체합니다.

```
embedding = DeterministicFakeEmbedding(size=64)
vectorstore = InMemoryVectorStore(embedding=embedding)
vectorstore.add_documents(chunks)

retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
```

14.5 검색 실행

검색 결과에는 `page_content`와 `metadata`를 함께 남깁니다. RAG에서는 이 결과가 모델 답변의 근거가 됩니다.

```
query = "Which framework uses checkpointers?"
results = retriever.invoke(query)

for idx, doc in enumerate(results, 1):
    print(idx, doc.metadata["source"], "=>", doc.page_content)
```

14.6 검색 품질 미니 평가

생성형 답변을 만들기 전에 "정답 source가 top-k 안에 있는가"를 확인합니다.

```
test_cases = [
    ("state checkpoint memory", "langgraph"),
    ("agent middleware tools", "langchain"),
    ("skills and subagents", "deepagents"),
]
```

```
for question, expected in test_cases:
    hits = retriever.invoke(question)
    sources = [doc.metadata["source"] for doc in hits]
    print(question, "=>", sources, "PASS", expected in sources)
```

14.7 RAG로 넘어가기 전 체크리스트

항목	확인 질문
chunk	너무 길거나 짧지 않은가?
metadata	출처, 페이지, 섹션을 보존하는가?
retriever	top-k와 필터가 질문 유형에 맞는가?
evaluation	정답 source가 검색되는지 측정했는가?

정리

항목	내용
다룬 기술	`Document`, splitter, embedding, vector store, retriever
핵심 개념	Semantic Search는 RAG의 검색 계층이며, 생성 답변 전에 독립 평가할 수 있습니다.
다음 단계	`09_custom_workflow_and_rag.ipynb`, `07_examples/01_rag_agent.ipynb`

REFERENCES

- `docs/langchain/knowledge-base.md`
- `docs/langchain/retrieval.md`
- `docs/langchain/rag.md`
- `docs/concepts/context.md`

PART

III

LangGraph

Stateful Agent Orchestration

이 파트에서 다루는 내용

- 1. 소개
- 2. Graph API
- 3. Functional API
- 4. 워크플로
- 5. 에이전트
- 6. 영속성과 메모리
- 7. 스트리밍
- 8. 인터럽트와 타임 트래블
- 9. 서브그래프
- 10. 프로덕션
- 11. 로컬 서버
- 12. 내구적 실행
- 13. API 가이드와 Pregel
- 14. Fault Tolerance

- 15. Backward Compatibility
- 16. Case Studies

01 LangGraph 소개

상태 기반 에이전트 오케스트레이션 프레임워크

학습 목표

LangGraph의 핵심 개념과 두 가지 API(Graph API, Functional API)를 이해합니다.

1.1 LangGraph란?

LangGraph는 LangChain 생태계의 저수준 오케스트레이션 프레임워크입니다.

LangChain 3계층 구조

계층	역할	설명
Deep Agents	고수준	사전 구축된 에이전트 시스템
LangChain	에이전트	LLM 에이전트 구축 도구
LangGraph	워크플로	상태 기반 오케스트레이션

핵심 특징

- 상태 관리: TypedDict 기반 상태 정의 및 리듀서
- 지속성: checkpointer로 상태를 자동 저장
- 스트리밍: 실시간 토큰 단위 출력
- Human-in-the-loop: 실행 중 사람이 개입할 수 있도록 중단·재개
- 내구성 실행: 장애 발생 시 자동 복구

1.2 핵심 개념

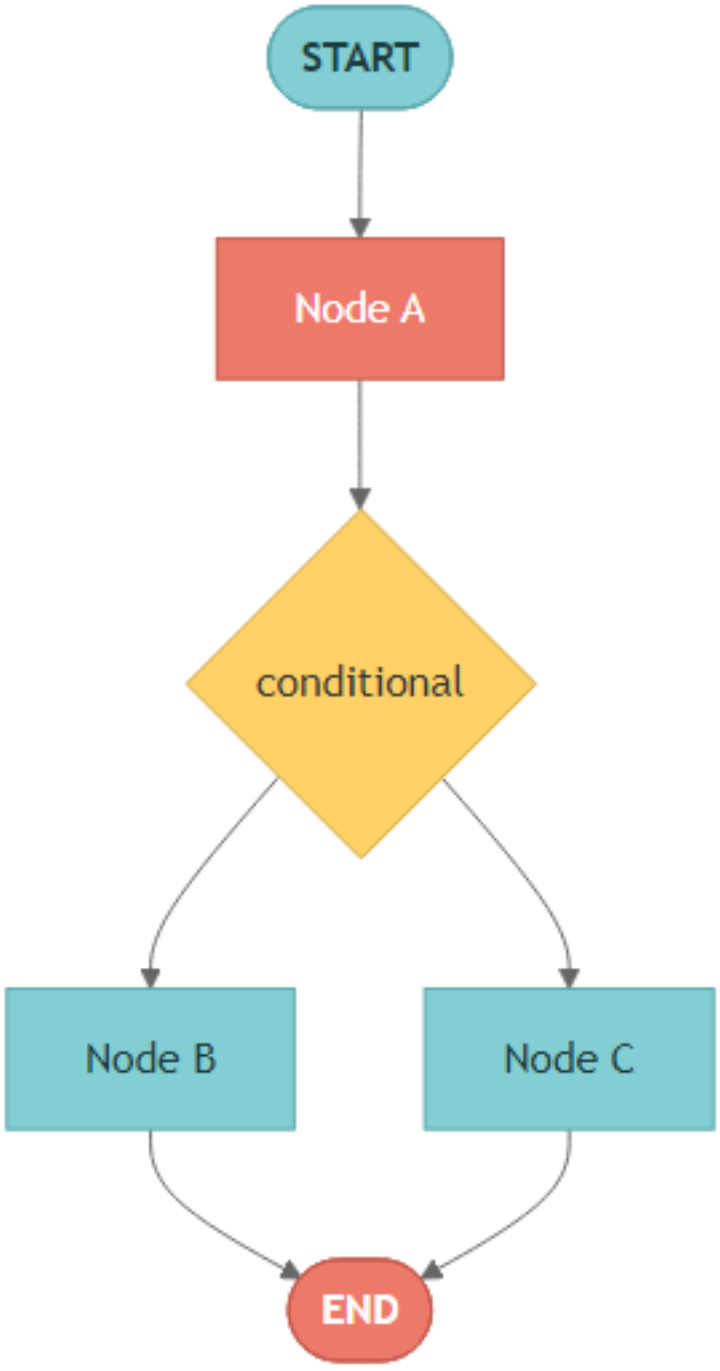
LangGraph는 그래프 구조를 기반으로 워크플로를 정의합니다.

구성 요소

개념	설명
노드(Node)	처리 단위 — Python 함수로 정의

엣지(Edge)	노드 간 연결, 조건부 분기 가능
상태(State)	TypedDict로 정의, 노드 간 공유 데이터
체크포인터(Checkpointer)	각 단계 상태 자동 저장

그래프 구조 다이어그램



1.3 두 가지 API

LangGraph는 동일한 기능을 두 가지 스타일로 구현할 수 있습니다.

특성	Graph API	Functional API
접근 방식	선언적 (노드+엣지)	명령적 (Python 제어 흐름)
상태 관리	명시적 State + 리듀서	함수 스코프, 리듀서 불필요
시각화	그래프 시각화 지원	미지원
체크포인팅	슈퍼스텝마다 새 체크포인트	태스크별, 기존 체크포인트에 저장
적합 상황	복잡한 워크플로, 팀 개발	기존 코드 마이그레이션, 간단한 흐름

1.4 환경 설정 및 설치 확인

필요한 패키지가 올바르게 설치되어 있는지 확인합니다.

```
import importlib

packages = {
    "langgraph": "langgraph",
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
}

print("=" * 50)
print("LangGraph 환경 확인")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"  OK  {package_name}: {version}")
    except ImportError:
        print(f"  ERR {package_name}: 설치되지 않음")
```

```
==================================================
LangGraph 환경 확인
==================================================
  OK  langgraph: installed
  OK  langchain: 1.2.10

  OK  langchain-openai: installed
```

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

required = ["OPENAI_API_KEY"]
optional = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("API 키 상태:")
for key in required:
    print(f"  {'OK' if os.environ.get(key) else 'MISSING'} {key} (필수)")
```

```
for key in optional:
    print(f"  {'OK' if os.environ.get(key) else '--'} {key} (선택)")
```

```
API 키 상태:
  OK OPENAI_API_KEY (필수)
  OK TAVILY_API_KEY (선택)
  -- LANGSMITH_API_KEY (선택)
```

```
# Core import verification
from langgraph.graph import StateGraph, START, END
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, interrupt
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, AIMessage
from langchain_openai import ChatOpenAI

print("모든 핵심 임포트 완료")
```

```
모든 핵심 임포트 완료
```

1.5 Graph API 맛보기

Graph API는 선언적 방식으로 워크플로를 정의합니다.

1. `StateGraph(State)` — 상태 스키마로 그래프 빌더 생성
2. `add_node()` — 노드(함수) 등록
3. `add_edge()` — 노드 간 연결
4. `compile()` — 실행 가능한 그래프 생성
5. `invoke()` — 그래프 실행

1.6 Functional API 맛보기

Functional API는 명령적 방식으로 워크플로를 정의합니다.

- `@task` — 단위 작업 정의 (체크포인팅 단위)
- `@entrypoint` — 워크플로 진입점 정의
- 일반 Python 제어 흐름(`if` , `for` , `while` 등)을 그대로 사용

요약

항목	설명
LangGraph	상태 기반 에이전트 오케스트레이션 프레임워크
Graph API	`StateGraph` 로 명시적 상태 흐름 정의

Functional API	`\@entrypoint` + `\@task` 로 함수형 워크플로
핵심 개념	State (상태), Node (노드), Edge (엣지)
체크포인터	상태 지속성, 멀티턴 대화, 타임 트래블 지원

02 Graph API 기초

StateGraph로 워크플로 만들기

학습 목표

StateGraph, 노드, 엣지, 조건부 분기, 상태 리듀서를 이해합니다.

2.1 환경 설정

LLM 모델과 필요한 모듈을 불러옵니다.

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

2.2 StateGraph 기본 구조

StateGraph를 사용하는 기본 흐름입니다:

1. `StateGraph(State)` — 상태 스키마로 그래프 빌더 생성
2. `add_node()` — 노드(함수) 등록
3. `add_edge()` — 노드 간 연결
4. `compile()` — 실행 가능한 그래프 생성
5. `invoke()` — 그래프 실행

```
StateGraph(State) → add_node() → add_edge() → compile() → invoke()
```

2.3 상태 리듀서

`Annotated`로 상태 업데이트 방식을 지정합니다.

리듀서란?

- 리듀서는 상태 필드가 어떻게 업데이트되는지 결정합니다
- 리듀서 없음: 단순 덮어쓰기 (override)
- `operator.add` : 리스트 항목을 누적 (append)

- 커스텀 함수로 리듀서 정의도 가능

2.4 조건부 엣지

상태에 따라 다른 노드로 분기합니다.

- `add_conditional_edges(source, routing_function)` — 라우팅 함수의 반환값이 다음 노드 이름
- 라우팅 함수는 `Literal` 타입 힌트를 사용하면 시각화에 도움

```
START → classify → [route] → weather → END
                           → math    → END
                           → general → END
```

2.5 메시지 기반 상태

LLM 에이전트에 적합한 `MessagesState` 를 사용합니다.

- `MessagesState` 는 `messages: Annotated[list[AnyMessage], add_messages]` 를 포함하는 사전 정의된 상태
- `add_messages` 리듀서가 메시지 리스트를 자동으로 누적
- LLM 응답을 메시지 히스토리에 자연스럽게 추가

2.6 입출력 스키마

그래프의 입출력을 내부 상태와 분리합니다.

- `StateGraph(InternalState, input_schema=InputSchema, output_schema=OutputSchema)`
- 입력 스키마: 외부에서 받는 데이터만 포함
- 출력 스키마: 외부로 내보내는 데이터만 포함
- 내부 상태: 중간 처리용 필드 포함 (외부에 노출되지 않음)

2.7 런타임 컨텍스트 — `context_schema` + `Runtime[ContextSchema]`

State에 넣지 않을 의존성(LLM provider, DB connection, user id 등)을 호출 시점에 주입합니다.

- 그래프 컴파일 시 `context_schema=ContextSchema` 지정
- 노드 시그니처에 `runtime: Runtime[ContextSchema]` 추가
- `graph.invoke({...}, context={...})` 으로 전달
- `runtime.execution_info` — thread/run/checkpoint/attempt
- `runtime.server_info` — LangGraph Server의 assistant/graph/user 정보
- `runtime.drain_requested` — graceful shutdown 신호

2.8 `add_node()` 고급 옵션 — retry / timeout / cache / error_handler / defer

`add_node()` 는 일반 함수 외에 신뢰성·성능 제어 파라미터를 받습니다.

파라미터	타입	용도
`retry_policy`	`RetryPolicy(max_attempts=..., retry_on=...)`	일시 장애 자동 재시도. 기본은 대부분의 예외, 단 `ValueError` / `TypeError` 등은 제외
`timeout`	`TimeoutPolicy(run_timeout=..., idle_timeout=...)`	초과 시 `NodeTimeoutError`. async 노드 전용, attempt별 독립 적용
`cache_policy`	`CachePolicy(ttl=..., key_func=...)`	입력 해시 기반 결과 캐싱. 컴파일 시 `cache=InMemoryCache()` 필요
`error_handler`	`Callable[[NodeError], Command]`	retry 소진 후 복구용 `Command` 반환
`defer`	`bool`	다른 분기가 끝날 때까지 노드 실행 지연 (fan-out/fan-in)

2.9 `Send` + `Command` — 동적 fan-out 과 부모 그래프 라우팅

- `Send("worker", {...})` — 조건부 엣지에서 동적으로 워커 인스턴스를 생성, 각자 다른 state를 받음 (map-reduce)
- `Command(update=..., goto=...)` — 노드 반환값에서 state 갱신 + 라우팅을 동시에 표현
- `Command(graph=Command.PARENT, ...)` — 서브그래프 노드에서 부모 그래프로 점프

요약

개념	설명
StateGraph	상태 스키마 기반 그래프 빌더
Node	Python 함수로 정의된 처리 단위
Edge	노드 간 고정 연결 (`add_edge`)
Conditional Edge	상태 기반 동적 분기 (`add_conditional_edges`)
Reducer	`Annotated` + `operator.add` 로 상태 누적 방식 정의
MessagesState	LLM 대화용 사전 정의된 상태
Input/Output Schema	내부 상태와 외부 입출력 분리

`context_schema` + `Runtime`	State에 넣지 않는 호출 시점 의존성 주입
`add_node()` 옵션	`retry_policy` / `timeout` / `cache_policy` / `error_handler` / `defer`
`Send` + `Command`	동적 fan-out (map-reduce), state 갱신 + 라우팅 동시 처리, `Command.PARENT` 로 부모 그래프 점프

03 Functional API

@entrypoint와 @task로 워크플로 만들기

학습 목표

Functional API의 `@entrypoint`, `@task` 패턴과 단기 메모리를 이해합니다.

3.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

3.2 @task — 비동기 작업 단위

- `@task` 데코레이터로 감싸면 체크포인팅 가능
- 호출 시 즉시 Future 객체 반환, `.result()` 로 대기

3.3 병렬 태스크 실행

여러 태스크를 동시에 실행합니다.

3.4 previous — 단기 메모리 (이전 실행 결과 접근)

3.5 entrypoint.final — 반환값과 체크포인트 저장값 분리

3.6 결정론성 요구사항

비결정적 작업은 반드시 `@task` 로 감싸야 합니다.

3.7 LLM 에이전트 (Functional API)

while 루프로 ReAct 에이전트 구현

3.8 RetryPolicy + timeout — `\@task` 신뢰성 강화

`@task(retry_policy=..., timeout=...)` 로 일시 장애 자동 재시도와 타임아웃을 함께 설정합니다.

- `RetryPolicy(retry_on=ValueError)` — 특정 예외만 재시도하도록 좁힐 수 있음
- `RetryPolicy(retry_on=NodeTimeoutError)` — timeout 초과를 재시도로 처리
- `@entrypoint(timeout=5.0)` 으로 워크플로 전체에도 타임아웃 지정 가능

3.9 캐싱 — `\@entrypoint(cache=InMemoryCache())` + `\@task(cache_policy=CachePolicy(ttl=120))`

같은 입력으로 호출되는 비싼 태스크의 결과를 재사용합니다.

- 워크플로 전체에 `cache=InMemoryCache()` 또는 `SqliteCache(...)` 부착
- 태스크별로 `cache_policy=CachePolicy(ttl=초)` 지정
- `key_func` 로 캐시 키 생성 로직 커스터마이즈 가능

3.10 Human-in-the-loop — 구조화된 `interrupt()`

`interrupt(payload)` 로 실행을 일시 중단하고 사람의 입력을 기다립니다.

- 단순 문자열 대신 `dict` 를 전달해 질문 + 컨텍스트 + 기대 입력 포맷을 함께 노출
- 재개는 `Command(resume=...)` — 같은 `thread_id` 에 입력으로 전달
- `interrupt()` 호출은 태스크 내부 다른 작업보다 먼저 와야 함 (재개 시 부수 효과 중복 방지)

요약

기능	설명
`\@task`	비동기 작업, 체크포인팅, 병렬 실행
`\@entrypoint`	워크플로 진입점, 실행 관리
주입 파라미터	`previous` / `store` / `writer` / `config` 를 자동 주입
`.result()`	Future 결과 동기 대기
`previous`	이전 실행 결과 접근 (단기 메모리)
`entrypoint.final(value=..., save=...)`	반환값 ≠ 저장값 분리
`RetryPolicy(retry_on=...)`	`\@task` 에서 특정 예외만 재시도
`\@entrypoint(cache=InMemoryCache())` + `CachePolicy(ttl=120)`	태스크 결과 캐싱
`interrupt({...})`	구조화된 HITL — 재개는 `Command(resume=...)`

04 워크플로 패턴

5가지 핵심 패턴

학습 목표

Prompt Chaining, Parallelization, Routing, Orchestrator-Worker, Evaluator-Optimizer 패턴을 이해합니다.

4.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)

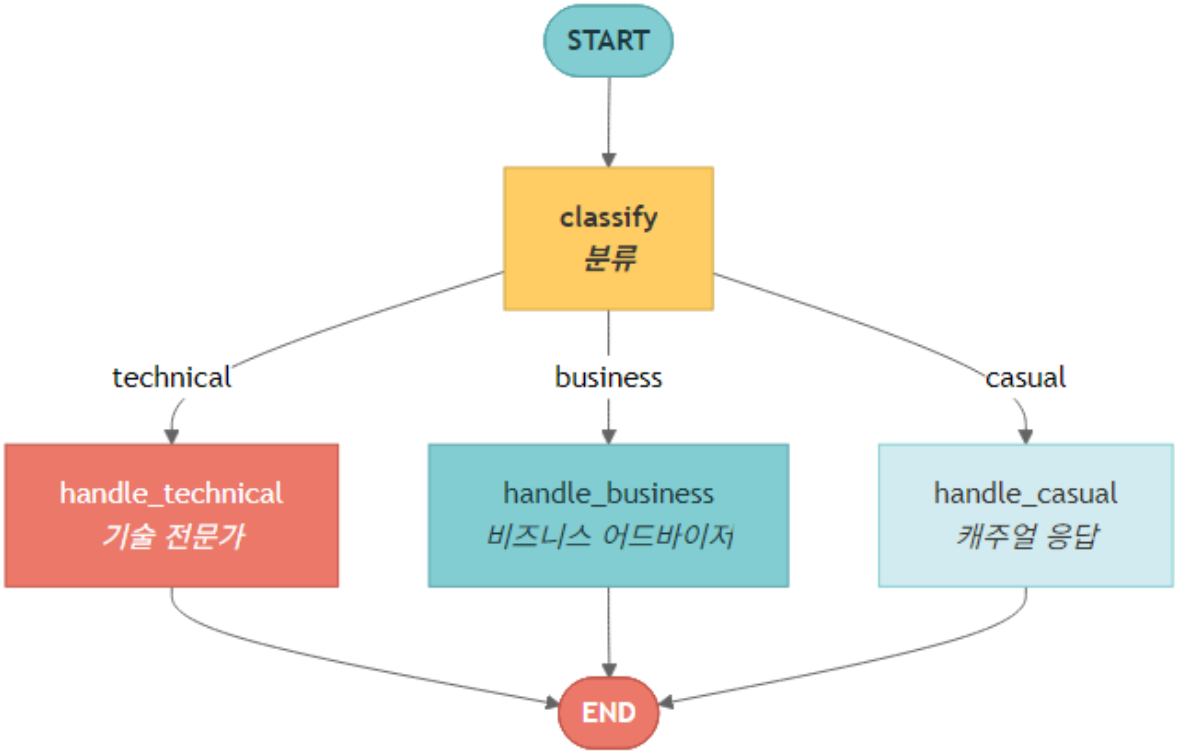
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

4.2 Prompt Chaining — 순차적 LLM 호출

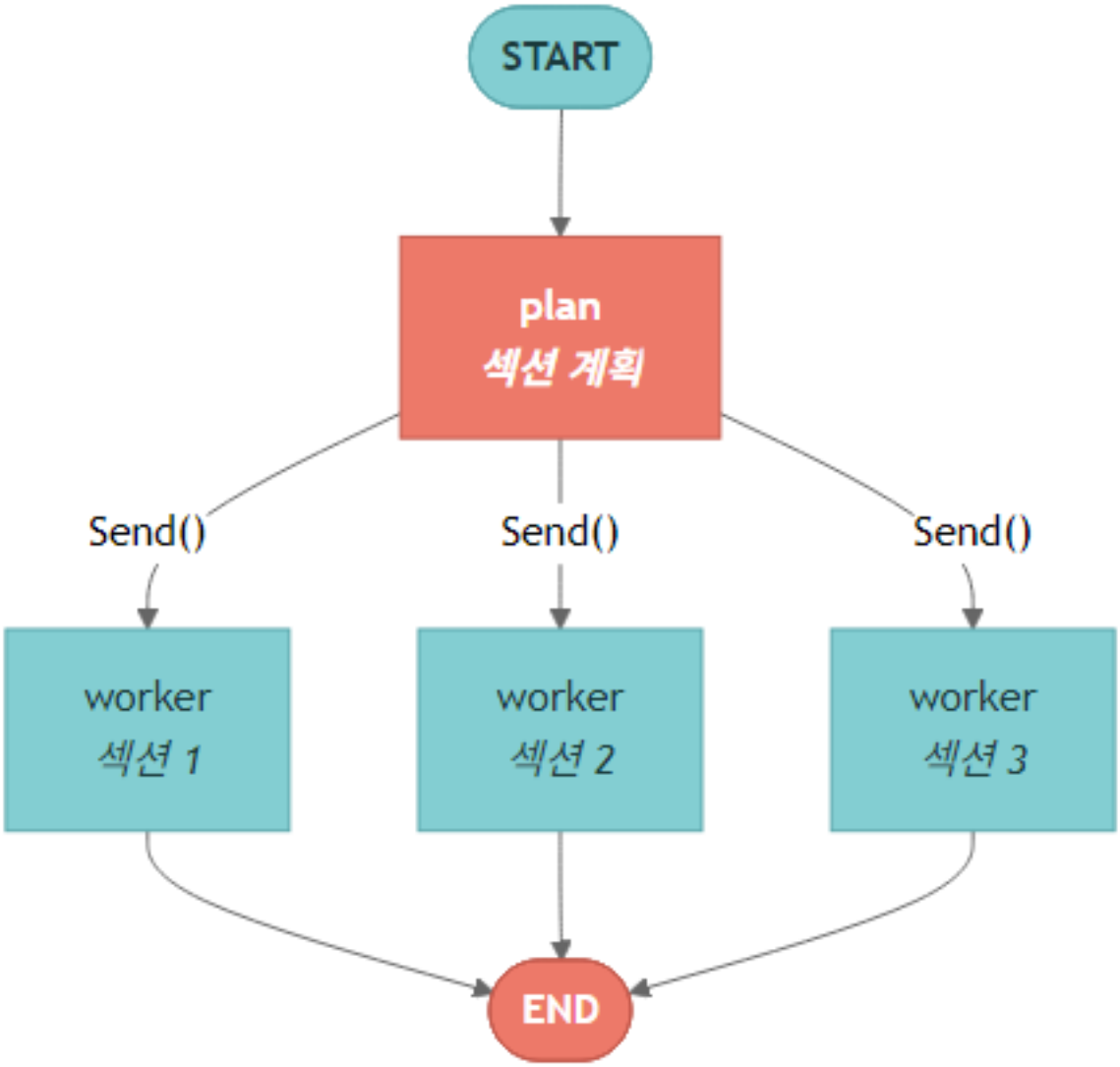
- 각 단계의 출력이 다음 단계의 입력이 됨
- 용도: 번역 → 검증 → 교정, 분석 → 요약 → 포맷팅

4.3 Parallelization — 독립적 태스크의 동시 실행

4.4 Routing — 분류 기반 분기



4.5 Orchestrator-Worker — Send()로 동적 워커 생성



4.6 Evaluator-Optimizer — 생성-평가 반복 루프

4.7 패턴 비교표

패턴	결정적	병렬	반복	적합 상황
Prompt Chaining	O	X	순차	단계별 변환
Parallelization	O	O	X	독립 분석

Routing	O	X	X	분류 기반 처리
Orchestrator-Worker	O	O	X	동적 하위 작업
Evaluator-Optimizer	X	X	O	품질 개선 루프

05 에이전트 구축

Graph API와 Functional API로 ReAct 에이전트 만들기

학습 목표

도구를 사용하는 LLM 에이전트를 두 가지 API로 구현합니다.

- Graph API: `StateGraph`와 조건부 엣지로 ReAct 루프를 명시적으로 구성
- Functional API: `@entrypoint` + `while` 루프로 간결하게 구현
- 도구 바인딩: `@tool` 데코레이터와 `bind_tools()`로 LLM에 도구 연결
- 메모리: 체크포인터로 대화 상태를 유지하는 에이전트

5.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

5.2 도구 정의 — @tool 데코레이터와 bind_tools()

LangChain의 `@tool` 데코레이터를 사용하면 일반 Python 함수를 LLM이 호출할 수 있는 도구로 바꿀 수 있습니다. `bind_tools()`는 이 도구들의 스키마를 모델에 바인딩하여, LLM이 적절한 시점에 도구를 선택하고 인자를 생성하게 합니다.

```
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """a를 b로 나눕니다."""
    return a / b
```

```
tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

print("모델에 바인딩된 도구:")
for t in tools:
    print(f"  - {t.name}: {t.description}")
```

```
모델에 바인딩된 도구:
  - add: 두 수를 더합니다.
  - multiply: 두 수를 곱합니다.
  - divide: a를 b로 나눕니다.
```

5.3 Graph API 에이전트 — StateGraph로 ReAct 루프 구현

ReAct(Reasoning + Acting) 패턴은 세 가지 요소로 구성됩니다:

- LLM 노드: 현재 메시지를 보고 도구 호출 여부를 결정합니다
- Tool 노드: LLM이 선택한 도구를 실제로 실행합니다
- 조건부 엣지: `tool_calls` 가 있으면 `tool_node` 로, 없으면 `END` 로 라우팅합니다

```
START → llm → [tool_calls?] → tools → llm → ... → END
```

5.4 실행 흐름 시각화 — 스트리밍으로 각 단계 관찰

`stream_mode="updates"` 를 사용하면 각 노드가 실행될 때마다 업데이트를 받을 수 있습니다. 에이전트가 어떤 순서로 도구를 호출하고 결과를 처리하는지 단계별로 볼 수 있습니다.

5.5 Functional API 에이전트 — @entrypoint + while 루프

Functional API는 그래프를 명시적으로 구성하지 않고, 일반 Python 코드처럼 에이전트를 작성합니다.

- `@entrypoint` : 에이전트의 진입점을 정의합니다
- `@task` : 개별 작업 단위를 정의합니다
- `while` 루프: 도구 호출이 없을 때까지 반복합니다

5.6 메모리가 있는 에이전트 — 체크포인터로 대화 유지

체크포인터(`InMemorySaver`)를 `compile()` 에 전달하면 에이전트가 이전 대화를 기억합니다. 같은 `thread_id` 를 사용하면 이전 대화 컨텍스트가 자동으로 유지됩니다.

요약

개념	설명
`\@tool`	Python 함수를 LLM 호출 가능한 도구로 변환
`bind_tools()`	도구 스키마를 모델에 바인딩
Graph API 에이전트	`StateGraph` + 조건부 엣지로 ReAct 루프 명시적 구현
Functional API 에이전트	`\@entrypoint` + `while` 루프로 간결하게 구현
`tool_calls`	LLM 응답에 포함된 도구 호출 정보
`ToolMessage`	도구 실행 결과를 LLM에게 전달하는 메시지
체크포인터	대화 상태를 저장하여 멀티턴 에이전트 구현

06

지속성과 메모리

체크포인터와 메모리 스토어

학습 목표

체크포인터로 상태를 저장하고, 스토어로 장기 메모리를 구현합니다.

- 체크포인터: 각 실행 단계의 상태를 자동으로 저장하고 복원
- 상태 조회: `get_state()` 와 `get_state_history()` 로 저장된 상태 확인
- 상태 수정: `update_state()` 로 외부에서 상태 변경
- 스레드 독립성: 서로 다른 `thread_id` 는 완전히 독립된 상태
- InMemoryStore: 스레드 간 공유되는 장기 메모리 (standalone 및 그래프 연동)
- Checkpointer vs Store: 최신 문서 기준으로 thread-scoped state와 cross-thread memory를 구분
- 대화 길이 관리: `trim_messages` 와 `RemoveMessage` 로 메시지 관리
- Durable Execution: 실패 시 마지막 체크포인트에서 재개

6.1 환경 설정

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
```

6.2 체크포인터 — 각 실행 단계의 상태를 자동으로 저장합니다

LangGraph는 다양한 체크포인터 구현체를 제공합니다 (`BaseCheckpointSaver` 를 구현).

구현체	패키지	용도
`InMemorySaver`	`langgraph-checkpoint` (기본)	개발/테스트, 메모리에만 저장
`SqliteSaver` / `AsyncSqliteSaver`	`langgraph-checkpoint-sqlite`	로컬 개발, 파일 영속화
`PostgresSaver` / `AsyncPostgresSaver`	`langgraph-checkpoint-postgres`	프로덕션 (LangSmith가 사용)
`MongoDBSaver`	`langgraph-checkpoint-mongodb`	프로덕션

`RedisSaver` / `AsyncRedisSaver`	`langgraph-checkpoint-redis`	프로덕션
`OracleSaver`	`langgraph-checkpoint-oracle`	프로덕션
`CosmosDBSaverSync` / `CosmosDBSaver`	`langchain-azure-cosmosdb`	Microsoft Entra ID 지원

`compile()`에 체크포인터를 전달하면 각 노드 실행 후 상태가 자동으로 저장됩니다.

6.3 get_state() — 현재 저장된 상태 조회

`get_state()`는 지정된 스레드의 최신 체크포인트 상태를 반환합니다. 메시지 수, 체크포인트 ID, 다음 실행할 노드 등을 확인할 수 있습니다.

```
state = graph.get_state(config)
print(f"스레드: {config['configurable']['thread_id']}")
print(f"메시지 수: {len(state.values['messages'])}")
print(f"체크포인트 ID: {state.config['configurable']['checkpoint_id']}")
print(f"다음 노드: {state.next}")
```

```
스레드: session-1
메시지 수: 4
체크포인트 ID: 1f1196e1-c938-68ee-8004-4f52838ad157
다음 노드: ()
```

6.4 get_state_history() — 전체 실행 이력 조회

`get_state_history()`는 해당 스레드의 모든 체크포인트를 최신순으로 반환합니다. 그래프 실행의 전체 이력을 추적할 수 있습니다.

```
print("상태 이력 (최신순):")
for i, snapshot in enumerate(graph.get_state_history(config)):
    msg_count = len(snapshot.values.get("messages", []))
    print(f"  [{i}] 체크포인트={snapshot.config['configurable']['checkpoint_id'][:20]}... 메시지
={msg_count}")
    if i >= 4:
        print("  ... (생략)")
        break
```

```
상태 이력 (최신순):
  [0] 체크포인트=1f1196e1-c938-68ee-8... 메시지=4
  [1] 체크포인트=1f1196e1-bf0f-6ddf-8... 메시지=3
  [2] 체크포인트=1f1196e1-bf0f-6dde-8... 메시지=2
  [3] 체크포인트=1f1196e1-bf0a-6139-8... 메시지=2
  [4] 체크포인트=1f1196e1-abd9-65c9-8... 메시지=1
  ... (생략)
```

6.5 update_state() — 저장된 상태를 외부에서 수정

`update_state()` 로 체크포인트에 저장된 상태를 프로그래밍 방식으로 수정할 수 있습니다. 예를 들어 시스템 노트를 추가하거나 사용자 선호도를 반영하는 작업이 가능합니다.

6.6 스레드 독립성 — 다른 thread_id는 완전히 독립된 상태

각 `thread_id` 는 완전히 독립된 대화 상태를 가집니다. 다른 스레드의 대화 내용은 서로 영향을 주지 않습니다.

6.6.5 최신 문서 기준 — Checkpointer와 Store를 분리해서 생각하기

LangGraph 최신 문서는 지속성을 checkpointer와 store로 나누어 설명합니다. 둘 다 "저장"처럼 보이지만 범위와 목적이 다릅니다.

구분	Checkpointer	Store
저장 범위	`thread_id` 안의 그래프 상태	여러 thread가 공유하는 장기 메모리
대표 용도	이어 실행, time travel, 상태 조회	사용자 프로필, 선호도, 지식 베이스
연결 방식	`graph.compile(checkpointer=...)`	`graph.compile(store=...)`
노드 접근	`get_state()` , `get_state_history()`	`runtime.store.put/search/get`

따라서 "대화 한 스레드를 복원"하려면 checkpointer, "사용자 A의 선호를 다음 스레드에서도 기억"하려면 store를 사용합니다.

6.7 InMemoryStore — 스레드 간 공유 장기 메모리

`InMemoryStore` 는 스레드 간에 공유되는 키-값 저장소입니다. 사용자 프로필, 선호도 등 스레드를 넘어 유지해야 하는 정보를 저장합니다.

- `put()` : 네임스페이스와 키로 데이터 저장
- `get()` : 특정 항목 조회
- `search()` : 네임스페이스 내 검색

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# 데이터 저장
store.put(("users",), "alice", {"favorite_color": "blue", "city": "Seoul"})
store.put(("users",), "bob", {"favorite_color": "red", "city": "Tokyo"})

# 데이터 조회
alice = store.get(("users",), "alice")
print(f"Alice: {alice.value}")

# 검색
results = store.search(("users",))
print(f"\n전체 사용자 ({len(results)}명):")
for item in results:
    print(f"  {item.key}: {item.value}")
```

```
Alice: {'favorite_color': 'blue', 'city': 'Seoul'}

전체 사용자 (2명):
  alice: {'favorite_color': 'blue', 'city': 'Seoul'}
  bob: {'favorite_color': 'red', 'city': 'Tokyo'}
```

6.7.5 InMemoryStore를 그래프와 함께 사용하기 — `Runtime[Context]` 패턴

`compile(store=store)` 로 그래프에 스토어를 연결하면 노드 함수에서 스토어에 접근할 수 있습니다.

LangGraph 1.x 권장 패턴은 `Runtime[Context]` 입니다.

- `context_schema=Context` 를 `StateGraph()` 에 전달
- 노드 시그니처에 `runtime: Runtime[Context]` 파라미터 선언
- `runtime.context.user_id` 로 타입-안전한 사용자 식별자 접근 (config dict의 `user_id` 키 대신)
- `runtime.store` (또는 `store` 파라미터)로 스토어 접근
- `runtime.store.aput(...)` / `runtime.store.asearch(...)` 비동기 메서드도 사용 가능

`graph.invoke(input, config, context=Context(user_id="..."))` 로 호출 시점에 context를 주입합니다.

6.7.6 대화 길이 관리 — trim_messages와 RemoveMessage

대화가 길어지면 LLM의 컨텍스트 윈도우를 초과할 수 있습니다. LangGraph는 두 가지 방법으로 메시지를 관리합니다:

`trim_messages`

- 토큰 수 기준으로 오래된 메시지를 자동으로 잘라냅니다
- `strategy="last"` : 최근 메시지만 유지
- `start_on="human"` : 잘린 결과가 항상 사용자 메시지로 시작하도록 보장
- 원본 상태는 수정하지 않고 잘린 메시지 목록만 반환합니다 (체크포인트에는 전체 이력이 유지됨)

`RemoveMessage`

- 특정 메시지를 체크포인트에서 영구적으로 삭제합니다
- `MessagesState` 의 리듀서가 `RemoveMessage` 를 감지하여 해당 메시지를 제거합니다
- 오래된 메시지를 정리해 저장 공간을 절약할 때 유용합니다

6.8 Durable Execution — 실패 시 마지막 체크포인트에서 재개

체크포인터를 사용하면 Durable Execution이 가능합니다. 실행 중 오류가 발생해도 마지막으로 성공한 체크포인트에서 재개할 수 있습니다. 이미 완료된 노드는 다시 실행하지 않으므로 비용과 시간을 절약합니다.

아래 예제에서는 3단계 파이프라인을 구성합니다:

1. step_1: 데이터 수집 (항상 성공)
2. step_2: 데이터 분석 (항상 성공)
3. step_3: 외부 API 호출 (첫 번째 실행에서 실패, 재시도 시 성공)

`attempt_count` 를 통해 첫 실행에서 step_3이 실패하고, 두 번째 `invoke()` 호출 시 step_1과 step_2를 건너뛰고 step_3에서만 재개되는 것을 확인합니다.

6.9 프로덕션 체크포인터 / 스토어 — 코드 레퍼런스

`InMemorySaver`는 데모용이며, 프로덕션에서는 DB-backed 체크포인터/스토어를 사용합니다. 모두 `from_conn_string()` 컨텍스트 매니저 패턴을 따르고, 최초 1회 `setup()`을 호출해 스키마를 생성해야 합니다.

> · NOTE
>
> 아래 셀은 의존 DB가 없으면 실행하지 않습니다 — API 패턴 참고용 코드 레퍼런스입니다.

```
# ============================================================================
# 프로덕션 체크포인터 — API 레퍼런스 (실제 실행은 의존 DB가 있을 때만)
# ============================================================================
# 모든 체크포인터는 BaseCheckpointSaver 인터페이스를 구현하므로
# 그래프 측 코드는 동일하고 compile(checkpointer=...) 한 줄만 바뀝니다.

reference_code = r'''
# --- PostgreSQL (sync) ---
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()  # 최초 1회 — 스키마 생성
    graph = builder.compile(checkpointer=checkpointer)

# --- PostgreSQL (async) — 비동기 그래프 실행에 사용 ---
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

async with AsyncPostgresSaver.from_conn_string(DB_URI) as checkpointer:
    await checkpointer.setup()
    graph = builder.compile(checkpointer=checkpointer)
    await graph.ainvoke(..., config={"configurable": {"thread_id": "1"}})

# --- MongoDB ---
from langgraph.checkpoint.mongodb import MongoDBSaver

with MongoDBSaver.from_conn_string("localhost:27017") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

# --- Redis ---
from langgraph.checkpoint.redis import RedisSaver

with RedisSaver.from_conn_string("redis://localhost:6379") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

# --- Oracle ---
from langgraph.checkpoint.oracle import OracleSaver

DB_URI = "oracle://user:password@localhost:1521/?service_name=FREEPDB1"
with OracleSaver.from_conn_string(DB_URI) as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)
'''

print(reference_code)
print("패턴 요약:")
print("  1) from_conn_string(DB_URI) 컨텍스트 매니저 진입")
print("  2) 최초 1회 setup() 호출 → 스키마/인덱스 생성")
print("  3) compile(checkpointer=...) 로 그래프에 연결")
print("  4) 동기/비동기 변형: PostgresSaver vs AsyncPostgresSaver, RedisSaver vs AsyncRedisSaver, ...")
```

6.9.1 프로덕션 Store

`InMemoryStore`도 마찬가지로 프로덕션 구현체로 교체할 수 있습니다. `BaseStore` 인터페이스를 따르므로 노드 측 코드(`runtime.store.put/search/...`)는 그대로 둡니다.

- `PostgresStore` — `langgraph.store.postgres`
- `RedisStore` — `langgraph.store.redis`
- `OracleStore` — `langgraph.store.oracle` (vector search 내장 지원)
- `MongoDBStore` — `langgraph.store.mongodb`

```
# ============================================================================
# 프로덕션 Store — API 레퍼런스
# ============================================================================
store_reference = r'''
# --- PostgresStore ---
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    graph = builder.compile(store=store)

# --- RedisStore ---
from langgraph.store.redis import RedisStore

with RedisStore.from_conn_string("redis://localhost:6379") as store:
    graph = builder.compile(store=store)

# --- OracleStore (vector search 내장) ---
from langgraph.store.oracle import OracleStore

DB_URI = "oracle://user:password@localhost:1521/?service_name=FREEPDB1"
with OracleStore.from_conn_string(DB_URI) as store:
    graph = builder.compile(store=store)

# --- Semantic search 옵션 (모든 store 공통) ---
from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": init_embeddings("openai:text-embedding-3-small"),
        "dims": 1536,
        "fields": ["food_preference", "$"],
    }
)
memories = store.search(namespace, query="What does the user like to eat?", limit=3)
'''
print(store_reference)
print("setup() 한 번이면 끝 — 이후 노드 코드는 runtime.store.put/search/aput/asearch 그대로.")
```

요약

개념	설명
체크포인터	각 노드 실행 후 상태를 자동 저장 (`InMemorySaver` / `SqliteSaver` / `PostgresSaver` / `MongoDBSaver` / `RedisSaver` / `OracleSaver` / `CosmosDBSaver`)
`get_state()`	현재 스레드의 최신 체크포인트 상태 조회
`get_state_history()`	스레드의 전체 체크포인트 이력 조회 (최신순)
`update_state()`	저장된 상태를 프로그래밍 방식으로 수정

스레드 독립성	서로 다른 `thread_id` 는 완전히 독립된 상태
`InMemoryStore`	스레드 간 공유되는 키-값 장기 메모리 저장소 (`BaseStore` 구현)
Checkpointer vs Store	thread-scoped state와 cross-thread memory를 분리 설계
`compile(store=store)` + `Runtime[Context]`	`context_schema` 로 user_id 등을 타입-안전하게 주입, 노드에서 `runtime.context.user_id` 로 namespace 구성
`trim_messages`	토큰 수 기준으로 오래된 메시지를 잘라내어 LLM에 전달 (체크포인트 유지)
`RemoveMessage`	체크포인트에서 특정 메시지를 영구적으로 삭제
Durable Execution	실패 시 마지막 성공 체크포인트에서 재개
프로덕션	`from_conn_string()` + `setup()` 패턴 — `AsyncPostgresSaver` , `OracleSaver` , `OracleStore` 등

07 스트리밍

실시간으로 에이전트 실행 관찰

학습 목표

LangGraph의 다양한 스트리밍 모드를 이해하고 활용합니다.

- `values`, `updates`, `messages`, `custom`, `debug` 모드의 차이를 이해합니다
- 각 스트리밍 모드의 적절한 사용 사례를 파악합니다
- 여러 스트리밍 모드를 동시에 사용하는 방법을 익힙니다

7.1 환경 설정

7.2 스트리밍 모드 비교

LangGraph는 다양한 스트리밍 모드를 제공합니다. 각 모드는 서로 다른 수준의 정보를 실시간으로 전달합니다.

모드	설명	용도
`values`	각 단계의 전체 상태	디버깅, 상태 추적
`updates`	각 노드가 반환한 업데이트만	진행 상황 표시
`messages`	메시지 토큰 단위	채팅 UI
`custom`	사용자 정의 이벤트	커스텀 진행률
`debug`	전체 디버그 정보	개발 중 디버깅

7.3 stream_mode="values" — 전체 상태 스냅샷

`values` 모드는 각 노드 실행 후 전체 상태를 반환합니다. 그래프가 어떻게 진행되는지 전체 흐름을 추적할 때 유용합니다.

7.4 stream_mode="updates" — 노드별 업데이트

`updates` 모드는 각 노드가 반환한 업데이트 값만 전달합니다. 어떤 노드가 어떤 변경을 만들었는지 정확히 파악할 수 있습니다.

7.5 stream_mode="messages" — 토큰 단위 스트리밍

`messages` 모드는 LLM이 생성하는 토큰을 실시간으로 전달합니다. 채팅 UI에서 타이핑 효과를 구현할 때 가장 적합합니다.

7.6 여러 스트리밍 모드 동시 사용

`stream_mode`에 리스트를 전달하면 여러 모드를 동시에 사용할 수 있습니다. 반환되는 이벤트는 `(mode, data)` 튜플 형태입니다.

7.7 stream_mode="custom" — 사용자 정의 스트리밍

`custom` 모드는 노드 내부에서 임의의 데이터를 직접 스트리밍할 수 있게 합니다.

`langgraph.config`의 `get_stream_writer()`를 호출하면 `writer` 함수를 얻을 수 있고, 이 함수에 딕셔너리 등의 데이터를 전달하면 그래프 실행 중 실시간으로 클라이언트에 전송됩니다.

활용 사례:

- 긴 작업의 진행률(progress) 보고
- LangChain을 사용하지 않는 외부 LLM의 청크 단위 스트리밍
- 노드 내부의 중간 결과를 즉시 전달

> ➡️ TIP
>
> `stream_mode="custom"`으로 그래프를 스트리밍하면 `writer()`로 전송한 데이터만 수신됩니다.

7.8 version="v2" — 타입-안전 통일 스트림 (LangGraph 1.1+)

`version="v2"`를 opt-in하면 모든 청크가 `StreamPart` dict로 통일됩니다. v1은 모드/서브그래프 조합에 따라 dict/tuple이 섞여 호출 측 분기 코드가 복잡했습니다.

StreamPart 구조

```
{
    "type": "values" | "updates" | "messages" | "custom" | ...,
    "ns":   (),    # 서브그래프 namespace 튜플
```

```
    "data": ...,   # 모드별 payload
}
```

이점

- 타입 안전성: `langgraph.types` 의 `UpdatesStreamPart`, `CustomStreamPart` 등으로 편집기가 `data` 를 자동 내로잉
- 일관된 분기 코드: 항상 `chunk["type"]` 으로 분기 → v1처럼 tuple vs dict 판별 불필요
- Pydantic / dataclass 자동 강제: `stream_mode="values"` 에서 state가 선언된 타입으로 coerce
- 후방 호환: v1 동작 유지, v2는 순수 opt-in

v1 → v2 비교

측면	v1 (기본)	v2 (opt-in)
반환 형태	모드/서브그래프 조합에 따라 dict/tuple 혼재	항상 `StreamPart` dict
모드 식별	tuple 첫 원소(다중모드) / 암묵적	`chunk["type"]` 명시 필드
namespace	`subgraphs=True` 시에만 tuple 첫 원소	항상 `chunk["ns"]`
타입 추론	제한적	TypedDict로 자동 내로잉
Pydantic/dataclass state	유지 (dict)	values 모드에서 자동 인스턴스화

마이그레이션 전략

1. 새 코드는 `version="v2"` 기본값으로 작성
2. 기존 v1 호출부는 그대로 두어도 무방 (v1은 계속 동작)
3. 타입 안전성이 필요한 핫패스 / 신기능부터 점진 마이그레이션
4. Pydantic state 쓰는 그래프는 자동 강제 덕분에 이득이 가장 큼

7.9 `nostream` 태그 — 백그라운드 LLM을 messages 스트림에서 제외

같은 그래프 안에서 사용자에게 보여줄 응답 모델과 내부 보조 모델(라우팅·요약·툴 결정 등)이 함께 호출되면, 두 모델의 토큰이 모두 `messages` 스트림에 흘러들어 UI가 노이즈를 받습니다.

해결책: 내부 모델에 `nostream` 태그를 부여하면 `stream_mode="messages"` 출력에서 자동으로 제외됩니다.

```
from langchain_anthropic import ChatAnthropic

internal_model = ChatAnthropic(model_name="claude-haiku-4-5-20251001").with_config(
    {"tags": ["nostream"]}
)
# internal_model의 토큰은 messages 스트림에 노출되지 않는다
```

`disable_streaming=True` / `streaming=False` 로 모델 자체의 스트리밍을 끄는 것과는 다릅니다 — `nostream` 은 그래프 스트림에서만 가립니다.

7.10 태그 / 마지막 청크 감지 — `chunk_position == "last"`

`messages` 모드 청크에는 메시지 자체뿐 아니라 풍부한 메타데이터가 함께 옵니다.

- `metadata["tags"]` — 모델 호출에 부여한 태그 리스트 → `["joke"]`, `["poem"]` 처럼 모델별 필터링
- `metadata["langgraph_node"]` — 어떤 노드가 호출한 토큰인지
- `metadata["chunk_position"]` — `"first"` / 중간(없음) / `"last"` 중 하나. `"last"` 가 들어오면 해당 메시지의 스트리밍이 끝났음을 의미해 UI에서 cursor 정리·전송 확정에 사용

```
joke_model = init_chat_model(model="gpt-5.4-mini", tags=["joke"])
poem_model = init_chat_model(model="gpt-5.4-mini", tags=["poem"])

async for chunk in graph.astream(
    {"topic": "cats"},
    stream_mode="messages",
    version="v2",
):
    if chunk["type"] != "messages":
        continue
    msg, meta = chunk["data"]

    # 1) 태그로 모델 분기
    if meta.get("tags") != ["joke"]:
        continue

    # 2) 토큰 출력
    if msg.content:
        print(msg.content, end="", flush=True)

    # 3) 마지막 청크에서 마무리 처리
    if meta.get("chunk_position") == "last":
        print("  <-- joke 모델 완료")
```

7.11 `GraphOutput` — `invoke(..., version="v2")` 의 반환 타입

`stream()` 뿐 아니라 `invoke()` 에도 `version="v2"` 를 줄 수 있습니다. 이때 반환값은 dict가 아니라 `GraphOutput` 객체입니다.

속성	타입	설명
`.value`	`StateT`	그래프 최종 state (Pydantic / dataclass면 자동 인스턴스화)
`.interrupts`	`tuple[Interrupt, ...]`	실행 중 발생한 interrupt들. 없으면 빈 튜플

→ `if result.interrupts:` 한 줄로 "최종 결과 vs interrupt 대기" 분기 가능. → `result["key"]` 같은 dict 스타일 접근도 여전히 동작 (deprecated 경로).

7.12 Event Streaming v3 — projection 기반 앱 스트림

LangGraph의 `stream_events(..., version="v3")` 는 raw `stream_mode` 위에 projection 계층을 얹습니다. `stream_mode` 는 런타임 이벤트를 직접 파싱할 때 좋고, v3 event streaming은 앱 코드가 `stream.messages` ,

`stream.values` , `stream.subgraphs` , `stream.output` , `stream.interrupts` , `stream.extensions` 처럼 목적별 핸들을 소비할 때 좋습니다.

필요	권장 API
노드별 raw update 확인	`graph.stream(..., stream_mode="updates")`
custom event 직접 처리	`graph.stream(..., stream_mode="custom")`
UI에서 메시지·상태·서브그래프를 분리 표시	`graph.stream_events(..., version="v3")`
interrupt 이후 재개 표시	`stream.interrupted` , `stream.interrupts`
typed projection 확장	`StreamTransformer` , `StreamChannel`

```python
# Event Streaming v3 — custom_graph의 values/output projection을 실제 소비합니다.
stream = custom_graph.stream_events(
    {"topic": "LangGraph Event Streaming v3"},
    version="v3",
)
print("values projection:")
for snapshot in stream.values:
    print(snapshot)
print("final output:", stream.output)
print("interrupted:", stream.interrupted)
```

요약

항목	설명
`values` / `updates`	각 단계의 전체 상태 / 변경분만
`messages`	LLM 토큰 + metadata (`tags` , `langgraph_node` , `chunk_position`)
`custom`	`get_stream_writer()` 로 임의 데이터 전송
`debug`	전체 실행 트레이스
여러 모드 동시	v1: `(mode, data)` 튜플 / v2: `chunk["type"]`
`stream_events(..., version="v3")`	projection 기반 — `messages` / `tool_calls` / `values` / `subgraphs` / `output`
interrupt projections	`stream.interrupted` , `stream.interrupts` 로 중단 상태 확인
custom extensions	`stream.extensions` 와 `StreamChannel` 로 도메인 UI 채널 확장

08 인터럽트와 타임 트래블

실행 중단, 승인, 되감기

학습 목표

`interrupt()` 로 실행을 중단하고, `Command(resume=...)` 로 재개합니다. 타임 트래블로 이전 상태로 돌아갑니다.

- Human-in-the-loop 패턴을 구현할 수 있습니다
- 승인 응답에 `response_schema` 를 붙여 resume 입력 형식을 명확히 한다
- Functional API에서도 interrupt를 사용할 수 있습니다
- 체크포인트 히스토리를 활용한 타임 트래블을 수행할 수 있습니다
- `update_state()` 로 외부에서 상태를 수정할 수 있습니다

8.1 환경 설정

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI

# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
print("모델 준비 완료")
```

8.2 interrupt() — 실행을 중단하고 사람의 입력을 기다립니다

- `interrupt(value)` : 현재 상태를 체크포인트에 저장하고 실행을 중단합니다
- `interrupt(value, response_schema=...)` : 사람이 돌려줄 값의 스키마를 함께 제시합니다
- `Command(resume=value)` : 중단된 지점에서 값을 전달하며 재개합니다

민감한 작업 전 사람의 승인을 받거나, 추가 정보를 입력받을 때 사용합니다.

8.3 Command(resume=…) — 중단된 실행을 재개합니다

`Command(resume=value)` 를 사용하면 `interrupt()` 가 호출된 지점에서 실행이 재개됩니다. `resume` 에 전달한 값이 `interrupt()` 의 반환값이 됩니다.

8.4 Functional API에서의 interrupt

Functional API(`@entrypoint` , `@task`)에서도 `interrupt()` 를 동일하게 사용할 수 있습니다.

8.5 Common Patterns — 5가지 표준 interrupt 패턴

공식 문서는 interrupt 활용을 5가지 패턴으로 분류합니다. 모두 `interrupt()` + `Command(resume=...)` 조합이며, 차이는 resume payload의 형태와 노드 내부 로직입니다.

패턴	resume 형태	용도
Approval	`bool` (`True` / `False`)	API 호출·DB 변경 같은 critical action 직전 승인
Review & Edit	`str` 또는 edited dict	LLM 출력 검토·수정 후 재주입
Tool Interrupt	`{"action": "approve", ...}` decision dict	tool 함수 내부에 interrupt를 두고 부분 편집 + 승인
Input Validation	임의 값, 루프 안에서 재시도	유효한 값이 들어올 때까지 다시 묻기
Multiple Interrupts	`{interrupt_id: 응답값}` map	병렬 노드의 동시 interrupt에 id로 매칭

`Command(resume=...)` 는 단일 값 / dict 어떤 형태든 받을 수 있고, 노드의 `interrupt()` 반환값으로 그대로 전달됩니다.

8.5.1 패턴 1 — Approval (`resume=True/False`)

`Command(goto=...)` 와 결합해 승인 여부에 따라 라우팅합니다.

8.5.2 패턴 2 — Review & Edit (`resume="edited text"`)

LLM 출력을 사람이 검토·수정한 뒤 그래프에 다시 주입합니다. resume 값이 그대로 state에 반영됩니다.

8.5.3 패턴 3 — Tool Interrupt (`resume={"action": "approve", ...}`)

tool 함수 내부에 `interrupt()` 를 두면 tool 실행 직전 사람이 승인·편집할 수 있습니다. resume payload는 보통 `{"action": "approve", ...}` 형태의 decision dict로, 승인 여부와 부분 편집을 함께 전달합니다.

8.5.4 패턴 4 — Input Validation (루프 안 interrupt)

루프 안에서 `interrupt()` 를 반복 호출해, 유효한 값이 들어올 때까지 다시 묻습니다. 노드 안 interrupt 호출 순서는 일관성이 보장되므로 루프 안에서도 안전합니다.

8.5.5 패턴 5 — Multiple Interrupts (`resume={interrupt_id: 응답}`)

병렬 노드가 동시에 interrupt를 일으키면, interrupt id를 키로 하는 resume map으로 응답을 일대일 매칭합니다.

```
interrupted = graph.invoke({"vals": []}, config, version="v2")

resume_map = {
    i.id: f"answer for {i.value}" for i in interrupted.interrupts
}
result = graph.invoke(Command(resume=resume_map), config, version="v2")
```

`interrupted.interrupts`는 v2의 `GraphOutput.interrupts`로, 각 `Interrupt`에는 고유 `id`와 노드가 넘긴 `value`가 들어 있습니다. resume map의 키가 일치해야 해당 노드로 값이 전달됩니다.

8.6 타임 트래블 — Replay vs Fork

LangGraph의 체크포인트 시스템은 모든 실행 상태를 저장합니다. `get_state_history()`로 이전 체크포인트를 조회하고, 그 시점에서 두 가지 동작을 할 수 있습니다.

동작	API	효과
Replay	`invoke(None, prev.config)`	그 시점부터 다시 실행. `next`로 표시된 노드부터 재실행되고, 이전 노드 결과는 캐시에서 재사용
Fork	`update_state(prev.config, values=...)` → 반환된 `fork_config`로 `invoke(None, fork_config)`	그 시점에서 state를 수정한 새 branch 생성 → 대체 경로 탐색. 원본 thread는 그대로 유지

> **! WARNING**
>
> 주의 — Replay는 결과를 "캐시에서 읽기만 하는" 게 아니라 실제로 다시 실행합니다. LLM 호출 / API 호출 / interrupt가 다시 발생하며 결과가 달라질 수 있습니다.

8.6.1 Replay 표준 패턴 — `invoke(None, prev.config)`

문서 표준 예제로 다시 짚어봅니다. 2-노드 그래프(`generate_topic` → `write_joke`)에서 `write_joke` 직전 체크포인트로 돌아가 replay하면, `generate_topic`은 캐시된 결과를 재사용하고 `write_joke`만 재실행됩니다.

8.6.2 Fork 표준 패턴 — `update_state(prev.config, values=...)` + `as_node`

같은 `before_joke` 시점에서 topic을 바꿔 다른 농담을 만들어봅니다.

- `update_state(before_joke.config, values={"topic": "chickens"})` → 수정된 state를 적용한 새 checkpoint(fork_config) 반환
- `invoke(None, fork_config)`로 fork branch 실행 → 원본 thread는 그대로 유지
- `as_node="generate_topic"` 옵션으로 "이 update가 generate_topic이 만든 것"임을 명시 → 후속 노드(`write_joke`)부터 실행 재개

8.7 update_state() — 외부에서 상태 직접 수정

`update_state()` 로 외부에서 그래프의 상태를 직접 수정할 수 있습니다. 디버깅, 테스트, 또는 수동 개입이 필요한 경우에 유용합니다.

채널에 reducer가 있으면 값이 병합되고, reducer가 없으면 덮어쓰기됩니다. `MessagesState` 의 `messages` 채널은 reducer로 append되므로 아래 호출은 메시지 목록 끝에 새 메시지를 추가합니다.

8.8 `version="v2"` — `GraphOutput` 으로 인터럽트 분기 (LangGraph 1.1+)

v1의 `invoke()` 는 state dict에 인터럽트 정보가 섞여 반환됩니다. 호출 측에서 "이번 결과가 최종인지, 인터럽트 대기 중인지" 구분하려면 `graph.get_state(config)` 를 따로 호출해야 했습니다.

v2에서는 `GraphOutput` 객체가 반환됩니다.

속성	타입	설명
`.value`	`StateT`	최종 state (Pydantic/dataclass면 자동 인스턴스화)
`.interrupts`	`tuple[Interrupt, ...]`	실행 중 발생한 인터럽트 목록

→ `if result.interrupts:` 한 줄로 분기 가능. 위 Common Patterns 코드도 모두 이 형태로 사용했습니다.

서브그래프 타임트래블 버그 수정 (1.1)

LangGraph 1.1은 인터럽트 + 서브그래프 타임트래블에서 과거 `RESUME` 값을 재사용하던 버그도 함께 수정했습니다. 서브그래프를 쓰며 타임트래블하는 코드가 있다면 1.1로 올리고, 가능하면 `version="v2"` 도 함께 적용하길 권장합니다.

8.8b 승인 응답 스키마 — `response_schema`

승인 UI가 여러 필드를 받아야 할 때는 `interrupt(..., response_schema=...)` 로 사람이 돌려줄 값의 구조를 함께 전달합니다. 기존 `interrupt(value)` 호출은 그대로 유효하며, 스키마는 검토 UI와 resume payload의 계약을 분명히 하기 위한 보강입니다. Pydantic 모델을 스키마로 쓰면 resume dict가 모델 객체로 변환되므로 노드에서는 `decision.approved` 처럼 속성으로 읽습니다.

```
from pydantic import BaseModel
from langgraph.types import interrupt, Command
class ApprovalResponse(BaseModel):
    approved: bool
    comment: str

def schema_review_node(state: dict) -> dict:
    decision = interrupt({"draft": state["draft"]}, response_schema=ApprovalResponse)
    return {"approved": decision.approved, "comment": decision.comment}
print(Command(resume={"approved": True, "comment": "검토 완료"}))
```

요약

기능	API	비고
`interrupt(value, response_schema=...)`	`langgraph.types`	실행 중단, JSON-serializable value와 선택적 응답 스키마 전달
`Command(resume=value)`	`langgraph.types`	중단 지점에서 재개 — `True/False`, `str`, `dict`, `{id: 응답}` map 모두 가능
Approval 패턴	`resume=True/False`	critical action 승인/거부 + `Command(goto=...)`
Review & Edit 패턴	`resume="edited text"`	LLM 출력 검토·수정
Tool Interrupt 패턴	`resume={"action": "approve", ...}`	tool 함수 내부 interrupt, 부분 편집 가능
Input Validation 패턴	루프 안 `interrupt()`	유효한 값까지 재요청
Multiple Interrupts 패턴	`resume={interrupt_id: 응답}`	병렬 노드 동시 interrupt
`get_state_history()`	Graph	체크포인트 이력 (최신순)
Replay	`invoke(None, prev.config)`	그 시점부터 재실행 — LLM 호출은 다시 발생
Fork	`update_state(prev.config, values=..., as_node=...)` → `invoke(None, fork_config)`	새 branch 생성 — 원본 thread 유지
`update_state()`	Graph	외부에서 상태 수정 — reducer 있으면 병합, 없으면 덮어쓰기
`invoke(..., version="v2")`	LangGraph 1.1+	`GraphOutput.value` / `.interrupts` 분기

핵심 규칙:

- `interrupt()`를 try/except로 감싸지 말 것 (인터럽트 예외를 잡아버림)
- interrupt 호출 순서를 일관되게 유지 — 노드 내부 분기에 따라 reorder/skip 금지
- side effect는 interrupt 이후 또는 별도 노드에 둘 것 (재실행 시 중복 방지)

09 서브그래프

그래프 안의 그래프

학습 목표

서브그래프로 복잡한 워크플로를 모듈화합니다.

9.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
```

9.2 서브그래프 개념

- 서브그래프: 다른 그래프의 노드로 사용되는 독립적인 그래프
- 장점: 모듈화, 재사용, 팀별 독립 개발
- 각 서브그래프는 자체 상태(State)를 가짐
- 부모 <-> 서브그래프 간 상태는 공유 키로 매핑

9.3 서브그래프 만들기

9.4 부모 그래프에 서브그래프 추가

9.4.1 패턴 1: 래퍼 노드를 통한 서브그래프 호출 (상태 스키마가 다른 경우)

위 예시(9.4)에서는 부모 그래프와 서브그래프가 동일한 키(`text` , `word_count` , `char_count`)를 공유했기 때문에, 컴파일된 서브그래프를 `add_node()` 에 직접 전달할 수 있었습니다.

실무에서는 부모 그래프와 서브그래프의 상태 스키마가 완전히 다른 경우가 많습니다. 이때는 래퍼 함수(wrapper function)를 사용하여:

1. 부모 상태에서 필요한 필드를 추출하여 서브그래프 입력으로 변환
2. 서브그래프를 실행
3. 서브그래프 출력을 부모 상태 형식으로 매핑

합니다. 공식 문서에서 Pattern 1: Call Subgraph Inside a Node라고 부르는 방식입니다.

9.5 LLM 기반 서브그래프

전문 에이전트를 서브그래프로 구성합니다.

9.6 서브그래프 스트리밍

`subgraphs=True`를 주면 서브그래프 내부 단계도 스트리밍됩니다.

v1 (기본) — `(namespace, data)` 튜플

```
for chunk in graph.stream(inputs, stream_mode="updates", subgraphs=True):
    ns, data = chunk  # tuple unpack
    ...
```

v2 — `chunk["type"]` / `chunk["ns"]` / `chunk["data"]` (권장)

```
for chunk in graph.stream(inputs, stream_mode="updates", subgraphs=True, version="v2"):
    chunk["type"]  # "updates"
    chunk["ns"]    # () for root, ("node_2:<id>",) for subgraph
    chunk["data"]  # {"node_name": {"key": "value"}}
```

v2는 모든 청크가 `StreamPart` dict로 통일되므로 tuple vs dict 판별이 필요 없습니다.

9.7 서브그래프 Checkpointer 3가지 모드

서브그래프 `compile(checkpointer=...)` 인자에 따라 동작이 달라집니다.

모드	`checkpointer=`	동작
Per-Invocation (기본)	`None` (또는 미지정)	매 호출마다 fresh 상태로 시작. 부모의 checkpointer를 상속해 interrupt와 durable execution은 단일 호출 내에서 동작
Per-Thread (stateful)	`True`	동일 thread에서 호출이 누적되어 대화 이력 유지
Stateless	`False`	checkpoint 자체를 만들지 않음. 일반 함수 호출처럼 동작 — interrupt 불가, 프로세스 crash 시 복구 불가

특성	Stateless	Per-Invocation	Per-Thread
`checkpointer=`	`False`	`None`	`True`
Interrupt (HITL)	불가	가능	가능
멀티턴 메모리	없음	없음	있음
상태 조회	불가	현재 호출만	가능
동일 subgraph 병렬 호출	가능	가능	불가 (namespace 충돌)

Per-Thread 주의점 — 병렬 tool 호출 금지

per-thread 서브그래프를 tool로 노출하면, LLM이 이 tool을 동시에 여러 번 호출할 수 있습니다. 두 호출이 같은 namespace에 동시에 쓰려 하면 checkpoint conflict가 발생합니다. `ToolCallLimitMiddleware`로 동시 호출을 막아야 합니다.

```
# 3가지 checkpointer 모드 — API 레퍼런스 (create_agent를 subagent로 쓰는 예)
from langchain.agents import create_agent
from langchain.tools import tool
from langgraph.checkpoint.memory import MemorySaver

@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    return f"Info about {fruit_name}"

# === 1) Per-Invocation (기본) ===
# subagent에 checkpointer 미지정 → 부모로부터 상속, 호출마다 fresh
fruit_subagent_per_invocation = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    system_prompt="You are a fruit expert.",
)

# === 2) Per-Thread (stateful) ===
fruit_subagent_per_thread = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    system_prompt="You are a fruit expert.",
    checkpointer=True,  # ← 핵심
)

# === 3) Stateless ===
fruit_subagent_stateless = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    system_prompt="You are a fruit expert.",
    checkpointer=False,  # ← interrupt 불가
)

# === Per-Thread를 tool로 노출할 때 — 병렬 호출 차단 ===
reference_code = r'''
from langchain.agents.middleware import ToolCallLimitMiddleware

@tool
def ask_fruit_expert(question: str) -> str:
    """Ask the fruit expert."""
    response = fruit_subagent_per_thread.invoke(
        {"messages": [{"role": "user", "content": question}]},
    )
    return response["messages"][-1].content

agent = create_agent(
    model="gpt-5.4-mini",
```

```
    tools=[ask_fruit_expert],
    middleware=[
        ToolCallLimitMiddleware(tool_name="ask_fruit_expert", run_limit=1),
    ],
    checkpointer=MemorySaver(),
)
'''
print(reference_code)
print("3가지 모드 객체 생성 완료:",
      type(fruit_subagent_per_invocation).__name__,
      type(fruit_subagent_per_thread).__name__,
      type(fruit_subagent_stateless).__name__)
```

9.8 Namespace Isolation — 여러 Per-Thread 서브그래프 격리

서로 다른 per-thread 서브그래프 여러 개를 같이 쓰면 namespace가 겹쳐 state 충돌이 생길 수 있습니다. 각 서브그래프를 고유 노드 이름을 가진 별도 `StateGraph` 로 감싸 안정적인 namespace를 부여합니다.

핵심 패턴:

- `agent = create_agent(model=..., name=name, ...)` — agent에 이름 부여
- `StateGraph(MessagesState).add_node(name, agent)` — 같은 이름으로 그래프 노드 등록
- 이 노드 이름이 LangGraph 내부에서 checkpoint namespace 접두사가 됨

```
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent
from langchain.tools import tool


@tool
def veggie_info(veggie_name: str) -> str:
    """Look up veggie info."""
    return f"Info about {veggie_name}"


def create_sub_agent(model: str, *, name: str, **kwargs):
    """고유 노드 이름으로 agent를 감싸 namespace를 격리합니다."""
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)        # 고유 이름 → 안정적 namespace
        .add_edge("__start__", name)
        .compile()
    )


# 서로 다른 두 per-thread subagent — namespace가 격리됨
fruit_agent_isolated = create_sub_agent(
    "gpt-5.4-mini",
    name="fruit_agent",
    tools=[fruit_info],
    system_prompt="You are a fruit expert.",
    checkpointer=True,
)

veggie_agent_isolated = create_sub_agent(
    "gpt-5.4-mini",
    name="veggie_agent",
    tools=[veggie_info],
    system_prompt="You are a veggie expert.",
    checkpointer=True,
)

print("두 per-thread subagent가 서로 다른 namespace로 격리되었습니다.")
print("  - fruit_agent: namespace prefix = ('fruit_agent:<uuid>',)")
```

```
print("  - veggie_agent: namespace prefix = ('veggie_agent:<uuid>',)")
print("→ 두 agent의 checkpoint state가 충돌하지 않습니다.")
```

요약

개념	설명
서브그래프	독립적으로 컴파일된 그래프를 노드로 사용
Pattern 1 (Call inside node)	상태 스키마가 다를 때 — 래퍼 함수로 변환
Pattern 2 (Add as node)	공유 키가 있을 때 — `add_node(name, subgraph)` 직접
스트리밍 v1	`subgraphs=True` → `(namespace, data)` 튜플
스트리밍 v2	`subgraphs=True, version="v2"` → `chunk["type"]` / `chunk["ns"]` / `chunk["data"]`
Per-Invocation (`checkpointer=None`)	매 호출 fresh, 부모 checkpointer 상속
Per-Thread (`checkpointer=True`)	thread별 누적 메모리, 병렬 호출 금지
Stateless (`checkpointer=False`)	checkpoint 없음, interrupt 불가
Namespace Isolation	여러 per-thread subagent는 `StateGraph().add_node(name, agent)` 로 감싸 격리
`get_state(config, subgraphs=True)`	서브그래프 내부 상태 검사 (parent에 checkpointer 필요)

10 프로덕션

테스트, 배포, 관측성

학습 목표

LangGraph 앱을 테스트, 배포, 모니터링하는 방법을 알아봅니다.

10.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

10.2 앱 구조 — langgraph.json

- `langgraph.json` : 그래프 정의, 의존성, 환경 변수 설정
- `langgraph dev` : 로컬 개발 서버 실행

```
import json

config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:graph"
    },
    "env": ".env"
}
print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("명령어:")
print("  $ pip install 'langgraph-cli[inmem]'")
print("  $ langgraph dev  # http://localhost:2024에서 로컬 서버 시작")
```

```
langgraph.json 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./agent.py:graph"
  },
  "env": ".env"
}
```

```
명령어:
  $ pip install 'langgraph-cli[inmem]'
  $ langgraph dev  # http://localhost:2024에서 로컬 서버 시작
```

10.3 LangGraph Studio — 시각적 디버깅 도구

Studio는 `langgraph dev` 실행 시 자동으로 제공됩니다.

Key Features (7가지):

1. Real-time Visualization — 프롬프트, 도구 호출, 결과, 최종 출력 등 모든 단계가 실시간으로 렌더링
2. Interactive Testing — 다양한 입력으로 실행하고 중간 상태를 UI에서 직접 검사
3. Hot-reloading — 프롬프트나 도구 시그니처 수정이 서버 재시작 없이 즉시 반영
4. Trace Inspection — 프롬프트, 도구 인자, 반환값, 토큰 수, 지연 시간을 추적
5. Exception Capture — 예외 발생 시 주변 상태와 함께 캡처되어 디버깅에 활용
6. Thread Replay — 대화 스레드를 임의 지점부터 다시 실행하여 변경 검증
7. Optional Tracing — `LANGSMITH_TRACING=false`로 외부 전송 없이 로컬 실행만 유지

사용 방법:

```
$ langgraph dev
# https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024 에서 접속
# Safari 사용자는 langgraph dev --tunnel 사용
```

10.4 Agent Chat UI

Agent Chat UI는 LangChain 에이전트용 Next.js 채팅 인터페이스입니다. `create_agent`로 만든 에이전트와 바로 연동됩니다.

설치 — npx 사용 (권장):

```
$ npx create-agent-chat-app --project-name my-chat-ui
$ cd my-chat-ui
$ pnpm install
$ pnpm dev
```

설치 — Git clone:

```
$ git clone https://github.com/langchain-ai/agent-chat-ui.git
$ cd agent-chat-ui
$ pnpm install
$ pnpm dev
```

Hosted 버전: https://agentchat.vercel.app 에 접속해 에이전트 deployment URL이나 로컬 서버 주소를 입력합니다.

연결 정보:

- Graph ID — `langgraph.json`의 `graphs` 섹션 키
- Deployment URL — 에이전트 서버 주소(로컬은 `http://localhost:2024`)
- LangSmith API key — 선택(로컬 서버 사용 시 불필요)

기능:

- 실시간 스트리밍 채팅
- 도구 호출 / 결과 렌더링
- Time-travel debugging, state forking
- Generative UI 지원
- Human-in-the-loop 인터럽트 감지/처리

10.5 테스트 — 결정론적 에이전트 테스트

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Graph to test
class TestState(TypedDict):
    input: str
    output: str


def process(state: TestState) -> dict:
    return {"output": state["input"].upper()}


builder = StateGraph(TestState)

builder.add_node("process", process)
builder.add_edge(START, "process")
builder.add_edge("process", END)

graph = builder.compile()


# Unit tests
def test_process():
    result = graph.invoke({"input": "hello"})

    assert result["output"] == "HELLO", f"HELLO 예상, {result['output']} 반환됨"

    print("  OK test_process")


def test_empty_input():
    result = graph.invoke({"input": ""})

    assert result["output"] == "", f"빈 문자열 예상, {result['output']} 반환됨"

    print("  OK test_empty_input")


print("테스트 실행 중:")

test_process()
test_empty_input()

print("모든 테스트 통과!")
```

```
테스트 실행 중:
  OK test_process
  OK test_empty_input
모든 테스트 통과!
```

10.6 LLM 에이전트 테스트 — GenericFakeChatModel 사용

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage, AnyMessage
from langgraph.graph import StateGraph, START, END, MessagesState

# Deterministic fake model
fake_model = GenericFakeChatModel(
    messages=iter(
        [
            AIMessage(content="The answer is 42."),
        ]
    )
)

def chatbot(state: MessagesState) -> dict:
    return {
        "messages": [fake_model.invoke(state["messages"])]
    }

builder = StateGraph(MessagesState)

builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

test_graph = builder.compile()

result = test_graph.invoke(
    {
        "messages": [HumanMessage(content="테스트")]
    }
)

assert "42" in result["messages"][-1].content

print("GenericFakeChatModel 테스트 통과!")
print(f"  응답: {result['messages'][-1].content}")
```

```
GenericFakeChatModel 테스트 통과!
  응답: The answer is 42.
```

10.7 배포 옵션

1. LangSmith Cloud (managed):

GitHub 저장소를 LangSmith Deployments에서 연결하면 자동 배포됩니다(약 15분 소요). 배포 완료 후 Studio 버튼으로 그래프를 띄우고, Deployment details에서 API URL을 복사합니다.

2. Self-hosted Docker:

```
$ langgraph build -t my-agent
$ docker run -p 2024:2024 my-agent
```

3. API 호출 (배포 후):

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="your-deployment-url", api_key="your-langsmith-api-key")
```

```
for chunk in client.runs.stream(
    None,                                           # thread_id=None → stateless run
    "agent",                                        # langgraph.json의 graph name
    input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    stream_mode="updates",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)
```

REST 호출:

```
curl -s --request POST \
    --url <DEPLOYMENT_URL>/runs/stream \
    --header 'Content-Type: application/json' \
    --header "X-Api-Key: <LANGSMITH API KEY>" \
    --data '{"assistant_id": "agent", "input": {"messages": [{"role": "human", "content": "What is
LangGraph?"}]}, "stream_mode": "updates"}'
```

10.8 관측성 — LangSmith 트레이싱

환경 변수 (`.env`):

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2-...
LANGSMITH_PROJECT=my-agent-project   # 선택, 미설정 시 'default'
```

`LANGSMITH_TRACING` / `LANGSMITH_API_KEY` 두 개는 필수, `LANGSMITH_PROJECT` 는 선택입니다.

자동 추적 항목:

- 각 노드 실행 시간
- LLM 입출력, 토큰 사용량
- 도구 호출 및 결과
- 상태 변화
- 에러 및 재시도

Selective tracing — `tracing_context`

특정 구간만 켜거나 끄려면 `langsmith.tracing_context` 컨텍스트 매니저를 사용합니다. 프로젝트 / 태그 / 메타데이터를 동적으로 지정할 수도 있습니다.

```
import langsmith as ls

# 이 호출만 트레이싱
with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "Send a test email"}]})

# 동적 프로젝트 + 태그 + 메타데이터
with ls.tracing_context(
    project_name="email-agent-test",
    enabled=True,
    tags=["production", "email-assistant", "v1.0"],
    metadata={"user_id": "user_123", "session_id": "session_456"},
):
    agent.invoke({"messages": [{"role": "user", "content": "Send a welcome email"}]})
```

`invoke()` 의 `config` 인자로 태그/메타데이터를 한 번에 주입할 수도 있습니다.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "Send a welcome email"}]},
    config={
        "tags": ["production", "email-assistant", "v1.0"],
        "metadata": {"user_id": "user_123", "session_id": "session_456"},
    },
)
```

Data privacy — `LangChainTracer` + `with_config`

민감 정보를 트레이스에 남기지 않으려면 `LangChainTracer` 에 anonymizer를 적용한 `Client` 를 주입하고, 컴파일된 그래프에 `.with_config({"callbacks": [tracer]})` 로 부착합니다.

```
from langchain_core.tracers.langchain import LangChainTracer
from langgraph.graph import StateGraph, MessagesState
from langsmith import Client
from langsmith.anonymizer import create_anonymizer

anonymizer = create_anonymizer([
    {"pattern": r"\b\d{3}-?\d{2}-?\d{4}\b", "replace": "<ssn>"},
])

tracer_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=tracer_client)

graph = (
    StateGraph(MessagesState)
    # .add_node(...).add_edge(...)
    .compile()
    .with_config({"callbacks": [tracer]})
)
```

위 패턴은 SSN처럼 정규식으로 식별 가능한 패턴을 로깅 직전에 마스킹한 뒤 LangSmith로 전송합니다.

노드별 trace 정책 — `trace_policy`

운영 그래프에서는 모든 노드를 같은 깊이로 남기지 않습니다. `StateGraph.add_node(..., trace_policy=...)` 는 특정 노드의 트레이스 수집 방식을 좁히는 지점입니다. 민감한 입력을 다루는 노드, 고빈도 라우터, 비용이 큰 모델 노드를 구분해 LangSmith trace의 노이즈와 개인정보 노출을 줄입니다.

```
from langgraph.graph import StateGraph, START, END
from langgraph.types import TracePolicy

def route(state): return {"route": "answer"}
def answer(state): return {"result": "done"}

builder = StateGraph(dict)
builder.add_node("route", route, trace_policy=TracePolicy(process_inputs=lambda _: {"redacted": True}))
builder.add_node("answer", answer)
builder.add_edge(START, "route"); builder.add_edge("route", "answer"); builder.add_edge("answer", END)
```

10.9 Pregel 런타임 개요

- Pregel은 LangGraph의 내부 실행 엔진
- Graph API와 Functional API 모두 Pregel 위에서 실행됨
- 핵심 개념: 슈퍼스텝, 채널, 체크포인트
- 슈퍼스텝: 동일 레벨의 노드가 병렬 실행되는 단위
- 일반적으로 직접 사용할 필요 없음 (Graph/Functional API가 추상화)

LangGraph 실행 모델:

```
[Super-step 1] Node A, Node B (병렬)
     ↓ 상태 업데이트
[Super-step 2] Node C (A, B 결과 기반)
     ↓ 상태 업데이트
[Super-step 3] Node D
     ↓
END
```

각 슈퍼스텝:

1. 해당 노드들 병렬 실행
2. 상태 업데이트 (리듀서 적용)
3. 체크포인트 저장
4. 다음 슈퍼스텝 결정

10.10 프로덕션 체크리스트

항목	도구	설명
단위 테스트	pytest	개별 노드 함수 테스트
통합 테스트	GenericFakeChatModel	API 호출 없이 전체 흐름
지속성	PostgresSaver	프로덕션 체크포인터
관측성	LangSmith	트레이싱, 모니터링
배포	langgraph deploy	관리형 배포
UI	Agent Chat UI	사용자 인터페이스

요약

주제	핵심 내용
앱 구조	`langgraph.json` 으로 프로젝트 설정
Studio	`langgraph dev` 로 시각적 디버깅

테스트	결정론적 테스트 + GenericFakeChatModel
배포	Platform, Docker, Cloud 옵션
관측성	LangSmith 트레이싱
런타임	Pregel 슈퍼스텝 실행 모델 → #link(“13_api_guide_and_pregel.ipynb”)[13번 노트북]에서 심화

11 로컬 서버

학습 목표

LEARNING OBJECTIVES

- → `langgraph dev` CLI로 개발 서버를 실행하는 방법을 안다
- → LangGraph Studio와 연동하여 시각적으로 디버깅한다
- → `langgraph.json` 설정 파일을 작성한다
- → Python SDK로 로컬 서버를 호출한다
- → 배포 준비 과정을 이해한다

11.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

11.2 LangGraph CLI 설치

LangGraph 로컬 서버를 실행하려면 CLI를 먼저 설치해야 합니다. `langgraph-cli[inmem]` 패키지에는 인메모리 모드가 포함되어 개발·테스트에 적합합니다.

```
# LangGraph CLI 설치 명령어
print("=== pip으로 설치 (Python >= 3.11) ===")
print('  $ pip install -U "langgraph-cli[inmem]"')
print()
print("=== uv로 설치 ===")
print('  $ uv add "langgraph-cli[inmem]"')
print()
print("설치 후 확인:")
print("  $ langgraph --version")
```

```
=== pip으로 설치 (Python >= 3.11) ===
  $ pip install -U "langgraph-cli[inmem]"

=== uv로 설치 ===
  $ uv add "langgraph-cli[inmem]"

설치 후 확인:
  $ langgraph --version
```

11.3 프로젝트 생성

`langgraph new` 명령으로 프로젝트 템플릿을 생성할 수 있습니다. 템플릿을 지정하지 않으면 인터랙티브 메뉴가 표시됩니다.

```python
# 프로젝트 생성 명령어
print("=== 템플릿으로 새 프로젝트 생성 ===")
print("  $ langgraph new my-agent --template new-langgraph-project-python")
print()
print("=== 인터랙티브 메뉴로 생성 ===")
print("  $ langgraph new my-agent")
print()
print("=== 생성 후 의존성 설치 ===")
print("  # pip 사용")
print("  $ cd my-agent && pip install -e .")
print()
print("  # uv 사용")
print("  $ cd my-agent && uv sync")
print()
print("=== 생성되는 파일 구조 ===")
print("  my-agent/")
print("  ├── langgraph.json   # 그래프 설정")
print("  ├── .env.example     # 환경 변수 템플릿")
print("  ├── pyproject.toml   # 의존성 정의")
print("  └── src/")
print("      └── agent.py     # 에이전트 코드")
```

```
=== 템플릿으로 새 프로젝트 생성 ===
  $ langgraph new my-agent --template new-langgraph-project-python

=== 인터랙티브 메뉴로 생성 ===
  $ langgraph new my-agent

=== 생성 후 의존성 설치 ===
  # pip 사용
  $ cd my-agent && pip install -e .

  # uv 사용
  $ cd my-agent && uv sync

=== 생성되는 파일 구조 ===
  my-agent/
  ├── langgraph.json   # 그래프 설정
  ├── .env.example     # 환경 변수 템플릿
  ├── pyproject.toml   # 의존성 정의
  └── src/
      └── agent.py     # 에이전트 코드
```

11.4 langgraph.json 설정

`langgraph.json` 은 LangGraph 프로젝트의 핵심 설정 파일입니다. 그래프 정의 위치, 의존성, 환경 변수 파일 등을 지정합니다.

```python
import json

# langgraph.json 설정 예시
config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./src/agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("주요 필드:")
print('  dependencies: 설치할 패키지 경로 목록')
print('  graphs:       그래프 이름 → 모듈:변수 매핑')
print('  env:          환경 변수 파일 경로')
```

```
langgraph.json 예시:
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./src/agent.py:graph"
  },
  "env": ".env"
}

주요 필드:
  dependencies: 설치할 패키지 경로 목록
  graphs:       그래프 이름 → 모듈:변수 매핑
  env:          환경 변수 파일 경로
```

11.5 개발 서버 실행

`langgraph dev` 명령으로 로컬 개발 서버를 실행합니다.

```
$ langgraph dev
```

예상 출력:

```
Ready!
- API: http://127.0.0.1:2024
- Docs: http://127.0.0.1:2024/docs
- Studio: https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024
```

URL	용도
`http://127.0.0.1:2024`	API 엔드포인트
`http://127.0.0.1:2024/docs`	API 문서 (Swagger)
`https://smith.langchain.com/studio/...`	LangGraph Studio 인터페이스

> ➡️ TIP
>
> Safari 사용자: `langgraph dev --tunnel` 플래그를 사용하여 localhost 서버에 대한 보안 연결을 설정하세요.

11.6 LangGraph Studio 연동

LangGraph Studio는 `langgraph dev` 실행 시 자동으로 제공되는 시각적 디버깅 도구입니다.

주요 기능:

기능	설명
그래프 시각화	노드와 엣지 구조를 한눈에 파악
실시간 추적	각 노드의 실행 과정을 실시간으로 관찰
상태 검사	각 단계의 상태(state) 값을 확인·수정
인터랙티브 테스트	입력을 바꿔가며 그래프 실행 테스트

Studio는 브라우저 기반이므로 별도 설치가 필요 없습니다. 커스텀 서버 주소를 사용하는 경우, Studio URL의 `baseUrl` 파라미터를 변경하면 됩니다.

11.7 Python SDK — 비동기 클라이언트

비동기 클라이언트는 `langgraph_sdk.get_client()` 로 생성합니다. `asyncio` 기반으로 동작하며, 스트리밍 응답을 효율적으로 처리합니다.

```
# 비동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())
""")
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

```
from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())

# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.
```

11.8 Python SDK — 동기 클라이언트

동기 클라이언트는 `langgraph_sdk.get_sync_client()` 로 생성합니다. 비동기가 필요 없는 간단한 스크립트나 테스트에 적합합니다.

```
# 동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)
""")
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)

# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.
```

11.9 REST API 호출

LangGraph 로컬 서버는 REST API를 제공합니다. `curl` 이나 HTTP 클라이언트로 직접 호출할 수 있습니다.

```
curl -s --request POST \
    --url "http://localhost:2024/runs/stream" \
    --header 'Content-Type: application/json' \
    --data '{
        "assistant_id": "agent",
        "input": {
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?"
            }]
        },
        "stream_mode": "messages-tuple"
    }'
```

API 문서는 `http://localhost:2024/docs` 에서 확인할 수 있습니다.

11.10 배포 준비 체크리스트

로컬 개발이 완료되면 프로덕션 배포를 준비합니다.

항목	설명	확인
`langgraph.json`	그래프 경로·의존성·환경 변수 설정 완료	☐
`.env` 파일	API 키 등 환경 변수 설정	☐
의존성 정리	`pyproject.toml` 또는 `requirements.txt` 정리	☐
로컬 테스트	`langgraph dev` 로 로컬에서 정상 동작 확인	☐
Studio 확인	LangGraph Studio에서 그래프 구조 확인	☐
SDK 테스트	Python SDK 또는 REST API로 호출 테스트	☐
영속 저장소	프로덕션용 체크포인터(예: PostgresSaver) 설정	☐
관측성	LangSmith 또는 Langfuse 트레이싱 설정	☐

요약

주제	핵심 내용
CLI 설치	`pip install "langgraph-cli[inmem]"` 또는 `uv add`
프로젝트 생성	`langgraph new` 로 템플릿 기반 프로젝트 생성
langgraph.json	그래프 경로, 의존성, 환경 변수를 정의하는 설정 파일

Studio	`langgraph dev` 실행 시 자동 제공되는 시각적 디버깅 도구
SDK 비동기	`get_client()` 로 비동기 스트리밍 호출
SDK 동기	`get_sync_client()` 로 동기 스트리밍 호출
REST API	`curl` 로 `/runs/stream` 엔드포인트 직접 호출

> **REFERENCES**
>
> - Run a Local Server

12 내구성 실행

학습 목표

LEARNING OBJECTIVES

- → 내구성 실행(Durable Execution)의 개념과 필요성을 이해한다
- → 체크포인터와 내구성 실행의 관계를 안다
- → `@entrypoint` + `@task` 로 내구성을 보장하는 방법을 익힌다
- → 내구성 모드(exit, async, sync)의 차이를 이해한다
- → 장애 시나리오에서의 복구 과정을 안다

12.1 환경 설정

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

12.2 내구성 실행 개념

내구성 실행(Durable Execution)이란 프로세스나 워크플로가 핵심 지점에서 진행 상태를 저장하여, 일시 중지 후 나중에 정확히 중단된 위치에서 재개할 수 있는 기법입니다.

왜 필요한가?

시나리오	설명
장애 복구	서버 장애 시 처음부터 다시 실행하지 않고 중단 지점에서 재개
상태 영속	긴 실행 시간의 워크플로 중간 결과를 보존
Human-in-the-loop	사람의 승인을 기다리는 동안 상태를 유지

LangGraph는 checkpointer로 내구성 실행을 지원합니다.

12.3 핵심 요구사항

내구성 실행을 구현하려면 세 가지 요소가 필요합니다:

1. 영속 계층 (Persistence Layer)

체크포인터로 워크플로 진행 상태를 기록합니다. 예: `InMemorySaver` (개발용), `PostgresSaver` (프로덕션용)

1. 스레드 식별자 (Thread ID)

워크플로 인스턴스의 실행 기록을 추적하는 고유 ID입니다. 같은 `thread_id` 를 사용하면 이전 실행을 이어서 재개할 수 있습니다.

1. 태스크 래핑 (Task Wrapping)

비결정적(non-deterministic) 연산과 부수 효과(side-effect) 연산을 태스크로 감싸서 재개 시 재실행을 방지합니다.

12.4 내구성 모드 비교

LangGraph는 성능과 일관성 사이의 균형을 맞추는 세 가지 모드를 제공합니다:

모드	동작	트레이드오프
`"exit"`	완료/에러/인터럽트 시에만 영속화	최고 성능, 중간 복구 불가
`"async"`	다음 단계 실행 중 비동기로 영속화	적절한 균형, 약간의 크래시 위험
`"sync"`	다음 단계 실행 전 동기로 영속화	최대 내구성, 성능 비용

대부분의 사용 사례에서는 기본 모드(`"exit"`)로 충분합니다. 미션 크리티컬한 워크플로에서는 `"sync"` 모드를 고려하세요.

12.5 문제가 있는 코드

부수 효과(API 호출 등)를 태스크로 감싸지 않으면 재개 시 동일한 API 호출이 다시 실행될 수 있습니다.

```
# 문제가 있는 접근 방식: 부수 효과를 직접 호출
print("""# BAD: 부수 효과가 태스크로 감싸지지 않음
def call_api(state: State):
    # 이 API 호출은 재개 시 다시 실행됨!
    result = requests.get(state['url']).text[:100]
    return {"result": result}
""")
print("문제점:")
print("  1. 장애 후 재개 시 API가 다시 호출됨")
print("  2. 비결정적 결과가 달라질 수 있음")
print("  3. 중복 요청으로 부작용 발생 가능")
```

```
# BAD: 부수 효과가 태스크로 감싸지지 않음
def call_api(state: State):
    # 이 API 호출은 재개 시 다시 실행됨!
    result = requests.get(state['url']).text[:100]
    return {"result": result}
```

```
문제점:
  1. 장애 후 재개 시 API가 다시 호출됨
  2. 비결정적 결과가 달라질 수 있음
  3. 중복 요청으로 부작용 발생 가능
```

12.6 @task로 개선

`@task` 데코레이터로 부수 효과를 감싸면 재개 시 이전 결과를 체크포인트에서 복원하여 재실행을 방지합니다.

```python
# 개선된 접근 방식: @task로 부수 효과 래핑
print("""# GOOD: @task로 부수 효과를 감쌈
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # 각 요청이 개별 태스크로 실행됨
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}
""")
print("개선 효과:")
print("  1. 재개 시 체크포인트에서 결과 복원")
print("  2. API 중복 호출 방지")
print("  3. 각 태스크가 독립적으로 추적됨")
```

```
# GOOD: @task로 부수 효과를 감쌈
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # 각 요청이 개별 태스크로 실행됨
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}

개선 효과:
  1. 재개 시 체크포인트에서 결과 복원
  2. API 중복 호출 방지
  3. 각 태스크가 독립적으로 추적됨
```

12.7 Graph API에서의 내구성

StateGraph에 체크포인터를 연결하면 각 노드 실행 후 상태가 자동 저장됩니다.

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class DocState(TypedDict):
    topic: str
    draft: str
    final: str
```

```
def write_draft(state: DocState) -> dict:
    return {"draft": f"Draft about {state['topic']}"}


def finalize(state: DocState) -> dict:
    return {"final": f"Final: {state['draft']}"}


checkpointer = InMemorySaver()

builder = StateGraph(DocState)
builder.add_node("write_draft", write_draft)
builder.add_node("finalize", finalize)
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=checkpointer)

# 실행 (thread_id로 실행 추적)
config = {"configurable": {"thread_id": "doc-1"}}
result = graph.invoke({"topic": "LangGraph"}, config)
print("결과:", result)
```

```
결과: {'topic': 'LangGraph', 'draft': 'Draft about LangGraph', 'final': 'Final: Draft about LangGraph'}
```

12.8 Functional API에서의 내구성

`@entrypoint`와 `@task`를 조합하면 Functional API에서도 내구성을 보장할 수 있습니다.

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"


@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"


func_checkpointer = InMemorySaver()


@entrypoint(checkpointer=func_checkpointer)
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    reviewed = review_draft(draft).result()
    return reviewed


config = {"configurable": {"thread_id": "func-1"}}
result = write_document.invoke("Durable Execution", config)
print("결과:", result)
```

```
결과: Reviewed: Draft about Durable Execution
```

12.9 장애 복구 시나리오

같은 `thread_id` 로 재실행하면 체크포인트에서 이전 상태를 복원하여 이어서 실행합니다.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class PipelineState(TypedDict):
    data: str
    step: int
    result: str


call_count = 0


def step_one(state: PipelineState) -> dict:
    global call_count
    call_count += 1
    print(f"  step_one 실행 (호출 횟수: {call_count})")
    return {"data": state["data"].upper(), "step": 1}


def step_two(state: PipelineState) -> dict:
    print(f"  step_two 실행")
    return {"result": f"Processed: {state['data']}", "step": 2}


recovery_saver = InMemorySaver()

builder = StateGraph(PipelineState)
builder.add_node("step_one", step_one)
builder.add_node("step_two", step_two)
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", END)

pipeline = builder.compile(checkpointer=recovery_saver)

# 첫 번째 실행
config = {"configurable": {"thread_id": "recovery-1"}}
print("=== 첫 번째 실행 ===")
result = pipeline.invoke(
    {"data": "hello", "step": 0, "result": ""},
    config
)
print(f"결과: {result}")

# 체크포인트 확인
print("\n=== 체크포인트에서 상태 복원 ===")
saved = pipeline.get_state(config)
print(f"저장된 상태: {saved.values}")
print(f"step_one 총 호출 횟수: {call_count}")
```

```
=== 첫 번째 실행 ===
  step_one 실행 (호출 횟수: 1)
  step_two 실행
결과: {'data': 'HELLO', 'step': 2, 'result': 'Processed: HELLO'}

=== 체크포인트에서 상태 복원 ===
저장된 상태: {'data': 'HELLO', 'step': 2, 'result': 'Processed: HELLO'}
step_one 총 호출 횟수: 1
```

12.10 재개 시작점

워크플로가 재개될 때 API에 따라 시작점이 다릅니다:

API	재개 시작점	설명
StateGraph	중단된 노드의 시작	해당 노드를 처음부터 다시 실행
서브그래프	부모 노드 → 서브그래프 내 중단 노드	부모 노드에서 시작 후 서브그래프 내 해당 노드로 이동
Functional API	`\@entrypoint` 시작	엔트리포인트에서 시작, `\@task` 결과는 캐시에서 복원

핵심 차이:

- StateGraph: 노드 단위 재개 (중단된 노드만 재실행)
- Functional API: 엔트리포인트부터 재실행하되, 완료된 `@task` 는 캐시 결과 사용

12.11 프로덕션 내구성 패턴

프로덕션 환경에서 내구성을 보장하기 위한 모범 사례:

1. 멱등성(Idempotent) 연산 구현

같은 요청을 여러 번 실행해도 결과가 같도록 설계합니다. 멱등성 키(idempotency key)로 중복 처리를 방지합니다.

1. 부수 효과 분리

API 호출, 파일 쓰기 등의 부수 효과를 개별 `@task` 로 분리합니다. 순수 로직과 부수 효과를 명확히 구분합니다.

1. 비결정적 코드 래핑

난수 생성, 타임스탬프 등 비결정적 연산도 `@task` 로 감쌉니다.

1. 영속 저장소 사용

개발: `InMemorySaver` 프로덕션: `PostgresSaver` 또는 외부 데이터베이스

1. 스레드 ID 관리

각 워크플로 인스턴스에 고유한 `thread_id` 를 부여합니다. 장애 복구 시 동일한 `thread_id` 로 재개합니다.

12.12 LangGraph 1.2 Fault Tolerance

내구성 실행은 checkpoint를 통해 재개 지점을 보존합니다. LangGraph 1.2의 fault tolerance는 여기에 노드 시도 단위 제어를 추가합니다.

기능	핵심 API	언제 쓰나
노드 timeout	`TimeoutPolicy` , `NodeTimeoutError`	외부 API가 오래 멈추는 경우

재시도	`RetryPolicy`	일시적 네트워크/5xx 실패
보정 handler	`error_handler=...` , `NodeError` , `Command`	Saga/보상 트랜잭션
Graceful shutdown	`RunControl.request_drain()` , `GraphDrained`	배포·점검 전 안전 중단

```
# LangGraph 1.2 fault tolerance 패턴
from importlib.metadata import version

print("설치된 langgraph:", version("langgraph"))
print("필요 버전: langgraph>=1.2.0")

example = r'''
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy, TimeoutPolicy
from langgraph.runtime import Runtime  # heartbeat 사용 시

def payment_error_handler(state: State, error: NodeError) -> Command:
    # retry 모두 소진 후 실행되는 보정 함수
    return Command(
        update={"status": f"compensated: {error.error}"},
        goto="finalize",
    )

builder.add_node(
    "charge_payment",
    charge_payment,
    timeout=TimeoutPolicy(idle_timeout=30),                    # NodeTimeoutError
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
    error_handler=payment_error_handler,                       # 보정 트랜잭션
)

# 장기 작업 노드: heartbeat로 idle timeout 갱신
async def long_running_node(state: State, runtime: Runtime) -> dict:
    for batch in fetch_batches():
        process(batch)
        runtime.heartbeat()
    return {"result": "done"}

builder.add_node(
    "long_running_node",
    long_running_node,
    timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
'''
print(example)
```

Graceful shutdown — `RunControl` / `GraphDrained` / `Runtime.drain_requested`

장기 실행 그래프를 즉시 강제 종료하지 않고 현재 superstep을 끝낸 뒤 checkpoint를 남기려면 `RunControl.request_drain()` 을 사용합니다. drain이 요청되면 런은 `GraphDrained` 로 멈추고 같은 config로 나중에 재개할 수 있습니다.

노드 내부에서는 `Runtime.drain_requested` 로 drain 신호를 직접 확인할 수 있습니다.

```
# Graceful shutdown 패턴 — langgraph>=1.2
example = r'''
from langgraph.runtime import RunControl, Runtime
from langgraph.errors import GraphDrained

# 1) 호출 측: 외부 시그널(SIGTERM 등)에 drain 요청
control = RunControl()
control.request_drain("sigterm")

try:
```

```
    result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
    print(f"Drained: {e.reason}")
    # checkpoint는 이미 저장됨 — 같은 config로 재개 가능

# 같은 config로 재개
graph.invoke(None, config)

# 2) 노드 내부: drain 신호를 직접 감지
async def my_node(state: State, runtime: Runtime) -> dict:
    if runtime.drain_requested:
        return {"status": "skipped"}
    # ... 일반 처리
    return {"status": "done"}
'''
print(example)
```

12.13 DeltaChannel — checkpoint 저장량 최적화

기본 checkpoint는 superstep마다 채널 값을 전체 저장합니다. 메시지 목록처럼 계속 커지는 append-heavy 채널은 장기 thread에서 저장량이 크게 늘어납니다. LangGraph 1.2의 `DeltaChannel`은 누적 전체값 대신 delta를 저장해 checkpoint 비용을 낮춥니다.

- `snapshot_frequency=K`로 K step마다 전체 snapshot을 남겨 읽기 지연을 제한합니다.
- beta 기능이므로 프로덕션 적용 전 저장량, 읽기 지연, 마이그레이션 비용을 측정합니다.
- 내구성 실행 설계에서는 `DeltaChannel`을 저장 최적화, `TimeoutPolicy` / `error_handler`를 실패 복구 제어로 분리해 이해합니다.

```
# DeltaChannel 개념 예시 — 실제 실행은 langgraph>=1.2에서 확인하세요.
example = r'''
from typing_extensions import Annotated, TypedDict
from langgraph.channels.delta import DeltaChannel

def append(state: list[str], writes: list[list[str]]) -> list[str]:
    return state + [item for batch in writes for item in batch]

class State(TypedDict):
    # append-heavy 메시지/로그 채널에서 checkpoint delta 저장을 고려
    items: Annotated[list[str], DeltaChannel(reducer=append, snapshot_frequency=50)]
'''
print(example)
```

요약

주제	핵심 내용
내구성 개념	중단 지점에서 재개할 수 있는 실행 기법
핵심 요구사항	영속 계층 + 스레드 ID + 태스크 래핑
내구성 모드	exit(기본), async(균형), sync(최대 내구성)
@task	부수 효과를 감싸서 재실행 방지

Graph API	`checkpointer` 연결로 노드별 자동 저장
Functional API	`\@entrypoint` + `\@task` 로 내구성 보장
장애 복구	같은 `thread_id` 로 체크포인트에서 재개

> **REFERENCES**
>
> - Durable Execution
> - Fault Tolerance

13 API 선택 가이드와 Pregel

학습 목표

LEARNING OBJECTIVES

- → Graph API와 Functional API의 차이를 비교한다
- → 동일한 에이전트를 두 API로 구현한다
- → Pregel 런타임의 내부 구조를 이해한다
- → 슈퍼스텝 실행 모델을 안다
- → 프로젝트에 맞는 API를 선택하는 기준을 세운다

13.1 환경 설정

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

13.2 Graph API vs Functional API 개요

LangGraph는 에이전트 워크플로를 구축하기 위한 두 가지 API를 제공합니다. 두 API는 동일한 Pregel 런타임 위에서 실행되며, 하나의 애플리케이션에서 함께 사용할 수 있습니다.

특성	Graph API	Functional API
추상화 방식	노드와 엣지로 구성된 그래프	데코레이터 기반 함수
상태 관리	TypedDict 스키마로 명시적 관리	함수 스코프 내 로컬 변수
제어 흐름	조건부 엣지, 라우팅	일반 Python 제어문 (if/else, for)
시각화	그래프 구조 자동 시각화	제한적

보일러플레이트	상대적으로 많음	최소화

13.3 빠른 선택 가이드

상황	추천 API	이유
복잡한 워크플로 시각화 필요	Graph API	노드/엣지 구조가 자동으로 다이어그램 생성
병렬 실행 경로	Graph API	여러 노드가 자연스럽게 병렬 실행
멀티 에이전트 팀	Graph API	에이전트 간 명확한 역할 분리
기존 코드에 최소 변경	Functional API	데코레이터만 추가하면 됨
단순 선형 워크플로	Functional API	보일러플레이트 없이 빠르게 구현
빠른 프로토타이핑	Functional API	상태 스키마 정의 불필요

13.4 Graph API 구현

에세이 작성 → 점수 매기기 워크플로를 Graph API로 구현합니다.

```
from typing import TypedDict
from langgraph.constants import START
from langgraph.graph import StateGraph


class Essay(TypedDict):
    topic: str
    content: str | None
    score: float | None


def write_essay(essay: Essay):
    return {"content": f"Essay about {essay['topic']}"}


def score_essay(essay: Essay):
    return {"score": 10}


builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

graph_app = builder.compile()

result = graph_app.invoke({"topic": "LangGraph"})
print("Graph API 결과:", result)
```

```
Graph API 결과: {'topic': 'LangGraph', 'content': 'Essay about LangGraph', 'score': 10}
```

13.5 Functional API 구현

동일한 에세이 워크플로를 Functional API로 구현합니다.

```
from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


class EssayResult(TypedDict):
    topic: str
    content: str | None
    score: float | None


@task
def write_essay_func(topic: str) -> str:
    return f"Essay about {topic}"


@task
def score_essay_func(content: str) -> float:
    return 10


func_saver = InMemorySaver()


@entrypoint(checkpointer=func_saver)
def essay_pipeline(topic: str) -> dict:
    content = write_essay_func(topic).result()
    score = score_essay_func(content).result()
    return {"topic": topic, "content": content, "score": score}


config = {"configurable": {"thread_id": "essay-1"}}
result = essay_pipeline.invoke("LangGraph", config)
print("Functional API 결과:", result)
```

```
Functional API 결과: {'topic': 'LangGraph', 'content': 'Essay about LangGraph', 'score': 10}
```

13.6 비교 분석

위 두 구현을 나란히 비교합니다:

항목	Graph API	Functional API
상태 정의	`TypedDict` 스키마 필수	선택적 (로컬 변수 가능)
노드 연결	`add_edge()`, `add_conditional_edges()`	일반 함수 호출
체크포인터	`compile(checkpointer=...)`	`\@entrypoint(checkpointer=...)`
코드 라인 수	상대적으로 많음	간결함
시각화	`graph.get_graph().draw_mermaid()` 지원	제한적
병렬 실행	엣지 구조로 자연스럽게 지원	`\@task` 병렬 실행으로 지원

디버깅	Studio에서 노드별 상태 확인	함수 단위 트레이싱

13.7 두 API 결합

하나의 애플리케이션에서 두 API를 함께 사용할 수 있습니다. 복잡한 멀티 에이전트 조정은 Graph API로, 단순한 데이터 파이프라인은 Functional API로 처리하는 패턴이 일반적입니다.

```python
# Graph API: 복잡한 멀티 에이전트 조정
coordinator = StateGraph(CoordinatorState)
coordinator.add_node("planner", planner_agent)
coordinator.add_node("executor", executor_agent)
coordinator.add_node("reviewer", reviewer_agent)
# ...

# Functional API: 단순 데이터 처리
@entrypoint(checkpointer=saver)
def preprocess(data: str) -> str:
    cleaned = clean_data(data).result()
    validated = validate_data(cleaned).result()
    return validated
```

복잡도가 높아지면 Functional에서 Graph로, 과도하게 설계된 경우 Graph에서 Functional로 마이그레이션 할 수 있습니다.

13.8 Pregel 런타임 개요

Pregel은 LangGraph의 내부 실행 엔진입니다. `StateGraph`를 컴파일하거나 `@entrypoint`를 사용하면 내부적으로 Pregel 인스턴스가 생성됩니다.

이름은 Google의 Pregel 알고리즘에서 따왔으며, 대규모 병렬 그래프 연산을 효율적으로 처리합니다.

핵심 구성 요소:

구성 요소	역할
액터(Actor)	채널에서 데이터를 읽고 처리 결과를 채널에 씀
채널(Channel)	액터 간 데이터 통신을 담당

실행 3단계 (매 스텝마다):

1. Plan — 이번 스텝에서 실행할 액터를 결정
2. Execute — 선택된 액터를 병렬로 실행 (완료, 실패, 또는 타임아웃까지)
3. Update — 새로운 값으로 채널을 갱신

실행할 액터가 없거나 최대 스텝에 도달하면 종료됩니다.

13.9 Pregel 직접 사용

일반적으로 Pregel을 직접 사용할 필요는 없지만, 내부 동작 원리를 이해하기 위해 간단한 예제를 살펴봅니다.

```python
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

# 단일 노드: 입력을 두 번 반복
node1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

result = app.invoke({"a": "foo"})
print("Pregel 결과:", result)
# 'foo' + 'foo' = 'foofoo'
```

```
Pregel 결과: {'b': 'foofoo'}
```

13.10 채널 타입

Pregel은 5가지 채널 타입을 제공합니다.

Type	Purpose
`LastValue`	가장 최근 값을 저장. 입력/출력에 적합한 기본 채널
`EphemeralValue`	실행 step 내부에서만 유효한 임시 값
`Topic`	설정 가능한 PubSub. 다중 값 + optional 중복 제거 / 누적
`BinaryOperatorAggregate`	현재 값과 업데이트에 binary operator 적용(러닝 토탈 등)
`DeltaChannel` (beta, `langgraph\ ⩾ 1.2`)	step별 incremental delta만 저장. 자주 쓰이고 빠르게 자라는 채널(메시지 리스트 등)에 적합

```python
from langgraph.channels import (
    EphemeralValue,
    LastValue,
    Topic,
    BinaryOperatorAggregate,
)
from langgraph.pregel import Pregel, NodeBuilder

# --- 1. LastValue: 최신 값만 유지 ---
node_lv = (
    NodeBuilder()
    .subscribe_only("input")
    .do(lambda x: x.upper())
    .write_to("output")
)
```

```
app_lv = Pregel(
    nodes={"node": node_lv},
    channels={
        "input": EphemeralValue(str),
        "output": LastValue(str),
    },
    input_channels=["input"],
    output_channels=["output"],
)
print("LastValue:", app_lv.invoke({"input": "hello"}))

# --- 2. Topic: 여러 값을 누적 ---
node_t1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_t2 = (
    NodeBuilder()
    .subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app_topic = Pregel(
    nodes={"node1": node_t1, "node2": node_t2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("Topic:", app_topic.invoke({"a": "foo"}))

# --- 3. BinaryOperatorAggregate: 리듀서 적용 ---
def reducer(current, update):
    if current:
        return current + " | " + update
    return update

node_b1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_b2 = (
    NodeBuilder()
    .subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app_agg = Pregel(
    nodes={"node1": node_b1, "node2": node_b2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("BinaryOperatorAggregate:", app_agg.invoke({"a": "foo"}))
```

```
LastValue: {'output': 'HELLO'}
Topic: {'c': ['foofoo', 'foofoofoofoo']}
BinaryOperatorAggregate: {'c': 'foofoo | foofoofoofoo'}
```

DeltaChannel (beta, `langgraph\ ⩾ 1.2`)

`DeltaChannel`은 매 step의 full 누적값이 아니라 incremental delta만 저장합니다. 메시지 리스트처럼 자주 쓰이고 무한히 자라는 채널의 checkpoint 비용을 줄이는 데 사용합니다.

- Reducer는 associative해야 하며 write 시점이 아니라 reconstruction 시점에 실행됩니다.
- `snapshot_frequency=K`로 K step마다 full snapshot을 한 번 기록해 reconstruction 비용을 제한합니다.
- Bulk reducer 시그니처는 `(current_state, sequence_of_writes) → new_state` 입니다.

```python
# DeltaChannel 개념 예시 — 실제 실행은 langgraph>=1.2에서 확인하세요.
example = r'''
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) -> list:
    # (current_state, sequence_of_writes) -> new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
'''
print(example)
```

13.11 슈퍼스텝 실행 모델

Pregel은 슈퍼스텝(Super-step) 단위로 실행됩니다. 각 슈퍼스텝에서 동일 레벨의 노드(액터)가 병렬로 실행되고, 모든 노드가 완료되면 채널이 업데이트된 후 다음 슈퍼스텝으로 넘어갑니다.

```
[슈퍼스텝 1] Node A, Node B  (병렬 실행)
     ↓ 채널 업데이트
[슈퍼스텝 2] Node C  (A, B 결과 기반)
     ↓ 채널 업데이트
[슈퍼스텝 3] Node D
     ↓
END
```

슈퍼스텝의 특징:

- 같은 슈퍼스텝 내 노드는 서로의 출력을 볼 수 없음 (이전 스텝의 채널 값만 참조)
- 모든 노드가 완료되어야 다음 스텝 진행
- 체크포인터가 설정된 경우, 각 슈퍼스텝 후 상태 저장
- 실행할 노드가 없으면 자동 종료

13.12 API 선택 기준 가이드

최종 선택을 위한 의사결정 프레임워크입니다:

1단계: 복잡도 평가
- 노드가 3개 이하이고 선형 흐름 → Functional API
- 조건 분기, 병렬 경로, 순환 구조 → Graph API

2단계: 팀 협업
- 혼자 또는 소규모 팀 → 어느 쪽이든 가능
- 여러 팀원이 각 노드를 담당 → Graph API (시각화와 역할 분리)

3단계: 기존 코드 활용
- 기존 절차적 코드에 LangGraph 기능 추가 → Functional API
- 새로운 워크플로를 처음부터 설계 → Graph API

4단계: 발전 가능성
- 프로토타입에서 시작 → Functional API → 복잡해지면 Graph API로 마이그레이션
- 처음부터 확장성 고려 → Graph API

요약

주제	핵심 내용
Graph API	노드/엣지 기반, 시각화 강점, 복잡한 워크플로에 적합
Functional API	데코레이터 기반, 최소 보일러플레이트, 선형 워크플로에 적합
비교	동일 런타임, 함께 사용 가능, 복잡도에 따라 선택
Pregel	LangGraph의 내부 실행 엔진, 액터-채널 모델
채널	LastValue, EphemeralValue, Topic, BinaryOperatorAggregate, DeltaChannel(beta)
슈퍼스텝	동일 레벨 노드 병렬 실행 → 채널 업데이트 → 다음 스텝
선택 기준	복잡도, 팀 협업, 기존 코드, 발전 가능성으로 판단

> REFERENCES
>
> - Choosing between Graph and Functional APIs
> - LangGraph Runtime (Pregel)

14 Fault Tolerance

RetryPolicy, Timeout, Error Handler

학습 목표

LEARNING OBJECTIVES

- → LangGraph의 재시도, 타임아웃, 에러 핸들러를 구분한다
- → `RetryPolicy` 로 일시적 장애를 자동 재시도한다
- → async node timeout과 `NodeTimeoutError` 를 안전하게 관찰한다
- → `error_handler` 와 `Command` 로 실패 후 보정 경로를 구성한다
- → `set_node_defaults()` 와 노드별 설정의 우선순위를 이해한다

개요

기능	핵심 API	사용 상황
Retries	`RetryPolicy`	네트워크/Rate limit/일시적 5xx
Timeouts	`timeout=...` , `NodeTimeoutError`	외부 호출이 오래 멈추는 경우
Error handling	`error_handler=...` , `Command`	실패 후 보정·fallback 라우팅
Graph defaults	`set_node_defaults(...)`	여러 노드에 공통 정책 적용

이 장은 외부 API를 실제로 느리게 만들지 않고, deterministic fake node로 장애를 재현합니다.

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

```
from langgraph.types import RetryPolicy, Command
from langgraph.errors import NodeError, NodeTimeoutError
```

1) 재시도 — transient failure를 시스템이 흡수

실패해도 같은 입력으로 다시 시도해도 안전한 작업에는 `RetryPolicy` 를 붙입니다. 아래 node는 첫 호출에서만 실패하고 두 번째 호출에서 성공합니다.

```
class FaultState(TypedDict):
    result: str
    attempts: int
    error: str

calls = {"n": 0}

def flaky_api(state: FaultState) -> dict:
    calls["n"] += 1
    if calls["n"] < 2:
        raise ValueError("temporary network error")
    return {"result": "ok", "attempts": calls["n"]}
```

```
retry_graph = (
    StateGraph(FaultState)
    .add_node("flaky_api", flaky_api, retry_policy=RetryPolicy(
        max_attempts=3, initial_interval=0.01, jitter=False, retry_on=ValueError,
    ))
    .add_edge(START, "flaky_api")
    .add_edge("flaky_api", END)
    .compile()
)
```

```
retry_result = retry_graph.invoke({"result": "", "attempts": 0, "error": ""})
print(retry_result)
assert retry_result["attempts"] == 2
```

2) 타임아웃 — 오래 멈춘 async node 중단

LangGraph의 node timeout은 async node에서 안전하게 동작합니다. sync Python 함수는 같은 프로세스에서 강제 취소할 수 없으므로 timeout 대상이 아닙니다.

```
import asyncio

async def slow_api(state: FaultState) -> dict:
    await asyncio.sleep(0.05)
    return {"result": "too late"}

timeout_graph = (
    StateGraph(FaultState)
    .add_node("slow_api", slow_api, timeout=0.01)
    .add_edge(START, "slow_api")
    .add_edge("slow_api", END)
    .compile()
)
```

```
try:
    await timeout_graph.ainvoke({"result": "", "attempts": 0, "error": ""})
except NodeTimeoutError as exc:
    print(type(exc).__name__, str(exc).split("(")[0].strip())
```

3) 에러 핸들러 — 실패를 fallback 경로로 라우팅

재시도를 모두 소진했거나, 재시도하면 안 되는 실패라면 `error_handler`가 보정 경로를 만들 수 있습니다. 핸들러는 `Command(update=..., goto=...)`로 상태 업데이트와 다음 노드를 동시에 지정합니다.

```
def broken_service(state: FaultState) -> dict:
    raise RuntimeError("downstream unavailable")

def recover(state: FaultState, error: NodeError) -> Command:
    return Command(
        update={"error": str(error.error), "result": "fallback"},
        goto="finalize",
    )

def finalize(state: FaultState) -> dict:
    return {"result": state["result"] + " | finalized"}
```

```
handler_graph = (
    StateGraph(FaultState)
    .add_node("broken_service", broken_service, error_handler=recover)
    .add_node("finalize", finalize)
    .add_edge(START, "broken_service")
    .add_edge("finalize", END)
    .compile()
)
handler_result = handler_graph.invoke({"result": "", "attempts": 0, "error": ""})
print(handler_result)
```

4) Graph defaults — 공통 정책과 노드별 override

동일한 retry/timeout 정책을 여러 노드에 붙일 때는 `set_node_defaults()`를 먼저 선언합니다. 특정 노드에 별도 정책을 주면 노드 설정이 우선합니다.

```
def stable_node(state: FaultState) -> dict:
    return {"result": "stable"}

common_retry = RetryPolicy(
    max_attempts=2, initial_interval=0.01, jitter=False, retry_on=ValueError,
)
default_graph = (
    StateGraph(FaultState)
    .set_node_defaults(retry_policy=common_retry)
    .add_node("stable", stable_node)
    .add_edge(START, "stable")
    .add_edge("stable", END)
    .compile()
)
print(default_graph.invoke({"result": "", "attempts": 0, "error": ""}))
```

운영 체크리스트

질문	판단 기준
재시도해도 안전한가?	멱등성, 중복 결제/중복 쓰기 위험 확인
얼마나 기다릴 것인가?	사용자 SLA, 외부 API p95, graph 전체 timeout
실패 후 보정이 필요한가?	취소, 환불, fallback, 사용자 안내
어디까지 기본 정책으로 둘 것인가?	전역 defaults vs 노드별 override

REFERENCES

- LangGraph Fault Tolerance: https://docs.langchain.com/oss/python/langgraph/fault-tolerance
- LangGraph Durable Execution: ../docs/langgraph/06-durable-execution.md
- LangGraph Graph API: ../docs/langgraph/19-graph-api.md

15 Backward Compatibility

LangGraph 버전 변화에 안전하게 대응하기

학습 목표

LEARNING OBJECTIVES

- → LangGraph API 변경을 노트북 코드에 안전하게 반영하는 방법을 익힙니다.
- → runtime feature detection과 compatibility checklist를 만듭니다.
- → migration note를 테스트와 함께 관리하는 방식을 확인합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
import inspect
from langgraph.graph import StateGraph
from langgraph.types import RetryPolicy

features = {
    "add_node_retry_policy": "retry_policy" in inspect.signature(StateGraph.add_node).parameters,
    "retry_policy_class": RetryPolicy.__name__,
}
features
```

15.1 호환성은 import 성공만으로 판단하지 않습니다

API가 존재해도 parameter, return type, behavior가 바뀔 수 있으므로 signature와 smoke test를 함께 봅니다.

```
def require_feature(name: str, ok: bool) -> str:
    if not ok:
        return f"SKIP: {name} is not available"
    return f"OK: {name}"

require_feature("add_node.retry_policy", features["add_node_retry_policy"])
```

15.2 migration checklist

버전 업데이트 PR에는 코드 변경보다 먼저 이 체크리스트를 붙입니다.

```python
migration_checklist = [
    "공식 changelog 확인",
    "기존 notebook smoke 실행",
    "deprecated import 검색",
    "새 API 예제 1개 추가",
]

for item in migration_checklist:
    print("-", item)
```

15.3 compatibility matrix

영역	확인 방법
import path	`python -c` 또는 notebook smoke
graph compile	최소 StateGraph compile
persistence	checkpointer thread_id smoke
streaming	event/stream mode smoke

정리

항목	내용
다룬 기술	feature detection, migration checklist, compatibility matrix
핵심 개념	버전 호환성은 문서 확인 + 작은 smoke test + migration note로 관리합니다.

REFERENCES

- `docs/langgraph/backward-compatibility.md`
- `docs/langgraph/changelog-py.md`
- `docs/langgraph/fault-tolerance.md`

16 Case Studies

공식 사례를 설계 패턴으로 읽기

학습 목표

LEARNING OBJECTIVES

- → LangGraph 공식 case study를 구현 복붙이 아니라 architecture pattern으로 분해합니다.
- → 문제, state, nodes, persistence, evaluation 관점의 리뷰 템플릿을 만듭니다.
- → 새 agent 프로젝트 시작 전에 case study checklist를 적용합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

16.1 case study 리뷰 템플릿

공식 사례는 “무엇을 만들었나”보다 “왜 graph가 필요했나”를 중심으로 읽습니다.

```
review_template = {
    "problem": "어떤 반복/분기/상태 문제가 있었나?",
    "state": "장기적으로 보존해야 할 state는 무엇인가?",
    "nodes": "결정적 node와 agentic node는 어떻게 나뉘나?",
    "eval": "성공 기준은 무엇인가?",
}

review_template
```

16.2 샘플 사례 분해

아래는 고객지원 그래프를 case study 형식으로 요약한 예시입니다.

```
case = {
    "problem": "문의 유형에 따라 billing/technical/general 경로가 달라진다.",
    "state": ["message", "owner", "resolution"],
```

```
    "nodes": ["triage", "resolve", "approval"],
    "eval": "owner routing accuracy and resolution completeness",
}

case
```

16.3 설계 결정 기록

case study를 읽고 내 프로젝트에 적용할 때는 decision record를 남깁니다.

```
decision = {
    "chosen": "StateGraph",
    "because": "handoff state and approval gates must be explicit",
    "rejected": "single create_agent loop",
    "risk": "routing policy drift",
}

decision
```

정리

항목	내용
다룬 기술	case study review, architecture template, decision record
핵심 개념	사례는 기능 목록이 아니라 설계 선택의 근거로 읽어야 합니다.

REFERENCES

- `docs/langgraph/case-studies.md`
- `docs/langgraph/thinking-in-langgraph.md`
- `docs/langgraph/workflows-agents.md`

PART

IV

Deep Agents

Framework-Agnostic Agent Building

이 파트에서 다루는 내용

- 1. 소개
- 2. 퀵스타트
- 3. 커스터마이징
- 4. 백엔드
- 5. 서브에이전트
- 6. 메모리와 스킬
- 7. 고급 기능
- 8. AgentHarness
- 9. 프레임워크 비교
- 10. 샌드박스와 ACP
- 11. 비동기 서브에이전트
- 12. 프로덕션 준비
- 13. 컨텍스트 엔지니어링
- 14. 스트리밍

- 15. 권한 관리
- 16. Models and Tools
- 17. Programmatic Subagents
- 18. Event Streaming
- 19. Permissions Practice
- 20. Quality Profiles and Rubrics

01 Deep Agents 소개

학습 목표

LEARNING OBJECTIVES

- → Deep Agents가 무엇인지 이해한다
- → SDK와 Deep Agents Code(`dcode`)의 차이를 파악한다
- → 핵심 개념 6가지(Planning, Context Management, Backends, Subagents, Memory, Quality Gates)를 이해한다
- → 다른 프레임워크와의 차이를 비교한다
- → 설치 상태를 확인한다

1. Deep Agents란?

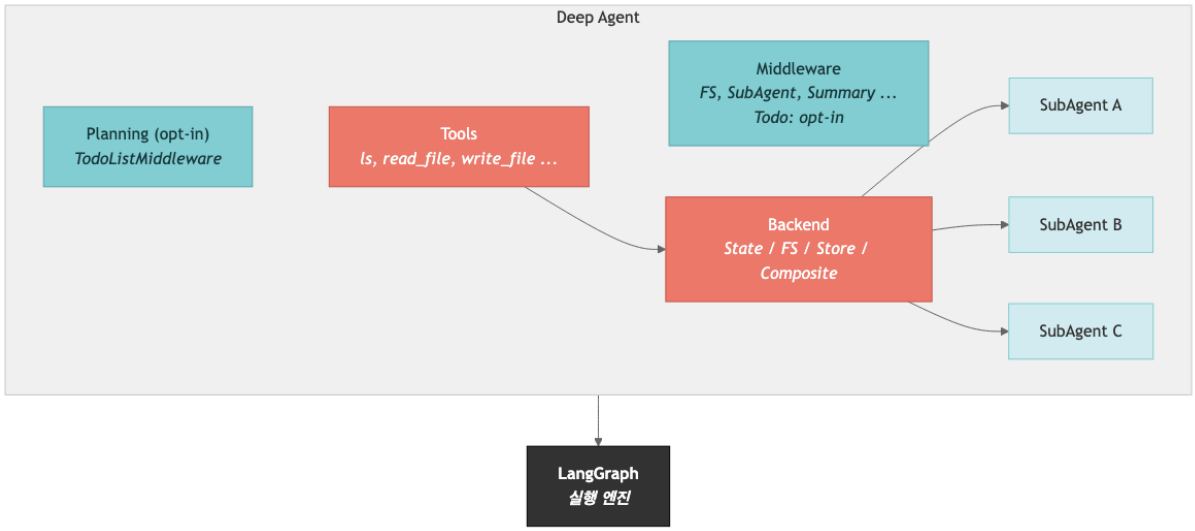
Deep Agents는 LangChain 팀이 만든 에이전트 하네스(Agent Harness) 프레임워크입니다. 복잡한 멀티 스텝 작업을 수행하는 자율 에이전트를 쉽게 구축할 수 있도록, 아래 기능들을 조립합니다:

- 컨텍스트 관리 — 파일 시스템 도구(`ls` , `read_file` , `write_file` , `edit_file` , `delete`)와 요약 압축
- 셸 실행 — 샌드박스 백엔드 격리 또는 제한된 로컬 실행
- JavaScript 인터프리터 — QuickJS 런타임으로 가벼운 도구 조합 (셸/네트워크 차단)
- 플러거블 백엔드 — in-memory state, local disk, LangGraph store, ContextHub, sandbox
- 서브에이전트 위임 — 컨텍스트 격리와 병렬 작업
- 태스크 플래닝(선택) — `TodoListMiddleware` 를 추가해 `write_todos` 노출
- 장기 메모리 — LangGraph Memory Store 기반 스레드 간 지식 유지
- 파일 시스템 권한 — 선언적 read/write/delete 규칙
- Human-in-the-Loop — `interrupt_on` 승인 워크플로
- 재사용 가능한 스킬 — 특화 워크플로와 gated skill tools
- 시스템 프롬프트 커스터마이징 훅

LangChain의 기본 에이전트 컴포넌트 위에 구축되었으며, LangGraph를 실행 엔진(durable execution + streaming)으로 사용합니다.

> ➡️ TIP
>
> 이 교육 자료의 모델 설정: 본 과정에서는 OpenAI `gpt-5.4` 모델을 사용합니다. `OPENAI_API_KEY` 환경 변수를 설정하고, `ChatOpenAI(model="gpt-5.4")` 를 사용합니다. Deep Agents의 모델은 예제에서 명시적으로 지정합니다.

아키텍처 개요



2. Deep Agents 제품군

Deep Agents 생태계는 세 가지 형태로 제공됩니다:

구분	Deep Agents SDK	Deep Agents Code	ACP Integration
패키지	`deepagents`	`deepagents-code` (`dcode`)	ACP 커넥터
용도	프로그래밍 방식으로 에이전트 구축	터미널 코딩 에이전트	Zed 등 코드 에디터 내장
설치	`pip install -qU deepagents langchain-{provider}`	`curl -LsSf https://langch.in/dcode | bash` 또는 `uvx --from deepagents-code dcode --help`	Zed 확장 설치
사용 방식	Python 코드에서 `create_deep_agent()` 호출	터미널에서 `dcode` 실행	에디터의 ACP 클라이언트 호출
커스터마이징	완전한 API 접근 (도구·백엔드·미들웨어)	`~/.deepagents/config.toml`, `AGENTS.md`, `SKILL.md`, slash commands	SDK 그대로 + 에디터 UI

적합한 경우	앱에 에이전트 통합, 자동화 파이프라인	대화형/비대화형 코딩 어시스턴트	에디터 내부 워크플로

`{provider}` 자리에는 `anthropic`, `openai`, `google-genai`, `openrouter`, `fireworks`, `baseten`, `ollama` 중 사용 모델에 맞는 값을 넣습니다.

> ➡️ TIP
>
> 이 교육 자료에서는 SDK를 중심으로 다루고, CLI 사용은 Deep Agents Code(`dcode`) 기준으로 소개합니다.

---

3. 핵심 개념 6가지

3.1 Planning (태스크 플래닝)

계획은 opt-in입니다. LangChain의 `TodoListMiddleware` 를 추가하면 에이전트가 `write_todos` 도구로 복잡한 작업을 구조화된 태스크 리스트로 분해합니다. 각 태스크는 `pending` → `in_progress` → `completed` 상태로 추적됩니다.

3.2 Context Management (컨텍스트 관리)

에이전트가 작업하면서 생성되는 방대한 정보(파일 내용, 검색 결과 등)를 효율적으로 관리합니다:

- 오프로딩: 큰 출력은 파일 시스템 도구로 디스크에 저장하고 포인터만 유지
- 요약: 컨텍스트가 모델 한도에 가까워지면 `SummarizationMiddleware` 가 대화 이력을 압축

3.3 Backends (스토리지 백엔드)

에이전트의 파일 시스템은 플러거블 백엔드로 구현됩니다:

- `StateBackend` — 에이전트 상태에 파일 저장 (스레드 한정, 기본값)
- `FilesystemBackend` — 로컬 디스크 접근 (`virtual_mode=True` 기본)
- `StoreBackend` — 크로스 스레드 영속 저장소 (`namespace` 필수)
- `ContextHubBackend` — LangSmith Hub 저장소
- `CompositeBackend` — 경로별 라우팅
- 샌드박스 — Modal / Daytona / Deno / local VFS

3.4 Subagents (서브에이전트)

메인 에이전트가 전문 서브에이전트에게 작업을 위임합니다. 기본 isolated 모드는 호출마다 독립 문맥을 사용하고, 실험적 `mode="fork"` 는 부모 문맥을 이어받습니다.

3.5 Memory & Skills (장기 메모리·스킬)

- Memory — `StoreBackend` + `AGENTS.md` 로 항상 주입되는 컨벤션
- Skills — `SKILL.md` 기반 progressive disclosure와 `metadata.include_tools`
- JavaScript Interpreter — QuickJS로 도구 조합/중간 상태 유지 (`deepagents ≥ 0.6`)

3.6 Quality Gates (런타임 평가)

`RubricMiddleware`는 criterion별 평가와 grader hook을 사용해 실행 결과를 점검하고, 필요하면 같은 실행 안에서 수정을 유도합니다.

---

4. 다른 프레임워크와의 비교

기능	LangChain Deep Agents	OpenCode	Claude Agent SDK
모델 지원	모델 무관 (Anthropic, OpenAI 등 100+)	75+ 프로바이더 (Ollama 포함)	Claude 전용
라이선스	MIT	MIT	MIT (SDK) / 독점 (Claude Code)
SDK	Python, TypeScript + CLI	터미널, 데스크톱, IDE	Python, TypeScript
샌드박스	도구로 통합 (Modal, Daytona 등)	미지원	미지원
플러거블 백엔드	O (State, FS, Store, Composite)	X	X
타임 트래블	O (LangGraph)	X	O
관측성	LangSmith 네이티브	X	X
파일 도구 기본 내장	O	O	O
Human-in-the-Loop	O (미들웨어)	O	O

> ➡️ TIP
>
> Deep Agents의 핵심 차별점: 플러거블 백엔드, 샌드박스 통합, LangSmith 관측성

---

5. 설치 확인

```
pip install -qU deepagents langchain-openai
# 다른 프로바이더를 쓰려면 langchain-anthropic, langchain-google-genai,
# langchain-openrouter, langchain-fireworks, langchain-baseten, langchain-ollama 중 선택
```

아래 셀을 실행하여 `deepagents` 패키지가 올바르게 설치되었는지 확인합니다.

```
# deepagents 패키지 버전 확인
import deepagents
print(f"deepagents 버전: {deepagents.__version__}")
```

```
deepagents 버전: 0.4.4
```

```
# 주요 모듈 임포트 확인
from deepagents import create_deep_agent, SubAgent, CompiledSubAgent
from deepagents import FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware
from deepagents.backends import StateBackend, FilesystemBackend, StoreBackend, CompositeBackend
from deepagents.backends.protocol import BackendProtocol

print("모든 주요 모듈을 성공적으로 임포트했습니다!")
```

```
모든 주요 모듈을 성공적으로 임포트했습니다!
```

```
# 의존 패키지 버전 확인
import importlib.metadata

print(f"langchain 버전: {importlib.metadata.version('langchain')}")
print(f"langgraph 버전: {importlib.metadata.version('langgraph')}")
```

```
langchain 버전: 1.2.10
langgraph 버전: 1.0.10
```

```
# create_deep_agent 함수 시그니처 확인
import inspect

sig = inspect.signature(create_deep_agent)
print("create_deep_agent() 파라미터:")
for name, param in sig.parameters.items():
    default = param.default if param.default is not inspect.Parameter.empty else "(필수)"
    print(f"  - {name}: {default}")
```

```
create_deep_agent() 파라미터:
  - model: None
  - tools: None
  - system_prompt: None
  - middleware: ()
  - subagents: None
  - skills: None
  - memory: None
  - response_format: None
  - context_schema: None
  - checkpointer: None
  - store: None
  - backend: None
  - interrupt_on: None
  - debug: False
  - name: None
  - cache: None
```

핵심 정리

항목	내용
Deep Agents	LangChain 기반 에이전트 하네스 프레임워크
핵심 함수	`create_deep_agent()`
실행 엔진	LangGraph (`CompiledStateGraph` 반환)
모델 표준	본 교재는 OpenAI `gpt-5.4` 를 명시적으로 지정
모델 접근	`ChatOpenAI(model="gpt-5.4")` 또는 `provider:model-name` 문자열
핵심 개념	Context Management, Backends, Subagents, Memory & Skills, opt-in Planning, Quality Gates
기본 도구	`ls` , `read_file` , `write_file` , `edit_file` , `delete` , `glob` , `grep` , `task`
계획 도구	`TodoListMiddleware()` 를 추가하면 `write_todos` 사용 가능
인터프리터	QuickJS 기반 JavaScript runtime (`deepagents\ ⩾ 0.6` , 셸·네트워크 불가)
품질 게이트	`RubricMiddleware` 로 runtime 평가와 재시도 제어
CLI	Deep Agents Code(`dcode`) — `deepagents-code` 패키지 기반 터미널 에이전트

02 첫 번째 에이전트 만들기

학습 목표

LEARNING OBJECTIVES

- → `.env` 파일에서 API 키를 로드하는 방법을 익힌다
- → `create_deep_agent()` 로 기본 에이전트를 생성한다
- → `agent.invoke()` 와 `agent.stream()` 으로 에이전트를 실행한다
- → Tavily 검색 도구를 연동하는 리서치 에이전트를 만든다
- → 기본 파일/위임 도구와 opt-in 계획 도구의 역할을 이해한다

1. 설치 & API 키 설정

설치

```
# uv 권장
uv init
uv add deepagents tavily-python langchain-openai
uv sync

# 또는 pip
pip install -qU deepagents tavily-python langchain-openai
```

`langchain-openai` 자리는 사용 모델 프로바이더로 바꿉니다 (`langchain-anthropic` , `langchain-google-genai` , `langchain-openrouter`).

API 키

`.env` 파일에 모델 프로바이더 키와 Tavily 키를 설정합니다. 본 노트북은 OpenAI를 기본으로 합니다.

```
# OpenAI (본 노트북 기본)
OPENAI_API_KEY=your-key

# 다른 프로바이더를 쓰면 추가
ANTHROPIC_API_KEY=your-key       # claude-sonnet-4-6
```

```
GOOGLE_API_KEY=your-key          # gemini-3.5-flash
OPENROUTER_API_KEY=your-key      # openrouter:anthropic/claude-sonnet-4-6

# 공통
TAVILY_API_KEY=your-key
```

➡️ TIP

`.env.example` 파일을 복사하여 `.env` 로 만들면 됩니다.

```
# 환경 변수 로드
from dotenv import load_dotenv
import os

load_dotenv()

assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY가 설정되지 않았습니다!"
print("API 키가 정상적으로 로드되었습니다.")
```

```
API 키가 정상적으로 로드되었습니다.
```

---

2. 가장 간단한 에이전트 만들기

`create_deep_agent()` 는 Deep Agents의 핵심 함수입니다. 모델을 명시하고 호출하면 파일 도구와 기본 하네스가 구성됩니다. 계획 도구가 필요한 예제에서는 `TodoListMiddleware()` 를 `middleware` 에 추가합니다.

본 노트북은 OpenAI `gpt-5.4` 를 사용합니다. 모델은 `ChatOpenAI` 객체로 넘기거나, `"provider:model-name"` 문자열로 전달할 수 있습니다.

```
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

# OpenAI gpt-5.4 모델 설정 (Deep Agents 기본은 anthropic:claude-sonnet-4-6)
model = ChatOpenAI(model="gpt-5.4")

# 기본 에이전트 생성
agent = create_deep_agent(model=model)

print(f"에이전트 타입: {type(agent).__name__}")
print("에이전트가 성공적으로 생성되었습니다!")
```

```
에이전트 타입: CompiledStateGraph
에이전트가 성공적으로 생성되었습니다!
```

`create_deep_agent()` 는 LangGraph의 `CompiledStateGraph` 를 반환합니다. LangGraph의 모든 실행 메서드(`invoke` , `stream` , `batch` 등)를 그대로 쓸 수 있습니다.

---

3. 에이전트 실행 — `invoke()`

에이전트에 메시지를 전달하여 실행합니다. 입력 형식은 `{"messages": [{"role": "user", "content": "..."}]}` 딕셔너리입니다.

4. Tavily 검색 도구 연동 — 리서치 에이전트

커스텀 도구를 추가하면 에이전트의 기능을 확장할 수 있습니다. 여기서는 Tavily 웹 검색 도구를 연동합니다.

도구 정의 방법

Python 함수의 docstring이 도구 설명으로, 타입 힌트가 파라미터 스키마로 변환됩니다.

```
from typing import Literal
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])


def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색할 질문 또는 키워드
        max_results: 반환할 최대 결과 수
        topic: 검색 주제 카테고리
        include_raw_content: 원본 콘텐츠 포함 여부
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )


print(f"도구 이름: {internet_search.__name__}")
print(f"도구 설명: {internet_search.__doc__.strip().splitlines()[0]}")
```

```
도구 이름: internet_search
도구 설명: 인터넷에서 정보를 검색합니다.
```

```
# 리서치 에이전트 생성 — 검색 도구 + 커스텀 시스템 프롬프트
research_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 전문 리서처입니다. 사용자의 질문에 대해 인터넷 검색을 수행하고, 결과를 정리하여 한국어로 보고서를 작성합니다.",
)

print("리서치 에이전트가 생성되었습니다!")
```

```
리서치 에이전트가 생성되었습니다!
```

5. 도구 확인

`create_deep_agent()` 가 제공하는 기본 도구와 opt-in 계획 도구입니다:

도구	설명
`ls`	디렉토리 내용 목록 (메타데이터 포함)
`read_file`	파일 읽기 (줄 번호 포함, 이미지 지원)
`write_file`	파일 쓰기 또는 기존 파일 교체
`edit_file`	파일 내 텍스트 교체 (`old_string` → `new_string`)
`delete`	delete 지원 백엔드에서 파일 또는 prefix 삭제
`glob`	패턴 기반 파일 검색 (예: `**/*.py`)
`grep`	파일 내용 검색 (정규식 지원)
`task`	서브에이전트 호출
`write_todos`	`TodoListMiddleware()` 를 추가한 경우 구조화된 태스크 리스트 관리

> ➡️ TIP
>
> 이 도구들은 모두 백엔드(Backend)를 통해 동작합니다. 기본값은 `StateBackend` 로, 에이전트 상태에 파일이 저장됩니다.

6. 스트리밍 출력 — `stream()`

`agent.stream()` 을 사용하면 에이전트의 실행 과정을 실시간으로 볼 수 있습니다. `stream_mode` 에 따라 다른 수준의 정보를 받습니다:

- `"updates"` — 각 단계 완료 시 상태 업데이트
- `"messages"` — 개별 토큰 스트리밍
- `"custom"` — 사용자 정의 이벤트

핵심 정리

항목	내용
에이전트 생성	`create_deep_agent(model, tools, system_prompt)`
동기 실행	`agent.invoke({"messages": [...]})`
스트리밍 실행	`agent.stream({"messages": [...]}, stream_mode="updates")`
커스텀 도구	Python 함수 + docstring + 타입 힌트
모델 객체	`ChatOpenAI(model="gpt-5.4")` 또는 다른 프로바이더
모델 문자열	`provider:model-name` — `anthropic:claude-sonnet-4-6`, `openai:gpt-5.4`, `google_genai:gemini-3.5-flash`, `openrouter:anthropic/claude-sonnet-4-6`
Deep Agents 기본	`anthropic:claude-sonnet-4-6` (모델 미지정 시)

03 에이전트 커스터마이징

학습 목표

LEARNING OBJECTIVES

- → 다양한 LLM 프로바이더와 모델을 선택하는 방법을 익힌다
- → 효과적인 시스템 프롬프트를 작성한다
- → docstring 기반 커스텀 도구를 만든다
- → `response_format` 으로 구조화된 출력(Pydantic)을 생성한다
- → 미들웨어 아키텍처를 이해한다

```python
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")

# OpenAI gpt-5.4 모델 초기화 (Deep Agents 기본은 anthropic:claude-sonnet-4-6)
from deepagents import create_deep_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
print(f"기본 모델: {model.model_name}")
```

```
환경 설정 완료

기본 모델: gpt-4.1
```

1. 모델 선택

Deep Agents는 LangChain ChatModel 객체 또는 `provider:model` 포맷으로 다양한 LLM을 지원합니다. 모델을 지정하지 않으면 기본값은 `anthropic:claude-sonnet-4-6` 입니다.

본 노트북에서는 OpenAI `gpt-5.4` 를 기본 모델로 사용합니다.

프로바이더	모델 예시	환경 변수	비고
OpenAI	`openai:gpt-5.4`	`OPENAI_API_KEY`	본 노트북 기본
Anthropic	`anthropic:claude-sonnet-4-6`	`ANTHROPIC_API_KEY`	Deep Agents 기본
Google	`google_genai:gemini-3.5-flash`	`GOOGLE_API_KEY`	
Azure	`azure_openai:gpt-4o`	`AZURE_OPENAI_*`	엔터프라이즈
AWS Bedrock	`bedrock:anthropic.claude-sonnet-4-6`	AWS 자격 증명	
OpenRouter	`openrouter:anthropic/claude-sonnet-4-6`	`OPENROUTER_API_KEY`	100+ 모델

자동 재시도(기본 6회), `max_retries`, `timeout` 파라미터가 내장되어 있어 네트워크 장애·rate limit·서버 에러를 견딥니다.

```
# OpenAI gpt-5.4 모델 사용 (위에서 초기화한 model 객체)
agent_oai = create_deep_agent(
    model=model,
)

print(f"에이전트 생성 완료: {type(agent_oai).__name__}")

# 참고: 다른 프로바이더를 사용하려면 해당 API 키를 설정하고 아래처럼 호출
# agent_anthropic = create_deep_agent(model="anthropic:claude-sonnet-4-6")  # Deep Agents 기본
# agent_gemini = create_deep_agent(model="google_genai:gemini-3.5-flash")
# agent_openrouter = create_deep_agent(model="openrouter:anthropic/claude-sonnet-4-6")
```

```
에이전트 생성 완료: CompiledStateGraph
```

2. 커스텀 시스템 프롬프트

시스템 프롬프트는 에이전트의 역할, 행동 규칙, 출력 형식을 정의합니다. 기본 프롬프트 위에 추가되므로, 도메인 특화 지침을 작성하면 됩니다.

3. 커스텀 도구 만들기

Python 함수를 도구로 변환하는 규칙:

1. 함수 이름 → 도구 이름
2. docstring → 도구 설명 (에이전트가 도구 선택 시 참조)
3. 타입 힌트 → 파라미터 스키마 (자동 생성)
4. 기본값 → 선택적 파라미터

```
import math
```

```python
def calculate_compound_interest(
    principal: float,
    annual_rate: float,
    years: int,
    compounds_per_year: int = 12,
) -> dict:
    """복리 이자를 계산합니다.

    Args:
        principal: 원금 (원)
        annual_rate: 연이율 (예: 0.05 = 5%)
        years: 투자 기간 (년)
        compounds_per_year: 연간 복리 횟수 (기본: 12 = 월복리)
    """
    amount = principal * (1 + annual_rate / compounds_per_year) ** (compounds_per_year * years)
    interest = amount - principal
    return {
        "원금": f"{principal:,.0f}원",
        "최종 금액": f"{amount:,.0f}원",
        "이자 수익": f"{interest:,.0f}원",
        "수익률": f"{(interest / principal) * 100:.2f}%",
    }


def convert_temperature(
    value: float,
    from_unit: str,
    to_unit: str,
) -> str:
    """온도 단위를 변환합니다.

    Args:
        value: 변환할 온도 값
        from_unit: 원래 단위 ('celsius', 'fahrenheit', 'kelvin')
        to_unit: 변환할 단위 ('celsius', 'fahrenheit', 'kelvin')
    """
    # 먼저 섭씨로 변환
    if from_unit == "fahrenheit":
        celsius = (value - 32) * 5 / 9
    elif from_unit == "kelvin":
        celsius = value - 273.15
    else:
        celsius = value

    # 목표 단위로 변환
    if to_unit == "fahrenheit":
        result = celsius * 9 / 5 + 32
    elif to_unit == "kelvin":
        result = celsius + 273.15
    else:
        result = celsius

    return f"{value} {from_unit} = {result:.2f} {to_unit}"


# 커스텀 도구를 사용하는 에이전트 생성
calculator_agent = create_deep_agent(
    model=model,
    tools=[calculate_compound_interest, convert_temperature],
    system_prompt="당신은 계산과 단위 변환을 도와주는 어시스턴트입니다. 항상 도구를 사용하여 정확한 계산을 수행하세
요.",
)

print("계산 에이전트가 생성되었습니다!")
```

```
계산 에이전트가 생성되었습니다!
```

4. 구조화된 출력 — `response_format`

에이전트의 최종 응답을 Pydantic 모델로 구조화할 수 있습니다. 프로그래밍으로 처리하기 쉬운 형태의 출력을 받을 수 있습니다.

> · NOTE
>
> `create_deep_agent()` 풀 시그니처는 17개 파라미터를 받습니다 (참고용): `model` , `tools` , `system_prompt` , `middleware` , `subagents` , `skills` , `memory` , `permissions` , `backend` , `interrupt_on` , `response_format` , `context_schema` , `checkpointer` , `store` , `debug` , `name` , `cache` . `permissions` / `context_schema` / `checkpointer` / `store` / `cache` 는 17번 advanced 노트북과 13번 production 가이드에서 자세히 다룹니다.

```python
from pydantic import BaseModel, Field


# 구조화된 출력 스키마 정의
class BookRecommendation(BaseModel):
    """도서 추천 결과"""
    title: str = Field(description="책 제목")
    author: str = Field(description="저자")
    reason: str = Field(description="추천 이유 (2~3문장)")
    difficulty: str = Field(description="난이도: 초급/중급/고급")


class BookRecommendationList(BaseModel):
    """도서 추천 목록"""
    topic: str = Field(description="추천 주제")
    books: list[BookRecommendation] = Field(description="추천 도서 목록")


# response_format을 사용하는 에이전트
book_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 도서 추천 전문가입니다. 사용자의 관심 분야에 맞는 책을 추천합니다.",
    response_format=BookRecommendationList,
)

print("도서 추천 에이전트 생성 완료")
```

```
도서 추천 에이전트 생성 완료
```

---

5. 미들웨어 아키텍처

`create_deep_agent()` 는 내부적으로 미들웨어 스택을 구성합니다. 미들웨어는 에이전트의 동작을 확장하는 플러그인 레이어입니다.

시스템 프롬프트 조립 순서

```
USER (사용자가 넘긴 system_prompt)
  → BASE/CUSTOM (SDK 기본 또는 profile)
  → SUFFIX (모델별 튜닝)
```

호출자(USER)의 의도가 항상 프레임워크 가이드보다 앞에 옵니다.

기본 미들웨어 스택

위치	미들웨어	조건
base	`FilesystemMiddleware`	항상 파일 도구 제공
base	`SubAgentMiddleware`	inline subagent가 있을 때 `task` 제공
base	`SummarizationMiddleware`	컨텍스트 압축
base	`PatchToolCallsMiddleware`	잘못된 도구 호출 보정
user	사용자가 넘긴 `middleware=[...]`	base 뒤, tail 앞에 삽입
tail	`SkillsMiddleware`	`skills=[...]` 또는 같은 이름 middleware 제공 시
tail	`MemoryMiddleware`	`memory=[...]` 사용 시
tail	`HumanInTheLoopMiddleware`	`interrupt_on` 사용 시

`write_todos`는 0.7 계열에서 기본 주입이 아닙니다. 계획 도구를 수업에 포함하려면 LangChain의 `TodoListMiddleware()`를 `middleware`에 명시적으로 추가합니다.

같은 `name`을 가진 사용자 미들웨어를 넘기면 해당 기본 슬롯을 대체할 수 있습니다. 예를 들어 `SkillsMiddleware`라는 이름의 미들웨어는 tail의 자동 Skills 슬롯을 교체합니다. 이 규칙은 LangChain 일반 미들웨어 중복 처리와 구분해서 이해해야 합니다.

```
# 미들웨어 임포트 확인
from deepagents.middleware import (
    FilesystemMiddleware,
    MemoryMiddleware,
    SubAgentMiddleware,
    SkillsMiddleware,
    SummarizationMiddleware,
)

print("사용 가능한 미들웨어:")
for mw in [FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware, SkillsMiddleware,
SummarizationMiddleware]:
    print(f"  - {mw.__name__}")
```

```
사용 가능한 미들웨어:
  - FilesystemMiddleware
  - MemoryMiddleware
  - SubAgentMiddleware
  - SkillsMiddleware
  - _DeepAgentsSummarizationMiddleware
```

> · NOTE
>
> 참고: 미들웨어는 `create_deep_agent()`가 자동으로 구성하므로, 대부분의 경우 직접 다룰 필요가 없습니다. 커스텀 미들웨어는 `middleware` 파라미터로 추가할 수 있으며, 고급 사용자를 위한 기능입니다.

핵심 정리

항목	방법
모델 선택	`model="provider:model-name"` 또는 `ChatOpenAI(model="gpt-5.4")`
Deep Agents 기본 모델	모델 미지정은 deprecated이므로 수업 예제에서는 명시
시스템 프롬프트	`system_prompt="역할과 규칙을 정의"` (USER → BASE → SUFFIX 순으로 조립)
커스텀 도구	함수 + docstring + 타입 힌트 → `tools=[func]`
구조화된 출력	`response_format=PydanticModel` → `result["structured_response"]`
미들웨어	파일·서브에이전트·요약은 하네스가 조립, 계획은 `TodoListMiddleware()` 로 opt-in
같은 이름 미들웨어	기본 슬롯을 대체할 수 있음 (`SkillsMiddleware` 등)
풀 시그니처	`model` , `tools` , `system_prompt` , `middleware` , `subagents` , `skills` , `memory` , `permissions` , `backend` , `interrupt_on` , `response_format` , `state_schema` , `context_schema` , `checkpointer` , `store` , `debug` , `name` , `cache`

04 스토리지 백엔드

학습 목표

LEARNING OBJECTIVES

- → 백엔드가 에이전트의 파일 시스템을 어떻게 구현하는지 이해한다
- → 5가지 내장 백엔드의 특성과 사용 시나리오를 파악한다
- → `CompositeBackend` 로 경로별 백엔드 라우팅을 구성한다
- → `BackendProtocol` 을 구현하여 커스텀 백엔드를 만든다

```python
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")

from langchain_openai import ChatOpenAI

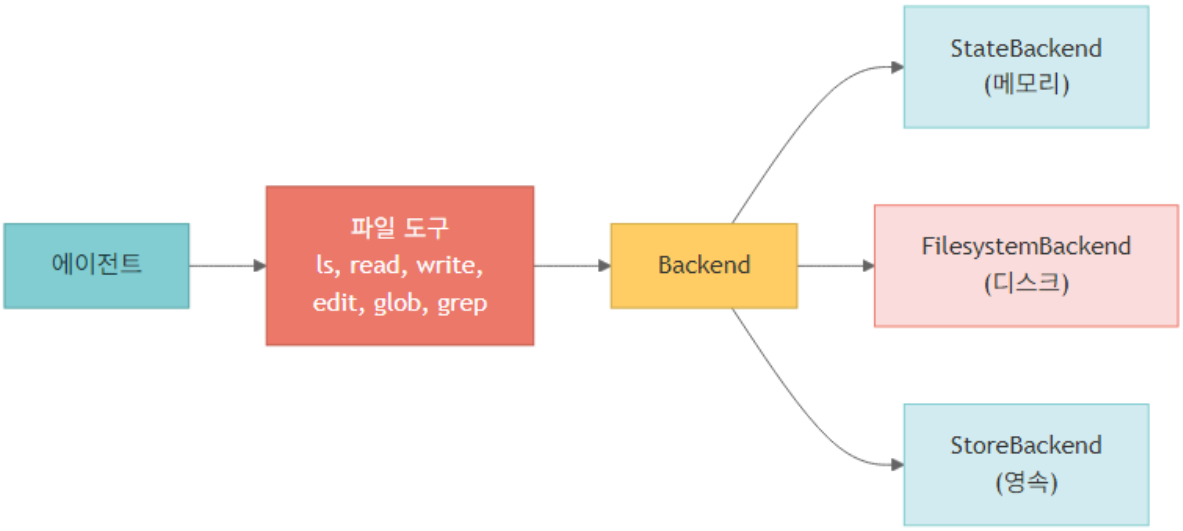
model = ChatOpenAI(model="gpt-5.4")
```

```
환경 설정 완료
```

1. 백엔드란?

Deep Agents의 빌트인 파일 도구(`ls` , `read_file` , `write_file` , `edit_file` , `delete` , `glob` , `grep`)는 모두 백엔드(Backend) 를 거쳐 동작합니다. `read_file` 은 모든 백엔드에서 이미지 파일을 멀티모달 content로 반환합니다.

백엔드는 에이전트가 파일을 읽고 쓰는 스토리지 계층을 추상화합니다.



사용 가능한 백엔드

백엔드	저장 위치	영속성	사용 시나리오
`StateBackend`	에이전트 상태 (LangGraph state)	스레드 내	임시 작업, 스크래치패드 (기본값)
`FilesystemBackend`	로컬 디스크	영구	로컬 파일 접근, 코딩 에이전트
`LocalShellBackend`	디스크 + 셸 실행	영구	개발 환경 (보안 주의)
`StoreBackend`	LangGraph `BaseStore`	크로스 스레드	장기 메모리, 사용자 선호도
`ContextHubBackend`	LangSmith Hub 저장소	영구	공유 가능한 파일 트리, lazy fetch + write-through
`CompositeBackend`	경로별 라우팅	혼합	메모리 + 임시 파일 병용
샌드박스	Modal / Daytona / Deno / local VFS	격리	격리된 셸·파일 실행

> ➡️ **TIP**
>
> 최신 예제에서는 백엔드를 `BackendProtocol` 인스턴스로 만들어 전달합니다. `StoreBackend` 는 `namespace` 를 명시하고, 실제 store는 `create_deep_agent(store=...)` 로 함께 넘깁니다. `namespace=lambda rt: (...)` 처럼 런타임을 받는 namespace factory는 여전히 유효합니다.

```
# 백엔드 임포트 확인
from deepagents.backends import (
    StateBackend,
    FilesystemBackend,
    StoreBackend,
    CompositeBackend,
)
```

```
# ContextHubBackend는 langsmith-hub 통합이 필요 (선택적)
try:
    from deepagents.backends import ContextHubBackend
    print("ContextHubBackend 사용 가능")
except ImportError:
    print("ContextHubBackend는 LangSmith Hub 의존성이 필요합니다 (선택).")

from deepagents.backends.protocol import BackendProtocol

print("모든 백엔드 클래스를 성공적으로 임포트했습니다!")
```

```
모든 백엔드 클래스를 성공적으로 임포트했습니다!
```

---

2. StateBackend (기본값)

에이전트 상태(LangGraph state)에 파일을 저장합니다. 에페메럴(ephemeral): 대화 스레드 내에서만 파일이 유지됩니다.

특징

- `create_deep_agent()` 에서 `backend` 를 지정하지 않으면 자동 사용
- 파일이 체크포인트를 통해 에이전트 턴 간에는 유지됨
- 스레드가 종료되면 파일 소멸
- 외부 스토리지 없이 바로 사용 가능

---

3. FilesystemBackend — 로컬 디스크 접근

에이전트가 실제 로컬 파일 시스템에 접근할 수 있게 합니다.

주요 옵션

- `root_dir` — 접근 가능한 루트 디렉토리 (기본: 현재 디렉토리)
- `virtual_mode=True` — 기본값이며 `..` , `~` , `root_dir` 밖의 절대 경로를 차단합니다
- `max_file_size_mb` — 읽을 수 있는 최대 파일 크기

`write_file` 은 같은 경로의 기존 파일을 교체할 수 있습니다. 백엔드가 `delete` 를 지원하면 `delete` 도 노출되므로, 쓰기 허용 정책은 삭제 가능성까지 함께 검토해야 합니다.

⚠️ 보안 주의사항

> ➡️ TIP
>
> `FilesystemBackend` 는 에이전트에게 실제 파일 시스템 접근 권한을 부여합니다. 에이전트가 `.env` , API 키, 자격 증명 같은 접근 가능한 파일을 읽을 수 있고, 네트워크 도구와 결합되면 시크릿이 유출될 수 있습니다. 프로덕션 환경에서는 `virtual_mode=True` 와 승인 정책, 또는 샌드박스 백엔드를 함께 사용하세요.

4. StoreBackend — 크로스 스레드 영속 저장소

LangGraph의 `BaseStore`를 활용하여 대화 스레드를 넘어서 파일을 영속적으로 저장합니다.

특징

- 다른 스레드에서도 같은 파일에 접근 가능 (Redis, PostgreSQL, 클라우드 구현 지원)
- LangSmith 배포 시 PostgreSQL 기반 store가 자동 프로비저닝
- `namespace`는 필수 격리 경계입니다. 데모처럼 고정 tuple을 쓰거나, `namespace=lambda rt: (...)`로 런타임 context를 읽습니다.
- `StoreBackend(namespace=...)` 인스턴스와 `create_deep_agent(store=...)`를 함께 전달합니다.

namespace 패턴

```
# fixed demo namespace
StoreBackend(namespace=lambda rt: ("demo-user",))

# user-scoped runtime context
StoreBackend(namespace=lambda rt: (rt.context.user_id,))
```

제거된 backend factory 패턴과 namespace factory를 혼동하지 마세요. namespace callable은 현재 계약에서 유효합니다.

```
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

# InMemoryStore — 개발용 (프로덕션에서는 PostgresStore 등 사용)
store = InMemoryStore()
checkpointer = MemorySaver()

# StoreBackend — 사전 생성 인스턴스 (deepagents>=0.5.2 권장 패턴)
# namespace 콜러블은 LangGraph Runtime 객체를 받음
store_backend = StoreBackend(
    namespace=lambda rt: ("demo-user",),  # 데모용 고정 namespace; 운영 시 rt.context.user_id 사용
)

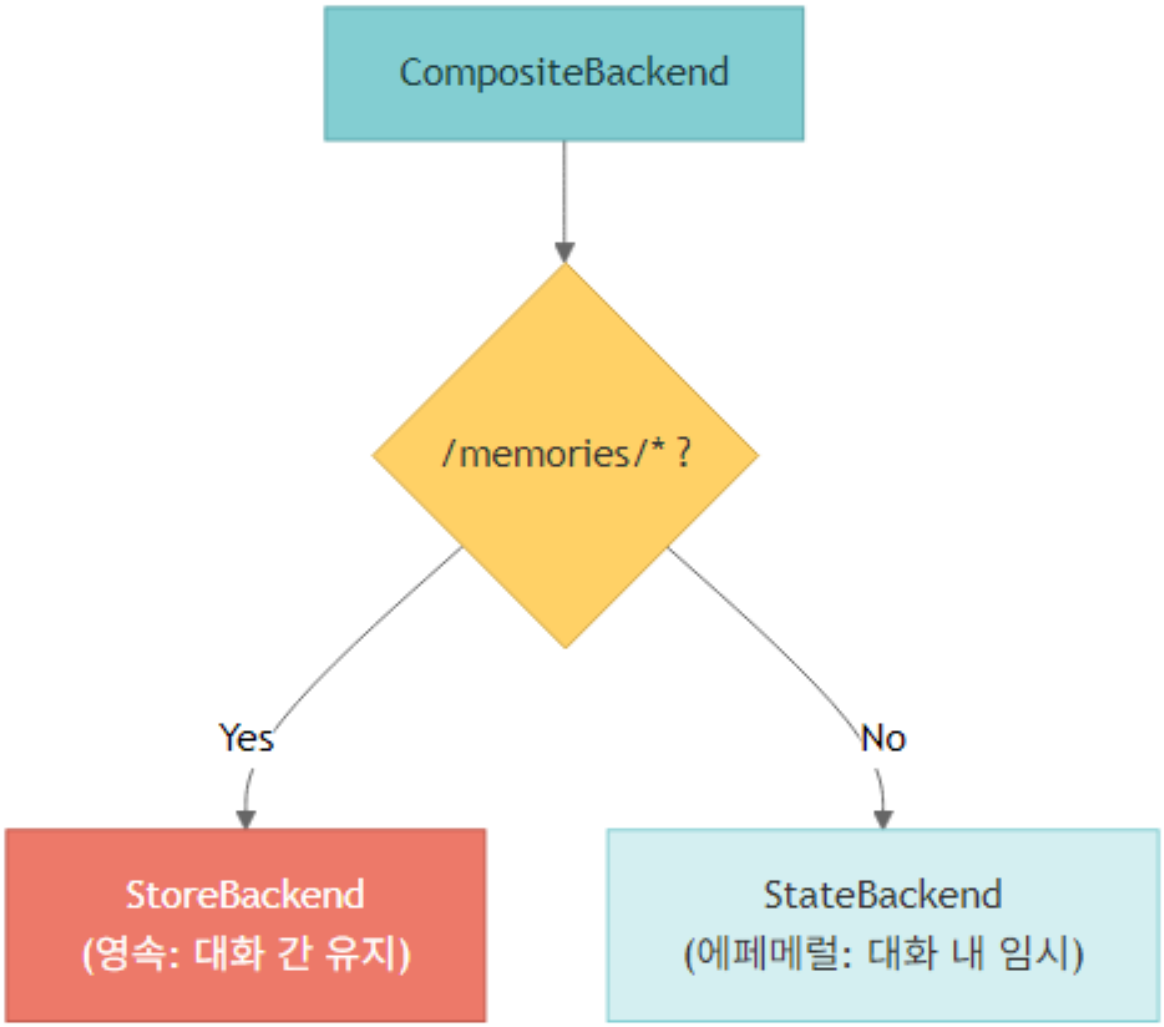
store_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 메모를 관리하는 어시스턴트입니다. 한국어로 응답하세요.",
    backend=store_backend,
    store=store,
    checkpointer=checkpointer,
)

print("StoreBackend 에이전트가 생성되었습니다!")
```

```
StoreBackend 에이전트가 생성되었습니다!
```

5. CompositeBackend — 경로별 라우팅

서로 다른 경로를 서로 다른 백엔드로 라우팅합니다. 가장 일반적인 패턴: `/memories/*` 는 영속 저장, 나머지는 에페메럴




```
# CompositeBackend — 사전 생성 인스턴스 사용 (deepagents>=0.5.2 권장)
composite_store = InMemoryStore()
composite_checkpointer = MemorySaver()

composite_backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: ("composite-demo",),
        ),
    },
)

composite_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 메모 관리 어시스턴트입니다.
- 영구 저장이 필요한 메모는 /memories/ 경로에 저장하세요.
- 임시 작업 파일은 루트(/) 경로에 저장하세요.
한국어로 응답하세요.""",
```

```
    backend=composite_backend,
    store=composite_store,
    checkpointer=composite_checkpointer,
)

print("CompositeBackend 에이전트가 생성되었습니다!")
```

```
CompositeBackend 에이전트가 생성되었습니다!
```

> · NOTE
>
> 참고: `CompositeBackend` 는 라우트 프리픽스를 제거한 후 저장합니다. 예: `/memories/preferences.txt` → 내부적으로 `/preferences.txt` 로 저장 하지만 에이전트는 항상 전체 경로(`/memories/preferences.txt`)로 접근합니다.

---

6. LocalShellBackend – 셸 실행

`LocalShellBackend` 는 `FilesystemBackend` 에 셸 명령 실행 기능(`execute` 도구)을 추가합니다.

⚠️ 보안 경고

> · NOTE
>
> 호스트 시스템에서 사용자 권한으로 `subprocess.run(shell=True)` 가 직접 실행됩니다. 샌드박스 없음, CPU/메모리/디스크 무제한, 비가역적입니다. 개발 환경에서만 사용하고, 공유 또는 프로덕션 시스템에서는 샌드박스 백엔드를 사용하세요.

정책 적용 방법

1. 권한 시스템 – `permissions=[...]` 파라미터로 read/write 규칙을 선언적으로 정의
2. 정책 훅 – 백엔드를 상속하거나 wrap (예: `GuardedBackend` 가 특정 prefix 의 `write()` / `edit()` 거부)

```
from deepagents.backends import LocalShellBackend

# ⚠ 개발 환경에서만 사용하세요!
agent = create_deep_agent(
    model=model,
    backend=LocalShellBackend(root_dir="./workspace", virtual_mode=True),
    interrupt_on={"execute": True},  # 셸 명령은 승인 필요
)
```

> · NOTE
>
> 이 노트북에서는 안전상의 이유로 `LocalShellBackend` 를 직접 실행하지 않습니다.

---

7. 커스텀 백엔드 구현

`BackendProtocol`을 구현하면 나만의 백엔드를 만들 수 있습니다.

필수 메서드

메서드	반환 타입	설명
`ls(path)`	`LsResult`	디렉토리 내용 목록 (`path` , `is_dir` , `size` , `modified_at`)
`read(file_path, offset=0, limit=2000)`	`ReadResult`	파일 읽기. 없으면 `ReadResult(error=...)`
`write(file_path, content)`	`WriteResult`	백엔드 정책에 따라 생성 또는 교체
`edit(file_path, old_string, new_string, replace_all=False)`	`EditResult`	`old_string`의 유일성 강제, 성공 시 `occurrences` 포함
`grep(pattern, path=None, glob=None)`	`GrepResult`	결과 매치. 오류 시 raise 대신 `GrepResult(error=...)` 반환
`glob(pattern, path="/")`	`list[FileInfo]`	글로브 매치, 비어 있으면 `[]`
`delete(file_path)`	`DeleteResult`	선택 메서드. 구현한 백엔드에서만 `delete` 노출

이전 메서드명(`ls_info` , `grep_raw` , `glob_info`)은 새 교육 코드에 사용하지 않습니다. 도구 출력 문자열과 백엔드의 구조화된 반환값도 구분해서 다룹니다.

```python
# 간단한 커스텀 백엔드 예시: 읽기 전용 딕셔너리 기반
# deepagents>=0.7 BackendProtocol 메서드명과 결과 타입 사용
from deepagents.backends.protocol import (
    FileInfo, LsResult, ReadResult, WriteResult, EditResult, GrepResult, GrepMatch,
)


class ReadOnlyDictBackend:
    """딕셔너리에 파일을 저장하는 읽기 전용 백엔드 예시"""

    def __init__(self, files: dict[str, str]):
        self._files = files

    def ls(self, path: str = "/") -> LsResult:
        entries = [
            FileInfo(path=p, is_dir=False, size=len(c), modified_at=None)
            for p, c in self._files.items()
            if p.startswith(path)
        ]
        return LsResult(entries=sorted(entries, key=lambda e: e["path"]))

    def read(self, file_path: str, offset: int = 0, limit: int = 2000) -> ReadResult:
        content = self._files.get(file_path)
        if content is None:
            return ReadResult(error=f"파일 없음: {file_path}")
        lines = content.splitlines()
        selected = lines[offset:offset + limit]
        text = "\n".join(f"{i + offset + 1}\t{line}" for i, line in enumerate(selected))
        next_offset = offset + limit if offset + limit < len(lines) else None
        return ReadResult(
            file_data={"content": text, "encoding": "utf-8"},
```

```
            total_lines=len(lines), start_line=offset + 1,
            end_line=offset + len(selected), next_offset=next_offset,
            no_lines_requested=(limit == 0),
        )

    def write(self, file_path: str, content: str) -> WriteResult:
        return WriteResult(error="읽기 전용 백엔드입니다.")

    def edit(self, file_path: str, old_string: str, new_string: str, replace_all: bool = False) ->
EditResult:
        return EditResult(error="읽기 전용 백엔드입니다.")

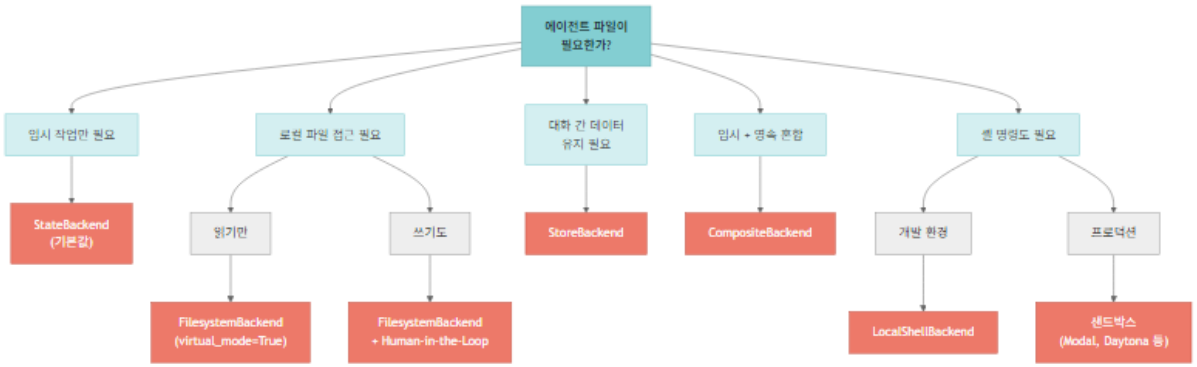
    def grep(self, pattern: str, path: str | None = None, glob: str | None = None) -> GrepResult:
        import re
        matches = []
        for fpath, content in self._files.items():
            for i, line in enumerate(content.splitlines(), 1):
                if re.search(pattern, line):
                    matches.append(GrepMatch(path=fpath, line=i, text=line))
        return GrepResult(matches=matches)

    def glob(self, pattern: str, path: str = "/") -> list[FileInfo]:
        import fnmatch
        return [
            FileInfo(path=p, is_dir=False, size=len(c), modified_at=None)
            for p, c in self._files.items()
            if p.startswith(path) and fnmatch.fnmatch(p, pattern)
        ]


# 사용 예시
custom_backend = ReadOnlyDictBackend({
    "/docs/guide.md": "# 가이드\n이것은 가이드 문서입니다.\n## 설치 방법\npip install deepagents",
    "/docs/faq.md": "# FAQ\nQ: 지원하는 모델은?\nA: Anthropic, OpenAI 등 다양한 모델을 지원합니다.",
})

# 커스텀 백엔드 동작 확인
print("파일 목록:", custom_backend.ls("/docs/"))
print()
print("파일 내용:")
print(custom_backend.read("/docs/guide.md").file_data["content"])
print()
print("쓰기 결과:", custom_backend.write("/docs/new.md", "초안"))
print("검색 결과:", custom_backend.grep("설치"))
```

백엔드 선택 가이드



핵심 정리

백엔드	특징	파라미터
`StateBackend`	에페메럴, 기본값	`backend` 생략 시 자동
`FilesystemBackend`	로컬 디스크	`root_dir` , `virtual_mode=True` 기본
`LocalShellBackend`	디스크 + 셸 실행	`root_dir` , `virtual_mode` , 승인 정책
`StoreBackend`	크로스 스레드 영속	`namespace` + `create_deep_agent(store=...)`
`ContextHubBackend`	LangSmith Hub	lazy fetch + write-through 캐시
`CompositeBackend`	경로별 라우팅	`default` + `routes` (긴 prefix 우선)
샌드박스	격리된 실행	Modal / Daytona / Deno / local VFS

`BackendProtocol` 메서드	반환 타입
`ls` , `read` , `write` , `edit` , `grep` , `glob`	`LsResult` , `ReadResult` , `WriteResult` , `EditResult` , `GrepResult` , `list[FileInfo]`
`delete`	`DeleteResult` (선택)

새 예제는 사전 생성된 백엔드 인스턴스를 전달하고, `StoreBackend` 는 namespace와 store를 함께 다룹니다.

05 서브에이전트와 태스크 위임

학습 목표

LEARNING OBJECTIVES

- → 서브에이전트가 해결하는 문제(컨텍스트 블로트)를 이해한다
- → `SubAgent` dict와 `CompiledSubAgent` 로 서브에이전트를 정의한다
- → 빌트인 general-purpose 서브에이전트를 이해하고 오버라이드한다
- → 컨텍스트 전파와 네임스페이스 키를 활용한다
- → 멀티 서브에이전트 파이프라인 패턴을 구현한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
assert os.environ.get("TAVILY_API_KEY"), "TAVILY_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
# OpenAI gpt-5.4 모델 설정 (Deep Agents 기본은 anthropic:claude-sonnet-4-6)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

```
모델 설정 완료: gpt-4.1
```

1. 서브에이전트가 필요한 이유

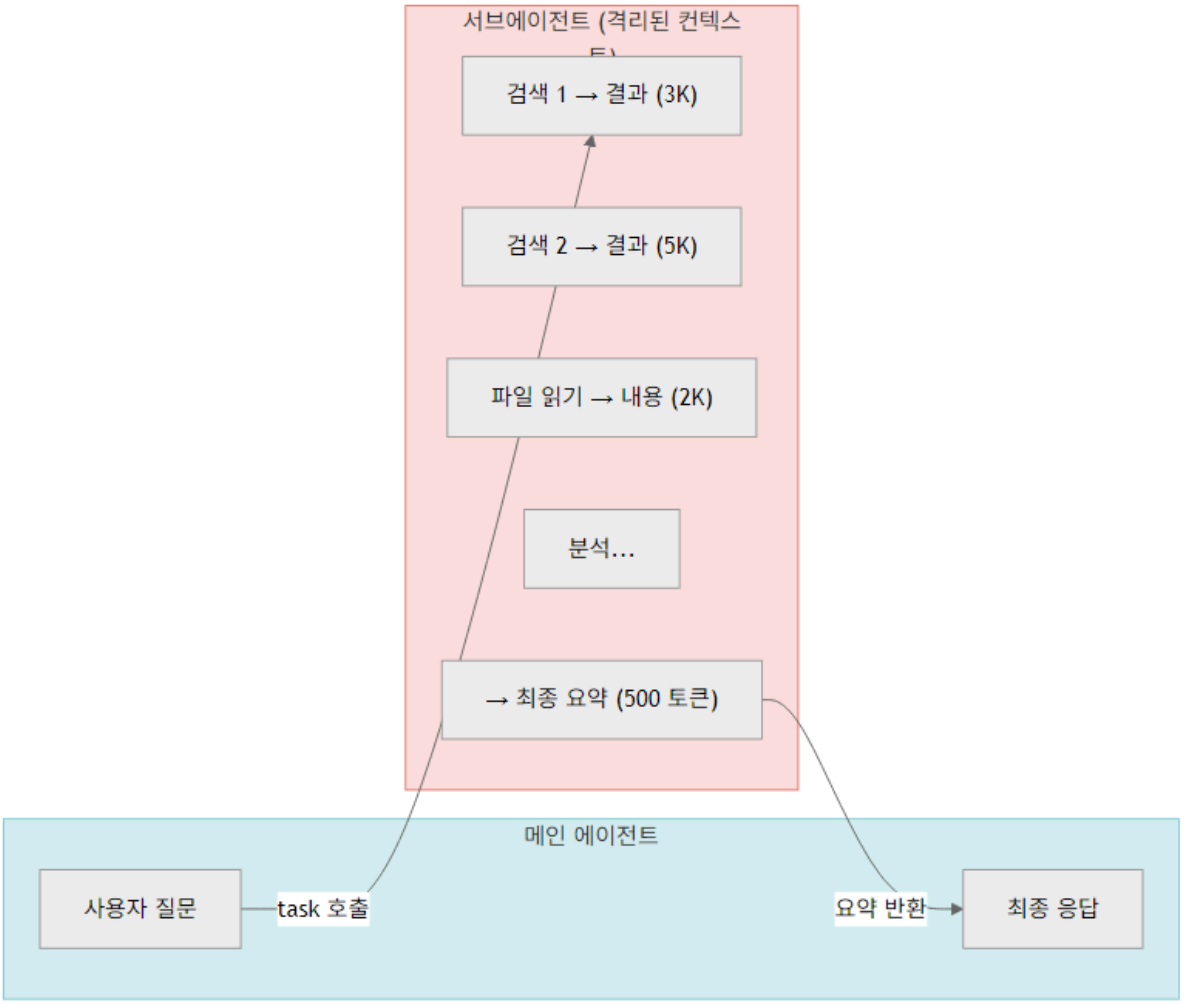
컨텍스트 블로트(Context Bloat) 문제

에이전트가 도구를 사용할 때마다 입력/출력이 컨텍스트 윈도우에 쌓입니다:

- 웹 검색 결과 (수천 토큰)
- 파일 내용 읽기 (수백 수천 줄)
- 데이터베이스 쿼리 결과

중간 결과들이 메인 에이전트의 컨텍스트를 가득 채우면, 정작 중요한 정보를 놓칠 수 있습니다.

서브에이전트의 해결 방식



메인 에이전트는 500 토큰짜리 요약만 받으므로 컨텍스트가 깔끔하게 유지됩니다.

서브에이전트 사용 기준

상황	서브에이전트 사용

다단계 작업으로 중간 결과가 많음	✅ 사용
전문 지식/도구가 필요한 도메인	✅ 사용
다른 모델이 더 적합한 작업	✅ 사용
단순하고 한 번에 끝나는 작업	❌ 불필요
중간 결과가 메인 에이전트에 필요	❌ 불필요

2. SubAgent 정의하기 (dict 기반)

`SubAgent` 는 딕셔너리 형태로 정의합니다. `SubAgentMiddleware` 가 자동으로 `task()` 도구를 메인 에이전트에 주입해서, 메인이 서브에이전트 이름으로 작업을 위임합니다. 동기 서브에이전트가 존재하면 `SubAgentMiddleware` 는 필수 미들웨어로 자동 부착되며 `excluded_middleware` 로 제거할 수 없습니다.

필수 필드

필드	타입	설명
`name`	`str`	고유 식별자
`description`	`str`	역할 설명 (메인 에이전트가 호출 판단에 사용)
`system_prompt`	`str`	서브에이전트 지침 (부모로부터 상속하지 않음)
`tools`	`list`	서브에이전트가 쓸 도구 (지정 시 부모 도구를 덮어씀)

선택 필드

필드	타입	설명
`model`	`str` \	`BaseChatModel`
모델 오버라이드 (`"provider:model"` 또는 chat model 객체)	`middleware`	`list`
추가 미들웨어 (상속되지 않음)	`interrupt_on`	`dict[str, bool]`
HITL 설정 (메인 설정을 오버라이드)	`skills`	`list[str]`
격리된 스킬 디렉토리	`response_format`	`ResponseFormat`
구조화 출력 (deepagents ≥ 0.5.3)	`permissions`	`list[FilesystemPermission]`
부모 규칙을 완전히 대체		

Dispatch + 계획 패턴

메인 에이전트는 `TodoList`(write_todos)로 요청을 순서 있는 작업으로 분해한 뒤 각 항목을 `task()` 호출로 위임합니다. 스트리밍 중에는 `lc_agent_name` 메타데이터로 어느 서브에이전트의 출력인지 식별할 수 있습니다.

```
from typing import Literal
from tavily import TavilyClient
from deepagents import create_deep_agent

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])


def internet_search(
    query: str,
    max_results: int = 5,
    topic: Literal["general", "news", "finance"] = "general",
    include_raw_content: bool = False,
) -> dict:
    """인터넷에서 정보를 검색합니다.

    Args:
        query: 검색할 질문 또는 키워드
        max_results: 반환할 최대 결과 수
        topic: 검색 주제 카테고리
        include_raw_content: 원본 콘텐츠 포함 여부
    """
    return tavily_client.search(
        query,
        max_results=max_results,
        include_raw_content=include_raw_content,
        topic=topic,
    )


# 리서치 서브에이전트 정의
# `model`을 명시하면 메인과 다른 모델을 쓸 수도 있다 — 예: 가벼운 작업은 gpt-5.4-mini
research_subagent = {
    "name": "researcher",
    "description": "인터넷에서 주제를 심층 조사하고 핵심 정보를 요약합니다. 리서치가 필요한 질문에 사용하세요.",
    "system_prompt": """당신은 전문 리서처입니다.
인터넷 검색을 통해 정확한 정보를 수집하고, 핵심만 추출하여 간결하게 요약합니다.
결과는 항상 한국어로 작성하며, 출처를 함께 표기합니다.
최종 결과는 500단어 이내로 요약하세요.""",
    "tools": [internet_search],
}

print(f"서브에이전트 정의 완료: {research_subagent['name']}")
print(f"설명: {research_subagent['description'][:50]}...")
```

```
서브에이전트 정의 완료: researcher
설명: 인터넷에서 주제를 심층 조사하고 핵심 정보를 요약합니다. 리서치가 필요한 질문에 사용하세요...
```

```
# 서브에이전트를 포함하는 메인 에이전트 생성
main_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 프로젝트 매니저입니다.
사용자의 요청을 분석하고, 필요하면 전문 서브에이전트에게 작업을 위임합니다.
서브에이전트의 결과를 종합하여 최종 답변을 작성합니다.
한국어로 응답하세요.""",
    subagents=[research_subagent],
)

print("메인 에이전트 생성 완료 (서브에이전트: researcher)")
```

```
메인 에이전트 생성 완료 (서브에이전트: researcher)
```

3. CompiledSubAgent — 커스텀 LangGraph 그래프 연결

미리 컴파일된 LangGraph 그래프를 서브에이전트로 쓸 수 있습니다. 조건 분기, 반복 같은 복잡한 워크플로가 필요한 경우에 유용합니다.

```
from deepagents import CompiledSubAgent

# 별도의 에이전트를 CompiledSubAgent로 래핑하는 예시
# 실제로는 create_deep_agent()로 만든 그래프도 사용 가능
custom_graph = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 데이터 분석 전문가입니다. 데이터를 수집하고 통계적으로 분석하여 인사이트를 도출합니다.",
)

# CompiledSubAgent로 래핑
data_analyst_subagent: CompiledSubAgent = {
    "name": "data-analyst",
    "description": "데이터를 수집하고 분석하여 통계적 인사이트를 제공합니다.",
    "runnable": custom_graph,
}

print(f"CompiledSubAgent 정의 완료: {data_analyst_subagent['name']}")
```

```
CompiledSubAgent 정의 완료: data-analyst
```

---

4. General-Purpose 서브에이전트

Deep Agents는 별도로 정의하지 않아도 빌트인 general-purpose 서브에이전트를 자동 제공합니다.

기본 동작

- 메인 에이전트와 같은 시스템 프롬프트 사용
- 메인 에이전트와 같은 도구 접근 (filesystem 도구 포함)
- 메인 에이전트와 같은 모델 사용 (이 노트북에서는 OpenAI `gpt-5.4`, Deep Agents 표준 기본은 `anthropic:claude-sonnet-4-6`)
- 메인 에이전트의 스킬을 자동 상속 (커스텀 서브에이전트와 다른 점)

비활성화 또는 오버라이드

- 완전 비활성화: 하네스 프로파일에서 `GeneralPurposeSubagentProfile(enabled=False)`
- 대체: `name="general-purpose"` 로 커스텀 서브에이전트 정의

```
# general-purpose 서브에이전트 오버라이드 예시
custom_gp_agent = create_deep_agent(
    model=model,
    tools=[internet_search],
    system_prompt="당신은 멀티태스크 코디네이터입니다.",
    subagents=[
        research_subagent,
        {
            # 이름을 "general-purpose"로 설정하면 빌트인을 오버라이드
            "name": "general-purpose",
            "description": "범용 에이전트로, 리서치 외의 일반적인 멀티스텝 작업을 처리합니다.",
```

```
            "system_prompt": "당신은 범용 어시스턴트입니다. 주어진 작업을 단계별로 처리하세요.",
            "tools": [internet_search],
        },
    ],
)

print("general-purpose 서브에이전트를 오버라이드한 에이전트 생성 완료")
```

```
general-purpose 서브에이전트를 오버라이드한 에이전트 생성 완료
```

실험적 fork 모드 구성

기본 서브에이전트 호출은 isolated 모드입니다. 부모 대화 문맥을 이어받아야 하는 제한적인 경우에는 `mode="fork"` 를 명시합니다. fork 서브에이전트는 부모 스킬을 상속하므로 별도 `skills` 필드를 함께 두지 않습니다.

```
# fork 서브에이전트 구성만 검증합니다. 모델 호출은 하지 않습니다.
fork_reviewer = {
    "name": "fork-reviewer",
    "description": "부모 문맥을 이어받아 짧은 리뷰 체크리스트를 작성합니다.",
    "system_prompt": "부모 대화의 요구사항을 기준으로 빠진 검증 항목만 요약하세요.",
    "tools": [],
    "mode": "fork",
}
fork_agent = create_deep_agent(model=model, system_prompt="리뷰를 조율합니다.", subagents=[fork_reviewer])
print(f"fork 서브에이전트 생성 완료: {fork_reviewer['name']} / mode={fork_reviewer['mode']}")
```

---

5. 컨텍스트 전파

런타임 컨텍스트는 자동으로 모든 서브에이전트에 전파됩니다. `context_schema` 로 구조를 정의하고, `config` 의 `context` 키로 값을 전달합니다.

네임스페이스 키로 서브에이전트별 컨텍스트 전달

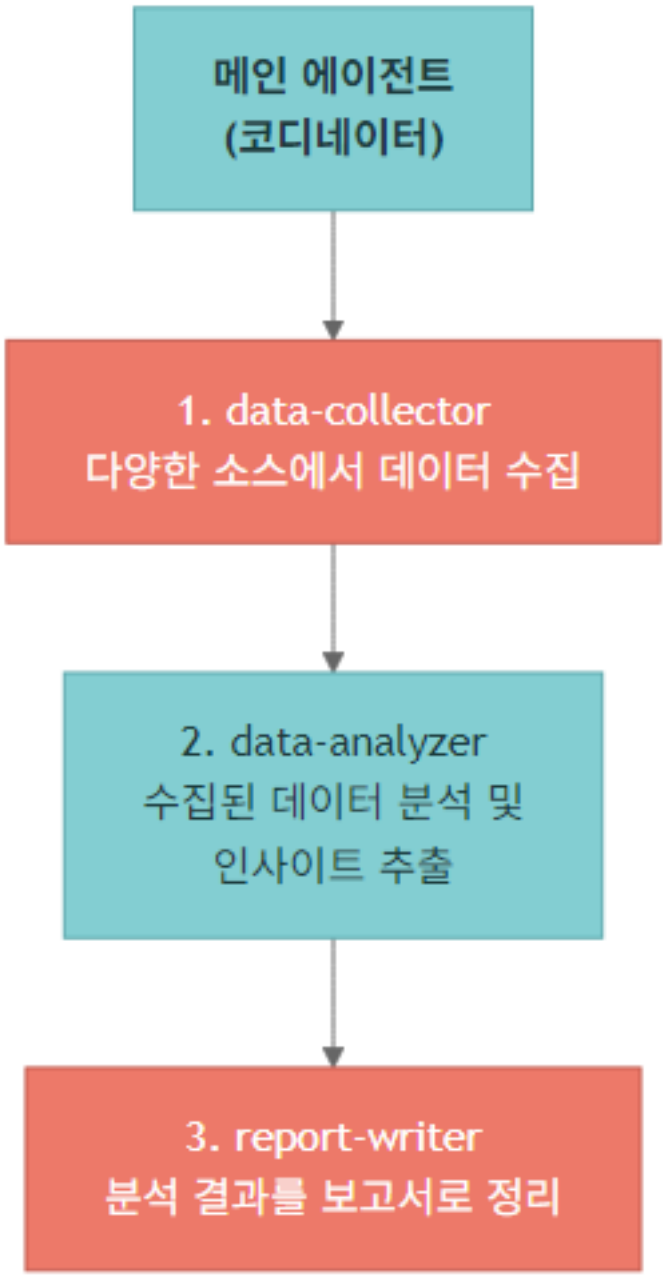
`"서브에이전트이름:키"` 형식을 쓰면, 특정 서브에이전트에만 전달되는 설정을 추가할 수 있습니다.

```
config = {
    "context": {
        "user_id": "user-123",              # 모든 에이전트에 전파
        "researcher:max_depth": 3,           # researcher에만 전달
        "data-analyst:strict_mode": True,    # data-analyst에만 전달
    }
}
```

---

6. 멀티 서브에이전트 파이프라인

여러 서브에이전트를 조합하여 수집 → 분석 → 작성 파이프라인을 구성할 수 있습니다.




```
# 멀티 서브에이전트 파이프라인
pipeline_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 프로젝트 코디네이터입니다.
사용자의 요청을 분석하고, 적절한 서브에이전트에게 순서대로 작업을 위임합니다:
1. 먼저 data-collector로 정보를 수집합니다.
2. 수집된 정보를 data-analyzer에게 전달하여 분석합니다.
3. 분석 결과를 report-writer에게 전달하여 보고서를 작성합니다.
최종 보고서를 사용자에게 전달합니다. 한국어로 응답하세요.""",
    subagents=[
        {
            "name": "data-collector",
            "description": "다양한 소스에서 원시 데이터와 정보를 수집합니다.",
            "system_prompt": """당신은 데이터 수집 전문가입니다.
주어진 주제에 대해 인터넷 검색을 수행하고, 관련 데이터를 최대한 수집합니다.
```

```
수집한 데이터를 구조화하여 반환하세요.""",
            "tools": [internet_search],
        },
        {
            "name": "data-analyzer",
            "description": "수집된 데이터를 분석하여 핵심 인사이트를 추출합니다.",
            "system_prompt": """당신은 데이터 분석 전문가입니다.
제공된 데이터에서 패턴, 트렌드, 핵심 인사이트를 추출합니다.
분석 결과를 불릿 포인트로 정리하세요.""",
            "tools": [],
        },
        {
            "name": "report-writer",
            "description": "분석 결과를 바탕으로 전문적인 보고서를 작성합니다.",
            "system_prompt": """당신은 테크니컬 라이터입니다.
분석 결과를 바탕으로 명확하고 읽기 쉬운 보고서를 작성합니다.
보고서는 다음 구조를 따릅니다: 개요 → 핵심 발견 → 결론""",
            "tools": [],
        },
    ],
)

print("멀티 서브에이전트 파이프라인 에이전트 생성 완료")
```

```
멀티 서브에이전트 파이프라인 에이전트 생성 완료
```

7. 베스트 프랙티스

1. 명확한 description 작성

메인 에이전트는 `description`을 보고 어떤 서브에이전트를 호출할지 결정합니다. 서브에이전트의 역할과 사용 시기를 명확하게 기술하세요.

2. 기본 isolated와 실험적 fork 구분

기본 서브에이전트 호출은 isolated 모드입니다. 호출마다 별도 문맥에서 작업하므로 필요한 입력을 한 번에 전달해야 합니다. `mode="fork"`는 부모의 유효 대화 문맥과 prompt-producing middleware를 이어받는 실험적 모드입니다. fork는 부모 skills를 상속하며 별도 `skills` 설정을 거부합니다. 이것을 장기 메모리나 호출 간 상태 보존으로 확대 해석하지 않습니다.

3. 최소 도구 원칙

서브에이전트에는 필요한 도구만 제공하세요. 불필요한 도구는 컨텍스트를 낭비하고 오동작을 유발합니다.

4. 간결한 결과 반환

서브에이전트의 시스템 프롬프트에 "결과를 간결하게 요약해서 반환"하도록 명시하세요. 중간 로그 전체를 반환하면 메인 에이전트 컨텍스트가 다시 비대해집니다.

5. 상태 공유가 필요하면 파일 또는 store 사용

서브에이전트 간 정보를 공유하려면 `write_file`로 중간 결과를 저장하거나 `StoreBackend` 기반 경로를 사용합니다. 단순히 이전 task 호출을 기억한다고 가정하지 않습니다.

핵심 정리

항목	내용
SubAgent	dict 기반 정의: `name`, `description`, `system_prompt`, `tools`
CompiledSubAgent	커스텀 LangGraph 그래프를 `runnable`로 연결
General-Purpose	빌트인 기본 서브에이전트 (명시적으로 같은 이름을 정의하면 오버라이드)
기본 모드	isolated: 호출마다 독립 문맥, 필요한 입력을 한 번에 제공
fork 모드	`mode="fork"` : 부모 문맥을 이어받는 실험적 모드, 부모 skills 상속
컨텍스트 전파	`context_schema`와 `config["context"]`로 런타임 context 전달
상태 공유	파일 또는 StoreBackend 경로로 명시적으로 공유

06 장기 메모리 & 스킬

학습 목표

LEARNING OBJECTIVES

- → `CompositeBackend` + `StoreBackend` 로 장기 메모리를 구현한다
- → 크로스 스레드 메모리 공유 패턴을 이해한다
- → `AGENTS.md` 로 에이전트에 컨텍스트를 주입한다
- → 스킬(SKILL.md)의 구조와 Progressive Disclosure를 이해한다
- → Skills vs Memory의 차이를 파악한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
# OpenAI gpt-5.4 모델 설정 (Deep Agents 기본은 anthropic:claude-sonnet-4-6)
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1. 장기 메모리가 필요한 이유

기본 `StateBackend` 에이전트는 대화 스레드가 끝나면 모든 정보를 잊습니다. 실제 어시스턴트라면 아래 정보를 대화 간에 유지해야 합니다:

- 사용자 선호도 (코딩 스타일, 사용 언어)
- 프로젝트 컨벤션 (아키텍처 결정, 네이밍 규칙)
- 이전 대화에서 받은 피드백

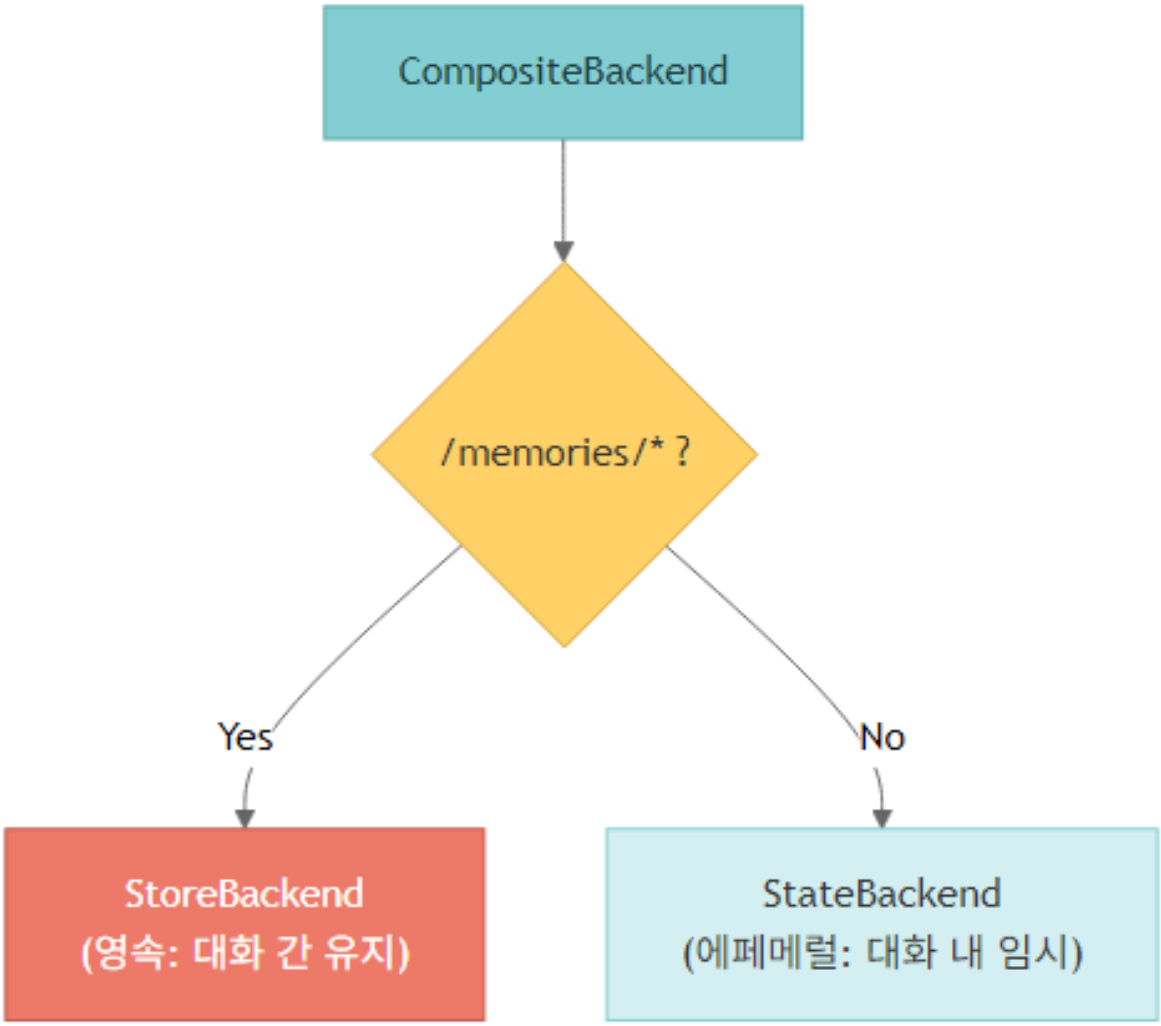
- 자주 참조하는 정보 (API 문서, 설정값)

정보 유형 분류 (deepagents 0.5+)

유형	의미	저장 예	메커니즘
Episodic	과거 경험 — 대화 세션, 문제 해결 궤적	지난 스레드 히스토리	Checkpointer (thread 단위)
Procedural	재사용 가능한 절차·스킬·워크플로	`SKILL.md`	Skills (on-demand)
Semantic	사실·선호·정책	`AGENTS.md`, `/memories/*.txt`	StoreBackend (always-on 파일)

세 유형을 하나의 백엔드로 몰아넣지 말고 성격에 맞는 메커니즘에 분산시키는 것이 기본 원칙입니다.

해결 방식: CompositeBackend



`/memories/` 경로에 저장된 파일은 어떤 대화 스레드에서든 접근할 수 있습니다.

```python
from deepagents import create_deep_agent
from deepagents.backends import StateBackend, StoreBackend, CompositeBackend, FilesystemBackend
from langgraph.store.memory import InMemoryStore
from langgraph.checkpoint.memory import MemorySaver

# 프로덕션에서는 PostgresStore 사용:
# from langgraph.store.postgres import PostgresStore

# 1. 스토어와 체크포인터 생성
store = InMemoryStore()          # 개발용
checkpointer = MemorySaver()     # 에이전트 상태 유지


# 2. CompositeBackend — 사전 생성 인스턴스 (deepagents>=0.5.2)
#    /memories/만 영속, 나머지는 에페메럴
#    namespace 콜러블은 LangGraph Runtime 객체를 받음
memory_backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: ("demo-user",),  # 운영: rt.context.user_id
        ),
    },
)


# 3. 에이전트 생성
memory_agent = create_deep_agent(
    model=model,
    system_prompt="""당신은 개인 어시스턴트입니다.
사용자가 기억해 달라고 하는 정보는 /memories/ 폴더에 저장하세요.
이전에 저장한 메모리가 있으면 참고하여 응답하세요.
한국어로 응답하세요.""",
    backend=memory_backend,
    store=store,
    checkpointer=checkpointer,
)

print("장기 메모리 에이전트 생성 완료")
```

```
장기 메모리 에이전트 생성 완료
```

---

2. 크로스 스레드 메모리 공유

`StoreBackend`에 저장된 데이터는 스레드 간에 공유됩니다. 아래 예제에서 스레드 1에서 저장한 선호도를 스레드 2에서 읽어봅니다.

---

3. AGENTS.md를 통한 컨텍스트 주입

`memory` 파라미터를 사용하면, 에이전트가 시작될 때 AGENTS.md 파일을 자동으로 로드하여 시스템 프롬프트에 주입합니다.

AGENTS.md란?

에이전트에게 항상 적용되어야 하는 규칙, 컨벤션, 컨텍스트 정보를 담는 마크다운 파일입니다.

특징

- 에이전트가 시작할 때 항상 로드 (on-demand 아님)
- `<agent_memory>` 태그로 시스템 프롬프트에 주입
- 여러 소스 지정 가능 (배열)
- 에이전트가 `edit_file` 도구로 AGENTS.md를 스스로 업데이트 가능

Memory 스코프 패턴

스코프	namespace	용도
agent-scoped	`(assistant_id,)`	조직 공통 컨벤션, 도메인 지식 (사용자 간 공유)
user-scoped	`(user_id,)` 또는 `(assistant_id, user_id)`	개인화 – 운영 기본
context-driven	`lambda rt: (rt.context.user_id,)`	호출자가 user_id 를 직접 주입 (멀티 테넌트 SaaS)

```
from deepagents.backends import StoreBackend

# 컨텍스트로 user_id 를 주입하는 패턴 (멀티 테넌트)
user_scoped = StoreBackend(
    namespace=lambda rt: (rt.context.user_id,),
)
```

```
import tempfile

# 임시 디렉토리 생성 – FilesystemBackend의 root_dir로 사용
tmp_dir = tempfile.mkdtemp()
print(f"임시 디렉토리 생성: {tmp_dir}")
```

```
임시 디렉토리 생성: C:\Users\HEESU\AppData\Local\Temp\tmpj97x7phs
```

---

4. 스킬 (Skills)

스킬은 에이전트에게 도메인 전문 지식을 부여하는 모듈화된 지침 세트입니다.

Memory vs Skills

비교	Memory (AGENTS.md)	Skills (SKILL.md)
로딩	항상 로드 (Always)	필요 시 로드 (On-demand)
파일 형식	`AGENTS.md`	`SKILL.md` (YAML 프론트매터)
적합한 용도	항상 적용되는 규칙/컨벤션	특정 태스크에 필요한 큰 컨텍스트

토큰 효율	항상 소비	점진적 공개로 절약
크기	간결할수록 좋음	대용량 가능 (10MB 제한)
업데이트	에이전트가 edit_file로 수정 가능	보통 정적

Progressive Disclosure (점진적 공개)

스킬은 한 번에 전부 로드하지 않습니다:

1. 처음에는 프론트매터(이름, 설명)만 로드
2. 사용자 요청과 관련된 스킬을 에이전트가 판단
3. 필요한 스킬의 전체 내용을 그때 로드

이 방식으로 토큰을 절약하면서 필요한 전문 지식에 접근할 수 있습니다.

SKILL.md 구조

```yaml
---
name: web-research
description: >
  체계적인 웹 리서치를 수행하기 위한 단계별 가이드.
metadata:
  category: research
  include_tools: web_search summarize_page
---
```

`metadata.include_tools` 는 공백으로 구분한 문자열입니다. YAML 리스트가 아니며, Agent Skills 표준의 `allowed-tools` 와도 의미가 다릅니다. `allowed-tools` 는 승인 정책에 가깝고, `include_tools` 는 skill을 읽거나 pin했을 때 모델에게 추가로 공개할 도구 이름입니다.

스킬 도구를 사용하려면 `create_deep_agent(skills=[...])` 만으로 끝나지 않습니다. `SkillsMiddleware(backend=..., sources=[...], tools=[...])` 또는 resolver를 직접 구성해 skill이 요청한 도구를 연결합니다. 도구는 skill을 읽은 뒤 또는 입력 state의 `pinned_skills` 로 pin된 동안에만 공개됩니다.

스킬을 지원하는 백엔드

백엔드	시나리오
`StateBackend`	`invoke(files={...})` 와 `create_file_data()` 로 시드
`StoreBackend`	영속 namespace 에서 로드 (`InMemoryStore` , `PostgresStore`)
`FilesystemBackend`	에이전트 root 기준 디스크 읽기
`CompositeBackend`	스킬 파일은 `StoreBackend` , 실행은 샌드박스로 분리

SkillsMiddleware로 도구와 pinned skill 연결

`metadata.include_tools` 에 적은 이름은 실제 파이썬 함수와 자동으로 연결되지 않습니다. `SkillsMiddleware` 에 도구 목록을 넘기거나, 스킬 이름과 런타임을 받아 도구를 반환하는 resolver를 제공합니다. 특정 스킬을 항상 열어 두고 싶으면 생성자 옵션이 아니라 입력 state의 `pinned_skills` 에 넣습니다.

```
from pathlib import Path

skill_dir = Path(tmp_dir) / "skills" / "review-style"
skill_dir.mkdir(parents=True, exist_ok=True)
skill_text = "---\nname: review-style\ndescription: 코드 리뷰 톤과 체크리스트를 제공합니다.\nmetadata:\n
include_tools: summarize_code\n---\n보안, 정확성, 테스트를 우선 확인하세요.\n"
skill_dir.joinpath("SKILL.md").write_text(skill_text, encoding="utf-8")
print(skill_dir.joinpath("SKILL.md").read_text().split("---", 2)[1].strip())
```

```
from deepagents.middleware import SkillsMiddleware

def summarize_code(code: str) -> str:
    """코드 길이를 짧게 요약합니다."""
    return f"검토 대상 코드 길이: {len(code)}자"
skills_backend = FilesystemBackend(root_dir=tmp_dir, virtual_mode=True)
list_mw = SkillsMiddleware(backend=skills_backend, sources=["/skills/"], tools=[summarize_code])
skilled_agent = create_deep_agent(model=model, backend=skills_backend, middleware=[list_mw])
print("도구 목록 기반 SkillsMiddleware 생성 완료")
```

```
def skill_tool_resolver(skill_name, runtime):
    return [summarize_code] if skill_name == "review-style" else []

resolver_mw = SkillsMiddleware(backend=skills_backend, sources=["/skills/"], tools=skill_tool_resolver)
skill_run_input = {
    "messages": [{"role": "user", "content": "이 코드를 review-style로 검토해 주세요."}],
    "pinned_skills": ["review-style"],
}
print(skill_run_input["pinned_skills"])
```

```
# 스킬을 사용하는 에이전트 생성
skilled_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 시니어 개발자입니다. 사용 가능한 스킬을 활용하여 작업을 수행하세요.",
    backend=FilesystemBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],  # 스킬 소스 디렉토리
)

print("스킬 에이전트 생성 완료")
```

```
스킬 에이전트 생성 완료
```

---

5. 스킬 소스 우선순위

여러 스킬 소스를 지정하면, 나중에 나온 소스가 우선합니다 (last wins).

```
skills=[
    "/skills/base/",     # 기본 스킬
    "/skills/user/",     # 사용자 스킬 (base 덮어쓰기 가능)
    "/skills/project/",  # 프로젝트 스킬 (최우선)
]
```

같은 이름의 스킬이 여러 소스에 있으면, 마지막 소스의 스킬이 사용됩니다.

---

6. 서브에이전트의 스킬 상속

서브에이전트 유형	스킬 상속
General-purpose (빌트인)	메인 에이전트의 스킬을 자동 상속
커스텀 SubAgent	명시적 `skills` 파라미터 필요

```
# 커스텀 서브에이전트에 스킬 부여
subagent = {
    "name": "reviewer",
    "description": "코드 리뷰 전문 에이전트",
    "system_prompt": "...",
    "tools": [],
    "skills": ["/skills/code-review/"],  # 명시적 스킬 경로
}
```

핵심 정리

Skills vs Memory 레이어링

비교	Memory (AGENTS.md)	Skills (SKILL.md)
로딩	항상 주입 (always-on)	Progressive disclosure (on-demand)
형식	`AGENTS.md`	`SKILL.md` + frontmatter (`name` , `description` , optional `metadata.include_tools`)
레이어링	사용자 + 프로젝트 결합	마지막 소스 우선 (last wins)
적합 용도	항상 적용되는 컨벤션	대용량 task-specific 컨텍스트와 gated tools
토큰 효율	항상 소비	필요 시 로드
업데이트	에이전트가 `edit_file` 로 수정 가능	보통 정적

요약

항목	내용
장기 메모리	`CompositeBackend` + `StoreBackend(namespace=...)` 로 `/memories/` 영속화
프로덕션 store	`from langgraph.store.postgres import PostgresStore`
namespace 패턴	`lambda rt: (rt.context.user_id,)` — 컨텍스트 주입 user_id

AGENTS.md	`memory=["/path/AGENTS.md"]` → 항상 시스템 프롬프트에 주입
Skills	`skills=["/skills/"]` 또는 직접 구성한 `SkillsMiddleware`
skill tools	`metadata.include_tools: tool_a tool_b` + `SkillsMiddleware(tools=...)`
pinned skills	`agent.invoke({"messages": [...], "pinned_skills": ["skill-name"]}, config)` 입력 state로 전달
정보 유형	Episodic (checkpointer) / Procedural (skills) / Semantic (store)

07 고급 기능

학습 목표

LEARNING OBJECTIVES

- → Human-in-the-Loop 워크플로를 구현한다
- → 다양한 스트리밍 모드와 네임스페이스 시스템을 이해한다
- → 샌드박스(Modal, Daytona, Runloop) 연동 개념을 파악한다
- → `CodeInterpreterMiddleware` 와 QuickJS interpreter의 역할을 이해한다
- → `RubricMiddleware` 로 런타임 평가 게이트를 구성한다
- → ACP(Agent Client Protocol)로 에디터와 연동하는 방법을 안다
- → Deep Agents Code(`dcode`) 사용법을 익힌다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

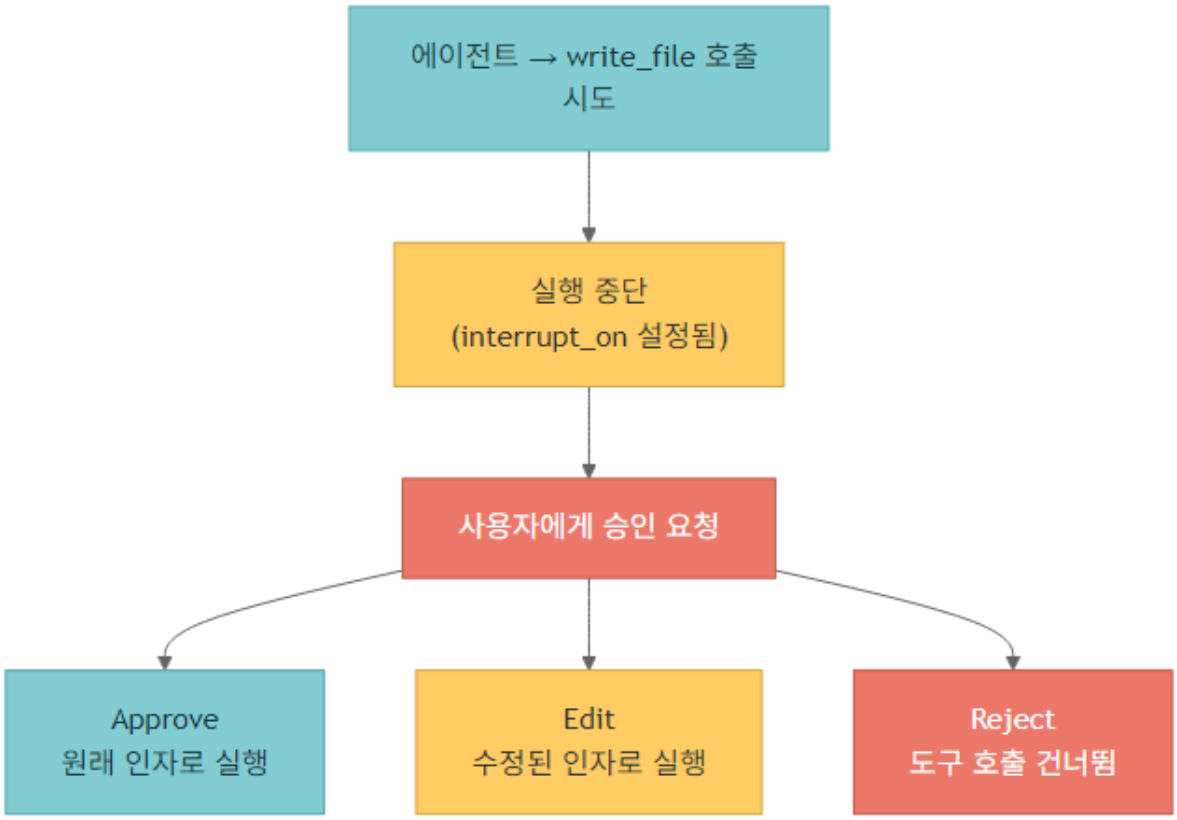
print(f"모델 설정 완료: {model.model_name}")
```

```
모델 설정 완료: gpt-4.1
```

1. Human-in-the-Loop (HITL)

에이전트가 민감한 도구를 호출할 때 사람의 승인을 요구하는 워크플로입니다.

작동 방식



4가지 결정 타입 (`Command(resume={"decisions": [...]})`)

결정	동작
`approve`	제안된 인자 그대로 도구 실행
`edit`	`edited_action.name/args` 로 인자를 수정한 뒤 실행
`reject`	호출 자체를 건너뜀
`respond`	도구 실행 없이 `message` 를 도구 결과로 반환

필수 요구사항

- Checkpointer: 중단/재개 사이의 에이전트 상태를 유지하기 위해 반드시 필요 (`MemorySaver` 등)
- `version="v2"` : invoke/stream 호출에 모두 지정 — interrupt 지원의 표준 경로
- 동일한 `thread_id` : 초기 invoke와 resume invoke가 같은 `thread_id` 를 공유해야 함

`interrupt_on` 설정 옵션

```
interrupt_on = {
    "tool_name": True,                                      # 4가지 결정 모두 허용
    "tool_name": False,                                     # 인터럽트 없음
    "tool_name": {"allowed_decisions": ["approve", "reject"]},  # 부분 집합
}
```

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

# interrupt_on으로 승인이 필요한 도구 지정
hitl_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 파일 관리 어시스턴트입니다. 한국어로 응답하세요.",
    checkpointer=MemorySaver(),  # 필수!
    interrupt_on={
        "write_file": True,   # 파일 쓰기 전 승인 필요
        "edit_file": True,    # 파일 편집 전 승인 필요
    },
)

print("Human-in-the-Loop 에이전트 생성 완료")
print("write_file, edit_file 호출 시 승인을 요구합니다.")
```

```
Human-in-the-Loop 에이전트 생성 완료
write_file, edit_file 호출 시 승인을 요구합니다.
```

---

2. 스트리밍 심화

Deep Agents는 LangGraph의 스트리밍 인프라 위에서 작동합니다.

v2 통합 포맷 (표준)

`agent.stream(..., stream_mode=..., subgraphs=True, version="v2")` 한 경로가 표준입니다. 모든 청크는 동일한 3-필드 구조입니다.

```
{
    "type": "updates" | "messages" | "custom",
    "ns":   tuple,            # 이벤트 발생 위치(메인/서브에이전트 라우팅 키)
    "data": Any,              # type 별 payload
}
```

스트림 모드

모드	data	사용 시나리오
`"updates"`	`{node_name: state_update}`	노드/스텝 진행 추적
`"messages"`	`(token, metadata)`	토큰 스트리밍, `tool_call_chunks` 조립
`"custom"`	도구가 `get_stream_writer()` 로 방출한 객체	커스텀 진행률/상태

리스트(`stream_mode=["updates", "messages", "custom"]`)를 주면 세 모드를 한 루프에서 받습니다.

네임스페이스 시스템

`ns` 튜플이 이벤트 소스를 식별합니다.

튜플	의미
`()`	메인 에이전트
`("tools:abc123",)`	`task` 도구가 스폰한 서브에이전트
`("tools:abc123", "model_request:def456")`	해당 서브에이전트 내부 노드

서브에이전트 이벤트 판정:

```
is_subagent = any(seg.startswith("tools:") for seg in chunk["ns"])
```

```
from typing import Literal

LOCAL_DOC_SNIPPETS = {
    "langgraph": "LangGraph supports persistence, interrupts, subgraphs, streaming, and fault
tolerance.",
    "python 3.13": "Python 3.13 improves the REPL, error messages, and standard-library behavior.",
}


def internet_search(query: str, max_results: int = 3, topic: Literal["general", "news"] = "general") ->
dict:
    """로컬 문서 스니펫을 검색합니다. 라이브 harness에서는 외부 검색 키가 필요 없습니다."""
    matches = [v for k, v in LOCAL_DOC_SNIPPETS.items() if k in query.lower()]
    return {"query": query, "topic": topic, "results": matches[:max_results] or
list(LOCAL_DOC_SNIPPETS.values())[:max_results]}


# 서브에이전트 포함 에이전트
stream_agent = create_deep_agent(
    model=model,
    system_prompt="당신은 리서치 코디네이터입니다. 한국어로 응답하세요.",
    subagents=[
        {
            "name": "researcher",
            "description": "로컬 문서 스니펫을 통해 정보를 조사합니다.",
            "system_prompt": "검색 도구 결과만 근거로 요청된 정보를 간결하게 요약하세요.",
            "tools": [internet_search],
        }
    ],
)

print("스트리밍 데모 에이전트 생성 완료")
```

```
스트리밍 데모 에이전트 생성 완료
```

커스텀 진행률 이벤트 (`get_stream_writer` + `stream_mode="custom"`)

도구 내부에서 임의 구조체를 방출해 UI에 노출합니다. 업로드 진행률·처리 건수·중간 상태 등에 활용합니다.

```
from langchain.tools import tool
from langgraph.config import get_stream_writer


@tool
def analyze_data(topic: str) -> str:
```

```
    """주제 데이터를 분석하고 진행률을 스트리밍합니다."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0, "topic": topic})
    # ... 실제 분석 작업 ...
    writer({"status": "complete", "progress": 100, "topic": topic})
    return f"분석 완료: {topic}"


custom_example = r'''
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "..."}]},
    stream_mode="custom",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])  # {"status": ..., "progress": ...}
'''
print(custom_example)
```

3. 샌드박스 (Sandbox)

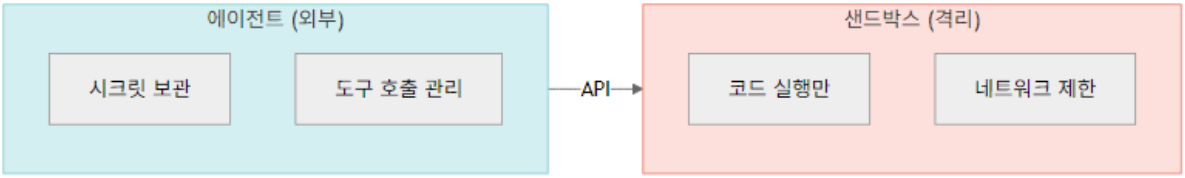
샌드박스는 에이전트가 격리된 환경에서 코드를 실행할 수 있게 합니다. 호스트 시스템의 파일, 네트워크, 자격 증명에 접근하지 못하므로 안전합니다.

지원 프로바이더

프로바이더	특징	적합한 용도
Modal	GPU 지원, ML 워크로드	AI/ML 작업
Daytona	TypeScript/Python, 빠른 콜드 스타트	웹 개발
Runloop	일회용 devbox, 격리 실행	코드 테스트

아키텍처 패턴

샌드박스를 도구로 사용 (권장)



⚠️ 보안 주의사항

- 절대 샌드박스 안에 시크릿을 넣지 마세요 — 에이전트가 유출할 수 있습니다
- 자격 증명은 외부 도구에서만 관리
- Human-in-the-Loop으로 민감한 작업 승인
- 불필요한 네트워크 접근 차단

```
# 샌드박스 연동 코드 예시 — Modal (docs/deepagents/11-sandboxes.md)
sandbox_example_code = '''
# pip install langchain-modal deepagents
import modal
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox

app = modal.App.lookup("your-app")
modal_sandbox = modal.Sandbox.create(app=app)
backend = ModalSandbox(sandbox=modal_sandbox)

agent = create_deep_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="You are a Python coding assistant with sandbox access.",
    backend=backend,
)

try:
    result = agent.invoke({
        "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
    })
finally:
    modal_sandbox.terminate()  # 필수: 리소스 해제
'''

print("샌드박스 연동 코드 예시 (참고용):")
print(sandbox_example_code)
```

```
샌드박스 연동 코드 예시 (참고용):

# pip install deepagents-modal
from deepagents.backends.sandbox import ModalSandbox

agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    backend=ModalSandbox(
        image="python:3.12-slim",
        gpu="T4",  # GPU 지원
    ),
)
```

---

4. Interpreters — CodeInterpreterMiddleware

Deep Agents 0.6의 interpreter는 에이전트 루프 안에 QuickJS 기반 코드 실행 공간을 제공합니다. 샌드박스가 외부 환경을 조작하는 격리 실행이라면, interpreter는 도구 호출 조합·중간 변수 보관·구조화 데이터 변환을 모델 컨텍스트 밖에서 수행하는 내부 작업 공간입니다.

설치

```
pip install -U "deepagents[quickjs]"
# 또는
uv add "deepagents[quickjs]"
```

언제 쓰나

용도	예

도구 호출 조합	검색 결과 20개를 loop로 점수화
서브에이전트 fan-out/fan-in	후보별 `task(...)` 호출 후 코드로 합성
구조화 데이터 처리	JSON 정렬·그룹화·검증
컨텍스트 절약	큰 중간 결과는 interpreter 변수에 유지

Programmatic Tool Calling (PTC)와 programmatic subagents

PTC는 `ptc=["web_search"]` 처럼 allowlist를 지정한 일반 도구를 QuickJS 내부의 `tools.*` async 함수로 노출합니다. 도구 이름은 camelCase로 변환됩니다(예: `web_search` → `tools.webSearch`).

서브에이전트 호출은 PTC allowlist가 아니라 interpreter의 top-level `task(...)` 함수로 다룹니다. `CodeInterpreterMiddleware(subagents=True)` 가 기본값이며, interpreter 내부에서 `Promise.all` 로 병렬 위임 패턴을 만들 수 있습니다.

```
const topics = ["retrieval", "memory", "evaluation"];
const reports = await Promise.all(
  topics.map((topic) => task({
    description: `Research ${topic}`,
    subagent_type: "general-purpose",
  })),
);
```

민감 도구를 PTC로 노출할 때는 parent-level `interrupt_on` 이 개별 interpreter dispatch마다 자동 적용된다고 가정하지 말고, 최소 권한 allowlist를 둡니다.

`CodeInterpreterMiddleware` 주요 옵션

Parameter	Default	Purpose
`memory_limit`	`64*1024*1024` (64 MB)	QuickJS heap memory limit (bytes)
`timeout`	`5.0`	Per-eval 타임아웃(초)
`max_ptc_calls`	`256`	한 eval에서 허용되는 `tools.*` 호출 수
`tool_name`	`"eval"`	Interpreter 도구 이름
`max_result_chars`	`4000`	반환 결과 최대 문자 수
`capture_console`	`True`	`console.log` 출력 캡처
`subagents`	`True`	top-level `task(...)` 함수 노출 여부
`ptc`	`None`	PTC allowlist
`mode`	`None`	`thread` 등 스냅샷/상태 보존 모드
`snapshot_between_turns`	`None`	명시적 turn 간 snapshot 제어
`max_snapshot_bytes`	`None`	snapshot 최대 크기

```
# Interpreter 구성 예시 — deepagents[quickjs] extra 필요
interpreter_example = r'''
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.4",
    checkpointer=MemorySaver(),
    middleware=[CodeInterpreterMiddleware(ptc=["web_search"], subagents=True, mode="thread")],
)

# QuickJS 안에서는 task(...)와 tools.webSearch(...)를 분리해서 사용합니다.
'''
print(interpreter_example)
```

현재 설치 버전에서의 Programmatic Subagents 호환성 gate

공식 Programmatic Subagents 패턴은 QuickJS interpreter 안의 top-level `task(...)` 함수를 사용합니다. 이 저장소의 기본 venv는 `deepagents==0.6.10` 이지만 `langchain_quickjs` extra가 설치되어 있지 않으므로, 기본 실행 경로에서는 기능 감지 → 일반 `SubAgent` fan-out/fan-in fallback 순서로 설명합니다.

이렇게 두면 학습자는 공식 패턴의 위치를 이해하면서도, 기본 gpt-4.1 실행 harness는 추가 QuickJS 의존성 없이 통과합니다.

```
from importlib.metadata import PackageNotFoundError, version
from importlib.util import find_spec

try:
    deepagents_version = version("deepagents")
except PackageNotFoundError:
    deepagents_version = "not installed"
quickjs_available = find_spec("langchain_quickjs") is not None
fallback_pattern = "Programmatic task(...) demo" if quickjs_available else "ordinary SubAgent fan-out/
fan-in fallback"
print("deepagents:", deepagents_version)
print("langchain_quickjs available:", quickjs_available)
print("recommended path:", fallback_pattern)
```

---

5. RubricMiddleware — 런타임 LLM-as-a-judge

`RubricMiddleware` 는 에이전트 실행 결과를 rubric 기준으로 평가하고, 기준을 만족하지 못하면 같은 invocation 안에서 추가 수정을 유도하는 미들웨어입니다. 오프라인 평가는 LangSmith/agentevals로 회귀를 막고, 런타임 rubric은 실제 요청마다 "이번 답변이 기준을 지켰는가"를 확인하는 방어선으로 둡니다.

항목	설명
`model`	평가를 맡을 judge 모델
`system_prompt`	품질 기준과 실패 시 수정 지시
`max_iterations`	평가→수정 반복 상한
`on_evaluation`	평가 결과 로깅/모니터링 콜백

주의할 점은 비용과 지연입니다. 모든 요청에 무조건 붙이기보다, 고위험 작업·최종 산출물·외부 전송 직전처럼 품질 게이트가 필요한 지점에 선택 적용합니다.

```
# RubricMiddleware 구성 예시 — 실행 전 품질 기준을 먼저 코드로 고정
rubric_example = r'''
from deepagents import RubricMiddleware, create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

agent = create_deep_agent(
    model="openai:gpt-5.4",
    checkpointer=MemorySaver(),
    middleware=[RubricMiddleware(
        model="openai:gpt-5.4-mini",
        system_prompt="정확성, 근거, 안전성을 1~5점으로 평가하고 부족하면 수정 지시",
        max_iterations=3,
    )],
)
'''
print(rubric_example)
```

---

6. ACP (Agent Client Protocol)

ACP는 코딩 에이전트와 에디터/IDE 간의 통신을 표준화하는 프로토콜입니다.

지원 에디터

- Zed — 네이티브 통합
- JetBrains IDEs — 빌트인 지원
- VS Code — vscode-acp 플러그인
- Neovim — ACP 호환 플러그인

MCP vs ACP

프로토콜	용도
MCP (Model Context Protocol)	외부 도구 통합
ACP (Agent Client Protocol)	에디터-에이전트 통합

```
# ACP 서버 구현 예시 (참고용) — docs/deepagents/14-acp.md
acp_example_code = '''
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP


async def main() -> None:
    agent = create_deep_agent(
        model="google_genai:gemini-3.5-flash",
        system_prompt="You are a helpful coding assistant",
        checkpointer=MemorySaver(),
```

```
    )
    server = AgentServerACP(agent)
    await run_agent(server)


if __name__ == "__main__":
    asyncio.run(main())
'''

print("ACP 서버 구현 예시 (참고용):")
print(acp_example_code)

print("Toad로 띄우기: uv tool install -U batrachian-toad && toad acp 'python acp_server.py' .")
```

```
ACP 서버 구현 예시 (참고용):

# pip install deepagents-acp
from deepagents import create_deep_agent
from deepagents_acp import AgentServerACP
from langgraph.checkpoint.memory import MemorySaver

# 에이전트 생성
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt="당신은 코딩 어시스턴트입니다.",
    checkpointer=MemorySaver(),
)

# ACP 서버 실행 (stdio 모드)
server = AgentServerACP(agent)
server.run()
```

---

7. Deep Agents Code (`dcode`)

SDK 위에 구축된 터미널 코딩 에이전트입니다. 현재 CLI 문서는 `deepagents-code` 패키지와 `dcode` 명령을 기준으로 설명합니다.

설치 및 실행

```
# 설치 스크립트
curl -LsSf https://langch.in/dcode | bash

# 설치 없이 도움말 확인
uvx --from deepagents-code dcode --help

# 실행
dcode
```

주요 옵션

옵션	설명
`-M/--model MODEL`	모델 선택
`-n/--non-interactive`	비대화형 모드 (단일 태스크 실행)
`-S/--shell-allow-list`	허용할 셸 명령 지정

`--interpreter`	interpreter 활성화
`--interpreter-tools`	interpreter에서 사용할 도구 지정
`--stdin`	표준 입력에서 prompt 읽기
`--json`	명령 출력 JSON 형식
`--acp`	ACP server over stdio로 실행

인터랙티브 명령어

명령	설명
`/model`	모델 변경
`/remember`	메모리에 정보 저장
`/tokens`	토큰 사용량 확인
`!command`	셸 명령 실행

설정과 메모리

- 글로벌 설정: `~/.deepagents/config.toml`
- 프로젝트 지침: `AGENTS.md`
- 스킬: `SKILL.md` 기반 progressive disclosure
- 서브에이전트: 설정 파일 또는 프로젝트 지침으로 역할 정의

```
# Deep Agents Code 비대화형 모드 예시 (셸에서 실행)
dcode_examples = """
# 기본 사용
dcode

# 특정 모델로 비대화형 실행
dcode -M openai:gpt-5.4 -n "이 프로젝트의 README.md를 검토해 줘"

# 셸 허용 목록과 함께 실행
dcode -S "pytest,python,rg" -n "테스트 실패 원인을 찾아 줘"
"""

print("dcode 사용 예시 (터미널에서 실행):")
print(dcode_examples)
```

전체 교육 자료 정리

노트북	주제	핵심 API
01	소개	`deepagents.__version__`
02	퀵스타트	`create_deep_agent()`, `invoke()`, `stream()`

03	커스터마이징	`model` , `system_prompt` , `tools` , `response_format`
04	백엔드	`StateBackend` , `FilesystemBackend` , `StoreBackend` , `CompositeBackend`
05	서브에이전트	`SubAgent` , `CompiledSubAgent` , `subagents`
06	메모리 & 스킬	`memory` , `skills` , `AGENTS.md` , `SKILL.md`
07	고급 기능	`interrupt_on` , streaming, `CodeInterpreterMiddleware` , `RubricMiddleware` , Sandbox, ACP, `dcode`

Context Engineering 보강 (docs/deepagents/14-context-engineering.md)

- `@dynamic_prompt` 미들웨어로 요청 시점 데이터(`request.runtime.context` , `request.runtime.store`)를 시스템 프롬프트에 주입
- `context_schema` 로 `user_id` / `org_id` 를 선언하면 모든 서브에이전트·도구에 자동 전파
- 도구는 `ToolRuntime[Context]` 를 받아 `runtime.context.user_id` 로 런타임 값 조회

Permissions (docs/deepagents/16-permissions.md, `deepagents\>=0.5.2`)

- `FilesystemPermission(operations=["read"|"write"], paths=[...], mode="allow"|"deny")`
- first-match-wins 평가, 매치 없으면 기본 `allow`
- built-in FS 도구에만 적용 (커스텀 도구·MCP·샌드박스 `execute` 는 우회)
- 서브에이전트 `permissions` 는 부모 규칙을 전면 대체 (부분 오버라이드 아님)

추가 업데이트 포인트

- `task(...)` 는 interpreter의 top-level 함수이며, PTC의 `tools.*` allowlist와 분리해 설명합니다.
- 현재 기본 venv에서는 `langchain_quickjs` 를 감지한 뒤 없으면 일반 `SubAgent` fallback 경로를 사용합니다.
- Deep Agents CLI 명령은 `deepagents-cli` 가 아니라 Deep Agents Code의 `dcode` 를 기준으로 사용합니다.

08 에이전트 하네스

학습 목표

LEARNING OBJECTIVES

- → AgentHarness의 개념과 역할을 이해한다
- → 하네스의 핵심 기능(계획, 파일시스템, 태스크 위임)을 안다
- → 컨텍스트 관리(오프로딩, 요약)를 이해한다
- → 코드 실행과 Human-in-the-Loop을 설정한다
- → 스킬과 메모리 시스템을 연동한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

```
모델 설정 완료: gpt-4.1
```

1. AgentHarness 개념

AgentHarness는 장기 실행 자율 에이전트를 위한 포괄적 기능 제공자입니다. 에이전트가 복잡한 멀티 스텝 작업을 수행할 때 필요한 인프라를 하나로 묶어 제공합니다.

하네스가 제공하는 핵심 기능

기능	설명
Filesystem	가상/로컬 파일 읽기·쓰기·검색 (멀티모달 `read_file` 포함)
Task Delegation	서브에이전트를 통한 격리·병렬·전문화 작업 위임
Context Management	설정 가능한 임계값 기반 오프로딩 + 요약 압축
Code Execution	샌드박스 백엔드의 `execute` 셸 도구 또는 QuickJS interpreter
Human-in-the-Loop	민감 작업에 대한 사람 승인
Skills & Memory	전문 워크플로(Agent Skills 표준) + 영속적 컨텍스트(`AGENTS.md`)
Planning	필요할 때 `TodoListMiddleware` 를 추가해 `write_todos` 노출

`create_deep_agent()` 는 파일·서브에이전트·요약·skills/memory 계층을 조립합니다. 계획 도구는 자동 전제가 아니라 수업 목적에 맞게 명시적으로 켭니다.

```
from langchain.agents.middleware import TodoListMiddleware

harness_config = {
    "model": "gpt-5.4",
    "filesystem": "built-in file tools",
    "subagents": "task delegation",
    "planning": TodoListMiddleware.__name__,
}

for key, value in harness_config.items():
    print(f"{key}: {value}")
```

---

2. 계획 도구

계획이 필요한 예제에서는 LangChain의 `TodoListMiddleware` 를 `middleware` 목록에 추가합니다. 그러면 에이전트가 `write_todos` 도구로 복잡한 작업을 구조화된 태스크 리스트로 분해할 수 있습니다.

```
from langchain.agents.middleware import TodoListMiddleware

agent = create_deep_agent(
    model=model,
    middleware=[TodoListMiddleware()],
)
```

각 태스크는 상태를 가집니다:

상태	설명
`pending`	아직 시작하지 않음
`in_progress`	현재 진행 중
`completed`	완료됨

```python
# write_todos 도구 — 구조화된 태스크 리스트 예시
todo_list = [
    {"task": "프로젝트 구조 분석", "status": "completed"},
    {"task": "API 엔드포인트 설계", "status": "in_progress"},
    {"task": "데이터베이스 스키마 작성", "status": "pending"},
    {"task": "테스트 코드 작성", "status": "pending"},
    {"task": "문서화", "status": "pending"},
]

print("=== 에이전트 태스크 리스트 ===")
for i, item in enumerate(todo_list, 1):
    icon = {"completed": "[x]", "in_progress": "[-]", "pending": "[ ]"}
    print(f"  {icon[item['status']]} {i}. {item['task']}")
```

```
=== 에이전트 태스크 리스트 ===
  [x] 1. 프로젝트 구조 분석
  [-] 2. API 엔드포인트 설계
  [ ] 3. 데이터베이스 스키마 작성
  [ ] 4. 테스트 코드 작성
  [ ] 5. 문서화
```

---

3. 가상 파일시스템

하네스는 구성 가능한 파일시스템 백엔드를 통해 표준 파일 작업을 지원합니다.

도구	설명
`ls`	디렉토리 목록 (size, modified time 메타데이터 포함)
`read_file`	줄 번호와 offset/limit 지원, 멀티모달 반환: 이미지(PNG/JPG/GIF/WebP/HEIC), 비디오(MP4/MOV/AVI), 오디오(WAV/MP3/AAC/FLAC), 문서(PDF/PPT)
`write_file`	파일 생성
`edit_file`	exact string replacement (옵션: global replace)
`glob`	패턴 기반 파일 검색 (예: `**/*.py`)
`grep`	내용 검색, 다양한 출력 모드
`execute`	셸 명령 실행 (sandbox 백엔드 전용)

```python
# 파일시스템 도구 사용 예시 (참고용)
fs_operations = {
    "ls": 'ls(path="/project/src")',
    "read_file": 'read_file(path="/project/src/main.py")',
    "write_file": 'write_file(path="/project/config.yaml", content="debug: true")',
    "edit_file": 'edit_file(path="/project/src/main.py", old="v1", new="v2")',
    "glob": 'glob(pattern="**/*.py")',
    "grep": 'grep(pattern="TODO", path="/project/src")',
}

print("=== 파일시스템 도구 호출 예시 ===")
for tool_name, call_example in fs_operations.items():
    print(f"  {tool_name:12s} -> {call_example}")
```

```
=== 파일시스템 도구 호출 예시 ===
  ls           -> ls(path="/project/src")
  read_file    -> read_file(path="/project/src/main.py")
  write_file   -> write_file(path="/project/config.yaml", content="debug: true")
  edit_file    -> edit_file(path="/project/src/main.py", old="v1", new="v2")
  glob         -> glob(pattern="**/*.py")
  grep         -> grep(pattern="TODO", path="/project/src")
```

---

4. 태스크 위임 — 서브에이전트

하네스는 메인 에이전트가 임시 서브에이전트를 생성하여 격리된 멀티 스텝 태스크를 수행할 수 있게 합니다.

서브에이전트의 장점

장점	설명
컨텍스트 격리	서브에이전트 실행이 메인 컨텍스트를 오염시키지 않음
병렬 실행	여러 서브에이전트를 동시에 실행 가능
전문화	각 서브에이전트에 특화된 도구와 프롬프트 제공
토큰 효율	결과 압축으로 메인 에이전트의 토큰 절약

```python
# 서브에이전트 위임 구성 예시 (참고용)
subagent_config = [
    {
        "name": "researcher",
        "description": "인터넷 검색으로 정보를 조사합니다.",
        "system_prompt": "검색 결과를 간결하게 요약하세요.",
        "tools": ["internet_search"],
    },
    {
        "name": "coder",
        "description": "코드를 작성하고 테스트합니다.",
        "system_prompt": "깔끔하고 테스트 가능한 코드를 작성하세요.",
        "tools": ["write_file", "execute"],
    },
]

print("=== 서브에이전트 구성 ===")
for sa in subagent_config:
    print(f"  [{sa['name']}] {sa['description']}")
    print(f"    도구: {', '.join(sa['tools'])}")
```

```
=== 서브에이전트 구성 ===
  [researcher] 인터넷 검색으로 정보를 조사합니다.
    도구: internet_search
  [coder] 코드를 작성하고 테스트합니다.
    도구: write_file, execute
```

---

5. 컨텍스트 관리

장기 실행 에이전트의 가장 큰 과제는 컨텍스트 윈도우 한계입니다. 하네스는 두 가지 기법으로 이를 해결합니다.

입력 컨텍스트 조립

시스템 프롬프트, 지침, 메모리 가이드라인, 스킬 정보, 파일시스템 문서를 종합하여 초기 프롬프트를 구성합니다.

런타임 컨텍스트 압축

기법	동작	트리거
오프로딩	큰 도구 결과를 디스크에 저장, 활성 메모리에 파일 포인터 + 프리뷰만 유지	configurable threshold (예: 20,000 토큰)
요약	대화 히스토리를 구조화 요약(session intent / artifacts / next steps)으로 압축	모델 `max_input_tokens` 의 85% 또는 `ContextOverflowError`

원본 메시지는 파일시스템 스토리지에 보존되므로 정보 손실이 없습니다. 자세한 정책은 `docs/deepagents/14-context-engineering.md` 참조.

```python
# 컨텍스트 관리 설정 예시 (참고용) — threshold는 configurable
context_config = {
    "offloading": {
        "enabled": True,
        "threshold_tokens": 20000,  # configurable
        "storage": "filesystem",
    },
    "summarization": {
        "enabled": True,
        "trigger_ratio": 0.85,        # max_input_tokens의 85%
        "recent_message_keep": 0.10,  # 최근 10% 보존
        "fallback_trigger_tokens": 170000,
    },
}

print("=== 컨텍스트 관리 설정 ===")
for section, settings in context_config.items():
    print(f"\n[{section}]")
    for key, value in settings.items():
        print(f"  {key}: {value}")
```

```
=== 컨텍스트 관리 설정 ===

[offloading]
  enabled: True
  threshold_tokens: 20000
  storage: filesystem

[summarization]
  enabled: True
  trigger: window_limit_approach
  preserve_original: True
```

6. 코드 실행

하네스는 두 가지 코드 실행 경로를 지원합니다.

Sandbox 백엔드 — `execute` 셸 도구

샌드박스 백엔드(Modal/Daytona/Runloop/LangSmith/AgentCore 등)를 backend로 주면 `execute` 도구가 노출됩니다. 격리된 환경에서 임의 셸 명령을 실행합니다.

QuickJS Interpreter — `CodeInterpreterMiddleware`

`langchain-quickjs`의 `CodeInterpreterMiddleware`를 추가하면 에이전트 루프 안에 QuickJS 기반 코드 실행 공간이 생깁니다. 셸/네트워크 접근 없이 결정적 도구 조합·데이터 변환에 적합합니다.

구분	Sandbox	Interpreter
실행 위치	외부 격리 런타임/컨테이너	에이전트 루프 내부 QuickJS
주 용도	파일·패키지·셸·데이터 분석	도구 조합·중간 상태·구조화 변환
보안 경계	provider 정책	QuickJS + PTC allowlist
컨텍스트 절약	실행 결과만 모델로 반환	변수 공간을 interpreter에 유지

설치: `pip install -U "deepagents[quickjs]"`. 자세한 옵션 10가지와 PTC는 `docs/deepagents/17-interpreters.md` 참조.

```python
# 코드 실행 예시 (참고용)

# 1) Sandbox execute 도구
execute_examples = [
    {"command": "python -c 'print(2+2)'", "desc": "Python 코드 실행"},
    {"command": "pip install requests",      "desc": "패키지 설치"},
    {"command": "pytest tests/",              "desc": "테스트 실행"},
]

print("=== Sandbox execute 도구 예시 ===")
for ex in execute_examples:
    print(f"  $ {ex['command']}")
    print(f"    -> {ex['desc']}")

# 2) QuickJS Interpreter — CodeInterpreterMiddleware
interpreter_snippet = r'''
from deepagents import create_deep_agent
from langchain_quickjs import CodeInterpreterMiddleware

agent = create_deep_agent(
    model="openai:gpt-5.4",
    middleware=[CodeInterpreterMiddleware(ptc=["task"])],  # task만 PTC 노출
)
'''
print()
print("=== QuickJS Interpreter 예시 ===")
print(interpreter_snippet)
```

```
=== 샌드박스 execute 도구 예시 ===
  $ python -c 'print(2+2)'
    -> Python 코드 실행
  $ pip install requests
    -> 패키지 설치
```

```
  $ pytest tests/
    -> 테스트 실행
```

---

7. Human-in-the-Loop

선택적 인터럽트 설정으로 지정된 도구 호출 시 사람의 승인을 요구합니다.

```python
# Human-in-the-Loop 설정 예시 (참고용)
hitl_config = {
    "interrupt_on": {
        "write_file": True,   # 파일 쓰기 전 승인
        "edit_file": True,    # 파일 편집 전 승인
        "execute": True,      # 명령 실행 전 승인
    }
}

print("=== Human-in-the-Loop 설정 ===")
print("승인이 필요한 도구:")
for tool, enabled in hitl_config["interrupt_on"].items():
    status = "승인 필요" if enabled else "자동 실행"
    print(f"  {tool}: {status}")

print("\n승인 옵션: approve(승인), reject(거부), edit(수정)")
```

```
=== Human-in-the-Loop 설정 ===
승인이 필요한 도구:
  write_file: 승인 필요
  edit_file: 승인 필요
  execute: 승인 필요

승인 옵션: approve(승인), reject(거부), edit(수정)
```

---

8. 스킬과 메모리

스킬 (Skills)

Agent Skills 표준을 따르는 전문 워크플로입니다. 관련성이 있을 때 점진적으로 로드되어 토큰 소비를 줄입니다.

- 각 스킬은 `SKILL.md` 파일로 정의
- 트리거 조건에 따라 자동 활성화
- 도구, 프롬프트, 워크플로를 캡슐화

메모리 (Memory)

AGENTS.md 형식의 영속적 컨텍스트 파일입니다. 대화를 넘어서 재사용 가능한 가이드라인, 선호도, 프로젝트 지식을 제공합니다.

구분	위치	범위
글로벌 메모리	`~/.deepagents/\<agent\>/memories/`	모든 프로젝트

프로젝트 메모리	`.deepagents/AGENTS.md`	현재 프로젝트

```
# 스킬과 메모리 설정 예시 (참고용)
skills_config = [
    {"name": "code-review", "trigger": "코드 리뷰 요청 시"},
    {"name": "test-writer", "trigger": "테스트 작성 요청 시"},
    {"name": "doc-generator", "trigger": "문서화 요청 시"},
]

memory_config = {
    "global": "~/.deepagents/my-agent/memories/",
    "project": ".deepagents/AGENTS.md",
}

print("=== 스킬 설정 ===")
for skill in skills_config:
    print(f"  [{skill['name']}] 트리거: {skill['trigger']}")

print("\n=== 메모리 설정 ===")
for scope, path in memory_config.items():
    print(f"  {scope}: {path}")
```

```
=== 스킬 설정 ===
  [code-review] 트리거: 코드 리뷰 요청 시
  [test-writer] 트리거: 테스트 작성 요청 시
  [doc-generator] 트리거: 문서화 요청 시

=== 메모리 설정 ===
  global: ~/.deepagents/my-agent/memories/
  project: .deepagents/AGENTS.md
```

요약

주제	핵심 개념	핵심 API/도구
하네스 개념	장기 실행 에이전트를 위한 기능 조립	`create_deep_agent()`
계획 도구	opt-in 태스크 리스트 관리	`TodoListMiddleware`, `write_todos`
파일시스템	가상/로컬 파일 작업 + 멀티모달 read_file	`ls`, `read_file`, `write_file`, `edit_file`, `delete`, `glob`, `grep`, `execute`
서브에이전트	기본 isolated 위임, 실험적 fork 문맥 상속	`subagents`, `task`, `mode="fork"`
컨텍스트 관리	configurable threshold 오프로딩 + 요약	middleware stack
코드 실행	Sandbox `execute` + QuickJS `CodeInterpreterMiddleware`	`execute`, `eval`
HITL	approve/edit/reject/respond 재개	`interrupt_on`, `Command(resume=...)`

스킬/메모리	전문 워크플로 + 영속적 컨텍스트	`SKILL.md`, `metadata.include_tools`, `pinned_skills`, `AGENTS.md`

Profiles 보강

Profile은 provider/model별 도구 설명, 제외 도구, 추가 middleware, 프롬프트 suffix를 묶습니다. Profile이 조정하는 것은 실행 환경이고, Rubric은 결과물이 충족해야 할 완료 기준입니다.

> REFERENCES
>
> - Deep Agents Harness
> - Interpreters
> - Profiles
> - Context Engineering

09 외부 프레임워크 비교

학습 목표

LEARNING OBJECTIVES

- → Deep Agents, LangGraph, LangChain의 심화 차이를 이해한다
- → OpenCode, Claude Agent SDK와 비교한다
- → 아키텍처, 유연성, 생태계를 비교 분석한다
- → 사용 사례별 추천 프레임워크를 안다
- → 마이그레이션 고려사항을 이해한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

```
모델 설정 완료: gpt-4.1
```

1. 비교 개요

AI 에이전트 프레임워크를 선택할 때는 모델 지원, 아키텍처, 생태계, 라이선스 등 여러 요소를 따져봐야 합니다.

세 가지 주요 프레임워크를 비교합니다:

- LangChain Deep Agents — 모델 무관(model-agnostic) 에이전트 하네스
- OpenCode — 터미널/데스크톱/IDE 기반 코딩 에이전트
- Claude Agent SDK — Anthropic의 Claude 전용 에이전트 SDK

2. Deep Agents vs OpenCode vs Claude Agent SDK

기본 비교

특성	LangChain Deep Agents	OpenCode	Claude Agent SDK
모델 지원	모델 무관 (Anthropic, OpenAI, 100+ 제공자)	75+ 제공자 (Ollama 포함 로컬)	Claude 모델 전용
라이선스	MIT	MIT	MIT (SDK), 독점 (Claude Code)
SDK	Python, TypeScript + CLI	터미널, 데스크톱, IDE 확장	Python, TypeScript
샌드박스	통합 도구로 사용 가능	미지원	미지원
상태 관리	타임 트래블 지원	미지원	타임 트래블 지원
Observability	LangSmith 네이티브	없음	없음

```
# 프레임워크 비교 테이블 출력
frameworks = {
    "LangChain Deep Agents": {
        "모델 지원": "100+ 제공자 (model-agnostic)",
        "라이선스": "MIT",
        "SDK": "Python, TypeScript, CLI",
        "샌드박스": "통합 지원",
        "타임 트래블": "지원",
    },
    "OpenCode": {
        "모델 지원": "75+ 제공자 (로컬 포함)",
        "라이선스": "MIT",
        "SDK": "터미널, 데스크톱, IDE",
        "샌드박스": "미지원",
        "타임 트래블": "미지원",
    },
    "Claude Agent SDK": {
        "모델 지원": "Claude 전용",
        "라이선스": "MIT (SDK)",
        "SDK": "Python, TypeScript",
        "샌드박스": "미지원",
        "타임 트래블": "지원",
    },
}

print("=== 프레임워크 비교 ===")
for name, features in frameworks.items():
    print(f"\n[{name}]")
    for key, value in features.items():
        print(f"  {key}: {value}")
```

```
=== 프레임워크 비교 ===

[LangChain Deep Agents]
  모델 지원: 100+ 제공자 (model-agnostic)
  라이선스: MIT
  SDK: Python, TypeScript, CLI
  샌드박스: 통합 지원
  타임 트래블: 지원

[OpenCode]
  모델 지원: 75+ 제공자 (로컬 포함)
  라이선스: MIT
  SDK: 터미널, 데스크톱, IDE
  샌드박스: 미지원
  타임 트래블: 미지원

[Claude Agent SDK]
  모델 지원: Claude 전용
  라이선스: MIT (SDK)
  SDK: Python, TypeScript
  샌드박스: 미지원
  타임 트래블: 지원
```

3. 핵심 기능 비교

공통 기능

세 프레임워크 모두 다음 기능을 지원합니다:

- 파일 작업 (읽기, 쓰기, 편집)
- 쉘 명령 실행
- 검색 기능 (grep, glob)
- 계획 기능 (태스크 리스트)
- Human-in-the-Loop (권한 프레임워크는 상이)

차별화 기능

기능	Deep Agents	OpenCode	Claude Agent SDK
코어 도구	파일·쉘·검색·계획	파일·쉘·검색·계획	파일·쉘·검색·계획
샌드박스 통합	통합 도구로 사용 가능 (Modal/ Daytona/Runloop/LangSmith/ AgentCore)	없음	없음
플러거블 백엔드	스토리지·파일시스템 (StateBackend/ StoreBackend/ FilesystemBackend/ CompositeBackend)	없음	없음
가상 파일시스템	플러거블 백엔드 + 멀티모달 read_file	없음	없음
네이티브 트레이싱	LangSmith	없음	없음

1. Execution Model — 핵심 차별점

축	Deep Agents	Claude Agent SDK
실행 backend	Pluggable (local, virtual, remote, custom)	Local filesystem 한정
에이전트 위치	샌드박스 내부 또는 외부에서 원격 명령 실행	샌드박스 내부 + local FS

Deep Agents는 "에이전트가 샌드박스 안에서 돌든 밖에서 원격으로 명령을 돌리든" 둘 다 지원합니다. Claude Agent SDK는 sandbox-internal + local filesystem으로 한정됩니다.

2. Deployment & Multi-Tenancy — 핵심 차별점

축	Deep Agents	Claude Agent SDK
Managed deployment	LangSmith 제공	직접 구축 필요
Self-hosted	Docker 공식 지원	직접 구축 필요
Multi-tenant	scoped threads, per-user sandboxes, RBAC 빌트인	서버·인증·스트리밍 직접 구현

Claude Agent SDK는 "the server, auth, and streaming layer"를 사용자가 직접 만들어야 합니다. Deep Agents는 LangSmith 매니지드 + Docker 자체호스팅 모두 제공합니다.

3. Per-Model Configuration — 핵심 차별점

축	Deep Agents	Claude Agent SDK
설정 방식	Harness profiles (provider/model 키로 자동 적용)	Code-based tuning
라이선스	MIT	MIT

Deep Agents의 `HarnessProfile`은 provider/model별 system prompt suffix, 도구 visibility, 제외 미들웨어 등을 자동 적용합니다. Claude Agent SDK는 코드에서 직접 분기해야 합니다.

---

4. 아키텍처 비교

LangChain Deep Agents

- 플러거블 스토리지 백엔드 — 상태, 파일시스템, 스토어를 독립적으로 구성
- 가상 파일시스템 — 로컬, 인메모리, 샌드박스 백엔드 교체 가능
- LangGraph 기반 — 그래프 실행 엔진으로 복잡한 워크플로 지원
- 미들웨어 시스템 — 에이전트 동작을 세밀하게 커스터마이징

OpenCode

- 터미널 네이티브 — 가볍고 빠른 시작
- 75+ 모델 제공자 — Ollama를 통한 로컬 모델 지원

- LSP 통합 — 코드 편집에 특화된 에디터 기능

Claude Agent SDK

- Claude 최적화 — Claude 모델에 특화된 기능
- 타임 트래블 — 상태 분기(branching) 지원
- 간결한 API — 빠른 프로토타이핑에 적합

---

5. 사용 사례별 추천

사용 사례	추천 프레임워크	이유
프로덕션 에이전트 앱	Deep Agents	플러거블 백엔드, 옵저버빌리티, 샌드박스
멀티 모델 에이전트	Deep Agents	100+ 모델 제공자 지원
터미널 코딩 어시스턴트	OpenCode	가볍고 빠른 시작, 로컬 모델
Claude 전용 앱	Claude Agent SDK	Claude 최적화, 간결한 API
빠른 프로토타이핑	Claude Agent SDK	간결한 API, 빠른 설정
복잡한 멀티 에이전트 시스템	Deep Agents	서브에이전트, 컨텍스트 관리
로컬 모델 사용	OpenCode	Ollama 네이티브 지원

```python
# 사용 사례별 프레임워크 추천 도우미
def recommend_framework(use_case: str) -> str:
    """사용 사례에 따라 프레임워크를 추천합니다."""
    recommendations = {
        "production": ("Deep Agents", "플러거블 백엔드, 옵저버빌리티, 샌드박스"),
        "multi-model": ("Deep Agents", "100+ 모델 제공자 지원"),
        "terminal": ("OpenCode", "가볍고 빠른 시작, 로컬 모델"),
        "claude-only": ("Claude Agent SDK", "Claude 최적화, 간결한 API"),
        "prototyping": ("Claude Agent SDK", "간결한 API, 빠른 설정"),
        "multi-agent": ("Deep Agents", "서브에이전트, 컨텍스트 관리"),
        "local-model": ("OpenCode", "Ollama 네이티브 지원"),
    }
    if use_case in recommendations:
        fw, reason = recommendations[use_case]
        return f"{fw} — {reason}"
    return "해당 사용 사례를 찾을 수 없습니다."

# 테스트
test_cases = ["production", "terminal", "claude-only", "multi-agent"]
print("=== 프레임워크 추천 ===")
for case in test_cases:
    print(f"  {case}: {recommend_framework(case)}")
```

```
=== 프레임워크 추천 ===
  production: Deep Agents — 플러거블 백엔드, 옵저버빌리티, 샌드박스
  terminal: OpenCode — 가볍고 빠른 시작, 로컬 모델
  claude-only: Claude Agent SDK — Claude 최적화, 간결한 API
  multi-agent: Deep Agents — 서브에이전트, 컨텍스트 관리
```

---

6. 생태계 비교

항목	Deep Agents	OpenCode	Claude Agent SDK
커뮤니티	LangChain 생태계 (대규모)	GitHub 커뮤니티	Anthropic 커뮤니티
문서	공식 문서 + LangSmith 연동	GitHub README	Anthropic 공식 문서
통합	LangChain, LangGraph, LangSmith	LSP, 터미널	Claude API
패키지 관리	pip/uv	go install / brew	pip/npm
에디터 통합	ACP (Zed, JetBrains, VS Code, Neovim)	자체 에디터	없음

7. 마이그레이션 고려사항

프레임워크 간 마이그레이션 시 확인할 핵심 사항:

공통 고려사항

1. 모델 호환성 – 사용 중인 모델이 대상 프레임워크에서 지원되는지 확인
2. 도구 호환성 – 커스텀 도구의 인터페이스 변환 필요
3. 상태 관리 – 체크포인트/메모리 마이그레이션 방법 확인
4. 옵저버빌리티 – 트레이싱/로깅 솔루션 대체 방안

Deep Agents로 마이그레이션 시 장점

- LangChain 도구 재사용 – 기존 LangChain 도구를 그대로 사용 가능
- LangGraph 호환 – LangGraph 그래프와 연동 가능
- 점진적 마이그레이션 – 기존 코드를 단계적으로 전환 가능

주의사항

- Claude Agent SDK에서 마이그레이션 시 Claude 전용 기능은 대체 구현 필요
- OpenCode에서 마이그레이션 시 터미널 UI 로직 분리 필요

요약

주제	핵심 개념	핵심 API/도구
3-way 비교	Deep Agents, OpenCode, Claude Agent SDK	모델 지원, 라이선스, SDK

핵심 기능	공통 도구 + 차별화 기능	샌드박스, 플러거블 백엔드
Execution Model	에이전트가 sandbox 내부/외부 어디서나 실행 vs sandbox-internal 한정	Pluggable backend
Deployment & Multi-Tenancy	LangSmith 매니지드 + Docker + 빌트인 multi-tenant	scoped threads, RBAC
Per-Model Configuration	Harness profiles vs code-based tuning	`HarnessProfile`, `register_harness_profile`
아키텍처	플러거블 vs 네이티브 vs 최적화	LangGraph, LSP, Claude API
사용 사례	프로덕션, 터미널, 프로토타이핑 등	`recommend_framework()`
생태계	커뮤니티, 문서, 통합, 에디터	LangSmith, ACP
마이그레이션	모델/도구/상태 호환성 확인	점진적 전환

> REFERENCES
>
> - Comparison with OpenCode and Claude Agent SDK

10 샌드박스와 ACP

학습 목표

LEARNING OBJECTIVES

- → 샌드박스 격리 개념과 보안 원칙을 이해한다
- → E2B, Modal, Docker 등 샌드박스 프로바이더를 비교한다
- → ACP(Agent Communication Protocol)의 개요와 용도를 안다
- → 에이전트–에디터 통합 패턴을 이해한다
- → 샌드박스 + ACP 통합 아키텍처를 설계한다

```
# 환경 설정
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY가 설정되지 않았습니다!"
print("환경 설정 완료")
```

```
환경 설정 완료
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")

print(f"모델 설정 완료: {model.model_name}")
```

```
모델 설정 완료: gpt-4.1
```

1. 샌드박스 개념

샌드박스는 AI 에이전트가 코드를 실행하고, 파일을 관리하고, 쉘 명령을 수행할 수 있는 격리된 실행 환경입니다.

왜 격리가 중요한가?

위험	격리 없을 때	샌드박스 사용 시
파일 시스템 접근	호스트 파일 변경/삭제 가능	격리된 파일시스템만 접근
네트워크 접근	무제한 외부 통신	제한된 네트워크 접근
자격 증명	환경 변수 유출 가능	시크릿 격리
시스템 영향	호스트 OS에 영향	호스트 시스템 보호

Deep Agents에서 샌드박스는 백엔드로 동작하며, 파일시스템 도구(`ls` , `read_file` , `write_file` , `edit_file` , `glob` , `grep`)와 `execute` 도구를 제공합니다.

---

2. 아키텍처 패턴

샌드박스 통합에는 두 가지 주요 패턴이 있습니다.

Agent-in-Sandbox

에이전트가 샌드박스 내부에서 실행되며, 네트워크 프로토콜로 외부와 통신합니다.

장점	단점
개발 환경과 동일한 경험	자격 증명 노출 위험
간단한 설정	인프라 복잡성 증가

Sandbox-as-Tool (권장)

에이전트가 외부에서 실행되며, 샌드박스 API를 호출하여 코드를 실행합니다.

장점	단점
에이전트 상태와 실행 분리	네트워크 지연
시크릿을 샌드박스 외부에 유지	
병렬 태스크 실행 가능	

```
# 두 가지 아키텍처 패턴 비교 (참고용)
print("=== 패턴 1: Agent-in-Sandbox ===")
print("  [샌드박스]")
print("    |-- 에이전트 (내부 실행)")
print("    |-- 파일시스템")
print("    |-- 코드 실행")
print("    <---> 네트워크 프로토콜 <---> 외부 시스템")

print()
print("=== 패턴 2: Sandbox-as-Tool (권장) ===")
print("  [호스트]")
print("    |-- 에이전트 (외부 실행)")
```

```
print("    |-- 자격 증명 관리")
print("    |-- API 호출 --> [샌드박스]")
print("                        |-- 파일시스템")
print("                        |-- 코드 실행")
```

```
=== 패턴 1: Agent-in-Sandbox ===
  [샌드박스]
    |-- 에이전트 (내부 실행)
    |-- 파일시스템
    |-- 코드 실행
    <---> 네트워크 프로토콜 <---> 외부 시스템

=== 패턴 2: Sandbox-as-Tool (권장) ===
  [호스트]
    |-- 에이전트 (외부 실행)
    |-- 자격 증명 관리
    |-- API 호출 --> [샌드박스]
                       |-- 파일시스템
                       |-- 코드 실행
```

---

3. 샌드박스 프로바이더 비교 (Deep Agents 0.4+ 공식 패키지)

패키지	공급자	특징
`langchain-modal`	Modal	GPU 지원, ML/AI 워크로드, 데이터 처리
`langchain-daytona`	Daytona	TypeScript/Python, 빠른 콜드 스타트
`langchain-runloop`	Runloop	일회용 devbox 기반 격리
`langsmith[sandbox]`	LangSmith	LangSmith Deployments 통합 (private beta)
`langchain-agentcore-codeinterpreter`	AgentCore	AWS Bedrock 기반 코드 인터프리터

공통 흐름: 공급자 SDK로 샌드박스 생성 → `*Sandbox` 래퍼로 backend 생성 → `create_deep_agent(backend=...)` → `try/finally` 로 정리(`terminate` / `stop` / `shutdown`).

```python
# Modal 샌드박스 예시 (참고용) — docs/deepagents/11-sandboxes.md
import textwrap

modal_example = textwrap.dedent('''
    # pip install langchain-modal deepagents
    import modal
    from deepagents import create_deep_agent
    from langchain_anthropic import ChatAnthropic
    from langchain_modal import ModalSandbox

    app = modal.App.lookup("your-app")
    modal_sandbox = modal.Sandbox.create(app=app)
    backend = ModalSandbox(sandbox=modal_sandbox)

    agent = create_deep_agent(
        model=ChatAnthropic(model="claude-sonnet-4-6"),
        system_prompt="You are a Python coding assistant with sandbox access.",
        backend=backend,
    )

    try:
        result = agent.invoke({
```

```
            "messages": [{"role": "user", "content": "Create a small Python package and run pytest"}],
        })
    finally:
        modal_sandbox.terminate()  # 필수: 리소스 해제
''')

print("=== Modal 샌드박스 예시 ===")
print(modal_example)
```

```
=== Modal 샌드박스 설정 ===
  provider: modal
  image: python:3.12-slim
  gpu: T4
  timeout: 300

코드 예시 (참고용):
  from deepagents.backends.sandbox import ModalSandbox
  agent = create_deep_agent(
      model="gpt-4.1",
      backend=ModalSandbox(image="python:3.12-slim", gpu="T4"),
  )
```

---

4. 보안 가이드라인

절대 샌드박스 안에 시크릿을 넣지 마세요

컨텍스트에 주입된 에이전트는 환경 변수나 마운트된 파일로 저장된 자격 증명을 읽고 유출할 수 있습니다.

안전한 관행

1. 자격 증명은 외부 도구에서만 관리 — 샌드박스 외부의 전용 도구 사용
2. Human-in-the-Loop — 민감한 작업에 사람 승인 요구
3. 네트워크 접근 차단 — 불필요한 아웃바운드 연결 차단
4. 아웃바운드 모니터링 — 예기치 않은 외부 연결 감시
5. 출력 검토 — 샌드박스 출력을 애플리케이션에 적용하기 전 검토

---

5. 파일 전송과 라이프사이클

파일 접근 방법

방법	설명
에이전트 파일시스템 도구	`execute()` 를 통한 직접 파일 작업
파일 전송 API	`uploadFiles()` , `downloadFiles()` 로 시드/아티팩트 관리

라이프사이클 관리

샌드박스는 불필요한 비용을 막기 위해 명시적 종료가 필요합니다. 채팅 애플리케이션에서는 대화 스레드별 고유 샌드박스에 TTL(Time-to-Live) 설정을 사용합니다.

```
# 라이프사이클 모드 (참고용) — docs/deepagents/11-sandboxes.md
lifecycle_modes = {
    "Thread-scoped (기본)": {
        "설명": "대화 thread 1개당 샌드박스 1개. 첫 run에서 생성, 같은 thread의 다음 turn에서 재사용",
        "정리": "idle TTL로 자동 정리",
        "적합": "멀티 턴 대화, 사용자별 격리",
    },
    "Assistant-scoped": {
        "설명": "같은 assistant의 모든 thread가 샌드박스 1개를 공유. 파일·패키지·레포지토리가 대화 사이에 누적",
        "정리": "반드시 TTL 또는 주기적 스냅샷 설정 필요 (누적 무한정)",
        "적합": "개발 환경 공유, 캐시 누적이 이득인 워크로드",
    },
}

file_operations = [
    "uploadFiles(['/local/data.csv'], '/sandbox/data/')",
    "downloadFiles(['/sandbox/output/result.json'], '/local/results/')",
]

print("=== 라이프사이클 모드 ===")
for mode, attrs in lifecycle_modes.items():
    print(f"\n[{mode}]")
    for k, v in attrs.items():
        print(f"  {k}: {v}")

print("\n=== 파일 전송 예시 ===")
for op in file_operations:
    print(f"  {op}")
```

```
=== 라이프사이클 설정 ===
  ttl_seconds: 1800
  auto_shutdown: True
  thread_isolation: True

=== 파일 전송 예시 ===
  uploadFiles(['/local/data.csv'], '/sandbox/data/')
  downloadFiles(['/sandbox/output/result.json'], '/local/results/')
```

---

6. ACP 개요

ACP(Agent Client Protocol)는 코딩 에이전트와 개발 환경(에디터/IDE) 간의 통신을 표준화하는 프로토콜입니다.

MCP vs ACP

프로토콜	용도	대상
MCP (Model Context Protocol)	외부 도구 통합	에이전트 ↔ 외부 서비스
ACP (Agent Client Protocol)	에디터-에이전트 통합	에이전트 ↔ 에디터/IDE

ACP를 쓰면 에이전트가 에디터와 직접 상호작용하여 코드 편집, 파일 탐색, 터미널 명령을 수행할 수 있습니다.

---

7. ACP 서버 구현

```
# ACP 서버 구현 예시 (참고용) — docs/deepagents/14-acp.md
acp_server_code = '''
# pip install deepagents-acp
import asyncio

from acp import run_agent
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP


async def main() -> None:
    agent = create_deep_agent(
        model="google_genai:gemini-3.5-flash",
        system_prompt="You are a helpful coding assistant",
        checkpointer=MemorySaver(),
    )
    server = AgentServerACP(agent)
    await run_agent(server)


if __name__ == "__main__":
    asyncio.run(main())
'''

print("=== ACP 서버 구현 예시 ===")
print(acp_server_code)

print("설치: pip install deepagents-acp")
print("실행: python acp_server.py  (stdio 모드)")
```

```
=== ACP 서버 구현 예시 ===

# pip install deepagents-acp
from deepagents import create_deep_agent
from deepagents_acp import AgentServerACP
from langgraph.checkpoint.memory import MemorySaver

# 에이전트 생성
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt="당신은 코딩 어시스턴트입니다.",
    checkpointer=MemorySaver(),
)

# ACP 서버 실행 (stdio 모드)
server = AgentServerACP(agent)
server.run()

설치: pip install deepagents-acp
실행: python acp_server.py (stdio 모드)
```

8. ACP 지원 에디터

에디터	통합 방식
Zed	네이티브 통합
JetBrains IDEs	빌트인 지원

Visual Studio Code	vscode-acp 플러그인
Neovim	ACP 호환 플러그인

Zed 설정 예시

```
// Zed settings.json
{
  "agent_servers": [
    {
      "command": "python",
      "args": ["acp_server.py"],
      "env": {
        "ANTHROPIC_API_KEY": "sk-..."
      }
    }
  ]
}
```

추가 도구: Toad (`batrachian-toad`)

Toad는 ACP 서버를 로컬 개발 도구로 실행하기 위한 프로세스 관리자입니다. `uv tool` 로 설치합니다.

```
# 설치
uv tool install -U batrachian-toad

# 실행
toad acp "python path/to/your_server.py" .
# 또는
toad acp "uv run python path/to/your_server.py" .
```

---

9. 샌드박스 + ACP 통합

샌드박스와 ACP를 결합하면 에디터에서 에이전트를 제어하면서, 코드 실행은 격리된 환경에서 수행하는 완전한 아키텍처를 구현할 수 있습니다.

통합 아키텍처

```
[에디터/IDE] <-- ACP --> [에이전트] <-- API --> [샌드박스]
    |                        |                      |
  코드 편집               태스크 관리            코드 실행
  파일 탐색               컨텍스트 관리          파일 격리
  터미널 UI               도구 호출              보안 환경
```

장점

- 에디터에서 직접 에이전트와 상호작용
- 코드 실행은 안전한 샌드박스에서 수행
- 시크릿은 호스트(에이전트 측)에서만 관리

```
# 샌드박스 + ACP 통합 예시 (참고용)
integrated_config = '''
# pip install langchain-modal deepagents deepagents-acp
import asyncio

import modal
from acp import run_agent
from deepagents import create_deep_agent
from langchain_anthropic import ChatAnthropic
from langchain_modal import ModalSandbox
from langgraph.checkpoint.memory import MemorySaver

from deepagents_acp.server import AgentServerACP


async def main() -> None:
    app = modal.App.lookup("your-app")
    modal_sandbox = modal.Sandbox.create(app=app)
    backend = ModalSandbox(sandbox=modal_sandbox)

    agent = create_deep_agent(
        model=ChatAnthropic(model="claude-sonnet-4-6"),
        system_prompt="당신은 코딩 어시스턴트입니다.",
        backend=backend,
        checkpointer=MemorySaver(),
        interrupt_on={"execute": True},  # 코드 실행 전 승인
    )

    server = AgentServerACP(agent)
    try:
        await run_agent(server)
    finally:
        modal_sandbox.terminate()


if __name__ == "__main__":
    asyncio.run(main())
'''

print("=== 샌드박스 + ACP 통합 예시 ===")
print(integrated_config)

print("이 구성의 효과:")
print("  1. 에디터에서 ACP를 통해 에이전트와 상호작용")
print("  2. 코드 실행은 Modal 샌드박스에서 안전하게 수행")
print("  3. execute 호출 시 Human-in-the-Loop 승인 필요 (interrupt_on)")
```

```
=== 샌드박스 + ACP 통합 예시 ===

from deepagents import create_deep_agent
from deepagents.backends.sandbox import ModalSandbox
from deepagents_acp import AgentServerACP
from langgraph.checkpoint.memory import MemorySaver

# 샌드박스 백엔드 + ACP 서버 통합
agent = create_deep_agent(
    model="gpt-4.1",
    system_prompt="당신은 코딩 어시스턴트입니다.",
    backend=ModalSandbox(image="python:3.12-slim"),
    checkpointer=MemorySaver(),
    interrupt_on={"execute": True},  # 코드 실행 전 승인
)

# ACP로 에디터와 연결
server = AgentServerACP(agent)
server.run()

이 구성의 효과:
  1. 에디터에서 ACP를 통해 에이전트와 상호작용
  2. 코드 실행은 Modal 샌드박스에서 안전하게 수행
  3. execute 호출 시 Human-in-the-Loop 승인 필요
```

요약

주제	핵심 개념	핵심 API/도구
샌드박스 개념	격리된 실행 환경으로 호스트 보호	`execute`, 파일시스템 도구
아키텍처 패턴	Agent-in-Sandbox vs Sandbox-as-Tool	Sandbox-as-Tool 권장
프로바이더	Modal(GPU), Daytona(빠른 시작), Runloop(일회용), LangSmith[sandbox], AgentCore	`ModalSandbox`, `DaytonaSandbox`, `RunloopSandbox`
라이프사이클	Thread-scoped(기본) vs Assistant-scoped(공유, TTL 필수)	`terminate` / `stop` / `shutdown`
보안	시크릿 외부 관리, HITL, 네트워크 차단	`interrupt_on`
ACP 개요	에디터-에이전트 통신 표준화	`AgentServerACP`, `run_agent`
ACP 서버	asyncio + `run_agent` 로 stdio 모드 띄우기	`deepagents-acp`
Toad CLI	ACP 서버 프로세스 관리자	`uv tool install -U batrachian-toad`, `toad acp`
에디터 통합	Zed, JetBrains, VS Code, Neovim	ACP 프로토콜
통합 패턴	에디터 ↔ 에이전트 ↔ 샌드박스	ACP + Sandbox 결합

REFERENCES

- Sandboxes
- Agent Client Protocol (ACP)

11 비동기 서브에이전트

AsyncSubAgent · Non-blocking · Mid-flight steering

Deep Agents 0.5.0의 플래그십 기능 `AsyncSubAgentMiddleware`는 슈퍼바이저가 블록되지 않고 서브에이전트를 백그라운드로 기동합니다. 장시간 리서치·코딩·병렬 서브에이전트 조율처럼 수 분에서 수 시간 단위 작업에서 기존 동기 서브에이전트의 한계를 넘는 인프라입니다. 단, 이 기능은 Agent Protocol 서버가 필요합니다 — LangSmith Deployments 또는 `langgraph dev` 같은 자체 호스트 환경에서만 동작합니다.

학습 목표

LEARNING OBJECTIVES

- → 동기 서브에이전트와 `AsyncSubAgent`의 실행 모델 차이를 설명한다
- → `start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks` 5개 도구의 역할을 구분한다
- → ASGI(co-deploy)와 HTTP(원격) 전송 모드를 비교하고 하이브리드 구성을 만든다
- → `async_tasks` state 채널이 컨텍스트 압축을 넘어 상태를 보존하는 원리를 이해한다
- → Mid-flight steering 패턴(update/cancel)과 `--n-jobs-per-worker` 슬롯 튜닝을 안다

11.1 기존 동기 서브에이전트의 한계

기존 서브에이전트는 동기였습니다. `task` 도구가 호출되면 슈퍼바이저는 서브에이전트가 끝날 때까지 멈추고, 사용자는 그 동안 새 지시를 줄 수 없었습니다. 분 단위 이상의 리서치 작업을 동기로 돌리면 프런트엔드가 응답 없이 굳어 있고, 사용자는 "아직 살아 있는가"를 확인할 방법이 없었습니다.

0.5.0의 `AsyncSubAgentMiddleware`는 이 제약을 제거합니다.

- **Non-blocking 실행**: `start_async_task`가 task id만 즉시 반환. 슈퍼바이저는 곧바로 사용자와 대화를 계속합니다.
- **Mid-flight steering**: 실행 중인 서브에이전트에 follow-up 지시를 보내거나 취소할 수 있습니다.
- **독립 스레드**: 각 서브에이전트 태스크는 자체 thread와 run을 가집니다. 슈퍼바이저의 컨텍스트 압축이 일어나도 상태가 손실되지 않습니다.

11.2 슈퍼바이저에게 주입되는 5개 도구

`AsyncSubAgentMiddleware`는 슈퍼바이저에게 다섯 개 도구를 주입합니다.

도구	역할
`start_async_task`	서브에이전트 백그라운드 기동. task id 즉시 반환
`check_async_task`	현재 상태 조회, 완료되었으면 최종 출력 추출
`update_async_task`	실행 중인 태스크의 같은 스레드에 새 지시 주입 (interrupt 전략)
`cancel_async_task`	서버에 cancel 신호 전송, 태스크를 `cancelled`로 마킹
`list_async_tasks`	추적 중인 모든 태스크의 현재 상태 일괄 조회

11.3 기본 사용

`AsyncSubAgent` 스펙 리스트를 `subagents`에 전달하면 `create_deep_agent`가 `AsyncSubAgentMiddleware`를 자동 부착합니다.

```python
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="정보 수집과 종합이 필요한 리서치 작업",
        graph_id="researcher",
    ),
    AsyncSubAgent(
        name="coder",
        description="코드 생성/리뷰 작업",
        graph_id="coder",
        url="https://coder-deployment.langsmith.dev",  # 원격 HTTP 호출
    ),
]

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    subagents=async_subagents,
)
```

AsyncSubAgent 핵심 필드

필드	설명
`name`	슈퍼바이저가 참조하는 고유 식별자
`description`	어떤 태스크에 위임할지 판단 근거. 동기 서브에이전트와 동일하게 중요
`graph_id`	`langgraph.json`의 graph 이름과 일치해야 함
`url`	선택. 없으면 ASGI(in-process), 있으면 원격 HTTP 호출
`headers`	선택. 자체 호스트 서버의 인증 헤더

11.4 `async_tasks` state 채널

태스크 메타데이터는 메시지 히스토리와 분리된 `async_tasks` state 채널에 저장됩니다. 레코드 하나에는 다음이 들어갑니다:

- task id
- agent name
- thread id, run id
- status (`pending` / `running` / `success` / `error` / `cancelled`)
- `created_at` , `updated_at`

이 분리 구조 덕분에 슈퍼바이저의 컨텍스트가 압축(summarization)되어 오래된 메시지가 요약으로 대체되어도 task id는 사라지지 않습니다. 압축 후에도 `check_async_task` / `update_async_task` 를 그대로 호출할 수 있습니다.

> **! WARNING**
>
> 커스텀 state reducer나 middleware로 state를 재구성할 때 `async_tasks` 채널을 덮어쓰면 실행 중인 태스크 추적을 잃습니다. 병합 전략을 명시하세요.

11.5 전송 모드

ASGI (co-deploy, 기본 권장)

`url` 을 생략하면 서브에이전트가 슈퍼바이저와 같은 프로세스에서 함수 호출로 실행됩니다. 네트워크 레이턴시 없이, 동일 `langgraph.json` 에 두 그래프를 등록합니다.

```
// langgraph.json
{
  "dependencies": ["."],
  "graphs": {
    "supervisor": "./agent.py:supervisor",
    "researcher": "./subagents/researcher.py:graph",
    "coder": "./subagents/coder.py:graph"
  },
  "env": ".env"
}
```

```
AsyncSubAgent(
    name="researcher",
    description="...",
    graph_id="researcher",   # url 없음 → ASGI
)
```

HTTP (원격)

독립 배포된 서브에이전트를 `url` 로 호출합니다. 리서치 전용 저가 모델, 코딩 전용 GPU 배포처럼 리소스 프로파일이 다른 서브에이전트를 따로 스케일하고 싶을 때 씁니다.

```
import os

AsyncSubAgent(
    name="coder",
    description="...",
    graph_id="coder",
    url="https://coder-deployment.langsmith.dev",
    headers={"x-api-key": os.environ["CODER_API_KEY"]},
)
```

하이브리드

둘을 섞어도 됩니다. 경량 서브에이전트는 ASGI, 리소스 집약 서브에이전트는 원격 HTTP로 배포 토폴로지를 워크로드에 맞게 나눕니다.

11.6 실행 라이프사이클

1. **Launch** — `start_async_task`가 새 thread 생성, run 시작, task id 즉시 반환
2. **Check** — `check_async_task`가 상태 조회, 완료 시 최종 출력 추출
3. **Update** — `update_async_task`가 기존 thread에 history를 유지한 채 새 instruction으로 interrupt 기반 새 run 기동
4. **Cancel** — `cancel_async_task`가 `runs.cancel()` 호출 후 cancelled로 마킹
5. **List** — 비종결 태스크의 live status를 병렬 조회, 종결된 건 캐시 반환

11.7 Mid-flight steering 패턴

대화 도중 사용자가 방향을 바꾸면 슈퍼바이저가 `update_async_task`를 호출합니다.

```
사용자: 경쟁사 리서치 시작해줘.
슈퍼바이저: [start_async_task(agent="researcher", ...)] → task_abc123

사용자: 아, Series A 이상만 봐줘.
슈퍼바이저: [update_async_task(task_id="task_abc123",
                               instruction="범위를 Series A 이상으로 좁혀줘")]

사용자: 그만, 대신 SaaS 쪽으로 다시 파줘.
슈퍼바이저: [cancel_async_task(task_id="task_abc123")]
           [start_async_task(agent="researcher",
                             description="SaaS 경쟁사 리서치")]
```

`update_async_task`는 같은 스레드에 새 run을 만들므로, 서브에이전트는 이전 탐색 결과를 그대로 이어받고 새 지시만 얹습니다. 완전히 새 태스크로 시작할 때만 cancel + start 조합을 씁니다.

11.8 로컬 개발: `--n-jobs-per-worker`

`langgraph dev`의 기본 worker pool은 작아서 동시 서브에이전트 기동 시 큐잉이 발생합니다. 슬롯 수를 늘려 줍니다.

```
langgraph dev --n-jobs-per-worker 10
```

서브에이전트 3개를 동시에 돌리는 슈퍼바이저는 최소 **4 슬롯**이 필요합니다(슈퍼바이저 1 + 서브에이전트 3). 여유 있게 10 20으로 잡아 두세요.

11.9 v2 통합 스트리밍으로 라이프사이클 관찰

비동기 서브에이전트의 Pending/Running/Complete 신호는 별도 v3 projection이 아니라 `agent.stream(...)` 한 경로로 받습니다. `subgraphs=True` + `version="v2"` 로 메인/서브 이벤트를 통일하고, `async_tasks` 채널 변화는 `updates` 모드의 `data` 에 함께 흘러나옵니다.

```
async for chunk in agent.astream(
    {"messages": [{"role": "user", "content": "경쟁사 3개 동시 리서치"}]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # async_tasks 채널 변화, check_async_task / list_async_tasks 결과
        for node, data in chunk["data"].items():
            print(f"step={node}")
    elif t == "messages":
        # 슈퍼바이저 토큰 + tool_call_chunks
        ...
    elif t == "custom":
        # 도구 내부 get_stream_writer 진행률
        ...
```

신호	감지 지점
Pending	슈퍼바이저가 `start_async_task` tool_call을 방출한 순간
Running	`async_tasks[task_id].status` 가 `running` 으로 바뀐 `updates` 청크
Complete	`status` 가 `success` / `error` / `cancelled` 중 하나로 종결된 `updates` 청크

UI 카드 상태를 이 세 신호에 매핑하면 별도 폴링 없이 라이프사이클이 자동으로 흐릅니다. 자세한 스트리밍 분기는 14장(스트리밍)을 참고합니다.

11.10 동기 vs 비동기 선택 기준

축	Sync SubAgent	AsyncSubAgent
실행	슈퍼바이저 블록, 완료까지 대기	즉시 task id 반환, 슈퍼바이저 계속 진행
결과 회수	자동으로 슈퍼바이저에 반환	`check_async_task` 로 폴링 필요
Mid-task 지시	불가	`update_async_task` 로 가능
취소	불가	`cancel_async_task` 로 가능
상태 유지	스테이트리스 (일회성)	자체 thread에 상태 유지, 상호작용 가능

인프라 요구	특별 요구 없음	Agent Protocol 서버 필요
적합 작업	수초 수십초, 결과만 필요한 작업	분 시간 단위 장시간, 중간 개입 가능한 작업

판단 흐름

- 작업이 수초 내 끝나고 슈퍼바이저가 결과를 바로 써야 한다 → **Sync**
- 작업이 수 분 이상이거나, 여러 서브에이전트를 병렬로 돌리고 싶다 → **Async**
- 사용자가 도중에 방향을 바꾸거나 중단시킬 수 있다 → **Async**
- LangSmith Deployments를 쓰지 않고 Agent Protocol 서버도 띄울 수 없다 → **Sync만 가능**

11.11 주의사항

- **Agent Protocol 의존성**: `AsyncSubAgent`를 선언했는데 실행 환경이 Agent Protocol을 지원하지 않으면 초기화 단계에서 실패
- `async_tasks` **채널은 보존하라**: 커스텀 state reducer나 middleware로 state를 재구성할 때 이 채널을 덮어쓰면 추적 상실
- **폴링 비용**: `check_async_task`를 너무 자주 호출하지 않도록 시스템 프롬프트에 가이드를 넣음 (예: "한 번에 2 3개 태스크를 기동한 뒤 사용자 반응을 기다려라")
- **장시간 태스크 정리**: 장기 미종결 태스크는 `list_async_tasks` + `cancel_async_task`로 정기적으로 정리. LangSmith Deployments는 checkpoint 기반이라 비용이 남음
- **HTTP 모드 타임아웃**: 원격 URL은 헤더·네트워크 타임아웃·재시도 정책을 별도로 확인

핵심 정리

- `AsyncSubAgent`는 슈퍼바이저를 블록하지 않고 서브에이전트를 백그라운드로 기동하는 0.5.0 플래그십 기능
- 5개 도구(`start` / `check` / `update` / `cancel` / `list`)로 라이프사이클을 제어
- ASGI는 co-deploy 기본, HTTP는 리소스 분리 시 사용, 하이브리드 가능
- `async_tasks` state 채널이 컨텍스트 압축에도 태스크 추적을 유지
- 수 분 이상 걸리거나 사용자 개입이 필요한 작업, 다중 병렬 서브에이전트에 적합

12 프로덕션 준비

Thread · User · Assistant 기반 10대 영역

로컬 프로토타입을 다중 사용자·다중 테넌트·장기 운영 가능한 프로덕션 에이전트로 전환할 때 짚어야 할 열 가지 영역을 다룹니다. Deep Agent를 실서비스에 올리기 직전, 또는 올렸는데 운영 이슈가 쌓일 때 이 장을 체크리스트로 씁니다.

학습 목표

LEARNING OBJECTIVES

- → Thread·User·Assistant 세 추상을 프로덕션 설계의 출발점으로 이해한다
- → `langgraph.json` + LangSmith Deployments가 자동 프로비저닝하는 인프라를 안다
- → 멀티 테넌트 authz, 엔드유저 자격증명, 메모리 스코핑을 구분한다
- → 샌드박스 thread-scoped vs assistant-scoped 패턴을 선택한다
- → 내구성·레이트 리밋·에러 3층·PII·실시간 프런트엔드를 미들웨어와 SDK로 구성한다

12.1 세 가지 핵심 추상

프로덕션 Deep Agent는 세 가지 핵심 추상 위에서 운영됩니다.

- **Thread** — 한 번의 대화. 메시지·파일·체크포인트의 단위
- **User** — 인증된 신원. 자원 소유권과 접근 범위의 단위
- **Assistant** — 설정된 에이전트 인스턴스 (프롬프트·도구·모델 조합)

이 세 개념 위에서 관리할 영역이 열 개이며, 각 영역은 독립적으로 도입할 수 있고 리스크 프로파일에 따라 골라 적용합니다.

12.2 LangSmith Deployments

`deepagents deploy` CLI 또는 LangSmith Deployment로 배포하면 다음 인프라가 자동 프로비저닝됩니다.

- Assistants / Threads / Runs API
- Store + Checkpointer (퍼시스턴스)
- 인증, Webhook, Cron, Observability

- MCP/A2A 노출 옵션

```
// langgraph.json 최소 설정
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./agent.py:agent"
  },
  "env": ".env"
}
```

12.3 멀티 테넌트 접근 제어

커스텀 auth/authz 핸들러

LangSmith Deployments는 커스텀 인증으로 사용자 신원을 확립하고, 별도 authorization 핸들러로 thread / assistant / store namespace 접근을 제어합니다. 핸들러는 리소스에 소유권 메타데이터를 태깅하고 사용자별 가시성을 필터링하며, HTTP 403으로 접근을 거부합니다.

Workspace RBAC

팀 단위 권한(LangSmith 자체 기능).

역할	권한
Workspace Admin	전체 권한
Workspace Editor	생성/수정, 삭제·멤버 관리 불가
Workspace Viewer	읽기 전용

12.4 엔드유저 자격증명 관리

에이전트가 사용자 대신 외부 서비스(GitHub, Slack, Gmail 등)를 호출할 때 자격증명을 에이전트 코드 밖에서 관리합니다.

Agent Auth (OAuth 2.0)

관리형 OAuth 플로우입니다. 첫 호출 때 사용자에게 consent URL을 interrupt로 제시하고, 토큰를 받으면 자동으로 resume·refresh합니다.

```
from langchain_auth import Client
from langchain.tools import tool, ToolRuntime

auth_client = Client()

@tool
async def github_action(runtime: ToolRuntime):
    """사용자 명의로 GitHub 작업 수행."""
    auth_result = await auth_client.authenticate(
        provider="github",
```

```
        scopes=["repo", "read:org"],
        user_id=runtime.server_info.user.identity,
    )
    # auth_result.token으로 GitHub API 호출
```

Sandbox Auth Proxy

샌드박스에서 실행되는 사용자 코드(또는 에이전트가 생성한 코드)가 외부 API를 호출할 때 프록시가 자격증명을 주입합니다. API 키가 샌드박스 안 코드에 노출되지 않습니다.

```
{
  "proxy_config": {
    "rules": [
      {
        "name": "openai-api",
        "match_hosts": ["api.openai.com"],
        "inject_headers": {
          "Authorization": "Bearer ${OPENAI_API_KEY}"
        }
      }
    ]
  }
}
```

`${SECRET_KEY}` 는 워크스페이스 시크릿에서 해석됩니다.

12.5 메모리 영속 스코핑

`StoreBackend` 의 `namespace` 함수로 메모리 범위를 정합니다. `CompositeBackend` 가 `/memories/` 만 `StoreBackend` 로 라우팅하고 나머지는 `StateBackend` 휘발성으로 두는 것이 기본 패턴입니다.

User-scoped (권장 기본값)

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (
                    rt.server_info.assistant_id,
                    rt.server_info.user.identity,
                ),
            ),
        },
    ),
    system_prompt=(
        "대화 시작 시 /memories/instructions.txt를 읽어라. "
        "지속 가치 있는 인사이트는 갱신한다."
    ),
)
```

Assistant-scoped / Organization-scoped
같은 assistant를 쓰는 모든 사용자가 공유(assistant-scoped)하거나 조직 전체에 공유(org-scoped)하는 구성도 가능합니다. 조직 공유는 read-only로 두는 것이 원칙입니다.

> ! WARNING
>
> **Prompt injection 경고**: 공유 메모리는 프롬프트 인젝션 벡터입니다. 사용자가 조작 가능한 범위에 쓰기 권한을 주지 말 것. 자세한 정책은 Part IV ch15(권한 관리) 참고.

12.6 실행 격리 — Sandbox

호스트 파일시스템과 네트워크를 그대로 노출하지 말고 샌드박스를 씁니다.

Thread-scoped sandbox (가장 흔한 패턴)

```python
from daytona import CreateSandboxFromSnapshotParams, Daytona
from deepagents import create_deep_agent
from langchain_core.runnables import RunnableConfig
from langchain_daytona import DaytonaSandbox

client = Daytona()

async def agent(config: RunnableConfig):
    thread_id = config["configurable"]["thread_id"]
    try:
        sandbox = await client.find_one(labels={"thread_id": thread_id})
    except Exception:
        sandbox = await client.create(
            CreateSandboxFromSnapshotParams(
                labels={"thread_id": thread_id},
                auto_delete_interval=3600,  # TTL
            )
        )
    return create_deep_agent(
        model="google_genai:gemini-3.5-flash",
        backend=DaytonaSandbox(sandbox=sandbox),
    )
```

Assistant-scoped sandbox
모든 스레드가 한 샌드박스를 공유해 도구 체인 캐시와 설치물을 보존해야 할 때 씁니다.

12.7 내구성·Async I/O

매 스텝 체크포인트
LangSmith Deployments는 자동으로 체크포인터를 붙입니다. 매 스텝 상태가 저장되므로:

- **Indefinite interrupt**: HITL 승인 대기가 며칠이어도 정확히 멈춘 자리에서 재개
- **Time travel**: 임의 체크포인트 시점으로 되감아 분기 실행
- **Audit trail**: 결제·관리자 동작 같은 민감 연산 직전 상태 감사

```
await agent.ainvoke(
    {"messages": [...]},
    config={"configurable": {"thread_id": "thread-abc"}},
)
```

Async I/O

LLM 앱은 I/O 바운드입니다. 비동기 도구와 미들웨어 훅(`abefore_agent` , `astream`)을 쓰면 처리량이 늘어납니다.

12.8 레이트 리밋과 비용 제어

```
from deepagents import create_deep_agent
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolCallLimitMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    middleware=[
        ModelCallLimitMiddleware(run_limit=50),
        ToolCallLimitMiddleware(run_limit=200),
    ],
)
```

`run_limit` 는 `invoke` 한 번마다 리셋, `thread_limit` 는 스레드 수명 동안 누적합니다. 폭주·무한 루프가 비용 폭탄이 되기 전에 잘라냅니다.

12.9 에러 핸들링 3층

분류	예시	전략	미들웨어
Transient	타임아웃, rate limit, 네트워크 일시 실패	자동 재시도 (backoff)	`ModelRetryMiddleware` , `ToolRetryMiddleware`
Recoverable	잘못된 도구 인자, 파싱 실패	모델에 피드백 → 재시도	도구 래퍼 에러 메시지
Human required	권한 없음, 불명확한 요청	에이전트 일시 정지	HITL `interrupt_on`

```
from langchain.agents.middleware import (
    ModelRetryMiddleware,
    ModelFallbackMiddleware,
    ToolRetryMiddleware,
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    middleware=[
        ModelRetryMiddleware(max_retries=3, backoff_factor=2.0, initial_delay=1.0),
        ModelFallbackMiddleware("gpt-5.4"),
```

```
        ToolRetryMiddleware(
            max_retries=2,
            tools=["search", "fetch_url"],
            retry_on=(TimeoutError, ConnectionError),
        ),
    ],
)
```

12.10 데이터 프라이버시와 실시간 프런트엔드

PIIMiddleware

이메일·카드 번호·주민번호 등 PII를 입출력 경계에서 가공합니다. 전략은 `redact` (삭제), `mask` (마스킹), `hash` (해시), `block` (차단)이며 커스텀 detector도 등록할 수 있습니다. PII가 로그에 섞이기 전 입력 단계에서 처리하는 게 핵심입니다. LangSmith 트레이스에도 마스킹된 채로 기록됩니다.

`useStream` 훅

`@langchain/react`의 `useStream` 훅으로 실시간 스트리밍·재연결·히스토리 로드를 한 번에 처리합니다.

```
import { useStream } from "@langchain/react";

function App() {
  const stream = useStream<typeof agent>({
    apiUrl: "https://your-deployment.langsmith.dev",
    assistantId: "agent",
    reconnectOnMount: true,
    fetchStateHistory: true,
  });
}
```

서브에이전트를 많이 띄우는 Deep Agent는 서브그래프 이벤트까지 스트리밍해 UI에 서브에이전트 진행 카드를 보여줍니다(`streamSubgraphs: true`).

12.11 프로덕션 체크리스트

- [ ] `langgraph.json`에 graph·env가 선언되어 있다
- [ ] 사용자별 authz 핸들러가 thread/store에 적용된다
- [ ] 외부 서비스 토큰은 Agent Auth/Proxy로 관리되고 코드에 하드코딩되지 않는다
- [ ] `/memories/`는 `StoreBackend`로 라우팅되고 스코프가 명시적이다(user/assistant/org)
- [ ] 공유 메모리는 read-only이거나 쓰기 권한이 엄격히 제한된다
- [ ] 호스트 파일시스템·셸에 직접 노출되지 않고 샌드박스를 경유한다
- [ ] `ModelCallLimitMiddleware` / `ToolCallLimitMiddleware`로 비용 상한이 설정되어 있다
- [ ] 재시도·fallback·HITL이 전부 구성되어 있다
- [ ] PII가 입출력 경계에서 마스킹된다
- [ ] 프런트엔드가 `reconnectOnMount` + `fetchStateHistory`를 사용한다

핵심 정리

- Thread·User·Assistant 세 추상 위에서 10대 운영 영역을 독립적으로 도입
- `langgraph.json` + LangSmith Deployments가 체크포인터·Store·인증·관측을 자동 프로비저닝
- 메모리 스코프는 user-scoped 기본, 공유 스코프는 반드시 read-only
- 에러는 3층(Transient/Recoverable/Human)으로 분류해 미들웨어와 HITL로 대응
- PII는 모델 입력 단계와 트레이스 전송 단계 두 레이어에서 이중 방어

13 컨텍스트 엔지니어링

Write big, read selective

Deep Agent가 모델의 제한된 컨텍스트 창 안에서 무엇을 보여주고, 언제 오프로드하고, 어떻게 다시 끌어올지 결정하는 설계를 정리합니다. 에이전트가 길어질수록 품질이 떨어지거나 요약에 삼켜질 때, 또는 서브에이전트·스킬·메모리를 한 파이프라인에서 정리하고 싶을 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- → 컨텍스트 엔지니어링의 4층 구조(Layered / Dynamic / Compression / Retrieval)를 이해한다
- → 시스템 프롬프트의 9단계 자동 합성 순서를 안다
- → 자동 offloading(>20k 토큰)과 자동 summarization(85%)의 트리거를 구분한다
- → `@dynamic_prompt` 와 `context_schema` 로 런타임 컨텍스트를 전파한다
- → 서브에이전트 격리와 `/memories/` 라우팅으로 컨텍스트 오염을 방지한다

13.1 4층 구조

Deep Agents의 컨텍스트 엔지니어링은 네 층으로 구성됩니다.

1. **Layered input context** — system prompt, AGENTS.md, SKILL.md, tool 설명이 정해진 순서로 합성
2. **Dynamic / runtime context** — `@dynamic_prompt` 와 `ToolRuntime` 으로 요청 시점 데이터 주입
3. **Automatic compression** — offloading(>20k 토큰 → 디스크), summarization(85% → 구조화 요약)
4. **Filesystem-centric retrieval** — `read_file` / `grep` / 서브에이전트 격리로 선별적 재주입

각 층은 독립적으로 끄고 켤 수 있고, 기본값만으로도 장시간 대화의 90%는 처리하도록 맞춰져 있습니다.

13.2 Layered input context — 시스템 프롬프트 9단계 합성

시스템 프롬프트는 다음 순서로 자동 합성됩니다.

1. Custom system prompt (사용자가 준 지시)

2. Base agent prompt (planning/filesystem/subagent 기반 지시)
3. To-do list prompt
4. Memory prompt (`AGENTS.md` , 설정 시 항상 로드)
5. Skills prompt (스킬 위치와 frontmatter 목록만)
6. Filesystem prompt (도구 문서)
7. Subagent prompt (위임 가이드)
8. Middleware prompts (커스텀 추가분)
9. Human-in-the-loop prompt

AGENTS.md: 항상 로드되는 반영구 컨텍스트

프로젝트 규약·사용자 선호처럼 모든 대화에 적용해야 하는 내용을 넣습니다.

```
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    memory=["/project/AGENTS.md", "~/.deepagents/preferences.md"],
)
```

항상 로드되므로 최소한으로 유지합니다. 상세 워크플로우는 SKILL.md로 빼는 것이 원칙입니다.

13.3 `@dynamic_prompt` — 요청 시점 데이터 주입

정적 문자열로 충분하지 않고 요청 시점 데이터(사용자 역할, 조직 ID, 현재 시각, store 내용 등)로 지시를 바꿔야 할 때 씁니다. 미들웨어는 `request.runtime.context` 와 `request.runtime.store` 를 읽어 동적 프롬프트를 만듭니다. 동적 프롬프트는 미들웨어 레이어에서 결정되고, 도구는 별도 설정 없이 `ToolRuntime` 으로 런타임 값을 받습니다.

13.4 Progressive skill loading

스킬은 2단계로 로드됩니다.

- **Startup**: `SKILL.md` 의 frontmatter(이름·설명)만 읽음 → 수백 토큰 수준
- **On demand**: 사용자 요청이 해당 스킬에 매칭될 때만 본문 전체 로드

```
agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    skills=["/skills/research/", "/skills/web-search/"],
)
```

스킬 수십 개를 등록해도 초기 토큰 비용은 frontmatter 분량뿐입니다.

Skill tools와 pinned skills

스킬은 본문만이 아니라 그 스킬을 읽은 뒤 노출할 도구까지 선언할 수 있습니다. frontmatter의 `metadata.include_tools` 에 공백으로 구분한 도구 이름을 둡니다. YAML 리스트나 `allowed-tools` 와는 의미가 다르며, 실제 도구 목록이나 resolver는 `SkillsMiddleware` 에 연결합니다.

```
---
name: issue-triage
description: GitHub 이슈 분류와 응답 초안 작성
metadata:
  include_tools: list_issues create_issue
---
```

```
from deepagents.backends import StateBackend
from deepagents.middleware import SkillsMiddleware

backend = StateBackend()
skills = SkillsMiddleware(
    sources=["/skills/"],
    backend=backend,
    tools=[list_issues, create_issue],
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    middleware=[skills],
    backend=backend,
)
```

앱이 이미 사용할 스킬을 알고 있다면 `pinned_skills` 를 생성자 인자가 아니라 실행 입력 state에 넣습니다. 이렇게 하면 모델이 스킬을 고르기 전에 해당 스킬 본문과 연결 도구가 먼저 열립니다.

```
agent.invoke({
    "messages": [{"role": "user", "content": "LangGraph 문서를 기준으로 답해줘"}],
    "pinned_skills": ["langgraph-docs"],
})
```

13.5 Runtime context propagation

`context_schema` 로 선언된 컨텍스트는 에이전트뿐 아니라 모든 서브에이전트·도구에 자동 전달됩니다.

```
from dataclasses import dataclass
from deepagents import create_deep_agent
from langchain.tools import tool, ToolRuntime

@dataclass
class Context:
    user_id: str
    api_key: str

@tool
def fetch_user_data(query: str, runtime: ToolRuntime[Context]) -> str:
    """현재 사용자 데이터를 조회한다."""
    user_id = runtime.context.user_id
    return f"Data for user {user_id}: {query}"

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    tools=[fetch_user_data],
    context_schema=Context,
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "Get my recent activity"}]},
    context=Context(user_id="user-123", api_key="sk-..."),
)
```

이 구조 덕분에 도구는 메시지에 API 키를 담지 않고도 런타임 값을 읽고, 서브에이전트도 같은 값을 자동으로 상속합니다.

> ➡️ TIP
>
> **도구 문서화 품질이 곧 컨텍스트.** 도구 선택은 모델이 설명만 보고 합니다. docstring + Args 섹션을 구체적으로 써야 토큰 낭비가 줄어듭니다. `@tool(parse_docstring=True)` 로 Args 자동 파싱을 활성화하세요.

13.6 자동 offloading (>20k 토큰)

컨텍스트가 커지면 자동으로 두 가지 오프로딩이 발동됩니다.

- **Input offloading**: 큰 파일 write/edit 결과는 컨텍스트가 85% 용량을 넘으면 실제 내용 대신 file pointer reference로 치환
- **Result offloading**: 도구 응답이 20,000 토큰을 초과하면 스토리지에 저장되고, 컨텍스트에는 파일 경로 + 첫 10라인 프리뷰만 남음

큰 결과는 일단 디스크에 쓰고, 필요할 때 `read_file` / `grep` 으로 부분만 끌어옵니다. 이것이 filesystem-centric 설계의 핵심입니다.

13.7 자동 summarization (85% 트리거)

오프로드할 것이 더 없는데 컨텍스트가 여전히 크면 구조화 요약이 발동됩니다. LLM이 현재 대화를 Session intent, Artifacts created, Next steps 세 요소로 압축합니다. 이 요약이 기존 대화 히스토리를 대체하고 에이전트는 계속 진행합니다.

항목	기본값
트리거	모델 `max_input_tokens` 의 85%
최근 보존	10%
Fallback	170,000 토큰 트리거, 최근 6 메시지 보존
즉시 트리거	`ContextOverflowError` 발생 시

수동 summarization 도구

자동 트리거 외에 에이전트가 명시적으로 요약을 호출하게 하고 싶을 때 미들웨어를 추가합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import StateBackend
from deepagents.middleware.summarization import create_summarization_tool_middleware

backend = StateBackend()
model = "google_genai:gemini-3.5-flash"

agent = create_deep_agent(
    model=model,
```

```
    middleware=[create_summarization_tool_middleware(model, backend)],
)
```

스트리밍 시 요약 토큰은 UI에 노출하지 않도록 필터링할 수 있습니다 (`metadata.get("lc_source") == "summarization"`).

13.8 Subagent isolation

서브에이전트는 자체 컨텍스트에서 실행되고 슈퍼바이저에게는 최종 리포트 하나만 돌려줍니다. 중간 도구 호출과 검색 결과는 서브에이전트 컨텍스트에 남아 슈퍼바이저 창이 깨끗하게 유지됩니다.

```
research_subagent = {
    "name": "researcher",
    "description": "특정 주제 리서치 수행",
    "system_prompt": (
        "You are a research assistant.\n"
        "IMPORTANT: Return only the essential summary (under 500 words).\n"
        "Do NOT include raw search results or detailed tool outputs."
    ),
    "tools": [web_search],
}
```

서브에이전트 프롬프트에 "최종 요약만 반환해라"를 명시하는 것이 컨텍스트 오염 방지의 첫 방어선입니다.

`mode="fork"` — 부모 대화 이어받기

기본값인 `mode="isolated"` 는 위임된 task 설명만 서브에이전트에 전달합니다. 반대로 `mode="fork"` 는 부모의 대화 히스토리와 시스템 프롬프트 prefix를 이어받아, 이미 진단한 버그를 다른 worker가 이어 고치거나 사고 분석 초안을 계속 작성할 때 씁니다.

```
fixer = {
    "name": "fixer",
    "description": "부모가 이미 진단한 수정 작업을 이어서 처리",
    "mode": "fork",
    "tools": [read_file, edit_file],
}
```

fork는 장기 메모리가 아닙니다. 현재 대화 prefix를 한 번 이어받는 실행 모드일 뿐이며, `skills` 설정은 fork 서브에이전트에서 거부됩니다. 별도 지시가 꼭 필요할 때만 `system_prompt` 를 addendum으로 붙이고, 일반적으로는 prompt cache 재사용을 위해 비워 둡니다.

13.9 CompositeBackend로 `/memories/` 라우팅

일부 경로만 영속 스토어, 나머지는 휘발성 상태입니다. 에이전트는 같은 `write_file` / `read_file` 로 접근하지만 내부적으로는 다른 백엔드가 처리합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend
from langgraph.store.memory import InMemoryStore

backend = CompositeBackend(
```

```
    default=StateBackend(),
    routes={
        "/memories/": StoreBackend(
            namespace=lambda rt: (rt.server_info.user.identity,),
        ),
    },
)

agent = create_deep_agent(
    model="google_genai:gemini-3.5-flash",
    store=InMemoryStore(),
    backend=backend,
    system_prompt=(
        "When users tell you their preferences, save them to "
        "/memories/user_preferences.txt so you remember them in future conversations."
    ),
)
```

13.10 Filesystem-centric 아키텍처

Deep Agents의 모든 컨텍스트 압축 전략은 결국 하나로 모입니다.

> ➡️ TIP
>
> **"Write big, read selective."** 큰 산출물은 디스크에 쓰고, 다시 쓸 땐 `read_file` / `grep` 으로 필요한 부분만 주입한다.

이 구조가 있기에:

- 도구 결과가 20k 토큰을 넘어도 컨텍스트는 파일 레퍼런스 + 10라인 프리뷰만 유지
- 서브에이전트는 원하는 만큼 탐색해도 슈퍼바이저 창에 영향 없음
- 메모리는 항상 로드가 아니라 필요할 때 `read_file("/memories/...")` 로 가져옴
- 스킬은 frontmatter만 보이고 본문은 on-demand

13.11 세 가지 패턴

패턴 1: 장시간 리서치

- `AGENTS.md` 는 조사 양식·인용 규칙만 (최소)
- 스킬 3 10개 등록, frontmatter만 노출
- 주제별 서브에이전트가 병렬로 탐색 → 최종 요약만 회수
- 큰 검색 결과는 자동 파일 오프로드 → `grep` 으로 다시 쿼리

패턴 2: 개인화 어시스턴트

- `/memories/` 라우팅으로 사용자별 선호 누적
- `@dynamic_prompt` 로 사용자 역할별 톤·가드레일 주입
- `context_schema` 에 `user_id` / `org_id` 선언해 전 서브에이전트에 전파
- 공유 정책은 `/policies/` (read-only) 네임스페이스

패턴 3: 비용 최적화

- 자동 summarization 트리거를 기본(85%)보다 조금 낮추기
- 서브에이전트 system_prompt에 "under 500 words" 명시
- 도구 응답이 큰 도구는 요약본 + 파일 포인터를 반환하도록 래핑

13.12 주의사항

- **AGENTS.md 과적재 금지** — 항상 로드되므로 5KB 넘기면 체감됩니다
- **스킬 frontmatter는 설명이 전부** — 매칭이 안 되면 본문이 영영 안 읽힘
- **Summarization은 파괴적** — 중요한 중간 산출물은 먼저 파일로 저장
- **서브에이전트가 raw dump를 반환하면** 슈퍼바이저 창이 오히려 더 빨리 터짐
- **도구 설명 품질이 곧 토큰 효율** — 모호한 docstring은 재시도 비용을 낳음

핵심 정리

- 컨텍스트 엔지니어링은 Layered / Dynamic / Compression / Retrieval 4층 구조
- 자동 offloading(20k 결과)과 자동 summarization(85%)이 기본 압축 메커니즘
- `@dynamic_prompt`와 `context_schema`가 런타임 컨텍스트를 전 그래프에 전파
- 서브에이전트 격리와 `/memories/` 라우팅이 컨텍스트 오염 방지의 두 축
- 전체 설계는 "Write big, read selective" 원칙 — 큰 산출물은 디스크, 재주입은 선별적

14 스트리밍

subgraphs=True + v2 네임스페이스

메인 에이전트는 물론 서브에이전트 내부 이벤트까지 실시간으로 UI에 노출하고, 커스텀 진행률 이벤트까지 방출하는 방법을 다룹니다. 장시간 실행 에이전트의 진행 상황을 사용자에게 보여주거나 서브에이전트 카드 UI를 만들 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- → `subgraphs=True` 와 `version="v2"` 조합으로 서브에이전트 내부까지 관찰한다
- → 네임스페이스(`ns`) 튜플로 메인/서브 이벤트를 라우팅한다
- → `updates` · `messages` · `custom` 세 모드를 동시에 구독한다
- → `get_stream_writer` 로 커스텀 진행률 이벤트를 방출한다
- → 서브에이전트 라이프사이클(Pending/Running/Complete) 3신호를 UI 카드에 매핑한다

14.1 세 가지 핵심

Deep Agents 스트리밍의 핵심은 세 가지입니다.

- `subgraphs=True` + `version="v2"` — 서브에이전트 내부 이벤트까지 관찰
- **네임스페이스(`ns`)** — 이벤트가 어느 서브그래프에서 왔는지 튜플로 구분
- **Stream mode 조합** — `updates` (스텝), `messages` (토큰/도구), `custom` (커스텀 이벤트)

v2 포맷은 모든 청크를 `{"type", "ns", "data"}` 통일 구조로 돌려줍니다. v1의 중첩 튜플보다 라우팅이 단순합니다.

14.2 기본 사용: 서브에이전트 스트리밍 활성화

```
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "Research quantum computing advances"}]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
```

```
):
    print(chunk)
```

`subgraphs=True`를 주지 않으면 서브에이전트 내부 동작은 슈퍼바이저 레벨에서 `task` 도구 호출 결과로만 보입니다. UI에서는 "블랙박스에서 뭔가 하다가 결과가 나오는" 경험이 됩니다.

14.3 네임스페이스로 이벤트 라우팅

모든 v2 청크는 `ns` 필드에 튜플을 담고 있으며, 이 튜플이 이벤트 발생 위치를 식별합니다.

튜플	의미
`()`	메인 에이전트 이벤트
`("tools:abc123",)`	메인 에이전트의 `task` 도구 호출로 스폰된 서브에이전트
`("tools:abc123", "model_request:def456")`	해당 서브에이전트 내부의 model request 노드

서브에이전트 이벤트 여부 판정:

```
is_subagent = any(segment.startswith("tools:") for segment in chunk["ns"])
```

메인 대화 패널과 서브에이전트 카드를 UI에서 나눌 때 이 판정이 기본 분기점입니다.

14.4 Stream mode: `updates`

노드 스텝 단위로 "지금 어느 단계를 실행 중인가"를 보여줍니다. 서브에이전트 라이프사이클 추적에 특히 유용합니다.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "updates":
        for node_name, data in chunk["data"].items():
            print(f"Step: {node_name}")
```

14.5 Stream mode: `messages`

LLM 토큰을 조각 단위로 받습니다. 도구 호출이 발생하면 각 청크에 `tool_call_chunks`가 붙어 도구 이름과 args가 스트리밍으로 쌓입니다.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="messages",
```

```
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "messages":
        token, metadata = chunk["data"]
        if token.tool_call_chunks:
            for tc in token.tool_call_chunks:
                print(f"Tool: {tc['name']}, Args: {tc.get('args')}")
        else:
            print(token.content, end="")
```

토큰 단위 출력과 도구 호출 조립을 같은 루프에서 처리할 수 있습니다.

14.6 커스텀 진행률 이벤트

도구 내부에서 스트리밍 작성자를 꺼내 임의 구조체를 방출합니다. 업로드 진행률, 처리 건수, 중간 상태 등을 UI에 내보낼 때 씁니다.

```
from langchain.tools import tool
from langgraph.config import get_stream_writer

@tool
def analyze_data(topic: str) -> str:
    """주제에 대한 데이터를 분석한다."""
    writer = get_stream_writer()
    writer({"status": "starting", "progress": 0})
    # ... 분석 작업 ...
    writer({"status": "complete", "progress": 100})
    return "done"
```

수신 측에서는 `stream_mode="custom"`을 구독합니다.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode="custom",
    subgraphs=True,
    version="v2",
):
    if chunk["type"] == "custom":
        print(chunk["data"])
```

14.7 다중 모드 동시 구독

리스트로 넘기면 루프 하나에서 세 종류 이벤트를 모두 받습니다.

```
for chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
    version="v2",
):
    t = chunk["type"]
    if t == "updates":
        # 스텝 진행
        ...
    elif t == "messages":
```

```
        # 토큰 / 도구 호출
        ...
    elif t == "custom":
        # 진행률 / 커스텀 이벤트
        ...
```

프로덕션 UI에서는 보통 이 세 모드를 함께 구독해 각자의 패널에 분배합니다.

14.8 서브에이전트 라이프사이클 추적

메인 에이전트 관점에서 서브에이전트는 세 이벤트로 식별합니다.

1. **Pending** — 메인이 `name="task"` 인 tool_call을 방출한 순간
2. **Running** — `("tools:<id>",)` 네임스페이스에서 첫 이벤트가 도착한 순간
3. **Complete** — 해당 `task` 호출의 ToolMessage가 메인 `tools` 노드에 돌아온 순간

UI 카드 상태를 이 세 신호에 매핑하면 "어떤 서브에이전트가 지금 탐색 중인지" 바로 보입니다.

14.9 v2 통합 포맷

모든 청크는 다음 세 필드를 가집니다.

```
{
    "type": "updates" | "messages" | "custom",
    "ns": tuple,
    "data": Any,
}
```

v1의 중첩 튜플 포맷은 `type` 마다 자료 구조가 달라 분기가 복잡했습니다. v2는 프런트엔드 라우팅을 `(type, ns prefix)` 하나로 단순화합니다.

14.10 프런트엔드 연동

React/Vue/Svelte/Angular는 `@langchain/react` 등의 `useStream` 훅으로 이 스트림을 자동 처리합니다. 서브에이전트 카드 렌더링, 재연결, 히스토리 로드까지 내장돼 있습니다.

```
import { useStream } from "@langchain/react";

const stream = useStream<typeof agent>({
  apiUrl: "https://your-deployment.langsmith.dev",
  assistantId: "agent",
  reconnectOnMount: true,
  fetchStateHistory: true,
});

stream.submit(
  { messages: [{ type: "human", content: text }] },
  {
    streamSubgraphs: true,
    config: { recursionLimit: 10000 },
  },
);
```

14.11 주의사항

- `subgraphs=True` **없으면 서브에이전트 내부는 보이지 않는다** — UI 진행 카드에 필수
- `version="v2"` **를 명시하라** — 레거시 포맷으로 떨어지면 네임스페이스 처리가 달라짐
- **커스텀 이벤트 스키마는 고정** — 도구마다 제각각 쓰면 UI 라우팅이 깨짐 (예: `{"status", "progress", "message"}` 공통 필드)
- **요약(summarization) 토큰은 노출 여부를 결정**해야 함 (`metadata.get("lc_source") == "summarization"` 로 필터링)
- **재시도 이벤트도 스트리밍됨** — `ModelRetryMiddleware` 가 붙어 있으면 실패-재시도 플로우가 그대로 보이므로 UI에서 집계

핵심 정리

- v2 포맷 + `subgraphs=True` 가 서브에이전트 관찰의 입구
- `ns` 튜플 prefix로 메인/서브 이벤트를 라우팅
- `updates` / `messages` / `custom` 세 모드를 리스트로 동시 구독하는 것이 프로덕션 기본
- `get_stream_writer` 로 도구에서 임의의 진행률 이벤트 방출
- Pending/Running/Complete 3신호를 UI 카드 상태에 매핑

15 권한 관리

FilesystemPermission · first-match-wins

Deep Agents의 built-in 파일시스템 도구(`ls` , `read_file` , `glob` , `grep` , `write_file` , `edit_file` , `delete`)에 선언적 allow/deny/interrupt 규칙을 적용해 경로 기반 접근 제어를 강제합니다. 프롬프트 인젝션 방어, 읽기 전용 에이전트, 특정 디렉터리만 허용하는 워크스페이스 격리를 구성할 때 이 장을 씁니다.

학습 목표

LEARNING OBJECTIVES

- → `FilesystemPermission` 의 3요소(`operations` , `paths` , `mode`)를 이해한다
- → first-match-wins 평가 규칙과 기본값(allow)을 안다
- → 읽기 전용·워크스페이스 격리·민감 파일 보호·read-only memory 4가지 패턴을 구분한다
- → 서브에이전트 permission 상속과 전면 대체 규칙을 인지한다
- → permission이 닿지 않는 커스텀 도구·MCP·샌드박스 셸의 보완 전략을 안다

15.1 적용 범위

`permissions` 는 built-in 파일시스템 도구에만 적용되는 경로 기반 규칙입니다. 다음은 우회됩니다:

- 커스텀 도구
- MCP 도구
- 샌드박스의 `execute` 셸 명령

보안 경계를 permission만으로 완성할 수 없습니다. 샌드박스에서 임의 셸 명령이 가능한 구성에서는 CompositeBackend의 라우팅 제약과 함께 설계해야 합니다. 커스텀 도구의 추가 검증·감사는 backend policy hooks가 담당합니다.

15.2 평가 규칙: first-match-wins

규칙은 리스트 순서대로 평가되고, `operations` 와 `paths` 가 현재 호출과 매치되는 첫 번째 규칙이 결과를 결정합니다. 어떤 규칙에도 매치되지 않으면 기본값은 **allow**입니다. 그러므로 구체적인 deny/allow를 앞에 두고 일반적인 fallback을 뒤에 놓아야 합니다.

15.3 규칙 3요소

각 `FilesystemPermission` 은 세 필드를 갖습니다.

필드	값	설명
`operations`	`list["read" | "write"]`	`read` = `ls` / `read_file` / `glob` / `grep` , `write` = `write_file` / `edit_file` / `delete`
`paths`	`list[str]`	글로브 패턴, `**` 재귀, `{a,b}` 선택자 지원
`mode`	`"allow" | "deny" | "interrupt"`	기본 `"allow"`

`mode="interrupt"` 는 일치하는 built-in 쓰기 도구 호출을 실행 전 승인 대기로 멈춥니다. 재개 방식은 일반 `interrupt_on` 승인 흐름과 같고, `delete` 는 파일은 정확한 경로 규칙을, 디렉터리는 대상과 모든 하위 경로의 write 권한을 보수적으로 검사합니다.

15.4 패턴 1: 읽기 전용 에이전트

모든 쓰기를 전역 차단합니다. 조사·감사·리포트 생성 에이전트에 알맞습니다.

```
from deepagents import create_deep_agent, FilesystemPermission

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.5 패턴 2: 워크스페이스 격리

`/workspace/` 아래만 허용하고 나머지는 전면 거부합니다. first-match-wins이므로 allow를 먼저, deny는 catch-all로 뒤에 둡니다.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
```

```
        ),
    ],
)
```

15.6 패턴 3: 특정 파일만 보호

`/workspace/.env`와 예제 디렉터리는 건드리지 못하게 막고, 나머지 `/workspace/`는 자유롭게 허용합니다.

```
agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=[
        # 가장 구체적인 deny를 최상단에
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/.env", "/workspace/examples/**"],
            mode="deny",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/workspace/**"],
            mode="allow",
        ),
        FilesystemPermission(
            operations=["read", "write"],
            paths=["/**"],
            mode="deny",
        ),
    ],
)
```

15.7 패턴 4: Read-only memory / policies

`/memories/` · `/policies/` 경로의 쓰기만 막습니다. 읽기는 자유입니다. 공유 메모리와 조직 정책이 프롬프트 인젝션으로 오염되는 것을 막는 기본 방어선입니다.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend, StateBackend, StoreBackend

agent = create_deep_agent(
    model=model,
    backend=CompositeBackend(
        default=StateBackend(),
        routes={
            "/memories/": StoreBackend(
                namespace=lambda rt: (rt.server_info.user.identity,),
            ),
            "/policies/": StoreBackend(
                namespace=lambda rt: (rt.context.org_id,),
            ),
        },
    ),
    permissions=[
        FilesystemPermission(
            operations=["write"],
            paths=["/memories/**", "/policies/**"],
            mode="deny",
        ),
```

```
    ],
)
```

메모리와 정책은 애플리케이션 코드로만 갱신하고, 에이전트는 읽기만 합니다.

15.8 Subagent 상속

기본 동작은 부모 에이전트의 permissions가 서브에이전트에 그대로 상속되는 것입니다. 서브에이전트 스펙에 `permissions`를 주면 부모 규칙을 완전히 대체합니다(부분 오버라이드 아님). 대체할 때는 catch-all deny까지 직접 넣어야 안전합니다.

```
parent_rules = [
    FilesystemPermission(operations=["read"], paths=["/workspace/**"], mode="allow"),
    FilesystemPermission(operations=["write"], paths=["/**"], mode="deny"),
]

agent = create_deep_agent(
    model=model,
    backend=backend,
    permissions=parent_rules,
    subagents=[
        {
            "name": "auditor",
            "description": "읽기 전용 코드 리뷰어",
            "system_prompt": "Review the code for issues.",
            "permissions": [
                # 쓰기 전역 차단
                FilesystemPermission(
                    operations=["write"], paths=["/**"], mode="deny",
                ),
                # /workspace만 읽기 허용
                FilesystemPermission(
                    operations=["read"], paths=["/workspace/**"], mode="allow",
                ),
                # 나머지 읽기 차단
                FilesystemPermission(
                    operations=["read"], paths=["/**"], mode="deny",
                ),
            ],
        },
    ],
)
```

감사 전용 서브에이전트는 읽기 권한만 주고 메인 에이전트는 넓은 권한을 유지하는 식으로 분리할 수 있습니다.

15.9 CompositeBackend 제약: sandbox-default

`CompositeBackend`의 default가 sandbox일 때, 모든 permission path는 선언된 route prefix 안에 있어야 합니다. 샌드박스가 `execute` 도구로 임의 셸 명령을 돌릴 수 있기 때문입니다. 경로 기반 규칙은 셸 수준 파일 접근을 막지 못하므로, 라우팅 외부 경로에 permission을 다는 것은 가짜 안전감입니다.

```
from deepagents import create_deep_agent, FilesystemPermission
from deepagents.backends import CompositeBackend

composite = CompositeBackend(
    default=sandbox,
    routes={"/memories/": memories_backend},
```

```
)

# 유효: /memories/ 라우트 안에 있음
agent = create_deep_agent(
    model=model,
    backend=composite,
    permissions=[
        FilesystemPermission(
            operations=["write"], paths=["/memories/**"], mode="deny",
        ),
    ],
)
```

알려진 route 바깥 경로에 규칙을 걸면 `NotImplementedError`가 발생합니다. 샌드박스 내부 파일시스템을 제어하려면 permission이 아니라 샌드박스 구성 자체(허용 바이너리, 네트워크 정책, 볼륨 마운트 범위)로 풀어야 합니다.

15.10 프롬프트 인젝션 방어 관점

공유 메모리, 조직 정책, 외부에서 주입되는 문서는 모두 인젝션 벡터입니다. 대응 계층은 다음과 같습니다.

1. **Read-only 강제**: `/memories/**`, `/policies/**`에 write deny (패턴 4)
2. **Workspace 격리**: 에이전트가 건드릴 수 있는 경로를 `/workspace/**`로 한정
3. **민감 파일 보호**: `.env` 류는 패턴 3처럼 개별 deny
4. **Subagent scope 축소**: 감사/조회 전용 서브에이전트는 read-only로 별도 구성
5. **커스텀 도구/샌드박스 경계**: permission이 닿지 않으므로 policy hook + 샌드박스 정책으로 보완

15.11 Permissions vs backend policy hooks

용도	사용 대상
Built-in FS 도구의 경로 기반 allow/deny	`permissions`
커스텀 검증(데이터 유효성, 로깅, rate limit)	backend policy hooks
커스텀 도구 / MCP 도구 제어	도구 자체 래퍼 / middleware
샌드박스 내부 파일·네트워크 통제	샌드박스 설정 (허용 바이너리·네트워크 정책)

Permission은 선언적 간이 규칙, policy hook은 로직이 필요한 통제입니다. 역할을 나눠 씁니다.

15.12 주의사항

- **first-match-wins**: 규칙 순서가 결과를 바꿈. 더 구체적인 deny/allow를 위로
- **매치 실패 = allow**: 의도치 않게 허용되는 걸 막으려면 마지막에 catch-all deny를 두는 것이 안전
- **operations 누락**: 하나의 규칙에 `"read"`만 넣으면 쓰기는 별도 규칙이 없는 한 기본 허용
- **Subagent 오버라이드는 전면 대체**: 부분 수정 불가. 부모 규칙 중 유지할 것은 복사해 넣어야 함

- **Permission은 완전한 보안 경계가 아니다**: 샌드박스·네트워크·시크릿 관리와 함께 설계되어야 의미가 있음

핵심 정리

- `FilesystemPermission` 의 3요소(`operations` / `paths` / `mode`)와 first-match-wins 평가 규칙이 기본
- 4가지 전형 패턴(읽기 전용 / 워크스페이스 격리 / 특정 파일 보호 / read-only memory) 조합으로 대부분 대응
- Subagent permission 오버라이드는 전면 대체이므로 catch-all deny 복사에 주의
- CompositeBackend + sandbox-default 구성에서는 route prefix 밖 permission이 `NotImplementedError`
- Permission은 선언적 간이 규칙이며 커스텀 도구·MCP·샌드박스 셸은 policy hook과 샌드박스 정책으로 보완

16 Models and Tools

Deep Agents의 실행 표면 설계

학습 목표

LEARNING OBJECTIVES

- → Deep Agents에서 model, tool, permission boundary가 맡는 역할을 구분합니다.
- → 도구 docstring과 schema가 에이전트 행동에 주는 영향을 확인합니다.
- → 실제 모델 호출 없이도 도구 명세와 위험도를 점검하는 방법을 익힙니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain.tools import tool
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
```

12.1 도구는 “함수”가 아니라 “계약”입니다

LLM은 Python 구현이 아니라 이름, 설명, 인자 schema를 보고 도구를 선택합니다.

```
@tool
def summarize_note(text: str, max_bullets: int = 3) -> str:
    """Summarize a note into a short Korean bullet list."""
    sentences = [s.strip() for s in text.split(".") if s.strip()]
    return "\n".join(f"- {s}" for s in sentences[:max_bullets])

print(summarize_note.invoke({"text": "A. B. C.", "max_bullets": 2}))
```

12.2 도구 schema 점검

교육 자료에서는 tool schema를 출력해 학습자가 “모델이 무엇을 보는지” 이해하게 합니다.

```
schema = summarize_note.args_schema.model_json_schema()

print("tool name:", summarize_note.name)
print("description:", summarize_note.description)
print("properties:", sorted(schema["properties"]))
```

12.3 도구 위험도 분류

도구를 만들 때는 실행 전 approval이 필요한지 먼저 분류합니다.

```
tool_policy = {
    "summarize_note": "allow",
    "write_file": "approve",
    "execute_shell": "deny-or-sandbox",
}

for name, policy in tool_policy.items():
    print(f"{name:16s} -> {policy}")
```

12.4 모델 설정은 호출부에서 통일합니다

이 저장소는 교육 기본 모델을 `gpt-5.4` 로 둡니다. 실제 실행 전에는 provider key와 비용 정책을 확인합니다.

```
MODEL_NAME = os.getenv("COURSE_MODEL", "gpt-5.4")
agent_config = {
    "model": MODEL_NAME,
    "tools": [summarize_note],
    "backend": FilesystemBackend(root_dir=".", virtual_mode=True),
}
agent_config["model"]
```

12.5 에이전트 생성은 실행과 분리합니다

아래 셀은 호출하지 않고 agent 객체만 구성합니다. 실제 `.invoke()` 는 API key, 비용, 권한 정책을 확인한 뒤 실행합니다.

```
agent = create_deep_agent(
    model=agent_config["model"],
    tools=agent_config["tools"],
    backend=agent_config["backend"],
    system_prompt="You are a concise course assistant.",
)

type(agent).__name__
```

12.6 설계 점검표

질문	기준
모델	provider, 비용, latency, context window가 맞는가?
도구	이름과 docstring이 구체적인가?
권한	read/write/execute 위험도가 분리됐는가?
backend	local, store, sandbox 중 목적에 맞는가?

정리

항목	내용
다룬 기술	`\@tool` , tool schema, `create_deep_agent` , `FilesystemBackend`
핵심 개념	Deep Agent의 품질은 model보다 tool 계약과 권한 경계에서 크게 갈립니다.
다음 단계	`13_programmatic_subagents.ipynb` , `14_event_streaming.ipynb`

> REFERENCES
>
> - `docs/deepagents/models.md`
> - `docs/deepagents/tools.md`
> - `docs/deepagents/permissions.md`
> - `docs/deepagents/backends.md`

17 Programmatic Subagents

코드로 fan-out/fan-in 설계하기

학습 목표

LEARNING OBJECTIVES

- → 일반 `SubAgent`, `AsyncSubAgent`, programmatic subagents의 차이를 구분합니다.
- → 현재 환경에서 programmatic 기능을 실행할 수 있는지 dependency gate로 확인합니다.
- → 기능이 없을 때도 동일한 설계 의도를 deterministic fallback으로 연습합니다.

```
from dotenv import load_dotenv
import importlib.util, os

load_dotenv(override=True)
```

```
import deepagents

capabilities = {
    "deepagents_version": getattr(deepagents, "__version__", "unknown"),
    "quickjs_available": importlib.util.find_spec("langchain_quickjs") is not None,
    "SubAgent": hasattr(deepagents, "SubAgent"),
    "AsyncSubAgent": hasattr(deepagents, "AsyncSubAgent"),
}
capabilities
```

13.1 언제 programmatic subagent가 필요한가

모델이 알아서 `task`를 호출하게 두는 대신, 코드가 작업 목록을 만들고 여러 worker 결과를 모아야 할 때 필요합니다.

```
tasks = [
    {"name": "coverage", "question": "공식 문서와 로컬 문서의 누락은?"},
    {"name": "tests", "question": "검증은 어떻게 할 것인가?"},
    {"name": "risks", "question": "외부 서비스 위험은?"},
]

[t["name"] for t in tasks]
```

13.2 deterministic fallback으로 fan-out/fan-in 연습

실제 subagent 실행 대신 worker 함수를 사용해 결과 수집 형태를 고정합니다.

```
def worker(task: dict) -> dict:
    return {
        "name": task["name"],
        "finding": f"{task['question']} → checklist로 관리",
    }

worker_results = [worker(task) for task in tasks]
worker_results
```

13.3 fan-in synthesis

여러 worker의 결과는 그대로 출력하지 말고, 최종 의사결정 기준으로 합칩니다.

```
summary = {
    "total_workers": len(worker_results),
    "findings": [item["finding"] for item in worker_results],
    "next_action": "공식 slug별 action matrix를 구현 체크리스트로 전환",
}

summary
```

13.4 실제 기능으로 승격할 때

조건	확인
dependency	`langchain_quickjs` 또는 공식 요구 extras 설치 여부
fallback	미설치 환경에서 실행 가능한 대체 예제
observability	worker별 trace/tag 분리
reducer	병렬 결과를 덮어쓰지 않고 누적

정리

항목	내용
다룬 기술	dependency gate, fan-out/fan-in, deterministic fallback
핵심 개념	programmatic subagent는 “모델에게 맡기는 위임”이 아니라 “코드가 제어하는 병렬 작업 분해”입니다.

다음 단계	`14_event_streaming.ipynb`

> **REFERENCES**
>
> - `docs/deepagents/programmatic-subagents.md`
> - `docs/deepagents/subagents.md`
> - `docs/deepagents/async-subagents.md`
> - `docs/langgraph/workflows-agents.md`

18 Event Streaming

Deep Agents 실행을 관찰 가능한 이벤트로 보기

학습 목표

LEARNING OBJECTIVES

- → 일반 streaming과 event streaming의 차이를 구분합니다.
- → todo, tool, subagent, filesystem 이벤트를 UI/로그 친화적 형태로 정리합니다.
- → 실제 장기 실행 agent 없이도 event consumer를 먼저 설계합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

14.1 이벤트 모델

Deep Agents frontend와 운영 로그에서는 “최종 답변”보다 중간 이벤트가 중요합니다.

```
events = [
    {"type": "todo", "payload": {"task": "outline", "status": "done"}},
    {"type": "tool", "payload": {"name": "read_file", "status": "done"}},
    {"type": "subagent", "payload": {"name": "researcher", "status": "running"}},
    {"type": "message", "payload": {"text": "초안을 작성합니다."}},
]

len(events)
```

14.2 이벤트 consumer 만들기

consumer는 provider별 raw event를 받아 UI나 로그에 필요한 projection으로 바꿉니다.

```
def project_event(event: dict) -> str:
    kind = event["type"]
    payload = event["payload"]
```

```
    if kind == "tool":
        return f"tool:{payload['name']}:{payload['status']}"
    if kind == "subagent":
        return f"subagent:{payload['name']}:{payload['status']}"
    return f"{kind}:{payload}"

[project_event(event) for event in events]
```

14.3 UI 상태로 누적하기

event stream은 append-only log로 보존하고, 화면 상태는 projection으로 만듭니다.

```
state = {"todos": [], "tools": [], "subagents": [], "messages": []}
for event in events:
    bucket = event["type"] + "s"
    if bucket in state:
        state[bucket].append(event["payload"])

state
```

14.4 운영 체크리스트

항목	질문
event schema	type과 payload가 안정적인가?
replay	같은 event log로 UI 상태를 복원할 수 있는가?
privacy	tool input/output에 민감정보가 섞이지 않는가?
fallback	streaming 미지원 환경에서 final output만으로 동작하는가?

정리

항목	내용
다룬 기술	event log, projection, UI state accumulation
핵심 개념	Event streaming은 답변 생성보다 "진행 상태를 안전하게 관찰하는 계약"입니다.
다음 단계	`08_integration/26_deepagents_frontend/` 후보

REFERENCES

- `docs/deepagents/event-streaming.md`
- `docs/deepagents/streaming.md`

- `docs/deepagents/frontend/subagent-streaming.md`
- `docs/langchain/event-streaming.md`

19 Permissions

Deep Agents 권한 경계를 먼저 설계하기

학습 목표

LEARNING OBJECTIVES

- → read, write, execute 도구의 위험도를 분리합니다.
- → approval/deny/sandbox 정책을 표로 설계합니다.
- → 권한 정책을 작은 함수로 테스트합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

15.1 권한 매트릭스

Deep Agent는 파일, shell, sandbox를 다룰 수 있으므로 권한 정책이 먼저 필요합니다.

```
permission_matrix = {
    "read_file": "allow",
    "write_file": "approve",
    "edit_file": "approve",
    "delete": "approve-high-risk",
    "execute": "sandbox-only",
}
permission_matrix
```

15.2 정책 evaluator

정책은 문서에만 두지 말고 테스트 가능한 함수로 표현합니다.

```
def decide(tool_name: str) -> str:
    return permission_matrix.get(tool_name, "deny")
```

```
for tool in ["read_file", "write_file", "delete", "execute", "unknown"]:
    print(tool, "=>", decide(tool))
```

15.3 approval 메시지

사람 승인이 필요한 도구는 이유와 예상 변경 범위를 함께 보여줘야 합니다.

```
def approval_message(tool_name: str, target: str) -> str:
    policy = decide(tool_name)
    if policy != "approve":
        return f"{tool_name}: {policy}"
    return f"승인 필요: {tool_name} will modify {target}"

approval_message("write_file", "docs/example.md")
```

정리

항목	내용
다룬 기술	permission matrix, policy evaluator, approval message
핵심 개념	권한은 agent 실행 후 수습하는 것이 아니라 tool 설계 전에 정합니다.

REFERENCES

- `docs/deepagents/permissions.md`
- `docs/deepagents/human-in-the-loop.md`
- `docs/deepagents/sandboxes.md`

20 Quality Profiles and Rubrics

모델별 설정과 완료 기준

학습 목표

LEARNING OBJECTIVES

- → Harness profile과 rubric의 역할을 구분합니다.
- → 모델/provider별 설정을 profile 형태로 관리하는 이유를 이해합니다.
- → LLM judge 없이도 rubric evaluator의 구조를 연습합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from deepagents import HarnessProfile, register_harness_profile

profile = HarnessProfile(system_prompt_suffix="Respond with concise Korean bullets.")
register_harness_profile("course:demo", profile)

type(profile).__name__
```

16.1 Rubric은 완료 기준입니다

Rubric은 “좋아 보이는 답변”이 아니라 산출물이 반드시 만족해야 할 criterion 목록입니다.

```
rubric = [
    {"name": "has_summary", "check": lambda text: "요약" in text},
    {"name": "has_next_step", "check": lambda text: "다음" in text},
]

rubric[0]["name"]
```

16.2 deterministic rubric evaluator

실제 `RubricMiddleware`는 criterion별 평가 커버리지를 요구하고, grader hook과 runtime self-revision loop에 연결됩니다. 이 셀의 evaluator는 SDK middleware가 아니라 수업용 결정적 축소판입니다.

이 구분이 중요합니다. 수업용 evaluator는 LLM 호출 없이 구조를 이해하기 위한 장치이고, `RubricMiddleware`는 실행 중 결과 품질을 검사하는 runtime middleware입니다.

```
def evaluate(text: str) -> dict:
    results = {item["name"]: item["check"](text) for item in rubric}
    return {"passed": all(results.values()), "criteria": results}

evaluate("요약: 완료. 다음: 테스트 실행")
```

16.3 Profile과 Rubric의 경계

항목	역할
profile	모델/provider별 harness 기본값, 도구 설명, 제외 도구, 추가 middleware 조정
rubric	결과물이 만족해야 할 품질 기준
eval	batch/offline 회귀 평가
middleware	runtime self-revision loop와 grader hook

Profile은 실행 환경을 바꾸고, rubric은 완료 기준을 바꿉니다. 같은 품질 문제를 다루더라도 두 레이어를 섞지 않아야 회귀 원인을 추적할 수 있습니다.

정리

항목	내용
다룬 기술	`HarnessProfile`, rubric criteria, deterministic evaluator, `RubricMiddleware` 계약
핵심 개념	profile은 실행 환경, rubric은 완료 기준을 관리합니다.
주의	수업용 결정적 evaluator와 SDK `RubricMiddleware`를 같은 것으로 설명하지 않습니다.

REFERENCES

- `docs/deepagents/profiles.md`
- `docs/deepagents/rubric.md`
- `docs/langchain/test/evals.md`

PART

V

고급 패턴

Advanced Agent Patterns

이 파트에서 다루는 내용

- 0. 마이그레이션
- 1. 미들웨어
- 2. 멀티 에이전트 — 서브에이전트
- 3. 멀티 에이전트 — 핸드오프와 라우터
- 4. 컨텍스트와 메모리
- 5. Agentic RAG
- 6. SQL 에이전트
- 7. 데이터 분석
- 8. 음성 에이전트
- 9. 프로덕션 배포
- 10. Deep Agent from Scratch
- 11. Custom Workflow Agent

00 v0 → v1 마이그레이션 가이드

LangChain/LangGraph v0에서 v1으로 전환할 때 알아야 할 브레이킹 체인지와 코드 매핑을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → v1 패키지 구조 변경과 import 경로를 이해한다
- → `create_react_agent` → `create_agent` 마이그레이션을 수행한다
- → 미들웨어 기반 동적 프롬프트, 상태 관리, 컨텍스트 주입을 적용한다
- → 표준 콘텐츠 블록과 구조화된 출력 전략을 활용한다

0.1 패키지 구조 변경

v1에서 `langchain` 네임스페이스가 에이전트 구축에 필요한 5개 핵심 모듈로 대폭 축소되었습니다:

v1 모듈	역할	주요 API
`langchain.agents`	에이전트 생성과 상태 관리	`create_agent` , `AgentState`
`langchain.messages`	메시지 타입과 콘텐츠 블록	`HumanMessage` , `AIMessage` , `ToolMessage` , `content_blocks`
`langchain.tools`	도구 정의와 런타임 주입	`\@tool` , `BaseTool` , `ToolRuntime`
`langchain.chat_models`	모델 초기화	`init_chat_model`
`langchain.embeddings`	임베딩 유틸리티	임베딩 모델 래퍼

레거시 코드 마이그레이션 — `langchain-classic`
기존에 `langchain` 패키지에서 쓰던 Chains, Retrievers, Hub, 인덱싱 API 등은 모두 `langchain-classic` 이라는 별도 패키지로 분리되었습니다. 기존 코드를 유지해야 한다면 `pip install langchain-classic` 으로 설치한 뒤 import 경로를 변경합니다:

```
# v0 — 기존 방식
from langchain.chains import LLMChain
from langchain.retrievers import MultiQueryRetriever
from langchain import hub

# v1 — langchain-classic으로 이전
from langchain_classic.chains import LLMChain
from langchain_classic.retrievers import MultiQueryRetriever
from langchain_classic import hub
```

이 분리로 v1의 `langchain` 패키지는 에이전트 빌딩에만 집중하는 경량 구조가 되었고, 레거시 기능은 독립적으로 유지보수됩니다.

0.2 에이전트 생성 API 변경

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent

model = ChatOpenAI(model="gpt-5.4")

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

# v0: from langgraph.prebuilt import create_react_agent
# v1: from langchain.agents import create_agent
agent = create_agent(
    model=model,
    tools=[add],
    system_prompt="당신은 수학 어시스턴트입니다.",  # v0: prompt=
)
print("\u2713 v1 에이전트 생성 완료")
```

```
✓ v1 에이전트 생성 완료
```

주요 변경점

v0	v1	비고
`from langgraph.prebuilt import create_react_agent`	`from langchain.agents import create_agent`	import 경로 + 함수명
`prompt=`	`system_prompt=`	파라미터명
`ToolNode` 지원	미지원	함수/BaseTool/dict만
`pre_hooks`, `post_hooks`	`middleware=[]`	통합 미들웨어
Pydantic/dataclass 상태	`TypedDict` only	상태 스키마

0.3 상태 스키마 — TypedDict only

v1에서 커스텀 상태는 반드시 `TypedDict` 기반 `AgentState`를 상속해야 합니다.

0.4 런타임 컨텍스트 주입 (신규)

v1에서는 `context_schema`와 `context` 파라미터로 불변 런타임 데이터를 에이전트에 전달할 수 있습니다. 사용자 ID, 역할, 세션 정보처럼 요청마다 달라지지만 실행 중에는 변하지 않는 데이터를 에이전트와 도구에 안전하게 넘기는 패턴입니다.

작동 방식:

1. `@dataclass`로 컨텍스트 스키마를 정의합니다.
2. `create_agent(context_schema=...)`로 스키마를 등록합니다.
3. `agent.invoke(..., context=ContextInstance(...))`로 런타임에 값을 전달합니다.
4. 도구에서는 `ToolRuntime[ContextType]` 파라미터로 컨텍스트에 접근합니다.

컨텍스트는 에이전트 상태(`AgentState`)와 달리 도구 호출 간에 변경되지 않는 읽기 전용 데이터입니다. 상태는 에이전트 루프 중 업데이트되지만, 컨텍스트는 `invoke` 호출 시 고정됩니다.

0.5 동적 프롬프트 — 미들웨어 방식 (신규)

v0의 정적 프롬프트 대신, `@dynamic_prompt`로 런타임 컨텍스트에 따라 프롬프트를 동적으로 생성합니다.

0.6 도구 에러 처리 — `\@wrap_tool_call` (신규)

v0의 `handle_tool_errors` 대신, v1은 미들웨어로 도구 에러를 처리합니다.

```
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def handle_errors(request, handler):
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"도구 오류: {e}",
            tool_call_id=request.tool_call["id"],
        )

agent = create_agent(
    model=model,
    tools=[add],
    middleware=[handle_errors],
)
print("\u2713 에러 핸들링 미들웨어 적용")
```

```
✓ 에러 핸들링 미들웨어 적용
```

미들웨어 6단계 훅 한눈에 보기

v0의 `pre_hooks` / `post_hooks` 2단계가 v1에서는 6단계로 세분화되었습니다.

훅	스타일	실행 시점	대표 용도
`@before_agent`	node	에이전트 실행 시작 1회	초기화, 인증, 사용자 컨텍스트 로드
`@before_model`	node	매 모델 호출 직전	메시지 가공, 로깅, 호출 횟수 카운팅
`@wrap_model_call`	wrap	모델 호출 감싸기	재시도, 캐싱, `request.override(...)` 로 동적 구성
`@wrap_tool_call`	wrap	도구 호출 감싸기	도구 재시도, 에러 변환, 감사 로그
`@after_model`	node	매 모델 응답 직후	가드레일, 가공, `{"jump_to": "end"}` 점프
`@after_agent`	node	에이전트 실행 종료 1회	정리, 메트릭 전송

- Node-style (`before_*` / `after_*`): 등록 순서대로 순차 실행. `after_*` 는 역순.
- Wrap-style (`wrap_*`): 핸들러 호출 여부를 0/1/N 회로 제어 → 재시도와 캐싱에 적합.

```python
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from typing import Callable

@wrap_model_call
def cache_or_call(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    cached = lookup_cache(request.messages)
    if cached:
        return cached
    response = handler(request)
    store_cache(request.messages, response)
    return response
```

0.7 표준 콘텐츠 블록 & 구조화된 출력 (신규)

v1에서 메시지는 프로바이더 무관한 `content_blocks` 를 지원합니다. 구조화된 출력은 두 가지 전략으로 분리되었습니다.

전략	import	동작	적용 시점
`ToolStrategy`	`from langchain.agents.structured_output import ToolStrategy`	인공 도구 호출로 스키마를 강제	모든 프로바이더 호환
`ProviderStrategy`	`from langchain.agents.structured_output import ProviderStrategy`	OpenAI/Anthropic 등 네이티브 structured output API 사용	지원 모델 한정, 토큰 효율

```python
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from pydantic import BaseModel
```

```python
class Answer(BaseModel):
    summary: str
    confidence: float

agent = create_agent(
    model="gpt-5.4",
    tools=[],
    response_format=ToolStrategy(Answer),
)
```

OpenAI Responses API 사용 시 메시지 콘텐츠는 기본적으로 표준 블록 형식으로 직렬화됩니다. v0 동작이 필요하면 `ChatOpenAI(..., output_version="v0")` 으로 명시합니다 (기본은 `"v1"`).

0.8 스트리밍 변경

항목	v0	v1
에이전트 노드명	`"agent"`	`"model"`
`.text`	메서드 `.text()`	프로퍼티 `.text`

0.9 기타 브레이킹 체인지

변경	설명
Python 3.10+	모든 LangChain 패키지가 Python 3.10 이상을 요구합니다. 3.9 이하는 미지원됩니다.
반환 타입	채팅 모델의 반환 타입이 `BaseMessage` 에서 `AIMessage` 로 고정되었습니다.
OpenAI Responses API	메시지 콘텐츠가 기본적으로 표준 블록 형식입니다. `output_version="v0"` 으로 이전 동작을 복구할 수 있습니다.
Anthropic max_tokens	기본값이 1024에서 모델별 자동 설정으로 변경되었습니다.
AIMessage.example	`example` 파라미터가 제거되었습니다. `additional_kwargs` 를 사용하세요.
AIMessageChunk	`chunk_position` 속성이 추가되었습니다 (마지막 청크에 `'last'` 값).
`.text` 프로퍼티	`.text()` 메서드가 `.text` 프로퍼티로 변경되었습니다.
파일 인코딩	파일이 기본적으로 UTF-8 인코딩으로 열립니다.

요약 — 마이그레이션 체크리스트

- [ ] Python 3.10+ 확인
- [ ] `create_react_agent` → `create_agent` 변경

- [ ] `prompt=` → `system_prompt=` 변경
- [ ] 상태 스키마를 `TypedDict` 기반 `AgentState`로 전환
- [ ] `pre_hooks` / `post_hooks` → `middleware=[]`
- [ ] `ToolNode` → 함수/BaseTool로 교체
- [ ] `.text()` → `.text` 프로퍼티
- [ ] 스트리밍 노드명 `"agent"` → `"model"` 확인
- [ ] 레거시 import를 `langchain-classic`으로 이전

01 미들웨어 시스템 심화

v1 최대 신기능

학습 목표

LEARNING OBJECTIVES

- → 에이전트 루프의 6단계 미들웨어 훅(`before_agent` / `before_model` / `wrap_model_call` / `wrap_tool_call` / `after_model` / `after_agent`)을 이해한다
- → 프로바이더 무관 빌트인(Summarization, HITL, Model/Tool Call Limit, ModelFallback, PII, LLMToolSelector, ContextEditing)과 프로바이더 전용(AnthropicPromptCaching, BedrockPromptCaching, PatchToolCalls) 미들웨어를 구분해 사용한다
- → `ModelRequest` / `ToolCallRequest` / `ModelResponse` / `ExtendedModelResponse` 타입을 활용해 데코레이터·클래스 기반 커스텀 미들웨어를 작성한다
- → `request.override(...)` , `@hook_config(can_jump_to=...)` , `state_schema=` 옵션으로 동적 구성과 에이전트 점프를 구현한다
- → 다중 미들웨어 조합 시 실행 순서(`before` 순방향, `after` 역방향, `wrap` 중첩)를 정확히 예측한다

1.1 환경 설정

미들웨어는 에이전트 실행의 각 단계에 훅(hook)을 삽입하여 모니터링, 변환, 신뢰성, 거버넌스를 구현하는 v1의 핵심 기능입니다. `create_agent` 함수의 `middleware` 파라미터에 미들웨어 인스턴스 리스트를 전달해 사용합니다.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

1.2 미들웨어 아키텍처 개요

에이전트 루프는 모델 호출 → 도구 선택 → 도구 실행 → 종료 판단의 반복 사이클입니다. 미들웨어는 이 사이클의 6단계에 훅(hook)을 삽입해 세밀하게 제어합니다.

6단계 훅

훅	스타일	실행 시점	대표 용도
`before_agent`	node	에이전트 실행 시작 1회	인증, 사용자 컨텍스트 로드
`before_model`	node	매 모델 호출 직전	프롬프트 보강, 카운팅, 로깅
`wrap_model_call`	wrap	모델 호출 감싸기	재시도, 캐싱, `request.override(...)` 동적 구성
`wrap_tool_call`	wrap	도구 호출 감싸기	도구 재시도, 에러 변환, 감사 로그
`after_model`	node	매 모델 응답 직후	가드레일, `{"jump_to": "end"}` 점프
`after_agent`	node	에이전트 실행 종료 1회	정리, 메트릭 전송

두 가지 훅 스타일

- Node-style (`before_*` , `after_*`): 순차 실행. 로깅·검증·상태 업데이트에 적합.
- Wrap-style (`wrap_*`): 핸들러 호출 여부를 0/1/N 회로 제어. 재시도·캐싱·동적 라우팅에 적합.

시그니처와 타입

훅	인자	반환
`wrap_model_call(request, handler)`	`request: ModelRequest` , `handler: Callable[[ModelRequest], ModelResponse]`	`ModelResponse` 또는 `ExtendedModelResponse`
`wrap_tool_call(request, handler)`	`request: ToolCallRequest` , `handler: Callable[[ToolCallRequest], ToolMessage \\], [Command]`	`ToolMessage` 또는 `Command`
`before_model(state, runtime)` / `after_model(state, runtime)`	`state: AgentState` , `runtime: Runtime`	`dict], [None` (`jump_to` 지원)

- `ModelRequest` 는 `messages` , `tools` , `model` , `system_message` , `response_format` , `state` , `runtime` 필드를 노출합니다.
- `ToolCallRequest` 는 `tool_call` , `state` , `runtime` 필드를 노출합니다.
- `ExtendedModelResponse` 는 `model_response` + `command` 로, `wrap_model_call` 에서 영구 상태 업데이트를 보낼 때 씁니다.

미들웨어는 에이전트의 핵심 로직을 건드리지 않고도 모니터링, 변환, 신뢰성, 거버넌스 등 횡단 관심사(cross-cutting concerns)를 깔끔하게 분리할 수 있게 해줍니다.

```python
from langchain.agents import create_agent
from langchain.agents.middleware import (
    SummarizationMiddleware,
    HumanInTheLoopMiddleware,
)

agent = create_agent(
    model="gpt-5.4", tools=[],
    middleware=[
        SummarizationMiddleware(model="gpt-5.4-mini", trigger=("messages", 50)),
        HumanInTheLoopMiddleware(interrupt_on={}),
    ],
)
```

1.3 SummarizationMiddleware

대화가 길어져 컨텍스트 윈도우를 초과할 때 이전 대화를 자동으로 요약·압축합니다. 장시간 실행되는 대화, 다중 턴 대화, 전체 대화 맥락을 보존해야 하는 애플리케이션에 적합합니다.

주요 파라미터

파라미터	설명	예시
`model`	요약 생성에 사용할 경량 모델 (비용 절감)	`"gpt-5.4-mini"`
`trigger`	요약 트리거 조건	`("tokens", 4000)` , `("messages", 50)` , `("fraction", 0.8)`
`keep`	요약 후 유지할 최근 컨텍스트	`("messages", 20)`
`token_counter`	커스텀 토큰 카운팅 함수	선택적
`summary_prompt`	커스텀 요약 프롬프트 템플릿	선택적

`trigger` 는 토큰 수, 메시지 수, 윈도우 비율 중 하나를 기준으로 설정할 수 있으며, 조건에 도달하면 `keep` 에 지정된 최근 메시지를 제외한 나머지를 요약문으로 교체합니다.

```python
from langchain.agents.middleware import SummarizationMiddleware

summarizer = SummarizationMiddleware(
    model="gpt-5.4-mini",
    trigger=("tokens", 4000),
    keep=("messages", 20),
)
```

1.4 HumanInTheLoopMiddleware

고위험 도구 호출 전에 에이전트 실행을 멈추고 인간 승인을 기다립니다. 데이터베이스 쓰기, 금융 거래, 이메일 전송 같은 고위험 작업이나 인간 감독이 필요한 컴플라이언스 워크플로우에 씁니다.

`checkpointer` 필수 — 중단 후 상태를 복원하려면 체크포인터가 반드시 있어야 합니다.

결정 유형

결정	설명	사용 방법
`approve`	도구 호출 승인 및 실행	`Command(resume="approve")`
`edit`	도구 인자 수정 후 실행	`Command(resume={"type": "edit", "args": {...}})`
`reject`	도구 호출 거부	`Command(resume={"type": "reject", "reason": "..."})`

`interrupt_on` 딕셔너리에서 각 도구별로 승인 정책을 설정합니다. `False` 로 설정하면 해당 도구는 중단 없이 실행됩니다.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
        "read_email": False,
    }
)
```

```
agent = create_agent(
    model="gpt-5.4", tools=[],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
)
```

1.5 ModelCallLimitMiddleware & ToolCallLimitMiddleware

무한 루프나 과도한 API 비용을 막는 호출 제한 미들웨어입니다.

ModelCallLimitMiddleware

에이전트가 모델을 호출하는 횟수를 제한합니다. 폭주하는 에이전트 방지, 프로덕션 비용 제어, 테스트 시 호출 예산 관리에 씁니다.

ToolCallLimitMiddleware

도구 호출 횟수를 전역적으로 또는 특정 도구별로 제한합니다. 비용이 높은 외부 API 호출 제한, 검색/DB 쿼리 빈도 제어, 특정 도구의 레이트 리밋 적용에 유용합니다.

공통 파라미터

파라미터	설명
`thread_limit`	전체 스레드(모든 invoke)에서의 최대 호출 수
`run_limit`	단일 invoke 실행에서의 최대 호출 수

`exit_behavior`	`"end"` (정상 종료), `"error"` (예외 발생), `"continue"` (에러 메시지와 함께 계속 — ToolCallLimit 전용)

ToolCallLimitMiddleware는 추가로 `tool_name` 파라미터를 받아 특정 도구에만 제한을 적용할 수 있습니다.

```
from langchain.agents.middleware import ModelCallLimitMiddleware

model_limit = ModelCallLimitMiddleware(
    thread_limit=10,
    run_limit=5,
    exit_behavior="end",
)
```

```
from langchain.agents.middleware import ToolCallLimitMiddleware

# 전역 제한
global_tool_limit = ToolCallLimitMiddleware(thread_limit=20, run_limit=10)

# 특정 도구 제한
search_limit = ToolCallLimitMiddleware(
    tool_name="search",
    thread_limit=5, run_limit=3,
    exit_behavior="continue",
)
```

1.6 ModelFallbackMiddleware

주 모델 실패 시 대체 모델 체인으로 자동 전환합니다. 프로덕션 장애 대응, 비용 최적화(비싼 모델 → 저렴한 모델 폴백), 멀티 프로바이더 중복성(OpenAI + Anthropic 등) 확보에 유용합니다.

생성자에 폴백 모델을 순서대로 전달하면, 주 모델 호출이 실패할 때 지정된 순서로 대체 모델을 시도합니다. 모든 폴백이 실패하면 최종 에러가 발생합니다.

```
from langchain.agents.middleware import ModelFallbackMiddleware

# gpt-5.4 실패 -> gpt-5.4-mini -> claude
fallback = ModelFallbackMiddleware(
    "gpt-5.4-mini",
    "claude-sonnet-4-6",
)
```

1.7 PIIMiddleware

개인 식별 정보(PII)를 자동 탐지하고 설정된 전략에 따라 처리합니다. 의료/금융 컴플라이언스, 고객 서비스 에이전트의 로그 세정, 민감한 사용자 데이터 처리에 씁니다.

빌트인 PII 타입

`email` , `credit_card` , `ip` , `mac_address` , `url`

처리 전략

전략	동작	예시 (이메일)
`block`	예외 발생 — PII 발견 시 실행 중단	에러 발생
`redact`	`[REDACTED_TYPE]` 으로 교체	`[REDACTED_EMAIL]`
`mask`	부분 마스킹	`u**\@example.com`
`hash`	결정적 해싱	`a1b2c3d4...`

적용 범위

- `apply_to_input` : 사용자 입력 메시지 검사
- `apply_to_output` : AI 응답 메시지 검사
- `apply_to_tool_results` : 도구 실행 결과 검사

커스텀 탐지기

빌트인 PII 타입 외에 세 가지 방식으로 커스텀 탐지기를 만들 수 있습니다:

1. 정규식 문자열: 간단한 패턴 매칭
2. 컴파일된 정규식 (`re.compile`): 복잡한 정규식
3. 함수: 검증 로직이 필요한 고급 탐지 (반환: `list[dict]` — `text` , `start` , `end` 키 포함)

```
from langchain.agents.middleware import PIIMiddleware

email_pii = PIIMiddleware("email", strategy="redact", apply_to_input=True)
card_pii = PIIMiddleware("credit_card", strategy="mask", apply_to_input=True)
```

```
# 커스텀 탐지기: 정규식 문자열
api_key_pii = PIIMiddleware(
    "api_key",
    detector=r"sk-[a-zA-Z0-9]{32}",
    strategy="block",
)
```

```
import re

# 커스텀 탐지기: 컴파일된 정규식
phone_pii = PIIMiddleware(
    "phone_number",
    detector=re.compile(r"\+?\d{1,3}[\s.-]?\d{3,4}[\s.-]?\d{4}"),
    strategy="mask",
)
```

```
# 커스텀 탐지기: 함수 (SSN 예시)
def detect_ssn(content: str) -> list[dict]:
    matches = []
    for m in re.finditer(r"\d{3}-\d{2}-\d{4}", content):
        first = int(m.group(0)[:3])
        if first not in [0, 666] and not (900 <= first <= 999):
            matches.append({"text": m.group(0), "start": m.start(), "end": m.end()})
    return matches

ssn_pii = PIIMiddleware("ssn", detector=detect_ssn, strategy="hash")
```

1.8 LLMToolSelectorMiddleware

도구가 10개 이상일 때, 경량 LLM이 사용자 쿼리를 분석해 관련 도구만 선별합니다. 불필요한 도구 설명으로 인한 토큰 낭비를 줄이고, 모델이 관련 도구에 집중해 정확도를 높입니다.

주요 파라미터

파라미터	설명	기본값
`model`	도구 선택용 모델	에이전트의 메인 모델
`system_prompt`	커스텀 선택 지침	내장 프롬프트
`max_tools`	최대 선택 도구 수	전체
`always_include`	항상 포함할 도구 이름 리스트	`[]`

선택 모델로 `gpt-5.4-mini` 같은 경량 모델을 쓰면 비용을 절감하면서도 효과적인 도구 필터링이 가능합니다.

```python
from langchain.agents.middleware import LLMToolSelectorMiddleware

tool_selector = LLMToolSelectorMiddleware(
    model="gpt-5.4-mini",
    max_tools=3,
    always_include=["search"],
)
```

1.8b 신규 빌트인 — 프로바이더 캐싱·도구 호출 복구·컨텍스트 편집

기본 7종 외에 v1.1+에서 추가된 빌트인 미들웨어들입니다. 비용과 신뢰성에 직접 영향을 주므로 프로덕션 스택의 핵심입니다.

미들웨어	패키지	목적
`AnthropicPromptCachingMiddleware`	`langchain_anthropic.middleware`	Claude 시스템 프롬프트/도구 정의/이전 턴 캐시 히트화
`BedrockPromptCachingMiddleware`	`langchain_aws.middleware.prompt_caching`	Bedrock 호스팅 Claude에 동일 캐싱 적용
`PatchToolCallsMiddleware`	`deepagents.middleware.patch_tool_calls`	모델이 만든 잘못된 도구 호출(타입 오류, 알 수 없는 이름, 잘린 args)을 도구 노드 도달 전에 보정
`ModelFallbackMiddleware` (재)	`langchain.agents.middleware`	1차 모델 실패 시 순차 폴백
`ContextEditingMiddleware`	`langchain.agents.middleware`	토큰 한도 도달 시 오래된 도구 출력 정리

`AnthropicPromptCachingMiddleware`의 `ttl`은 `"5m"` 또는 `"1h"`, `min_messages_to_cache`로 캐싱 활성화 메시지 수를 조정합니다. `PatchToolCallsMiddleware`는 `create_deep_agent(...)` 사용 시 기본 스택에 자동 포함됩니다.

```
# AnthropicPromptCachingMiddleware — 긴 시스템 프롬프트/도구를 캐시 히트로
from langchain_anthropic import ChatAnthropic
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware
from langchain.agents import create_agent

cached_agent = create_agent(
    model=ChatAnthropic(model="claude-sonnet-4-6"),
    system_prompt="<긴 시스템 프롬프트>",
    middleware=[AnthropicPromptCachingMiddleware(ttl="5m", min_messages_to_cache=0)],
)
```

```
# BedrockPromptCachingMiddleware — Bedrock 호스팅 Claude 동일 캐싱
from langchain_aws import ChatBedrockConverse
from langchain_aws.middleware.prompt_caching import BedrockPromptCachingMiddleware

bedrock_agent = create_agent(
    model=ChatBedrockConverse(model="us.anthropic.claude-sonnet-4-5-20250929-v1:0"),
    system_prompt="<긴 시스템 프롬프트>",
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

```
# PatchToolCallsMiddleware — 잘못된 도구 호출을 도구 노드 도달 전에 보정
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware

patched_agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[],
    middleware=[PatchToolCallsMiddleware()],
)
# 참고: create_deep_agent(...) 사용 시 기본 미들웨어 스택에 자동 포함
```

```
# ContextEditingMiddleware — 토큰 임계 도달 시 오래된 도구 출력을 placeholder로 교체
from langchain.agents.middleware import ContextEditingMiddleware, ClearToolUsesEdit

ctx_edit_agent = create_agent(
    model="gpt-5.4",
    tools=[],
    middleware=[
        ContextEditingMiddleware(
            edits=[ClearToolUsesEdit(trigger=100_000, keep=3)],
        ),
    ],
)
```

1.9 커스텀 미들웨어 작성

두 가지 구현 방식이 있습니다:

1. 데코레이터 방식

단일 훅, 간단한 로직에 적합합니다. `@before_model`, `@after_model`, `@wrap_model_call`, `@wrap_tool_call` 데코레이터를 사용합니다.

2. 클래스 방식 (`AgentMiddleware`)

여러 훅을 조합하거나 설정이 필요한 경우 `AgentMiddleware` 를 상속합니다. sync/async 구현을 동시에 제공할 수 있습니다.

커스텀 상태

미들웨어는 `NotRequired` 타입 힌트로 에이전트 상태를 확장할 수 있습니다. 실행 간 값 추적, 훅 간 데이터 공유, 레이트 리밋이나 감사 로깅 같은 횡단 관심사 구현에 활용합니다.

에이전트 점프

`after_model` 등에서 딕셔너리를 반환하여 에이전트 흐름을 제어할 수 있습니다:

- `{"jump_to": "end"}` — 에이전트 즉시 종료
- `{"jump_to": "tools"}` — 도구 실행 단계로 이동
- `{"jump_to": "model"}` — 모델 호출 단계로 이동

```python
from langchain.agents.middleware import before_model

@before_model
def log_before(state, runtime):
    """모델 호출 전 메시지 수를 기록합니다."""
    print(f"[LOG] 메시지 {len(state.get('messages', []))}개")
```

```python
from langchain.agents.middleware import after_model

@after_model
def validate_output(state, runtime):
    """가드: 금지된 콘텐츠를 차단합니다."""
    last = state["messages"][-1].content
    if "FORBIDDEN" in last:
        return {"jump_to": "end"}
```

```python
from typing import Callable
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

@wrap_model_call
def retry_on_error(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """실패 시 모델 호출을 최대 2회 재시도합니다."""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception:
            if attempt == 2: raise
```

```python
# wrap_tool_call — 도구 호출을 감싸 감사 로그/에러 변환
from typing import Callable
from langchain.agents.middleware import wrap_tool_call, ToolCallRequest
from langchain.messages import ToolMessage
from langgraph.types import Command

@wrap_tool_call
def audit_tool(
    request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage | Command],
) -> ToolMessage | Command:
    """도구 호출 전후로 감사 로그를 남기고 예외를 ToolMessage 로 변환."""
```

```
    name = request.tool_call["name"]
    print(f"[AUDIT] tool={name} args={request.tool_call['args']}")
    try:
        return handler(request)
    except Exception as e:
        return ToolMessage(
            content=f"도구 오류: {e}",
            tool_call_id=request.tool_call["id"],
        )
```

```
# request.override(...) — wrap_model_call 에서 모델/도구/시스템 프롬프트를 단일 호출 한정으로 교체
@wrap_model_call
def dynamic_config(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """상태 기반으로 모델 호출 시 system_prompt 와 tools 를 동적으로 교체."""
    if request.state.get("expert_mode"):
        request = request.override(
            system_prompt="당신은 시니어 분석가입니다.",
            # tools=[...specialist_tools],
        )
    return handler(request)
```

```
# @hook_config(can_jump_to=[...]) — after_model 에서 합법 점프 대상 선언
from typing import Any
from langchain.agents.middleware import after_model, hook_config, AgentState
from langchain.messages import AIMessage
from langgraph.runtime import Runtime

@after_model
@hook_config(can_jump_to=["end"])
def block_forbidden(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    last = state["messages"][-1]
    if "BLOCKED" in getattr(last, "content", ""):
        return {
            "messages": [AIMessage("요청에 응답할 수 없습니다.")],
            "jump_to": "end",
        }
    return None
```

```
# ExtendedModelResponse — wrap_model_call 에서 모델 응답 + 영구 상태 업데이트를 함께 반환
from langchain.agents.middleware import ExtendedModelResponse

@wrap_model_call
def track_tokens(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ExtendedModelResponse:
    response = handler(request)
    return ExtendedModelResponse(
        model_response=response,
        command=Command(update={"last_model_call_tokens": 150}),
    )
```

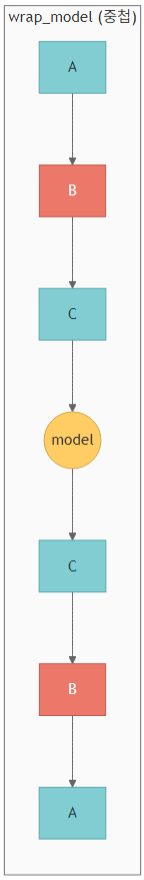
```
from langchain.agents.middleware import AgentMiddleware

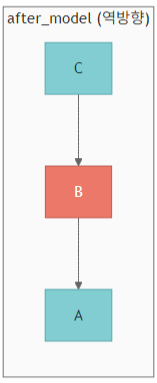
class AuditMiddleware(AgentMiddleware):
    def __init__(self, log_file="audit.log"):
        self.log_file = log_file
    def before_model(self, state, config):
        print(f"[AUDIT] before -> {self.log_file}")
    def after_model(self, state, config):
        print(f"[AUDIT] after -> {self.log_file}")
```

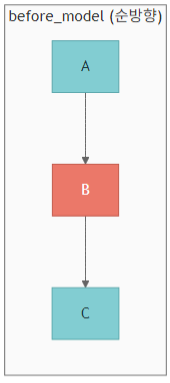
1.10 미들웨어 실행 순서

다중 미들웨어 등록 시 실행 순서를 정확히 이해해야 예상치 못한 동작을 막을 수 있습니다.

`middleware=[A, B, C]` 등록 시:


wrap_model (중첩)
A
B
C
model
C
B
A


after_model (역방향)
C
B
A


before_model (순방향)
A
B
C

실전 팁

- PII 검출은 로깅보다 먼저 등록해야 로그에 PII가 포함되지 않습니다.
- 폴백 미들웨어는 재시도 미들웨어보다 뒤에 배치하여, 재시도 실패 후 폴백이 작동하도록 합니다.
- `wrap` 훅에서 `next_fn`을 호출하지 않으면 이후 미들웨어와 실제 호출이 모두 건너뛰어집니다.

```
@before_model
def mw_a(state, runtime): print("before A")

@before_model
def mw_b(state, runtime): print("before B")

@before_model
def mw_c(state, runtime): print("before C")

# 실행: A -> B -> C (after_model이면 C -> B -> A)
```

1.11 미들웨어 조합 (Stacking)

프로덕션 환경에서는 여러 미들웨어를 함께 사용해 종합적인 에이전트 거버넌스를 구현합니다. 미들웨어는 등록 순서에 따라 실행되므로, 보안(PII) → 신뢰성(폴백) → 비용 제어(호출 제한) → 컨텍스트 관리(요약) → 최적화(도구 선택) → 감독(HITL) 순으로 배치하는 것이 권장됩니다.

이런 조합에서 각 미들웨어는 단일 책임 원칙을 지키면서도, 전체적으로 강력한 프로덕션 에이전트 파이프라인을 구성합니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    PIIMiddleware, ModelFallbackMiddleware,
    ModelCallLimitMiddleware, SummarizationMiddleware,
    HumanInTheLoopMiddleware, LLMToolSelectorMiddleware,
    ContextEditingMiddleware, ClearToolUsesEdit,
)
from deepagents.middleware.patch_tool_calls import PatchToolCallsMiddleware
from langgraph.checkpoint.memory import InMemorySaver
```

```
middleware_stack = [
    PIIMiddleware("email", strategy="redact", apply_to_input=True),
    PatchToolCallsMiddleware(),
    ModelFallbackMiddleware("gpt-5.4-mini", "claude-sonnet-4-6"),
    ModelCallLimitMiddleware(thread_limit=50, run_limit=10),
    ContextEditingMiddleware(edits=[ClearToolUsesEdit(trigger=100_000, keep=3)]),
    SummarizationMiddleware(model="gpt-5.4-mini", trigger=("fraction", 0.8)),
]

production_agent = create_agent(
    model="gpt-5.4", tools=[], checkpointer=InMemorySaver(), middleware=middleware_stack,
)
```

요약

항목	핵심 내용

아키텍처	6단계 훅: `before_agent` / `before_model` / `wrap_model_call` / `wrap_tool_call` / `after_model` / `after_agent`
타입	`ModelRequest`, `ToolCallRequest`, `ModelResponse`, `ExtendedModelResponse`
프로바이더 무관 빌트인	Summarization, HITL, ModelCallLimit, ToolCallLimit, ModelFallback, PII, LLMToolSelector, ContextEditing
프로바이더 전용	AnthropicPromptCaching, BedrockPromptCaching, PatchToolCalls (Deep Agents 기본 스택)
커스텀	데코레이터(`\@before_model`, `\@wrap_model_call`, `\@wrap_tool_call` 등) / `AgentMiddleware` 클래스
동적 구성	`request.override(system_prompt=, tools=, model=, response_format=)`
상태 점프	`\@hook_config(can_jump_to=["end"], ["tools"], ["model"])`
영구 상태	`ExtendedModelResponse(model_response=..., command=Command(update={...}))`
실행 순서	`before` : 순방향, `after` : 역방향, `wrap` : 중첩
프로덕션 스택	PII → PatchToolCalls → Fallback → Limit → ContextEditing → Summarization → ToolSelector → HITL

02 멀티에이전트: Subagents

감독자 패턴

학습 목표

LEARNING OBJECTIVES

- → 감독자 → 서브에이전트 → 도구의 3계층 아키텍처를 설계한다
- → 서브에이전트를 `@tool` 로 래핑하거나 Deep Agents의 `SubAgentMiddleware` 로 일괄 등록한다
- → 서브에이전트 상태 관리(상속 vs `checkpointer=True`), HITL, `ToolRuntime[None, CustomState]` 컨텍스트 주입을 익힌다
- → 디스패치 도구 노출 3가지(시스템 프롬프트 열거 / `Literal` 제약 / 도구 기반 발견)와 비동기 5-tool 패턴(start/check/update/cancel/list)을 구분해 적용한다

2.1 환경 설정

Subagents 패턴은 중앙 감독자(Supervisor) 에이전트가 전문화된 서브에이전트들을 도구처럼 호출해 작업을 위임하는 멀티에이전트 아키텍처입니다. 캘린더와 이메일 도메인을 처리하는 개인 비서 시스템을 구축합니다.

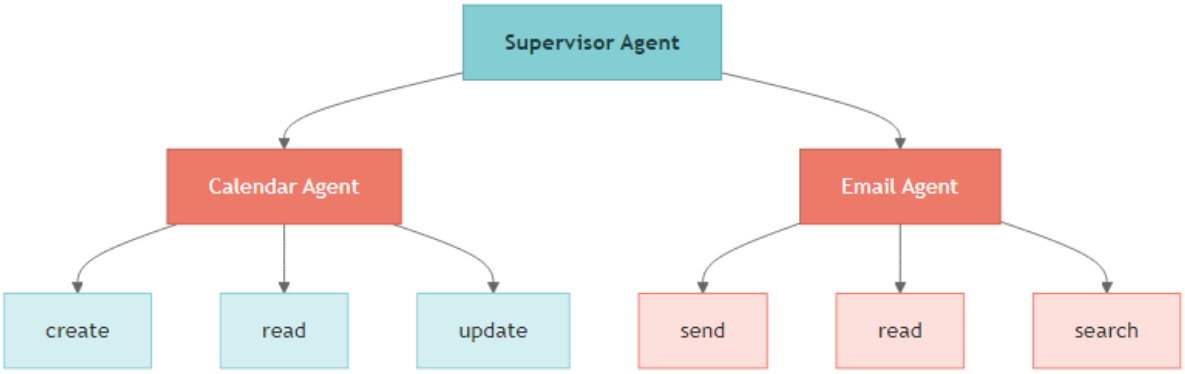
```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

2.2 Subagents 아키텍처 개요

Subagents 패턴은 3계층 아키텍처로 구성됩니다. 감독자가 모든 라우팅을 담당하고, 서브에이전트는 사용자와 직접 상호작용하지 않으며, 결과를 감독자에게 반환합니다.



계층	역할	특징
저수준 도구	외부 서비스 직접 호출 (Calendar API, Email API)	단순한 함수 래퍼
서브에이전트	도메인별 추론 + 도구 조합	전문 시스템 프롬프트, 독립적 도구 세트
감독자	작업 분해, 위임, 결과 집계	전체 대화 기억, 서브에이전트 = 도구로 취급

핵심 특성

- 중앙 집중 제어: 모든 라우팅이 감독자를 통해 흐릅니다
- 컨텍스트 격리: 서브에이전트는 매번 깨끗한 컨텍스트 윈도우에서 실행되어 컨텍스트 비대화를 방지합니다
- 병렬 실행: 여러 서브에이전트를 한 턴에서 동시에 호출할 수 있습니다
- 도구 기반 호출: 서브에이전트를 `@tool`로 래핑하여 감독자에게 일반 도구처럼 노출합니다

사용 시점

서브에이전트 패턴은 여러 도메인(캘린더, 이메일, CRM 등)을 관리하면서 서브에이전트가 사용자와 직접 대화할 필요 없고, 중앙화된 워크플로 관리가 필요할 때 적합합니다. 도구가 적은 단순한 시나리오에서는 단일 에이전트로 충분합니다.

2.3 저수준 도구 정의

3계층 아키텍처의 최하층인 저수준 도구를 정의합니다. 외부 서비스(Calendar API, Email API)와 직접 상호작용하는 단순한 함수 래퍼입니다. 실제 프로덕션에서는 Google Calendar API, Email Service 등과 연동하지만, 여기서는 학습용 스텁(stub) 구현을 사용합니다.

도구 설계 시 주의할 점:

- 하나의 도구는 하나의 기능만 담당 (단일 책임 원칙)
- 동일한 도구를 여러 서브에이전트에 중복 할당하지 않아야 합니다
- docstring을 명확하게 작성하여 LLM이 도구를 올바르게 선택할 수 있게 합니다

```
from langchain_core.tools import tool

@tool
```

```python
def create_calendar_event(
    title: str, start_time: str, end_time: str,
    attendees: list[str] = None,
) -> str:
    """새 캘린더 이벤트를 생성합니다."""
    return f"이벤트 '{title}' 생성됨: {start_time} ~ {end_time}"
```

```python
@tool
def read_calendar_events(date: str) -> str:
    """날짜(YYYY-MM-DD)의 캘린더 이벤트를 조회합니다."""
    return f"{date}에 이벤트가 없습니다."
```

```python
@tool
def send_email(to: str, subject: str, body: str) -> str:
    """이메일 메시지를 전송합니다."""
    return f"{to}에게 이메일 전송됨: '{subject}'"

@tool
def read_emails(folder: str = "inbox", limit: int = 10) -> str:
    """폴더에서 최근 이메일을 읽습니다."""
    return f"{folder}에 이메일 3개"
```

```python
@tool
def search_emails(query: str, limit: int = 10) -> str:
    """검색어로 이메일을 검색합니다."""
    return f"'{query}' 검색 결과 2건"
```

2.4 서브에이전트 생성

각 서브에이전트는 `create_agent()` 로 생성하며, 세 가지 핵심 요소를 갖습니다:

1. 전문화된 시스템 프롬프트: 도메인별 역할과 행동 지침을 정의합니다
2. 도메인별 도구 세트: 해당 도메인의 도구만 할당하여 관심사를 분리합니다
3. `name` 식별자: 감독자가 서브에이전트를 구분하고 호출할 때 사용합니다

서브에이전트의 세분화 수준(granularity)은 도메인 단위(캘린더, 이메일 등)가 권장됩니다. 너무 세분화하면 감독자의 라우팅 부담이 커지고, 너무 통합하면 컨텍스트 격리의 이점이 줄어듭니다.

상태 관리 모드

서브에이전트는 두 가지 체크포인트 모드를 지원합니다.

모드	설정	동작
상속(Inherited) — 기본	`checkpointer` 미지정	매 호출이 깨끗한 상태로 시작. 부모 체크포인터를 투명하게 공유. 인터럽트·병렬 실행에 안전
영구(Persistent)	`checkpointer=True`	서브에이전트가 호출 간 자체 대화 히스토리를 유지. 이전 턴을 부모와 독립적으로 기억해야 할 때

```
calendar_agent_persistent = create_agent(
    model="claude-sonnet-4-6",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="당신은 캘린더 어시스턴트입니다.",
    name="calendar_agent",
    checkpointer=True,   # 호출 간 자체 히스토리 유지
)
```

> ➡️ TIP
>
> 서브그래프의 `get_state` 는 nested agent state 를 반환하지 않습니다(정적 발견 한계). 인터럽트 중에는 노드 함수에서 상태를 확인하세요.

```
from langchain.agents import create_agent

calendar_agent = create_agent(
    model="gpt-5.4",
    tools=[create_calendar_event, read_calendar_events],
    system_prompt="당신은 캘린더 어시스턴트입니다. ISO 8601 날짜 형식을 사용하세요.",
    name="calendar_agent",
)
```

```
email_agent = create_agent(
    model="gpt-5.4",
    tools=[send_email, read_emails, search_emails],
    system_prompt="당신은 이메일 어시스턴트입니다. 메시지를 전문적으로 작성하세요.",
    name="email_agent",
)
```

2.5 서브에이전트를 도구로 래핑

서브에이전트를 감독자에게 노출하는 표준 패턴은 `@tool` 데코레이터로 감싸는 것입니다. 래핑 함수 내부에서 `subagent.invoke()` 를 호출하고, 마지막 메시지의 `content` 를 반환합니다.

이 패턴의 장점:

- 감독자 입장에서 서브에이전트는 일반 도구와 동일하게 취급됩니다
- 서브에이전트의 내부 구현이 바뀌어도 감독자에 영향을 주지 않습니다
- 입출력 형식을 래핑 함수에서 자유롭게 커스터마이징할 수 있습니다

입출력 전략 선택: 쿼리만 전달(간단)할 수도 있고, 전체 컨텍스트를 전달(정교)할 수도 있습니다. 결과 반환 시에도 최종 결과만 반환하거나 전체 히스토리를 반환하는 선택지가 있습니다.

2.6 감독자 에이전트 조립

감독자는 래핑된 서브에이전트 도구들을 `tools` 에 전달받아 생성됩니다. 감독자의 시스템 프롬프트에는 작업 분해(task decomposition) 및 위임(delegation) 지침을 포함합니다.

감독자 설계 시 고려사항:

- 에러 처리: 서브에이전트 실패를 감독자가 적절히 처리해야 합니다

- 결과 집계: 여러 서브에이전트의 결과를 통합하여 사용자에게 일관된 응답을 제공합니다
- 승인 범위: 상태를 변경하는 작업(이메일 전송, 이벤트 생성)에만 HITL을 적용합니다

```
supervisor = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt=(
        "당신은 개인 비서입니다. 복잡한 요청을 "
        "하위 작업으로 분해하고 적절한 에이전트에 위임하세요."
    ),
)
```

2.7 실행 테스트

```
User: "내일 2시 Sarah 미팅 잡고 초대 이메일 보내줘"
Supervisor → call_calendar → create_calendar_event
Supervisor → call_email → send_email
Supervisor: "미팅과 초대 이메일 완료"
```

2.8 HITL (Human-in-the-Loop) 통합

`HumanInTheLoopMiddleware`와 `checkpointer`를 결합하면 고위험 도구 호출(이메일 전송, 일정 생성 등) 전에 사용자 승인을 요청할 수 있습니다. 에이전트가 보호된 도구를 호출하려 하면 실행이 일시 중지되고, 사용자가 검토합니다.

승인 응답 유형

응답	설명	코드
승인(Approve)	도구 호출을 그대로 실행	`Command(resume="approve")`
편집(Edit)	도구 인자를 수정 후 실행	`Command(resume={"type": "edit", "args": {...}})`
거부(Reject)	도구 호출을 취소	`Command(resume={"type": "reject", "reason": "..."})`

HITL은 상태를 변경하는 작업(send_email, create_calendar_event 등)에만 적용하는 것이 권장됩니다. 읽기 전용 작업에는 불필요한 마찰을 줄이기 위해 적용하지 않습니다.

Subagent Design Best Practices

원칙	설명
명확한 설명	메인 에이전트가 언제 위임할지 알 수 있도록 구체적인 `description` 작성
전문화된 프롬프트	출력 형식, 제약, 금지 사항을 서브에이전트별로 분리

완전한 지시	기본 isolated 모드는 호출마다 새 문맥이므로 필요한 입력·파일 경로·완료 기준을 한 번에 전달
fork 신중 사용	`mode="fork"` 는 부모 문맥을 이어받는 실험적 모드이며, 부모 skills를 상속하고 별도 skills를 받지 않음
상태 공유 명시	공유가 필요하면 파일이나 StoreBackend 경로를 사용하고, task 호출 간 기억을 가정하지 않음

```
supervisor_hitl = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    checkpointer=InMemorySaver(),
    middleware=[hitl],
    system_prompt="당신은 개인 비서입니다.",
)
```

2.9 컨텍스트 주입 — `ToolRuntime[None, CustomState]`

서브에이전트를 호출하는 도구는 `ToolRuntime[ContextSchema, StateSchema]` 시그니처로 부모 에이전트의 상태와 컨텍스트에 접근할 수 있습니다. 두 번째 타입 매개변수가 부모 `state_schema` 이므로, 여기서 메시지 히스토리·사용자 ID·임의의 상태 키를 끌어와 서브에이전트 입력에 주입합니다.

- 첫 번째 매개변수가 `None` 이면 컨텍스트 스키마 없이 상태만 주입합니다.
- `runtime.context.<field>` 로 정적 런타임 컨텍스트(사용자 ID, DB 연결)에 접근합니다.
- `runtime.state["messages"]` 등 부모 상태 키를 그대로 읽을 수 있습니다.

```
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState

class SupervisorState(AgentState):
    user_id: str

@tool
def call_research_agent(query: str, runtime: ToolRuntime[None, SupervisorState]) -> str:
    """부모 상태의 메시지 히스토리를 함께 전달."""
    history = runtime.state["messages"]
    result = research_agent.invoke({"messages": history + [{"role": "user", "content": query}]})
    return result["messages"][-1].content
```

매 프롬프트마다 텍스트로 반복 주입하지 않고도 사용자 신원·타임존·중간 결과를 서브에이전트에 흘려보낼 수 있어 토큰 비용과 일관성이 모두 개선됩니다.

```
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import AgentState, create_agent

@dataclass
class UserContext:
    user_email: str
    user_name: str
    timezone: str

class SupervisorState(AgentState):
```

```
    # 부모 에이전트가 보유한 추가 상태 키 예시
    user_id: str
```

```
supervisor_ctx = create_agent(
    model="gpt-5.4",
    tools=[call_calendar, call_email],
    system_prompt="당신은 개인 비서입니다.",
    context_schema=UserContext,
    state_schema=SupervisorState,
)
```

```
# 도구에서 부모 상태 + 컨텍스트 동시 접근
@tool
def send_email_ctx(
    to: str, subject: str, body: str,
    runtime: ToolRuntime[UserContext, SupervisorState],
) -> str:
    """현재 사용자로 이메일을 전송합니다."""
    sender = runtime.context.user_email
    user_id = runtime.state.get("user_id", "anon")
    return f"[{user_id}] {sender}에서 {to}로 이메일 전송: '{subject}'"
```

2.10 비동기 실행 패턴 — 5-tool 워크플로

서브에이전트의 실행 모드는 두 가지입니다.

모드	동작	사용 시점
동기(Synchronous)	감독자가 서브에이전트 완료를 대기 후 다음 진행	결과가 다음 작업에 필요할 때 (기본값)
비동기(Asynchronous)	즉시 task id 반환, 백그라운드 실행, 도중에 지시·취소 가능	장시간 작업, 병렬 다중 서브에이전트, mid-flight steering 필요 시

Deep Agents 0.5+ 의 `AsyncSubAgentMiddleware` 는 감독자에게 다음 5개 도구를 자동 주입합니다 — 비동기 라이프사이클의 단위 연산을 그대로 매핑한 형태입니다.

도구	역할
`start_async_task`	서브에이전트 백그라운드 기동, task id 즉시 반환
`check_async_task`	상태 조회, 완료 시 최종 출력 추출
`update_async_task`	같은 thread 에 follow-up 지시 주입 (interrupt)
`cancel_async_task`	서버에 cancel 신호 전송
`list_async_tasks`	추적 중 모든 task 의 live 상태 일괄 조회

비동기 모드는 Agent Protocol 서버(LangSmith Deployments, `langgraph dev`)를 요구합니다.
`async_tasks` state 채널이 메시지 히스토리와 분리되어 있어, 감독자 컨텍스트가 요약되어도 task id 가 유실되지 않습니다.

아래 학습용 스텁은 5개 도구의 시그니처와 흐름을 직접 구현해 본 예시입니다. 실제 프로덕션에서는 `AsyncSubAgent(name=..., graph_id=...)` 를 `create_deep_agent(subagents=...)` 에 넘기면 충분합니다.

```
import uuid
from datetime import datetime

job_store: dict[str, dict] = {}

@tool("start_async_task", description="서브에이전트 백그라운드 기동, task id 반환")
def start_async_task(agent_name: str, instruction: str) -> str:
    """비동기 서브에이전트 작업을 시작하고 task id를 반환합니다."""
    task_id = "task_" + uuid.uuid4().hex[:12]
    job_store[task_id] = {
        "agent": agent_name,
        "status": "running",
        "instruction": instruction,
        "result": None,
        "created_at": datetime.utcnow().isoformat(),
    }
    return f"started {task_id} (agent={agent_name})"

@tool("check_async_task", description="task 상태 조회 / 완료 시 결과 추출")
def check_async_task(task_id: str) -> str:
    job = job_store.get(task_id)
    if not job:
        return f"not_found: {task_id}"
    # 학습용 스텁: 한 번 호출되면 완료로 표시
    if job["status"] == "running":
        job["status"] = "success"
        job["result"] = f"{job['agent']} 결과: {job['instruction'][:60]}"
    return f"{task_id} status={job['status']} result={job.get('result')!r}"
```

```
@tool("update_async_task", description="실행 중 task 에 follow-up 지시 주입")
def update_async_task(task_id: str, instruction: str) -> str:
    job = job_store.get(task_id)
    if not job:
        return f"not_found: {task_id}"
    job["instruction"] = instruction
    job["status"] = "running"   # interrupt 기반 재기동
    return f"updated {task_id} with new instruction"

@tool("cancel_async_task", description="task 취소")
def cancel_async_task(task_id: str) -> str:
    job = job_store.get(task_id)
    if not job:
        return f"not_found: {task_id}"
    job["status"] = "cancelled"
    return f"cancelled {task_id}"

@tool("list_async_tasks", description="추적 중 task 일괄 조회")
def list_async_tasks() -> str:
    if not job_store:
        return "no tasks"
    return "\n".join(
        f"{tid} agent={j['agent']} status={j['status']}"
        for tid, j in job_store.items()
    )
```

2.11 단일 디스패치 도구 패턴

도구 패턴에는 두 가지 접근법이 있습니다.

패턴	설명	장점

에이전트별 도구	서브에이전트마다 별도의 래핑 도구 생성	세밀한 제어, description 커스터마이징 용이
단일 디스패치 도구	하나의 파라미터화된 도구로 모든 서브에이전트 호출	확장성 우수, 서브에이전트 추가/제거가 독립적

단일 디스패치 도구가 감독자에게 어떤 서브에이전트가 있는지 알려주는 방식은 3가지로 나뉩니다.

방식	적합한 규모	트레이드오프
시스템 프롬프트 열거	10개 미만, 정적 레지스트리	가장 단순. 프롬프트 수동 업데이트 필요
`Literal` / Enum 제약	10개 미만, 타입 안전	스키마 레벨 검증, 프롬프트 부담 없음
도구 기반 발견	대규모/동적 레지스트리	Progressive disclosure, 와이어링 복잡

`Literal` 제약 패턴은 `agent_name` 파라미터에 가능한 값을 타입으로 못박아 LLM 이 잘못된 이름을 만들 수 없게 합니다. 대규모 레지스트리라면 `list_agents` 도구를 따로 두고 감독자가 필요할 때 조회하도록 합니다.

Deep Agents — `SubAgentMiddleware` 로 일괄 등록

수동으로 `@tool` 래핑하는 대신, Deep Agents 의 `SubAgentMiddleware` 가 `task` 디스패치 도구와 표준 시스템 프롬프트를 자동 주입합니다. 동기 서브에이전트가 하나라도 있으면 자동 부착되며 `excluded_middleware` 로 제거할 수 없습니다.

```
supervisor_dispatch = create_agent(
    model="gpt-5.4",
    tools=[dispatch],
    system_prompt=(
        "delegate 도구를 사용하여 작업을 라우팅하세요. "
        "에이전트: 'calendar'(일정), 'email'(이메일)."
    ),
)
```

```
# Deep Agents — SubAgentMiddleware 로 일괄 등록 (수동 @tool 래핑 불필요)
from deepagents.backends import StateBackend
from deepagents.middleware.subagents import SubAgentMiddleware

supervisor_dag = create_agent(
    model="claude-sonnet-4-6",
    middleware=[
        SubAgentMiddleware(
            backend=StateBackend(),
            subagents=[
                {
                    "name": "calendar",
                    "description": "캘린더 이벤트 생성/조회",
                    "system_prompt": "당신은 캘린더 어시스턴트입니다. ISO 8601 사용.",
                    "tools": [create_calendar_event, read_calendar_events],
                    "model": "claude-sonnet-4-6",
                },
                {
                    "name": "email",
                    "description": "이메일 전송/읽기/검색",
                    "system_prompt": "당신은 이메일 어시스턴트입니다.",
                    "tools": [send_email, read_emails, search_emails],
                    "model": "gpt-5.4",   # 서브에이전트별 모델 오버라이드
                },
            ],
```

```
        ),
    ],
)
# 감독자는 자동 주입된 `task(subagent_name, instruction)` 도구로 위임
```

요약

항목	핵심
3계층	도구 → 서브에이전트(`create_agent`) → 감독자
수동 래핑	`\@tool` + `subagent.invoke()` → 마지막 content 반환
Deep Agents	`SubAgentMiddleware` (`subagents=[{...}]`) — `task` 디스패치 도구 자동 주입
체크포인트 모드	상속(기본) vs `checkpointer=True` (서브에이전트 자체 히스토리)
격리	서브에이전트 = 깨끗한 컨텍스트, 감독자만 전체 기억
HITL	`HumanInTheLoopMiddleware` + 체크포인터, decision: approve/edit/reject
컨텍스트	`ToolRuntime[ContextSchema, ParentState]` — 부모 state·context 동시 주입
비동기 5-tool	`start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks`
디스패치 노출	프롬프트 열거 / `Literal` 제약 / 도구 기반 발견

03 멀티에이전트: Handoffs & Router

상태 머신과 병렬 라우팅

학습 목표

LEARNING OBJECTIVES

- → Handoffs 패턴(단일 에이전트 + `@wrap_model_call` + `request.override(system_prompt=, tools=)`)으로 상태 기반 동적 구성을 구현한다
- → 핸드오프 도구가 `Command(update={"current_step": ..., "messages": [ToolMessage(tool_call_id=...)]})` 로 상태와 대화 히스토리를 동시에 업데이트하는 규약을 익힌다
- → 서브그래프 멀티 에이전트 경로(`Command.PARENT`)와 단일 에이전트 미들웨어 경로를 비교하고, 서브그래프 사이 핸드오프 시 `AIMessage` + `ToolMessage` 2개만 전달하는 규칙을 따른다
- → Router 패턴(전용 분류 단계 → `Send` API fan-out → Reducer fan-in)으로 병렬 라우팅과 결과 합성을 구현한다
- → Router 와 Supervisor(=Subagents)의 차이를 용어 수준에서 구분한다

Part A — Handoffs: Customer Support 상태 머신

3.1 환경 설정

이 노트북은 두 가지 멀티에이전트 패턴을 다룹니다:

- Part A — Handoffs: 단일 에이전트가 상태 변수에 따라 동적으로 프롬프트와 도구를 교체하는 상태 머신 패턴
- Part B — Router: 쿼리를 분류하여 전문 에이전트들에게 병렬로 라우팅하고 결과를 합성하는 패턴

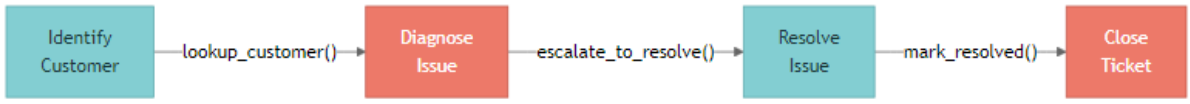
```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

3.2 Handoffs 개요

Handoffs 패턴은 단일 에이전트가 상태 변수에 따라 동적으로 행동을 바꾸는 아키텍처입니다. 여러 에이전트를 전환하는 것이 아니라, 하나의 에이전트가 단계(step)에 따라 다른 시스템 프롬프트와 도구 세트를 사용합니다.



두 가지 구현 경로

경로	구조	사용 시점
단일 에이전트 + 미들웨어 (권장)	하나의 `create_agent` + `\@wrap_model_call` 이 `request.override(system_prompt=, tools=)` 로 단계별 구성 교체	대부분의 핸드오프 시나리오
서브그래프 멀티 에이전트	단계마다 독립 그래프, `Command(goto=..., graph=Command.PARENT)` 로 부모 그래프 사이 이동	각 단계가 자체 reflection/retrieval 등 복잡한 내부 그래프를 가질 때

핵심 메커니즘 (단일 에이전트 경로)

메커니즘	설명
`current_step`	현재 단계를 추적하는 상태 변수. 미들웨어가 이 값을 보고 다음 호출 구성을 결정
`Command(update={...})`	핸드오프 도구가 반환. `current_step` 변경 + 필요한 데이터 저장 + `messages` 동기화
`\@wrap_model_call` + `request.override(...)`	LLM 호출 직전 `system_prompt` 와 `tools` 를 단계별 설정으로 단일 호출 한정 교체

핸드오프 도구 시그니처

핸드오프 도구는 `ToolRuntime[None, SupportState]` 를 받아 `runtime.tool_call_id` 를 얻고, `Command.update.messages` 에 그 id 와 매칭되는 `ToolMessage` 를 함께 넣어야 합니다.

```python
from langchain.tools import tool, ToolRuntime
from langgraph.types import Command
from langchain.messages import ToolMessage

@tool
def transfer_to_resolve(runtime: ToolRuntime[None, SupportState]) -> Command:
    return Command(update={
        "current_step": "resolve_issue",
        "messages": [ToolMessage(
            content="Transferred to resolve step",
```

```
            tool_call_id=runtime.tool_call_id,
        )],
    })
```

`tool_call_id` 가 누락되거나 매칭되지 않으면 LLM 이 대화 히스토리를 malformed 로 거부합니다. (LangChain 의 도구 호출/응답 쌍 규약)

서브그래프 경로의 메시지 규칙

서브그래프 멀티 에이전트로 갈 때는 핸드오프 경계를 정확히 두 개의 메시지로만 흘려보냅니다.

1. 송신 에이전트의 `AIMessage` — 원본 도구 호출 포함
2. 매칭되는 `tool_call_id` 가 들어간 `ToolMessage` — 핸드오프 ack

이 외의 메시지가 더 끼거나 빠지면 수신 에이전트가 잘못된 컨텍스트로 시작합니다.

사용 시점

순차적 제약(sequential constraints)이 필요하거나, 각 상태에서 사용자와 직접 대화하거나, 다단계 플로우(예: 고객 지원)에서 정보를 특정 순서로 수집해야 할 때 적합합니다.

3.3 SupportState 정의

`AgentState` 를 상속하여 `current_step` 필드를 추가합니다. 이 필드가 상태 머신의 현재 노드를 결정하며, `Literal` 타입으로 유효한 단계를 제한합니다.

기본값은 `"identify_customer"` 로, 모든 대화가 고객 식별 단계에서 시작됩니다. 이후 도구가 `Command(update={"current_step": "..."})` 를 반환하면 자동으로 다음 단계로 전이됩니다.

```
from langchain.agents import AgentState
from typing import Literal

class SupportState(AgentState):
    current_step: Literal[
        "identify_customer", "diagnose_issue",
        "resolve_issue", "close_ticket",
    ] = "identify_customer"
```

3.4 단계별 도구 정의

각 단계(step)에는 해당 단계의 역할에 맞는 도구들이 할당됩니다. 도구는 두 가지 유형으로 나뉩니다:

- 상태 전이 도구: `Command(update={...})` 를 반환하여 `current_step` 을 변경하고 추가 데이터를 상태에 저장합니다. `result` 필드로 LLM에게 보여줄 문자열 결과도 함께 전달합니다.
- 일반 도구: 문자열을 반환하며 상태를 변경하지 않습니다. 정보 조회 등에 사용됩니다.

설계 권장사항:

- 상태 전이는 `Command` 를 반환하는 도구를 통해서만 이루어져야 합니다
- 역방향 전이(이전 단계로 되돌아가기)도 필요 시 허용합니다
- 미들웨어에서 유효하지 않은 전이를 검증하여 단계 건너뛰기를 방지합니다

```
from langchain_core.tools import tool
from langgraph.types import Command

# --- Identify Customer ---
@tool
def lookup_customer(email: str) -> Command:
    """이메일로 고객을 조회합니다."""
    return Command(
        update={"customer": {"name": "Alice", "id": "C-1234"}, "current_step": "diagnose_issue"},
        result="고객 찾음: Alice (C-1234). 진단 단계로 이동합니다.",
    )
```

```
# --- Diagnose Issue ---
@tool
def check_service_status(service_name: str) -> str:
    """서비스의 현재 상태를 확인합니다."""
    return f"서비스 '{service_name}': 정상 (99.9% 가동률)"
```

```
@tool
def escalate_to_resolve(diagnosis: str) -> Command:
    """진단 후 해결 단계로 이동합니다."""
    return Command(
        update={"diagnosis": diagnosis, "current_step": "resolve_issue"},
        result=f"진단 완료: {diagnosis}. 해결 단계로 이동합니다.",
    )
```

```
# --- Resolve Issue ---
@tool
def apply_fix(fix_type: str, customer_id: str) -> Command:
    """고객 계정에 수정 사항을 적용합니다."""
    return Command(
        update={"resolution": {"type": fix_type}},
        result=f"수정 적용됨: {customer_id}에 {fix_type}",
    )
```

```
@tool
def mark_resolved(summary: str) -> Command:
    """이슈를 해결 완료로 표시하고 종료 단계로 이동합니다."""
    return Command(
        update={"current_step": "close_ticket", "resolution_summary": summary},
        result="해결됨. 종료 단계로 이동합니다.",
    )
```

```
# 정식 핸드오프 시그니처 — ToolRuntime.tool_call_id 를 ToolMessage 에 echo
from langchain.tools import ToolRuntime
from langchain.messages import ToolMessage

@tool
def transfer_to_resolve(
    diagnosis: str,
    runtime: ToolRuntime[None, SupportState],
) -> Command:
    """진단 후 해결 단계로 이동 (tool_call_id echo 패턴)."""
    return Command(update={
        "current_step": "resolve_issue",
        "diagnosis": diagnosis,
        "messages": [ToolMessage(
            content=f"진단 완료: {diagnosis}. 해결 단계로 이동합니다.",
            tool_call_id=runtime.tool_call_id,
        )],
    })
```

```
# --- Close Ticket ---
@tool
def send_satisfaction_survey(customer_id: str) -> str:
    """만족도 설문을 전송합니다."""
    return "설문 전송 완료."

@tool
def close_ticket(ticket_id: str, notes: str) -> str:
    """지원 티켓을 종료합니다."""
    return f"티켓 {ticket_id} 종료됨."
```

3.5 @wrap_model_call 미들웨어

`@wrap_model_call` 미들웨어는 Handoffs 패턴의 핵심입니다. LLM 호출을 가로채어 `current_step`에 따라 시스템 프롬프트와 사용 가능 도구를 동적으로 교체합니다.

동작 순서:

1. 미들웨어가 상태에서 `current_step` 값을 읽습니다
2. `STEP_CONFIG` 딕셔너리에서 해당 단계의 설정(프롬프트 + 도구)을 조회합니다
3. `config`를 오버라이드하여 LLM에 전달합니다
4. `next_fn(state, config)`으로 수정된 설정으로 LLM을 호출합니다

Handoffs의 핵심 메커니즘이 바로 이것입니다: 단일 에이전트가 상태에 따라 완전히 다른 페르소나와 능력을 갖게 됩니다. 다중 에이전트 없이도 미들웨어 하나로 동적 행동 변경을 달성합니다.

```
STEP_CONFIG = {
    "identify_customer": {
        "tools": [lookup_customer],
        "system_prompt": "고객을 식별하세요. 이메일 또는 계정 ID를 요청하세요.",
    },
    "diagnose_issue": {
        "tools": [check_service_status, escalate_to_resolve],
        "system_prompt": "이슈를 진단하세요. 도구를 사용한 후 escalate_to_resolve를 호출하세요.",
    },
}
```

```
STEP_CONFIG["resolve_issue"] = {
    "tools": [apply_fix, mark_resolved],
    "system_prompt": "이슈를 해결하세요. 수정을 적용한 후 mark_resolved를 호출하세요.",
}
STEP_CONFIG["close_ticket"] = {
    "tools": [send_satisfaction_survey, close_ticket],
    "system_prompt": "고객에게 감사를 전하고, 설문을 보내고, 티켓을 종료하세요.",
}
```

```
from typing import Callable
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

@wrap_model_call
def step_middleware(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """current_step 에 따라 system_prompt 와 tools 를 단일 호출 한정으로 교체."""
    step = request.state.get("current_step", "identify_customer")
    cfg = STEP_CONFIG[step]
    request = request.override(
```

```
        system_prompt=cfg["system_prompt"],
        tools=cfg["tools"],
    )
    return handler(request)
```

3.6 에이전트 생성 및 실행 흐름

에이전트 생성 시 모든 도구를 등록하되, `state_schema=SupportState`와 미들웨어를 지정합니다. 미들웨어가 런타임에 `current_step`에 따라 도구를 필터링하므로, 각 단계에서는 해당 단계의 도구만 LLM에 노출됩니다.

실행 흐름 예시:

```
[identify_customer] User: "로그인이 안 돼요. 이메일: alice@example.com"
  → lookup_customer("alice@example.com")
  ← Command(update={customer: {...}, current_step: "diagnose_issue"})

[diagnose_issue] Agent: "계정을 찾았습니다. 어떤 문제가 있나요?"
  → check_service_status("auth-service") → "healthy"
  → escalate_to_resolve("3회 로그인 실패로 잠김")

[resolve_issue] → apply_fix("reset_password", "C-1234")
  → mark_resolved("비밀번호 재설정 완료")

[close_ticket] → send_satisfaction_survey("C-1234")
  → close_ticket("T-5678", notes="비밀번호 재설정")
```

각 단계 전이는 `Command`를 반환하는 도구에 의해 자동으로 이루어집니다.

```
from langchain.agents import create_agent

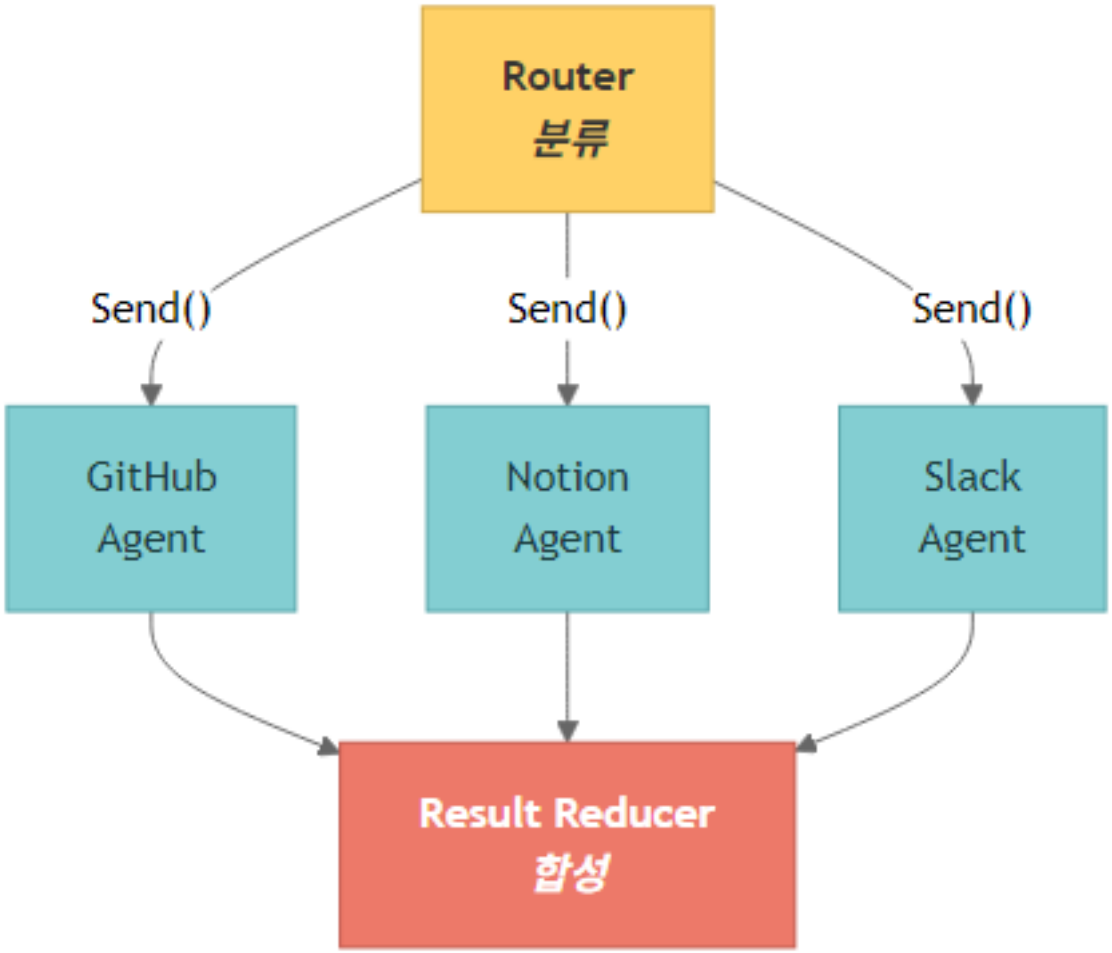
all_tools = [
    lookup_customer, check_service_status,
    escalate_to_resolve, apply_fix, mark_resolved,
    send_satisfaction_survey, close_ticket,
]
support_agent = create_agent(
    model="gpt-5.4", tools=all_tools,
    state_schema=SupportState, middleware=[step_middleware],
)
```

```
# [identify_customer]
#   User: "Can't log in. Email: alice@example.com"
#   Agent -> lookup_customer("alice@example.com")
#        <- Command(update={current_step: "diagnose_issue"})
#
# [diagnose_issue] (auto transition)
#   Agent -> check_service_status("auth-service")
#   Agent -> escalate_to_resolve("Locked out")
#
# [resolve_issue] -> [close_ticket]
```

Part B — Router: 병렬 라우팅과 결과 합성

3.7 Router 개요

Router 패턴은 입력을 분류하여 전문 에이전트들에게 라우팅하는 아키텍처입니다. Subagents 패턴과 달리, Router는 전용 분류 단계(단일 LLM 호출 또는 규칙 기반 로직)를 거쳐 쿼리를 배분합니다.



파이프라인

단계	역할	구현
분류(Classification)	쿼리를 분석하여 관련 소스와 서브쿼리를 생성	`with_structured_output(QueryClassification)`
병렬 디스패치	분류된 각 소스에 동시에 서브쿼리 전달	`Send` API
결과 합성(Reduction)	모든 에이전트 결과를 수집하여 통합 응답 생성	Reducer 노드 + LLM

Router vs. Supervisor — 용어 정리

LangChain 공식 분류는 다음과 같이 두 패턴을 명확히 구분합니다.

항목	Router	Supervisor (= Subagents)
의사결정 위치	전용 라우팅 단계 – 단일 LLM 호출 또는 규칙 기반	대화 안에서 메인 에이전트(=감독자)가 매 턴 판단
대화 인식	기본 없음 (stateless), 도구 wrapping 으로 stateful 화	항상 stateful, 진행 맥락에 따라 다음 액션 결정
사용 시점	입력 카테고리가 분명, 결정적 경량 분류로 충분	유연한 대화 중심 오케스트레이션, 진행에 따라 다음 단계가 바뀌는 워크플로
병렬 fan-out	자연스럽게 `Send` 로 다중 소스 동시 조회	동시 호출 가능하지만 감독자 판단에 의존

요컨대 별개의 지식 도메인(vertical)이 명확히 구분되어 있고 병렬 조회가 필요할 때가 Router 의 자리이고, 대화 흐름이 분기·재진입을 반복하면 Supervisor 가 적합합니다.

아키텍처 모드

- Stateless: 각 요청이 독립적으로 라우팅됩니다 (메모리 없음)
- Stateful: 대화 히스토리를 유지하여 멀티턴 상호작용을 지원합니다. Stateless 라우터를 도구로 래핑하거나, 라우터 자체가 상태를 직접 관리하는 방식이 있습니다

3.8 RouterState 및 분류 스키마

`QueryClassification` 은 Pydantic 모델로, LLM의 `with_structured_output()` 을 통해 쿼리를 구조화된 형태로 분류합니다. `RouterState` 는 분류 결과, 소스 목록, 서브쿼리, 에이전트 결과를 추적합니다.

분류 스키마의 핵심 필드:

- `sources` : 어떤 지식 소스가 관련 있는지 (복수 선택 가능)
- `reasoning` : 왜 해당 소스를 선택했는지 설명
- `sub_queries` : 소스별로 최적화된 서브쿼리 (원래 쿼리를 각 소스에 맞게 재구성)

```python
from pydantic import BaseModel, Field
from typing import Literal

class SubQuery(BaseModel):
    """소스별 하위 쿼리."""
    source: Literal["github", "notion", "slack"] = Field(description="지식 소스.")
    query: str = Field(description="해당 소스에 최적화된 검색 쿼리.")

class QueryClassification(BaseModel):
    """사용자 쿼리의 분류 결과."""
    sources: list[Literal["github", "notion", "slack"]] = Field(
        description="관련 지식 소스."
    )
    reasoning: str = Field(description="해당 소스를 선택한 이유.")
    sub_queries: list[SubQuery] = Field(description="소스별 하위 쿼리.")
```

```python
from langchain.agents import AgentState

class RouterState(AgentState):
    classification: QueryClassification = None
    sources: list[str] = []
```

```
    sub_queries: list[SubQuery] = []
    agent_results: list[dict] = []
```

3.9 분류 노드

`with_structured_output`으로 쿼리를 소스별로 분류하고, 각 소스에 최적화된 서브쿼리를 생성합니다.

분류 예시

사용자 쿼리	분류 소스	이유
“auth 서비스 배포 방법”	`["github", "notion"]`	배포 코드는 GitHub, 절차 문서는 Notion에 존재
“API 변경 결정 경위”	`["slack", "notion"]`	논의는 Slack, 결정 문서는 Notion에 기록
“로그인 버그 PR”	`["github"]`	PR은 GitHub에만 존재
“온보딩 프로세스와 스타터 레포”	`["github", "notion", "slack"]`	레포는 GitHub, 프로세스는 Notion, 맥락은 Slack

서브쿼리 생성이 중요합니다: “auth 서비스 배포”라는 원본 쿼리를 GitHub에는 `"auth service deployment scripts CI/CD pipeline"`, Notion에는 `"auth service deployment process procedure runbook"`으로 각각 최적화합니다.

3.10 병렬 라우팅 (Send API)

`Send` API는 분류된 각 소스에 동시에 서브쿼리를 디스패치합니다. `Send(node_name, payload)` 형태로, 그래프의 특정 노드에 데이터를 병렬로 전달합니다.

```
from langgraph.types import Command, Send

def dispatch(state):
    classifications = state["classification"].sources
    return [Send(c["agent"], {"messages": [...], "query": c["query"]}) for c in classifications]
```

Router 패턴의 핵심 강점이 바로 이 병렬 실행입니다. 여러 지식 소스를 순차적으로 조회하면 지연 시간이 합산되지만, `Send`를 통한 fan-out은 가장 느린 소스의 응답 시간만큼만 걸립니다. 각 `Send`는 독립된 페이로드로 노드를 동시에 호출하므로, 부모 그래프는 모든 fan-out 결과가 reducer에 도착할 때까지 한 번에 대기합니다.

단일 라우팅 vs 다중 fan-out

- `Command(goto=agent)` — 하나의 에이전트로 라우팅 (분류 결과가 단일일 때)
- `[Send(agent, payload), ...]` — 다중 에이전트 fan-out (분류 결과가 복수일 때)

새로운 소스 추가하기

1. 소스별 도구를 정의합니다
2. 전문 에이전트를 생성합니다
3. `QueryClassification.sources`에 새 소스를 추가합니다
4. 그래프에 에이전트 노드를 추가합니다
5. Reducer에 연결합니다

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def search_github_code(query: str) -> str:
    """GitHub 저장소를 검색합니다."""
    return f"'{query}'에 대한 GitHub 결과"

@tool
def search_notion_pages(query: str) -> str:
    """Notion 워크스페이스를 검색합니다."""
    return f"'{query}'에 대한 Notion 결과"
```

```
@tool
def search_slack_messages(query: str) -> str:
    """Slack 메시지를 검색합니다."""
    return f"'{query}'에 대한 Slack 결과"
```

```
github_agent = create_agent(
    model="gpt-5.4", tools=[search_github_code],
    system_prompt="GitHub에서 코드와 PR을 검색합니다.",
    name="github_agent",
)
notion_agent = create_agent(
    model="gpt-5.4", tools=[search_notion_pages],
    system_prompt="Notion에서 문서를 검색합니다.",
    name="notion_agent",
)
```

```
slack_agent = create_agent(
    model="gpt-5.4", tools=[search_slack_messages],
    system_prompt="Slack에서 토론을 검색합니다.",
    name="slack_agent",
)
```

```
from langgraph.types import Command, Send

def dispatch_to_agents(state):
    """하위 쿼리를 에이전트들에게 병렬로 전달합니다 (Send fan-out)."""
    cls = state["classification"]
    sq_dict = {sq.source: sq.query for sq in cls.sub_queries}
    return [
        Send(
            src,
            {
                "messages": [{"role": "user", "content": sq_dict.get(src, "")}],
                "source": src,
            },
        )
        for src in cls.sources
    ]
```

3.11 결과 합성

Reducer는 모든 에이전트의 결과를 수집하고, LLM으로 통합된 응답을 합성합니다. 합성 시 각 정보의 출처(source)를 인용하여 사용자가 어디서 온 정보인지 파악할 수 있게 합니다.

합성 프롬프트에서는 소스를 명시하도록 지시합니다. 예를 들어, "배포 스크립트는 GitHub의 `payment-service` 레포에 있고(GitHub), 배포 절차는 Notion의 'Payment Service Ops' 문서를 참고하세요(Notion)"와 같이 응답합니다.

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(RouterState)
graph.add_node("router", route_query)
graph.add_node("github", github_agent)
graph.add_node("notion", notion_agent)
graph.add_node("slack", slack_agent)
graph.add_node("reducer", reduce_results)
```

```
<langgraph.graph.state.StateGraph at 0x1bfb57ab230>
```

```
graph.add_edge(START, "router")
graph.add_conditional_edges("router", dispatch_to_agents)
graph.add_edge("github", "reducer")
graph.add_edge("notion", "reducer")
graph.add_edge("slack", "reducer")
graph.add_edge("reducer", END)

app = graph.compile()
```

요약

Part A — Handoffs

항목	핵심
패턴	단일 에이전트 + `current_step` 기반 동적 구성 (권장) / 서브그래프 멀티 에이전트 (복잡 케이스)
전이	핸드오프 도구가 `Command(update={"current_step": ..., "messages": [ToolMessage(tool_call_id=...)]})` 반환
도구 시그니처	`ToolRuntime[None, SupportState]` 로 `runtime.tool_call_id` echo
동적 구성	`\@wrap_model_call` 미들웨어 + `request.override(system_prompt=, tools=)`
서브그래프 규칙	핸드오프 경계에 `AIMessage` + `ToolMessage` 정확히 2개만 전달

Part B — Router

항목	핵심
분류	`with_structured_output(QueryClassification)`
fan-out	`from langgraph.types import Command, Send` → `[Send(agent, payload) for ...]`
단일 라우팅	`Command(goto=agent)`
합성	Reducer 노드에서 LLM 통합 응답, 출처 인용
Router vs Supervisor	분류 전용 단계 vs 대화 중심 오케스트레이션

04 컨텍스트 엔지니어링 & 메모리 심화

– Static/Dynamic Context, InMemoryStore, Skills 패턴

LangGraph의 컨텍스트 시스템과 장기 메모리(Store)를 심층 학습합니다. 정적/동적 런타임 컨텍스트부터 시맨틱 검색 기반 장기 메모리, 그리고 Progressive Disclosure(Skills) 패턴까지 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → 컨텍스트 엔지니어링의 2차원(Mutability x Lifetime) 매트릭스를 이해한다
- → `context_schema` + `@dataclass` 로 정적 런타임 컨텍스트를 구현한다
- → `state_schema` 와 `AgentState` 커스텀으로 동적 런타임 컨텍스트를 관리한다
- → `InMemoryStore` 의 namespace, put, get, search API를 활용한다
- → 시맨틱 검색 기반 장기 메모리를 구축한다
- → 메모리 3유형(Semantic, Episodic, Procedural)을 구분하여 설계한다
- → Skills 패턴으로 Progressive Disclosure를 구현한다
- → Hot path vs Background 메모리 쓰기 전략을 비교한다

4.1 환경 설정

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

model = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

```
환경 준비 완료.
```

4.2 컨텍스트 엔지니어링 개요

컨텍스트 엔지니어링은 "올바른 정보를, 올바른 형식으로, 올바른 시점에" AI에 제공하는 시스템 설계입니다. 단순한 프롬프트 엔지니어링을 넘어, 컨텍스트를 런타임에 프로그래밍 방식으로 조립하는 아키텍처적 접근입니다.

에이전트가 실패하는 주된 원인은 두 가지입니다:

1. LLM 능력 부족
2. 컨텍스트 부족 또는 부적절한 컨텍스트 (더 빈번한 원인)

따라서 컨텍스트 엔지니어링은 AI 엔지니어의 핵심 역할이며, 에이전트 신뢰성의 근본적인 해결책입니다.

2차원 매트릭스: Mutability x Lifetime

Static (불변)	User ID, DB 연결, 도구 정의	설정 파일 등
Dynamic (가변)	대화 히스토리, 중간 결과	사용자 선호도, 학습된 메모리

3가지 컨텍스트 타입

타입	Mutability	Lifetime	예시	LangGraph 구현
Static Runtime	Static	Single run	User ID, DB conn	`context_schema`
Dynamic Runtime (State)	Dynamic	Single run	Messages, 중간결과	`state_schema`
Dynamic Cross-conv (Store)	Dynamic	Cross-conversation	선호도, 메모리	`InMemoryStore`

제어 가능한 3가지 컨텍스트 카테고리

카테고리	제어 대상	특성
Model Context	Instructions, 메시지 히스토리, 도구, 응답 형식	Transient (일시적)
Tool Context	도구 접근, 상태 읽기/쓰기, 런타임 컨텍스트	Persistent (영구적)
Life-cycle Context	단계 간 변환, 요약, 가드레일	Persistent (영구적)

LangChain은 미들웨어(middleware) 메커니즘으로 컨텍스트 엔지니어링을 구현합니다. `@dynamic_prompt`, `@wrap_model_call` 등의 미들웨어로 컨텍스트를 업데이트하거나 라이프사이클 단계 간 제어를 할 수 있습니다.

4.3 정적 런타임 컨텍스트 – `context_schema` + `\@dataclass`

에이전트 실행 중 변하지 않는 정보를 `context_schema` 로 주입합니다. `@dataclass` 로 스키마를 정의하고, 도구에서 `ToolRuntime[Context]` 로 접근합니다.

```python
from dataclasses import dataclass
from langchain.tools import tool, ToolRuntime
from langchain.agents import create_agent

@dataclass
class UserContext:
    user_id: str
    role: str
    department: str
```

```python
@tool
def get_permissions(runtime: ToolRuntime[UserContext]) -> str:
    """현재 사용자의 역할에 따른 권한을 조회합니다."""
    ctx = runtime.context
    perms = {"admin": "read,write,delete", "editor": "read,write"}
    return f"사용자 {ctx.user_id} ({ctx.department}): {perms.get(ctx.role, 'read')}"
```

핵심 포인트

- `context_schema` 에 `@dataclass` 를 전달하면 타입 안전한 컨텍스트를 사용할 수 있습니다
- 도구 함수에서 `runtime: ToolRuntime[Context]` 타입힌트로 자동 주입됩니다
- 실행 중에는 읽기 전용이며 변경되지 않습니다
- 적합한 데이터: User ID, DB 연결, API 키, 세션 메타데이터

4.4 동적 런타임 컨텍스트 – `state_schema` , `AgentState` 커스텀

에이전트가 메시지를 처리하고 도구를 호출하면서 변화하는 상태입니다. `AgentState` 를 상속하여 커스텀 필드를 추가합니다.

```python
from langchain.agents import AgentState

class RAGState(AgentState):
    """동적 검색 컨텍스트를 포함한 상태."""
    retrieved_docs: list[str]
    query_count: int

print(f"상태 키: {list(RAGState.__annotations__.keys())}")
```

```
상태 키: ['messages', 'jump_to', 'structured_response', 'retrieved_docs', 'query_count']
```

Static vs Dynamic 비교

구분	Static Runtime (`context_schema`)	Dynamic Runtime (`state_schema`)
변경 여부	불변 (읽기 전용)	가변 (노드가 업데이트)
전달 방식	`context=` 파라미터	invoke 입력 dict
접근 방법	`runtime.context.field`	`state["field"]`
적합한 데이터	인증 정보, 설정	대화 히스토리, 중간 결과

4.4b 런타임 메타데이터 – `runtime.execution_info` / `runtime.server_info`

`ToolRuntime` 은 `context` / `store` 외에 실행 추적용 두 채널을 노출합니다.

채널	설명	주요 필드
`runtime.execution_info`	현재 run 의 identity 와 retry 정보	`thread_id` , `run_id` , `attempt`
`runtime.server_info`	LangGraph Server 위에서 실행 중일 때만 채워지는 메타데이터 (로컬은 `None`)	`assistant_id` , `graph_id` , `user.identity`

> ➡️ TIP
>
> 두 채널은 `deepagents ⩾ 0.5.0` 또는 `langgraph ⩾ 1.1.5` 가 필요합니다. 도구·미들웨어 양쪽 구현에 동일하게 적용됩니다.

```python
# execution_info / server_info 노출 패턴
@tool
def whoami(runtime: ToolRuntime[UserContext]) -> str:
    """현재 run 의 identity 와 서버 메타데이터를 한 줄로 요약."""
    info = runtime.execution_info
    parts = [
        f"thread={info.thread_id}",
        f"run={info.run_id}",
        f"attempt={info.node_attempt}",
    ]
    server = runtime.server_info
    if server is not None:
        parts.append(f"assistant={server.assistant_id}")
        parts.append(f"graph={server.graph_id}")
        if server.user is not None:
            parts.append(f"user={server.user.identity}")
    else:
        parts.append("env=local")
    return " | ".join(parts)
```

4.5 장기 메모리 – InMemoryStore 기본 API

Cross-conversation 컨텍스트를 위해 `InMemoryStore` 를 사용합니다. 장기 메모리는 세션과 스레드를 초월하여 지속되는 사용자별 또는 앱 수준의 데이터입니다.

저장 구조

메모리는 JSON 문서 형태로 저장되며, 계층적 namespace로 조직화됩니다:

- namespace: 메모리를 분류하는 폴더 역할 (예: `(user_id, "preferences")`)
- key: 각 메모리의 고유 식별자 (예: `"theme"`)
- 네임스페이스에는 보통 사용자 ID나 조직 ID를 포함하여 정보 관리를 용이하게 합니다

기본 API

API	설명
`store.put(namespace, key, value)`	메모리 저장 (upsert)
`store.get(namespace, key)`	특정 키로 메모리 조회
`store.search(namespace)`	네임스페이스 내 전체 검색
`store.search(namespace, filter={...})`	필터 조건으로 검색

프로덕션 환경에서는 `InMemoryStore` 대신 DB 기반 Store (예: PostgreSQL)를 사용해야 합니다.

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
user_id = "user_42"
store.put((user_id, "preferences"), "theme", {"value": "dark"})
store.put((user_id, "preferences"), "language", {"value": "ko"})

item = store.get((user_id, "preferences"), "theme")
print(f"테마: {item.value}")
```

```
테마: {'value': 'dark'}
```

```
items = store.search((user_id, "preferences"))
for item in items:
    print(f"  [{item.key}] = {item.value}")

filtered = store.search(
    (user_id, "preferences"), filter={"value": "dark"}
)
print(f"필터 결과: {len(filtered)}건")
```

```
[theme] = {'value': 'dark'}
  [language] = {'value': 'ko'}
필터 결과: 1건
```

4.6 장기 메모리 – 시맨틱 검색

임베딩 함수를 설정하면 `InMemoryStore`가 시맨틱 검색을 지원합니다. `query` 파라미터로 의미 기반 유사도 검색을 수행합니다.

```
semantic_store = InMemoryStore(
    index={"embed": embeddings, "dims": 1536}
)
ns = ("user_42", "memories")
semantic_store.put(ns, "mem1", {"content": "pytest를 unittest보다 선호"})
semantic_store.put(ns, "mem2", {"content": "모든 함수에 타입 힌트 사용"})
semantic_store.put(ns, "mem3", {"content": "좋아하는 음식은 초밥"})
semantic_store.put(ns, "mem4", {"content": "ML 인프라 팀에서 근무"})
print("임베딩과 함께 메모리 4개 저장 완료.")
```

```
임베딩과 함께 메모리 4개 저장 완료.
```

```python
results = semantic_store.search(
    ("user_42", "memories"), query="testing preferences", limit=2
)
for r in results:
    print(f"  [{r.key}] {r.value['content']}")
```

```
[mem1] pytest를 unittest보다 선호
  [mem2] 모든 함수에 타입 힌트 사용
```

```python
results2 = semantic_store.search(
    ("user_42", "memories"), query="machine learning work", limit=2
)
for r in results2:
    print(f"  [{r.key}] {r.value['content']}")
```

```
[mem4] ML 인프라 팀에서 근무
  [mem2] 모든 함수에 타입 힌트 사용
```

기본 Store vs 시맨틱 Store 비교

기능	`InMemoryStore()`	`InMemoryStore(index={...})`
정확 키 조회	`get(ns, key)`	`get(ns, key)`
필터 검색	`search(ns, filter={...})`	`search(ns, filter={...})`
시맨틱 검색	불가	`search(ns, query="...", limit=N)`
프로덕션	`InMemoryStore` 대신 DB 백엔드 사용	PostgreSQL 기반 Store 권장

4.7 도구에서 Store 읽기/쓰기 – `ToolRuntime.store`

에이전트의 도구 내에서 Store에 접근하여 사용자 정보를 읽고 쓸 수 있습니다. `create_agent(store=...)` 로 Store를 연결하면 `runtime.store` 로 자동 주입됩니다.

읽기 패턴

도구에서 `runtime.store` 를 통해 저장된 사용자 정보를 조회합니다. `ToolRuntime[Context]` 타입힌트로 컨텍스트와 Store 모두에 접근할 수 있습니다.

쓰기 패턴

도구 파라미터로 사용자 입력을 받아 `store.put()` 으로 메모리를 저장합니다. 에이전트가 대화 중 학습한 정보를 영구 저장하는 방식입니다.

핵심 사항

- `runtime.store` : Store 인스턴스에 접근
- `runtime.context` : 정적 런타임 컨텍스트에 접근
- Store와 Context를 결합하면 "누구의(context) 어떤 정보(store)"를 체계적으로 관리할 수 있습니다

```
@tool
def get_user_info(runtime: ToolRuntime[UserContext]) -> str:
    """현재 사용자의 저장된 정보를 조회합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    info = store.get(("users",), user_id)
    return str(info.value) if info else "사용자 정보를 찾을 수 없습니다."
```

```
@tool
def save_preference(key: str, value: str, runtime: ToolRuntime[UserContext]) -> str:
    """사용자 선호도를 저장합니다."""
    store = runtime.store
    user_id = runtime.context.user_id
    store.put((user_id, "preferences"), key, {"value": value})
    return f"선호도 저장됨: {key}={value}"
```

```
# TypedDict 입력을 store 에 저장할 때 dict() 캐스트 — runtime.context.user_id 를 namespace 에 사용
from typing_extensions import TypedDict

class UserInfo(TypedDict):
    name: str
    language: str

@tool
def save_user_info(user_info: UserInfo, runtime: ToolRuntime[UserContext]) -> str:
    """사용자 프로필을 저장합니다. TypedDict → dict 캐스트 필수."""
    assert runtime.store is not None
    runtime.store.put(("users",), runtime.context.user_id, dict(user_info))
    return f"saved user_info for {runtime.context.user_id}"

@tool
def get_user_info_typed(runtime: ToolRuntime[UserContext]) -> str:
    """저장된 프로필을 조회합니다."""
    assert runtime.store is not None
    item = runtime.store.get(("users",), runtime.context.user_id)
    return str(item.value) if item else "Unknown user"
```

4.8 메모리 3유형: Semantic, Episodic, Procedural

장기 메모리는 인지과학에서 영감을 받은 세 가지 유형으로 분류됩니다. 각 유형에 따라 저장 구조와 활용 방식이 다릅니다.

유형	설명	예시	구조
Semantic	엔티티에 대한 사실적 지식	사용자 선호도, 프로필 정보	Profile 또는 Collection
Episodic	과거 경험과 이벤트 기억	Few-shot 예시, 과거 액션 로그	Collection
Procedural	수행 방법에 대한 규칙/지침	시스템 프롬프트 수정, 가이드라인	Profile (규칙 목록)

Semantic Memory – Profile vs Collection

Semantic 메모리는 저장 전략에 따라 두 가지 접근법이 있습니다:

접근법	구조	적합한 경우	예시
Profile	단일 JSON 문서, 지속 업데이트	소수의 잘 알려진 속성	`{"name": "Alice", "language": "Python", "preferred_style": "concise"}`
Collection	다수의 좁은 범위 문서, 높은 리콜	오픈엔드 또는 대규모 지식	`[{"topic": "testing", "content": "Prefers pytest"}, ...]`

Episodic Memory

과거에 유사한 상황에서 어떻게 행동했는지를 기록합니다. Few-shot 예시로 활용되어 에이전트가 과거 경험에서 학습할 수 있게 합니다.

Procedural Memory

에이전트의 행동 규칙을 저장합니다. 시스템 프롬프트를 동적으로 수정하는 효과를 가져, 에이전트가 사용자별 맞춤 지침을 따르도록 합니다.

```
mem_store = InMemoryStore(index={"embed": embeddings, "dims": 1536})
uid = "user_42"

# Semantic -- Profile (single JSON)
mem_store.put((uid, "profile"), "main", {
    "name": "Alice", "language": "Python",
    "preferred_style": "concise",
})
# Semantic -- Collection (multiple docs)
mem_store.put((uid, "facts"), "f1", {"content": "pytest 선호"})
```

```
# Episodic -- past experiences (few-shot)
mem_store.put((uid, "episodes"), "ep1", {
    "content": "SQL 최적화 -> EXPLAIN ANALYZE 사용",
})

# Procedural -- rules/guidelines
mem_store.put((uid, "procedures"), "rules", {
    "content": "항상 에러 처리를 포함. logging 사용.",
})
print("3가지 메모리 유형 모두 저장 완료.")
```

```
3가지 메모리 유형 모두 저장 완료.
```

```
# Episodic search: find similar past experiences
episodes = mem_store.search(
    (uid, "episodes"), query="database query help", limit=1
)
for ep in episodes:
    print(f"관련 에피소드: {ep.value['content']}")
```

```
관련 에피소드: SQL 최적화 -> EXPLAIN ANALYZE 사용
```

4.9 Progressive Disclosure – Skills 패턴

모든 컨텍스트를 프롬프트에 넣으면 토큰 비용이 늘고 정확도가 떨어집니다. Skills 패턴은 필요할 때만 관련 정보를 로드하는 Progressive Disclosure 방식입니다.

Skill의 구조

Skill은 `{name, description, content}` 로 구성된 지식 단위입니다:

- name: 스킬 식별자 (예: `"customers_schema"`)
- description: 짧은 설명 (시스템 프롬프트에 포함됨)
- content: 상세 내용 (`load_skill` 도구로 온디맨드 로드)

크기별 전략

크기	전략	예시
\<1K tokens	시스템 프롬프트에 직접 포함	테이블 이름, 고수준 관계
1–10K tokens	`load_skill` 도구로 온디맨드 로드	테이블 스키마, 쿼리 패턴, 베스트 프랙티스
\>10K tokens	페이지네이션으로 온디맨드 로드	대규모 참조 데이터, 과거 쿼리 로그

동작 흐름

1. 미들웨어가 모든 스킬의 이름과 설명을 시스템 프롬프트에 주입
2. 에이전트가 질문을 분석하고 필요한 스킬을 판단
3. `load_skill` 도구를 호출하여 상세 내용을 로드
4. 로드된 내용을 바탕으로 작업 수행

장점

장점	설명
토큰 효율성	현재 쿼리에 필요한 정보만 로드
확장성	수백 개의 테이블이 있는 DB도 지원
정확도	필요한 시점에 상세 스키마를 제공
비용 절감	요청당 입력 토큰 감소

```
skills = [
    {"name": "db_overview",
     "description": "모든 테이블의 고수준 개요",
     "content": "테이블: customers, orders, products"},
    {"name": "customers_schema",
     "description": "customers 테이블의 전체 스키마",
     "content": "CREATE TABLE customers (id INT PK, name VARCHAR)"},
]
SKILL_MAP = {s["name"]: s for s in skills}
print(f"스킬 {len(skills)}개 정의됨.")
```

```
스킬 2개 정의됨.
```

```python
from langchain_core.tools import tool

@tool
def load_skill(skill_name: str) -> str:
    """데이터베이스 스킬에 대한 상세 정보를 로드합니다."""
    skill = SKILL_MAP.get(skill_name)
    if skill is None:
        return f"찾을 수 없음. 사용 가능: {', '.join(SKILL_MAP.keys())}"
    return f"## {skill['name']}\n\n{skill['content']}"
```

4.10 Hot Path vs Background 메모리 쓰기

메모리를 언제 쓰느냐에 따라 사용자 응답 지연에 영향을 줍니다.

방식	타이밍	즉시 사용 가능?	지연 영향
Hot path	대화 루프 내 실시간	즉시 (다음 턴에 사용 가능)	응답 지연 증가
Background	별도 비동기 태스크	지연됨 (Eventual Consistency)	지연 영향 없음

Hot Path 쓰기

에이전트 루프 내 인라인으로 메모리를 저장합니다. 바로 다음 턴에서 해당 메모리를 써야 할 때 적합합니다. 예: 사용자가 방금 알려준 선호도를 즉시 반영해야 하는 경우.

Background 쓰기

별도의 프로세스나 비동기 태스크로 메모리를 저장합니다. Eventual Consistency가 허용되는 경우에 사용하며, 응답 지연에 영향을 주지 않습니다. 예: 대화 패턴 분석, 장기 학습 데이터 축적.

선택 기준

- 즉시 리콜이 필요한가? -> Hot path
- 지연 감소가 우선인가? -> Background
- 대부분의 경우 Background 쓰기를 선호합니다

```python
from langgraph.store.base import BaseStore

# Hot path: write inline (adds latency)
def reflect_node(state, store: BaseStore):
    """메모리를 인라인으로 추출하고 저장합니다."""
    last_msg = state["messages"][-1].content
    store.put(("user", "reflections"), "latest", {"content": last_msg})
    return state

print("Hot path: 즉시 저장, 다음 턴에 사용 가능.")
```

```
Hot path: 즉시 저장, 다음 턴에 사용 가능.
```

```
import asyncio

# Background: write in separate async task
async def background_memory_writer(state, store: BaseStore):
    """백그라운드에서 메모리를 저장합니다 (지연 없음)."""
    last_msg = state["messages"][-1].content
    await store.aput(
        ("user", "reflections"), "latest", {"content": last_msg}
    )
print("Background: 최종 일관성, 지연 없음.")
```

```
Background: 최종 일관성, 지연 없음.
```

4.11 프로덕션 Store — PostgreSQL

`InMemoryStore` 는 프로세스가 종료되면 데이터가 사라집니다. 프로덕션에서는 `langgraph-checkpoint-postgres` 의 `PostgresStore` 로 교체합니다.

Store	설치	용도
`InMemoryStore`	`langgraph` 에 기본 포함	개발·테스트·일회성 데모
`PostgresStore`	`pip install langgraph-checkpoint-postgres`	프로덕션 — 프로세스 간 영속, 동시 에이전트, 벡터 유사도 검색, 운영 도구 지원

```
from langgraph.store.postgres import PostgresStore
from langchain.agents import create_agent

DB_URI = "postgresql://user:pass@localhost:5432/agentdb"

with PostgresStore.from_conn_string(
    DB_URI,
    index={"embed": embeddings, "dims": 1536},
) as store:
    store.setup()   # 최초 1회 — 테이블/인덱스 생성

    agent = create_agent(
        model="claude-sonnet-4-6",
        tools=[save_user_info, get_user_info_typed],
        context_schema=UserContext,
        store=store,
    )
```

> ➡️ TIP
>
> Store 는 반드시 `create_agent(store=...)` 로 명시 주입해야 `runtime.store` 가 채워집니다. 누락 시 도구의 `runtime.store` 는 `None` 이 되어 LTM 호출이 모두 실패합니다.

요약

컨텍스트 엔지니어링 3요소

요소	구현	API
정적 런타임	`\@dataclass Context` + `context_schema=Context` + `context=Context(...)`	`runtime.context.user_id`
동적 런타임	`state_schema=AgentState` 커스텀	`state["field"]`
장기 메모리	`create_agent(store=store)` 명시 주입	`runtime.store.put / get / search`

런타임 메타데이터

채널	필드	비고
`runtime.execution_info`	`thread_id`, `run_id`, `attempt`	항상 채워짐
`runtime.server_info`	`assistant_id`, `graph_id`, `user.identity`	LangGraph Server 위에서만, 로컬은 `None`

메모리 3유형

유형	용도	Namespace 예시
Semantic	사용자 프로필/사실	`(user_id, "profile")`, `(user_id, "facts")`
Episodic	과거 경험 (few-shot)	`(user_id, "episodes")`
Procedural	규칙/프롬프트 수정	`(user_id, "procedures")`

Store 선택

Store	설치	용도
`InMemoryStore`	포함	개발/테스트
`PostgresStore`	`langgraph-checkpoint-postgres`	프로덕션, 영속, 벡터 유사도

Best Practices

원칙	설명
정적 컨텍스트 최소화	현재 태스크에 필요한 것만 포함
Namespace 구조화	`runtime.context.user_id` 를 namespace 에 포함
TypedDict → dict 캐스트	`runtime.store.put(..., dict(user_info))` 로 안전하게 직렬화
시맨틱 검색 우선	정확 매칭보다 임베딩 기반 검색이 확장성 우수
Background 쓰기 선호	즉시 리콜 불필요시 background 로 지연 감소

Skills 패턴 적용	대규모 컨텍스트는 Progressive Disclosure

05 Agentic RAG

– LangGraph로 직접 구축

Retrieval-Augmented Generation(RAG)을 세 가지 방법으로 구현합니다: LangChain RAG Agent, LangChain RAG Chain, 그리고 LangGraph StateGraph 기반 커스텀 RAG. 문서 관련성 평가, 쿼리 리라이트, 조건부 라우팅 등 심화 패턴을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → RAG 파이프라인(인덱싱 -> 검색 -> 생성)의 전체 구조를 이해한다
- → `RecursiveCharacterTextSplitter`로 문서를 청킹한다
- → `InMemoryVectorStore`로 벡터 스토어를 구축한다
- → LangChain `create_agent` + `@tool`로 RAG Agent를 구현한다
- → `@dynamic_prompt` 미들웨어로 RAG Chain(단일 LLM 호출)을 구현한다
- → LangGraph `StateGraph`로 커스텀 RAG 에이전트를 구축한다
- → `GradeDocuments` 구조화 출력으로 문서 관련성을 평가한다
- → 쿼리 리라이트와 조건부 라우팅을 구현한다

5.1 환경 설정

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI, OpenAIEmbeddings

llm = ChatOpenAI(model="gpt-5.4")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("환경 준비 완료.")
```

```
환경 준비 완료.
```

5.2 RAG 개요

RAG(Retrieval-Augmented Generation)는 외부 지식을 검색하여 LLM 응답의 정확도를 높이는 패턴입니다. LLM 의 두 가지 한계 — 유한한 컨텍스트(Finite context)와 정적 학습 지식(Static knowledge) — 을 검색으로 보완합니다.

5가지 빌딩 블록

빌딩 블록	역할
Document loaders	Google Drive, Slack, Notion 등 외부 소스에서 표준화된 `Document` 객체로 데이터 수집
Text splitters	큰 문서를 컨텍스트 윈도우에 맞는 청크로 분할
Embedding models	텍스트를 벡터로 변환. 의미적으로 유사한 콘텐츠가 가까운 위치에 군집화
Vector stores	임베딩을 저장하고 유사도 기반으로 검색하는 전문 데이터베이스
Retrievers	비정형 쿼리에 대해 관련 문서를 반환하는 통일된 인터페이스

3가지 RAG 아키텍처

아키텍처	설명	레이턴시	유연성	제어성	적합 사례
2-Step RAG	검색 후 생성. 예측 가능한 순서	빠름	낮음	높음	FAQ, 문서 봇
Agentic RAG	에이전트가 추론 중 검색 여부와 방법을 결정	가변	높음	중간	리서치 어시스턴트
Hybrid RAG	두 방식 결합 + 쿼리 강화·검색 검증·답변 품질 체크 등 반복 단계 추가	가변	높음	높음	프로덕션 어시스턴트

이 노트북에서는 Agentic RAG(LangChain `create_agent` + retriever tool)와 Hybrid RAG(LangGraph `StateGraph` Rewrite → Retrieve → Agent 3-노드 패턴)를 모두 구현합니다.

5.3 문서 로딩 & 청킹

문서 로더 (Document Loaders)

문서 로더는 다양한 소스에서 원시 콘텐츠를 읽어 `page_content` 와 `metadata` 필드를 가진 `Document` 객체로 반환합니다.

로더	소스	패키지
`PyPDFLoader`	PDF 파일	`pypdf`

`TextLoader`	텍스트 파일	내장
`CSVLoader`	CSV 파일	내장
`WebBaseLoader`	웹 페이지	`beautifulsoup4`
`DirectoryLoader`	디렉토리 내 파일들	내장

텍스트 분할 (Text Splitting)

`RecursiveCharacterTextSplitter`는 `\n\n` -> `\n` -> ` ` -> `""` 순으로 재귀적으로 분할하여 의미적 연관성을 유지합니다. 가장 범용적인 분할기로 권장됩니다.

파라미터	설명	권장값
`chunk_size`	청크 최대 문자 수	500–2000 (작으면 정밀 검색, 크면 맥락 보존)
`chunk_overlap`	인접 청크 공유 문자 수	chunk_size의 10–20% (경계 정보 손실 방지)

기타 분할기

분할기	적합한 경우
`MarkdownHeaderTextSplitter`	마크다운 문서
`HTMLHeaderTextSplitter`	HTML 문서
`TokenTextSplitter`	토큰 예산 기반 분할
`CodeTextSplitter`	소스 코드 (언어 인식)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

raw_docs = [
    Document(page_content="LangGraph는 LLM으로 상태 기반 멀티 액터 "
        "애플리케이션을 구축하기 위한 프레임워크입니다.",
        metadata={"source": "langgraph-docs"}),
    Document(page_content="에이전트는 도구를 사용하여 외부 시스템과 "
        "상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다.",
        metadata={"source": "agent-guide"}),
]
print(f"문서 {len(raw_docs)}개 로드됨.")
```

```
문서 2개 로드됨.
```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
)
splits = text_splitter.split_documents(raw_docs)

for i, doc in enumerate(splits):
    print(f"청크 {i}: {doc.page_content[:60]}...")
print(f"총 청크 수: {len(splits)}")
```

```
청크 0: LangGraph는 LLM으로 상태 기반 멀티 액터 애플리케이션을 구축하기 위한 프레임워크입니다....
청크 1: 에이전트는 도구를 사용하여 외부 시스템과 상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다....
총 청크 수: 2
```

5.4 벡터 스토어 구축

벡터 스토어는 임베딩을 인덱싱하고 유사도 검색을 수행하는 전문 데이터베이스입니다. `InMemoryVectorStore`는 개발/테스트용으로 적합합니다.

주요 벡터 스토어 비교

벡터 스토어	유형	적합한 경우
`InMemoryVectorStore`	인프로세스	개발, 소규모 데이터셋
`Chroma`	임베디드/클라이언트-서버	프로토타이핑, 중규모 데이터셋
`FAISS`	인프로세스	고성능 로컬 검색
`Pinecone`	매니지드 클라우드	프로덕션, 확장성
`PGVector`	PostgreSQL 확장	기존 PostgreSQL 인프라 활용

검색 유형

검색 타입	설명
`"similarity"`	표준 최근접 이웃 검색
`"mmr"`	Maximal Marginal Relevance – 관련성과 다양성의 균형 (중복 감소)
`"similarity_score_threshold"`	최소 유사도 점수 이상인 문서만 반환

```python
from langchain_core.vectorstores import InMemoryVectorStore

vector_store = InMemoryVectorStore.from_documents(
    documents=splits, embedding=embeddings,
)
test_results = vector_store.similarity_search("LangGraph", k=2)
for doc in test_results:
    print(f"  [{doc.metadata['source']}] {doc.page_content[:80]}")
print(f"벡터 스토어 준비 완료. 문서 {len(splits)}개.")
```

```
[langgraph-docs] LangGraph는 LLM으로 상태 기반 멀티 액터 애플리케이션을 구축하기 위한 프레임워크입니다.
  [agent-guide] 에이전트는 도구를 사용하여 외부 시스템과 상호작용합니다. ReAct 패턴은 추론과 행동을 번갈아 수행합니다.
벡터 스토어 준비 완료. 문서 2개.
```

5.5 검색 도구 정의

`response_format="content_and_artifact"` 를 사용하면 도구 출력을 두 부분으로 분리합니다:

- Content: 모델에 전달되는 문자열 표현 (추론에 사용)
- Artifact: 원본 Document 객체 (프로그래밍 방식으로 접근 가능하지만 모델에 전송되지 않음)

이 분리를 통해 모델에는 읽기 쉬운 텍스트를, 후속 처리에는 메타데이터가 포함된 원본 객체를 사용할 수 있습니다.

```python
from langchain_core.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """지식 베이스에서 관련 문서를 검색합니다."""
    docs = vector_store.similarity_search(query, k=4)
    serialized = "\n\n".join(
        f"출처: {d.metadata.get('source', '?')}\n{d.page_content}"
        for d in docs
    )
    return serialized, docs
```

5.6 LangChain RAG Agent — `create_agent` + `\@tool` (Agentic RAG)

가장 단순한 Agentic RAG 구현입니다. retriever 를 `@tool` 로 노출하고, 에이전트가 언제 검색할지 / 어떤 쿼리로 검색할지 를 자율적으로 결정합니다.

- `create_agent` 가 도구 호출 루프를 관리
- 검색 호출 자체가 도구이므로, 다른 도구(웹 검색, 계산기 등)와 자연스럽게 조합 가능
- 검색이 필요 없는 질문은 LLM 이 곧바로 답변 — 2-Step RAG 와의 가장 큰 차이

5.7 LangChain RAG Chain – `\@dynamic_prompt` 미들웨어

단일 LLM 호출로 RAG를 구현합니다. `@dynamic_prompt` 가 LLM 호출 전에 문서를 검색하고 시스템 프롬프트에 자동으로 주입합니다. 미들웨어 방식이므로 에이전트 루프 없이 단일 패스로 동작합니다.

특성	RAG Agent	RAG Chain
LLM 호출 수	다중 (에이전트 결정)	단일
검색 횟수	1회 이상 (에이전트 제어)	정확히 1회 (미들웨어 제어)
쿼리 재구성	자동	미지원
지연	높음 (여러 왕복)	낮음 (단일 패스)
비용	높음 (더 많은 토큰)	낮음 (더 적은 토큰)
투명성	에이전트 추론이 메시지에 노출	컨텍스트 주입이 암묵적

고급 활용: `@dynamic_prompt` 로 기본 컨텍스트를 주입하면서 retriever 도구를 함께 제공하여 두 접근법을 결합할 수도 있습니다.

```python
from langchain.agents.middleware import dynamic_prompt

@dynamic_prompt
def rag_prompt(request):
    """문서를 검색하여 시스템 프롬프트에 주입합니다."""
    user_msg = request.state["messages"][-1].content
    docs = vector_store.similarity_search(user_msg, k=4)
    ctx = "\n\n".join(d.page_content for d in docs)
    return f"컨텍스트를 기반으로 답변하세요:\n\n{ctx}"
```

5.8 LangGraph 커스텀 RAG — StateGraph 구축

`docs/langchain/23-custom-workflow.md` 의 Rewrite → Retrieve → Agent 3-노드 패턴을 일반화하여, 검색 품질 검증과 질문 리라이트를 추가한 Hybrid RAG 를 구현합니다.

노드 구성

노드 유형	노드 이름	역할
Model node (Rewrite)	`rewrite_question`	검색에 더 적합한 형태로 질문을 재작성 (구조화 출력 사용 가능)
Deterministic node (Retrieve)	`retrieve` (`ToolNode`)	벡터 유사도 검색을 수행
Agent node (Generate)	`generate_query_or_respond` / `generate_answer`	검색 결과를 추론하고 필요 시 추가 도구 호출

이 패턴은 노드 단위로 검증/재시도/분기를 쉽게 추가할 수 있어 Hybrid RAG 구현에 적합합니다.

```python
from langgraph.graph import MessagesState

class AgentState(MessagesState):
    """커스텀 RAG 에이전트 상태."""
    relevance: str  # "relevant" or "not_relevant"

print(f"AgentState 키: {list(AgentState.__annotations__)}")
```

```
AgentState 키: ['messages', 'relevance']
```

5.9 `generate_query_or_respond` 노드

진입 노드입니다. LLM이 retrieve 도구를 호출할지, 직접 응답할지 결정합니다.

5.10 `grade_documents` 노드 – 구조화 출력으로 관련성 평가

`GradeDocuments` 스키마로 LLM이 문서 관련성을 평가합니다. `with_structured_output` 으로 구조화된 응답을 받습니다.

```
from pydantic import BaseModel, Field
from typing import Literal

class GradeDocuments(BaseModel):
    """검색된 문서의 이진 관련성 점수."""
    relevance: Literal["relevant", "not_relevant"] = Field(
        description="문서가 관련이 있는지 여부."
    )
    reasoning: str = Field(description="간략한 설명.")

grader = llm.with_structured_output(GradeDocuments)
```

```
def grade_documents(state: AgentState):
    """
    검색된 문서의 관련성을 평가합니다.
    """

    msgs = state["messages"]

    user_q = next(
        (m.content for m in msgs if m.type == "human"),
        ""
    )

    tool_content = msgs[-1].content

    grade = grader.invoke(
        f"질문: {user_q}\n문서:\n{tool_content}\n"
        f"이 문서들이 관련이 있습니까?"
    )

    return {
        "relevance": grade.relevance,
        "messages": msgs
    }
```

5.11 `rewrite_question` 노드

검색된 문서가 관련 없을 때, 원래 질문을 더 구체적으로 리라이트하여 검색 품질을 높입니다.

5.12 `generate_answer` 노드

관련 문서가 확인되면, 검색 결과와 원본 질문을 결합하여 최종 답변을 생성합니다.

5.13 그래프 조립 & 실행

모든 노드를 `StateGraph`에 등록하고, 조건부 엣지(`tools_condition` , `relevance_router`)로 연결합니다.

```
from langgraph.graph import StateGraph, START, END
from langgraph.prebuilt import ToolNode, tools_condition

def relevance_router(state: AgentState):
    if state.get("relevance") == "relevant":
        return "generate_answer"
    return "rewrite_question"
```

```
graph = StateGraph(AgentState)
graph.add_node("gen_query", generate_query_or_respond)
```

```
<langgraph.graph.state.StateGraph at 0x29f10457080>
```

```
graph.add_node("retrieve", ToolNode([retrieve]))
graph.add_node("grade_documents", grade_documents)
graph.add_node("rewrite_question", rewrite_question)
graph.add_node("generate_answer", generate_answer)

graph.add_edge(START, "gen_query")
graph.add_conditional_edges(
    "gen_query", tools_condition,
    {"tools": "retrieve", "__end__": END},
)
```

```
<langgraph.graph.state.StateGraph at 0x29f10457080>
```

```
graph.add_edge("retrieve", "grade_documents")
graph.add_conditional_edges(
    "grade_documents", relevance_router,
    {"generate_answer": "generate_answer",
     "rewrite_question": "rewrite_question"},
)
graph.add_edge("rewrite_question", "gen_query")
graph.add_edge("generate_answer", END)

app = graph.compile()
print("그래프 컴파일 성공.")
```

```
그래프 컴파일 성공.
```

요약

세 가지 RAG 접근법 비교

특성	RAG Agent	RAG Chain	LangGraph 커스텀
LLM 호출 수	다중	단일	다중
검색 횟수	에이전트 결정	정확히 1회	커스텀
쿼리 재구성	자동	미지원	명시적 노드
관련성 평가	암묵적	없음	`GradeDocuments`
제어 수준	낮음	낮음	높음
구현 복잡도	낮음	최저	높음

핵심 LangGraph 패턴

패턴	구현
조건부 라우팅	`add_conditional_edges` + `tools_condition`
구조화 출력	`llm.with_structured_output(GradeDocuments)`
도구 노드	`ToolNode([retrieve])`
루프 제어	`rewrite_question` -\> `gen_query` 순환

06 SQL 에이전트 심화

– LangChain & LangGraph

자연어를 SQL 쿼리로 변환하는 에이전트를 두 가지 방법으로 구축합니다: LangChain `create_agent` + `SQLDatabaseToolkit` (간단 버전)과 LangGraph `StateGraph` (커스텀 버전). Human-in-the-Loop, `interrupt()` , `Command(resume=...)` 패턴을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → SQL 에이전트의 8단계 워크플로우를 이해한다
- → `SQLDatabase` 와 `SQLDatabaseToolkit` 의 4개 도구를 활용한다
- → LangChain `create_agent` 로 ReAct 기반 SQL Agent를 구현한다
- → `HumanInTheLoopMiddleware` 로 쿼리 실행 전 승인을 추가한다
- → LangGraph `StateGraph` 로 커스텀 SQL Agent를 구축한다
- → `bind_tools` 와 `tool_choice` 로 강제 도구 호출을 설정한다
- → `interrupt()` 와 `Command(resume=...)` 로 쿼리 리뷰를 구현한다

6.1 환경 설정 (SQLite + Chinook DB)

```
# %pip install langchain langchain-openai langchain-community langgraph sqlalchemy python-dotenv

from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
from langchain_community.utilities import SQLDatabase

llm = ChatOpenAI(model="gpt-5.4")
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print(f"Dialect: {db.dialect}")
```

```
Dialect: sqlite
```

6.2 SQL 에이전트 개요

SQL 에이전트는 자연어 질문을 SQL 쿼리로 변환하는 8단계 프로세스를 따릅니다:

```
1. 질문 수신 -> 2. 테이블 목록 -> 3. 관련 테이블 스키마
-> 4. SQL 쿼리 생성 -> 5. 쿼리 검증 -> 6. (선택) 사람 리뷰
-> 7. 쿼리 실행 -> 8. 결과 해석
```

왜 에이전트가 필요한가?

단순 text-to-SQL과 달리 에이전트 방식은 스키마 탐색 → 쿼리 생성 → 검증 → 실행의 반복 루프를 수행합니다. 잘못된 쿼리가 나오면 에이전트가 오류를 분석하고 쿼리를 재작성할 수 있어 정확도가 크게 높아집니다. 에이전트는 필요한 테이블의 스키마만 선택적으로 로드하므로 컨텍스트 윈도우를 효율적으로 사용합니다.

에이전트 실행 트레이스 예시

```
User: "지난달 매출 상위 5개 제품은?"

Agent -> sql_db_list_tables()
      <- "customers, orders, order_items, products, categories"

Agent -> sql_db_schema("orders, order_items, products")
      <- CREATE TABLE orders (id INT, order_date DATE, ...)
         CREATE TABLE order_items (order_id INT, product_id INT, quantity INT, price DECIMAL, ...)

Agent -> sql_db_query_checker("SELECT p.name, SUM(oi.quantity * oi.price) ...")
      <- "The query looks correct."

Agent -> sql_db_query(validated_query)
      <- [("Widget Pro", 45230.00), ("Gadget X", 38100.00), ...]
```

안전 수칙

우려사항	대응
SQL Injection	파라미터화된 쿼리 사용, Toolkit이 자동 처리
DML 실행	시스템 프롬프트에서 INSERT/UPDATE/DELETE 금지, DB 레벨 읽기 전용 권한 설정
비용 높은 쿼리	LIMIT 강제, Human-in-the-Loop으로 실행 전 승인
민감 데이터	`include_tables` / `exclude_tables` 로 접근 가능 테이블 제한, 컬럼 레벨 권한 설정
데이터 노출	데이터베이스 뷰(view) 또는 제한된 사용자 권한 활용

접근 가능 테이블 제한

프로덕션에서는 에이전트가 접근할 수 있는 테이블을 명시적으로 제한하는 것이 좋습니다:

```python
db = SQLDatabase.from_uri(
    "sqlite:///company.db",
    include_tables=["products", "orders", "order_items"],  # 허용 목록
    # exclude_tables=["users", "credentials"],             # 또는 차단 목록
)
```

6.3 SQLDatabaseToolkit

4개의 도구를 자동 생성합니다:

도구	기능
`sql_db_list_tables`	데이터베이스의 모든 테이블 이름 반환
`sql_db_schema`	CREATE TABLE 문 + 샘플 행 반환
`sql_db_query`	SQL 쿼리를 실행하고 결과 반환
`sql_db_query_checker`	LLM이 쿼리의 오류를 사전 검사

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()

for t in tools:
    print(f"  {t.name}: {t.description[:60]}...")
print(f"총 도구 수: {len(tools)}")
```

```
sql_db_query: Input to this tool is a detailed and correct SQL query, outp...
  sql_db_schema: Input to this tool is a comma-separated list of tables, outp...
  sql_db_list_tables: Input is an empty string, output is a comma-separated list o...
  sql_db_query_checker: Use this tool to double check if your query is correct befor...
총 도구 수: 4
```

6.4 LangChain SQL Agent – `create_agent` + ReAct

`create_agent`는 LangChain의 고수준 API로, 모델과 도구를 받아 ReAct(Reasoning + Acting) 루프를 자동으로 구성합니다. 에이전트는 시스템 프롬프트에 정의된 워크플로우를 따라 도구를 순서대로 호출합니다.

ReAct 루프 동작 원리

1. LLM이 사용자 질문과 대화 이력을 분석하여 다음에 호출할 도구를 결정합니다
2. 도구가 실행되고 결과가 대화 이력에 추가됩니다
3. LLM이 결과를 확인하고, 추가 도구 호출이 필요하면 1단계로 돌아갑니다
4. 최종 답변이 준비되면 텍스트 응답을 반환합니다

시스템 프롬프트의 역할

시스템 프롬프트는 에이전트의 행동 지침을 정의합니다. SQL 에이전트에서는 특히 다음을 명시해야 합니다:

- 도구 호출 순서: `list_tables` → `schema` → `query_checker` → `query` 순서 강제
- 안전 규칙: `LIMIT` 사용, DML 금지, 필요한 컬럼만 조회
- 오류 처리: 쿼리 오류 발생 시 재작성 지시
- SQL 방언: 현재 DB의 dialect(SQLite, PostgreSQL 등) 명시

```
system_prompt = (
    "당신은 SQL 에이전트입니다. 단계:\n"
```

```
    "1. sql_db_list_tables\n2. sql_db_schema\n"
    "3. 쿼리 작성 + sql_db_query_checker\n"
    "4. sql_db_query\n5. 결과를 해석하세요.\n"
    f"규칙: LIMIT 10 사용. DML 금지. Dialect: {db.dialect}"
)
```

```
from langchain.agents import create_agent

sql_agent = create_agent(
    model=llm, tools=tools, system_prompt=system_prompt,
)
print("LangChain SQL 에이전트 생성됨.")
```

```
LangChain SQL 에이전트 생성됨.
```

6.5 실행 테스트

6.6 HITL — `HumanInTheLoopMiddleware`

프로덕션에서는 SQL 쿼리 실행 전 사람 승인이 필요합니다. 에이전트가 만든 쿼리가 비용을 유발하거나, 예상 외 테이블에 접근하거나, 의도와 다른 결과를 반환할 수 있기 때문입니다.

`HumanInTheLoopMiddleware` 는 지정한 도구(`sql_db_query`) 호출을 가로채 그래프를 일시 중단합니다. 사람이 결정을 내리면 `Command(resume={"decisions": [...]})` 로 그래프를 재개합니다 (v2 invocation 필수).

4가지 decision 타입

타입	`Command(resume=...)` 값	동작
approve	`{"decisions": [{"type": "approve"}]}`	원본 인자 그대로 실행
edit	`{"decisions": [{"type": "edit", "edited_action": {"name": "sql_db_query", "args": {"query": "..."}}}]}`	수정된 인자로 실행
reject	`{"decisions": [{"type": "reject", "message": "..."}]}`	실행 거부, 대화에 메시지 추가
respond	`{"decisions": [{"type": "respond", "message": "..."}]}`	도구 호출 건너뛰고, 사람의 응답이 도구 결과를 대체

HITL이 필요한 이유

- 비용 제어: `LIMIT` 없는 풀 스캔 차단
- 데이터 보호: 민감 컬럼 접근 사전 차단
- 정확성 검증: 에이전트가 질문 의도를 잘못 해석한 경우 수정
- 감사 추적: 실행된 모든 쿼리에 대한 승인 기록 — DELETE / UPDATE 등 파괴적 쿼리는 반드시 HITL 필수

v2 invocation 필수

`Command(resume={"decisions": [...]})` 패턴은 `invoke(..., version="v2")` / `stream(..., version="v2")` 호출에서만 동작합니다. v2 결과는 `.interrupts`, `.value` 속성을 가진 `GraphOutput` 객체입니다.

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

# DELETE / UPDATE 가 위험한 환경: sql_db_query 를 전부 가로채거나,
# 특정 도구만 allowed_decisions 로 제한할 수 있습니다.
hitl = HumanInTheLoopMiddleware(
    interrupt_on={
        "sql_db_query": {"allowed_decisions": ["approve", "edit", "reject"]},
    },
    description_prefix="SQL 쿼리 실행 승인 필요",
)
sql_agent_hitl = create_agent(
    model=llm, tools=tools,
    system_prompt=system_prompt, middleware=[hitl],
)
print("HITL이 적용된 SQL 에이전트 생성됨.")
```

```
HITL이 적용된 SQL 에이전트 생성됨.
```

```
from langgraph.types import Command

config = {"configurable": {"thread_id": "sql-review-1"}}

# v2 invocation 필요 (version="v2")
# 1) 첫 실행 — interrupt 발생 시점까지 진행
# result = sql_agent_hitl.invoke(
#     {"messages": [{"role": "user", "content": "고객이 가장 많은 나라는?"}]},
#     config=config, version="v2",
# )
# print(result.interrupts)  # HITLRequest 확인

# Option 1: Approve
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [{"type": "approve"}]}),
#     config=config, version="v2",
# )

# Option 2: Edit
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [
#         {"type": "edit",
#          "edited_action": {
#              "name": "sql_db_query",
#              "args": {"query": "SELECT Country, COUNT(*) FROM Customer GROUP BY Country LIMIT 10"},
#          }}
#     ]}),
#     config=config, version="v2",
# )

# Option 3: Reject (DELETE/UPDATE 차단 시 사용)
# result = sql_agent_hitl.invoke(
#     Command(resume={"decisions": [
#         {"type": "reject", "message": "DELETE 가 의심됩니다. SELECT 로 다시 작성하세요."}
#     ]}),
#     config=config, version="v2",
# )
print("HITL 재개 옵션: approve / edit / reject / respond (v2 invocation 필수)")
```

```
HITL 재개 옵션: approve / edit / reject
```

6.7 LangGraph 커스텀 SQL Agent – StateGraph

LangChain `create_agent`는 빠르게 프로토타입을 만들 수 있지만, 노드 단위의 세밀한 제어가 필요하면 LangGraph `StateGraph`를 사용합니다. 각 단계를 독립적인 노드로 정의하면 다음을 실현할 수 있습니다:

- 조건부 분기: 쿼리 검증 실패 시 재생성 노드로 라우팅
- 강제 도구 호출: `bind_tools(tool_choice="...")`로 특정 노드에서 반드시 특정 도구 호출
- 세밀한 중단점: `interrupt()`로 원하는 노드에서 정확히 실행 중단
- 커스텀 상태: 쿼리 이력, 재시도 횟수 등을 상태에 추가

그래프 구조

```
START -> list_tables -> get_schema -> generate_query
      -> check_query -> execute_query -> END
```

각 노드는 공유 `State` 객체를 받아 메시지를 추가하며, 에이전트가 워크플로우를 진행하는 동안 대화 이력이 누적됩니다. `tools_condition`을 사용하면 `check_query` 결과에 따라 쿼리를 재생성하거나 실행으로 진행하는 조건부 분기를 구현할 수 있습니다.

LangChain `create_agent` 대비 장점

측면	`create_agent`	`StateGraph`
도구 호출 순서	LLM 자율 결정	그래프 엣지로 강제
오류 시 재시도	시스템 프롬프트에 의존	조건부 엣지로 명시적 구현
사람 리뷰	미들웨어 기반	`interrupt()` 기반, 위치 자유
디버깅	블랙박스	노드별 상태 확인 가능

```python
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class SQLState(TypedDict):
    messages: Annotated[list, add_messages]

print(f"SQLState 키: {list(SQLState.__annotations__)}")
```

```
SQLState 키: ['messages']
```

6.8 전용 노드 – `list_tables`, `get_schema`, `generate_query`, `check_query`

각 노드는 SQL 에이전트 워크플로우의 한 단계를 담당합니다.

6.9 `bind_tools` with `tool_choice` – 강제 도구 호출

`tool_choice` 파라미터로 특정 도구를 강제 호출하도록 설정합니다.

6.10 `interrupt()` 로 쿼리 리뷰

LangGraph의 `interrupt()` 함수는 그래프 실행을 일시 중단하고 외부 입력(사람의 리뷰)을 기다립니다. `HumanInTheLoopMiddleware` 와 달리 `interrupt()` 는 노드 내부 코드의 정확한 위치에서 중단할 수 있어 더 유연합니다.

동작 원리

1. 노드 함수 내에서 `interrupt(payload)` 를 호출하면 그래프 실행이 즉시 중단됩니다
2. `payload` 는 클라이언트에게 전달되어 리뷰 UI에 표시됩니다 (예: 생성된 SQL 쿼리)
3. 클라이언트가 `Command(resume=value)` 로 그래프를 재개하면, `interrupt()` 가 `value` 를 반환합니다
4. 노드 함수는 반환된 값에 따라 쿼리를 실행, 수정, 또는 거부합니다

`interrupt()` **vs** `HumanInTheLoopMiddleware`

특성	`interrupt()`	`HumanInTheLoopMiddleware`
적용 범위	노드 내 코드 레벨	도구 호출 레벨
유연성	임의 로직 구현 가능	도구 호출 가로채기만 가능
상태 접근	전체 State 접근 가능	도구 인자만 접근 가능
체크포인터	필수 (상태 저장 필요)	선택적

6.11 `Command(resume=...)` 패턴

`interrupt()` 로 중단된 그래프를 재개하려면 `Command(resume=...)` 를 사용합니다.

```
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

builder = StateGraph(SQLState)
builder.add_node("list_tables", list_tables_node)
builder.add_node("get_schema", get_schema_node)
builder.add_node("generate_query", generate_query_node)
builder.add_node("check_query", check_query_node)
builder.add_node("execute_query", execute_query_node)
```

```
<langgraph.graph.state.StateGraph at 0x1f6f9914b00>
```

```
builder.add_edge(START, "list_tables")
builder.add_edge("list_tables", "get_schema")
builder.add_edge("get_schema", "generate_query")
```

```
builder.add_edge("generate_query", "check_query")
builder.add_edge("check_query", "execute_query")
builder.add_edge("execute_query", END)

checkpointer = InMemorySaver()
sql_graph = builder.compile(checkpointer=checkpointer)
print("LangGraph SQL 에이전트 컴파일됨.")
```

```
LangGraph SQL 에이전트 컴파일됨.
```

요약

두 가지 SQL Agent 비교

특성	LangChain `create_agent`	LangGraph `StateGraph`
구현 복잡도	낮음 (5줄)	높음 (전용 노드)
제어 수준	ReAct 자동	노드 단위 커스텀
HITL	`HumanInTheLoopMiddleware`	`interrupt()` + `Command(resume=...)`
강제 도구 호출	미지원	`bind_tools(tool_choice=...)`
적합한 경우	빠른 프로토타입	프로덕션, 세밀한 제어

HITL 패턴

액션	`Command(resume=...)`
Accept	`{"action": "accept"}`
Edit	`{"action": "edit", "edited_query": "..."}`
Reject	`{"action": "reject", "reason": "..."}`

SQLDatabaseToolkit 4개 도구

도구	단계	용도
`sql_db_list_tables`	2	테이블 목록 확인
`sql_db_schema`	3	DDL + 샘플 데이터 조회
`sql_db_query_checker`	5	쿼리 사전 검증
`sql_db_query`	7	쿼리 실행

07 데이터 분석 에이전트

Deep Agents + 샌드박스

CSV 데이터를 입력받아 탐색적 데이터 분석(EDA)을 수행하고, 시각화 결과를 생성한 뒤 Slack으로 공유하는 자율 에이전트를 구축합니다.

Deep Agents SDK의 `create_deep_agent`, 백엔드 시스템, 빌트인 도구, 체크포인터를 활용합니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. 백엔드 선택 — `LocalShellBackend`(개발)과 `DaytonaSandbox` / `Modal` / `Runloop`(운영) 간 차이를 이해하고 선택할 수 있다
2. 커스텀 도구 정의 — `@tool` 데코레이터로 Slack 연동 등 외부 서비스 도구를 만들 수 있다
3. 에이전트 생성 — `create_deep_agent()`로 모델, 도구, 백엔드, 체크포인터를 조합한 에이전트를 구성할 수 있다
4. 스트리밍 관찰 — 에이전트의 분석 과정을 실시간으로 모니터링할 수 있다
5. 도구 활용 — `ls`, `read_file`, `write_file`, `edit_file`, `delete`, `glob`, `grep`, `execute`와 opt-in `write_todos`의 역할을 이해할 수 있다
6. 체크포인터 — `InMemorySaver`로 대화 상태를 유지하고 이어서 분석을 수행할 수 있다
7. 하네스 해부 — `create_agent()` + 미들웨어 관점으로 `create_deep_agent()`를 이해할 수 있다

7.1 환경 설정

Deep Agents SDK와 Tavily(웹 검색), Slack SDK를 설치합니다. 실행 환경에 따라 `pip` 또는 `uv`를 사용합니다.

```
from dotenv import load_dotenv

load_dotenv()
```

```
True
```

7.2 데이터 분석 에이전트 개요

Deep Agents의 데이터 분석 에이전트는 다음 파이프라인을 실행합니다:

```
CSV 입력 → 계획 수립(선택: TodoListMiddleware) → 파일 읽기(read_file) → 코드 생성 & 실행 → 반복 분석 → 결과 전달 (Slack)
```

실행 흐름 상세

단계	설명	사용 도구
Planning	복잡한 분석이면 `TodoListMiddleware` 를 추가해 `write_todos` 로 계획을 관리	`write_todos`
File Reading	CSV 구조, 컬럼명, 데이터 타입, 행 수를 파악. 이미지 파일도 멀티모달로 읽기 가능	`read_file`
Code Execution	pandas, matplotlib 등 Python 코드를 작성하여 백엔드에서 격리 실행	Backend `execute`
Iterative Analysis	초기 결과를 기반으로 추가 분석 수행, 웹 검색으로 도메인 맥락 확보	Tavily, `edit_file`
Result Delivery	분석 결과를 포맷팅하여 Slack 채널에 전송	`slack_send_message`

자율 에이전트의 핵심 특성

1. 계획 수립(선택): `TodoListMiddleware` 가 있을 때 `write_todos` 로 분석 작업을 세분화합니다.
2. 적응적 실행: 분석 중 오류가 발생하면 코드를 수정(`edit_file`)하고 재실행합니다.
3. 서브에이전트 위임: 복잡한 작업은 전문 서브에이전트를 생성하여 병렬 처리합니다.
4. 컨텍스트 관리: 파일 시스템 도구로 중간 결과를 저장하고 필요할 때 다시 참조합니다.

7.3 백엔드 선택

Deep Agents는 플러그형 백엔드 아키텍처로 파일시스템 및 코드 실행 환경을 제공합니다. 모든 백엔드는 동일한 `BackendProtocol` 을 구현하므로, 코드를 변경하지 않고 백엔드만 교체할 수 있습니다.

백엔드	용도	보안 수준	설정 난이도
`LocalShellBackend`	로컬 개발/테스트	낮음 (호스트 시스템 전체 접근)	설정 불필요
`Daytona`	클라우드 샌드박스 환경	높음 (격리된 컨테이너)	API 키 필요
`Modal`	서버리스 GPU/CPU 연산	높음	Modal 계정 필요
`Runloop`	관리형 클라우드 실행	높음	API 키 필요

백엔드 유형별 상세

- `StateBackend` (기본값): 파일을 LangGraph 에이전트 상태에 저장합니다. 스크래치패드 용도로 적합하며, 큰 출력물은 자동 제거됩니다.
- `FilesystemBackend` : `root_dir` 설정으로 로컬 디스크 접근을 제공합니다. `virtual_mode=True` 옵션으로 경로 제한 및 디렉터리 탐색 방지가 가능합니다.

- `LocalShellBackend` : 파일시스템 접근에 더해 `execute` 도구로 무제한 셸 명령 실행을 제공합니다. 호스트 시스템에 대한 전체 사용자 권한으로 실행됩니다.
- `CompositeBackend` : 경로별로 다른 백엔드를 라우팅합니다. 예: 임시 파일은 `StateBackend` , `/memories/` 는 `StoreBackend` .

샌드박스 보안 원칙

샌드박스는 에이전트가 코드를 안전하게 실행할 수 있는 격리된 환경을 제공합니다. 핵심 보안 원칙:

- 절대로 시크릿을 샌드박스 안에 넣지 마세요 – 컨텍스트 주입 공격으로 에이전트가 환경 변수나 마운트된 파일의 자격 증명을 읽어 유출할 수 있습니다
- 자격 증명은 샌드박스 외부의 도구에 보관하세요
- 민감한 작업에는 Human-in-the-Loop 승인을 사용하세요
- 불필요한 네트워크 접근을 차단하세요

> **! WARNING**
>
> 주의: `LocalShellBackend` 는 호스트 시스템에 대한 무제한 셸 실행 권한을 가집니다. 프로덕션에서는 반드시 샌드박스 백엔드를 사용하세요.

```
from deepagents.backends import LocalShellBackend

# 개발용 — 로컬 셸 백엔드
dev_backend = LocalShellBackend(virtual_mode=True)

# 운영용 — 클라우드 샌드박스 (택 1)
# prod_backend = ...  # 프로덕션에서는 클라우드 샌드박스 백엔드를 사용하세요
```

7.4 샘플 데이터 업로드

에이전트가 분석할 CSV 파일을 백엔드의 작업 디렉터리에 생성합니다. `create_deep_agent` 의 백엔드는 `write_file` 도구를 내장하고 있어 에이전트가 직접 파일을 쓸 수 있지만, 여기서는 미리 데이터를 준비합니다.

```
import os

workspace = "/tmp/analysis"
os.makedirs(workspace, exist_ok=True)

csv_content = (
    "region,quarter,revenue,units_sold\n"
    "Seoul,Q1,120000,340\nSeoul,Q2,135000,380\n"
    "Seoul,Q3,128000,355\nSeoul,Q4,150000,420\n"
    "Busan,Q1,85000,240\nBusan,Q2,92000,260\n"
    "Busan,Q3,88000,250\nBusan,Q4,105000,300"
)
```

```
with open(f"{workspace}/sales_2025.csv", "w") as f:
    f.write(csv_content)
print(f"CSV 저장됨: {workspace}/sales_2025.csv")
```

```
CSV 저장됨: /tmp/analysis/sales_2025.csv
```

7.5 커스텀 도구 – Slack 연동

`@tool` 데코레이터로 에이전트가 호출할 수 있는 커스텀 도구를 정의합니다. 도구의 docstring이 에이전트에게 사용법을 알려주는 역할을 합니다.

아래는 분석 결과를 Slack 채널로 전송하는 도구입니다.

```python
from langchain_core.tools import tool
try:
    from slack_sdk import WebClient
except ImportError:
    WebClient = None
    print("slack_sdk 미설치 -- Slack 도구는 스텁으로 작동합니다")

slack_client = WebClient(token=os.environ.get("SLACK_BOT_TOKEN", "xoxb-placeholder")) if WebClient else
None

@tool
def slack_send_message(message: str) -> str:
    """분석 결과를 Slack 채널로 전송합니다."""
    resp = slack_client.chat_postMessage(
        channel=os.environ.get("SLACK_CHANNEL_ID", "general"), text=message)
    return f"전송 완료. 타임스탬프: {resp['ts']}"
```

웹 검색 도구 (Tavily)

에이전트가 분석 중 도메인 맥락이 필요할 때 웹 검색을 수행할 수 있도록 Tavily 도구를 준비합니다.

```python
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def tavily_search(query: str) -> str:
    """분석에 관련된 정보를 웹에서 검색합니다."""
    results = tavily_client.search(query, max_results=5)
    return "\n".join(
        [r["content"] for r in results["results"]]
    )
```

7.6 에이전트 생성

`create_deep_agent()` 는 모델, 도구, 백엔드, 체크포인터, 시스템 프롬프트를 조합하여 에이전트를 생성합니다.

파라미터	타입	설명
`model`	`str`	모델 식별자 (기본값: `claude-sonnet-4-6`)
`tools`	`list`	에이전트가 사용할 도구 함수 목록
`backend`	`Backend`	코드 실행 및 파일시스템 백엔드

`checkpointer`	`Checkpointer`	상태 영속화 메커니즘
`system_prompt`	`str`	에이전트 행동 지침

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import TodoListMiddleware

backend = LocalShellBackend(virtual_mode=True)
checkpointer = InMemorySaver()
```

```
agent = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=backend,
    checkpointer=checkpointer,
    middleware=[TodoListMiddleware()],
    system_prompt="당신은 데이터 분석가입니다.",
)
```

7.6.5 `create_agent()` 로 해부해 보기

`create_deep_agent()` 는 교육과 실무에서 바로 쓰기 좋은 shortcut입니다. 내부 관점에서는 LangChain `create_agent()` 에 filesystem, subagent, skills/memory, 선택적 planning 미들웨어를 조합한 하네스라고 이해하면 됩니다.

관점	`create_deep_agent()`	`create_agent()` + middleware
목적	Deep Agent 기본 하네스 빠른 구성	필요한 정책만 직접 조립
장점	파일 도구·서브에이전트·skills/memory 기본 제공	최소 구성, 세밀한 제어
계획	`TodoListMiddleware()` 를 명시적으로 추가	같은 미들웨어를 직접 추가
교육 포인트	“완성형 하네스 사용”	“하네스 내부 구조 해부”

데이터 분석 에이전트처럼 파일 읽기·코드 실행·결과 저장이 핵심이면 `create_deep_agent()` 가 기본값입니다. 아주 작은 도구 호출 assistant나 미들웨어 실험은 `create_agent()` 로 시작해도 됩니다.

```
# 하네스 해부용 reference code — 실제 분석 에이전트는 위의 create_deep_agent() 사용
agent_anatomy = r'''
from langchain.agents import create_agent

base_agent = create_agent(
    model="openai:gpt-5.4",
    tools=[tavily_search, slack_send_message],
    system_prompt="당신은 데이터 분석가입니다.",
    # 필요 시 middleware=[...]로 정책을 직접 조합
)
'''
print(agent_anatomy)
```

7.7 실행 — 분석 요청

`agent.invoke()` 로 분석 요청을 전달하면 에이전트가 자율적으로 계획 수립 → 파일 읽기 → 코드 실행 → 결과 전달 파이프라인을 수행합니다.

7.8 스트리밍으로 분석 과정 관찰

`agent.stream()` 을 사용하면 에이전트의 실행 과정을 실시간으로 관찰할 수 있습니다. Deep Agents는 LangGraph의 스트리밍 인프라를 기반으로 서브에이전트의 실행까지 추적할 수 있는 강력한 모니터링을 제공합니다.

스트림 모드

모드	설명	사용 사례
Updates	각 단계(노드) 완료 시 이벤트 수신	진행 상황 대시보드, 로그
Messages	개별 토큰 스트리밍, 소스 에이전트 메타데이터 포함	실시간 채팅 UI
Custom	`get_stream_writer()` 로 커스텀 진행 이벤트 발행	분석 진행률 표시, 커스텀 알림

네임스페이스 시스템

스트리밍 이벤트에는 소스를 식별하는 네임스페이스가 포함됩니다:

- `()` (빈 튜플) = 메인 에이전트
- `("tools:abc123",)` = 도구 호출로 생성된 서브에이전트
- `("tools:abc123", "model_request:def456")` = 서브에이전트 내부의 모델 요청 노드

`subgraphs=True` 를 설정하면 서브에이전트의 실행까지 추적할 수 있으며, 여러 스트림 모드를 동시에 사용할 수도 있습니다:

```
for namespace, chunk in agent.stream(
    {"messages": [...]},
    stream_mode=["updates", "messages", "custom"],
    subgraphs=True,
):
    mode, data = chunk
    # 각 모드별로 다르게 처리
```

서브에이전트 라이프사이클 추적

서브에이전트는 세 단계를 거칩니다:

1. Pending – 메인 에이전트의 `model_request` 에 태스크 도구 호출이 포함될 때 감지
2. Running – `tools:UUID` 네임스페이스에서 이벤트가 발생할 때 시작
3. Complete – 메인 에이전트의 `tools` 노드가 결과를 반환할 때 완료

7.9 도구 활용

Deep Agents는 백엔드를 통해 파일 도구를 제공하고, 계획 도구는 `TodoListMiddleware`로 명시적으로 추가합니다.

도구	설명	사용 예시
`write_todos`	구조화된 작업 계획 수립 및 추적 (`TodoListMiddleware` 필요)	분석 단계별 TODO 리스트 생성
`ls`	디렉터리 내용 나열	CSV 파일 존재 여부 확인
`read_file`	파일 읽기 (이미지 멀티모달 지원)	CSV 구조 파악, 차트 이미지 확인
`write_file`	파일 쓰기 또는 기존 파일 교체	분석 스크립트, 결과 파일 저장
`edit_file`	기존 파일 수정 (find-and-replace)	코드 수정, 설정 파일 업데이트
`delete`	delete 지원 백엔드에서 파일 삭제	임시 산출물 정리
`glob`	glob 패턴 기반 파일 탐색	`*.csv` 패턴으로 데이터 파일 검색
`grep`	패턴 매칭 검색	특정 컬럼명이나 값 검색

커스텀 백엔드 구현

새 교육 코드에서는 `BackendProtocol`의 `ls`, `read`, `write`, `edit`, `grep`, `glob` 메서드를 구현합니다. `delete`는 선택 메서드이며 구현한 백엔드에서만 `delete`이 노출됩니다.

Skills 패턴 — 도메인 지식을 진보적 공개로 분리

복잡한 데이터 분석은 도메인별 컨텍스트가 비대해지기 쉽습니다. Skills는 `SKILL.md`와 선택적 `metadata.include_tools`로 도메인 지식을 분리하여, 에이전트가 필요할 때만 로드합니다.

```
skills/
└── pandas_eda/
    ├── SKILL.md       # name, description, metadata.include_tools
    └── helpers.py     # 결정적 헬퍼
```

`metadata.include_tools`는 공백으로 구분한 도구 이름 문자열입니다. 도구 구현은 agent tools 또는 `SkillsMiddleware(tools=...)`로 따로 제공해야 합니다.

7.10 체크포인터로 대화 유지

체크포인터는 에이전트 상태를 저장하여 중단 후 재개가 가능하게 합니다. `thread_id`로 대화 세션을 식별하며, 동일한 `thread_id`로 이어서 요청하면 이전 대화 맥락이 유지됩니다.

체크포인터	용도	영속성
`InMemorySaver`	개발/테스트	프로세스 종료 시 소멸

`SQLiteSaver`	로컬 영속	디스크에 저장 (`./agent_checkpoints.db`)
`PostgresSaver`	프로덕션	데이터베이스에 저장, 다중 인스턴스 지원

체크포인터의 역할

체크포인터는 단순한 대화 이력 저장을 넘어 다음을 가능하게 합니다:

- 중단 복구: 네트워크 오류나 타임아웃 시 마지막 완료된 단계부터 재개
- `interrupt()` 지원: Human-in-the-Loop에서 그래프 상태를 저장하여 사람의 응답 후 정확한 위치에서 재개
- 멀티턴 분석: 동일 세션에서 여러 분석 요청을 연속으로 처리하면서 이전 결과를 참조

샌드박스 수명 관리

샌드박스를 사용할 때는 명시적인 종료(shutdown)가 필요합니다. 종료하지 않으면 불필요한 비용이 발생합니다. 채팅 애플리케이션에서는 대화 스레드별로 고유한 샌드박스를 사용하고, TTL(Time-to-Live) 설정으로 자동 정리를 구성하세요.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
config = {"configurable": {"thread_id": "analysis-session-1"}}

agent_with_memory = create_deep_agent(
    model="gpt-5.4",
    tools=[tavily_search, slack_send_message],
    backend=LocalShellBackend(virtual_mode=True),
    checkpointer=checkpointer,
)
```

요약

주제	핵심 내용
백엔드	`LocalShell` (개발)은 호스트 셸 접근, 프로덕션에서는 `Daytona` / `Modal` / `Runloop` 샌드박스 사용
커스텀 도구	`\@tool` 데코레이터 + docstring으로 정의, 에이전트가 자율 호출
에이전트 생성	`create_deep_agent(model, tools, backend, checkpointer, middleware, system_prompt)`
계획	`TodoListMiddleware()` 를 명시적으로 추가하면 `write_todos` 사용 가능
스트리밍	`agent.stream(stream_mode="updates", subgraphs=True)` 로 실시간 관찰
파일 도구	`ls` , `read_file` , `write_file` , `edit_file` , `delete` , `glob` , `grep`
체크포인터	`InMemorySaver` (개발) → `SQLiteSaver` → `PostgresSaver` (프로덕션)

08 보이스 에이전트

STT/Agent/TTS 파이프라인

음성 입력을 텍스트로 변환(STT)하고, LangChain 에이전트가 처리한 뒤, 텍스트를 음성으로 합성(TTS)하여 실시간으로 돌려주는 보이스 에이전트를 구축합니다.

이 아키텍처는 Sandwich 패턴 (STT -> Agent -> TTS)으로, 각 계층이 스트리밍으로 연결되어 sub-700ms 레이턴시를 목표로 합니다.

핵심 설계 원칙

Sandwich 아키텍처의 핵심은 스트리밍 체이닝입니다. 각 레이어가 이전 레이어의 완전한 출력을 기다리지 않고, 부분 결과가 생성되는 즉시 다음 레이어로 전달합니다:

- STT: 부분 전사(partial transcript)를 실시간으로 생성하고, 발화 완료(end-of-speech) 감지 시 최종 전사를 emit
- Agent: `astream()` 으로 토큰 단위로 응답을 스트리밍 — 전체 응답 생성 완료를 기다리지 않음
- TTS: WebSocket 기반으로 텍스트 청크가 도착하는 즉시 오디오 합성 시작

이 구조 덕분에 전체 파이프라인의 지연 시간은 각 단계의 합이 아닌, 각 단계의 첫 출력까지의 시간 합으로 줄어듭니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. Sandwich 아키텍처 — STT -> Agent -> TTS 파이프라인의 구조와 데이터 흐름을 설명할 수 있다
2. 아키텍처 비교 — Sandwich 방식과 Speech-to-Speech(S2S) 방식의 장단점을 비교할 수 있다
3. STT 단계 — AssemblyAI 실시간 전사의 Producer-Consumer 패턴을 이해할 수 있다
4. 에이전트 단계 — LangChain `create_agent` 로 스트리밍 응답을 생성하는 에이전트를 구축할 수 있다
5. TTS 단계 — Cartesia WebSocket 기반 스트리밍 음성 합성의 동작 원리를 이해할 수 있다
6. RunnableGenerator — 비동기 제너레이터 체이닝으로 파이프라인을 조합할 수 있다
7. 성능 최적화 — 레이턴시 목표 달성을 위한 각 단계별 최적화 기법을 이해할 수 있다

> · NOTE
>
> 참고: STT(AssemblyAI)와 TTS(Cartesia)는 외부 유료 서비스입니다. 해당 셀은 개념 설명용 마크다운으로 제공되며, 에이전트 생성 부분만 실제 실행 가능한 코드입니다.

8.1 환경 설정

보이스 에이전트에 필요한 패키지와 각 역할입니다:

패키지	역할
`langchain` , `langchain-openai`	에이전트 생성 및 LLM 연결
`assemblyai`	실시간 음성-텍스트 변환 (STT)
`cartesia`	저지연 텍스트-음성 합성 (TTS)
`websockets`	실시간 양방향 통신 서버
`pyaudio`	마이크 오디오 캡처 (선택)

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(model="gpt-5.4")
```

8.2 보이스 에이전트 아키텍처 개요

보이스 에이전트는 3-레이어 파이프라인으로 구성됩니다:

```
Audio In -> [STT: AssemblyAI] -> Text -> [Agent: LangChain] -> Text -> [TTS: Cartesia] -> Audio Out
```

레이어	제공자	역할
STT	AssemblyAI	사용자 음성을 텍스트로 변환
Agent	LangChain	텍스트 쿼리를 처리하고 응답 생성
TTS	Cartesia	에이전트의 텍스트 응답을 음성으로 변환

각 레이어가 스트리밍으로 연결되어, 이전 단계의 완전한 출력을 기다리지 않고 부분 결과를 즉시 다음 단계로 전달합니다:

1. STT – 부분 전사 결과를 스트리밍 (발화 완료 감지 시 최종 전사)
2. Agent – 토큰 단위 스트리밍 응답 생성 (`astream()`)
3. TTS – 전체 응답 완료 전에 음성 합성 시작 (WebSocket 기반)

스트리밍이 중요한 이유

스트리밍 없이 동기적으로 처리하면, 각 단계가 완전히 끝나야 다음 단계가 시작됩니다. 예를 들어 에이전트가 200 토큰 응답을 생성하는 데 3초가 걸린다면, TTS는 3초 후에야 시작됩니다. 스트리밍에서는 에이전트의 첫 토큰이 나오는 즉시 TTS가 음성 합성을 시작하므로, 사용자는 에이전트가 아직 응답을 생성하는 동안에도 이미 음성을 듣기 시작합니다.

멀티모달 콘텐츠 블록과 `output_version="v1"`

LangChain 1.x 의 메시지는 텍스트, 이미지, 오디오, 비디오 등 다양한 modality 를 표현할 수 있는 content blocks 구조를 사용합니다. `ChatOpenAI(output_version="v1")` 로 모델을 생성하면, 응답이 표준 블록 리스트 (`[{"type": "text", "text": "..."}, {"type": "audio", "audio": {...}}]`) 형태로 직렬화됩니다.

블록 타입	용도
`text`	일반 텍스트 응답
`image` / `image_url`	이미지 입력·출력
`audio`	오디오 입력 (STT 전 단계) 또는 출력 (TTS 직전 단계)
`tool_use` / `tool_result`	도구 호출과 결과

Sandwich 아키텍처에서는 STT 가 텍스트를 만들어 에이전트에 넘기지만, 멀티모달 모델(gpt-5.4 audio, claude-sonnet-4-6 등) 을 직접 사용할 때는 오디오 블록을 메시지 콘텐츠로 직접 주입할 수 있습니다. `output_version="v1"` 은 이 직렬화 형식을 안정화합니다.

8.3 아키텍처 비교표

음성 에이전트를 구축하는 두 가지 접근법을 비교합니다.

Sandwich 아키텍처 (STT -\> Agent -\> TTS)

항목	내용
장점	각 컴포넌트 독립 교체 가능, 텍스트 기반 도구 활용 용이, 디버깅 편리 (중간 텍스트 로깅)
단점	3단계 직렬 처리로 레이턴시 누적, 감정/억양 등 비언어 정보 손실
적합 상황	도구 호출이 많은 복잡한 에이전트, 다국어 지원 필요 시

Speech-to-Speech (S2S)

항목	내용
장점	낮은 레이턴시, 음성 특성(감정, 강세) 보존, 자연스러운 대화
단점	도구 호출 통합 어려움, 모델 선택지 제한, 디버깅 어려움
적합 상황	단순 대화형 인터페이스, 감정 인식이 중요한 경우

> · NOTE
>
> 이 노트북에서는 도구 호출이 가능한 Sandwich 아키텍처에 집중합니다.

8.4 STT 단계 – AssemblyAI 실시간 전사

> ➡️ TIP
>
> 이 셀은 AssemblyAI API 키가 필요합니다. 개념 이해를 위한 참조 코드입니다.

AssemblyAI의 `RealtimeTranscriber`는 Producer-Consumer 패턴으로 동작합니다:

- Producer: 마이크에서 캡처한 오디오 청크를 WebSocket으로 전송
- Consumer: 부분 전사(partial) 및 최종 전사(final) 결과를 콜백으로 수신

두 작업이 동시에 진행되므로, 음성 전송 중에도 이전 발화의 전사 결과를 받을 수 있습니다.

전사 결과의 두 가지 유형

유형	클래스	설명
Partial	`RealtimeTranscript`	아직 발화 중인 단어들의 임시 전사. 사용자가 말하는 동안 실시간으로 업데이트됨
Final	`RealtimeFinalTranscript`	발화 종료(엔드포인팅)가 감지된 후의 확정 전사. 이 결과만 에이전트로 전달

AssemblyAI의 내장 VAD(Voice Activity Detection)가 발화 종료 시점을 자동으로 감지하므로, 별도의 무음 감지 로직이 필요하지 않습니다.

```
import assemblyai as aai

aai.settings.api_key = "your-assemblyai-key"

transcriber = aai.RealtimeTranscriber(
    sample_rate=16000,
    encoding=aai.AudioEncoding.pcm_s16le,
    on_data=on_transcription_data,
    on_error=on_transcription_error,
)

def on_transcription_data(transcript: aai.RealtimeTranscript):
    if isinstance(transcript, aai.RealtimeFinalTranscript):
        process_user_input(transcript.text)

def on_transcription_error(error: aai.RealtimeError):
    print(f"Transcription error: {error}")

transcriber.connect()
```

마이크 오디오 캡처

PCM 16-bit, 16kHz 단일 채널 오디오를 캡처하여 실시간으로 전사기에 전송합니다:

```
import pyaudio

audio = pyaudio.PyAudio()
stream = audio.open(
    format=pyaudio.paInt16,
```

```
    channels=1, rate=16000,
    input=True, frames_per_buffer=1024,
)

while True:
    data = stream.read(1024)
    transcriber.stream(data)
```

8.5 에이전트 단계 – LangChain 에이전트 활용

파이프라인의 핵심인 에이전트 단계입니다. `create_agent`로 생성한 에이전트는 텍스트 입력을 받아 도구 호출과 추론을 수행한 뒤 텍스트 응답을 스트리밍합니다.

보이스 에이전트용 시스템 프롬프트의 핵심은 간결하고 대화체인 응답을 유도하는 것입니다. 음성 출력은 텍스트와 달리 긴 응답이 사용자에게 부담이 되므로, 1 2문장 이내의 짧은 응답을 생성하도록 지시합니다.

비동기 스트리밍의 역할

에이전트의 `astream()` 메서드는 응답 토큰을 생성되는 즉시 yield합니다. 보이스 에이전트에서 특히 중요한 이유:

- TTS 조기 시작: 전체 응답 완료를 기다리지 않고 첫 토큰부터 음성 합성 가능
- 체감 지연 감소: 사용자가 에이전트의 응답을 더 빨리 듣기 시작
- 파이프라인 효율성: Agent와 TTS가 동시에 실행되어 총 처리 시간 단축

```
from langchain.agents import create_agent

def search_tool(query: str) -> str:
    """웹에서 최신 정보를 검색합니다."""
    return f"검색 결과: {query}"

def calendar_tool(action: str, details: str) -> str:
    """캘린더 이벤트를 관리합니다."""
    return f"캘린더 {action}: {details}"
```

```
agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool],
    system_prompt=(
        "당신은 유용한 음성 어시스턴트입니다. "
        "응답을 간결하고 대화체로 유지하세요."
    ),
)
```

비동기 스트리밍 응답 생성

`astream()`은 에이전트의 응답 토큰을 생성되는 즉시 yield합니다. TTS 단계가 전체 응답 완료를 기다리지 않고 음성 합성을 시작할 수 있게 합니다.

LangChain의 스트리밍은 크게 두 가지 방식을 제공합니다:

메서드	설명
`astream()`	에이전트 실행의 각 스텝별 출력을 스트리밍. 메시지, 도구 호출 결과 등을 포함

`astream_events()`	더 세분화된 이벤트 단위 스트리밍. 개별 토큰, LLM 시작/종료 등 상세 이벤트 제공

보이스 에이전트에서는 `astream()`으로 메시지 청크를 수신하고, 각 청크의 `content`를 추출하여 TTS로 전달합니다.

```
async def stream_agent_response(user_text: str):
    """에이전트 응답 토큰을 하나씩 스트리밍합니다."""
    async for chunk in agent.astream(
        {"messages": [{"role": "user",
                       "content": user_text}]}
    ):
        if "messages" in chunk:
            for msg in chunk["messages"]:
                if hasattr(msg, "content") and msg.content:
                    yield msg.content
```

8.6 TTS 단계 – Cartesia 스트리밍 음성 합성

> ➡️ TIP
>
> 이 셀은 Cartesia API 키가 필요합니다. 개념 이해를 위한 참조 코드입니다.

Cartesia는 WebSocket 기반 저지연 TTS를 제공합니다. 에이전트의 텍스트 스트림을 받아 부분 텍스트마다 즉시 오디오 청크를 생성합니다.

Cartesia TTS의 동작 방식

1. WebSocket 연결: `AsyncCartesia` 클라이언트로 WebSocket 세션을 열어 지속적인 연결 유지
2. 텍스트 청크 전송: 에이전트가 생성한 토큰/문장 단위로 텍스트를 전송
3. 오디오 청크 수신: 각 텍스트 청크에 대해 즉시 PCM 오디오 바이트를 수신
4. 클라이언트 전달: 수신된 오디오 바이트를 WebSocket으로 클라이언트에 스트리밍

WebSocket 기반이므로 HTTP 요청/응답 오버헤드 없이 양방향 실시간 통신이 가능합니다. `sonic-2` 모델은 자연스러운 음성과 낮은 지연 시간을 동시에 제공합니다.

```
import cartesia

cartesia_client = cartesia.AsyncCartesia(
    api_key="your-cartesia-key"
)

async def text_to_speech_stream(text_stream):
    ws = await cartesia_client.tts.websocket()
    async for text_chunk in text_stream:
        audio_chunks = ws.send(
            model_id="sonic-2",
            transcript=text_chunk,
            voice_id="your-voice-id",
            stream=True,
            output_format={
                "container": "raw",
                "encoding": "pcm_s16le",
                "sample_rate": 24000,
            },
```

```
        )
        async for audio in audio_chunks:
            yield audio["audio"]
    await ws.close()
```

8.7 파이프라인 조합 – RunnableGenerator

LangChain의 `RunnableGenerator`를 사용하면 비동기 제너레이터를 Runnable 파이프라인에 통합할 수 있습니다. STT 출력 -> Agent 처리 -> TTS 입력이라는 전체 데이터 흐름을 하나의 파이프라인으로 구성할 수 있습니다.

RunnableGenerator란?

`RunnableGenerator`는 비동기 제너레이터 함수를 LangChain의 `Runnable` 인터페이스로 래핑합니다. 이렇게 하면:

- `|` (파이프) 연산자로 다른 Runnable과 체이닝 가능
- LangChain의 `batch()`, `stream()` 등 표준 메서드 사용 가능
- 입력 스트림을 받아 변환된 출력 스트림을 생성하는 패턴에 적합

```
from langchain_core.runnables import RunnableGenerator

async def transform_input(input_stream):
    async for text in input_stream:
        async for token in stream_agent_response(text):
            yield token

agent_runnable = RunnableGenerator(transform_input)
```

전체 파이프라인 연결 (개념 코드)

`asyncio.Queue`를 사용하여 STT의 최종 전사 결과를 에이전트로 전달하고, 에이전트의 스트리밍 응답을 TTS로 중계합니다. `Queue`는 비동기 Producer-Consumer 패턴의 핵심 구성 요소입니다.

```
async def voice_pipeline(audio_input_stream):
    transcript_queue = asyncio.Queue()

    def on_final(transcript):
        if isinstance(transcript, aai.RealtimeFinalTranscript):
            transcript_queue.put_nowait(transcript.text)

    transcriber = aai.RealtimeTranscriber(
        sample_rate=16000, on_data=on_final
    )
    transcriber.connect()

    async for audio_chunk in audio_input_stream:
        transcriber.stream(audio_chunk)
        if not transcript_queue.empty():
            user_text = await transcript_queue.get()
            text_stream = stream_agent_response(user_text)
            async for audio in text_to_speech_stream(text_stream):
                yield audio
```

8.8 WebSocket 서버 – 실시간 양방향 통신

> ➡️ TIP
>
> 이 셀의 코드를 실행하면 서버가 시작됩니다. 개념 이해를 위한 참조 코드입니다.

WebSocket으로 클라이언트와 양방향 오디오 스트리밍을 처리합니다. `asyncio.gather`로 수신과 송신을 동시 실행합니다.

WebSocket을 사용하는 이유

보이스 에이전트는 양방향 실시간 통신이 필수입니다:

- 수신 (Receive): 클라이언트가 마이크 오디오를 서버로 지속적으로 전송
- 송신 (Send): 서버가 합성된 음성 오디오를 클라이언트로 지속적으로 전송

HTTP의 요청-응답 모델은 이런 양방향 스트리밍에 맞지 않습니다. WebSocket은 단일 TCP 연결 위에서 양방향 데이터 흐름을 지원하며, `asyncio.gather`를 사용하면 수신과 송신이 서로를 블록하지 않고 동시에 실행됩니다.

```
import websockets
import asyncio

async def handle_client(websocket):
    transcriber = create_transcriber()
    transcriber.connect()

    async def receive_audio():
        async for message in websocket:
            if isinstance(message, bytes):
                transcriber.stream(message)

    async def send_audio():
        async for transcript in transcription_queue:
            text_stream = stream_agent_response(transcript)
            async for audio in text_to_speech_stream(text_stream):
                await websocket.send(audio)

    await asyncio.gather(receive_audio(), send_audio())

async def main():
    async with websockets.serve(handle_client, "0.0.0.0", 8765):
        print("Voice agent server on ws://0.0.0.0:8765")
        await asyncio.Future()
```

8.9 성능 목표 – Sub-700ms 레이턴시

보이스 에이전트의 핵심 성능 지표는 발화 종료부터 첫 오디오 출력까지의 시간(Time to First Audio, TTFA)입니다. 자연스러운 대화 경험을 위해 이 지표가 700ms 미만이어야 합니다.

레이턴시 분석

단계	목표 시간	설명

STT 최종 전사	200ms	발화 종료 감지 -\> 최종 텍스트 (AssemblyAI 내장 엔드포인팅)
Agent 첫 토큰	300ms	텍스트 입력 -\> 첫 응답 토큰 생성 (TTFT: Time to First Token)
TTS 첫 오디오	150ms	첫 텍스트 토큰 -\> 첫 오디오 청크 (WebSocket 기반)
합계	\<700ms	발화 종료 -\> 첫 오디오 출력

최적화 기법

기법	효과	구현 방법
스트리밍 STT	전체 전사 대기 제거	`RealtimeTranscriber` + 부분 결과
스트리밍 Agent	전체 응답 완료 전 TTS 시작	`astream()` 사용 — `ainvoke()` 대신
스트리밍 TTS	첫 오디오까지 시간 단축	WebSocket 기반 합성
커넥션 풀링	연결 설정 지연 제거	WebSocket 재사용 (요청마다 새 연결 생성 방지)
VAD	무음 구간 처리 방지	AssemblyAI 내장 엔드포인팅
응답 캐싱	빈번한 질문 즉시 응답	자주 묻는 질문 캐싱

> ➡️ TIP
>
> 팁: Agent의 TTFT(첫 토큰까지 시간)가 전체 레이턴시에서 가장 큰 비중을 차지합니다. 짧은 시스템 프롬프트와 경량 모델 선택이 레이턴시 최적화의 핵심입니다.

8.10 에이전트에 도구 추가

보이스 에이전트의 강점은 Sandwich 아키텍처 덕분에 텍스트 기반 도구를 그대로 활용할 수 있다는 점입니다. 검색, 일정 관리, 날씨 조회 등 다양한 도구를 추가할 수 있습니다.

S2S(Speech-to-Speech) 방식과 가장 다른 점입니다. S2S 모델은 오디오를 직접 처리하므로 텍스트 기반 도구 호출이 어렵지만, Sandwich 아키텍처는 에이전트가 텍스트 영역에서 동작하므로 기존의 모든 LangChain 도구를 그대로 사용할 수 있습니다.

도구 설계 팁

음성 에이전트의 도구는 빠른 응답이 중요합니다. 도구 실행 시간이 길어지면 전체 파이프라인의 레이턴시가 커지므로:

- 가벼운 API 호출 위주로 설계
- 타임아웃 설정 필수
- 가능하면 캐싱 적용

```python
def weather_tool(city: str) -> str:
    """도시의 현재 날씨를 조회합니다."""
    return f"{city} 날씨: 15도, 구름 조금"

def reminder_tool(time: str, message: str) -> str:
    """지정된 시간에 알림을 설정합니다."""
    return f"{time}에 알림 설정됨: {message}"
```

```python
voice_agent = create_agent(
    model="gpt-5.4",
    tools=[search_tool, calendar_tool,
           weather_tool, reminder_tool],
    system_prompt=(
        "당신은 음성 어시스턴트입니다. "
        "1-2문장으로 간결하고 대화체로 응답하세요."
    ),
)
```

요약

주제	핵심 내용
Sandwich 아키텍처	STT -\> Agent -\> TTS, 각 레이어 독립 교체 가능, 도구 호출 지원
STT (AssemblyAI)	`RealtimeTranscriber`, Producer-Consumer 패턴, WebSocket 스트리밍
Agent (LangChain)	`create_agent` + `astream()`, 토큰 단위 스트리밍, 도구 통합
TTS (Cartesia)	WebSocket 기반 저지연 합성, 부분 텍스트 즉시 오디오 변환
파이프라인 조합	`RunnableGenerator`, 비동기 제너레이터 체이닝
성능 목표	Sub-700ms (STT 200ms + Agent 300ms + TTS 150ms)
도구 확장	텍스트 기반 도구를 그대로 음성 에이전트에 적용 가능

09 프로덕션 배포

– 테스트, 관측성, 배포

에이전트를 프로덕션에 배포하기 위한 전체 파이프라인을 다룹니다. 단위 테스트와 LangSmith 평가로 품질을 보장하고, 트레이싱으로 관측성을 확보한 뒤, LangGraph Platform으로 배포합니다.

```
개발 -> 테스트 (단위 + 평가) -> 관측성 (트레이싱) -> 배포 (LangGraph Platform)
```

왜 에이전트에 특화된 프로덕션 파이프라인이 필요한가?

에이전트는 전통적인 소프트웨어와 다른 특성을 갖습니다:

- 비결정적 실행: 동일한 입력에도 다른 도구 호출 순서, 다른 응답을 생성할 수 있음
- 상태 유지: 대화 기록과 체크포인트를 관리하는 장기 실행 프로세스
- 다중 컴포넌트: LLM, 도구, 메모리, 체크포인터 등 여러 시스템이 연동

따라서 단순한 단위 테스트를 넘어 궤적(trajectory) 평가, LLM-as-Judge, 트레이스 기반 모니터링 등 에이전트 전용 품질 보장 기법이 필요합니다.

학습 목표

이 노트북을 완료하면 다음을 수행할 수 있습니다:

1. 단위 테스트 – `GenericFakeChatModel`로 LLM 응답을 모킹하여 결정적 테스트를 작성할 수 있다
2. LangSmith 평가 – 데이터셋 생성, 평가자 정의, 자동화된 에이전트 평가를 수행할 수 있다
3. 트레이스 분석 – LangSmith 트레이싱으로 레이턴시, 토큰 사용량, 에러를 추적할 수 있다
4. LangGraph Studio – 시각적 디버깅 도구로 에이전트 실행 흐름을 분석할 수 있다
5. 배포 옵션 – LangGraph Platform, 셀프호스트, Cloud 배포 간 차이를 이해할 수 있다
6. langgraph.json – 배포 설정 파일을 구성하고 배포 명령어를 실행할 수 있다
7. Runtime rubric – Deep Agents `RubricMiddleware`로 실행 중 품질 게이트를 구성할 수 있다

9.1 환경 설정

테스트, 관측성, 배포에 필요한 패키지를 설치합니다.

9.2 프로덕션 파이프라인 개요

에이전트의 비결정적 특성 때문에 전통적인 소프트웨어 테스트만으로는 품질을 보장할 수 없습니다.

단계	목적	도구
개발	에이전트 구현 및 로컬 테스트	LangGraph, LangSmith Studio
테스트	단위 / 통합 / Evals 3단 테스트	`pytest`, `GenericFakeChatModel`, `agentevals`, LangSmith
관측성	트레이싱, 메트릭, 에러 추적	LangSmith Tracing
배포	프로덕션 환경 배포	LangGraph Platform

테스트 3분류 (Unit / Integration / Evals)

`docs/langchain/27-test.md` 는 에이전트 테스트를 3개 축으로 정리합니다.

유형	특징	적합 대상
Unit tests	`GenericFakeChatModel` / `InMemorySaver` 로 외부 호출 없이 결정적 검증. 빠르고 저렴	개별 도구, 파서, 프롬프트 포맷팅, 상태 전이
Integration tests	실제 네트워크 호출로 컴포넌트 협업, 자격 증명, 레이턴시 검증	전체 에이전트 흐름, 도구 호출 시퀀스
Evals	`agentevals` 패키지로 trajectory 평가. trajectory match 또는 LLM-as-judge	행동 패턴, 응답 품질 회귀

`agentevals` 는 다음 두 평가 전략을 제공합니다:

- Trajectory match — 기대 시퀀스와 단계별 비교. 매칭 모드 4종 (`strict` / `unordered` / `subset` / `superset`)
- LLM-as-judge — 루브릭 기반 정성 평가. 유연하지만 LLM 비용 + 비결정성

대규모 평가, 결과 추적, 실험 비교에는 LangSmith 를 사용합니다. CI/CD 비용 절감용으로는 `vcrpy` + `pytest-recording` 으로 HTTP 응답을 녹화·재생합니다.

9.3 Agent testing – LangSmith evaluation

`agentevals` 는 에이전트 trajectory를 평가하는 도구를 제공합니다. trajectory는 최종 답변에 도달하기까지의 도구 호출, 중간 판단, 의사결정 경로 전체를 뜻합니다.

전략	설명	장점	단점
Trajectory Match	기대한 단계와 실제 trajectory를 비교	재현성 높음, 회귀 감지 명확	기대 경로 작성 비용
LLM-as-Judge	rubric 기준으로 trajectory와 답변을 평가	복잡한 시나리오 대응	비용과 비결정성

RubricMiddleware	runtime에서 criterion별 커버리지와 grader hook을 사용	실행 중 품질 게이트 가능	grader 설계가 필요

Deep Agents의 `RubricMiddleware`는 단순 문자열 rubric과 다릅니다. criterion마다 평가 결과가 빠지지 않도록 설계하고, 오프라인 `evaluate`와 runtime self-revision의 책임을 나눕니다.

```
try:
    from langsmith import Client

    ls_client = Client()

    dataset = ls_client.create_dataset("agent-eval-v1")
    ls_client.create_examples(
        inputs=[{"query": "LangGraph란 무엇인가요?"}],
        outputs=[{"expected": "에이전트를 위한 프레임워크"}],
        dataset_id=dataset.id,
    )
    print("데이터셋 생성됨:", dataset.name)
except Exception as e:
    print(f"LangSmith 미설정 (건너뜀): {e}")
    ls_client = None
    dataset = None
```

```
LangSmith 미설정 (건너뜀): Authentication failed for /datasets. HTTPError('401 Client Error: Unauthorized for url:
https://api.smith.langchain.com/datasets', '{"detail":"Invalid token"}')
```

```
try:
    from agentevals.trajectory import create_trajectory_llm_as_judge

    evaluator = create_trajectory_llm_as_judge(
        rubric=(
            "에이전트가 적절한 도구를 사용했습니까? "
            "최종 답변이 정확했습니까?"
        ),
    )
    print("평가자 생성됨:", type(evaluator).__name__)
except ImportError:
    print("agentevals 미설치. 설치: pip install agentevals")
    evaluator = None
except Exception as e:
    print(f"평가자 생성 건너뜀 (LLM API 키 필요): {e}")
    evaluator = None
```

```
평가자 생성 건너뜀 (LLM API 키 필요): create_trajectory_llm_as_judge() got an unexpected keyword argument 'rubric'
```

9.3.5 Runtime RubricMiddleware — 실행 중 품질 게이트

`agentevals`와 LangSmith 평가는 배포 전/후 회귀를 확인하는 오프라인 평가에 가깝습니다. Deep Agents의 `RubricMiddleware`는 실제 요청 처리 중에 judge 모델을 호출해 답변을 점검하고, 기준을 만족하지 못하면 추가 수정을 유도하는 런타임 품질 게이트입니다.

사용 지점	권장 기준
외부 전송 직전	근거, 금지 표현, 개인정보 누락 여부
긴 분석 리포트	숫자·출처·결론 일관성

고위험 도구 실행 후	작업 결과와 사용자 요청의 일치 여부

모든 요청에 붙이면 비용과 지연이 커지므로, 실패 비용이 큰 구간에 선택적으로 적용합니다.

```
try:
    from deepagents import RubricMiddleware

    rubric = RubricMiddleware(
        model="openai:gpt-5.4-mini",
        system_prompt="정확성, 근거, 안전성 기준으로 평가하고 부족하면 수정 지시",
        max_iterations=2,
    )
    print("RubricMiddleware 준비됨:", type(rubric).__name__)
except ImportError:
    print("deepagents>=0.6.10 필요: pip install -U deepagents")
```

9.4 단위 테스트 패턴

`GenericFakeChatModel`을 사용하면 API 호출 없이 LLM 응답을 모킹하여 결정적 테스트를 작성할 수 있습니다. 응답 이터레이터를 받아 호출마다 하나씩 반환합니다.

왜 모킹이 필요한가?

에이전트 테스트에서 실제 LLM API를 호출하면 다음 문제가 발생합니다:

- 비결정적 결과: 동일 입력에도 매번 다른 응답이 올 수 있어 테스트 재현이 어려움
- 비용: 테스트를 실행할 때마다 API 비용 발생
- 속도: 네트워크 지연으로 테스트 속도 저하
- 가용성: API 장애 시 테스트 실패

`GenericFakeChatModel`은 미리 정의한 응답을 순서대로 반환하므로, 결정적이고 빠르며 무료인 테스트를 작성할 수 있습니다. 스트리밍 패턴도 지원하여 `astream()` 기반 코드도 테스트 가능합니다.

상태 지속성 테스트

`InMemorySaver` 체크포인터를 사용하면 여러 대화 턴에 걸친 상태 의존적 행동을 테스트할 수 있습니다. `thread_id`를 지정하여 같은 대화 컨텍스트를 유지하면서 여러 호출의 누적 상태를 검증합니다.

```
def search_tool(query: str) -> str:
    """웹에서 정보를 검색합니다."""
    return f"검색 결과: {query}"

def test_search_tool():
    """검색 도구가 예상 형식을 반환하는지 테스트합니다."""
    result = search_tool("test query")
    assert isinstance(result, str) and len(result) > 0
    print("통과: test_search_tool")

test_search_tool()
```

```
통과: test_search_tool
```

HTTP 요청 녹화/재생

CI/CD에서 API 비용을 줄이려면 `vcrpy`와 `pytest-recording`으로 HTTP 요청을 녹화하고 재생할 수 있습니다. 첫 실행 시 실제 API 호출을 녹화(cassette 파일로 저장)하고, 이후 실행에서는 녹화된 응답을 재생하여 네트워크 호출 없이 테스트합니다.

이 방식의 장점:

- 첫 실행: 실제 API와의 통합을 검증 (통합 테스트 역할)
- 이후 실행: 녹화된 응답으로 빠르고 결정적인 테스트 (단위 테스트 역할)
- CI/CD: API 키 없이도 테스트 실행 가능

```
import pytest

@pytest.fixture(scope="module")
def vcr_config():
    return {"record_mode": "once"}

@pytest.mark.vcr()
def test_agent_with_recorded_responses():
    result = agent.invoke("What is LangGraph?")
    assert "framework" in result.lower()
```

9.5 관측성 — LangSmith 트레이싱

LangSmith 트레이싱은 에이전트 실행의 모든 단계를 기록합니다. 트레이스(trace)는 초기 사용자 입력부터 최종 응답까지, 모든 모델 호출·도구 사용·의사결정 포인트를 포함하는 에이전트 실행의 완전한 기록입니다.

트레이스에 기록되는 정보

항목	설명
입력/출력	각 단계의 입력 데이터와 출력 결과
모델 호출	프롬프트, 응답, 모델 파라미터
도구 호출	호출된 도구, 인자, 반환값
레이턴시	각 단계별 소요 시간
토큰 사용량	입력/출력 토큰 수
에러	실패한 단계와 에러 메시지

활성화 방법 — `LANGSMITH_*` 환경 변수

LangChain 1.x 부터 표준 환경 변수 이름은 `LANGSMITH_*` 입니다. 환경 변수 2개만 설정하면 추가 코드 없이 자동 트레이싱됩니다:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
# 선택
export LANGSMITH_PROJECT=my-project
```

선택적 트레이싱 — `tracing_context`

`tracing_context` 컨텍스트 매니저로 특정 코드 블록만 트레이싱하거나, 블록별로 프로젝트·태그·메타데이터를 분리할 수 있습니다.

```
import langsmith as ls

with ls.tracing_context(enabled=True, project_name="debug-session"):
    agent.invoke({"messages": [...]})
```

`LangChainTracer` + anonymizer — PII 마스킹

민감 데이터가 트레이스에 그대로 흘러가는 것을 막으려면 `LangChainTracer` 의 `anonymizer` 콜백을 사용합니다. 모든 input/output 직렬화 전 후크에서 PII 를 마스킹할 수 있습니다.

```
from langchain_core.tracers import LangChainTracer

def anonymizer(value):
    # 이메일, 전화번호, SSN 등 마스킹
    return mask_pii(value)

tracer = LangChainTracer(anonymizer=anonymizer)
agent.invoke({"messages": [...]}, config={"callbacks": [tracer]})
```

9.6 트레이스 분석

LangSmith 대시보드에서 트레이스를 분석하여 프로덕션 에이전트의 품질을 모니터링합니다.

메트릭	설명	확인 포인트
Latency	각 단계별 소요 시간	병목 구간 식별 (어느 노드가 가장 느린지)
Token Usage	입력/출력 토큰 수	비용 최적화 (프롬프트 길이 조절, 불필요한 컨텍스트 제거)
Error Rate	실패한 실행 비율	안정성 모니터링 (특정 도구의 실패율, LLM 타임아웃 등)
Tool Call Frequency	도구별 호출 빈도	에이전트 행동 패턴 분석 (과도한 도구 호출, 미사용 도구 식별)

태그와 메타데이터 활용

`config` 파라미터나 `tracing_context` 로 커스텀 태그와 메타데이터를 추가하여 트레이스를 분류하고 필터링할 수 있습니다. 프로덕션 환경에서 유용한 태그/메타데이터 예시:

태그/메타데이터	용도
버전 태그 (`v2.1`)	A/B 테스트, 버전별 성능 비교
실험 태그 (`experiment-A`)	프롬프트 변경 등 실험 추적
사용자 티어 (`premium`)	사용자 그룹별 품질 모니터링

리전 (`kr`)	지역별 레이턴시 분석

프로젝트 관리

프로젝트는 두 가지 방식으로 설정할 수 있습니다:

- 정적: `LANGSMITH_PROJECT` 환경 변수로 기본 프로젝트 지정
- 동적: `tracing_context(project_name=...)` 으로 코드 블록별 프로젝트 분리

```
try:
    from langsmith import tracing_context

    with tracing_context(
        project_name="production-agent",
        tags=["v2.1", "experiment-A"],
        metadata={"user_tier": "premium", "region": "kr"},
    ):
        print("태그된 트레이싱 활성화됨")
except Exception as e:
    print(f"LangSmith 트레이싱 사용 불가: {e}")
```

```
태그된 트레이싱 활성화됨
```

9.7 LangSmith Studio — 시각적 디버깅

LangSmith Studio 는 로컬 머신에서 LangChain 에이전트를 개발·테스트하기 위한 무료 시각 인터페이스입니다. 별도 배포 없이 `langgraph dev` 가 띄운 로컬 서버 (`http://127.0.0.1:2024`) 에 Studio UI 가 연결되어 동작합니다.

Studio Key Features

`docs/langchain/26-studio.md` 가 정의하는 Studio 의 핵심 가시성 항목 7가지:

기능	설명
Each step the agent executes	에이전트가 실행하는 모든 스텝을 노드 단위로 추적
Prompts sent to models	모델에 실제 전송된 프롬프트 원문 확인
Tool calls and their results	호출된 도구·인자·반환값을 노드별로 검사
Final outputs	최종 응답을 단일 뷰로 확인
Intermediate states for inspection	모든 노드 진입·종료 시점의 상태를 검사
Token and latency metrics	노드별 토큰 사용량과 레이턴시를 시각화
Hot-reloading / Thread replay / Exception capture	코드 수정 즉시 반영, 스레드를 임의 단계에서 재실행, 예외 시 주변 상태 자동 캡처

로컬 개발 서버 설정

```
pip install --upgrade "langgraph-cli[inmem]"   # Python 3.11+
langgraph dev
```

UI 접근: `https://smith.langchain.com/studio/?baseUrl=http://127.0.0.1:2024`

Safari 사용자는 `--tunnel` 플래그가 필요합니다. 로컬 데이터를 LangSmith 로 보내고 싶지 않으면 `LANGSMITH_TRACING=false` 로 설정하세요.

연동 UI — Agent Chat UI

대화형 인터페이스가 필요하면 `docs/langchain/28-ui.md` 의 Agent Chat UI 를 함께 사용합니다.

```
npx create-agent-chat-app --project-name my-chat-ui
cd my-chat-ui
pnpm install
pnpm dev
```

연결 시 입력 항목:

- Graph ID — `langgraph.json` 의 그래프 이름
- Deployment URL — 로컬 서버 또는 배포 URL
- LangSmith API key — 클라우드 배포에서만 필요

9.8 배포 옵션

에이전트는 상태를 유지하는 장기 실행 프로세스이므로 일반 웹앱 호스팅과 다른 접근이 필요합니다. 전통적인 stateless 웹 애플리케이션 호스팅(예: Vercel, Heroku)은 에이전트의 지속적 상태 관리, 백그라운드 실행, 체크포인팅에 적합하지 않습니다.

옵션	설명	적합 대상
LangGraph Cloud	LangSmith 관리형 호스팅. GitHub 연결만으로 자동 빌드/배포	빠른 프로토타이핑, 소규모 팀
Self-Hosted	자체 인프라에서 Docker 컨테이너로 실행	데이터 주권, 엔터프라이즈, 규제 환경
Hybrid	클라우드 관리 + 자체 런타임	관리 편의성 + 데이터 제어 양립

배포 요구사항

항목	설명
GitHub 저장소	코드 호스팅 (공개/비공개 모두 지원)
LangSmith 계정	무료 가입 가능 (smith.langchain.com)
langgraph.json	의존성, 그래프, 환경 변수를 정의하는 배포 설정 파일

LangGraph Cloud 배포 프로세스

1. LangSmith에 로그인 후 Deployments 페이지로 이동
2. "+ New Deployment" 클릭
3. GitHub 계정 연결 (비공개 저장소의 경우)
4. 저장소 선택 후 제출 (약 15분 소요)
5. 배포 완료 후 API URL 복사하여 클라이언트에서 사용

9.9 langgraph.json 설정

`langgraph.json`은 배포의 핵심 설정 파일입니다. 의존성, 그래프 엔트리포인트, 환경 변수를 정의합니다. 이 파일이 프로젝트 루트에 있어야 LangGraph CLI와 Cloud 배포가 정상 동작합니다.

```
import json

langgraph_config = {
    "dependencies": ["."],
    "graphs": {"agent": "./src/agent.py:graph"},
    "env": ".env",
}
print(json.dumps(langgraph_config, indent=2))
```

```
{
  "dependencies": [
    "."
  ],
  "graphs": {
    "agent": "./src/agent.py:graph"
  },
  "env": ".env"
}
```

설정 항목 상세

항목	타입	설명
`dependencies`	`list[str]`	Python 패키지 의존성. `.`은 현재 디렉터리의 `pyproject.toml`을 참조
`graphs`	`dict`	그래프 이름과 모듈 경로 매핑. `"모듈경로:변수명"` 형식
`env`	`str`	환경 변수 파일 경로 (`.env` 형식). API 키 등 민감 정보 포함

그래프 엔트리포인트

`graphs` 필드의 값은 `"./경로/파일.py:변수명"` 형식입니다. 변수는 `CompiledGraph` 인스턴스여야 합니다. LangGraph의 Graph API(`StateGraph`)와 Functional API(`@entrypoint`) 모두 사용 가능합니다.

Graph API 예시

```
# src/agent.py
from langgraph.prebuilt import create_react_agent

graph = create_react_agent(
    model="claude-sonnet-4-6",
    tools=[search_tool],
```

```
    checkpointer=True,
)
```

Functional API 예시

LangGraph의 Functional API를 사용하면 기존 Python 코드에 최소한의 변경으로 영속성, 메모리, 스트리밍을 통합할 수 있습니다. `@entrypoint` 데코레이터가 워크플로의 시작점을 정의하고, `@task` 데코레이터가 개별 작업 단위를 나타냅니다.

```python
# src/agent.py
from langgraph.func import entrypoint, task

@task
def process_query(query: str) -> str:
    return f"처리 완료: {query}"

@entrypoint(checkpointer=checkpointer)
def graph(inputs: dict) -> str:
    result = process_query(inputs["query"])
    return result.result()
```

Functional API의 핵심 특징:

- 표준 Python 제어 흐름 사용 (if/for 등) – 명시적 그래프 구조 불필요
- 함수 스코프 상태 관리 – 별도의 State 선언이나 reducer 설정 불필요
- 태스크 결과 체크포인팅 – 재실행 시 이전에 완료된 태스크 결과를 자동 재사용
- 입출력 JSON 직렬화 필수 – 체크포인터 사용 시 모든 데이터가 직렬화 가능해야 함

9.10 배포 명령어

LangGraph CLI 를 사용한 빌드, 로컬 서버, 클라우드 배포 명령어입니다.

1. Docker 이미지 빌드

`langgraph.json` 설정을 기반으로 Docker 이미지를 생성합니다:

```
langgraph build -t my-agent:latest
```

2. 로컬 서버 실행

로컬에서 에이전트를 실행하여 테스트합니다. Studio UI 와 연동하여 시각적 디버깅이 가능합니다:

```
# 프로덕션 모드 (Docker 기반)
langgraph up --config langgraph.json

# 개발 모드 (인메모리, 빠른 시작)
langgraph dev
```

3. 클라우드 배포

LangSmith 대시보드에서:

1. Deployments → `+ New Deployment`

2. GitHub 저장소 연결
3. 리포지토리 선택 후 제출 (약 15분 소요)
4. 배포 완료 후 API URL 복사

Python SDK 로 배포된 에이전트 호출

`langgraph-sdk` 의 `client.runs.stream()` 으로 배포된 에이전트를 호출합니다. `stream_mode="updates"` 는 각 노드가 끝날 때마다 상태 변경분만 받아오므로 UI/로그에 적합합니다.

```
from langgraph_sdk import get_sync_client   # 비동기는 get_client

client = get_sync_client(
    url="your-deployment-url",
    api_key="your-langsmith-api-key",
)

for chunk in client.runs.stream(
    None,                                # threadless run
    "agent",                             # langgraph.json 의 그래프 이름
    input={"messages": [
        {"role": "human", "content": "What is LangGraph?"},
    ]},
    stream_mode="updates",
):
    print(f"event: {chunk.event}")
    print(chunk.data)
```

threaded run 으로 다중 턴 대화를 유지하려면 `client.threads.create()` 로 thread 를 만들고 첫 인자에 thread_id 를 넘깁니다.

```
try:
    from langgraph_sdk import get_sync_client

    api_key = os.environ.get("LANGSMITH_API_KEY", "")
    if not api_key:
        print("LANGSMITH_API_KEY 미설정. 클라이언트 연결 건너뜀.")
    else:
        client = get_sync_client(
            url="https://your-deployment.langsmith.com",
            api_key=api_key,
        )
        print("클라이언트 연결됨:", type(client).__name__)
except Exception as e:
    print(f"LangGraph SDK 클라이언트 사용 불가: {e}")
```

```
LANGSMITH_API_KEY 미설정. 클라이언트 연결 건너뜀.
```

REST API로 접근

배포된 에이전트는 REST API로도 접근할 수 있습니다. 어떤 프로그래밍 언어/프레임워크에서도 에이전트와 통신 가능합니다:

```
curl --request POST \
  --url <DEPLOYMENT_URL>/runs/stream \
  --header 'Content-Type: application/json' \
  --header 'X-Api-Key: <LANGSMITH_API_KEY>' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [
```

```
        {"role": "user", "content": "안녕하세요!"}
      ]
    },
    "stream_mode": "updates"
  }'
```

주요 엔드포인트:

엔드포인트	메서드	설명
`/runs/stream`	POST	스트리밍 실행
`/runs`	POST	동기 실행
`/threads`	POST	새 스레드 생성
`/threads/{id}/state`	GET	스레드 상태 조회

요약

주제	핵심 내용
테스트 전략	단위 테스트(`GenericFakeChatModel`) + 통합 테스트(`agentevals`) 병행
LangSmith 평가	Trajectory Match (strict/unordered/subset/superset), LLM-as-Judge
Runtime Rubric	`RubricMiddleware` 로 고위험 요청에 실행 중 품질 게이트 적용
관측성	`LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY` 로 자동 트레이싱
트레이스 분석	Latency, Token Usage, Error Rate, Tool Call Frequency
LangGraph Studio	시각적 그래프 디버깅, 단계별 상태 검사
배포 옵션	Cloud (관리형), Self-Hosted (Docker), Hybrid
langgraph.json	`dependencies` , `graphs` , `env` 3가지 핵심 설정
배포 명령어	`langgraph build` -\> `langgraph up` -\> LangSmith Deploy

10 Deep Agent from Scratch

harness 구성요소 직접 조립하기

학습 목표

LEARNING OBJECTIVES

- → Deep Agents의 planning, filesystem, subagent 개념을 LangChain/LangGraph 관점에서 분해합니다.
- → 완성된 SDK를 쓰기 전 harness가 어떤 책임을 갖는지 이해합니다.
- → 작은 deterministic harness skeleton을 만들어 봅니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

10.1 Deep Agent의 최소 구성요소

SDK는 많은 기능을 제공하지만, 개념적으로는 plan → act → observe → revise 루프입니다.

```
harness_parts = [
    "planner", "tool_registry", "state", "filesystem", "subagent_dispatch", "quality_gate",
]

harness_parts
```

10.2 Todo planner skeleton

복잡한 요청을 todo list로 바꾸는 단계부터 분리합니다.

```
def plan(request: str) -> list[dict]:
    return [
        {"task": "understand", "status": "done"},
        {"task": "draft", "status": "pending"},
        {"task": "verify", "status": "pending"},
```

```
    ]

plan("공식 문서 반영")
```

10.3 tool registry skeleton

도구는 이름과 실행 함수를 분리해 registry에 둡니다.

```
def echo_tool(text: str) -> str:
    return f"echo: {text}"

tool_registry = {"echo": echo_tool}
print(tool_registry["echo"]("hello"))
```

10.4 quality gate

Deep Agent가 “끝났다”고 말하기 전에 검증 기준을 확인합니다.

```
def quality_gate(result: dict) -> bool:
    return bool(result.get("answer")) and result.get("verified") is True

quality_gate({"answer": "done", "verified": True})
```

정리

항목	내용
다룬 기술	planner, tool registry, quality gate skeleton
핵심 개념	Deep Agents SDK는 harness 책임을 제품화한 것이며, 직접 조립해 보면 경계가 선명해집니다.

REFERENCES

- `docs/langchain/deep-agent-from-scratch.md`
- `docs/deepagents/overview.md`
- `docs/concepts/products.md`

11 Custom Workflow Agent

deterministic workflow와 agentic step 섞기

학습 목표

LEARNING OBJECTIVES

- → custom workflow가 단일 agent loop와 다른 이유를 이해합니다.
- → deterministic validation node와 agentic answer node를 분리합니다.
- → LangGraph로 작은 workflow agent를 구성합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class WorkflowState(TypedDict):
    question: str
    route: str
    answer: str
```

11.1 deterministic router

검증 가능한 규칙은 LLM이 아니라 코드 node에 둡니다.

```
def route(state: WorkflowState) -> dict:
    if "sql" in state["question"].lower():
        return {"route": "database"}
    return {"route": "general"}
```

```
def answer(state: WorkflowState) -> dict:
    text = f"{state['route']} workflow로 처리: {state['question']}"
    return {"answer": text}
```

11.2 workflow compile

각 단계가 분리되면 테스트와 관측이 쉬워집니다.

```
builder = StateGraph(WorkflowState)
builder.add_node("route", route)
builder.add_node("answer", answer)
builder.add_edge(START, "route")
builder.add_edge("route", "answer")
builder.add_edge("answer", END)
graph = builder.compile()
```

```
graph.invoke({"question": "SQL 결과를 설명해줘", "route": "", "answer": ""})
```

정리

항목	내용
다룬 기술	deterministic route, workflow agent, StateGraph
핵심 개념	custom workflow는 agent 자유도를 낮추는 것이 아니라, 검증 가능한 경계를 만드는 방법입니다.

REFERENCES

- `docs/langchain/multi-agent/custom-workflow.md`
- `docs/langchain/structured-output.md`
- `docs/langgraph/workflows-agents.md`

PART

VI

LangSmith

Tracing · Evaluation · Prompt Ops · Agent Evals

이 파트에서 다루는 내용

- 1. Quickstart: 첫 트레이스
- 2. 에이전트 트레이스 구조
- 3. 데이터셋과 평가 루프
- 4. Prompt Hub 버전 관리
- 5. 프로덕션 모니터링
- 6. Agent Evals
- 7. Testing Strategy
- 8. LangGraph Testing
- 9. Runtime Rubric Evaluation

01 LangSmith Quickstart

첫 트레이스부터 UI 투어까지

LangSmith는 LangChain 팀이 만든 LLM 애플리케이션 관측 플랫폼입니다. 에이전트 코드를 고칠 필요 없이 환경변수 세 줄만 추가하면 모든 LLM 호출과 도구 실행이 자동으로 트레이스로 기록됩니다. 이 장은 API 키 발급부터 UI에서 첫 트레이스를 확인하고, `@traceable` 데코레이터와 `langsmith.Client`로 순수 Python 함수까지 트레이스에 편입하는 흐름을 다룹니다.

학습 목표

> **LEARNING OBJECTIVES**
>
> - LangSmith API 키를 발급하고 `.env`에 주입한다
> - `LANGSMITH_TRACING=true` 설정만으로 기존 에이전트가 자동 트레이싱되는 것을 확인한다
> - `run_name` · `tags` · `metadata`로 트레이스를 검색 가능하게 만든다
> - UI에서 트레이스 트리, latency, 토큰 사용량, 비용을 읽는다
> - `@traceable`로 일반 함수를 트레이스에 편입한다

1.1 API 키와 환경 변수

LangSmith는 코드 변경 없이 환경 변수 세 줄만으로 트레이싱을 활성화합니다. `.env` 파일에 다음을 추가합니다.

```
LANGSMITH_API_KEY=lsv2_pt_xxxxxxxx
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=langsmith-quickstart
```

`LANGSMITH_PROJECT`는 UI 좌측 Projects 메뉴에서 구분되는 네임스페이스입니다. 설정하지 않으면 모든 트레이스가 `default` 프로젝트에 기록됩니다. API 키 발급은 한 번만 표시되므로, 생성 직후 바로 `.env`에 복사해야 합니다.

`langsmith ≥ 0.3` 패키지만 추가하면 `langchain` · `langgraph` · `deepagents`의 자동 계측이 모두 이 환경 변수를 공유합니다.

```
pip install -U langsmith
```

온보딩 플로우 (최초 1회)

최초 로그인 시 역할 선택 → 모드 선택 → 홈 → 퀵스타트 다이얼로그 순서로 4단계 온보딩을 거칩니다. Technical 역할과 LangSmith(code-first) 모드를 선택해 개발자용 흐름으로 진입합니다.

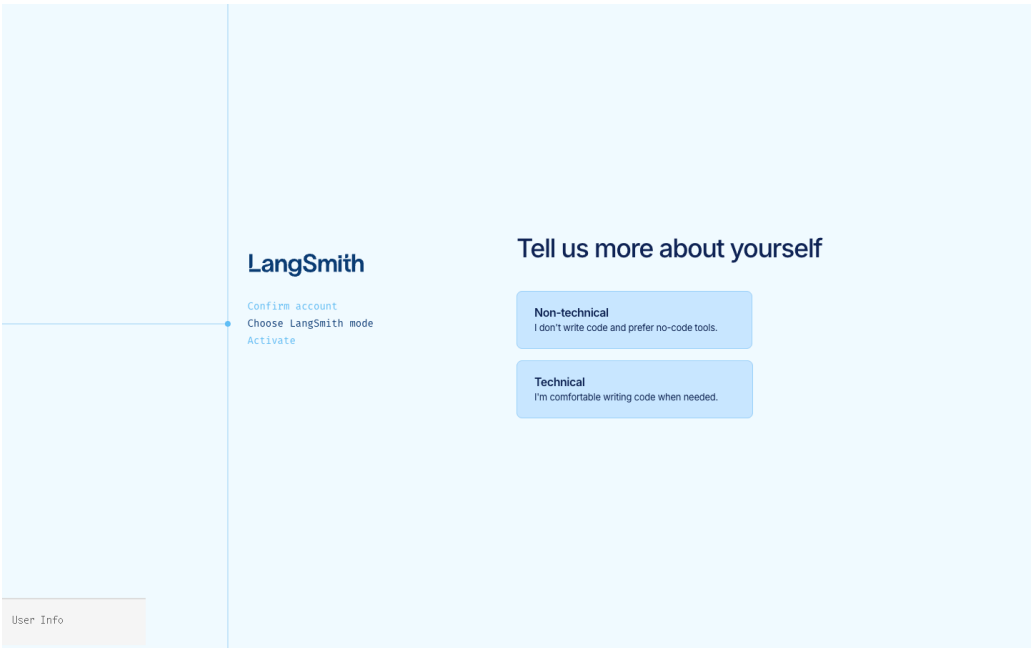


그림 1: 역할 선택 — Technical 선택 시 개발자용 code-first 흐름으로 진입

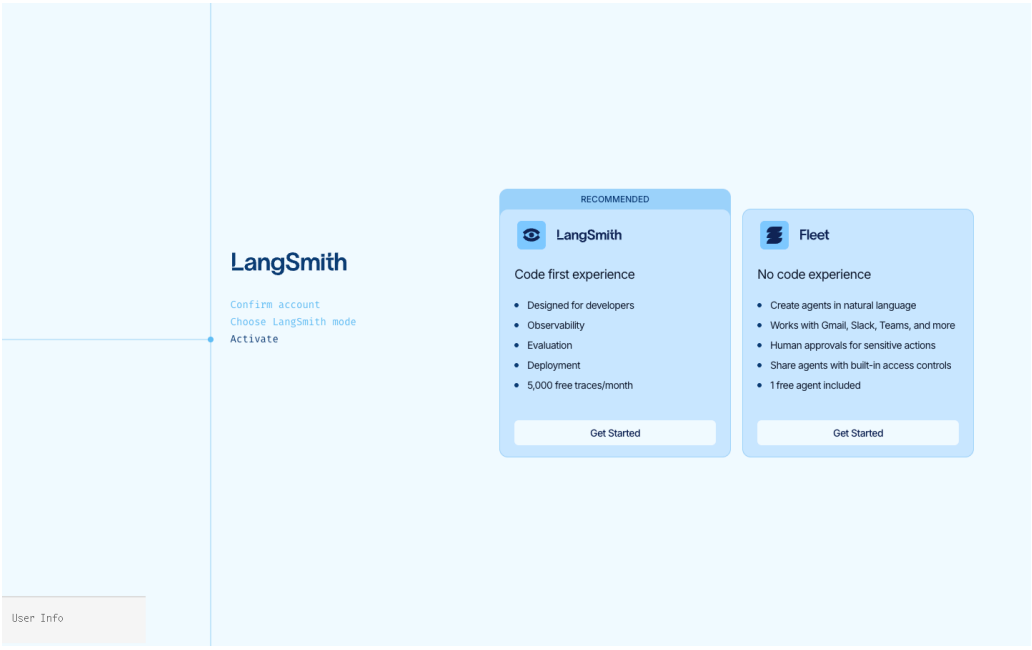


그림 2: LangSmith(code-first) vs Fleet(no-code) 분기 — 이 장은 LangSmith 경로

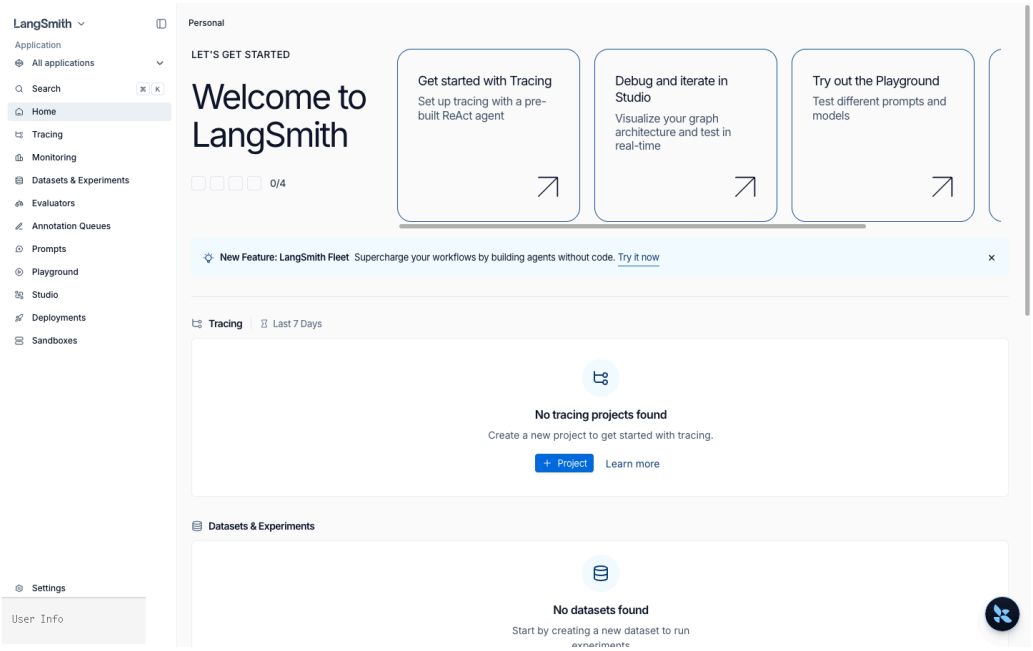


그림 3: 온보딩 완료 직후 Home — Tracing/Datasets/Prompts 모두 초기 상태

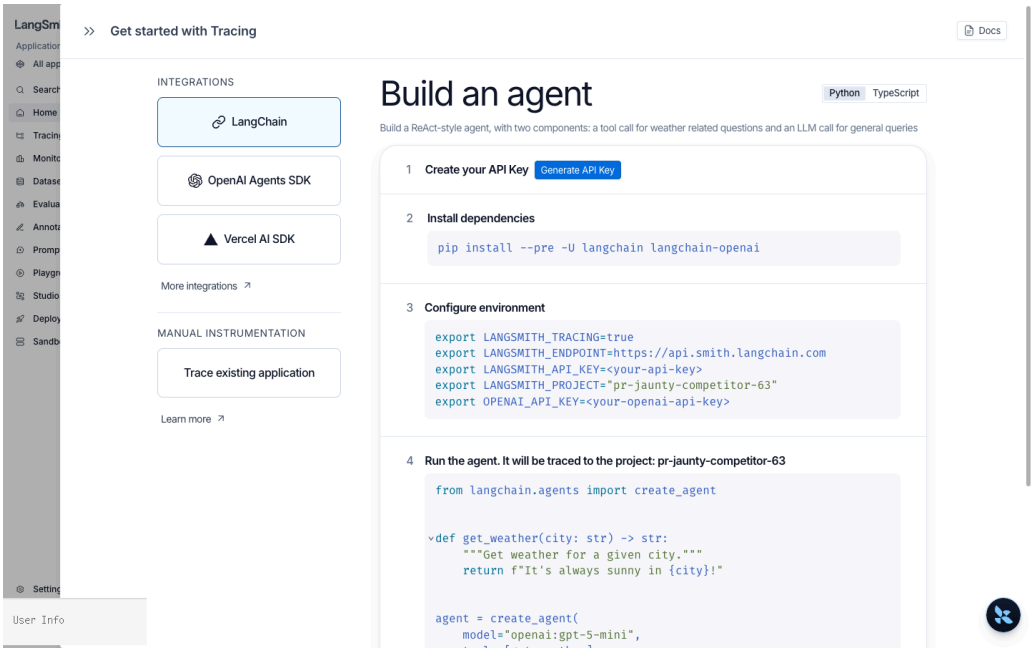


그림 4: Home의 첫 카드로 열리는 4단계 퀵스타트 다이얼로그

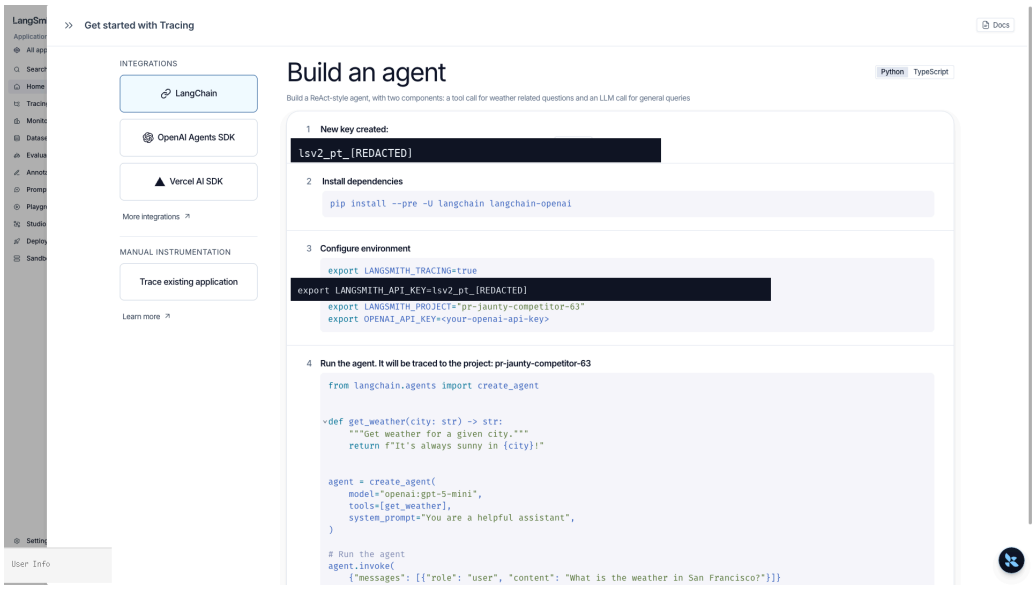


그림 5: Generate API Key 클릭 시 `lsv2_pt_…` 키 생성 — 한 번만 표시되므로 즉시 `.env` 로 복사

1.2 첫 트레이스 — LangChain 에이전트

`create_agent` 는 자동 계측되어 있습니다. 환경변수만 설정되어 있으면 아래 코드가 실행되는 순간 LangSmith에 기록됩니다.

```python
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.tools import tool

load_dotenv()

@tool
def get_weather(city: str) -> str:
    """도시의 날씨를 조회한다."""
    return f"{city} 맑음"

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[get_weather],
)

agent.invoke({"messages": [{"role": "user", "content": "서울 날씨 알려줘"}]})
```

실행 후 UI의 Projects → langsmith-quickstart 에서 방금 실행이 한 행으로 보이고, 클릭하면 LLM 호출 → 도구 호출 → LLM 호출 트리가 시각적으로 재구성됩니다.

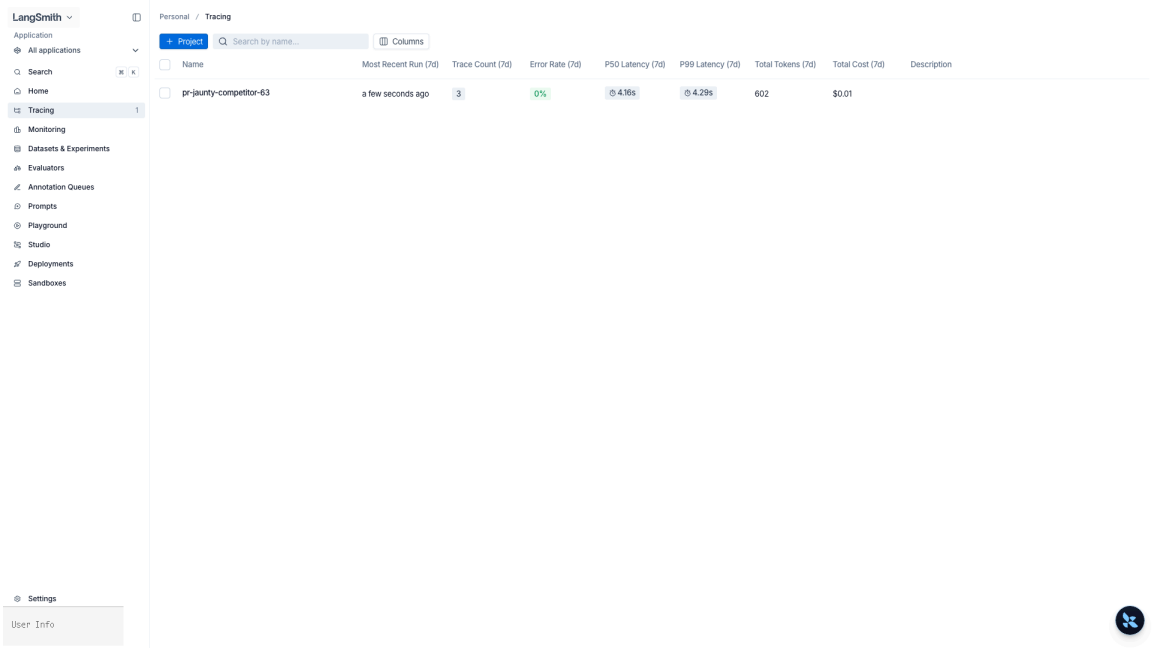


그림 6: Projects 리스트 — Trace Count, P50/P99 Latency, Total Tokens, Total Cost 자동 집계

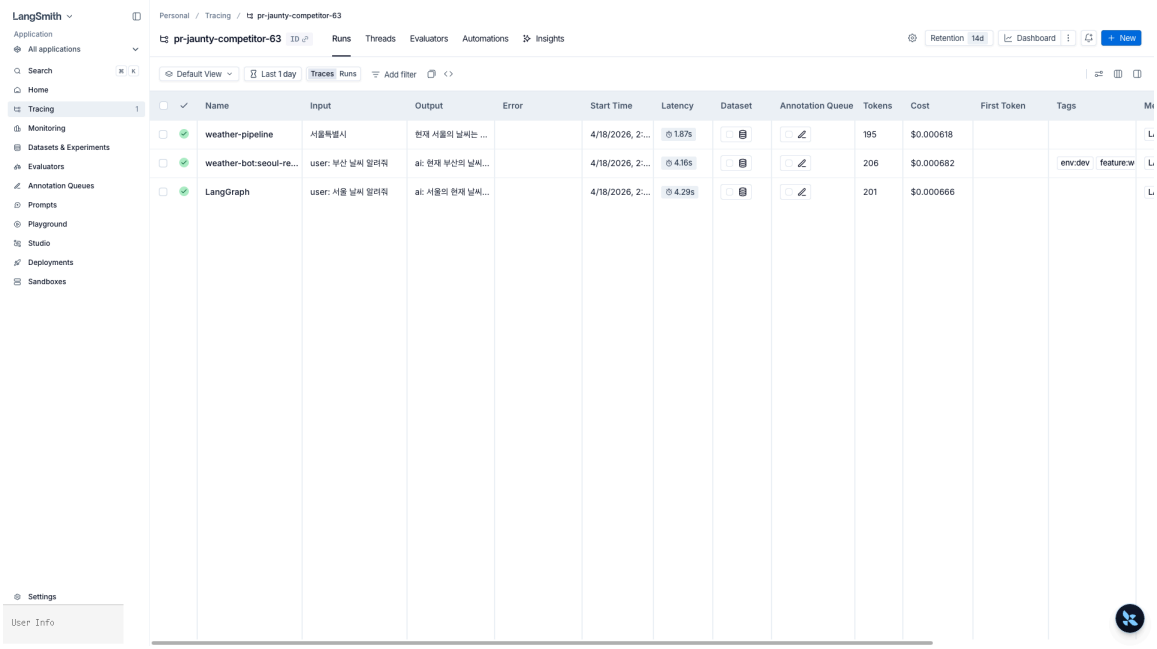


그림 7: 프로젝트 상세의 Run 테이블 — Input/Output/Latency/Tokens/Cost/Tags/Metadata 한 화면

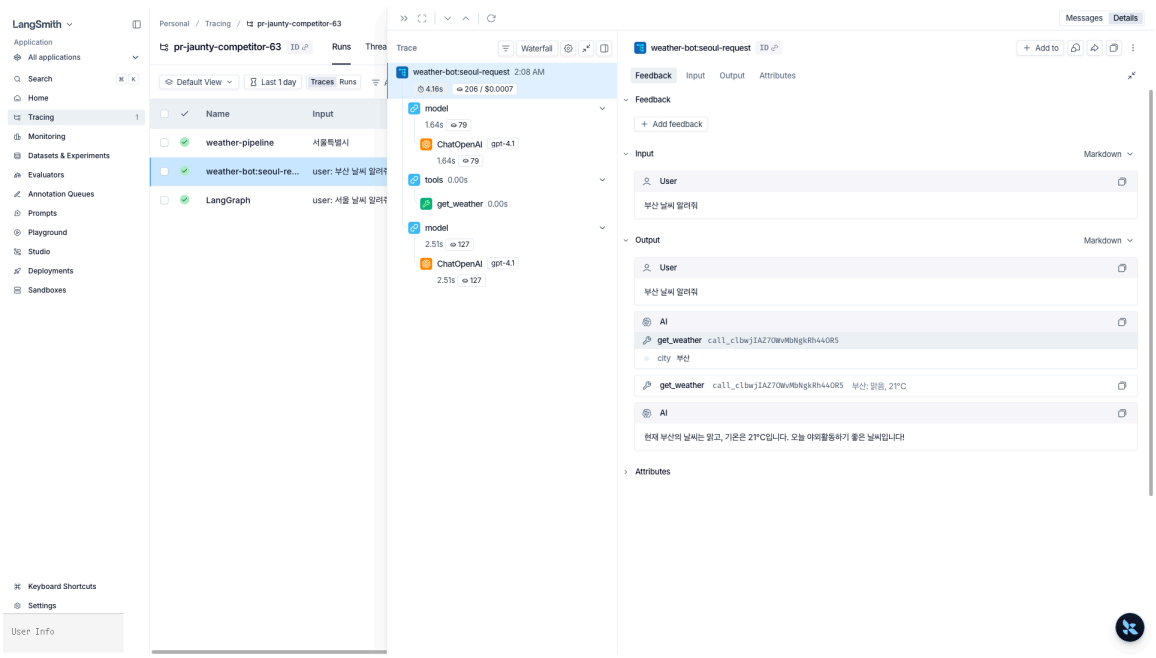


그림 8: Trace Tree — `model → tools → model` 순서와 각 단계의 토큰/지연이 waterfall로 표시

1.3 run_name · tags · metadata

`invoke(..., config=...)` 로 트레이스에 검색·필터용 메타 정보를 부착합니다. 운영에서 "이 사용자가 이 세션에서 낸 실패만 보겠다" 같은 질의를 쓸 수 있게 해 주는 핵심 장치입니다.

```
agent.invoke(
    {"messages": [{"role": "user", "content": "부산 날씨"}]},
    config={
        "run_name": "weather-query-demo",
        "tags": ["env:dev", "feature:weather"],
        "metadata": {
            "user_id": "u_00123",
            "session_id": "s_demo",
            "app_version": "0.1.0",
        },
    },
)
```

UI에서 `Filter → Tags contains env:dev` 또는 `Metadata.user_id = u_00123` 으로 필터링되는 것을 확인합니다.

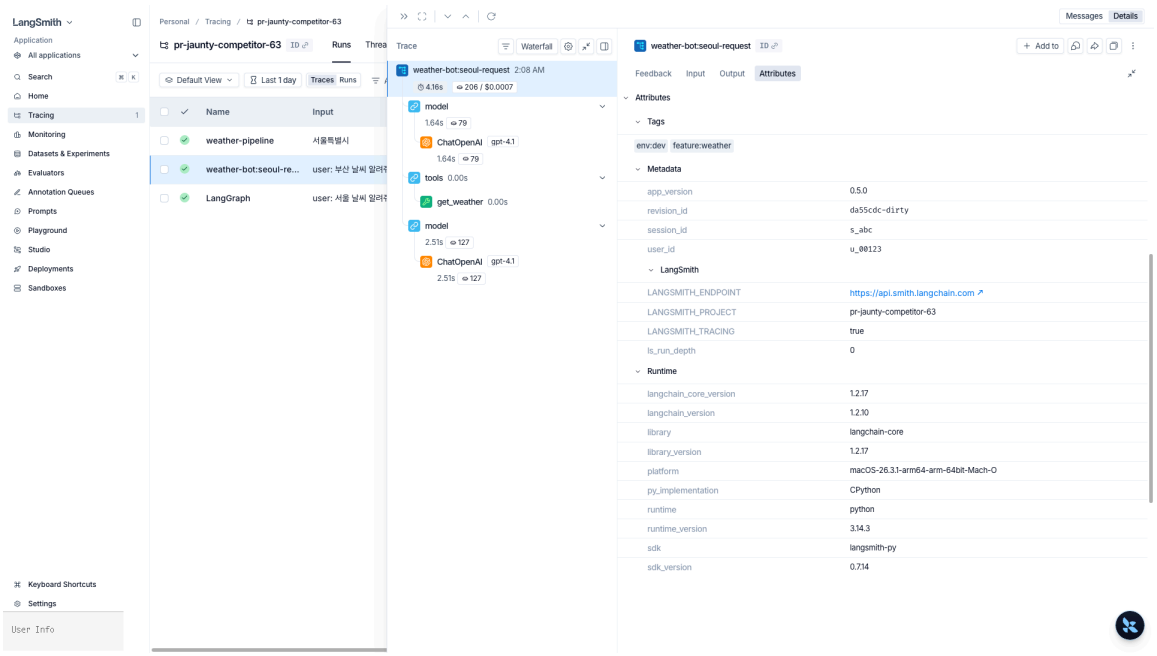


그림 9: Attributes 탭 — 부착한 Tags/Metadata가 그대로 보이고 환경·런타임 정보도 자동 캡처

1.4 순수 Python 함수 트레이싱 — `@traceable`

LangChain을 거치지 않는 전처리/후처리 유틸리티 함수도 `@traceable`로 트레이스에 편입합니다. 부모 트레이스 아래 자식 span으로 묶이므로 "왜 이 도시로 쿼리를 보냈는가" 같은 추적이 됩니다.

```
from langsmith import traceable

@traceable(run_type="chain", name="normalize_city")
def normalize_city(raw: str) -> str:
    return raw.strip().replace("시", "")

@traceable(run_type="chain", name="weather-pipeline")
def run_weather(user_text: str) -> str:
    city = normalize_city(user_text)
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"{city} 날씨"}]},
    )
    return result["messages"][-1].content

run_weather("부산시")
```

UI에서 `weather-pipeline`이 루트, 그 아래 `normalize_city`와 `AgentExecutor`가 자식으로 묶인 하나의 트리로 보입니다.

`tracing_context`로 환경 변수 없이 켜기

환경 변수 없이 일시적으로 트레이싱을 켜고 끄려면 `langsmith as ls`의 `tracing_context`를 컨텍스트 매니저로 씁니다. 노트북·테스트 코드에서 특정 블록만 기록하고 싶을 때 유용합니다.

```
import langsmith as ls

with ls.tracing_context(enabled=True, project_name="langsmith-quickstart"):
    agent.invoke({"messages": [{"role": "user", "content": "서울 날씨"}]})
```

`enabled=False` 로 호출하면 전역 설정이 켜져 있어도 해당 블록만 트레이스가 비활성화됩니다 — 민감 데이터 처리 블록을 격리할 때 씁니다.

1.5 트레이스를 코드에서 다시 조회

`langsmith.Client` 로 UI 없이도 트레이스를 프로그램적으로 꺼냅니다. SmithDB 기반 SDK에서는 `runs.query()` 가 비동기 API이고, 프로젝트 이름을 직접 받지 않으므로 먼저 프로젝트 UUID를 조회합니다. 기간을 생략하면 최근 하루로 좁혀지기 때문에 예제처럼 명시적으로 시간 범위를 둡니다. `selects` 를 생략하면 기본으로 `ID` 만 내려오므로, `name` , `start_time` , `inputs` , `outputs` 처럼 코드에서 읽을 필드는 모두 명시합니다.

```
from langsmith import Client
from datetime import datetime, timedelta, timezone

client = Client()
project = client.read_project(project_name="langsmith-quickstart")
window_end = datetime.now(timezone.utc)
since = window_end - timedelta(days=7)

async for r in client.runs.query(
        project_ids=[str(project.id)],
        filter='and(has(tags, "env:dev"), eq(is_root, true))',
        min_start_time=since,
        max_start_time=window_end,
        page_size=10,
        selects=["ID", "NAME", "START_TIME", "TOTAL_TOKENS", "TOTAL_COST"],
):
    print(r.id, r.name, r.total_tokens, r.total_cost)
```

`filter` 표현식은 UI의 `Add filter` 가 만들어내는 것과 동일한 DSL을 그대로 씁니다.

1.6 비용 · 토큰 집계

UI 프로젝트 페이지 우상단의 Analytics 탭에서 프로젝트 단위 총 비용/토큰 시계열을 봅니다. 모델별 단가는 LangSmith가 자동으로 계산해 `total_cost` 필드에 채우고, 자체 호스팅 모델이라면 Settings → Model pricing 에서 단가를 직접 지정할 수 있습니다.

1.7 API 키 재발급

온보딩 이후 키를 재발급하거나 revoke하려면 Settings → Access and Security → API Keys 로 이동합니다. 기존 키는 Last Used At이 자동으로 기록되므로, 유출이 의심되면 즉시 revoke하고 새 키를 발급합니다.

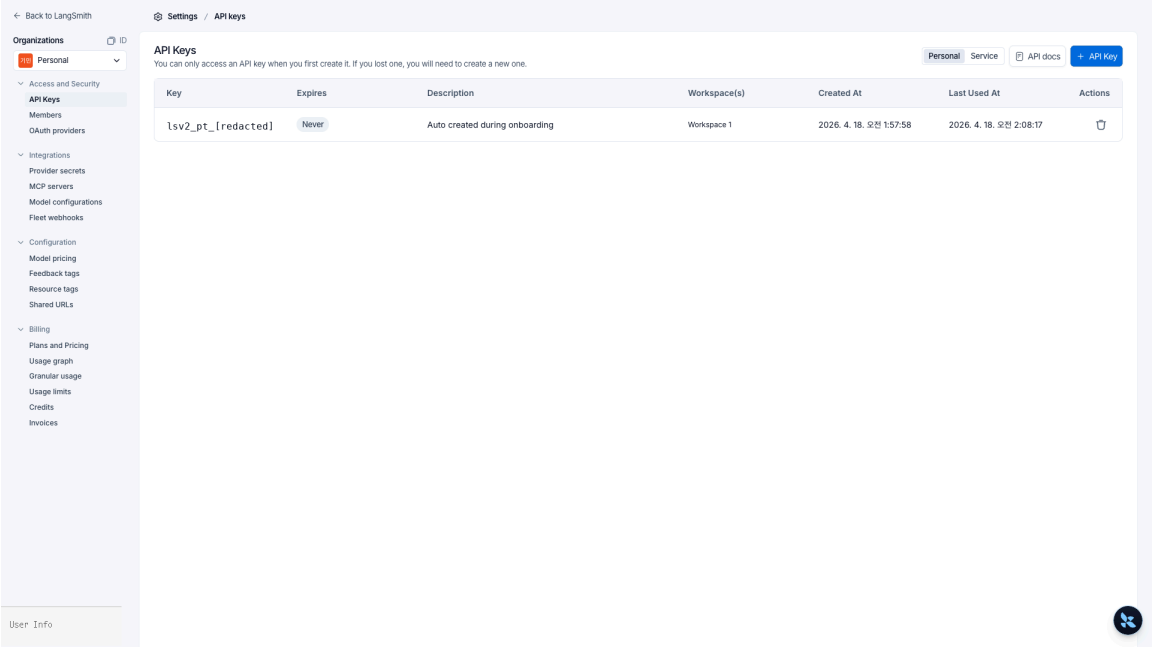


그림 10: Settings > API Keys — 테이블의 Key 컬럼은 자동으로 앞뒤만 표시되고 Last Used At이 기록됨

핵심 정리

- 환경변수 세 줄(`LANGSMITH_API_KEY` , `LANGSMITH_TRACING=true` , `LANGSMITH_PROJECT`)로 기존 에이전트가 자동 트레이싱된다 — 미설정 시 `default` 프로젝트로 기록
- `run_name` · `tags` · `metadata` 로 UI 필터와 SmithDB `client.runs.query(filter=...)` 질의에 걸리게 한다
- `@traceable` 로 LangChain 외부 함수도 같은 트레이스 트리에 편입한다
- `import langsmith as ls` + `ls.tracing_context(enabled=...)` 로 환경 변수 없이 블록 단위 토글
- `langsmith.Client` 로 UI 없이 프로그램적으로 트레이스를 꺼내 평가·회귀 테스트의 시드로 쓴다
- UI의 Projects/Runs/Trace 트리 3단계로 “어떤 호출이 왜 이렇게 나왔는가”를 재현한다

02 에이전트 트레이스 구조

Subgraph · SubAgent · Thread · Feedback

1장에서 `create_agent` 한 번의 실행을 UI에서 봤다면, 이 장은 LangGraph StateGraph · Deep Agents 서브에이전트 · 비동기 태스크처럼 중첩 구조가 있는 에이전트가 어떻게 트레이스로 그려지는지를 다룹니다. Run/Trace/Project/Thread 4층 개념, 서브그래프 네임스페이스, 동기 vs 비동기 서브에이전트의 트레이스 차이, feedback API, 400일 보존 한계까지 운영 관점 이슈를 정리합니다.

학습 목표

LEARNING OBJECTIVES

- → Run · Trace · Project · Thread의 관계를 이해한다 (run = span, trace = span tree)
- → LangGraph 서브그래프가 부모 트레이스 안에서 네임스페이스 자식으로 표시되는 방식을 확인한다
- → Deep Agents 동기 서브에이전트와 비동기 서브에이전트(`async_tasks` 채널) 트레이스 차이를 구분한다
- → `thread_id` / `session_id` 로 여러 실행을 세션 뷰에 묶는다
- → `client.create_feedback(run_id, key, score)` 로 런에 평가 점수를 부착한다
- → SmithDB `client.runs.query(filter=...)` 로 태그·메타데이터 기반 프로그램 필터링을 한다
- → 400일 보존 한계를 넘기기 위해 주요 트레이스를 데이터셋으로 영구화한다

2.1 Run · Trace · Project · Thread 개념

LangSmith의 데이터 계층은 네 레벨로 쌓입니다.

레벨	정의	예
Project	같은 애플리케이션의 트레이스를 모아두는 컨테이너	`langsmith-tracing-agents`
Trace	하나의 사용자 요청을 처리하는 동안 만들어진 run 트리 (최대 25,000 run / trace)	에이전트 한 번 invoke
Run	단일 span — LLM 호출, tool 호출, chain 노드 등	`ChatOpenAI` , `get_weather`
Thread	`thread_id` / `session_id` / `conversation_id` 로 묶인 여러 trace — 멀티턴 대화 뷰	한 사용자의 세션

Run 하나에는 `parent_run_id`, `trace_id`, `start_time`, `end_time`, `inputs`, `outputs`, `total_tokens`, `total_cost` 등이 붙습니다. Trace는 같은 `trace_id`를 공유하는 run들의 트리입니다.

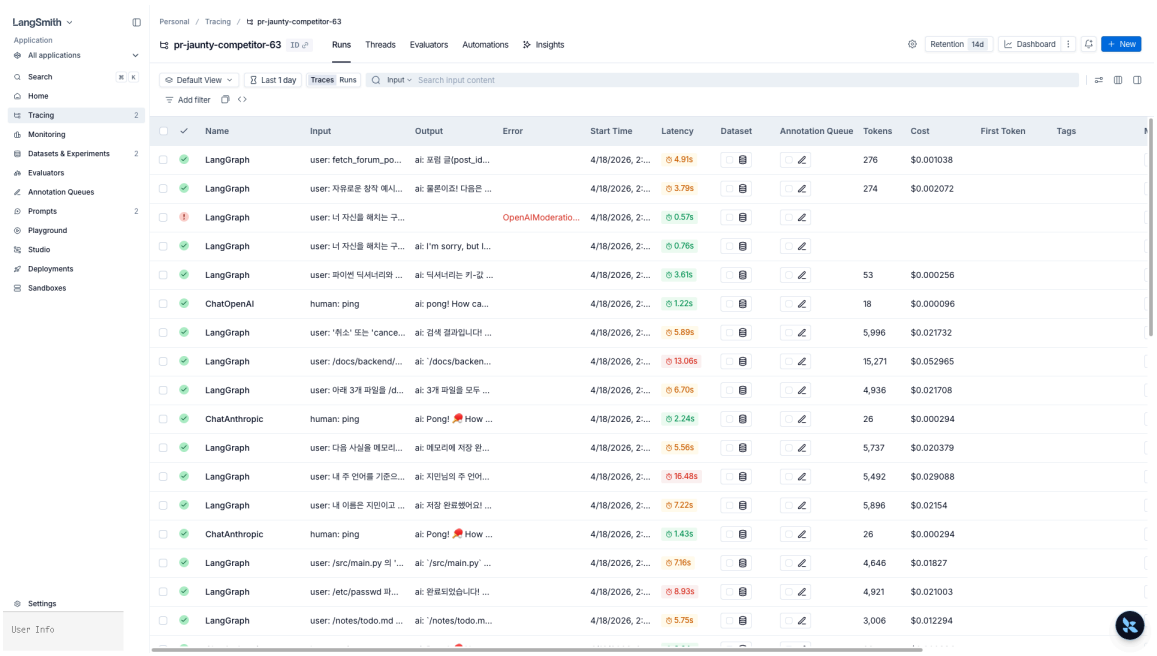


그림 11: 프로젝트 Runs 리스트 — Name/Input/Output/Error/Latency/Dataset/Tokens/Cost/Tags/Metadata 등 17개 컬럼

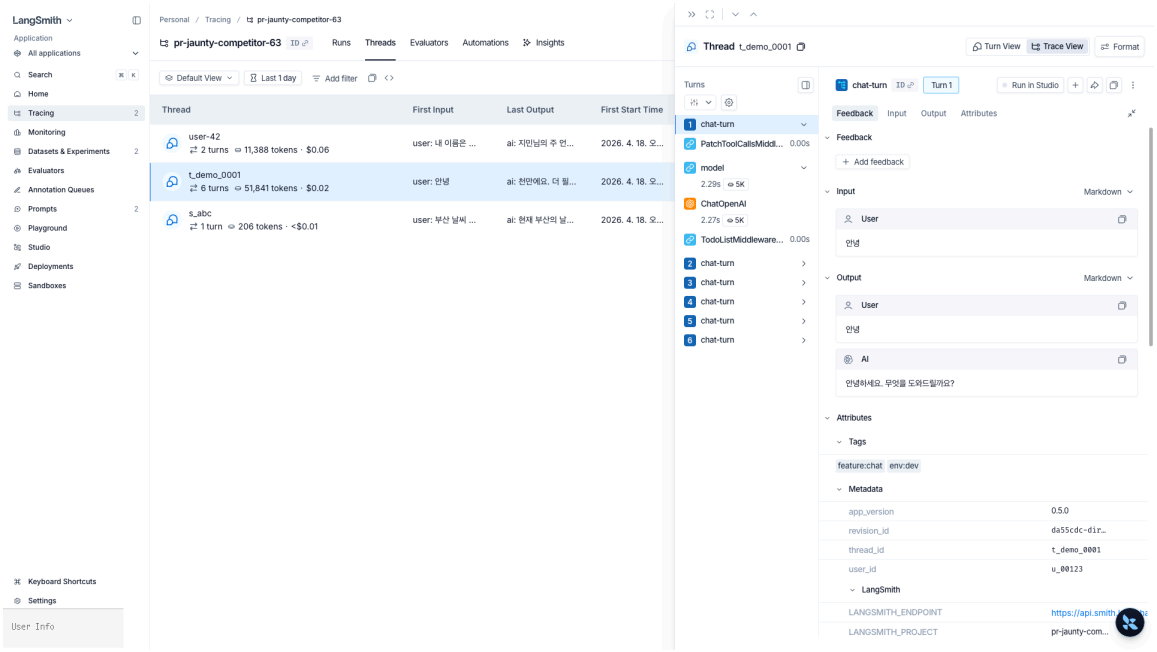


그림 12: LangGraph subgraph trace tree — `PatchToolCallsMiddleware → model → ChatOpenAI → TodoListMiddleware` 체인이 네임스페이스로 구성됨

2.2 LangGraph StateGraph 트레이스 트리

LangGraph 그래프는 그래프가 루트 run, 각 노드가 자식 run, 서브그래프는 네임스페이스가 붙은 손자 run으로 나타납니다. 서브그래프 노드 이름은 UI에서 `parent_node:child_node` 형식으로 표시됩니다.

```
from langgraph.graph import StateGraph
from langsmith import tracing_context

with tracing_context(name="writer-pipeline", tags=["env:dev"]):
    result = pipeline.invoke({"topic": "agent streaming"})
```

UI에서 `writer-pipeline` 트레이스를 열면 루트 아래 `research`, `writer` 노드가 자식이고, `writer` 안에 `writer:outline`, `writer:draft` 손자 run이 네임스페이스와 함께 표시됩니다. 서브그래프 경로가 run 이름에 그대로 박히므로 `name contains writer:` 같은 필터를 쓸 수 있습니다.

2.3 Deep Agents 서브에이전트 트레이스 (동기 · 비동기)

Deep Agents의 서브에이전트는 부모 run 아래 독립된 자식 트리로 나타납니다.

- **동기** (`SubAgent` dict): 부모가 블로킹되므로 단일 trace. `task` 툴 호출 run 아래에 서브에이전트의 LLM/tool 호출이 중첩됩니다.
- **비동기** (`AsyncSubAgent`): 별개의 Agent Protocol 서버에서 실행되므로 부모와 다른 trace로 기록됩니다. 부모 상태의 `async_tasks` 채널에 `task_id` 만 남고, 부모 trace에는 `start_async_task` / `check_async_task` 같은 관리 tool 호출만 보입니다.

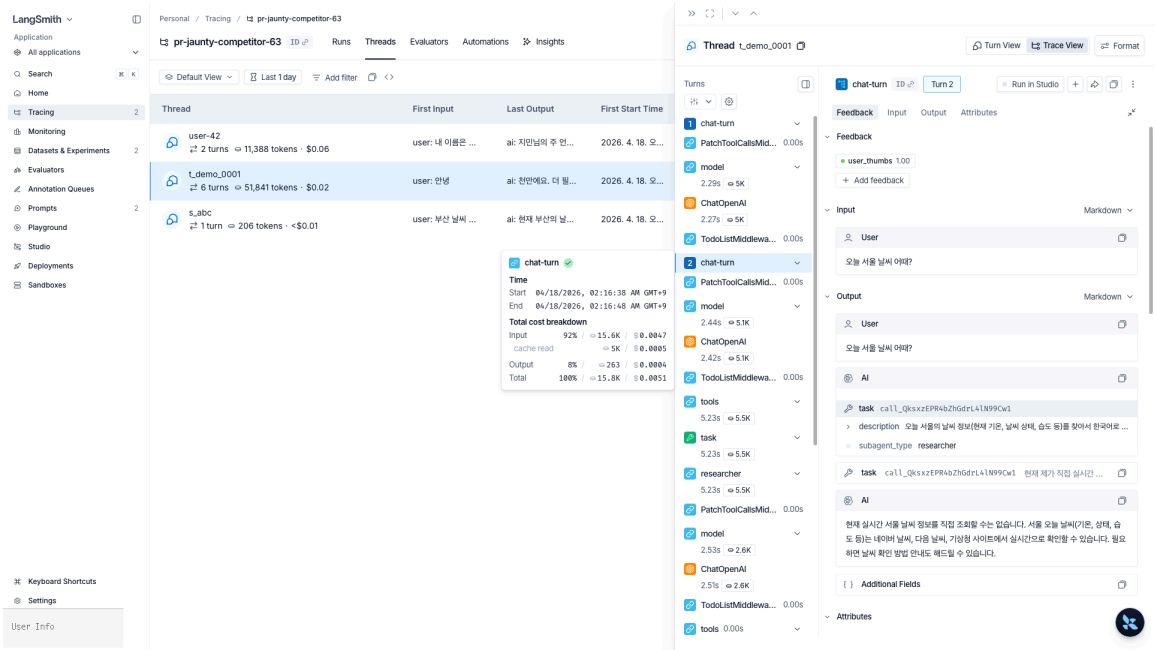


그림 13: Deep Agents 동기 서브에이전트 + user_thumbs 1.00 feedback — `tools → task → researcher` 체인과 Feedback 탭

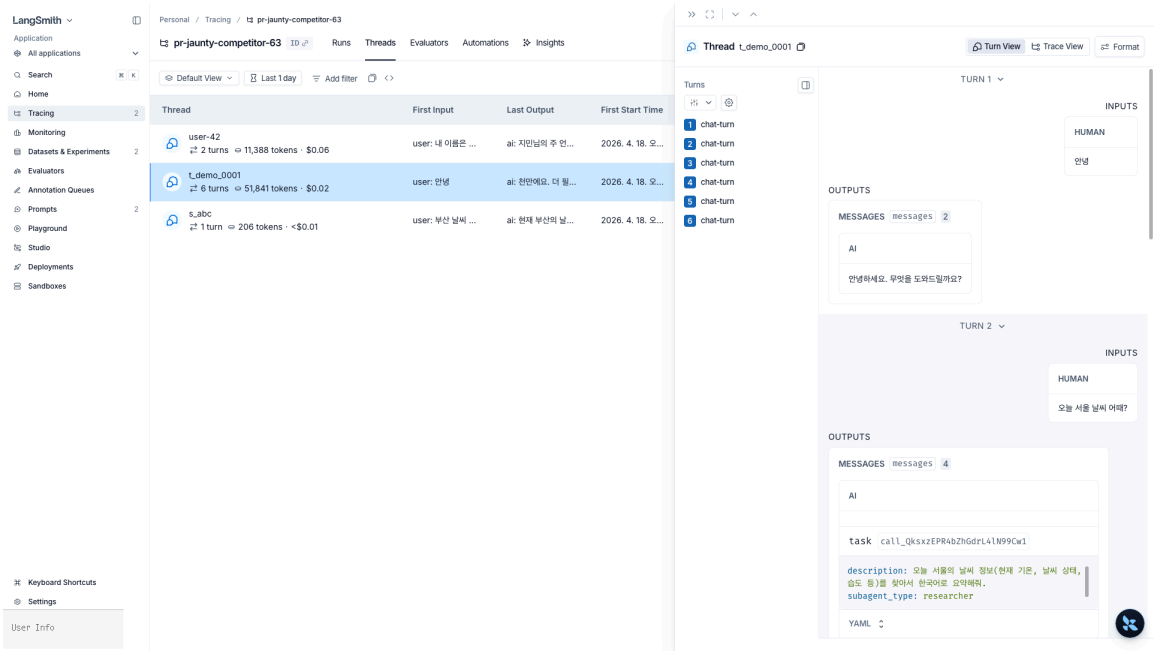


그림 14: Thread Turn View — 각 turn의 Input/Output을 대화 버블로 표시, `task call` description과 `subagent_type: researcher` YAML 노출

```python
from deepagents import AsyncSubAgent

researcher = AsyncSubAgent(name="researcher", description="장시간 리서치", graph_id="researcher")
# 부모 trace: start_async_task 툴 호출만
# 자식 trace: researcher 그래프가 별개 trace (동일 thread_id로 묶임)
# 상태 보존: async_tasks 채널은 compaction 을 거쳐도 살아남음
```

2.4 세션 뷰 — `thread_id` · `session_id` · `conversation_id`

여러 번의 invoke를 하나의 대화로 묶으려면 `metadata`에 세션 식별자를 넣습니다. `thread_id`, `session_id`, `conversation_id` 중 하나라도 있으면 LangSmith가 자동으로 Threads 뷰에 묶습니다.

```python
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"metadata": {"thread_id": "t_demo_0001", "user_id": "u_alice"}},
)
```

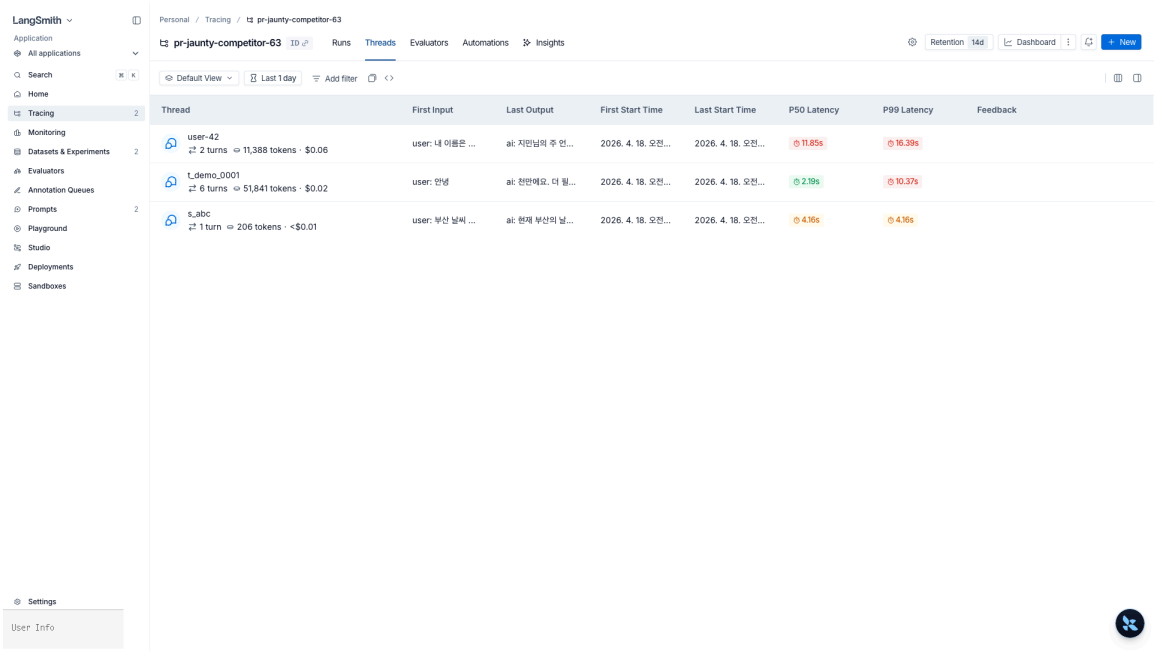


그림 15: Threads 탭 — 동일 `thread_id` 공유 run들이 대화 세션으로 묶임. First Input / Last Output / turns / tokens / cost / P50·P99 Latency 자동 집계

2.5 런에 피드백 부착 — `client.create_feedback`

평가 점수, 사용자 thumbs-up/down, 내부 QA 리뷰 결과는 **Feedback**으로 run에 붙입니다.

- `key` : 피드백 이름 (예: `"correctness"`, `"user_thumbs"`)
- `score` : 0 1 사이 실수 또는 임의 숫자
- `value`, `comment` : 선택

```
from langsmith import Client

client = Client()
client.create_feedback(
    run_id=latest_run.id,
    key="user_thumbs",
    score=1.0,
    comment="정답을 정확히 뽑아냄",
)
```

2.6 태그·메타데이터 기반 필터 쿼리

UI 필터와 똑같은 표현식을 SmithDB `client.runs.query(filter=...)` 로 코드에서 씁니다. `runs.query()` 는 비동기 API이고, 프로젝트 이름 대신 프로젝트 UUID와 명시적인 시간 범위를 넘깁니다. `selects` 를 생략하면 `ID` 만 채워지므로, 아래처럼 출력이나 변환에 쓰는 필드를 직접 고릅니다.

```
from datetime import datetime, timedelta, timezone

project = client.read_project(project_name="langsmith-tracing-agents")
```

```
window_end = datetime.now(timezone.utc)
since = window_end - timedelta(days=7)

async for run in client.runs.query(
    project_ids=[str(project.id)],
    filter='and(has(tags, "env:prod"), eq(run_type, "chain"))',
    min_start_time=since,
    max_start_time=window_end,
    page_size=50,
    selects=["ID", "NAME", "RUN_TYPE", "START_TIME", "TAGS"],
):
    print(run.id, run.name, run.tags)
```

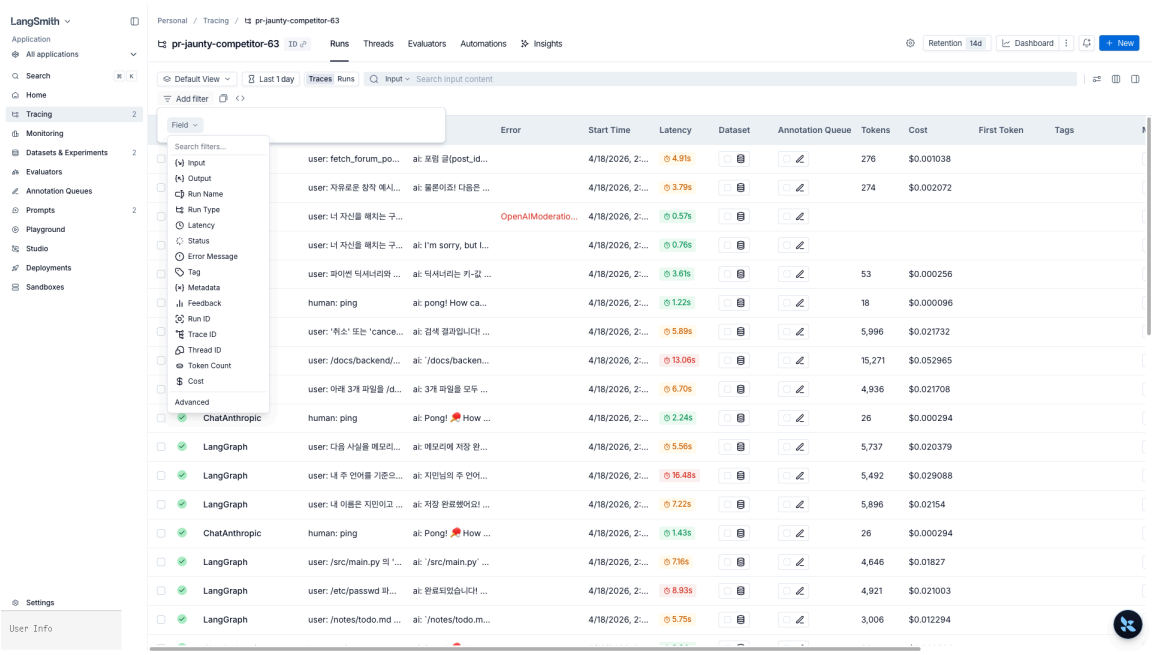


그림 16: Add filter 메뉴: Tag·Metadata가 별도 필드로 존재 — `tags contains env:dev` 같은 조건 쿼리 가능

2.7 `@traceable` 데코레이터 — run_type별 종류

`@traceable` 은 LangChain 외부 함수를 트레이스 트리에 편입하는 데 쓰며, `run_type` 으로 UI 아이콘과 필터 카테고리가 정해집니다. 자주 쓰는 값은 다음과 같습니다.

run_type	용도	UI 표시
`chain`	기본값 — 임의의 함수 / 파이프라인 스텝	체인 아이콘, `chain` 필터
`llm`	수동 LLM 호출 (provider SDK 직접 사용 등)	LLM 아이콘, 토큰/비용 자동 합산
`tool`	외부 시스템 호출 (검색·DB·API)	도구 아이콘, `tool` 필터
`retriever`	벡터스토어·검색 단계	retriever 아이콘, document 카운트 표시
`embedding`	임베딩 계산 단계	embedding 아이콘
`parser`	출력 파싱·후처리	parser 아이콘

```python
from langsmith import traceable

@traceable(run_type="retriever", name="vector-search")
def search(query: str) -> list[dict]:
    return vectorstore.similarity_search_with_score(query, k=4)

@traceable(run_type="tool", name="lookup-order")
def lookup_order(order_id: str) -> dict:
    return db.fetch_order(order_id)
```

2.8 PII 마스킹 — `LangChainTracer` + `Client(anonymizer=...)`

LangSmith로 보내기 직전에 input/output을 정규식으로 마스킹하려면 `create_anonymizer`로 패턴 목록을 만든 뒤 `Client(anonymizer=...)`에 주입합니다. 이 클라이언트로 만든 `LangChainTracer`를 `config={"callbacks": [...]}`로 꽂으면 해당 실행만 PII가 가려진 채 기록됩니다.

```python
from langsmith import Client
from langsmith.anonymizer import create_anonymizer
from langchain.callbacks.tracers import LangChainTracer

anonymizer = create_anonymizer([
    {"pattern": r"[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+", "replace": "<email>"},
    {"pattern": r"\b\d{4}[ -]?\d{4}[ -]?\d{4}[ -]?\d{4}\b",            "replace": "<card>"},
    {"pattern": r"010[ -]?\d{4}[ -]?\d{4}",                             "replace": "<phone>"},
])
masked_client = Client(anonymizer=anonymizer)
tracer = LangChainTracer(client=masked_client, project_name="prod-masked")

agent.invoke(
    {"messages": [{"role": "user", "content": "내 이메일 a@b.com, 010-1234-5678"}]},
    config={"callbacks": [tracer]},
)
```

전체 input/output을 통째로 차단하려면 `Client(hide_inputs=lambda _: {}, hide_outputs=lambda _: {})` 또는 `LANGSMITH_HIDE_INPUTS=true` 환경 변수를 씁니다. 5장에서 `PIIMiddleware`와 함께 이중 방어 패턴으로 다시 다룹니다.

2.9 Selective tracing — 일부 실행만 기록

전역 트레이싱을 켜둔 상태에서 특정 호출만 제외하거나, 특정 호출만 따로 다른 프로젝트로 보내는 패턴이 자주 필요합니다.

```python
import langsmith as ls

# 1) 환경변수가 켜져 있어도 이 블록만 비활성화
with ls.tracing_context(enabled=False):
    agent.invoke({"messages": [{"role": "user", "content": "민감 데이터..."}]})

# 2) 디버깅용 호출만 별도 프로젝트로 라우팅
with ls.tracing_context(project_name="langsmith-debug", tags=["mode:debug"]):
    agent.invoke({"messages": [{"role": "user", "content": "테스트 케이스"}]})
```

`tracing_context`는 thread-local로 적용되므로 비동기 환경에서도 안전합니다. 환경 변수와 컨텍스트 매니저, `config={"callbacks": [...]}` 세 경로의 우선순위는 가까운 스코프 우선입니다.

2.10 400일 보존 한계 → 데이터셋으로 영구화

SaaS LangSmith는 ingestion 시점부터 400일 후 trace가 삭제됩니다. 평가 회귀에 쓸 실행은 Dataset으로 영구화해야 합니다. 3장에서 자세히 다루고, 여기선 패턴만 확인합니다.

```
# langsmith 0.7+에서는 add_runs_to_dataset이 제거되었으므로
# create_examples로 run의 inputs/outputs를 직접 example로 변환한다.
from datetime import datetime, timedelta, timezone

project = client.read_project(project_name="langsmith-tracing-agents")
since = datetime.now(timezone.utc) - timedelta(days=30)
golden_runs = [
    r
    async for r in client.runs.query(
        project_ids=[str(project.id)],
        min_start_time=since,
        max_start_time=window_end,
        page_size=100,
        selects=["ID", "INPUTS", "OUTPUTS", "FEEDBACK_STATS"],
    )
    if is_human_approved(r.feedback_stats)
]
ds = client.create_dataset("agent-golden-traces",
                           description="사람이 승인한 황금 trace")
client.create_examples(
    dataset_id=ds.id,
    examples=[
        {"inputs": r.inputs,
         "outputs": r.outputs,
         "metadata": {"source_run_id": str(r.id)}}
        for r in golden_runs if r.outputs
    ],
)
```

핵심 정리

- Project → Trace → Run + Thread(세션 묶음) 4층 개념이 모든 UI 뷰의 기초
- LangGraph 서브그래프는 네임스페이스가 run 이름에 박히므로 필터 가능
- Deep Agents 동기 서브에이전트는 단일 trace, 비동기는 별개 trace — `async_tasks` 채널로 추적
- `thread_id` / `session_id` 메타데이터가 Threads 뷰 묶음을 트리거
- Feedback API + SmithDB `runs.query(filter=...)` 로 평가 루프의 프로그램적 연결
- `@traceable` 의 `run_type` (`chain` / `llm` / `tool` / `retriever` / `embedding` / `parser`)이 UI 아이콘과 필터 카테고리를 결정
- PII는 `create_anonymizer` + `Client(anonymizer=...)` + `LangChainTracer` 조합으로 트레이스 전송 직전 마스킹
- Selective tracing은 `ls.tracing_context(enabled=...)` 로 블록 단위 on/off — thread-local
- 400일 보존 한계를 넘기려면 Dataset으로 이관

03 데이터셋과 평가 루프

Code · LLM-as-judge · Pairwise · Summary · Online

트레이스는 "지금 어떻게 굴러가는지"를 보여주고, 평가는 "프롬프트·모델·코드를 바꿨을 때 더 좋아졌는지"를 답합니다. LangSmith는 트레이스를 그대로 데이터셋으로 끌어올려 code evaluator · LLM-as-judge · pairwise · summary 평가를 돌립니다. 수동 시드 데이터셋부터 프로덕션 trace 이관, 4종 evaluator, `evaluate` 러너, online evaluator까지 평가 파이프라인 전체를 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → `client.create_dataset` + `client.create_examples` 로 데이터셋과 예시를 만든다
- → 프로덕션 trace를 `client.create_examples(dataset_id=..., examples=[...])` 로 데이터셋에 이관한다
- → Code evaluator를 `(inputs, outputs, reference_outputs) → dict` 형식으로 작성한다
- → LLM-as-judge evaluator를 구조적 score로 돌린다
- → Pairwise / summary evaluator로 두 실험 비교와 데이터셋 수준 지표를 낸다
- → `from langsmith.evaluation import evaluate` 러너로 experiment를 실행한다
- → 프로덕션 trace에 **online evaluator**를 자동 적용한다

3.1 Dataset 생성 — 수동 예시 추가

도메인 전문가가 골든 Q&A를 직접 적어 `create_examples` 로 넣습니다. `inputs` 와 `outputs` 는 모두 dict입니다. `outputs` 는 reference 정답으로 code evaluator / LLM-as-judge의 비교 대상이 됩니다.

```
from langsmith import Client

client = Client()
dataset = client.create_dataset(
    "weather-bot-qa",
    description="도시 추출 골든 예시",
)

client.create_examples(
    dataset_id=dataset.id,
    inputs=[
        {"question": "서울 날씨 알려줘"},
```

```
        {"question": "부산시 기온은?"},
        {"question": "대전 주말에 비와?"},
    ],
    outputs=[
        {"city": "서울"},
        {"city": "부산"},
        {"city": "대전"},
    ],
)
```

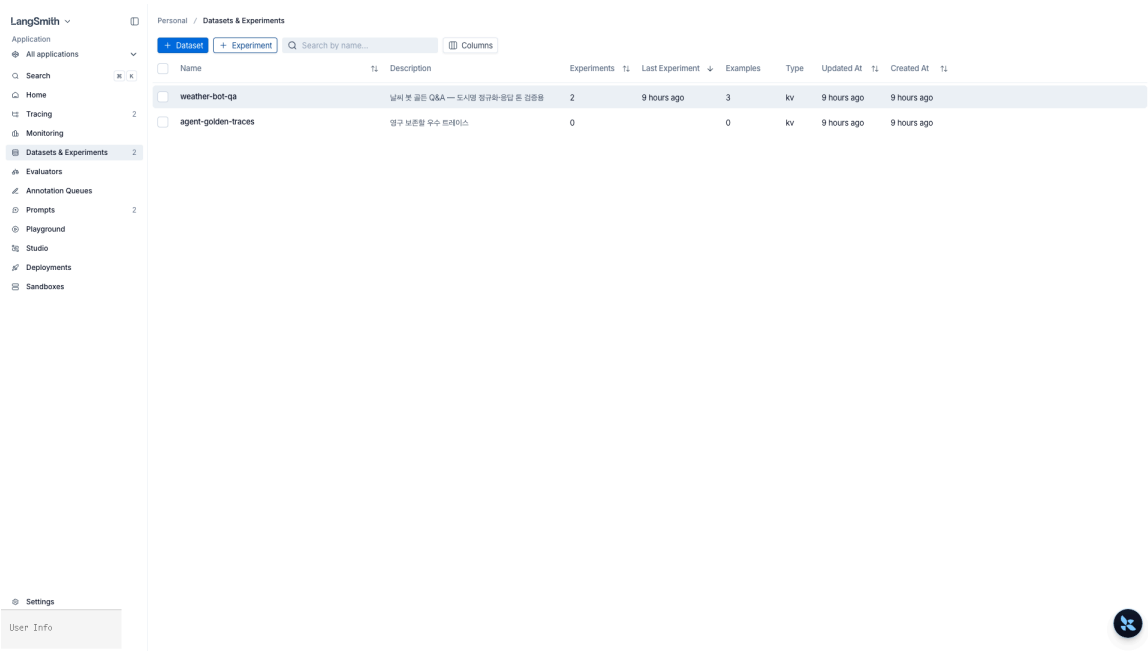


그림 17: Datasets & Experiments 리스트 — `weather-bot-qa` 와 `agent-golden-traces` 두 dataset

3.2 프로덕션 trace를 데이터셋으로 이관

수동 작성은 초기 시드에만 쓰고, 규모 있는 데이터는 프로덕션 trace에서 옵니다. `langsmith 0.7` 부터 `add_runs_to_dataset` 이 제거됐으므로 `client.create_examples(...)` 로 run의 `inputs` / `outputs` 를 example로 변환해 넣습니다. SmithDB query는 `selects` 를 생략하면 `ID` 만 채우므로, 예시처럼 `ID` , `INPUTS` , `OUTPUTS` , `FEEDBACK_STATS` 를 명시합니다. 운영에서는 Annotation Queue로 사람이 리뷰한 run만 올리는 게 보통입니다.

```
from datetime import datetime, timedelta, timezone

project = client.read_project(project_name="prod")
since = datetime.now(timezone.utc) - timedelta(days=30)
good_runs = [
    r
    async for r in client.runs.query(
        project_ids=[str(project.id)],
        min_start_time=since,
        page_size=100,
        selects=["ID", "INPUTS", "OUTPUTS", "FEEDBACK_STATS"],
    )
    if is_good(r)
]
client.create_examples(
    dataset_id=client.read_dataset(dataset_name="weather-bot-qa").id,
    examples=[
        {"inputs": r.inputs,
```

```
            "outputs": r.outputs,
            "metadata": {"source_run_id": str(r.id)}}
           for r in good_runs if r.outputs
    ],
)
```

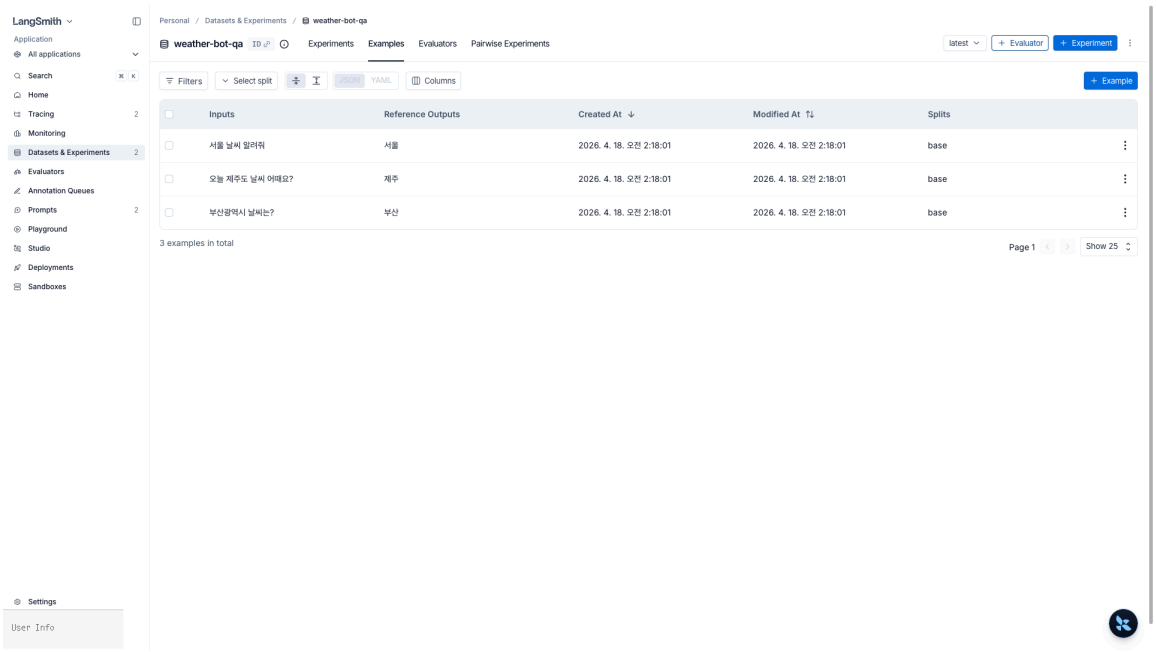


그림 18: Dataset Examples 탭 — 한국어 질문 Inputs와 기대 도시명 Reference Outputs. JSON/YAML 토글, Splits 컬럼 제공

3.3 평가 대상 + Code evaluator

예제용으로 “질문에서 도시명을 뽑는” LLM 함수를 대상(target)으로 씁니다. target은 `inputs: dict → outputs: dict` 형태면 됩니다.

Evaluator는 `(inputs, outputs, reference_outputs)` 를 받아 `{"key": ..., "score": ...}` dict를 돌려줍니다. 결정적 휴리스틱은 비용 0, 지연 0 ms — 가능한 한 많이 넣는 게 이득입니다.

```
def city_exact_match(inputs, outputs, reference_outputs):
    return {
        "key": "city_exact_match",
        "score": int(outputs["city"] == reference_outputs["city"]),
    }

def city_non_empty(inputs, outputs, reference_outputs):
    return {
        "key": "city_non_empty",
        "score": int(bool(outputs.get("city"))),
    }
```

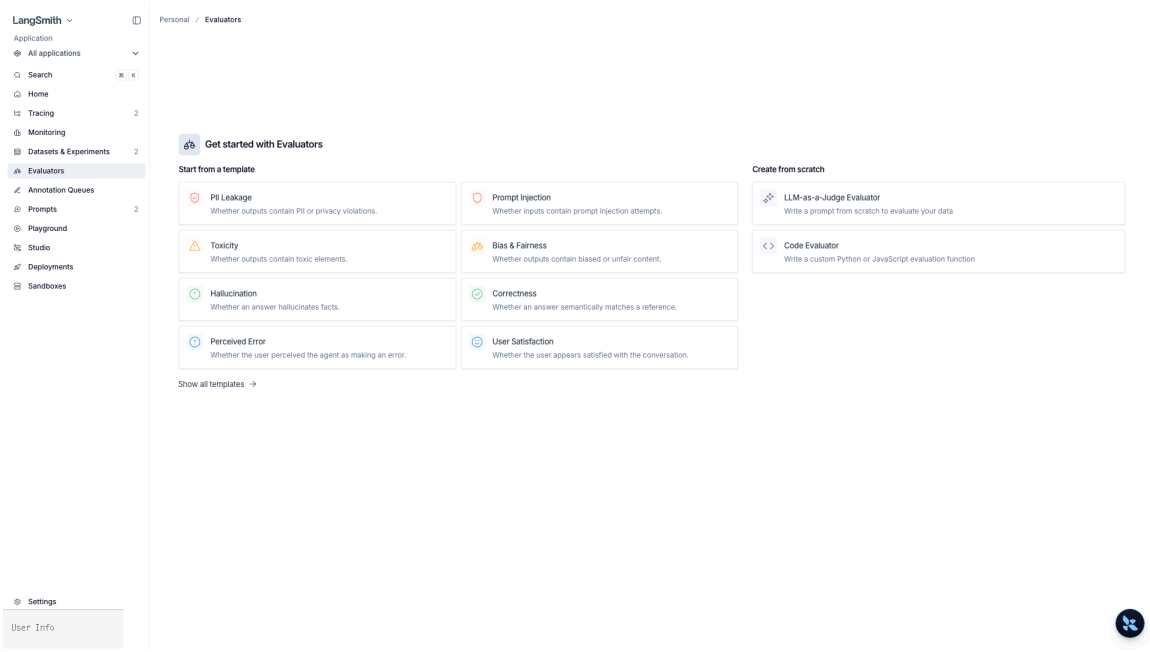


그림 19: Evaluator 템플릿 갤러리 — PII Leakage · Prompt Injection · Toxicity · Bias & Fairness · Hallucination · Correctness · Perceived Error · User Satisfaction + 직접 작성

3.4 LLM-as-judge evaluator

정답 문자열이 없거나 자연어 품질(톤·정확성·도움됨)을 재야 할 때 씁니다. 같은 LLM을 호출하지만 출력을 구조화된 score로 강제하는 것이 핵심입니다.

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Judgement(BaseModel):
    score: float   # 0.0 ~ 1.0
    reason: str

judge_llm = ChatOpenAI(model="gpt-5.4").with_structured_output(Judgement)

def semantic_city_match(inputs, outputs, reference_outputs):
    j = judge_llm.invoke(
        f"질문: {inputs['question']}\n"
        f"정답 도시: {reference_outputs['city']}\n"
        f"추출: {outputs['city']}\n"
        "0.0~1.0 score + reason",
    )
    return {"key": "semantic_city_match", "score": j.score, "comment": j.reason}
```

3.5 `evaluate` 러너 + experiment 이름

표준 러너는 두 가지 import 경로 모두 동일하게 동작합니다 — 새 코드는 짧은 쪽인 `from langsmith import evaluate` 를 권장합니다. `Client` 인스턴스가 있다면 `client.evaluate(...)` 로도 동일하게 호출할 수 있습니다. `experiment_prefix` 에 의미 있는 이름을 주면 UI Experiments 뷰에서 바로 비교됩니다.

```python
from langsmith import Client, evaluate     # 권장 (langsmith ≥ 0.3)
# from langsmith.evaluation import evaluate  # 구 경로, 여전히 호환

result = evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match, city_non_empty, semantic_city_match],
    experiment_prefix="city-extractor:gpt-5.4",
    disable_evaluator_tracing=True,
    metadata={"prompt_commit": "12344e88"},
)

# Client.evaluate(...) 도 동일 시그니처
client = Client()
result = client.evaluate(
    city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
)
```

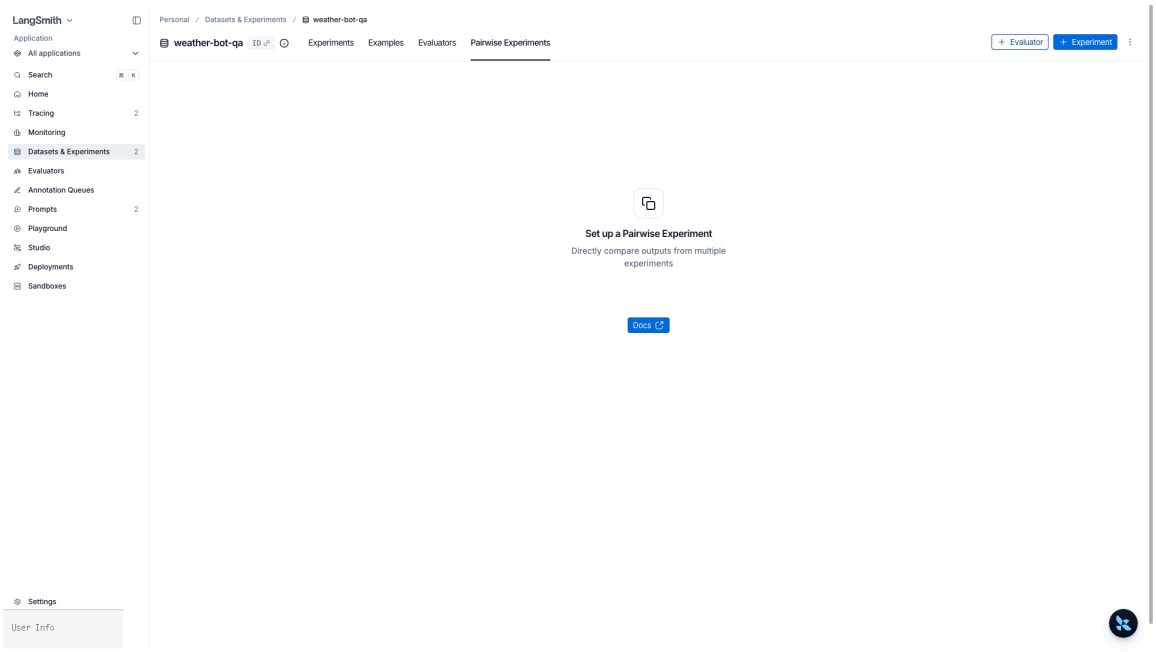


그림 20: Pairwise Experiments 탭 — 두 experiment의 output을 나란히 비교. `evaluate_comparative` API로 실행

`disable_evaluator_tracing=True`는 evaluator 내부 모델 호출이 별도 trace를 많이 만들지 않게 막습니다. evaluator 프롬프트를 디버깅할 때는 끄고, experiment 결과와 비용·노이즈 관리가 더 중요할 때만 켭니다.

3.6 Pairwise + Summary evaluator

- **Pairwise**: 두 experiment의 같은 example 출력을 놓고 "어느 쪽이 더 나은가"를 뽑습니다. A/B 프롬프트 실험에 씁니다. `evaluate_comparative` 러너
- **Summary**: 데이터셋 전체 수준의 지표(예: 정확도 매크로 평균). 시그니처가 run/example의 리스트를 받도록 다릅니다

```python
def macro_accuracy(runs, examples):
    correct = sum(
```

```
        r.outputs.get("city") == e.outputs["city"]
        for r, e in zip(runs, examples)
    )
    return {"key": "macro_accuracy", "score": correct / len(runs)}

evaluate(
    target=city_extractor,
    data="weather-bot-qa",
    evaluators=[city_exact_match],
    summary_evaluators=[macro_accuracy],
)
```

3.7 `agentevals` — 에이전트 trajectory 평가

도시 추출처럼 최종 출력만 보는 평가는 단순 string 비교로 충분하지만, 에이전트는 도구를 어떤 순서로 호출했는가가 같이 중요합니다. `agentevals` 패키지(`pip install agentevals`)는 LangChain의 메시지 트레이스를 기준으로 reference trajectory와 실제 trajectory를 비교하는 평가기를 제공합니다.

비교 모드는 네 가지입니다.

모드	일치 기준	언제 쓰나
`strict`	도구 이름·순서·인자까지 모두 일치	고정된 워크플로우의 회귀 테스트
`unordered`	도구 집합 일치, 호출 순서는 무관	parallel tool use, 순서가 중요하지 않은 파이프라인
`subset`	실제 trajectory ⊆ reference (allowed actions)	에이전트가 기대보다 적게 부른 경우만 허용
`superset`	reference ⊆ 실제 trajectory (required actions)	에이전트가 반드시 부르길 원하는 도구가 있을 때

```
from agentevals.trajectory.match import create_trajectory_match_evaluator

trajectory_strict = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
)

# evaluators 인자에 그대로 넘기면 LangSmith experiment의 점수로 합산
evaluate(
    target=run_agent,
    data="agent-golden-trajectories",
    evaluators=[trajectory_strict],
    experiment_prefix="agent:trajectory-strict",
)
```

LLM-as-judge로 trajectory를 자유 형식으로 평가하려면 `create_trajectory_llm_as_judge` 를 씁니다. 도구 이름 비교가 아니라 전체 reasoning chain의 적절성을 LLM이 판단하게 만듭니다.

```
from agentevals.trajectory.llm import create_trajectory_llm_as_judge

trajectory_judge = create_trajectory_llm_as_judge(
    model="openai:gpt-5.4",
    prompt="에이전트의 도구 호출 순서가 사용자의 의도를 효율적으로 달성했는가?",
)
```

> ➡️ TIP
>
> `agentevals` 는 `openevals` (범용 LLM 평가)와 한 쌍입니다. 최종 답변 품질은 `openevals` , 과정의 적절성은 `agentevals` 로 분리해 두 점수를 같은 experiment에 함께 붙이는 것이 권장 패턴입니다.

3.8 Online evaluator — 프로덕션 trace 자동 평가

Offline experiment는 배포 전 회귀 테스트에 쓰고, 운영 중에는 **online evaluator**로 실시간 feedback을 붙입니다. UI 흐름:

1. 프로젝트 → **Evaluators** 탭 → `+ Evaluator`
2. **LLM-as-judge** 선택, 평가 프롬프트 작성 (예: "응답이 사용자의 의도에 답하는가?")
3. 필터 지정 — 예: `has(tags, "env:prod")` 인 run만
4. **Sampling rate**를 0.1로 두면 매칭 trace의 10%만 평가 — 비용 제어
5. 저장하면 신규 trace에 자동으로 feedback key가 붙기 시작

과거 trace에도 소급하려면 **Apply to past runs**를 켜고 기간을 지정합니다.

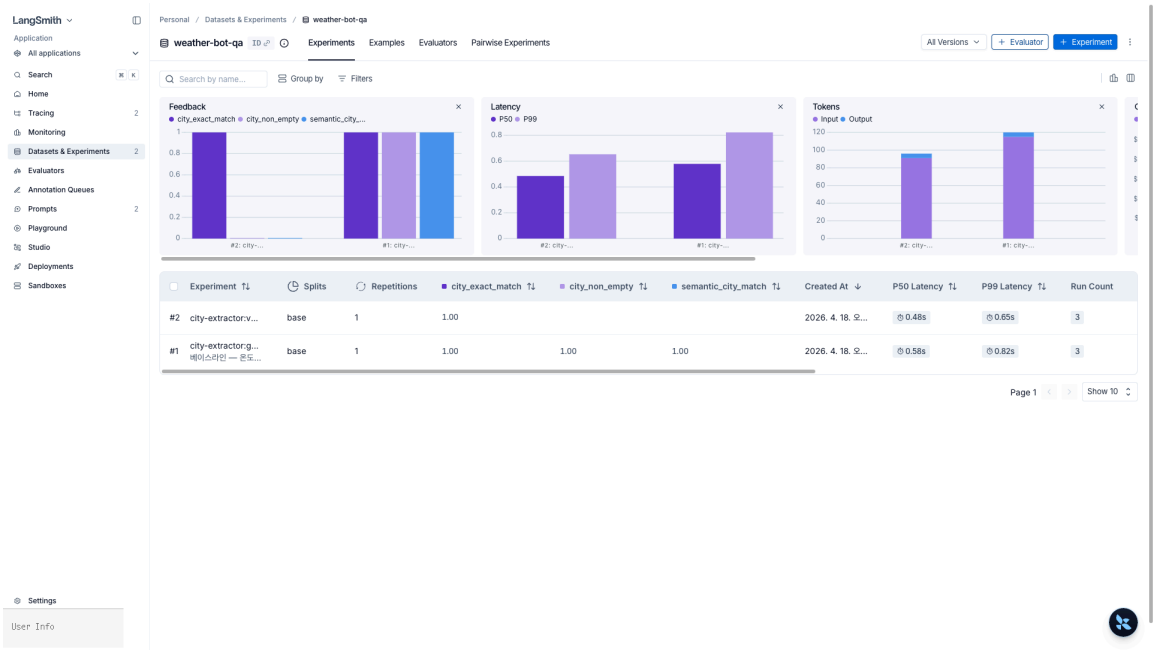


그림 21: Experiment 결과 + evaluator 차트 — Feedback 점수(`city_exact_match` / `city_non_empty` / `semantic_city_match`), Latency P50/P99, Tokens Input/Output 시계열

핵심 정리

- 데이터셋은 수동 시드 + 프로덕션 trace 이관의 조합으로 누적
- Evaluator 4종: Code(결정적, 저비용) / LLM-as-judge(자연어 품질) / Pairwise(A/B 비교) / Summary(데이터셋 수준)
- `from langsmith import evaluate` 또는 `Client.evaluate(...)` 둘 다 동일한 시그니처

- `agentevals` 의 trajectory match 4 모드(strict/unordered/subset/superset) + LLM-as-judge로 과정의 적절성 평가
- Online evaluator는 프로덕션 trace에 feedback key를 자동 부착, 대시보드·알림의 트리거가 됨
- `prompt_commit` 같은 metadata를 실험에 붙이면 "어떤 버전에서 낸 수치인지" 재현됩니다

04 Prompt Hub 버전 관리

Commit SHA · Tag · Playground

프롬프트는 사실상 코드지만, 엔지니어가 아닌 사람이 자주 고치고 주기도 짧습니다. 매번 재배포하지 않으려면 프롬프트를 애플리케이션 코드와 분리된 저장소에 두고 버전 핀을 박아야 합니다. LangSmith Prompt Hub가 그 역할을 합니다. push/pull API, commit SHA vs tag, 템플릿 엔진 선택, Playground 실험, CI 핀 전략을 다룹니다.

학습 목표

LEARNING OBJECTIVES

- → `client.push_prompt("name", object=...)` 로 프롬프트를 업로드한다
- → Commit SHA 고정 vs `prod` · `staging` 태그 참조의 차이를 이해한다
- → f-string vs mustache 템플릿의 변수 처리 차이를 비교한다
- → Playground에서 실험 → 커밋 → 태그 플로우를 숙지한다
- → 런타임에서 `client.pull_prompt("name:prod")` 로 프롬프트를 주입한다
- → CI 테스트에서 특정 커밋 해시를 고정해 회귀를 방지한다

4.1 Prompt 생성 · 푸시 — `Client.push_prompt(...)`

가장 단순한 형태는 `ChatPromptTemplate` 을 만든 뒤 `client.push_prompt("이름", object=prompt)` 로 올리는 것입니다. 첫 push에서 새 prompt가 생기고, 이후 push마다 새 commit이 쌓입니다. 반환 URL로 UI에서 바로 열립니다.

```python
from langchain_core.prompts import ChatPromptTemplate
from langsmith import Client

client = Client()
prompt = ChatPromptTemplate.from_messages([
    ("system", "너는 날씨 비서야. 도시명만 뽑아 알려줘."),
    ("user", "{question}"),
])

url = client.push_prompt(
    "weather-bot",
    object=prompt,
    description="질문에서 도시명을 뽑는 시스템 프롬프트",
```

```
    tags=["weather", "extraction"],
    is_public=False,                    # 조직 외부 공개 여부
)
print(url)  # https://smith.langchain.com/hub/...
```

`Client.push_prompt(...)`의 주요 인자는 다음과 같습니다.

- `object` — `ChatPromptTemplate` / `PromptTemplate` 등 LangChain 프롬프트 객체
- `description` — UI 카드에 노출되는 한 줄 설명
- `tags` — 검색·필터용 태그 (commit 태그가 아닌 prompt 메타데이터)
- `is_public` — public hub에 공개할지 여부 (기본 False)
- `parent_commit_hash` — 특정 커밋 기준으로 새 커밋 생성 (기본은 최신)

반환 URL에는 조직 식별용 query string이 붙을 수 있습니다. CI 핀에 쓸 commit SHA는 `url.split("/")[-1]`처럼 문자열 끝을 자르지 말고, URL path의 마지막 segment만 파싱해야 합니다.

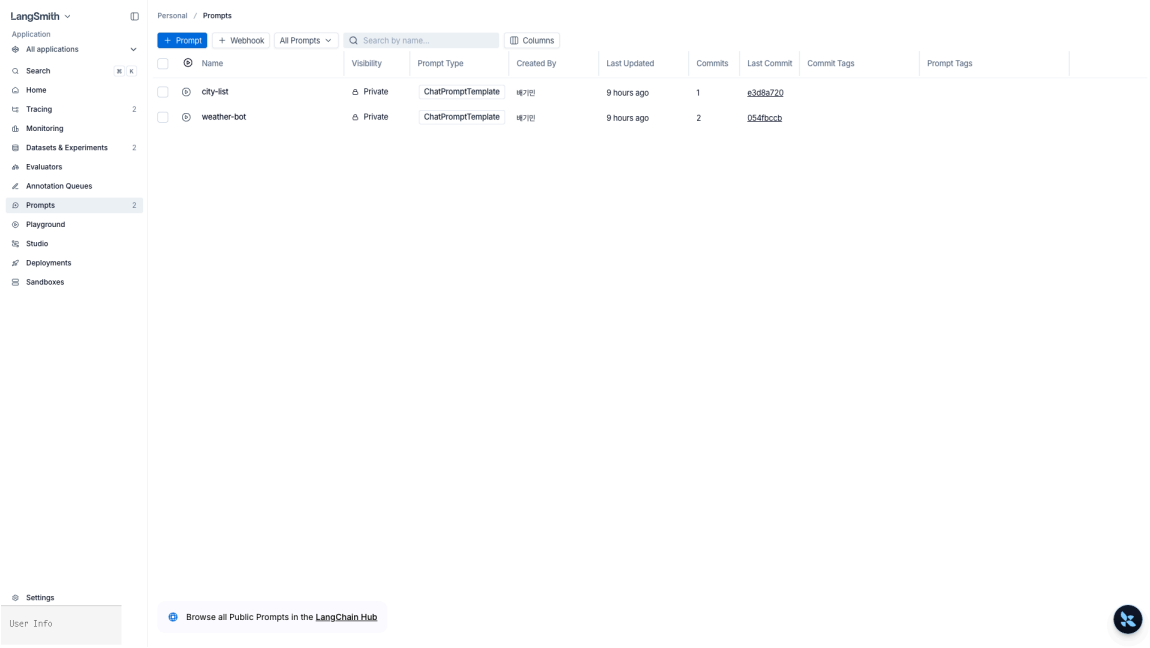


그림 22: Prompts 허브 목록 — `city-list`(1 commit), `weather-bot`(2 commits). Visibility, Last Commit 짧은 SHA 표시

4.2 Commit SHA 고정 vs 태그 (`prod`, `staging`)

프롬프트는 Git처럼 동작합니다.

참조 방식	예	특성	언제 쓰나
Commit SHA	`weather-bot:12344e88`	불변, 정확히 그 버전을 핀	CI 회귀 테스트, 재현 필요 시
Tag	`weather-bot:prod`, `:staging`	이동형 — 다른 커밋을 가리키도록 변경 가능	런타임 배포 슬롯

최신	`weather-bot`	가장 최근 커밋	개발 초기에만, 프로덕션 금지

태그는 애플리케이션 코드를 재배포하지 않고 프롬프트만 교체하는 핵심 장치입니다. UI Commits 뷰에서 특정 커밋에 `prod` 태그를 **promote**합니다.

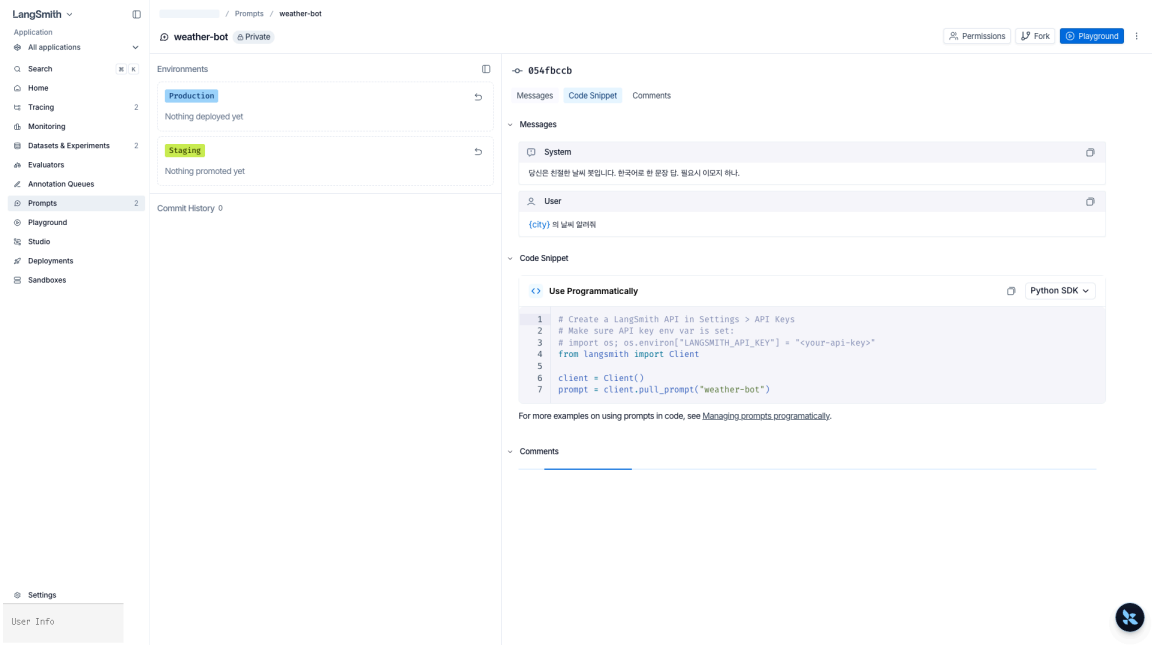


그림 23: Prompt 상세 — 상단 commit + 탭(Messages/Code Snippet/Comments), Environments 패널의 Production/Staging 슬롯

4.3 f-string vs mustache

두 템플릿 엔진의 차이는 실무에서 자주 문제가 됩니다.

항목	f-string (`{var}`)	mustache (`{{var}}`)
기본값	LangChain 기본	별도 선택
JSON 예시 포함	`{{` 이스케이프 필요	이스케이프 불필요
조건문 · 반복	미지원	`{{#users}}...{{/users}}`
중첩 키	제한적	`{{user.name}}`
Playground 변수 지정	자동 감지	수동 지정 (Inputs)

JSON/코드 예시가 많거나 반복·조건이 들어가면 mustache가, 단순 변수 치환이면 f-string이 편합니다.

4.4 Playground — 실험에서 커밋까지

UI의 **Playground**는 프롬프트 + 모델 + 입력 변수를 묶어 돌려보는 공간입니다. 흐름은:

1. 프롬프트 페이지에서 `Open in Playground` 클릭
2. 모델·temperature·출력 스키마·tools를 사이드 패널에서 조정
3. 변수 값을 넣고 `Run` — 결과와 token/cost가 곧바로 기록됨
4. `Compare` 로 같은 입력에 대한 여러 프롬프트/모델 출력 병렬 비교
5. 만족스러운 상태에서 `Save as...` → 새 커밋 생성, 필요하면 `prod` 태그로 promote

Playground에서 돌린 모든 run은 Experiments 뷰로 넘어가 3장의 데이터셋과 직접 연결됩니다.

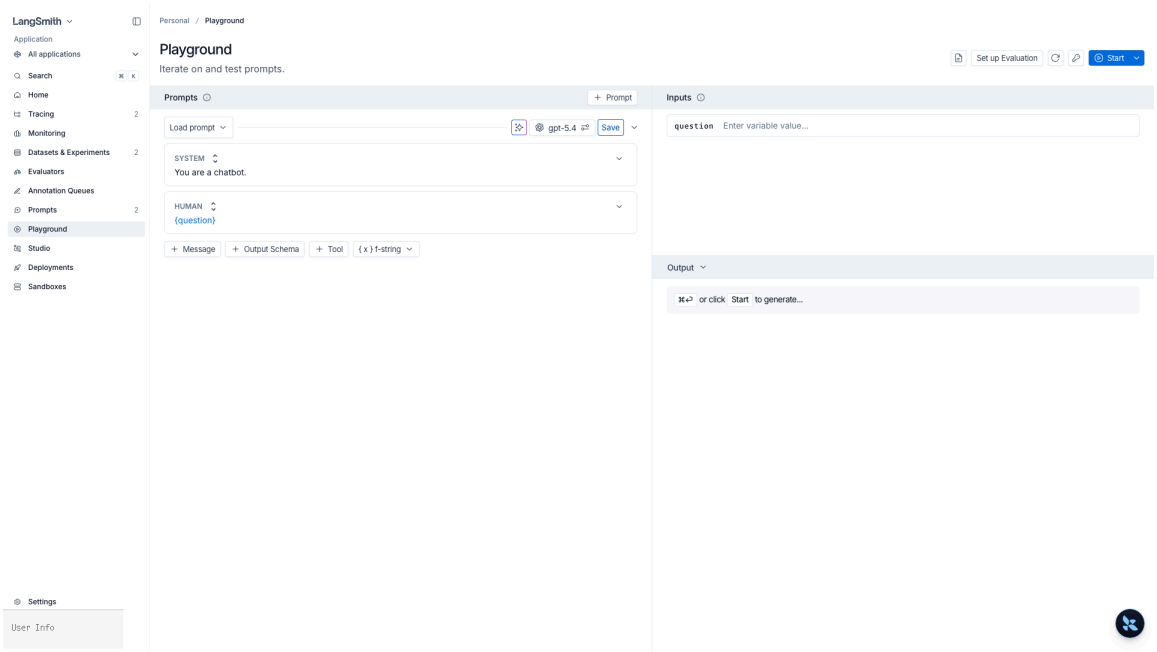


그림 24: Playground — SYSTEM/HUMAN 메시지 편집 + `{question}` 변수 입력 + Output 생성. f-string↔mustache 스위처 제공

4.5 런타임 주입 — `Client.pull_prompt(...)` → `create_agent`

애플리케이션에서는 배포 슬롯 태그를 `client.pull_prompt(...)` 로 당겨 LLM/에이전트에 바로 꽂습니다. 프롬프트를 고치면 재배포 없이 다음 요청부터 반영됩니다.

```python
from langchain.agents import create_agent

# 1) tag 또는 commit SHA 로 참조
prompt = client.pull_prompt("weather-bot:prod")              # 태그
# prompt = client.pull_prompt("weather-bot:12344e88")        # commit SHA
# prompt = client.pull_prompt("weather-bot", include_model=True)  # 저장된 모델 설정까지

agent = create_agent(
    model="openai:gpt-5.4",
    system_prompt=prompt.format_messages()[0].content,
    tools=[],
)
```

`Client.pull_prompt(...)` 의 주요 옵션은 다음과 같습니다.

- `prompt_identifier` — `"name"`, `"name:tag"`, `"name:SHA"` 세 가지 참조 방식
- `include_model` — True로 두면 Playground에서 저장한 모델·temperature까지 함께 반환되는 `RunnableSequence`

- 캐싱 — Python SDK가 commit 기준으로 ETag 캐싱을 하므로 같은 SHA를 반복해 받을 때 비용이 들지 않음

공개 Hub 프롬프트(`owner/name`)를 가져올 때는 신뢰한 프롬프트에 한해서 `dangerously_pull_public_prompt=True` 를 명시합니다. 공개 프롬프트는 직렬화된 LangChain 객체를 포함할 수 있으므로, 교육·테스트 코드에서도 의도를 코드에 남기는 편이 안전합니다.

4.6 CI에서 특정 커밋 해시 고정

프로덕션 배포 슬롯은 `:prod` 태그로 받고, 회귀 테스트는 commit SHA를 핀해야 합니다. 그래야 "통과한 테스트 이후 누군가 프롬프트를 고쳤다"는 사고를 자동으로 막습니다.

```python
# tests/test_prompt_regression.py
PINNED_SHA = "12344e88"  # CI 핀

def test_weather_prompt_still_extracts_city():
    prompt = client.pull_prompt(f"weather-bot:{PINNED_SHA}")
    # ... 구체 assertion
```

배포 패턴 요약

환경	참조	이유
dev / 로컬	`weather-bot` (최신)	즉시 반영
staging	`weather-bot:staging`	태그만 promote해서 테스트
prod	`weather-bot:prod`	태그 이동으로 무중단 롤아웃, 롤백은 태그 되돌리기
CI	`weather-bot:{SHA}`	재현 가능, 프롬프트 변경이 바로 테스트 실패로 감지

3장의 experiment에 `prompt_commit` 을 metadata로 붙이면 어떤 커밋 기준에서 낸 수치인지 UI에서 바로 추적됩니다.

핵심 정리

- 프롬프트 허브는 push → commit 축적 → tag promote 구조 (Git과 유사)
- 태그는 런타임 배포 슬롯, SHA는 CI 재현용 핀
- f-string vs mustache는 반복/조건/중첩 여부로 선택
- Playground에서 실험 → Save → promote 플로우가 비개발자 편집 경로
- CI에서 SHA 핀, 프로덕션에서 태그 참조 — 이 조합이 무중단 롤아웃 + 회귀 방지의 핵심

05 프로덕션 모니터링

대시보드 · 알림 · 샘플링 · PII

프로덕션은 "트레이스 한 번 찍어보자"가 아니라 실시간 대시보드 + 자동 평가 + 알림 + 개인정보 방어가 함께 돌아야 합니다. 이 장은 배포 후 필요한 LangSmith 기능을 운영 관점으로 묶습니다 — Monitoring 대시보드, autoeval rule, 사용자 feedback API, 알림 룰, 샘플링, PII 스크러빙, Slack/PagerDuty 웹훅 연동까지.

학습 목표

LEARNING OBJECTIVES

- → 프로젝트 대시보드(Overview/Analytics)에서 latency p50/p95, cost, 성공률을 읽는다
- → Online evaluator를 자동 실행되는 autoeval rule로 등록한다
- → 앱에서 `client.create_feedback(run_id, ...)` 로 사용자 thumbs/별점을 전송한다
- → Metadata 기반 알림 룰(실패율 > N% → webhook)을 이해한다
- → 고볼륨 프로덕션에서 `LANGSMITH_TRACING_SAMPLING_RATE` 로 샘플링한다
- → `PIIMiddleware` 와 `hide_inputs` / `anonymizer` 로 이중 방어한다
- → Slack / PagerDuty 웹훅 수신 패턴을 구성한다

5.1 프로젝트 대시보드

UI의 프로젝트 페이지에는 기본 3개 탭이 있습니다.

탭	보는 것	알림 연결
Runs	trace 목록, 필터, 빠른 검색	—
Monitor	latency p50·p95·p99, 성공률, error 분포, token/cost 시계열	Rules로 임계 알림
Evaluators	부착된 online evaluator와 score 분포	특정 key 점수 하락 알림

대시보드를 직접 구성하고 싶다면 `Dashboards` 에서 커스텀 차트를 묶을 수 있습니다. 같은 지표를 SmithDB `client.runs.query` 로 뽑아 Grafana 같은 사내 시스템에 붙여도 됩니다.

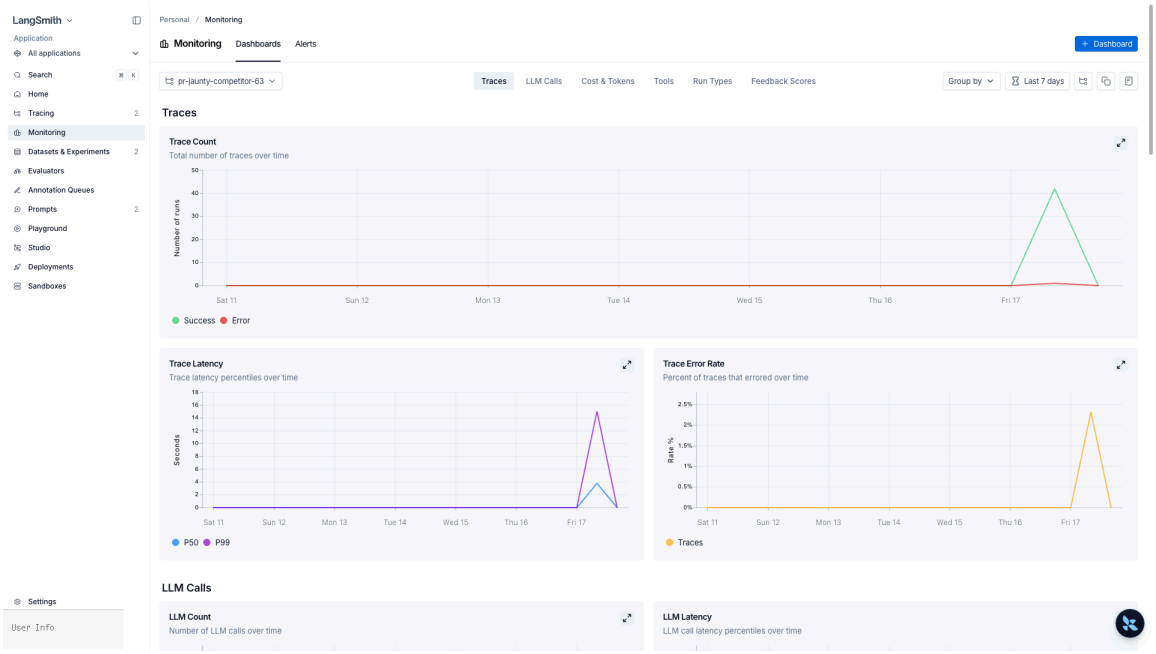


그림 25: Project Monitoring Dashboards — 6개 탭(Traces / LLM Calls / Cost & Tokens / Tools / Run Types / Feedback Scores) × 기간

5.2 Online evaluator — autoeval rule

3장에서 UI로 붙이는 흐름을 확인했다면, 여기선 운영 관점 설계입니다.

역할	설정 예
실시간 품질 게이지	`has(tags, "env:prod")` 인 run에 LLM-as-judge(`useful` score 0 1)
비용 관리	Sampling rate 0.05로 5%만 평가
회귀 감지	`useful` score 평균이 N% 이하로 떨어지면 Rules에서 webhook 트리거
특정 유즈케이스	`metadata.feature == "checkout"` 인 run에만 평가 붙이기

autoeval 결과는 feedback key로 저장되므로, 알림 룰·대시보드·Experiments 비교에 모두 씁니다.

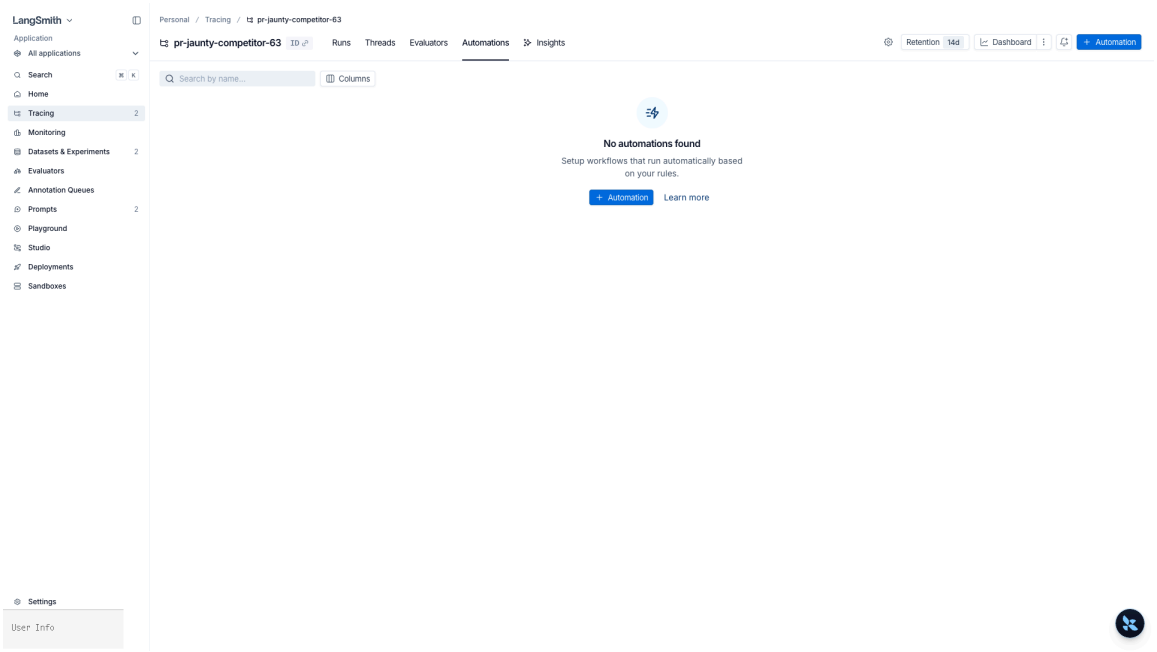


그림 26: 프로젝트 > Automations 탭 — `+ Automation`으로 조건(Feedback score < 0.5) → 액션(dataset 이관 / 웹훅 / annotation queue) 정의

5.3 사용자 feedback 수집 API

앱 UI의 thumbs-up / 별점 / "이 답 도움 안 됨" 버튼을 누르면 서버에서 `client.create_feedback`을 호출합니다. 클라이언트가 기다리지 않도록 **background**로 쏩니다 (Python SDK는 `trace_id`를 주면 자동 백그라운드).

```python
from langsmith import Client

client = Client()
client.create_feedback(
    trace_id=trace_id,
    key="user_thumbs",
    score=1.0,
    comment="정답",
)
```

5.4 Metadata 기반 알림 룰

UI의 프로젝트 → **Rules** 탭에서 다음 액션들을 조합합니다.

- `Add to annotation queue` (사람 리뷰)
- `Add to dataset` (골든셋 축적)
- `Trigger webhook` (Slack, PagerDuty, 자체 incident 시스템)
- `Extend data retention` (중요 trace 보존 연장)
- `Run online evaluator` (조건부 품질 평가)

필터 표현식 예

```
and(
  has(tags, "env:prod"),
  eq(status, "error")
)
```

액션 실행 순서(LangSmith 고정): annotation queue → dataset → webhook → online evaluator → code evaluator → alert. 샘플링 비율도 룰 단위로 다르게 줄 수 있습니다 (예: 0.5 = 매칭의 50%).

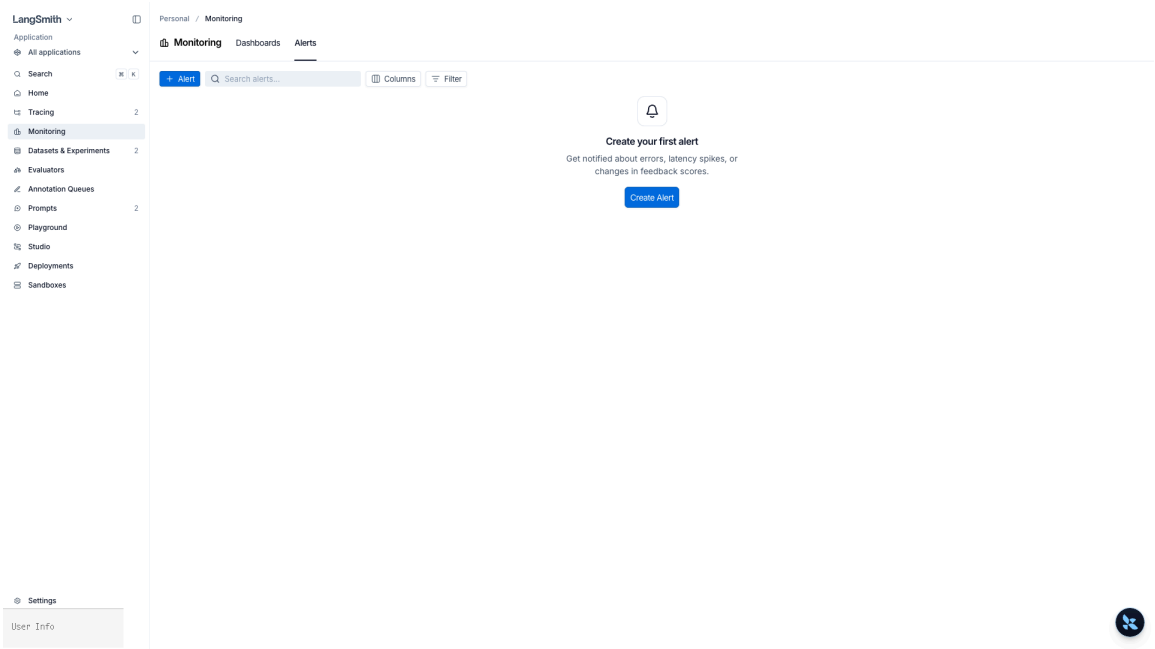


그림 27: Monitoring > Alerts — 조직 전체 알림 규칙 허브. `Create Alert` 클릭 시 모달 노출

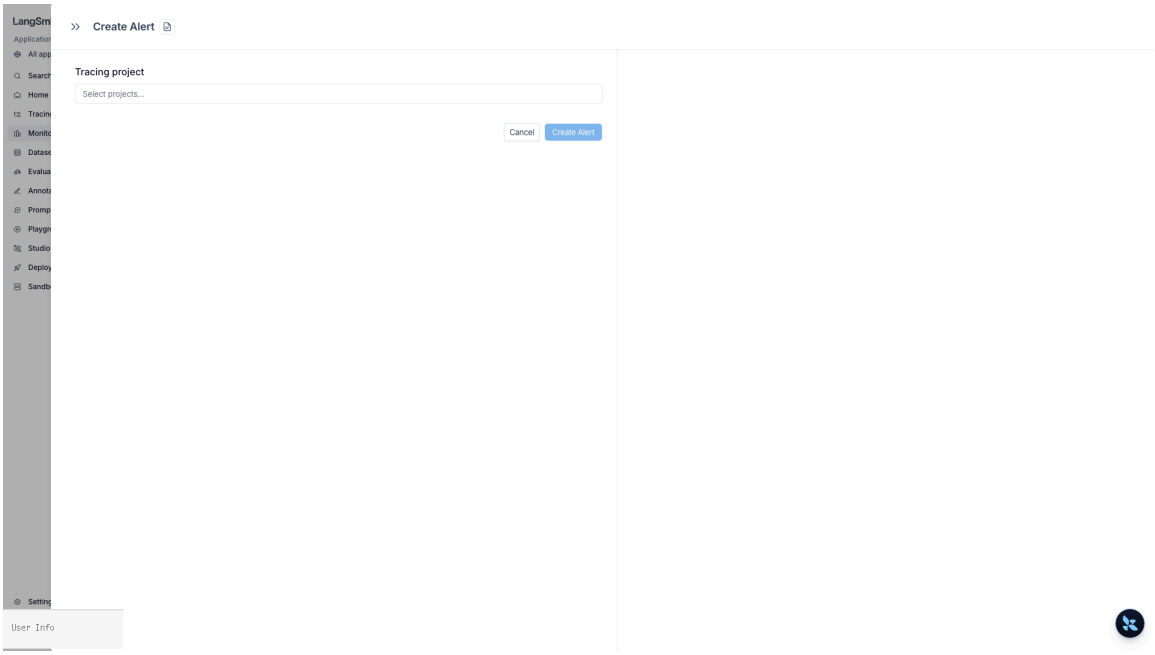


그림 28: Create Alert 모달 — Tracing project 선택 후 조건(에러율/지연/피드백 변화) 지정 → webhook URL 전송

5.5 고볼륨 샘플링

초당 수백 요청이 되면 모든 trace를 보내는 건 비용·네트워크 낭비입니다. 두 층으로 나눠 제어합니다.

층위	수단	특성
프로세스 전체	`LANGSMITH_TRACING_SAMPLING_RATE=0.1`	`@traceable`, `RunTree`, 자동 계측 모두 적용
요청별	`Client(tracing_sampling_rate=...)` + `tracing_context`	관리자/결제 등 중요 요청만 100%

조합하면 "일반 트래픽은 10%, 결제 트래픽은 100%" 같은 운영 정책을 만들 수 있습니다.

5.6 PII 스크러빙

두 레이어로 방어합니다.

1. **모델 입력 단계**: `langchain.agents.middleware.PIIMiddleware`가 LLM에 전달되는 메시지에서 이메일·카드번호·API 키를 차단/마스킹 — 모델 자체가 PII를 보지 않게 합니다
2. **트레이스 전송 단계**: LangSmith 클라이언트의 `hide_inputs` / `hide_outputs` 또는 `anonymizer`가 서버로 올라가는 input/output에서 한 번 더 씻어냅니다

둘 다 걸어야 "모델에 들어갔지만 트레이스엔 안 남는", "모델은 못 봤지만 trace에 남았다" 같은 구멍이 사라집니다.

```
# .env
LANGSMITH_HIDE_INPUTS=true
LANGSMITH_HIDE_OUTPUTS=true
```

5.7 Slack / PagerDuty 웹훅 연동

Rules의 webhook 액션은 설정한 URL로 POST 페이로드를 쏩니다. 받는 쪽에서 Slack/PagerDuty 포맷으로 변환해 알림을 띄웁니다.

```
from fastapi import FastAPI, Request
import httpx, os

app = FastAPI()
SLACK_URL     = os.environ["SLACK_INCOMING_WEBHOOK"]
PAGERDUTY_URL = os.environ.get("PAGERDUTY_EVENTS_V2_URL")

@app.post("/langsmith/alert")
async def on_alert(req: Request):
    event = await req.json()
    run_id  = event.get("run_id") or event.get("trace_id")
    status  = event.get("status", "unknown")
    tags    = event.get("tags", [])
    trace_url = f"https://smith.langchain.com/o/-/projects/-/r/{run_id}"

    async with httpx.AsyncClient() as hc:
        await hc.post(SLACK_URL, json={
```

```
            "text": f":rotating_light: LangSmith alert — status={status} tags={tags}\n<{trace_url}|trace
열기>",
        })
        if PAGERDUTY_URL and "critical" in tags:
            await hc.post(PAGERDUTY_URL, json={
                "routing_key": os.environ["PAGERDUTY_ROUTING_KEY"],
                "event_action": "trigger",
                "payload": {"summary": f"LangSmith {status}", "severity": "error", "source":
"langsmith"},
            })
    return {"ok": True}
```

UI Rules에서 이 엔드포인트 URL을 webhook 대상으로 등록하고, 필터를 `and(has(tags, "env:prod"), eq(status, "error"))` 로 잡으면 프로덕션 에러만 Slack으로 뜹니다.

5.8 LangSmith Deployments — 배포와 관측의 통합

LangGraph 그래프를 LangGraph Platform 으로 배포하면 LangSmith 프로젝트가 자동으로 생성·연결됩니다. 별도 환경 변수를 주입하지 않아도 그래프 실행이 곧바로 같은 조직의 LangSmith 프로젝트에 트레이스로 기록됩니다.

Deployment 화면	연결되는 LangSmith 자원	운영 행동
Deployments → Traces	같은 이름의 Project	실행 즉시 trace 확인
Deployments → Threads	Project의 Threads 뷰	멀티턴 대화 디버깅
Deployments → Monitoring	Project Monitor 탭과 동일 지표	latency · cost · 에러율 알람
Deployments → Settings	환경별 secret · `LANGSMITH_PROJECT` override	staging vs prod 프로젝트 분리

운영 패턴은 보통 다음과 같이 묶입니다.

- 배포는 LangGraph Platform이 담당 — `langgraph deploy` 로 commit 단위 무중단 롤아웃
- 관측은 LangSmith가 담당 — 같은 배포의 trace가 자동으로 프로젝트에 흘러 들어옴
- 알림은 5.4의 Rules로 — `metadata.deployment_id == "..."` 같은 필터로 배포 단위 분리
- 평가는 3장의 online evaluator를 prod 트래픽에 부착해 score 추세를 대시보드에 노출

`langsmith.Client` 는 동일 키로 배포 메타데이터까지 조회할 수 있어, 외부 사내 대시보드(Grafana 등)에서 LangGraph 배포 ID 기준으로 SLO를 따로 추적할 수 있습니다.

5.9 Insights Agent (유료)

프로젝트 > Insights 탭 — LangSmith의 **Insights Agent**로 production trace에서 usage pattern / common failure mode를 자동 추출합니다. 무료 플랜은 upgrade가 필요합니다.

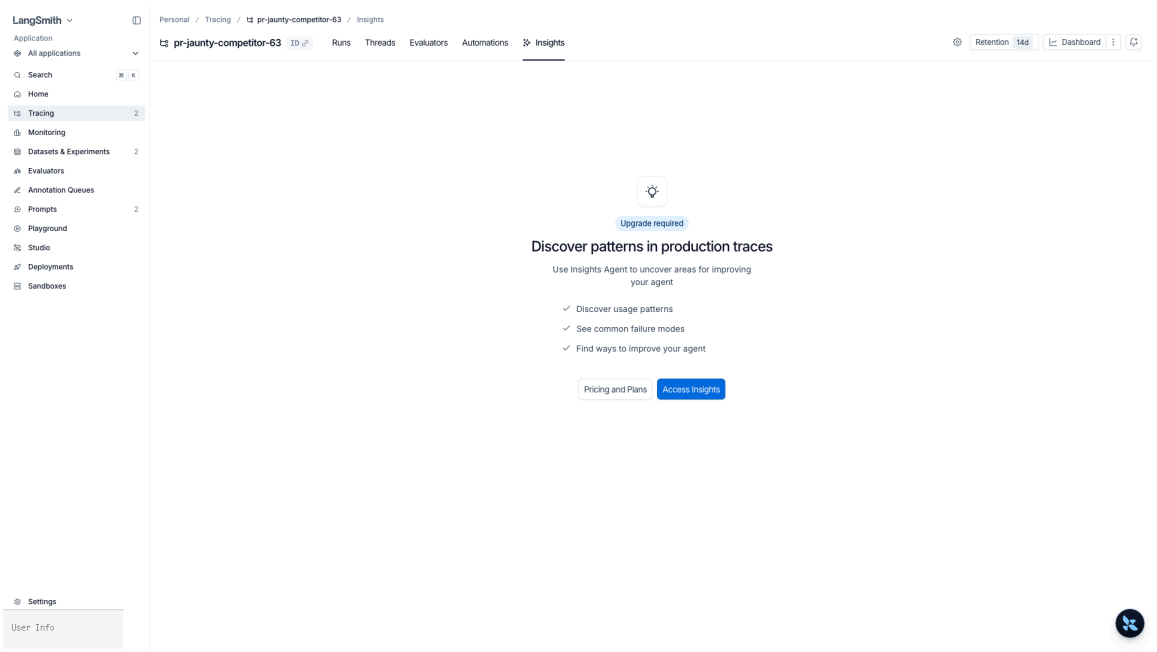


그림 29: Insights 탭 — Upgrade required 상태. 유료 플랜에서 production trace 자동 분석

핵심 정리

- 대시보드·Evaluators·Automations 세 탭이 프로덕션 관측의 기본축
- Online evaluator + Rules webhook이 “실시간 품질 → 알림” 연결 고리
- 사용자 feedback은 background로 쏘고, `key` 별로 점수 추이를 대시보드에 노출
- 샘플링은 전역(환경 변수) + 요청별(`tracing_context`) 두 층 조합
- PII는 `PIIMiddleware` (모델) + `anonymizer` / `hide_inputs` (트레이스) 이중 방어
- Webhook 수신 서비스가 Slack/PagerDuty로 라우팅 — 필터는 `env:prod` + `status=error` 조합이 기본
- LangGraph Platform 배포는 같은 이름의 LangSmith 프로젝트로 자동 연결 — 배포 ID 메타데이터로 SLO를 환경별로 분리

06 Agent Evals

trajectory match와 LLM-as-judge

학습 목표

LEARNING OBJECTIVES

- → `agentevals` 의 trajectory evaluator가 최종 답변이 아니라 도구 호출 경로를 평가하는 방식을 이해한다
- → `strict` , `unordered` , `subset` , `superset` match mode를 구분한다
- → LLM-as-judge evaluator로 reference trajectory와 실제 trajectory의 의미적 일치 여부를 평가한다
- → LangSmith `evaluate()` 와 연결해 agent trajectory 회귀 테스트를 남긴다

개요

기존 `03_datasets_and_evaluation.ipynb` 가 dataset/evaluate 전체 루프를 다뤘다면, 이 장은 에이전트 내부 경로에 집중합니다.

평가 대상	일반 evaluator	AgentEvals
최종 답변	문자열·JSON 정답 비교	가능하지만 주 목적은 아님
도구 호출	직접 파싱 필요	`tool_calls` 시퀀스 비교
경로 품질	LLM judge 직접 작성	trajectory 전용 prompt 제공
회귀 테스트	LangSmith experiment	LangSmith + trajectory evaluator

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
assert os.environ.get("LANGSMITH_API_KEY"), "LANGSMITH_API_KEY 미설정"
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY 미설정"
os.environ.setdefault("LANGSMITH_PROJECT", "langchain-langgraph-deepagents-notebooks")
```

```
from agentevals.trajectory.match import create_trajectory_match_evaluator
from agentevals.trajectory.llm import create_trajectory_llm_as_judge
```

1) 평가할 trajectory 만들기

trajectory는 LangChain message처럼 `role`, `content`, `tool_calls`를 가진 dict list로 표현할 수 있습니다. 핵심은 "어떤 도구를 어떤 인자로 호출했는가"입니다.

```
actual_trajectory = [
    {"role": "user", "content": "서울 날씨 알려줘"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "서울"}}]},
    {"role": "tool", "content": "서울: 맑음, 21°C"},
    {"role": "assistant", "content": "서울은 맑고 21도입니다."},
]
reference_trajectory = [
    {"role": "user", "content": "서울 날씨 알려줘"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather", "args": {"city": "서울"}}]},
]
```

2) Trajectory match evaluator

`trajectory_match_mode`는 실제 trajectory와 reference의 관계를 정합니다.

모드	의미
`strict`	순서와 호출이 정확히 같아야 함
`unordered`	같은 호출 집합이면 순서는 덜 중요
`subset`	실제 trajectory가 reference의 일부여야 함
`superset`	reference 호출을 포함하면 추가 호출 허용

```
trajectory_superset = create_trajectory_match_evaluator(
    trajectory_match_mode="superset",
    tool_args_match_mode="subset",
)
match_result = trajectory_superset(
    outputs=actual_trajectory,
    reference_outputs=reference_trajectory,
)
print(match_result)
assert match_result["score"] is True
```

3) LLM-as-judge trajectory 평가

reference가 너무 추상적이거나, tool 호출 순서보다 "합리적인 경로였는가"가 중요하면 LLM judge를 씁니다. 여기서는 작은 trajectory 한 건만 평가합니다.

```
trajectory_judge = create_trajectory_llm_as_judge(
    model="openai:gpt-5.4",
    use_reasoning=False,
)
judge_result = trajectory_judge(
    outputs=actual_trajectory,
```

```
    reference_outputs=reference_trajectory,
)
print(judge_result)
```

4) LangSmith `evaluate()` 와 연결

아래 셀은 실제 LangSmith dataset과 experiment를 만듭니다. 로컬 실행 harness에서는 기본적으로 remote write/evaluate 셀을 건너뛰고, `--allow-langsmith-mutations` 를 켰을 때만 실행합니다.

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()
DATASET_NAME = "agent-eval-v1"
try:
    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="Agent trajectory regression examples",
    )
except Exception:
    dataset = client.read_dataset(dataset_name=DATASET_NAME)
client.create_examples(dataset_id=dataset.id, examples=[{
    "inputs": {"question": "서울 날씨 알려줘"},
    "outputs": {"trajectory": reference_trajectory},
}])

def predict_agent_trajectory(inputs: dict) -> dict:
    return {"trajectory": actual_trajectory}

def trajectory_evaluator(inputs, outputs, reference_outputs):
    return trajectory_superset(
        outputs=outputs["trajectory"],
        reference_outputs=reference_outputs["trajectory"],
    )

results = evaluate(
    predict_agent_trajectory,
    data=DATASET_NAME,
    evaluators=[trajectory_evaluator],
    experiment_prefix="agent-eval-v1:gpt-5.4",
    metadata={"model": "gpt-5.4", "kind": "trajectory"},
)
print("Experiment:", results)
```

운영 기준

기준	권장
회귀 테스트	핵심 tool path는 `strict` 또는 `unordered`
유연한 agent	보조 호출을 허용하려면 `superset`
인자 비교	중요 필드는 `subset` , 전체 계약은 `exact`
reference 없음	LLM-as-judge로 합리성 평가
프로덕션 연결	LangSmith dataset + experiment prefix + 모델 metadata

REFERENCES

- LangChain Agent Evals: https://docs.langchain.com/oss/python/langchain/test/evals
- LangSmith Evaluation: https://docs.langchain.com/langsmith/evaluation
- AgentEvals GitHub: https://github.com/langchain-ai/agentevals

07 Testing Strategy

unit, integration, eval을 분리하기

학습 목표

LEARNING OBJECTIVES

- → Agent 테스트를 unit, integration, eval로 분리합니다.
- → 외부 API 없이 deterministic unit test를 작성합니다.
- → Agent Evals와 smoke test가 맡는 역할을 구분합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from langchain_core.language_models.fake_chat_models import FakeListChatModel
from langchain_core.messages import HumanMessage

model = FakeListChatModel(responses=["테스트 응답"])
model.invoke([HumanMessage(content="ping")]).content
```

7.1 테스트 피라미드

층	목적	외부 서비스
unit	함수, prompt, parser, graph node	없음
integration	provider, DB, retriever, server 연결	opt-in
eval	trajectory/품질/회귀 평가	선택적 LLM judge

7.2 Unit test: 순수 함수부터 고정

LLM 호출 전에 입력 정규화, tool routing, state update 같은 작은 단위를 먼저 고정합니다.

```
def normalize_question(text: str) -> str:
    return " ".join(text.strip().lower().split())

assert normalize_question("  Hello   Agent  ") == "hello agent"
print("unit test passed")
```

7.3 Integration gate

외부 서비스 테스트는 키가 있을 때만 실행합니다. 기본 smoke에서 실패하면 안 됩니다.

```
def can_run_openai_integration() -> bool:
    return bool(os.getenv("OPENAI_API_KEY")) and os.getenv("RUN_LIVE_TESTS") == "1"

print("run live integration:", can_run_openai_integration())
```

7.4 Eval은 “정답 문자열”보다 경로를 본다

Agent는 같은 답을 여러 문장으로 말할 수 있으므로, tool trajectory와 rubric을 함께 봅니다.

```
trajectory = [
    {"role": "user", "content": "서울 날씨"},
    {"role": "assistant", "tool_calls": [{"name": "get_weather"}]},
    {"role": "tool", "name": "get_weather", "content": "맑음"},
]

assert trajectory[1]["tool_calls"][0]["name"] == "get_weather"
```

7.5 CI/수동 검증 분리

검증	기본 CI	수동 opt-in
notebook JSON/cell ID	yes	yes
deterministic unit cells	yes	yes
provider integration	no	`RUN_LIVE_TESTS=1`
LangSmith mutation	no	`--allow-langsmith-mutations`

정리

항목	내용

다룬 기술	fake chat model, unit assertion, integration gate, trajectory check
핵심 개념	테스트 실패 원인을 작게 만들려면 unit/integration/eval을 섞지 않아야 합니다.
다음 단계	`06_agent_evals.ipynb` , `08_langgraph_testing.ipynb` 후보

REFERENCES

- `docs/langchain/test/index.md`
- `docs/langchain/test/unit-testing.md`
- `docs/langchain/test/integration-testing.md`
- `docs/langchain/test/evals.md`
- `docs/langgraph/test.md`

08 LangGraph Testing

state와 checkpoint를 검증하기

학습 목표

LEARNING OBJECTIVES

- → LangGraph node와 compiled graph를 분리해 테스트합니다.
- → state assertion으로 회귀를 잡습니다.
- → checkpointer가 필요한 테스트와 필요 없는 테스트를 구분합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class CounterState(TypedDict):
    count: int
```

8.1 node unit test

node는 순수 함수처럼 입력 state와 출력 update를 검증합니다.

```
def increment(state: CounterState) -> dict:
    return {"count": state["count"] + 1}

assert increment({"count": 1}) == {"count": 2}
print("node unit test passed")
```

8.2 graph integration test

compile된 graph는 end-to-end state를 검증합니다.

```
builder = StateGraph(CounterState)
builder.add_node("increment", increment)
builder.add_edge(START, "increment")
builder.add_edge("increment", END)
graph = builder.compile()

assert graph.invoke({"count": 0})["count"] == 1
```

8.3 checkpoint smoke

thread_id가 필요한 실행은 config까지 테스트해야 합니다.

```
checkpointer = InMemorySaver()
graph_with_memory = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "test-1"}}

result = graph_with_memory.invoke({"count": 2}, config=config)
assert result["count"] == 3
```

정리

항목	내용
다룬 기술	node unit test, graph integration test, checkpoint smoke
핵심 개념	LangGraph 테스트는 state update와 compiled graph behavior를 따로 검증합니다.

REFERENCES

- `docs/langgraph/test.md`
- `docs/langgraph/checkpointers.md`
- `docs/langchain/test/unit-testing.md`

09 Runtime Rubric Evaluation

실행 중 품질 게이트 만들기

학습 목표

LEARNING OBJECTIVES

- runtime rubric과 offline eval의 차이를 구분합니다.
- criterion별 feedback을 만들어 revision loop의 입력으로 사용하는 구조를 익힙니다.
- deterministic evaluator로 비용 없는 품질 게이트를 연습합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

9.1 Rubric criteria

Rubric은 최종 답변 직후 확인할 수 있는 명시적 기준이어야 합니다.

```
criteria = {
    "has_sources": lambda text: "참고" in text,
    "has_tests": lambda text: "검증" in text,
    "has_next_step": lambda text: "다음" in text,
}

list(criteria)
```

9.2 evaluator

criterion별 실패 사유를 revision prompt로 되돌릴 수 있게 만듭니다.

```
def grade(text: str) -> dict:
    checks = {name: fn(text) for name, fn in criteria.items()}
    feedback = [name for name, passed in checks.items() if not passed]
    return {"passed": all(checks.values()), "checks": checks, "feedback": feedback}
```

```
grade("요약과 다음 단계는 있지만 검증이 없습니다.")
```

9.3 offline eval과의 관계

방식	시점	목적
runtime rubric	응답 직후	self-revision, 필수 섹션 누락 방지
offline eval	batch/CI	regression, model 비교, 장기 품질 추세
smoke test	PR/로컬	실행 오류 탐지

정리

항목	내용
다룬 기술	rubric criteria, deterministic grader, feedback list
핵심 개념	runtime rubric은 모델 품질 평가가 아니라 완료 조건 미충족을 즉시 되돌리는 게이트입니다.

REFERENCES

- `docs/deepagents/rubric.md`
- `docs/langchain/test/evals.md`
- `docs/langsmith` 관련 LangSmith 노트북

PART

VII

실전 응용

Real-World Applications

이 파트에서 다루는 내용

- 1. RAG 에이전트
- 2. SQL 에이전트
- 3. 데이터 분석 에이전트
- 4. 머신러닝 에이전트
- 5. 딥 리서치 에이전트
- 6. 멀티모달 PDF RAG
- 7. Content Builder 에이전트
- 10. Personal Assistant Subagents
- 11. Customer Support Handoffs
- 12. Router Knowledge Base
- 13. Skills SQL Assistant

01 RAG 에이전트

5 building blocks + 3-Node 패턴

학습 목표

LEARNING OBJECTIVES

- → LangChain RAG의 5 building blocks(document loaders / text splitters / embedding models / vector stores / retrievers)를 이해한다
- → 2-Step / Agentic / Hybrid 세 가지 RAG 아키텍처의 차이를 안다
- → `Rewrite → Retrieve → Agent` 3-node 패턴으로 질의 재작성과 검색을 분리한다
- → `content_and_artifact` 반환 형식으로 검색 도구를 정의한다
- → `create_deep_agent` 로 RAG 에이전트를 만들고 v1 미들웨어를 적용한다
- → Skills 시스템으로 RAG 도메인 지식을 점진적 공개(Progressive Disclosure)한다

개요

항목	내용
프레임워크	LangChain v1 + Deep Agents
5 building blocks	Document loaders · Text splitters · Embedding models · Vector stores · Retrievers
아키텍처	2-Step (정적 RAG) · Agentic (도구 호출) · Hybrid (Rewrite → Retrieve → Agent)
에이전트 패턴	`content_and_artifact` 도구 → `create_deep_agent`
백엔드	`FilesystemBackend(root_dir=".", virtual_mode=True)`
스킬	`skills/rag-agent/SKILL.md` — RAG 도메인 지식 점진적 공개

> · NOTE
>
> 참고: `docs/langchain/24-retrieval.md` — 2-Step / Agentic / Hybrid RAG 아키텍처 비교

```python
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```python
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

RAG 5 building blocks

`docs/langchain/24-retrieval.md` 가 정의하는 다섯 가지 구성 요소를 노트북 흐름에 매핑하면 다음과 같습니다.

#	컴포넌트	역할	본 노트북 매핑
1	Document loaders	외부 소스에서 표준화된 `Document` 객체로 데이터를 적재	1단계: 6개 `Document` 직접 생성
2	Text splitters	큰 문서를 검색 가능한 청크로 분할	2단계: `RecursiveCharacterTextSplitter`
3	Embedding models	의미적으로 가까운 텍스트가 모이도록 벡터로 변환	3단계: `OpenAIEmbeddings(text-embedding-3-small)`
4	Vector stores	임베딩 저장 + 유사도 검색	3단계: `InMemoryVectorStore`
5	Retrievers	비정형 질의로부터 관련 문서 반환	4단계: `retrieve` 도구 (`similarity_search`)

세 가지 RAG 아키텍처

아키텍처	검색 시점	지연 시간	유연성	적합 시나리오
2-Step RAG	항상 검색 후 생성	Fast	Low	FAQ, 문서 봇
Agentic RAG	에이전트가 필요할 때 도구 호출	Variable	High	리서치 어시스턴트, 다중 도구
Hybrid RAG	Rewrite → Retrieve → Agent + 검증	Variable	High	품질 검증이 필요한 도메인

본 노트북은 Agentic RAG(에이전트가 `retrieve` 도구를 호출)를 기본으로 하고, 마지막에 `Rewrite → Retrieve → Agent` 3-node Hybrid 패턴을 한 번 더 보입니다.

1단계: 샘플 문서 생성

RAG 파이프라인의 첫 단계는 검색 대상 문서를 준비하는 것입니다. 실제 환경에서는 PDF, 웹 페이지, 데이터베이스 등에서 문서를 로드하지만, 여기서는 학습 목적으로 `Document` 객체를 직접 생성합니다.

```
from langchain_core.documents import Document

docs = [
    Document(page_content="LangChain은 LLM 애플리케이션 개발 프레임워크입니다. 도구, 체인, 에이전트를 지원합니다.",
metadata={"source": "langchain"}),
    Document(page_content="LangGraph는 상태 기반 워크플로를 구축하는 프레임워크입니다. 그래프 API와 Functional API
를 제공합니다.", metadata={"source": "langgraph"}),
    Document(page_content="Deep Agents는 올인원 에이전트 SDK입니다. create_deep_agent로 에이전트를 생성하고, 백엔
드와 서브에이전트를 지원합니다.", metadata={"source": "deepagents"}),
    Document(page_content="RAG는 검색 증강 생성의 약자로, 외부 지식을 LLM에 주입하여 정확한 응답을 생성합니다.",
metadata={"source": "rag"}),
    Document(page_content="벡터 스토어는 임베딩을 저장하고 유사도 검색을 수행하는 데이터베이스입니다. FAISS, Chroma
등이 있습니다.", metadata={"source": "vectorstore"}),
    Document(page_content="에이전트는 LLM이 도구를 사용하여 자율적으로 작업을 수행하는 시스템입니다. ReAct 패턴이 대
표적입니다.", metadata={"source": "agent"}),
]
print(f"문서 {len(docs)}개 생성 완료")
```

```
문서 6개 생성 완료
```

2단계: 텍스트 분할

큰 문서를 검색에 적합한 크기의 청크로 분할합니다. `RecursiveCharacterTextSplitter` 는 단락 → 문장 → 단어 순으로 자연스러운 경계에서 분할을 시도합니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=200, chunk_overlap=50
)
splits = splitter.split_documents(docs)
print(f"분할 결과: {len(splits)}개 청크")
```

```
분할 결과: 6개 청크
```

3단계: 벡터 스토어 구축

OpenAI 임베딩 모델로 텍스트를 벡터로 변환한 뒤 `InMemoryVectorStore` 에 저장합니다. 프로덕션에서는 FAISS나 Chroma 같은 영구 저장소를 씁니다.

```
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = InMemoryVectorStore.from_documents(splits, embeddings)
print(f"벡터 스토어 구축 완료 — {len(splits)}개 문서 임베딩됨")
```

```
벡터 스토어 구축 완료 — 6개 문서 임베딩됨
```

4단계: 검색 도구 정의 (content_and_artifact)

`response_format="content_and_artifact"` 패턴은 도구가 두 가지를 반환하게 합니다:

- content: 에이전트에게 보여줄 텍스트 요약
- artifact: 전체 Document 객체 (후속 처리용)

에이전트의 컨텍스트를 절약하면서 원본 데이터에도 접근할 수 있습니다.

```
from langchain.tools import tool

@tool(response_format="content_and_artifact")
def retrieve(query: str):
    """벡터 스토어에서 관련 문서를 검색합니다."""
    results = vectorstore.similarity_search(query, k=3)
    content = "\n\n".join(d.page_content for d in results)
    return content, results
```

5단계: 검색 도구 단독 테스트

에이전트에 연결하기 전에 도구가 올바르게 동작하는지 확인합니다.

```
result = retrieve.invoke({"query": "에이전트란 무엇인가?"})
print(result)
```

```
에이전트는 LLM이 도구를 사용하여 자율적으로 작업을 수행하는 시스템입니다. ReAct 패턴이 대표적입니다.

Deep Agents는 올인원 에이전트 SDK입니다. create_deep_agent로 에이전트를 생성하고, 백엔드와 서브에이전트를 지원합니다.

벡터 스토어는 임베딩을 저장하고 유사도 검색을 수행하는 데이터베이스입니다. FAISS, Chroma 등이 있습니다.
```

6단계: RAG 에이전트 생성 (v1 미들웨어 적용)

에서 프롬프트를 로드합니다. LangSmith Hub → Langfuse → 기본값 순으로 시도합니다.

미들웨어	역할
\	무한 루프 방지 — 최대 모델 호출 횟수 제한

\	검색 도구 실패 시 자동 재시도

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
from langchain.agents.middleware import (
    ModelCallLimitMiddleware,
    ToolRetryMiddleware,
)
from prompts import RAG_AGENT_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[retrieve],
    system_prompt=RAG_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    middleware=[
        ModelCallLimitMiddleware(run_limit=10),
        ToolRetryMiddleware(max_retries=2),
    ],
)
```

```
Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.

Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.

Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.

Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.

Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.
```

7단계: 단순 질의 및 비교 질의

단순 질의(하나의 검색)와 비교 질의(다중 검색)로 에이전트의 RAG 동작을 확인합니다.

8단계: Hybrid RAG — `Rewrite → Retrieve → Agent` 3-node 패턴

Agentic RAG 는 에이전트가 한 번에 검색·답변을 다 결정하지만, 도메인 어휘가 풍부할수록 질의 재작성(rewrite) 을 분리하면 재현율이 올라갑니다. LangGraph 의 `StateGraph` 로 3개 노드를 명시적으로 구성합니다.

```
사용자 입력
   │
   ▼
[1] rewrite   — 모델이 질의를 검색용 키워드로 재작성
   │
   ▼
[2] retrieve  — 벡터 스토어에서 후보 문서 K개 조회
   │
   ▼
[3] agent     — create_deep_agent 가 근거 기반 답변 생성
```

`retrieve` 도구는 4단계에서 이미 정의했으므로 그대로 재사용합니다.

요약

항목	핵심
5 building blocks	loaders · splitters · embeddings · vector stores · retrievers — RAG 표준 구성
벡터 스토어	`InMemoryVectorStore.from_documents()` — 임베딩 기반 유사도 검색
검색 도구	`\@tool(response_format="content_and_artifact")` — 요약 + 원본 분리
Agentic RAG	`create_deep_agent(model, tools=[retrieve], backend=..., skills=["/skills/"])`
Hybrid RAG	`Rewrite → Retrieve → Agent` 3-node `StateGraph`
스킬	`skills/rag-agent/SKILL.md` — Progressive Disclosure 로 토큰 절약

REFERENCES

- `docs/langchain/24-retrieval.md` — 5 building blocks · 2-Step / Agentic / Hybrid 아키텍처
- LangChain RAG Tutorial
- `docs/deepagents/10-skills.md`

다음 단계: → 02_sql_agent.ipynb: SQL 에이전트를 구축합니다.

02 SQL 에이전트

자연어 데이터베이스 질의 + HITL 4-decision

학습 목표

LEARNING OBJECTIVES

- → `SQLDatabaseToolkit` 으로 SQL 도구 4종을 자동 생성한다
- → AGENTS.md / Skills 기반 안전 규칙(READ-ONLY)을 적용한다
- → HITL 4-decision(`approve` / `edit` / `reject` / `respond`)을 모두 시연한다
- → `interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}` 로 도구별 정책을 좁힌다
- → `version="v2"` 호출과 `Command(resume={"decisions": [...]})` 재개 패턴을 익힌다

개요

항목	내용
프레임워크	LangChain v1 + Deep Agents
핵심 컴포넌트	`SQLDatabaseToolkit` , `SQLDatabase` , `InMemorySaver`
에이전트 패턴	AGENTS.md 안전 규칙 + Skills 기반 워크플로
HITL	`interrupt_on={"sql_db_query": {"allowed_decisions":[...]}}` + `Command(resume={"decisions":[...]})`
호출 버전	`version="v2"` — `GraphOutput` 의 `.interrupts` 필요
데이터베이스	Chinook (SQLite)
스킬	`skills/sql-agent/SKILL.md` — SQL 안전 규칙 + 쿼리 워크플로

· NOTE

참고: `docs/langchain/17-human-in-the-loop.md` — HITL 4-decision 사양

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: 데이터베이스 연결

Chinook은 디지털 음악 스토어의 샘플 데이터베이스입니다. Artist, Album, Track, Invoice 등의 테이블을 포함합니다.

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///../05_advanced/Chinook.db")
print(f"테이블: {db.get_usable_table_names()}")
```

```
테이블: ['Album', 'Artist', 'Customer', 'Employee', 'Genre', 'Invoice', 'InvoiceLine', 'MediaType', 'Playlist',
'PlaylistTrack', 'Track']
```

2단계: SQLDatabaseToolkit 도구 생성

`SQLDatabaseToolkit`은 데이터베이스 연결에서 4개의 도구를 자동 생성합니다:

도구	설명
`sql_db_list_tables`	사용 가능한 테이블 목록 조회
`sql_db_schema`	테이블 스키마(DDL) 조회
`sql_db_query`	SQL 쿼리 실행
`sql_db_query_checker`	쿼리 실행 전 문법 검증

```
from langchain_community.agent_toolkits import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=model)
sql_tools = toolkit.get_tools()
for t in sql_tools:
    print(f"  {t.name}: {t.description[:60]}")
```

```
sql_db_query: Input to this tool is a detailed and correct SQL query, outp
  sql_db_schema: Input to this tool is a comma-separated list of tables, outp
  sql_db_list_tables: Input is an empty string, output is a comma-separated list o
  sql_db_query_checker: Use this tool to double check if your query is correct befor
```

3단계: 프롬프트 로드 (LangSmith / Langfuse / 기본값)

의 함수가 프롬프트를 로드합니다:

1. LangSmith Hub – 가 있으면 Hub에서 pull
2. Langfuse – 가 있으면 Langfuse에서 로드
3. 기본값 – 둘 다 없으면 코드에 정의된 기본 프롬프트 사용

SQL 에이전트 프롬프트에는 READ-ONLY 안전 규칙과 워크플로가 포함되어 있습니다.

```
from prompts import SQL_AGENT_PROMPT

print(SQL_AGENT_PROMPT)
```

```
Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.

Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.

Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.

Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.

Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

당신은 SQL 에이전트입니다.

## 워크플로
1. sql_db_list_tables로 테이블 목록을 확인하세요
2. sql_db_schema로 관련 테이블의 스키마를 조회하세요
3. SQL 쿼리를 작성하고 sql_db_query_checker로 검증하세요
4. sql_db_query로 실행하고 결과를 해석하세요

## 안전 규칙
- READ-ONLY: SELECT만 허용. INSERT, UPDATE, DELETE, DROP 금지
- 항상 LIMIT 10을 사용하세요
- 쿼리 실행 전 반드시 스키마를 확인하세요
- 복잡한 쿼리는 write_todos로 단계별 계획을 세우세요
```

4단계: Skills 개념

Skills는 에이전트의 워크플로 가이드입니다. 반복되는 작업 패턴을 문서화하여 에이전트가 일관된 방식으로 작업하도록 합니다.

스킬	용도
`query-writing`	테이블 확인 → 스키마 조회 → SQL 작성 → 실행
`schema-exploration`	테이블 목록 → DDL 조회 → 관계 매핑

5단계: 기본 SQL 에이전트 생성

`create_deep_agent`에 SQL 도구와 AGENTS.md를 전달하여 에이전트를 생성합니다. `system_prompt`로 AGENTS.md 내용을 주입합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
)
```

6단계: HITL 에이전트 (4-decision 정책)

`create_deep_agent`의 `interrupt_on` 파라미터로 도구별 승인 정책을 설정합니다. `sql_db_query` 호출 전에 실행이 중단되고, `Command(resume={"decisions": [...]})` 로 재개합니다. 호출 시 `version="v2"` 가 필요합니다.

결정 유형	페이로드	동작
`approve`	`{"type": "approve"}`	도구를 그대로 실행
`edit`	`{"type": "edit", "edited_action": {"name":"sql_db_query", "args": {"query":"..."}}}`	인자 수정 후 실행
`reject`	`{"type": "reject", "message": "..."}`	실행 거부 + 에이전트에 피드백
`respond`	`{"type": "respond", "message": "..."}`	도구 호출 생략, 사람 응답이 ToolMessage가 됨

여기서는 위험한 데이터 변경 도구가 없으므로 `sql_db_query` 에 대해 네 결정 모두를 허용합니다. 운영 환경에서는 `allowed_decisions=["approve","reject"]` 처럼 좁혀 두는 것이 안전합니다.

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import ModelCallLimitMiddleware

hitl_agent = create_deep_agent(
    model=model,
    tools=sql_tools,
    system_prompt=SQL_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    interrupt_on={
        "sql_db_query": {
            "allowed_decisions": ["approve", "edit", "reject", "respond"],
        },
    },
    middleware=[
        ModelCallLimitMiddleware(run_limit=15),
    ],
)
```

7단계: 4-decision 시연

각 결정 유형이 어떻게 동작하는지 개별 thread 에서 확인합니다.

7-1. `approve` — 제안된 SQL 그대로 실행

7-2. `edit` — SQL 인자를 수정한 뒤 실행

검토자가 더 엄격한 `LIMIT` 을 강제하거나 컬럼을 추가하고 싶을 때 사용합니다. `edited_action.args` 로 도구 인자를 덮어씁니다.

7-3. `reject` — 실행 거부 후 에이전트에 피드백

`message` 가 ToolMessage 로 추가되어 에이전트가 다음 사고에서 이를 반영합니다.

7-4. `respond` — 도구 실행을 건너뛰고 사람이 직접 답변

`respond` 는 도구 호출 자체를 생략합니다. 검토자의 `message` 가 그대로 ToolMessage 가 되어 에이전트의 다음 응답에 반영됩니다. “이번엔 굳이 쿼리하지 말고 내가 알려준 값을 써” 같은 시나리오에 적합합니다.

요약

항목	핵심
도구 생성	`SQLDatabaseToolkit(db, llm).get_tools()` → 4개 SQL 도구 자동 생성
안전 규칙	Skills + 프롬프트로 READ-ONLY 정책 적용
HITL 정책	`interrupt_on={"sql_db_query": {"allowed_decisions": [...]}}`
HITL 호출	`version="v2"` → `GraphOutput.interrupts` 검사 → `Command(resume={"decisions":[...]})` 재개
4-decision	`approve` (그대로) / `edit` (인자 수정) / `reject` (피드백 후 거부) / `respond` (도구 생략, 사람 답변)

REFERENCES

- `docs/langchain/17-human-in-the-loop.md` — HITL 4-decision · `version="v2"` 사양
- `docs/deepagents/examples/03-text-to-sql-agent.md`
- LangChain SQL Agent Tutorial

다음 단계: → 03_data_analysis_agent.ipynb: 데이터 분석 에이전트를 구축합니다.

03 데이터 분석 에이전트

코드 실행과 멀티턴 분석

학습 목표

LEARNING OBJECTIVES

- → `LocalShellBackend` 로 코드 실행 환경을 구성한다
- → `execute` 와 파일 도구를 데이터 분석 흐름에 연결한다
- → 계획이 필요한 분석에서는 `TodoListMiddleware` 로 `write_todos` 를 명시적으로 켠다
- → 스트리밍과 멀티턴 대화로 반복 분석을 수행한다
- → v1 미들웨어(`SummarizationMiddleware` , `ModelCallLimitMiddleware`)를 적용한다

개요

항목	내용
프레임워크	Deep Agents + LangChain v1
커스텀 도구	`\@tool` 로 정의한 `get_csv_path` , `run_pandas`
백엔드	`LocalShellBackend(virtual_mode=True)`
기본 도구	`execute` , `ls` , `read_file` 등 파일/실행 도구
계획 도구	`TodoListMiddleware` 를 추가한 경우 `write_todos` 사용
패턴	스트리밍 (`stream(subgraphs=True)`) + 멀티턴 대화
스킬	`skills/data-analysis/SKILL.md` — 분석 체크리스트 + 코드 실행 규칙
대안 패턴	`CodeInterpreterMiddleware` — QuickJS 인터프리터 (`docs/deepagents/17-interpreters.md`)

```
from dotenv import load_dotenv
import os
```

```
load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: 백엔드 비교

Deep Agents는 다양한 백엔드를 지원합니다. 데이터 분석에는 코드 실행이 필요하므로 `LocalShellBackend` 를 씁니다.

백엔드	파일 접근	코드 실행	용도
`StateBackend`	상태 내 저장	❌	스크래치패드
`FilesystemBackend`	로컬 디스크	❌	파일 읽기/쓰기
`LocalShellBackend`	로컬 디스크	✅ `execute`	데이터 분석, 코드 실행
Sandboxes (Modal 등)	격리 환경	✅	프로덕션

> ➡️ TIP
>
> ⚠️ `LocalShellBackend` 는 호스트 시스템에서 명령을 실행합니다. 반드시 `virtual_mode=True` 를 사용하세요.

```
from deepagents.backends import LocalShellBackend

backend = LocalShellBackend(root_dir=".", virtual_mode=True)
```

2단계: 분석용 CSV 파일 생성

에이전트가 `execute` 로 pandas 코드를 실행하려면 파일 시스템에 CSV 파일이 있어야 합니다. `write_file` 빌트인 도구로 에이전트가 직접 파일을 쓸 수도 있지만, 여기서는 미리 준비합니다.

```
import tempfile, os

# 임시 디렉토리에 CSV 파일 생성
tmp_dir = tempfile.mkdtemp()
csv_path = os.path.join(tmp_dir, "sales.csv")

CSV_DATA = """date,product,region,sales,quantity
2024-01-15,Widget A,서울,150000,30
2024-01-15,Widget B,부산,89000,18
2024-02-10,Widget A,서울,175000,35
2024-02-10,Widget C,대구,62000,12
```

```
2024-03-05,Widget B,서울,134000,27
2024-03-05,Widget A,부산,98000,20
2024-03-20,Widget C,서울,71000,14
2024-04-01,Widget A,대구,112000,22"""

with open(csv_path, "w", encoding="utf-8") as f:
    f.write(CSV_DATA.strip())
print(f"CSV 저장: {csv_path}")
```

```
CSV 저장: C:\Users\HEESU\AppData\Local\Temp\tmpspv1awp9\sales.csv
```

3단계: `\@tool` + pandas 통합 도구 정의

두 가지 커스텀 도구를 `@tool` 데코레이터로 정의합니다. `@tool` 은 함수 시그니처와 docstring 으로부터 자동으로 LangChain 도구 스키마를 만듭니다.

도구	역할
`get_csv_path`	CSV 파일 경로 반환
`run_pandas`	에이전트가 작성한 pandas 코드를 직접 실행

> · NOTE
>
> `run_pandas` 는 `exec` 로 코드를 실행하므로, 노트북 학습 환경 외에서는 `LocalShellBackend.execute` 나 `CodeInterpreterMiddleware` (QuickJS)·샌드박스 백엔드로 격리하는 것이 안전합니다.

```
from langchain.tools import tool
import io, contextlib

@tool
def get_csv_path() -> str:
    """분석 대상 CSV 파일의 경로를 반환합니다."""
    return csv_path

@tool
def run_pandas(code: str) -> str:
    """pandas Python 코드를 실행합니다. print()로 결과를 출력하세요."""
    import pandas as pd, numpy as np
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "csv_path": csv_path}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "실행 완료 (출력 없음)"
    except Exception as e:
        return f"오류: {e}"
```

4단계: 에이전트 생성 (v1 미들웨어)

`LocalShellBackend` 와 커스텀 도구(`get_csv_path` , `run_pandas`)를 조합합니다.

미들웨어	역할
`SummarizationMiddleware`	대화가 길어지면 이전 메시지를 자동 요약하여 컨텍스트 절약
`ModelCallLimitMiddleware`	무한 루프 방지 — 최대 20회 모델 호출 제한

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import DATA_ANALYSIS_PROMPT

agent = create_deep_agent(
    model=model,
    tools=[get_csv_path, run_pandas],
    system_prompt=DATA_ANALYSIS_PROMPT,
    backend=LocalShellBackend(root_dir=tmp_dir, virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(model=model, trigger=("messages", 10)),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)
```

```
Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.

Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.

Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.

Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.

Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.
```

5단계: pandas 코드 실행으로 분석

에이전트에게 분석을 요청하면, `get_csv_path`로 파일 경로를 확인한 뒤 `run_pandas`로 pandas 코드를 직접 작성하고 실행합니다.

```
에이전트 실행 흐름:
1. get_csv_path() → CSV 파일 경로 확인
2. run_pandas("import pandas as pd; ...") → pandas 코드 실행
3. 결과 해석 및 답변 생성
```

6단계: 멀티턴 후속 질문

같은 `thread_id`를 쓰면 이전 대화의 맥락을 유지한 채 후속 질문을 할 수 있습니다. 에이전트는 이전 분석 결과를 기억합니다.

요약

빌트인 도구	백엔드	설명
`read_file`	모든 백엔드	파일 읽기 (이미지 포함)
`write_file`	모든 백엔드	파일 쓰기 또는 기존 파일 교체
`edit_file`	모든 백엔드	파일 내용 치환
`delete`	delete 지원 백엔드	파일 또는 prefix 삭제
`ls`	모든 백엔드	디렉토리 목록
`glob`	모든 백엔드	패턴으로 파일 검색
`grep`	모든 백엔드	파일 내용 검색
`execute`	LocalShell, Sandbox	명령 실행
`write_todos`	TodoListMiddleware	명시적으로 추가한 경우 작업 계획 관리

데이터 분석 실행 흐름

```
1. [Planning]    TodoListMiddleware를 추가한 경우 write_todos로 분석 계획 작성
2. [Reading]     read_file로 CSV 구조 확인
3. [Execution]   execute로 pandas 코드 실행
4. [Iteration]   후속 질문으로 분석 심화
5. [Delivery]    결과 요약과 근거 보고
```

이 노트북의 핵심 실행 경로는 `LocalShellBackend(virtual_mode=True)`와 제한된 코드 실행입니다. 계획 도구는 복잡한 요청에서만 켭니다.

요약

항목	핵심 포인트
Backend	`LocalShellBackend(virtual_mode=True)` — 코드 실행 지원
빌트인 도구	`execute` + 파일 도구, 삭제는 권한 정책과 함께 검토
계획	`write_todos` 는 `TodoListMiddleware` 를 추가한 경우 사용
Streaming	`stream(subgraphs=True)` — 실행 과정을 실시간 관찰
Multi-turn	`InMemorySaver` + 같은 `thread_id` — 대화 맥락 유지

> **REFERENCES**
>
> - `docs/deepagents/tutorials/data-analysis.md`
> - `docs/deepagents/06-backends.md`
>
> 다음 단계: → 04_ml_agent.ipynb: 머신러닝 에이전트를 구축합니다.

04 머신러닝 에이전트

`@tool` + scikit-learn 워크플로

학습 목표

LEARNING OBJECTIVES

- → `FilesystemBackend` 로 데이터 디렉토리를 설정하고, 에이전트가 자유롭게 파일을 탐색한다
- → NB03 의 `run_pandas` 패턴을 확장해 `@tool` + scikit-learn 을 통합한 `run_ml_code` 도구를 만든다
- → 에이전트가 빌트인 도구(`ls` , `read_file` , `glob`)로 데이터를 탐색하고 `run_ml_code` 로 학습·평가한다
- → 멀티턴 대화로 EDA → 전처리 → 모델 선택 → 학습 → 평가를 수행한다

개요

항목	NB03 (데이터 분석)	NB04 (머신러닝)
백엔드	`LocalShellBackend`	`FilesystemBackend`
데이터	매출 CSV (8행)	사용자 지정 CSV (데모: 유방암 569행)
커스텀 도구	`\@tool` + pandas — `get_csv_path` + `run_pandas`	`\@tool` + scikit-learn — `run_ml_code`
빌트인 도구	—	`ls` , `read_file` , `glob` (파일 탐색)
목적	집계, 통계, 추이 분석	EDA → 전처리 → 모델 학습 → 비교

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

NB03 vs NB04: 백엔드와 도구 확장

NB03에서는 `LocalShellBackend` + `run_pandas`로 pandas 코드를 실행했습니다. NB04에서는 두 가지를 확장합니다:

1. 백엔드: `FilesystemBackend(root_dir=DATA_DIR)` — 에이전트가 빌트인 도구(`ls`, `read_file`, `glob`)로 데이터 디렉토리를 자유롭게 탐색
2. 도구: `run_ml_code` — sklearn을 네임스페이스에 추가하여 ML 파이프라인 실행

```
# NB03: LocalShellBackend + run_pandas
backend = LocalShellBackend(root_dir=tmp_dir, virtual_mode=True)
ns = {"pd": pd, "np": np, "csv_path": csv_path}

# NB04: FilesystemBackend + run_ml_code
backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
ns = {"pd": pd, "np": np, "sklearn": sklearn, "DATA_DIR": DATA_DIR}
```

> ➡️ TIP
>
> `FilesystemBackend`는 `execute` 없이 파일 접근만 제공하므로 `LocalShellBackend`보다 안전합니다.

1단계: 데이터 디렉토리 설정

`DATA_DIR`을 변경하면 자신의 CSV 데이터를 사용할 수 있습니다. 에이전트가 `ls`, `glob`, `read_file` 빌트인으로 디렉토리를 탐색하여 어떤 파일이 있는지 파악합니다.

```
# 예시: 자신의 데이터 디렉토리 사용
DATA_DIR = "/path/to/your/data"
```

```
import tempfile
import pandas as pd
from sklearn.datasets import load_breast_cancer

# ── 데이터 디렉토리 설정 ──────────────────────────
# 자신의 CSV가 있는 디렉토리로 변경하세요.
# 아래는 데모용으로 breast_cancer 데이터를 CSV로 저장합니다.
DATA_DIR = tempfile.mkdtemp()

# 데모 데이터 생성 (자신의 CSV가 있으면 이 블록을 제거하세요)
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
df.to_csv(os.path.join(DATA_DIR, "breast_cancer.csv"), index=False)

print(f"DATA_DIR: {DATA_DIR}")
print(f"파일 목록: {os.listdir(DATA_DIR)}")
```

2단계: FilesystemBackend 생성

`FilesystemBackend`는 `root_dir` 아래의 파일에 대해 빌트인 도구를 제공합니다:

빌트인 도구	역할
`ls`	디렉토리 목록 조회
`read_file`	파일 내용 읽기
`glob`	패턴 기반 파일 검색
`write_file`	파일 쓰기 (결과 저장)

> ➡️ TIP
>
> `virtual_mode=True` 로 디렉토리 탈출(`..` , `~`)을 방지합니다.

```
from deepagents.backends import FilesystemBackend

backend = FilesystemBackend(root_dir=DATA_DIR, virtual_mode=True)
```

3단계: `\@tool` + scikit-learn 통합 도구 정의

NB03 의 `run_pandas` 를 확장해 `sklearn` 을 네임스페이스에 추가합니다. `@tool` 데코레이터로 함수를 도구로 등록하면 docstring 이 곧 모델에 노출되는 도구 설명이 됩니다.

`DATA_DIR` 을 네임스페이스에 전달해 에이전트가 디렉토리 내 어떤 CSV 든 로드할 수 있게 합니다.

> ➡️ TIP
>
> 파일 탐색은 빌트인 `ls` / `read_file` 로, 코드 실행은 `run_ml_code` 로 — 역할 분리

```
from langchain.tools import tool
import io, contextlib

@tool
def run_ml_code(code: str) -> str:
    """sklearn/pandas Python 코드를 실행합니다. print()로 결과를 출력하세요.
    사용 가능: pd, np, sklearn, os. DATA_DIR 변수로 데이터 디렉토리에 접근하세요."""
    import pandas as pd, numpy as np, sklearn
    buf = io.StringIO()
    ns = {"pd": pd, "np": np, "sklearn": sklearn, "os": os, "DATA_DIR": DATA_DIR}
    try:
        with contextlib.redirect_stdout(buf):
            exec(code, ns)
        return buf.getvalue() or "실행 완료 (출력 없음)"
    except Exception as e:
        return f"오류: {e}"
```

4단계: 에이전트 생성

에이전트의 워크플로:

1. 빌트인 `ls` / `glob` 으로 `DATA_DIR` 내 CSV 파일 탐색
2. 빌트인 `read_file` 로 데이터 미리보기
3. `run_ml_code` 로 EDA → 전처리 → 모델 학습/비교

미들웨어	역할
`ToolRetryMiddleware`	도구 실패 시 자동 재시도 (최대 2회)
`ModelCallLimitMiddleware`	무한 루프 방지 — 최대 20회 모델 호출 제한

```
from deepagents import create_deep_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    ToolRetryMiddleware,
    ModelCallLimitMiddleware,
)
from prompts import ML_AGENT_PROMPT

ml_agent = create_deep_agent(
    model=model,
    tools=[run_ml_code],
    system_prompt=ML_AGENT_PROMPT,
    backend=backend,
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        ToolRetryMiddleware(max_retries=2),
        ModelCallLimitMiddleware(run_limit=20),
    ],
)
```

5단계: 파일 탐색 + EDA 분석

에이전트에게 데이터 디렉토리를 탐색하고 분석하도록 요청합니다. 에이전트는 빌트인 `ls` 로 파일 목록을 확인한 뒤, `run_ml_code` 로 EDA를 수행합니다.

6단계: 모델 학습 + 비교

에이전트에게 적절한 모델 3개 이상을 학습하고 교차 검증으로 성능을 비교하도록 요청합니다. 에이전트가 스스로 알고리즘을 선택합니다.

7단계: 멀티턴 후속 — Feature Importance 분석

같은 `thread_id` 를 써서 이전 대화의 맥락을 유지한 채 후속 분석을 요청합니다.

8단계: 스트리밍 — 추가 분석

`stream(subgraphs=True)` 으로 에이전트의 실행 과정을 실시간으로 관찰합니다.

요약

항목	핵심
백엔드	`FilesystemBackend(root_dir=DATA_DIR)` — 사용자 데이터 디렉토리 설정
빌트인 도구	`ls` , `read_file` , `glob` — 파일 탐색
커스텀 도구	`\@tool` + scikit-learn — `run_ml_code` (pandas + numpy + sklearn)
워크플로	파일 탐색 → EDA → 전처리 → 모델 선택 → 교차 검증 비교
멀티턴	`InMemorySaver` + 동일 `thread_id` — 대화 맥락 유지

자신의 데이터 사용하기

```
# 1단계 셀에서 DATA_DIR만 변경하면 됩니다
DATA_DIR = "/path/to/your/data"  # CSV 파일이 있는 디렉토리
```

에이전트가 `ls` 로 파일을 탐색하고, `run_ml_code` 로 자유롭게 분석합니다.

REFERENCES

- `docs/deepagents/06-backends.md`
- `docs/deepagents/tutorials/data-analysis.md`
- scikit-learn 공식 문서

다음 단계: → 05_deep_research_agent.ipynb: 딥 리서치 에이전트를 구축합니다.

05 딥 리서치 에이전트

`create_deep_agent` + TodoListMiddleware + 서브에이전트 dispatch

학습 목표

LEARNING OBJECTIVES

- → `create_deep_agent` 의 하네스와 서브에이전트 구성으로 리서치 에이전트를 만든다
- → `TodoListMiddleware` 를 명시적으로 추가해 `write_todos` 기반 계획을 세우고 `task` 도구로 서브에이전트에 dispatch 한다
- → `think_tool` 로 전략적 반성(strategic reflection)을 구현한다
- → 5단계 워크플로(Plan → Delegate → Synthesize → Verify → Report)를 설계한다
- → AsyncSubAgent 5-tool 패턴(`start_async_task` / `check_async_task` / `update_async_task` / `cancel_async_task` / `list_async_tasks`)을 안내한다
- → v1 미들웨어(`SummarizationMiddleware` , `ModelCallLimitMiddleware` , `ModelFallbackMiddleware`)를 적용한다

개요

항목	내용
프레임워크	Deep Agents
진입점	`create_deep_agent(model, tools, subagents, ...)` — 하네스가 파일 도구와 서브에이전트를 조립
계획 도구	`TodoListMiddleware` 를 명시적으로 추가해 `write_todos` 사용
위임 도구	빌트인 `task` — 동기 서브에이전트 dispatch (Plan → Delegate)
반성 도구	커스텀 `think_tool` — 검색 후 전략적 반성
서브에이전트	researcher-1, researcher-2, fact-checker (3개)
워크플로	Plan → Delegate → Synthesize → Verify → Report

백엔드	`FilesystemBackend(root_dir=".", virtual_mode=True)`
스킬	`skills/deep-research/SKILL.md` — 리서치 방법론 + 인용 규칙
비동기 확장	`AsyncSubAgent` 5-tool 패턴 (`docs/deepagents/12-async-subagents.md`)

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4")
```

1단계: think_tool — 전략적 반성 도구

`think_tool` 은 에이전트가 행동하기 전에 "생각"을 기록하는 도구입니다. 의사결정 품질을 높이는 데 씁니다:

- 검색 결과를 분석하고 다음 행동을 계획
- 수집된 정보의 충분성을 평가
- 서브에이전트에게 위임할 작업을 구체화

```
from langchain.tools import tool

@tool
def think_tool(thought: str) -> str:
    """전략적 반성 — 현재 상황을 분석하고 다음 행동을 계획합니다."""
    return f"Reflection recorded: {thought}"
```

2단계: web_search 도구 (간소화)

실제 딥 리서치에서는 Tavily API를 사용하지만, 여기서는 학습 목적으로 간소화된 검색 도구를 정의합니다.

```
@tool
def web_search(query: str) -> str:
    """웹 검색을 수행합니다 (시뮬레이션)."""
    results = {
        "AI agent": "AI 에이전트는 자율적으로 작업을 수행하는 시스템입니다. 2024년 이후 급성장 중입니다.",
        "LangGraph": "LangGraph는 상태 기반 워크플로 프레임워크입니다. Graph API와 Functional API를 지원합니다.",
        "Deep Agents": "Deep Agents는 올인원 에이전트 SDK입니다. 서브에이전트, 백엔드, 스킬을 지원합니다.",
    }
    for key, val in results.items():
        if key.lower() in query.lower():
            return val
    return f"'{query}'에 대한 검색 결과: 관련 정보를 찾을 수 없습니다."
```

3단계: 5단계 리서치 워크플로 프롬프트

`prompts.load_prompt()` 가 프롬프트를 로드합니다 (LangSmith Hub → Langfuse → 기본값).

단계	이름	사용 도구	설명
1	Plan	`write_todos`	`TodoListMiddleware` 로 노출한 계획 도구 사용
2	Delegate	`task`	서브에이전트에 dispatch (비교 분석 시 병렬 최대 3개)
3	Synthesize	`think_tool`	수집 정보를 통합·요약
4	Verify	`task(fact-checker)`	사실 검증
5	Report	(LLM)	최종 보고서 작성

TodoList + dispatch 패턴

`write_todos` 는 다단계 리서치에서 메인 에이전트의 계획–기억–진행 상황을 외부 상태로 분리합니다. 이 도구는 기본 전제가 아니라 `TodoListMiddleware()` 를 추가했을 때 사용할 수 있습니다.

```
write_todos([
  {"content": "LangGraph 핵심 개념 조사", "status": "in_progress"},
  {"content": "Deep Agents 핵심 개념 조사", "status": "pending"},
  {"content": "fact-checker 로 사실 검증", "status": "pending"},
  {"content": "최종 보고서 작성", "status": "pending"},
])

task(subagent_type="researcher-1", description="LangGraph 핵심 개념 조사 ...")
task(subagent_type="researcher-2", description="Deep Agents 핵심 개념 조사 ...")
```

`task` 는 동기 서브에이전트 dispatch 도구입니다. 메인 에이전트는 서브에이전트 결과를 받아 todos 상태를 갱신하고 다음 단계로 넘어갑니다.

```
from prompts import RESEARCH_AGENT_PROMPT

print(RESEARCH_AGENT_PROMPT)
```

```
Prompt 'rag-agent-label:production' not found during refresh, evicting from cache.

Prompt 'sql-agent-label:production' not found during refresh, evicting from cache.

Prompt 'data-analysis-agent-label:production' not found during refresh, evicting from cache.

Prompt 'ml-agent-label:production' not found during refresh, evicting from cache.

Prompt 'deep-research-agent-label:production' not found during refresh, evicting from cache.

당신은 박사급 딥 리서치 에이전트입니다.

## 워크플로
1. **Plan**: write_todos로 리서치 계획을 세우세요
2. **Delegate**: 서브에이전트에게 조사를 위임하세요 (비교 분석 시 병렬)
3. **Synthesize**: 수집된 정보를 통합하세요
4. **Verify**: fact-checker에게 사실 검증을 요청하세요
5. **Report**: 최종 보고서를 작성하세요

## 규칙
- 검색 후 반드시 think_tool로 반성하세요
- 서브에이전트는 최대 3개까지 병렬 실행
```

```
- 인용은 [1], [2] 형식으로, 출처 섹션을 포함하세요
- 단순 주제는 서브에이전트 1개, 비교 분석은 2-3개 사용하세요
```

4단계: 서브에이전트 3개 정의

딥 리서치 에이전트는 3개의 전문 서브에이전트를 씁니다:

서브에이전트	역할	도구
`researcher-1`	주제 조사 담당	web_search, think_tool
`researcher-2`	비교/보완 조사	web_search, think_tool
`fact-checker`	사실 검증 담당	web_search

```
researcher_1 = {
    "name": "researcher-1",
    "description": "주제에 대한 심층 조사를 수행합니다",
    "system_prompt": "당신은 리서치 전문가입니다. 주제를 깊이 조사하고 핵심 정보를 요약하세요. 검색 후 think_tool로
반성하세요.",
    "tools": [web_search, think_tool],
}
```

```
researcher_2 = {
    "name": "researcher-2",
    "description": "보완적 관점에서 추가 조사를 수행합니다",
    "system_prompt": "당신은 보완 리서처입니다. 다른 관점에서 추가 정보를 수집하세요. 검색 후 think_tool로 반성하세
요.",
    "tools": [web_search, think_tool],
}
```

```
fact_checker = {
    "name": "fact-checker",
    "description": "수집된 정보의 사실 여부를 검증합니다",
    "system_prompt": "당신은 팩트체커입니다. 제공된 정보의 정확성을 검증하고, 오류가 있으면 지적하세요.",
    "tools": [web_search],
}
```

5단계: 딥 리서치 에이전트 생성 (v1 미들웨어)

모든 도구와 서브에이전트를 조합하여 최종 에이전트를 생성합니다. v1 미들웨어로 안정성과 신뢰성을 높입니다:

미들웨어	역할

로 체크포인팅을 활성화하여 중단된 리서치를 재개할 수 있습니다.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend
```

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents.middleware import (
    SummarizationMiddleware,
    ModelCallLimitMiddleware,
    ModelFallbackMiddleware,
    TodoListMiddleware,
)

research_agent = create_deep_agent(
    model=model,
    tools=[web_search, think_tool],
    subagents=[researcher_1, researcher_2, fact_checker],
    system_prompt=RESEARCH_AGENT_PROMPT,
    backend=FilesystemBackend(root_dir=".", virtual_mode=True),
    skills=["/skills/"],
    checkpointer=InMemorySaver(),
    middleware=[
        TodoListMiddleware(),
        SummarizationMiddleware(model=model, trigger=("messages", 15)),
        ModelCallLimitMiddleware(run_limit=30),
        ModelFallbackMiddleware("openai:gpt-5.4-mini"),
    ],
)
```

6단계: 리서치 실행

에이전트에게 리서치 주제를 부여하면 5단계 워크플로를 자동으로 수행합니다.

7단계: 스트리밍 — 네임스페이스 추적

`stream(subgraphs=True)`로 메인 에이전트와 서브에이전트의 실행 과정을 네임스페이스별로 추적합니다. 어떤 서브에이전트가 언제 호출되는지 실시간으로 확인할 수 있습니다.

서브에이전트 설계 모범 사례

원칙	설명
명확한 설명	`description`을 구체적으로 작성 — 메인 에이전트가 위임 대상을 선택하는 기준
전문 프롬프트	`system_prompt`에 출력 형식, 제약, 워크플로 포함
최소 도구	필요한 도구만 할당 — 불필요한 도구는 혼란 유발
간결한 결과	서브에이전트가 요약을 반환하도록 지시 — 원시 데이터 전달 금지

확장: AsyncSubAgent 5-tool 패턴

본 노트북의 `task` 는 동기 dispatch 입니다. 서브에이전트가 끝날 때까지 메인 에이전트가 블록되며 사용자도 새 지시를 줄 수 없습니다. 장시간 리서치 + 도중 방향 전환 + 병렬 fan-out 이 필요하면 Deep Agents 0.5+

의 `AsyncSubAgent` 로 전환하면 됩니다. `AsyncSubAgentMiddleware` 가 슈퍼바이저에게 다음 5개 도구를 주입합니다.

도구	역할
`start_async_task`	서브에이전트 백그라운드 기동, task id 즉시 반환 (non-blocking)
`check_async_task`	상태 조회, 완료 시 최종 출력 추출
`update_async_task`	같은 thread 에 새 지시 주입 (mid-flight steering)
`cancel_async_task`	실행 취소, 태스크를 `cancelled` 로 마킹
`list_async_tasks`	추적 중인 모든 태스크의 live status 일괄 조회

```python
from deepagents import AsyncSubAgent, create_deep_agent

async_subagents = [
    AsyncSubAgent(
        name="researcher",
        description="장시간 정보 수집과 종합이 필요한 리서치 작업",
        graph_id="researcher",
    ),
]

agent = create_deep_agent(
    model=model,
    subagents=async_subagents,
    # AsyncSubAgentMiddleware 가 자동 부착됨
)
```

축	`task` (Sync)	AsyncSubAgent (Async)
실행	슈퍼바이저 블록	즉시 task id 반환, 슈퍼바이저 계속
결과 회수	자동 반환	`check_async_task` 폴링
Mid-task 지시	불가	`update_async_task` 로 가능
취소	불가	`cancel_async_task`
인프라 요구	없음	Agent Protocol 서버 (LangSmith Deployments / `langgraph dev`)
적합 작업	수초 수십초	분 시간 단위, 중간 개입

상세 패턴(ASGI vs HTTP transport, mid-flight steering, `async_tasks` state channel)은 `docs/deepagents/12-async-subagents.md` 를 참고하세요.

요약

항목	핵심

진입점	`create_deep_agent(model, tools, subagents, ...)` — 하네스가 파일 도구와 서브에이전트 구성 조립
TodoList	`TodoListMiddleware()`를 명시적으로 추가하면 `write_todos` 사용 가능
dispatch	빌트인 `task` — 동기 서브에이전트 호출 (Plan → Delegate)
think_tool	전략적 반성 — 검색 후 분석, 다음 행동 계획
서브에이전트	researcher-1, researcher-2, fact-checker 병렬 실행
워크플로	Plan → Delegate → Synthesize → Verify → Report
컨텍스트 관리	서브에이전트 결과만 메인에 전달 — 중간 과정 격리
비동기 확장	`AsyncSubAgent` 5-tool: start / check / update / cancel / list

REFERENCES

- `docs/deepagents/examples/02-deep-research.md`
- `docs/deepagents/07-subagents.md` — 동기 서브에이전트
- `docs/deepagents/12-async-subagents.md` — AsyncSubAgent 5-tool 패턴
- `docs/deepagents/06-backends.md`

이전 단계: ← 04_ml_agent.ipynb: 머신러닝 에이전트

06 멀티모달 PDF RAG

PyMuPDF4LLM + v1 content blocks

학습 목표

LEARNING OBJECTIVES

- → PyMuPDF4LLM 으로 PDF 슬라이드를 1장 = 1 Document 구조로 변환한다
- → 슬라이드 내 이미지와 표 를 추출하고 RAG 검색 텍스트에 합친다
- → 비전 지원 LLM 에 멀티모달 content blocks 로 이미지를 전달해 설명을 생성한다
- → LangChain v1 의 `output_version="v1"` 직렬화 옵션을 켜서 표준 content blocks (`TextContentBlock` / `ImageContentBlock`) 로 응답을 받는다
- → `create_agent()` 와 `content_and_artifact` 검색 도구로 질의응답을 구현한다

개요

항목	내용
대상 문서	Stanford CS224N 2026 Lecture 5: Attention and Transformers 공개 PDF
추출 도구	`pymupdf4llm.to_markdown(page_chunks=True, write_images=True)`
문서 단위	PDF 1페이지/슬라이드 → LangChain `Document` 1개
멀티모달 보강	추출 이미지 경로 + 슬라이드 렌더링 PNG + LLM 이미지 설명
Content blocks	`{"type":"text", ...}` + `{"type":"image_url", ...}` 의 멀티모달 메시지
응답 직렬화	`ChatOpenAI(..., output_version="v1")` — `TextContentBlock` / `ImageContentBlock` 표준 형식
RAG 구조	`OpenAIEmbeddings` → `InMemoryVectorStore` → `\@tool` → `create_agent()`
스킬	`skills/multimodal-rag/SKILL.md` — 슬라이드 기반 멀티모달 RAG 규칙

```
# [6-1] : 환경 설정
from dotenv import load_dotenv
import os
import pymupdf4llm

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

# output_version="v1" — 응답이 표준 content blocks 형식으로 직렬화됨
# (TextContentBlock / ImageContentBlock / ReasoningContentBlock 등)
# 환경 변수 LC_OUTPUT_VERSION="v1" 로도 동일 효과를 줄 수 있음.
model = ChatOpenAI(model="gpt-5.4", output_version="v1")
vision_model = model  # gpt-5.4 는 비전 지원
print("모델 준비 완료 — output_version=v1")
```

LangChain v1 멀티모달 content blocks

`HumanMessage.content` 는 세 가지 형식을 받습니다.

1. string — `"Transformer 의 self-attention 한계는?"` 처럼 단순 텍스트
2. dict list — OpenAI 호환 멀티모달 형식: `[{"type":"text", ...}, {"type":"image_url", ...}]`
3. 표준 content blocks — `TextContentBlock` , `ImageContentBlock` , `AudioContentBlock` , `FileContentBlock` 등 LangChain 의 타입세이프 표현

비전 모델에 슬라이드 이미지를 보낼 때는 형식 2 (`multimodal_pdf_rag.describe_image_with_llm` 가 내부에서 사용) 가 가장 직관적입니다.

```
message = [
    {"type": "text", "text": "이 슬라이드를 한국어로 설명하세요."},
    {"type": "image_url", "image_url": {"url": "...."}},
]
response = vision_model.invoke([{"role": "user", "content": message}])
```

응답을 받을 때 `output_version="v1"` 을 켜 두면 `response.content` 가 자동으로 표준 content blocks 로 직렬화되어, 외부 시스템이 `TextContentBlock` / `ImageContentBlock` 을 타입세이프하게 소비할 수 있습니다 (`docs/langchain/05-messages.md`).

1) 공개 PDF 다운로드

이 예제는 Stanford CS224N의 공개 PDF 슬라이드를 사용합니다. 원본 PDF와 생성 이미지는 저장소에 커밋하지 않고 `07_examples/.cache/` 아래 런타임 캐시에 둡니다.

```
import sys
from pathlib import Path

root = Path.cwd()
if (root / "07_examples").exists():
    sys.path.append(str(root / "07_examples"))
```

```
from multimodal_pdf_rag import (
    DEFAULT_PAGES, DEFAULT_SOURCE_URL, add_llm_image_descriptions,
    build_vectorstore, download_pdf, extract_pdf_metadata,
    extract_slide_documents, extract_slide_metadata, format_pdf_metadata,
    format_slide_metadata, format_sources, make_slide_search_tool,
    metadata_as_json,
)
```

```
pdf_path = download_pdf(DEFAULT_SOURCE_URL)
selected_pages = DEFAULT_PAGES
if os.environ.get("FAST_MULTIMODAL_RAG_TEST"):
    selected_pages = [19, 51, 58]
print(f"PDF: {pdf_path}")
print(f"대상 슬라이드(0-based): {selected_pages}")
```

2) PDF 파일 메타데이터 추출

PDF 자체의 제목, 작성자, 생성일, 페이지 수, 파일 크기, 페이지 크기 같은 메타데이터를 먼저 추출합니다. 이 값은 전체 코퍼스의 출처 추적과 필터링 조건으로 사용할 수 있습니다.

```
pdf_metadata = extract_pdf_metadata(pdf_path, source_url=DEFAULT_SOURCE_URL)
print(format_pdf_metadata(pdf_metadata))
print(metadata_as_json({"pages_sample": pdf_metadata["pages"][:2]}))
```

3) 슬라이드 단위 Document 생성

`page_chunks=True`로 페이지별 Markdown을 받고, `write_images=True`로 슬라이드 안의 그림 영역을 PNG로 저장합니다. 동시에 PyMuPDF로 각 슬라이드 전체를 대표 이미지로 렌더링합니다.

> · NOTE
>
> 전체 70장을 처리하려면 `pages=None`으로 바꾸면 됩니다. 노트북 기본값은 실행 시간과 비용을 줄이기 위해 핵심 슬라이드만 선택합니다.

```
documents = extract_slide_documents(
    pdf_path,
    source_url=DEFAULT_SOURCE_URL,
    pages=selected_pages,
)
print(f"생성된 Document 수: {len(documents)}")
print(format_sources(documents))
```

```
slide_metadata = extract_slide_metadata(documents)
print(format_slide_metadata(slide_metadata))
print(metadata_as_json(slide_metadata[:2]))
```

```
from IPython.display import Image, display

sample_doc = documents[0]
display(Image(filename=sample_doc.metadata["slide_image_path"], width=480))
print(sample_doc.metadata["extracted_image_paths"][:2])
```

4) 슬라이드별 메타데이터와 표/이미지 확인

PyMuPDF4LLM은 표를 GitHub Flavored Markdown 형태로 본문에 넣습니다. 헬퍼 모듈은 Markdown 표 블록을 다시 분리해 `metadata["table_count"]` 와 `## Extracted tables` 섹션에 보존합니다.

```
table_docs = [doc for doc in documents if doc.metadata["table_count"] > 0]
print(f"표가 감지된 슬라이드: {[d.metadata['slide'] for d in table_docs]}")
if table_docs:
    print(table_docs[0].page_content[:1200])
```

5) LLM 기반 이미지 설명 생성

이미지 자체는 벡터 검색에 바로 들어가기 어렵습니다. 따라서 슬라이드 대표 이미지와 일부 추출 이미지를 비전 지원 LLM으로 설명하게 한 뒤, 그 설명을 Document 본문에 합칩니다.

```
caption_limit = len(documents) + 4

documents = add_llm_image_descriptions(
    documents, vision_model,
    max_descriptions=caption_limit,
)
print(documents[0].metadata["image_descriptions"][0][:700])
```

v1 응답의 표준 content blocks 확인

`output_version="v1"` 을 켜 두면 비전 모델의 응답 메시지에서 `.content_blocks` 프로퍼티로 `TextContentBlock` 들을 꺼낼 수 있습니다. 이 표현은 provider 별 차이를 흡수하므로 후속 파이프라인(예: 캡션 캐싱, UI 렌더링) 이 안전하게 소비할 수 있습니다.

```
from multimodal_pdf_rag import _image_to_data_url  # 데모용 헬퍼

sample_image = documents[0].metadata["slide_image_path"]
vision_msg = [
    {"type": "text", "text": "이 슬라이드의 핵심 메시지를 한 문장으로 요약하세요."},
    {"type": "image_url", "image_url": {"url": _image_to_data_url(sample_image)}},
]
resp = vision_model.invoke([{"role": "user", "content": vision_msg}])

# v1: .content_blocks 는 표준 content block 리스트 (TextContentBlock 등)
blocks = getattr(resp, "content_blocks", None)
print("block 개수:", len(blocks) if blocks else "n/a")
if blocks:
    for b in blocks:
        print(f"  type={b.get('type'):>15}  | preview={str(b)[:120]}")
print()
print("응답 미리보기:", str(resp.content)[:300])
```

6) 벡터 인덱스와 검색 도구 구성

LangChain v1 구조에 맞춰 `InMemoryVectorStore`를 만들고, `@tool(response_format="content_and_artifact")`로 검색 본문과 원본 Document artifact를 함께 반환합니다.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = build_vectorstore(documents, embeddings)
search_slides = make_slide_search_tool(vectorstore, k=4)
print("벡터 인덱스와 검색 도구 준비 완료")
```

```
query = "self-attention을 Transformer 블록으로 쓰려면 어떤 문제가 있나요?"
hits = vectorstore.similarity_search(query, k=3)
print(format_sources(hits))
print(hits[0].page_content[:1400])
```

7) LangChain v1 `create_agent()`로 질의응답

검색을 도구로 노출하면 에이전트가 질문에 맞춰 슬라이드를 검색하고, 텍스트·표·이미지 설명을 함께 근거로 답변합니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from prompts import MULTIMODAL_RAG_AGENT_PROMPT
```

```
rag_agent = create_agent(
    model=model,
    tools=[search_slides],
    system_prompt=MULTIMODAL_RAG_AGENT_PROMPT,
    middleware=[ModelCallLimitMiddleware(run_limit=6)],
    checkpointer=InMemorySaver(),
)
```

정리

항목	핵심
슬라이드 단위 문서화	`page_chunks=True`로 PDF 1페이지를 `Document` 1개로 구성
이미지 처리	`write_images=True` 추출 이미지 + PyMuPDF 슬라이드 렌더링 + LLM 설명
메타데이터 처리	PDF 파일 메타데이터 + 슬라이드별 표/이미지/레이아웃 메타데이터 추출
표 처리	Markdown 표 블록을 분리해 본문과 메타데이터에 보존

멀티모달 메시지	`{"type":"text", ...}` + `{"type":"image_url", ...}` content blocks 로 비전 모델 호출
v1 직렬화	`ChatOpenAI(..., output_version="v1")` → `.content_blocks` 로 `TextContentBlock` 접근
LangChain v1 구조	`InMemoryVectorStore` + `\@tool(content_and_artifact)` + `create_agent()`
확장 포인트	`pages=None` 으로 전체 PDF 처리, Chroma/FAISS 로 영속화, 캡션 캐시 추가

REFERENCES

- `docs/langchain/05-messages.md` — content blocks · `output_version="v1"`
- `docs/langchain/24-retrieval.md` — RAG 5 building blocks
- `07_examples/skills/multimodal-rag/SKILL.md`
- PyMuPDF4LLM 공식 문서
- PyMuPDF4LLM API: `to_markdown`
- LangChain PyMuPDF4LLM integration
- Stanford CS224N 2026 Lecture 5 PDF

다음 단계: → 08_integration/04_document_loaders에서 PDF·웹·구조화 문서 로더를 비교합니다.

07 Content Builder Agent

AGENTS.md + Skills + Subagents

학습 목표

LEARNING OBJECTIVES

- → 공식 Deep Agents Content Builder 예제의 메모리·스킬·서브에이전트 구조를 이해한다
- → `AGENTS.md` 로 브랜드 보이스를 주입하고, `SKILL.md` 로 콘텐츠 워크플로를 온디맨드 로드한다
- → `FilesystemBackend(virtual_mode=True)` 로 결과물을 안전하게 파일로 남긴다
- → 외부 검색·이미지 생성 의존성을 선택 기능으로 분리해 OpenAI만으로 실행 가능한 텍스트 파이프라인을 만든다

개요

항목	내용
공식 패턴	Content Builder Agent — memory(`AGENTS.md`), skills, subagents
실행 범위	텍스트 전용 LinkedIn/블로그 초안 생성
안전 장치	`local/` 하위 출력, `virtual_mode=True` , 외부 이미지 생성은 reference-only
검증 방식	gpt-4.1 harness에서 실제 1회 실행

공식 예제는 Tavily 검색과 Gemini 이미지 생성을 포함하지만, 이 노트북은 01 07 실행 harness에 맞춰 OpenAI + 로컬 파일시스템만 필수로 둡니다.

```
from dotenv import load_dotenv
import os

load_dotenv()
assert os.environ.get("OPENAI_API_KEY"), "OPENAI_API_KEY를 .env에 설정하세요"
```

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5.4", temperature=0)
```

1) Content Builder 작업 공간

공식 예제의 핵심은 "코드에 모든 지침을 박아넣지 않고, 파일 시스템에 지침을 둔다"입니다.

- `AGENTS.md` — 브랜드 보이스와 작성 원칙, 항상 로드
- `skills/content-builder/SKILL.md` — 콘텐츠 작성 절차, 필요할 때 로드
- `research/`, `linkedin/`, `blogs/` — 에이전트가 산출물을 저장하는 폴더

```
from pathlib import Path
import shutil

WORK_DIR = Path("local/content_builder_demo")
if WORK_DIR.exists():
    shutil.rmtree(WORK_DIR)
(WORK_DIR / "skills/content-builder").mkdir(parents=True)
(WORK_DIR / "research").mkdir()
```

```
brand_voice = """# BAEUM.AI Content Voice
Write in Korean, professional but approachable.
Lead with practical value, keep paragraphs short, and end with action.
Focus on AI agents, developer productivity, and production reliability.
"""
(WORK_DIR / "AGENTS.md").write_text(brand_voice, encoding="utf-8")
```

```
skill_text = """---
name: content-builder
description: Use for blog, LinkedIn, or technical content drafts.
---
# Content Builder
Research first, then write a hook, 3 concise insights, and a CTA.
Save LinkedIn posts to linkedin/<slug>/post.md.
Save blog posts to blogs/<slug>/post.md.
"""
(WORK_DIR / "skills/content-builder/SKILL.md").write_text(skill_text, encoding="utf-8")
```

2) 로컬 리서치 도구와 researcher 서브에이전트

라이브 강의·자동 테스트에서는 외부 검색 API 키가 항상 있지 않습니다. 그래서 여기서는 로컬 knowledge base를 검색 도구처럼 감싼 뒤, researcher 서브에이전트가 이를 요약해 파일로 저장하게 합니다.

```
from langchain.tools import tool

@tool
def topic_brief(topic: str) -> str:
    """Return a concise local research brief for a content topic."""
    return (
        f"{topic}: Deep Agents combine memory, skills, filesystem tools, "
        "and subagents so content workflows become repeatable."
    )
```

```
researcher = {
    "name": "researcher",
    "description": "Research a topic and save concise findings before writing.",
    "system_prompt": "Use topic_brief, then write findings to the requested path.",
    "tools": [topic_brief],
}
```

3) Deep Agent 구성

`create_deep_agent()` 가 콘텐츠 작성 구성요소를 조립합니다.

구성	역할
`memory=["/AGENTS.md"]`	브랜드 보이스를 시스템 프롬프트에 항상 주입
`skills=["/skills/"]`	`SKILL.md` 워크플로를 필요할 때 로드
`subagents=[researcher]`	`task` 도구로 리서치 위임
`FilesystemBackend`	`write_file` , `read_file` 등 파일 도구 제공

스킬 자체가 별도 도구를 공개해야 한다면 `metadata.include_tools` 와 직접 구성한 `SkillsMiddleware(tools=...)` 를 사용합니다. 이 노트북은 기본 도구와 서브에이전트만으로 실행되므로 `skills=["/skills/"]` 경로를 유지합니다.

```
from deepagents import create_deep_agent
from deepagents.backends import FilesystemBackend

agent = create_deep_agent(
    model=model, tools=[topic_brief], subagents=[researcher],
    backend=FilesystemBackend(root_dir=str(WORK_DIR), virtual_mode=True),
    memory=["/AGENTS.md"], skills=["/skills/"],
    system_prompt="You are a content builder. Use files for durable outputs.",
)
```

4) 실행 — LinkedIn 포스트 생성

요청에는 네 가지를 명시합니다.

1. researcher 서브에이전트 사용
2. 리서치 저장 위치
3. 콘텐츠 저장 위치
4. 짧고 검증 가능한 산출물 범위

```
request = """Use the researcher subagent to research Deep Agents content workflows.
Save research to research/content-workflows.md.
Then write a Korean LinkedIn post with one hook, three bullets, and one CTA.
Save it to linkedin/content-workflows/post.md."""
result = agent.invoke({"messages": [{"role": "user", "content": request}]})
print(result["messages"][-1].content[:800])
```

```
post_path = WORK_DIR / "linkedin/content-workflows/post.md"
research_path = WORK_DIR / "research/content-workflows.md"
print("research exists:", research_path.exists())
print("post exists:", post_path.exists())
print(post_path.read_text(encoding="utf-8")[:900])
```

5) 이미지·외부 검색은 선택 기능으로 분리

프로덕션 Content Builder는 Tavily 검색과 이미지 생성 도구를 붙일 수 있습니다. 다만 01 07 기본 실행 테스트에서는 비용·키 의존성을 피하기 위해 reference-only로 둡니다.

```
optional_tools_example = r"""
# Optional production tools — not required for this notebook smoke run.
# Add Tavily or image generation only when the matching API keys exist.
agent = create_deep_agent(
    model=model,
    tools=[web_search, generate_cover, generate_social_image],
    memory=["/AGENTS.md"], skills=["/skills/"],
)
"""
print(optional_tools_example)
```

요약

항목	핵심 내용
Memory	`AGENTS.md` 로 브랜드 보이스를 항상 주입
Skills	`SKILL.md` 로 블로그/소셜 작성 절차를 온디맨드 로드
Subagents	researcher가 리서치를 분리 수행해 메인 컨텍스트를 절약
FilesystemBackend	콘텐츠 산출물을 `local/` 아래 파일로 보존
실행 정책	OpenAI만 필수, 검색·이미지는 optional

REFERENCES

- Deep Agents Content Builder: https://docs.langchain.com/oss/python/deepagents/content-builder
- Deep Agents Skills: ../docs/deepagents/10-skills.md
- Deep Agents Subagents: ../docs/deepagents/07-subagents.md
- Local reference: ../docs/deepagents/examples/01-content-builder-agent.md

10 Personal Assistant Subagents

역할별 위임 패턴

학습 목표

LEARNING OBJECTIVES

- → supervisor가 사용자 요청을 역할별 subagent로 나누는 방식을 이해합니다.
- → 각 subagent의 책임과 tool scope를 좁게 정의합니다.
- → 실제 LLM 호출 없이 routing과 fan-in 결과 형식을 검증합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

10.1 역할 정의

개인 비서는 하나의 거대한 agent보다 calendar, email, research처럼 역할을 나누면 안전합니다.

```
subagents = {
    "calendar": {"tools": ["check_availability"], "risk": "approval"},
    "email": {"tools": ["draft_reply"], "risk": "review"},
    "research": {"tools": ["search_notes"], "risk": "allow"},
}

subagents
```

10.2 요청 라우팅

실제 모델 라우터 대신 keyword router로 구조를 먼저 확인합니다.

```
def route_request(text: str) -> str:
    lowered = text.lower()
    if "meeting" in lowered or "일정" in lowered:
        return "calendar"
```

```
    if "email" in lowered or "메일" in lowered:
        return "email"
    return "research"

route_request("내일 일정 확인하고 메일 초안도 준비해줘")
```

10.3 복합 요청 분해

하나의 요청이 여러 역할을 건드리면 supervisor가 task list를 만듭니다.

```
def plan_tasks(text: str) -> list[dict]:
    tasks = []
    for name in subagents:
        if name == route_request(text) or name in text.lower():
            tasks.append({"subagent": name, "input": text})
    return tasks or [{"subagent": "research", "input": text}]

plan_tasks("calendar 확인 후 email 초안 작성")
```

10.4 fan-in 결과 합치기

subagent 결과는 사용자에게 바로 노출하기 전에 supervisor가 정리합니다.

```
results = [
    {"subagent": "calendar", "result": "화요일 3시 가능"},
    {"subagent": "email", "result": "회의 제안 메일 초안 생성"},
]

summary = "\n".join(f"- {r['subagent']}: {r['result']}" for r in results)
print(summary)
```

정리

항목	내용
다룬 기술	supervisor, subagent role, routing, fan-in
핵심 개념	subagent는 역할과 권한을 좁혀 전체 assistant의 위험을 줄입니다.
다음 단계	`11_customer_support_handoffs.ipynb`

REFERENCES

- `docs/langchain/multi-agent/subagents.md`
- `docs/langchain/multi-agent/subagents-personal-assistant.md`
- `docs/deepagents/subagents.md`

11 Customer Support Handoffs

상태 전환형 멀티에이전트

학습 목표

LEARNING OBJECTIVES

- → handoff가 subagent delegation과 어떻게 다른지 구분합니다.
- → 고객지원 상태 머신을 deterministic하게 설계합니다.
- → escalation과 approval 조건을 명시합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class SupportState(TypedDict):
    message: str
    owner: str
    resolution: str
```

11.1 Handoff 기준

handoff는 “누가 다음 응답의 주체인가”가 바뀌는 패턴입니다.

```
def triage(state: SupportState) -> dict:
    msg = state["message"].lower()
    if "refund" in msg or "환불" in msg:
        return {"owner": "billing"}
    if "error" in msg or "오류" in msg:
        return {"owner": "technical"}
    return {"owner": "general"}
```

```
def resolve(state: SupportState) -> dict:
    templates = {
```

```
        "billing": "환불 정책을 확인하고 승인 요청을 준비합니다.",
        "technical": "오류 재현 정보와 로그를 요청합니다.",
        "general": "기본 안내를 제공합니다.",
    }
    return {"resolution": templates[state["owner"]]}
```

11.2 그래프로 표현하기

LangGraph를 쓰면 handoff 기록이 state에 남고 테스트하기 쉽습니다.

```
builder = StateGraph(SupportState)
builder.add_node("triage", triage)
builder.add_node("resolve", resolve)
builder.add_edge(START, "triage")
builder.add_edge("triage", "resolve")
builder.add_edge("resolve", END)
graph = builder.compile()
```

```
result = graph.invoke({
    "message": "구독 환불을 요청합니다.",
    "owner": "", "resolution": "",
})
result
```

11.3 승인 게이트

조건	처리
환불/결제	human approval
기술 오류	로그 요청 후 재현
일반 문의	자동 응답 가능

```
def needs_approval(state: SupportState) -> bool:
    return state["owner"] == "billing"

print("approval required:", needs_approval(result))
```

정리

항목	내용
다룬 기술	handoff, state machine, approval gate

핵심 개념	handoff는 작업 위임이 아니라 대화 주체와 상태의 전환입니다.
다음 단계	`12_router_knowledge_base.ipynb`

> **REFERENCES**
>
> - `docs/langchain/multi-agent/handoffs.md`
> - `docs/langchain/multi-agent/handoffs-customer-support.md`
> - `docs/langgraph/workflows-agents.md`

12 Router Knowledge Base

질문을 알맞은 지식원으로 보내기

학습 목표

> **LEARNING OBJECTIVES**
>
> → router와 handoff/subagent의 차이를 구분합니다.
>
> → 여러 지식원을 source별 retriever처럼 다루는 구조를 만듭니다.
>
> → 라우팅 결과를 간단한 테스트 케이스로 검증합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

12.1 지식원 정의

지식원이 많아질수록 하나의 retriever보다 먼저 source를 고르는 router가 유용합니다.

```
knowledge_sources = {
    "billing": ["환불은 결제일로부터 7일 이내 가능합니다."],
    "technical": ["오류 신고에는 로그와 재현 단계가 필요합니다."],
    "product": ["Deep Agents는 planning, files, subagents를 포함합니다."],
}

list(knowledge_sources)
```

12.2 deterministic router

실제 서비스에서는 structured output router를 쓰되, 먼저 규칙 기반 router로 테스트 모양을 고정합니다.

```
def route_source(question: str) -> str:
    q = question.lower()
    if "환불" in q or "refund" in q:
        return "billing"
```

```
    if "오류" in q or "error" in q:
        return "technical"
    return "product"

route_source("Deep Agents 기능은?")
```

12.3 source-local search

라우터가 고른 source 안에서만 검색하면 context가 작아지고 근거 관리가 쉬워집니다.

```
def retrieve(question: str) -> dict:
    source = route_source(question)
    docs = knowledge_sources[source]
    return {"source": source, "documents": docs}

retrieve("환불 조건 알려줘")
```

12.4 라우터 평가

라우터는 답변 품질과 별도로 source 선택 정확도를 평가합니다.

```
cases = [
    ("환불 가능한가요?", "billing"),
    ("앱 오류가 납니다", "technical"),
    ("Deep Agents가 뭐예요?", "product"),
]

for question, expected in cases:
    actual = route_source(question)
    print(question, actual, actual == expected)
```

정리

항목	내용
다룬 기술	router, source-local retrieval, routing eval
핵심 개념	Router는 답변을 생성하기 전에 “어디에서 찾을지”를 결정하는 계층입니다.
다음 단계	`13_skills_sql_assistant.ipynb` 후보

REFERENCES

- `docs/langchain/multi-agent/router.md`
- `docs/langchain/multi-agent/router-knowledge-base.md`
- `docs/langchain/structured-output.md`

13 Skills SQL Assistant

필요할 때만 SQL 지침 로드하기

학습 목표

LEARNING OBJECTIVES

- → SKILL.md progressive disclosure를 SQL assistant에 적용합니다.
- → SQL 안전 규칙을 query 실행 전 checklist로 검증합니다.
- → 실제 DB mutation 없이 read-only SQL 흐름을 연습합니다.

```
from dotenv import load_dotenv
import os

load_dotenv(override=True)
```

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE tracks (id INTEGER, name TEXT, genre TEXT)")
conn.executemany("INSERT INTO tracks VALUES (?, ?, ?)", [(1,"A","Jazz"),(2,"B","Rock")])
conn.commit()
```

13.1 SQL skill policy

SQL assistant는 query 작성 전에 안전 규칙을 먼저 로드해야 합니다.

```
sql_policy = [
    "SELECT only by default",
    "LIMIT required for exploration",
    "No INSERT/UPDATE/DELETE without approval",
]

sql_policy
```

13.2 query safety check

데이터베이스 호출 전에 안전 검사를 먼저 실행합니다. 이 교육용 예시는 읽기 전용 `SELECT`만 허용하고, 주요 mutation 명령을 차단하며, 탐색 쿼리가 무제한으로 실행되지 않도록 `LIMIT`을 필수로 요구합니다. 실제 운영 환경에서는 DB 권한, query timeout, row cap, parser 기반 validation을 함께 사용해야 합니다.

```
def is_safe_query(sql: str) -> bool:
    lowered = sql.lower().strip()
    forbidden = ["insert", "update", "delete", "drop", "alter"]
    has_limit = " limit " in f" {lowered} "
    return lowered.startswith("select") and has_limit and not any(word in lowered for word in forbidden)

print(is_safe_query("SELECT * FROM tracks LIMIT 5"))
print(is_safe_query("SELECT * FROM tracks"))
```

13.3 read-only 실행

안전 검사를 통과한 SELECT만 실행합니다.

```
query = "SELECT genre, COUNT(*) AS n FROM tracks GROUP BY genre LIMIT 10"
assert is_safe_query(query)

rows = conn.execute(query).fetchall()
rows
```

13.4 answer synthesis

SQL 결과와 정책 확인을 함께 보고합니다.

```
answer = {
    "query": query,
    "rows": rows,
    "policy_checked": True,
}

answer
```

정리

항목	내용
다룬 기술	SQL skill policy, static query check, read-only execution
핵심 개념	SQL assistant는 답변보다 먼저 안전한 query-writing workflow를 강제해야 합니다.

REFERENCES

- `docs/langchain/multi-agent/skills-sql-assistant.md`
- `docs/langchain/sql-agent.md`
- `07_examples/skills/sql-agent/SKILL.md`

PART

VIII

Integrations

LangChain 생태계 통합

이 파트에서 다루는 내용

- 1. 통합 카테고리 개요
- 2. Anthropic Prompt Caching
- 3. Claude Bash Tool
- 4. Claude Text Editor
- 5. Claude Memory
- 6. Anthropic File Search
- 7. Bedrock Prompt Caching
- 8. OpenAI Moderation

01 통합 카테고리 개요

LangChain 생태계 지도

LangChain·LangGraph·Deep Agents는 공통 에이전트 인터페이스 아래 프로바이더별 구현을 플러그인 형태로 연결합니다. Part II Part IV가 "에이전트를 어떻게 쓰는가"를 다뤘다면, 이 Part는 "에이전트를 무엇과 연결하는가"를 살핍니다. 13개 카테고리의 통합 표면을 한눈에 파악하고, 각 카테고리의 대표 패키지와 선택 기준을 정리합니다.

학습 목표

LEARNING OBJECTIVES

- → LangChain 생태계의 13개 통합 카테고리를 조감한다
- → Provider Middleware가 왜 별도 카테고리로 분리되는지 이해한다
- → 벤더 선택이 필요한 영역(Chat Models, Vector Stores)과 표준화된 영역(Middleware, Checkpointers)을 구분한다
- → 이 Part에서 다루는 7개 Provider Middleware 장의 맵을 그린다

1.1 기준 버전 스냅샷

이 Part의 코드는 다음 버전을 전제로 합니다. 최신 릴리스는 `docs/skills/langchain-dependencies.md` 로 확인하세요.

- `langchain` 1.3+ (event streaming v3)
- `langgraph` 1.2+ (per-node timeout · `DeltaChannel` · graceful shutdown)
- `deepagents` 0.6.1+ (`CodeInterpreterMiddleware` , async subagents, interpreter snapshots)

! WARNING

LangChain 1.2 `create_agent` 는 같은 미들웨어 클래스의 중복 인스턴스를 거부합니다. `AssertionError: Please remove duplicate middleware instances.` 가 발생하면 서브클래싱으로 두 인스턴스를 서로 다른 타입으로 분리하세요.

1.2 13개 통합 카테고리

카테고리	대표 패키지	선택 기준
Chat Models	`langchain-openai`, `langchain-anthropic`, `langchain-google-genai`, `langchain-aws`	모델 성능·비용·리전 요건
Embeddings	`langchain-openai`, `langchain-cohere`, `langchain-voyageai`	도메인 벤치마크·라이선스·다국어 지원
Vector Stores	`langchain-chroma`, `langchain-pinecone`, `langchain-pgvector`	운영 규모·메타데이터 필터링·관리형 여부
Document Loaders	`langchain-community.document_loaders`	소스 포맷(PDF, Notion, Slack 등)
Retrievers	`langchain.retrievers`, 벤더 특화 리트리버	하이브리드 검색·MMR·재랭커 통합
Text Splitters	`langchain-text-splitters`	언어·구조(코드 vs 프로즈)·오버랩 전략
Tools	`langchain-community.tools`, MCP 도구	외부 API 연동·인증 방식·호출 제약
Checkpointers	`langgraph-checkpoint-postgres`, `-sqlite`	내구성 요구·멀티 테넌트·백업 전략
Stores	`langgraph-store-postgres`, 벤더 관리형	영속 메모리 규모·검색 방식(키/벡터)
Sandboxes	`langchain-daytona`, `langchain-modal`, `langchain-runloop`	격리 수준·콜드 스타트·비용
Provider Middleware	`langchain-anthropic`, `langchain-aws`, `langchain-openai`	프로바이더 고유 기능(캐시·네이티브 도구·정책)
Observability	`langsmith`, `langfuse`, OpenTelemetry	SaaS vs 자체 호스팅·PII 정책
Providers (Hosted Runtime)	LangGraph Platform, Anthropic Skills, OpenAI Agents	관리형 런타임 vs 자체 호스팅·SDK 호환성

Provider Middleware는 프로바이더 서버 측에서 활성화되는 기능(프롬프트 캐시, 네이티브 도구, 컨텐츠 정책)을 LangChain 미들웨어 포맷으로 감싼 것입니다. 공식 문서에는 한 줄 정도만 언급되는 경우가 많아 실행 가능한 코드로 정리해 두는 편이 실용적입니다. 이 Part의 ch2 ch8에서 7개 미들웨어를 각각 1장씩 다룹니다.

1.3 Provider Middleware 로드맵

장	미들웨어	효과
2	`AnthropicPromptCachingMiddleware`	긴 시스템 프롬프트/도구 정의 서버 측 캐시 (5m/1h)
3	`ClaudeBashToolMiddleware`	Claude 네이티브 bash 도구 + 실행 정책 3종
4	`StateClaudeTextEditorMiddleware` / `FilesystemClaudeTextEditorMiddleware`	네이티브 text editor 6개 오퍼레이션
5	`StateClaudeMemoryMiddleware` / `FilesystemClaudeMemoryMiddleware`	`/memories/*` 경로 계약으로 모델 자기 메모
6	`StateFileSearchMiddleware`	상태 안 가상 파일의 glob/grep 검색
7	`BedrockPromptCachingMiddleware`	AWS Bedrock 경유 Claude/Nova 캐시
8	`OpenAIModerationMiddleware`	OpenAI Moderation API로 사전/사후 검사

1.4 새 통합 표면: UI · Code · 견고성

2026년 기준 통합 범위는 provider middleware만으로 끝나지 않습니다. 새 표면은 이 Part의 13개 카테고리 수를 바꾸지 않고, 기존 카테고리 위에 얹히는 연결 방식으로 읽는 것이 안전합니다.

표면	핵심	이 책에서 이어지는 위치
Generative UI / AG-UI	controlled · declarative · open-ended UI 생성 패턴과 frontend/server 이벤트 경계	Part 2/3의 agent state · streaming, Part 8의 provider/runtime 통합
Deep Agents Code	`/oss/deepagents/code`의 configuration · approval · extensions · plugins · MCP tools	Part 4의 permissions/skills/subagents/backend와 같은 개념을 코딩 에이전트 런타임으로 확장
RAG · fault tolerance · multimodal	retrieval, checkpointer/retry/resume, multimodal `read_file`이 Deep Agents 파일시스템과 결합	Part 4/5/7의 core agent · advanced patterns · examples와 연결

Generative UI는 LLM이 UI를 어느 정도까지 결정하느냐에 따라 controlled, declarative, open-ended로 나뉩니다. AG-UI는 이 흐름을 frontend와 agent server 사이의 이벤트 프로토콜로 연결하는 별도 트랙입니다. 따라서 이 장에서는 새 노트북을 만들지 않고, state/streaming/hosted runtime을 배운 뒤 frontend 통합 후보로 표시합니다.

Deep Agents Code 문서는 `configuration`, `approval`, `extensions`, `plugins`, `MCP tools` 처럼 코딩 에이전트 제품화 표면을 다룹니다. 여기서의 승인·확장·플러그인은 Part 4의 permissions, skills, subagents, backend와 같은 개념을 더 좁은 코딩 에이전트 실행 환경에 맞춘 것입니다. 별도 자격 증명이나 환경 변수를 추가로 전제하지 않습니다.

RAG, fault tolerance, multimodal은 새 카테고리가 아니라 기존 축의 결합입니다. RAG는 Retriever/Vector Store와 Deep Agents 파일 컨텍스트의 결합이고, fault tolerance는 LangGraph checkpointer · retry · resume과 운영 장의 연결이며, multimodal은 `read_file`이 이미지/PDF/오디오/비디오 입력을 모델별 제한 안에서 전달하는 문제입니다.

공식 참조	확인할 내용
LangChain Generative UI	controlled/declarative/open-ended UI 패턴
Deep Agents AG-UI	frontend/server 통합 이벤트 경계
Deep Agents Code configuration	코딩 에이전트 런타임 설정
Code approval modes	실행 전 승인 정책과 권한 경계
Code extensions	런타임 기능 확장과 스킬/메모리 연결
Code plugins	외부 도구 묶음과 배포 단위
Deep Agents fault tolerance	재시도·재개·체크포인터와 운영 안정성
Deep Agents multimodal	파일 백엔드와 멀티모달 입력 처리
Deep Agents RAG	검색·파일 컨텍스트·서브에이전트 결합

1.5 공통 패턴

이 Part의 모든 장은 세 단계 루프를 공유합니다.

1. **환경 설정** — `.env`에 해당 프로바이더 키를 넣고 패키지를 설치합니다.
2. **기본 사용** — `create_agent(..., middleware=[Middleware()])` 한 줄로 기능을 켭니다.
3. **검증** — `usage_metadata`, `tool_calls`, 또는 Moderation 응답을 읽어 기능이 실제로 동작하는지 확인합니다.

> ➡️ **TIP**
>
> 프로바이더 특화 미들웨어는 모델을 같이 정해야 합니다. Anthropic 미들웨어를 OpenAI 모델에 붙이면 `unsupported_model_behavior` 설정에 따라 경고만 나오거나 예외가 발생합니다. 멀티 프로바이더 파이프라인에서는 `ModelFallbackMiddleware`와 조합할 때 순서에 주의하세요.

1.6 Observability 환경변수 표준화

LangChain 1.1부터 트레이싱 관련 환경변수는 `LANGCHAIN_*` → `LANGSMITH_*` 로 표준화되었습니다. 옛 변수도 호환을 위해 일정 기간 동작하지만, 신규 코드는 새 이름만 씁니다.

변수	역할	비고
`LANGSMITH_TRACING`	트레이스 활성화 (`true` / `false`)	모든 `create_agent` 실행 자동 계측
`LANGSMITH_API_KEY`	LangSmith 계정 키	`ls__` 접두사
`LANGSMITH_PROJECT`	트레이스 분류 프로젝트 이름	`tracing_context` 로 호출 구간별 override 가능
`LANGSMITH_ENDPOINT`	self-host LangSmith용 엔드포인트	선택 — SaaS 사용 시 불필요

핵심 정리

- 통합은 13개 카테고리로 정리되며, 각 카테고리는 “프로바이더 플러그인 + 공통 인터페이스” 구조를 공유한다
- Provider Middleware는 프로바이더 서버 측 고유 기능을 LangChain 미들웨어로 감싼 영역
- 이 Part의 2 8장이 7개 Provider Middleware를 1장씩 실행 가능한 코드로 다룬다
- Generative UI/AG-UI, Deep Agents Code, RAG/fault tolerance/multimodal은 새 노트북 목록이 아니라 기존 state·streaming·backend·retrieval·operations 축과 연결해 읽는다
- 중복 인스턴스 제약, 모델 매칭, 멀티 프로바이더 폴백은 Provider Middleware 공통 주의점

02 Anthropic Prompt Caching

서버 측 캐시로 입력 토큰 90% 절감

Claude 모델 전용 프롬프트 캐시 미들웨어 `AnthropicPromptCachingMiddleware`는 긴 system prompt, 도구 정의, 초기 대화 컨텍스트를 Anthropic 서버 측에 5분 1시간 캐시해 입력 토큰 비용과 지연을 크게 줄입니다. 같은 에이전트가 같은 시스템 프롬프트로 여러 번 호출될 때, 두 번째 호출부터는 입력 토큰 단가가 약 90% 할인됩니다.

학습 목표

LEARNING OBJECTIVES

- → 4개 파라미터(`type` , `ttl` , `min_messages_to_cache` , `unsupported_model_behavior`)를 이해한다
- → `usage_metadata.input_token_details` 에서 `cache_creation` / `cache_read` 를 읽어 캐시 적중을 확인한다
- → 1시간 TTL beta와 5분 기본값의 선택 기준을 안다
- → 비(非) Anthropic 모델에 적용할 때 `warn` / `ignore` / `raise` 동작을 구분한다

2.1 언제 쓰나

- 긴 system prompt(수천 토큰의 기업 정책, 스타일 가이드)를 매 턴 전송하는 에이전트
- 큰 tool 정의 목록(20개 이상의 JSON schema)을 반복 전송하는 구성
- RAG 컨텍스트 재사용: 같은 문서 묶음을 여러 질의에 걸쳐 사용
- 멀티턴 대화에서 앞부분 컨텍스트가 안정적인 경우

2.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` 가 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import AnthropicPromptCachingMiddleware

load_dotenv()
```

2.3 기본 사용

미들웨어를 `create_agent` 의 `middleware` 리스트에 넣으면 LangChain이 system prompt · tool 정의 · 마지막 메시지 위치에 `cache_control: {"type": "ephemeral"}` 마커를 자동 부착합니다.

```
long_system_prompt = (
    "You are a senior software engineer... " * 200  # 약 2,000 토큰 가정
)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    system_prompt=long_system_prompt,
    tools=[],
    middleware=[AnthropicPromptCachingMiddleware()],
)
```

2.4 캐시 적중 확인

같은 에이전트를 연속 두 번 호출하고 `usage_metadata` 를 읽으면 캐시 적중 여부가 드러납니다. 1회차는 `cache_creation_input_tokens` 가, 2회차 이후는 `cache_read_input_tokens` 가 커집니다.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"요청 {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

```
0 {'cache_creation': 2048, 'cache_read': 0}
1 {'cache_creation': 0, 'cache_read': 2048}
```

2.5 파라미터 전체

파라미터	기본값	설명
`type`	`"ephemeral"`	현재 Anthropic이 지원하는 유일한 타입
`ttl`	`"5m"`	캐시 유지 시간. `"5m"` 또는 `"1h"`. 1시간은 Anthropic beta
`min_messages_to_cache`	`0`	메시지 수가 이 값 이상일 때만 cache_control 부착
`unsupported_model_behavior`	`"warn"`	비 Anthropic 모델 적용 시 `"warn"` / `"ignore"` / `"raise"`

1시간 TTL 예

```
AnthropicPromptCachingMiddleware(
    ttl="1h",
    min_messages_to_cache=2,
)
```

1시간 캐시는 긴 상담 세션이나 일간 배치 분석처럼 같은 시스템 프롬프트로 수 시간 반복 호출되는 경우에 맞습니다. 5분 기본값은 실시간 챗봇의 연속 입력용입니다.

2.6 비 Anthropic 모델 동작

이 미들웨어는 내부적으로 `cache_control` 블록을 주입하므로, 다른 공급자 모델에서는 무시되거나 에러로 이어집니다. 모델을 교체할 수 있는 파이프라인이라면 `unsupported_model_behavior` 를 명시하세요.

```
# 경고만 남기고 계속
AnthropicPromptCachingMiddleware(unsupported_model_behavior="warn")

# 조용히 스킵
AnthropicPromptCachingMiddleware(unsupported_model_behavior="ignore")

# 즉시 실패 (CI/테스트용)
AnthropicPromptCachingMiddleware(unsupported_model_behavior="raise")
```

2.7 `UsageMetadataCallbackHandler` 로 집계

스레드 전체의 캐시 적중률을 한 번에 집계하려면 `langchain.callbacks.UsageMetadataCallbackHandler` 를 붙입니다. 매 LLM 호출의 `usage_metadata` 가 누적되어 `cache_creation` / `cache_read` 까지 그대로 보존됩니다.

```
from langchain.callbacks import UsageMetadataCallbackHandler

handler = UsageMetadataCallbackHandler()
agent.invoke(
    {"messages": [{"role": "user", "content": "..."}]},
    config={"callbacks": [handler]},
)
print(handler.usage_metadata)  # {model_name: {input/output/cache_creation/cache_read}}
```

> ➡️ TIP
>
> Anthropic 응답을 `output_version="v1"` (`ChatAnthropic(..., output_version="v1")`)로 받으면 `cache_creation` / `cache_read` 키가 메시지 본문이 아닌 표준화된 `usage_metadata.input_token_details` 안으로 정리됩니다 — 멀티 프로바이더 집계 코드가 단순해집니다.

2.8 Bedrock과의 차이

AWS Bedrock 경유로 Claude를 호출할 때는 `BedrockPromptCachingMiddleware` (ch7)를 씁니다. 파라미터 이름과 의미는 거의 같지만, 대상 패키지와 TTL 제약(Nova 모델은 5m만 지원, tool 정의 캐시 미지원)이 다릅니다.

핵심 정리

- 긴 system prompt·tool 정의를 반복 전송하는 에이전트에서 입력 토큰 비용을 90% 절감
- `cache_creation_input_tokens` → `cache_read_input_tokens` 이동으로 캐시 적중 검증
- TTL `"5m"` 은 실시간 대화, `"1h"` 는 장시간 배치·상담 세션용
- 멀티 프로바이더 파이프라인은 `unsupported_model_behavior="warn"` 을 명시해 조용한 실패 방지

03 Claude Bash Tool

네이티브 bash_20250124 + 실행 정책

`ClaudeBashToolMiddleware`는 Claude 모델에 Anthropic 네이티브 `bash_20250124` 도구를 주입합니다. 범용 `ShellToolMiddleware`와 달리 Anthropic 서버가 bash 호출 스키마를 직접 관리하므로 프롬프트가 간결하고 tool schema 토큰이 거의 0에 수렴합니다. 이 장에서는 3종 실행 정책(Host/Docker/CodexSandbox)의 트레이드오프를 비교하고, 민감정보 마스킹을 위한 `redaction_rules`까지 다룹니다.

학습 목표

> **LEARNING OBJECTIVES**
>
> → `ClaudeBashToolMiddleware`의 4개 주요 파라미터를 이해한다
>
> → `HostExecutionPolicy` / `DockerExecutionPolicy` / `CodexSandboxExecutionPolicy`의 격리 수준을 구분한다
>
> → `RedactionRule`로 출력에서 토큰·키를 마스킹한다
>
> → Claude 네이티브 bash와 범용 shell 도구의 차이를 안다

3.1 언제 쓰나

- Claude에게 실제 쉘 명령을 실행시켜 코드 실행, 파일 조사, 빌드를 맡길 때
- Deep Agents처럼 장시간 작업 공간에서 세션 상태(cwd, 환경변수)를 유지해야 할 때
- 격리된 Docker 컨테이너에서 안전하게 임의 코드를 실행하고 싶을 때
- 쉘 출력에 API 키·자격증명이 섞일 가능성이 있어 출력 리다이렉션이 필요할 때

3.2 환경 설정

필요 패키지: `langchain`, `langchain-anthropic`. Docker 정책 예시를 실행하려면 로컬에 Docker 데몬이 떠 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import ClaudeBashToolMiddleware

load_dotenv()
```

3.3 호스트 실행 정책 (가장 단순)

`HostExecutionPolicy` 는 로컬 프로세스에서 명령을 실행합니다. 빠르고 설정이 간단하지만 격리가 없다 — 네트워크, 파일시스템, 환경변수에 그대로 접근하므로 신뢰할 수 있는 스크립트나 로컬 개발 용도로만 씁니다.

```
from langchain.agents.middleware import HostExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            workspace_root="/tmp/work",
            startup_commands=["export PATH=$HOME/.local/bin:$PATH"],
            execution_policy=HostExecutionPolicy(),
        ),
    ],
)
```

- `workspace_root` : 쉘 세션의 기본 디렉터리
- `startup_commands` : 세션 시작 시 자동 실행되는 명령들 (PATH 설정, venv 활성화 등)

3.4 Docker 실행 정책 (권장, 격리)

`DockerExecutionPolicy` 는 명령을 컨테이너 안에서 실행합니다. 호스트 파일시스템과 네트워크에서 완전히 분리되므로, 모델이 만든 임의 코드를 실행할 때는 사실상 이 정책을 기본으로 써야 합니다.

```
from langchain.agents.middleware import DockerExecutionPolicy

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(image="python:3.11"),
            startup_commands=["pip install requests numpy"],
        ),
    ],
)
```

컨테이너는 첫 bash 호출 시 생성되고 세션이 끝나면 정리됩니다. 패키지가 필요하다면 `startup_commands` 에 `pip install ...` 을 넣습니다.

3.5 Codex 샌드박스 정책

`CodexSandboxExecutionPolicy` 는 Anthropic이 제공하는 샌드박스 러너에서 실행합니다. 로컬 Docker 없이 격리가 필요할 때 쓰는 대안입니다. 네트워크 화이트리스트와 리소스 한도가 기본값으로 강하게 잡혀 있습니다.

3.6 출력 리다이렉션 (`redaction_rules`)

쉘 출력에 API 키·토큰·이메일 같은 민감정보가 흘러나올 수 있습니다. `RedactionRule` 을 리스트로 넘겨 도구 응답을 에이전트 컨텍스트에 들어가기 전에 마스킹합니다. LangChain 1.2부터 `RedactionRule` 은 `PIIMiddleware`

와 같은 `(pii_type, strategy, detector)` 시그니처를 씁니다 — 과거 `pattern=` / `replacement=` 인자는 제거되었습니다.

```
from langchain.agents.middleware import RedactionRule

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        ClaudeBashToolMiddleware(
            execution_policy=DockerExecutionPolicy(),
            redaction_rules=[
                RedactionRule(
                    pii_type="openai_api_key",
                    strategy="redact",
                    detector=r"sk-[a-zA-Z0-9]{32,}",
                ),
                RedactionRule(
                    pii_type="github_token",
                    strategy="redact",
                    detector=r"ghp_[a-zA-Z0-9]{36}",
                ),
            ],
        ),
    ],
)
```

3.7 일반 `@tool` 로 대체할 때

Claude 외 모델에서도 같은 흐름을 쓰려면 LangChain 1.3의 `@tool` 데코레이터로 직접 쉘 함수를 정의합니다. `extras` 필드(LangChain 1.2부터 도입)에 Anthropic programmatic tool calling 메타데이터를 주입하면 동일 함수를 Claude 전용 도구와 같은 톤으로 호출할 수 있습니다.

```
from langchain_core.tools import tool

@tool(extras={"anthropic_cache_control": {"type": "ephemeral"}})
def run_bash(command: str) -> str:
    """격리된 컨테이너에서 bash 명령을 실행하고 stdout/stderr를 반환합니다."""
    return run_in_container(command)  # 자체 구현체
```

> ➡️ **TIP**
>
> `@tool(extras=...)` 은 멀티 프로바이더 파이프라인의 부분 캐싱에 유용합니다. Claude는 `cache_control` 을 해석하고, 다른 프로바이더는 무시하므로 한 코드베이스를 그대로 양쪽에 노출할 수 있습니다.

3.8 네이티브 vs 범용 비교

항목	`ClaudeBashToolMiddleware`	범용 `ShellToolMiddleware`
도구 타입	Anthropic 네이티브 `bash_20250124`	일반 `@tool` 함수
지원 모델	Claude 전용	모든 모델

tool schema 토큰	서버 측 → 거의 0	매 턴 전송
세션 상태	cwd, env 누적 유지	구현에 따라 다름
실행 정책 API	공유 (`HostExecutionPolicy` 등)	공유

선택 기준: Claude만 쓴다면 네이티브가 싸고 깔끔합니다. 멀티 프로바이더 파이프라인이라면 범용 `ShellToolMiddleware` 로 통일합니다.

핵심 정리

- Claude 네이티브 bash 도구는 tool schema 토큰을 거의 0으로 만든다
- 실행 정책은 Host(격리 없음) → Docker(권장) → CodexSandbox(Anthropic 관리) 세 층으로 선택
- `redaction_rules` 로 도구 응답의 민감정보를 에이전트 컨텍스트 진입 전에 마스킹
- 멀티 프로바이더 파이프라인에서는 범용 `ShellToolMiddleware` 로 통일하는 것이 안전

04 Claude Text Editor

State vs Filesystem 변형

Claude 네이티브 `text_editor_20250728` 도구는 `view` / `create` / `str_replace` / `insert` / `delete` / `rename` 6가지 오퍼레이션을 지원합니다. LangChain은 이 도구를 두 변형 미들웨어로 제공합니다 — State 변형은 LangGraph 상태 안의 가상 파일로, Filesystem 변형은 실제 디렉터리에 씁니다. 산출물이 스레드 내에서만 쓰이는지, 디스크에 남아야 하는지로 선택합니다.

학습 목표

LEARNING OBJECTIVES

- → Claude 네이티브 text editor의 6가지 오퍼레이션을 사용한다
- → State 변형과 Filesystem 변형의 수명·공유 범위 차이를 구분한다
- → `allowed_path_prefixes` / `allowed_prefixes` 로 접근 경로를 제한한다
- → `root_path`, `max_file_size_mb` 로 디스크 변형의 경계를 설정한다

4.1 언제 쓰나

- 코드/문서 편집을 모델이 멀티스텝으로 수행해야 할 때 (diff를 한 번에 뽑기 어려운 큰 변경)
- 결과물이 그래프 상태 안에서만 살아 있으면 되는 경우: State 변형
- 결과물을 실제 리포지터리/디렉터리에 남겨야 하는 경우: Filesystem 변형
- 일반 `@tool` 로 `read_file` / `write_file` 을 짜는 대신 Claude가 학습해둔 도구 스키마를 재사용하고 싶을 때

4.2 환경 설정

필요 패키지: `langchain`, `langchain-anthropic`. `.env` 에 `ANTHROPIC_API_KEY`.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    FilesystemClaudeTextEditorMiddleware,
)
```

```
load_dotenv()
```

4.3 State 변형 — 그래프 상태 안 가상 파일

`StateClaudeTextEditorMiddleware`는 파일 내용을 LangGraph 상태의 `text_editor_files` 키에 저장합니다. 디스크에 쓰지 않으므로 스레드가 끝나면 사라지는 임시 작업에 어울립니다.

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(
            allowed_path_prefixes=["/src"],  # 가상 경로 제한
        ),
    ],
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "/src/hello.py에 hello world를 써줘"}]},
)
print(result.get("text_editor_files"))
```

- `allowed_path_prefixes` : 접근 허용 가상 경로 접두사. 미지정 시 전체 허용

> ➡️ TIP
>
> `allowed_path_prefixes` 밖 경로로 쓰려고 하면 도구가 에러를 반환하고, 모델은 그 에러를 읽은 뒤 허용 경로로 재시도합니다. 경로 제한은 에이전트 자율성과 안전의 타협점입니다.

4.4 Filesystem 변형 — 실제 디스크

`FilesystemClaudeTextEditorMiddleware`는 실제 디렉터리를 루트로 삼아 파일을 읽고 씁니다.

파라미터	기본값	설명
`root_path`	(필수)	파일 오퍼레이션의 실제 루트 디렉터리
`allowed_prefixes`	`["/"]`	허용 가상 경로 접두사 (root_path 기준 상대 경로)
`max_file_size_mb`	`10`	읽기/쓰기 허용 최대 파일 크기

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeTextEditorMiddleware(
            root_path="/tmp/editor-demo",
            allowed_prefixes=["/drafts", "/reports"],
            max_file_size_mb=5,
        ),
```

```
    ],
)
```

4.5 State vs Filesystem 선택 기준

축	State 변형	Filesystem 변형
저장소	LangGraph 상태 dict	실제 디렉터리
수명	스레드와 함께 소멸	영구 (디스크에 남음)
체크포인터 호환	상태에 포함 → 자동 저장	경로만 저장, 파일은 별도 관리
동시성	스레드별 완전 격리	같은 `root_path` 공유 시 충돌 가능
적합한 용도	임시 draft, 1회성 분석	실제 코드베이스 수정, 보고서 산출물

규칙: 결과물을 스레드 종료 후에도 보존해야 하면 Filesystem, 그렇지 않으면 State를 씁니다.

4.6 일반 파일 도구와의 관계

Deep Agents의 `FilesystemMiddleware` 나 커스텀 `@tool` 로 짠 `read_file` / `write_file` 도 같은 목적을 달성합니다. 차이는 Claude가 이 도구를 학습 단계에서 이미 봤다는 점 — 도구 스키마 토큰이 줄고 오류가 적습니다. Claude 전용 파이프라인이라면 이 미들웨어를 먼저 쓰고, 멀티 프로바이더라면 일반 파일 도구로 내리는 것이 기본 전략입니다.

핵심 정리

- Claude 네이티브 text editor는 tool schema 토큰을 거의 0으로 만들고 6가지 오퍼레이션을 지원
- 산출물 수명으로 State(스레드와 함께 소멸) vs Filesystem(디스크 영구 보존) 선택
- `allowed_path_prefixes` / `allowed_prefixes` 로 접근 경계를 강제
- Filesystem 변형은 `max_file_size_mb` 로 메모리 안전성 확보

05 Claude Memory

`/memories/*` 경로 계약

Claude 네이티브 `memory_20250818` 도구는 모델이 스스로 메모를 쓰고 다시 읽는 장기 기억을 구현합니다. 주입되는 시스템 프롬프트가 매 턴 "먼저 `/memories` 를 확인하라"고 지시하므로, 같은 사용자와 여러 세션에 걸쳐 취향이나 사실이 자연스럽게 쌓입니다. State와 Filesystem 두 변형의 지속성 차이를 파악하는 것이 이 장의 핵심입니다.

학습 목표

LEARNING OBJECTIVES

- → Claude 메모리 도구의 `/memories/` 경로 계약을 이해한다
- → State vs Filesystem 변형의 지속성 범위 차이를 구분한다
- → 주입되는 `system_prompt` 가 모델에게 메모리 활용을 지시하는 방식을 안다
- → Deep Agents `StoreBackend` 와의 역할 분담을 정리한다

5.1 언제 쓰나

- 같은 사용자와 여러 세션에 걸쳐 취향/사실을 기억해야 하는 챗봇
- 에이전트가 장시간 작업 중 중간 결과를 본인이 쓰고 본인이 다시 읽어야 하는 경우
- RAG와 달리 모델이 직접 메모 내용을 관리(쓰고 지우고 수정)하도록 맡기고 싶을 때

5.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeMemoryMiddleware,
    FilesystemClaudeMemoryMiddleware,
)
from langgraph.checkpoint.memory import MemorySaver

load_dotenv()
```

5.3 State 변형 — 스레드 내 지속

`StateClaudeMemoryMiddleware`는 메모 내용을 LangGraph 상태(`memory_files`)에 저장합니다. `MemorySaver` / `PostgresCheckpointer` 등 체크포인터를 쓰면 같은 thread_id로 재개할 때마다 자동 복원됩니다.

파라미터	기본값	설명
`allowed_path_prefixes`	`["/memories"]`	메모 저장 허용 경로. 보통 그대로 둔다
`system_prompt`	Anthropic 기본	매 턴 시작 전 "/memories를 먼저 확인하라"를 지시

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    checkpointer=MemorySaver(),
    middleware=[StateClaudeMemoryMiddleware()],
)

cfg = {"configurable": {"thread_id": "user-42"}}

agent.invoke(
    {"messages": [{"role": "user", "content": "내 이름은 지훈이야."}]},
    config=cfg,
)

# 같은 thread_id로 재호출 → 메모 재사용
result = agent.invoke(
    {"messages": [{"role": "user", "content": "내 이름 알지?"}]},
    config=cfg,
)
print(result["messages"][-1].content)
```

모델은 두 번째 호출에서 이름을 묻지 않고 `/memories`를 읽어 "지훈"으로 답합니다.

5.4 Filesystem 변형 — 프로세스 경계를 넘어

`FilesystemClaudeMemoryMiddleware`는 메모를 실제 디스크 디렉터리에 남깁니다. 프로세스가 종료되거나 체크포인터 저장소가 바뀌어도 디스크 파일이 있으면 그대로 다시 읽습니다.

파라미터	기본값	설명
`root_path`	(필수)	메모리를 저장할 실제 디렉터리
`allowed_prefixes`	`["/memories"]`	메모 작성 허용 가상 경로
`max_file_size_mb`	`10`	한 메모 파일 최대 크기
`system_prompt`	기본 제공	메모리 활용 지시 프롬프트

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        FilesystemClaudeMemoryMiddleware(
            root_path="/var/data/claude-memory",
            max_file_size_mb=2,
        ),
```

```
    ],
)
```

5.5 영속 체크포인터로 확장

`MemorySaver` 는 프로세스 내 사전(dict) 저장이므로 재시작 시 휘발됩니다. 운영 환경에서는 `langgraph-checkpoint-postgres` 또는 `-sqlite` 의 컨텍스트 매니저 패턴을 씁니다 — `from_conn_string()` 으로 풀을 얻고, 처음 한 번 `setup()` 을 호출해 스키마를 생성합니다.

```python
from langgraph.checkpoint.postgres import PostgresSaver

DB = "postgresql://user:pass@localhost/agent"

with PostgresSaver.from_conn_string(DB) as checkpointer:
    checkpointer.setup()  # 최초 1회 — 테이블/인덱스 생성

    agent = create_agent(
        model="anthropic:claude-sonnet-4-6",
        checkpointer=checkpointer,
        middleware=[StateClaudeMemoryMiddleware()],
    )
```

> ➡️ TIP
>
> 비동기 그래프는 `AsyncPostgresSaver.from_conn_string(DB)` 를 `async with` 로 받습니다. `setup()` 도 `await` 이 필요합니다. SQLite도 동일한 컨텍스트 매니저 API(`SqliteSaver` / `AsyncSqliteSaver`)를 따릅니다.

5.6 `runtime.context.user_id` 로 멀티 테넌트 분리

같은 디스크 디렉터리(또는 같은 Store)를 여러 사용자가 공유할 때는 네임스페이스로 격리해야 합니다. LangGraph 1.2부터 `runtime.context.user_id` 를 직접 읽어 Store의 namespace 키로 쓰는 패턴이 표준입니다.

```python
from langgraph.store.postgres import PostgresStore
from langgraph.runtime import get_runtime

with PostgresStore.from_conn_string(DB) as store:
    store.setup()

    def upsert_profile(profile: dict) -> str:
        rt = get_runtime()
        ns = ("profiles", rt.context.user_id)  # 사용자별 네임스페이스
        store.put(ns, "self", profile)
        return "ok"
```

> ➡️ TIP

> Claude Memory Middleware의 `/memories/*` 경로 계약과 LangGraph Store의 `(namespace, key)` 계약은 층위가 다릅니다. 전자는 Claude가 자율적으로 메모를 관리, 후자는 코드가 명시적으로 키를 부여 — 멀티 테넌트 격리는 Store 쪽이 자연스럽습니다.

5.7 Deep Agents `StoreBackend` 와의 관계

Deep Agents는 영구 메모리를 위한 `StoreBackend`를 별도로 제공합니다. 현재 API에서는 `StoreBackend(namespace=...)` 처럼 명시적 namespace가 필요하며, 역할은 Claude 네이티브 메모리와 비슷해 보여도 적용 범위가 다릅니다.

축	Claude Memory Middleware	Deep Agents `StoreBackend`
제공자	Anthropic 네이티브 도구	LangGraph Store API 래퍼
지원 모델	Claude 전용	모든 모델
저장 구조	`/memories/*` 파일	`(namespace, key) → dict`
검색 방식	파일 목록 + 내용 읽기	명시적 namespace 아래 파일 읽기/쓰기
적합한 용도	Claude가 자유롭게 메모 관리	구조화된 프로필·사실 저장

조합 팁: 둘을 함께 써도 됩니다. Claude가 단기 작업 노트를 `/memories`에 남기고, 장기 프로필은 Deep Agents `StoreBackend`에 넣어 다른 프로바이더 에이전트와 공유하는 패턴이 현실적입니다.

핵심 정리

- Claude 네이티브 메모리 도구는 `/memories/*` 경로 계약 + 자동 시스템 프롬프트로 "먼저 확인, 필요 시 갱신" 흐름을 강제한다
- State 변형은 체크포인터로 thread 재개 시 복원, Filesystem 변형은 디스크에 영구 보존
- `max_file_size_mb`로 모델이 한 메모에 몰아 쓰는 것을 방지
- Deep Agents `StoreBackend`는 명시적 namespace와 멀티 프로바이더 공유가 필요할 때 보완재

06 Anthropic File Search

가상 파일의 glob + grep

`StateFileSearchMiddleware` 는 그래프 상태 안에 있는 가상 파일(예: `text_editor_files` , `memory_files`)을 glob + grep으로 검색하는 Claude 네이티브 도구를 제공합니다. 텍스트 에디터/메모리 미들웨어가 "파일을 만들고 편집"한다면, 이 미들웨어는 그 위에서 "파일을 찾고 읽는" 역할입니다.

학습 목표

LEARNING OBJECTIVES

- → `StateFileSearchMiddleware(state_key=...)` 로 검색 대상 저장소를 선택한다
- → `StateClaudeTextEditorMiddleware` 와 조합해 "쓰고 → 찾고 → 읽는" 루프를 완성한다
- → `state_key="memory_files"` 로 메모리 파일도 검색 대상으로 돌린다
- → 중복 미들웨어 제약과 서브클래싱 회피 패턴을 안다

6.1 언제 쓰나

- 에이전트가 수십 수백 개의 가상 파일을 상태에 쌓아놓고 탐색해야 할 때
- 이름만 기억나는 파일을 패턴(`**/*.md`)으로 찾거나, 특정 키워드를 포함한 파일만 골라야 할 때
- 메모리에 쌓인 과거 노트를 모델이 스스로 grep 해서 참조하게 하고 싶을 때

6.2 환경 설정

필요 패키지: `langchain` , `langchain-anthropic` . `.env` 에 `ANTHROPIC_API_KEY` .

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_anthropic.middleware import (
    StateClaudeTextEditorMiddleware,
    StateFileSearchMiddleware,
)

load_dotenv()
```

6.3 텍스트 에디터 + 파일 검색 조합

검색은 단독으로 쓸 수 없습니다 — 파일이 먼저 상태에 있어야 합니다. 가장 많이 쓰는 조합은 text editor로 파일을 만들고 file search로 찾는 패턴입니다.

파라미터	기본값	설명
`state_key`	`"text_editor_files"`	검색할 파일 dict가 들어 있는 상태 키. `"memory_files"` 로 바꾸면 메모리 쪽 검색

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
    ],
)
```

6.4 1단계 — 여러 파일을 상태에 심는다

모델에게 `/docs/` 아래에 노트 파일 몇 개를 만들어 달라고 요청합니다. 이후 검색 쿼리에서 이 파일들이 대상이 됩니다.

```
cfg = {"configurable": {"thread_id": "search-demo"}}
agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": (
                    "/docs/architecture.md, /docs/api.md, /docs/release-notes.md 세 파일을 "
                    "간단한 초안으로 만들어 줘."
                ),
            }
        ]
    },
    config=cfg,
)
```

6.5 2단계 — 같은 에이전트가 glob/grep으로 검색

같은 스레드에서 이어 호출하면 이전 파일들이 상태에 남아 있습니다. 모델은 `StateFileSearchMiddleware` 가 제공하는 `glob` 과 `grep` 도구로 파일을 찾고 내용을 읽습니다.

```
result = agent.invoke(
    {
        "messages": [
            {
                "role": "user",
                "content": "/docs/ 아래 md 파일 중 'API'가 언급된 파일만 알려줘",
            }
        ]
```

```
    },
    config=cfg,
)
print(result["messages"][-1].content)
```

6.6 메모리 파일도 검색 대상으로

`state_key="memory_files"` 로 바꾸면 `StateClaudeMemoryMiddleware` 가 쌓아둔 메모를 검색할 수 있습니다.

> **! WARNING**
>
> **중복 미들웨어 제약** — LangChain 1.2 `create_agent` 는 같은 미들웨어 클래스의 중복 인스턴스를 거부합니다 (`AssertionError: Please remove duplicate middleware instances.`). 텍스트 에디터 파일과 메모리 파일 두 저장소를 동시에 검색하려면 한 번에 하나만 등록하거나, `StateFileSearchMiddleware` 를 서브클래싱해 별도 클래스로 분리해야 합니다.

```
class MemoryFileSearchMiddleware(StateFileSearchMiddleware):
    """`memory_files` 전용 검색 미들웨어."""

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    middleware=[
        StateClaudeTextEditorMiddleware(),
        StateClaudeMemoryMiddleware(),
        StateFileSearchMiddleware(state_key="text_editor_files"),
        MemoryFileSearchMiddleware(state_key="memory_files"),
    ],
)
```

6.7 Retrieval 5단계 building block에서의 위치

LangChain RAG는 Document Loaders → Text Splitters → Embeddings → Vector Stores → Retrievers 다섯 단계로 표준화되어 있습니다(08_integration의 04 05 카테고리).
`StateFileSearchMiddleware` 는 이 다섯 단계를 우회하는 lite 경로입니다 — 임베딩 비용 없이 현재 상태에 있는 파일을 리터럴 grep으로 뒤집니다.

축	State File Search	5단계 RAG 파이프라인
전처리	없음 — 상태에 들어온 그대로	Loader + Splitter + Embed + Index
검색 방식	literal glob + grep	vector similarity + filter
지연	밀리초 단위	임베딩 호출 + ANN
적합한 규모	수십 수백 파일	수만 수백만 청크

```
# 5단계 RAG로 확장할 때
from langchain.embeddings import init_embeddings
```

```python
from langchain_chroma import Chroma

embeddings = init_embeddings("openai:text-embedding-3-small")
vectorstore = Chroma(collection_name="docs", embedding_function=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

> ➡️ TIP
>
> `init_embeddings("provider:model")` 헬퍼는 `init_chat_model` 과 같은 패턴 — provider prefix 문자열로 임베딩 백엔드를 한 줄로 교체합니다. State File Search에서 벡터스토어 RAG로 넘어갈 때 코드 변경이 최소화됩니다.

6.8 vectorstore와의 선택 기준

- **정확한 파일명/패턴**이 중요하고 리터럴 grep으로 충분할 때
- 문서 수가 수십 수백 개 수준이고 매 턴 임베딩을 다시 돌릴 비용이 아까울 때
- 파일이 현재 세션에서 방금 만들어진 것이라 벡터 인덱스가 없을 때

대규모 코퍼스나 의미 검색이 필요하면 Part VIII Chat Models/Vector Stores 영역(향후 확장)으로 넘어갑니다.

핵심 정리

- `StateFileSearchMiddleware` 는 상태 안 가상 파일을 glob + grep으로 검색한다
- text editor + file search 조합이 "쓰고 → 찾고 → 읽는" 루프의 기본
- `state_key` 를 바꿔 메모리 파일도 검색 대상으로 돌리되, 중복 미들웨어 제약은 서브클래싱으로 우회
- 리터럴 검색이 충분할 때는 벡터스토어보다 이 미들웨어가 저비용

07 Bedrock Prompt Caching

AWS 경유 Claude/Nova 캐시

AWS Bedrock 경유로 Claude/Nova 모델을 호출할 때 `BedrockPromptCachingMiddleware`로 시스템 프롬프트 · 도구 정의 · 마지막 메시지 위치에 캐시 체크포인트를 자동으로 박아 입력 토큰 비용을 낮춥니다. `AnthropicPromptCachingMiddleware`(ch2)와 파라미터 이름과 의미는 거의 같지만 대상 패키지와 모델별 제약이 다릅니다.

학습 목표

LEARNING OBJECTIVES

- → `BedrockPromptCachingMiddleware`의 4개 파라미터를 이해한다
- → `usage_metadata.input_token_details`에서 `cache_creation` / `cache_read`를 읽어 적중을 검증한다
- → `ChatBedrock` vs `ChatBedrockConverse`에서 `type` 파라미터 처리 차이를 안다
- → Nova 모델의 제약(5m TTL 전용, tool 정의 캐시 미지원)을 구분한다

7.1 언제 쓰나

- AWS 계정·VPC 경계 안에서만 LLM을 호출해야 하는 기업 환경 (Bedrock 경유)
- Claude를 Anthropic 직접 API가 아닌 Bedrock 요금제로 이용할 때
- Amazon Nova 모델을 쓰면서 긴 시스템 프롬프트를 반복 사용할 때

7.2 환경 설정

필요 패키지: `langchain`, `langchain-aws`. AWS 자격증명(`AWS_ACCESS_KEY_ID` 등)과 리전이 `.env` 또는 환경에 설정돼 있어야 합니다. 모델별로 Bedrock 콘솔에서 모델 액세스를 먼저 허용해야 호출이 됩니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_aws.middleware import BedrockPromptCachingMiddleware

load_dotenv()
```

7.3 ChatBedrockConverse + Claude (권장)

Converse API는 Bedrock의 통합 인터페이스로, 다양한 모델을 같은 API로 호출합니다. `BedrockPromptCachingMiddleware`를 붙이면 system/tool/last-message 위치에 캐시 체크포인트가 자동으로 삽입됩니다.

> **! WARNING**
>
> 체크포인트가 실제로 작동하려면 캐시 대상 블록이 약 1,024 토큰 이상이어야 합니다. 짧은 system prompt에서는 `cache_creation`이 잡히지 않을 수 있습니다.

```
long_prompt = (
    "You are an enterprise policy assistant. Always cite section numbers. "
    * 200  # ~2,000 tokens
)

agent = create_agent(
    model="bedrock_converse:anthropic.claude-sonnet-4-20250514-v2:0",
    system_prompt=long_prompt,
    tools=[],
    middleware=[BedrockPromptCachingMiddleware(ttl="1h")],
)
```

7.4 캐시 적중 검증

두 번 연속 호출한 뒤 `usage_metadata.input_token_details`를 읽어 캐시 생성/적중을 확인합니다.

```
for i in range(2):
    result = agent.invoke(
        {"messages": [{"role": "user", "content": f"요청 {i}"}]},
    )
    last = result["messages"][-1]
    print(i, last.usage_metadata.get("input_token_details"))
```

`cache_creation`은 1회차에 채워지고, `cache_read`는 2회차 이후에 늘어납니다.

7.5 파라미터 전체

파라미터	기본값	설명
`type`	`"ephemeral"`	`ChatBedrock` 전용. `ChatBedrockConverse`는 이 값을 무시하고 `"default"` 사용
`ttl`	`"5m"`	`"5m"` 또는 `"1h"`. Nova 계열은 `"5m"`만 지원
`min_messages_to_cache`	`0`	메시지 수가 이 이상일 때만 cache 체크포인트 부착
`unsupported_model_behavior`	`"warn"`	캐시 미지원 모델에 대해 `"ignore"` / `"warn"` / `"raise"`

7.6 ChatBedrock (Invoke 모델) 예시

`ChatBedrock` 은 이전 세대 invoke-model 래퍼입니다. `BedrockPromptCachingMiddleware` 는 이쪽도 지원하며 `type="ephemeral"` 파라미터가 여기서는 실제로 반영됩니다. `ChatBedrock` 은 Anthropic 모델 한정으로 tool 정의 캐시와 확장 TTL(1h)을 모두 지원합니다.

```
from langchain_aws import ChatBedrock

model = ChatBedrock(model_id="anthropic.claude-sonnet-4-20250514-v2:0")

agent = create_agent(
    model=model,
    system_prompt=long_prompt,
    tools=[],
    middleware=[
        BedrockPromptCachingMiddleware(type="ephemeral", ttl="1h"),
    ],
)
```

7.7 Reasoning · Profiling

Claude 4.6/4.7 모델은 Bedrock에서도 extended thinking(reasoning)이 지원됩니다. 캐싱은 reasoning이 켜진 호출에도 적용되지만, reasoning 블록 자체는 캐시 미스로 매번 새로 계산되니 토큰 절감 효과가 무 reasoning 케이스보다 작습니다.

```
from langchain_aws import ChatBedrockConverse

model = ChatBedrockConverse(
    model_id="anthropic.claude-sonnet-4-20250514-v2:0",
    additional_model_request_fields={
        "reasoning_config": {"type": "enabled", "budget_tokens": 4096},
    },
)
print(model.profile.reasoning_output)  # True → 응답에 reasoning 블록 포함
```

> ➡️ TIP
>
> `ChatBedrockConverse(...).profile` (LangChain 1.1+)에서 `tool_calling` , `reasoning_output` , `max_input_tokens` 같은 모델 능력치를 확인할 수 있습니다 — 멀티 모델 라우팅 로직에서 capability check로 쓰면 깨끗합니다.

7.8 Amazon Nova 제약

항목	ChatBedrockConverse + Anthropic	ChatBedrockConverse + Nova
시스템 프롬프트 캐시	O	O

도구 정의 캐시	O	X
메시지 캐시	O	O (tool result 메시지 제외)
TTL `"1h"`	O	X (5m 전용)

Nova 모델을 쓸 때는 `ttl="5m"`으로 고정하고, `unsupported_model_behavior="warn"`을 설정해 실수로 `"1h"`를 지정했을 때 조용히 넘어가지 않도록 경고를 남기세요.

핵심 정리

- AWS 경계 안에서 Claude/Nova 사용 시 Bedrock 미들웨어로 입력 토큰 비용 절감
- `ChatBedrock`과 `ChatBedrockConverse`의 `type` 파라미터 처리 차이를 인지
- Nova 모델은 5m TTL + tool 정의 캐시 미지원 — 설정 시 `ttl="5m"` 고정
- 캐시 대상 블록은 약 1,024 토큰 이상이어야 실제 적중 발생

08 OpenAI Moderation

정책 위반 콘텐츠 사전/사후 차단

`OpenAIModerationMiddleware`는 OpenAI Moderation API로 사용자 입력 · 모델 출력 · 도구 결과를 자동 스캔하고, 정책 위반이 감지되면 대화를 차단·교체·예외로 처리합니다. 공개 챗봇의 혐오·폭력·자해 카테고리를 선제 차단하거나, 외부 도구 결과의 2차 오염을 막을 때 씁니다.

학습 목표

LEARNING OBJECTIVES

- → `check_input` / `check_output` / `check_tool_results` 세 스캔 지점의 의미와 비용 트레이드오프를 안다
- → `exit_behavior` 세 모드(`"end"` / `"error"` / `"replace"`)의 동작을 구분한다
- → 커스텀 `violation_message` 템플릿으로 사용자 응답을 다듬는다
- → 사전 구성된 OpenAI 클라이언트를 `client` / `async_client` 로 주입한다

8.1 언제 쓰나

- 공개 챗봇에서 혐오·폭력·자해 같은 Moderation 카테고리를 선제 차단할 때
- 도구 결과(예: 웹 검색, 이메일 본문)에 유해 콘텐츠가 섞여 모델에 들어가는 것을 막을 때
- 로그/감사용으로 위반 메타데이터를 응답 흐름에 남겨 보고서를 만들 때

8.2 환경 설정

필요 패키지: `langchain` , `langchain-openai` . `.env` 에 `OPENAI_API_KEY` 가 있어야 합니다.

```
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai.middleware import OpenAIModerationMiddleware

load_dotenv()
```

8.3 기본 사용 — 입출력 모두 검사, 위반 시 종료

기본 설정은 `check_input=True`, `check_output=True`, `exit_behavior="end"`. 사용자 입력이 Moderation API에 걸리면 모델 호출 없이 바로 위반 메시지로 대화를 종료합니다.

파라미터	기본값	설명
`model`	`"omni-moderation-latest"`	사용할 Moderation 모델
`check_input`	`True`	사용자 메시지를 모델에 넘기기 전 검사
`check_output`	`True`	모델 생성 응답을 사용자에게 반환하기 전 검사
`check_tool_results`	`False`	도구 실행 결과를 모델 입력 전에 검사
`exit_behavior`	`"end"`	`"end"` / `"error"` / `"replace"`
`violation_message`	기본 템플릿	위반 시 사용자에게 보여줄 텍스트
`client` / `async_client`	`None`	사전 구성된 OpenAI 클라이언트 주입 (선택)

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware()],
)
```

8.4 `exit_behavior="end"` — 위반 입력 즉시 종료

유해 카테고리(예: 자해 유도)가 감지되면 모델 호출 없이 위반 메시지로 바로 마감합니다. 응답 마지막 메시지에는 기본 위반 메시지가 담깁니다.

8.5 `exit_behavior="error"` — 예외로 빠르게 실패

테스트/배치 환경에서 위반을 조용히 넘기지 않고 예외로 띄울 때 씁니다. `OpenAIModerationError`가 발생하며 상위 파이프라인에서 catch해 감사 로그에 남깁니다.

```
from langchain_openai.middleware import OpenAIModerationError

agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[OpenAIModerationMiddleware(exit_behavior="error")],
)

try:
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
except OpenAIModerationError as e:
    # 감사 로그에 기록
    print("Moderation violation:", e.categories)
```

8.6 `exit_behavior="replace"` — 메시지만 교체하고 계속

출력 검사에서 위반이 잡히면 해당 응답만 위반 메시지로 교체하고 그래프 실행은 이어집니다. 멀티턴 대화에서 흐름은 끊지 않고 특정 응답만 정리할 때 쓰는 모드입니다.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[],
    middleware=[
        OpenAIModerationMiddleware(
            exit_behavior="replace",
            violation_message=(
                "이 응답은 안전 정책에 따라 표시할 수 없습니다 "
                "(카테고리: {categories}). 다른 질문을 해 주세요."
            ),
        ),
    ],
)
```

`violation_message`는 `{categories}`, `{category_scores}`, `{original_content}` 변수를 지원하는 템플릿 문자열입니다.

8.7 도구 결과 스캔 (`check_tool_results=True`)

웹 검색·크롤링·DB 조회 결과에 제3자 유해 콘텐츠가 섞여 모델에 들어가는 것을 막습니다. 비용은 도구 호출 수에 비례해 늘어나므로 신뢰할 수 없는 외부 소스를 쓰는 파이프라인에서만 켭니다.

```
agent = create_agent(
    model="openai:gpt-5.4",
    tools=[web_search_tool, fetch_url_tool],
    middleware=[
        OpenAIModerationMiddleware(
            check_input=True,
            check_output=True,
            check_tool_results=True,
        ),
    ],
)
```

8.8 LangSmith로 위반 흐름 관측

Moderation 차단은 드물지만 감사 로그에는 반드시 남아야 하는 사건입니다. LangSmith의 자동 트레이싱(`LANGSMITH_TRACING=true` + `LANGSMITH_API_KEY`)을 켜면 `OpenAIModerationMiddleware`가 트리거된 노드도 트레이스 트리에 정상적으로 박힙니다.

```
import os
from langsmith.run_helpers import tracing_context

os.environ["LANGSMITH_TRACING"] = "true"
# os.environ["LANGSMITH_API_KEY"] = "ls__..."
# os.environ["LANGSMITH_PROJECT"] = "moderation-audit"

with tracing_context(
    project_name="moderation-audit",
```

```
    tags=["moderation", "input-only"],
    metadata={"region": "kr"},
):
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
```

> ➡️ TIP
>
> `tracing_context` 는 특정 호출 구간만 다른 프로젝트/태그로 분리해 라우팅합니다 — 평소 트레이스는 `production` 에, Moderation 차단 케이스만 `moderation-audit` 에 모아두는 식의 운영이 자연스럽습니다. `LANGCHAIN_*` 환경변수는 1.1부터 `LANGSMITH_*` 로 표준화됐습니다 — 옛 변수도 한동안 동작하지만 신규 코드에서는 `LANGSMITH_*` 만 씁니다.

8.9 비용·지연 트레이드오프

설정	비용	지연	주요 효과
`check_input=True`	Mod 1 + Model 1	+1 RT	유해 입력이 모델에 들어가기 전 차단
`check_output=True`	Model 1 + Mod 1	+1 RT	모델이 잘못된 응답을 냈을 때 최종 사용자 보호
`check_tool_results=True`	도구 수 × Mod	도구당 +1	외부 오염 데이터 차단

> ➡️ TIP
>
> **실전 가이드**: 프로덕션에서는 `check_input` 만 항상 켜고 `check_output` 은 샘플링(예: 10%) 전략으로 운영하는 경우가 많습니다. `check_tool_results` 는 외부 신뢰할 수 없는 소스 전용으로 제한합니다.

핵심 정리

- 세 스캔 지점을 비용/지연 트레이드오프로 선택: input 항상 켜기, output 샘플링, tool_results 조건부
- `exit_behavior` 로 대화 종료(`end`)/즉시 예외(`error`)/메시지 교체(`replace`) 선택
- `violation_message` 템플릿으로 사용자 친화적 안내 구성
- `client` / `async_client` 로 사전 구성된 OpenAI 클라이언트를 재사용해 초기화 비용 절감

A 부록 A. 용어집

Book 전체에서 반복 등장하는 핵심 용어

이 부록은 책 전반에서 자주 등장하는 핵심 용어를 한 번에 다시 찾아볼 수 있도록 정리한 참고 섹션입니다. 처음부터 모두 외울 필요는 없으며, 낯선 용어가 나올 때마다 다시 찾아보는 방식으로 활용하면 됩니다.

> · NOTE
>
> 용어집의 목적은 API 문서를 대체하는 것이 아니라, 책을 읽을 때 자주 마주치는 개념을 빠르게 재확인할 수 있게 하는 데 있습니다. 구현 세부사항은 각 장의 본문과 참고 문서를 함께 보세요.

프레임워크와 제품

LangChain 모델, 도구, 프롬프트, 미들웨어를 조합해 에이전트와 LLM 애플리케이션을 빠르게 만드는 상위 프레임워크입니다.

LangGraph 상태, 노드, 엣지, 체크포인터를 기반으로 복잡한 워크플로와 장기 실행형 에이전트를 오케스트레이션하는 런타임/프레임워크입니다.

Deep Agents LangGraph 위에 계획, 파일시스템, 서브에이전트, 메모리, 샌드박스 같은 하네스 기능을 얹은 올인원 에이전트 SDK입니다.

LangSmith 트레이싱, 평가, 데이터셋 관리, 디버깅을 지원하는 관측성과 품질 관리 플랫폼입니다.

Langfuse 트레이싱과 관측성 수집을 위한 오픈소스 계열의 observability 도구입니다.

MCP Model Context Protocol의 약자로, 외부 도구와 리소스를 모델/에이전트에 표준 방식으로 연결하는 프로토콜입니다.

ACP Agent Client Protocol의 약자로, 에이전트를 에디터나 IDE 같은 클라이언트에 연결하기 위한 통신 프로토콜입니다.

에이전트와 실행 모델

Agent LLM이 도구를 사용하고 결과를 관찰하며 작업이 끝날 때까지 반복적으로 행동하는 실행 단위입니다.

ReAct Reasoning + Acting의 줄임말로, 모델이 추론과 도구 사용을 반복하며 문제를 해결하는 대표적인 에이전트 패턴입니다.

Workflow 단계와 순서가 비교적 명확하게 정의된 실행 흐름입니다. 에이전트보다 결정론적 성격이 강합니다.

Orchestrator 여러 단계나 여러 워커의 작업을 조정하고 전체 흐름을 관리하는 상위 제어자입니다.

Worker 오케스트레이터에게 위임받은 특정 작업을 수행하는 실행 단위입니다.

Subagent 메인 에이전트가 세부 작업을 위임하기 위해 호출하는 보조 에이전트입니다. 독립된 컨텍스트를 갖는 경우가 많습니다.

Handoff 현재 단계나 상태에 따라 다른 역할/에이전트로 제어를 넘기는 패턴입니다.

Router 입력이나 상태를 분류하여 적절한 처리 경로나 전문 에이전트로 보내는 패턴입니다.

Human-in-the-Loop 민감한 도구 실행이나 중요한 분기 지점에서 사람의 승인·수정·거부를 끼워 넣는 안전 장치입니다.

Interrupt 그래프나 에이전트 실행을 중간에서 멈추고 외부 입력을 기다리게 만드는 메커니즘입니다.

Resume 중단된 실행을 이전 상태에서 다시 이어가는 동작입니다. 보통 같은 `thread_id` 와 체크포인터가 필요합니다.

Durable Execution 실행 상태를 저장해 두었다가 장애나 중단 이후 마지막 성공 지점부터 다시 시작할 수 있게 하는 실행 방식입니다.

Pregel LangGraph 내부 실행 모델에 영향을 준 메시지 패싱 기반 계산 모델입니다. 노드가 슈퍼스텝 단위로 동작합니다.

Superstep Pregel 모델에서 여러 노드가 한 라운드로 실행되는 병렬 계산 단위를 뜻합니다.

상태와 메모리

State 에이전트나 그래프가 실행되는 동안 유지·업데이트하는 현재 작업 정보 묶음입니다.

AgentState LangChain/에이전트 실행에서 사용하는 기본 상태 스키마입니다. `messages` 등 예약 필드를 포함합니다.

MessagesState 메시지 리스트를 중심으로 상태를 관리하는 LangGraph의 편의 상태 타입입니다.

Checkpointer 실행 상태를 저장하고 복구하는 컴포넌트입니다. 멀티턴 대화, interrupt/resume, durable execution의 핵심입니다.

Thread ID 하나의 대화/실행 흐름을 식별하는 고유 ID입니다. 같은 `thread_id` 를 사용해야 이전 상태를 이어서 불러올 수 있습니다.

Runtime Context 실행 시점에만 주입되는 메타데이터·권한·사용자 정보 같은 컨텍스트입니다.

`context_schema` 실행 중 변하지 않는 정적 컨텍스트의 구조를 정의하는 스키마입니다.

`state_schema` 실행 중 계속 변하는 동적 상태의 구조를 정의하는 스키마입니다.

Short-term memory 하나의 스레드나 대화 안에서만 유지되는 메모리입니다. 주로 메시지 히스토리와 최근 작업 상태를 뜻합니다.

Long-term memory 대화가 끝난 뒤에도 남아 다음 스레드에서 재사용되는 메모리입니다. 사용자 선호도, 학습 결과 등에 적합합니다.

Store 스레드 바깥의 장기 데이터를 저장하고 검색하는 저장 계층입니다.

InMemoryStore 메모리 기반의 간단한 Store 구현입니다. 개발과 테스트에는 편리하지만 재시작 시 데이터가 사라집니다.

Semantic memory 사실·선호도·개념처럼 의미 기반으로 다시 찾아 쓸 수 있는 장기 기억입니다.

Episodic memory 특정 사건이나 상호작용 기록처럼 시간 순서가 중요한 기억입니다.

Procedural memory 에이전트가 따라야 할 규칙, 절차, 행동 방식에 해당하는 기억입니다.

도구와 인터페이스

Tool 에이전트가 호출할 수 있는 함수나 외부 작업 인터페이스입니다. 검색, 계산, 파일 읽기, API 호출 등이 여기에 해당합니다.

ToolRuntime 도구 실행 중 현재 상태, 컨텍스트, Store 등에 접근할 수 있게 해 주는 런타임 객체입니다.

`create_agent()` LangChain에서 기본 에이전트를 빠르게 생성하는 핵심 API입니다.

`create_deep_agent()` Deep Agents에서 계획, 파일, 서브에이전트 등을 포함한 하네스형 에이전트를 생성하는 API입니다.

`StateGraph` LangGraph에서 상태 기반 그래프 워크플로를 정의하는 빌더 클래스입니다.

Graph API 노드와 엣지를 명시적으로 선언해 그래프를 조립하는 LangGraph 프로그래밍 방식입니다.

Functional API Python 함수와 `@entrypoint`, `@task`로 워크플로를 표현하는 LangGraph 프로그래밍 방식입니다.

`@entrypoint` Functional API에서 워크플로의 시작 함수를 정의하는 데코레이터입니다.

`@task` 내구성 보장이 필요한 작업 단위를 감싸는 Functional API 데코레이터입니다.

`Send` LangGraph에서 동적으로 여러 워커 실행을 fan-out하는 데 쓰이는 API입니다.

`Command` 상태 업데이트, resume, 부모 그래프 전이 등을 표현하는 LangGraph의 제어 객체입니다.

Structured output 모델의 응답을 Pydantic 스키마나 명시된 구조로 강제하는 출력 방식입니다.

백엔드와 실행 환경

Backend Deep Agents에서 파일 읽기/쓰기와 실행 환경을 추상화하는 저장·실행 계층입니다.

StateBackend 대화 스레드 안에서만 유지되는 에페메럴 파일 저장 백엔드입니다.

FilesystemBackend 로컬 디스크에 접근하는 백엔드입니다. `virtual_mode=True`로 경로 제한을 두는 것이 일반적입니다.

StoreBackend LangGraph Store를 사용해 스레드 간 영속 저장을 제공하는 백엔드입니다.

CompositeBackend 경로별로 서로 다른 백엔드에 요청을 라우팅하는 혼합형 백엔드입니다.

LocalShellBackend 로컬 파일 접근에 더해 셸 명령 실행까지 제공하는 강력하지만 위험한 백엔드입니다.

Sandbox 호스트와 격리된 환경에서 코드 실행과 파일 작업을 수행하게 하는 실행 환경입니다.

Modal GPU와 AI/ML 워크로드에 강한 샌드박스/서버리스 실행 환경입니다.

Daytona 빠른 devbox provisioning과 개발 환경 중심 사용 사례에 적합한 샌드박스 제공자입니다.

Runloop 일회성 devbox와 격리 실행에 적합한 샌드박스 제공자입니다.

검색, 스트리밍, 품질 관리

RAG Retrieval-Augmented Generation의 약자로, 외부 지식을 검색해 모델 응답에 주입하는 패턴입니다.

Retriever 질문과 관련된 문서나 청크를 검색해 반환하는 컴포넌트입니다.

Embedding 텍스트를 의미 공간의 벡터로 바꿔 유사도 검색을 가능하게 만드는 표현입니다.

Vector store 임베딩을 저장하고 유사도 검색을 수행하는 저장소입니다.

Chunking 긴 문서를 검색과 컨텍스트 주입에 적합한 작은 단위로 분할하는 작업입니다.

SQL Agent 자연어 질문을 SQL 쿼리로 변환하고 실행 결과를 해석하는 에이전트입니다.

Streaming 모델 응답이나 실행 상태를 완료 전에 점진적으로 전달하는 방식입니다.

StreamEvent 스트리밍 중 전달되는 이벤트 단위입니다. 토큰, 도구 시작/종료, 상태 변경 등이 포함됩니다.

TTFT Time to First Token의 약자로, 모델이 첫 토큰을 내놓기까지 걸리는 시간입니다.

TTFA Time to First Audio의 약자로, 보이스 에이전트에서 첫 오디오 출력까지 걸리는 시간입니다.

Tracing 모델 호출, 도구 실행, 상태 전이, 에러를 기록해 실행 흐름을 추적하는 관측 기법입니다.

Evaluation 에이전트 출력 또는 실행 궤적의 품질을 측정하는 과정입니다.

LLM-as-Judge 다른 LLM이 응답 품질이나 실행 결과를 평가자 역할로 채점하는 방식입니다.

Trajectory 에이전트가 최종 답변에 도달하기까지 거친 도구 호출, 중간 추론, 상태 변화의 전체 흐름입니다.

Guardrail 입력·도구·출력 단계에서 안전성, 정책, 품질을 검증하는 제어 장치입니다.

PII Personally Identifiable Information의 약자로, 이메일·전화번호·주민번호처럼 개인을 식별할 수 있는 민감 정보입니다.